The Little Book of The Art of Computer Programming Solutions

Version 0.1.0

Duc-Tam Nguyen

2025-09-13

Table of contents

1.1 Algorithms **16**
1 [10] 16
2 [15] 17
3 [20] 17
4 [16] 18
5 [12] 19
6 [20] 19
7 [M21] 19
8 [M25] 22
9 [M30] 24

1.2.1 Mathematical induction **27**
1 [5] 27
2 [15] 28
3 [18] 28
4 [20] 30
5 [21] 30
6 [20] 31
7 [23] 32
8 [25] 33
9 [20] 34
10 [M22] 35
11 [M30] 36
12 [M25] 37
13 [M23] 39
14 [50] 41
15 [HM28] 41

1.2.2 Numbers, powers, and logarithms **43**
1 [00] 43
2 [00] 43
3 [02] 44
4 [05] 44
5 [05] 45
6 [10] 46

7 [M23] 47
8 [25] 48
9 [M23] 50
10 [18] 52
11 [10] 53
12 [02] 54
13 [M23] 54
14 [15] 55
15 [10] 56
17 [05] 56
18 [10] 57
19 [20] 58
20 [10] 59
21 [15] 60
22 [20] 60
23 [M25] 61
24 [15] 62
25 [22] 64
26 [M27] 65
27 [M30] 67
28 [M30] 69
29 [HM20] 71
30 [12] 72

1.2.3 Sums and products **74**

1 [10] 74
2 [01] 74
3 [13] 75
4 [10] 76
5 [HM20] 76
6 [HM20] 78
7 [HM23] 79
8 [HM25] 80
9 [05] 82
10 [05] 83
11 [03] 84
12 [10] 85
13 [10] 86
14 [11] 87
15 [M22] 88
16 [M22] 89
17 [M00] 91
18 [M20] 92

19 [20] 93
20 [25] 94
21 [M25] 96
22 [20] 97
23 [10] 98
24 [20] 100
25 [15] 101
26 [25] 101
27 [M20] 102
28 [M22] 103
29 [M30] 104
30 [M23] 106
31 [M20] 107
32 [M20] 108
33 [M30] 110
34 [M25] 112
35 [HM20] 113
36 [M23] 115
37 [M24] 117
38 [M25] 119
39 [M23] 121
40 [M24] 123
41 [M26] 124
42 [M18] 125
43 [M24] 127
44 [M26] 129
45 [M25] 131
46 [M30] 131
47 [M27] 133

1.2.4 Integer functions and elementary number theory **136**
1 [00] 136
2 [01] 136
3 [M10] 137
4 [M10] 138
5 [16] 139
6 [20] 140
7 [M15] 141
8 [00] 142
9 [05] 142
10 [00] 143
11 [00] 144
12 [00] 144

13 [M00] 144
14 [12] 145
15 [10] 145
16 [M10] 147
17 [M15] 148
18 [M15] 149
19 [M10] 150
20 [M15] 151
21 [M22] 152
22 [M10] 153
23 [M10] 154
24 [M20] 155
25 [M02] 156
26 [M15] 157
27 [M15] 158
28 [M25] 159
29 [M22] 160
30 [M30] 161
31 [M22] 163
32 [M18] 164
33 [M18] 165
34 [M21] 166
35 [M20] 167
36 [M23] 168
37 [M30] 170
38 [M26] 172
39 [HM35] 174
40 [HM46] 177
41 [M24] 180
42 [M24] 181
43 [M23] 182
44 [M24] 184
45 [M28] 185
46 [M29] 187
47 [M31] 189

1.2.5 Permutations and factorials **193**
1 [00] 193
2 [10] 194
3 [10] 195
4 [13] 195
5 [15] 197
6 [17] 198

7 [M10] 198
8 [HM15] 199
9 [M10] 201
10 [HM20] 202
11 [M15] 203
12 [M22] 204
Check 205
13 [M22] 206
14 [M28] 206
15 [HM15] 208
16 [HM15] 209
17 [HM20] 211
18 [M20] 212
19 [HM22] 214
20 [HM21] 216
21 [HM25] 218
22 [HM20] 219
23 [HM20] 221
24 [HM21] 222
25 [M20] 223

1.2.6 Binomial coefficients **226**
1 [00] 226
2 [00] 227
3 [00] 228
4 [10] 228
5 [05] 229
6 [10] 230
7 [12] 231
8 [00] 232
9 [01] 233
10 [M25] 234
11 [M20] 236
12 [M22] 237
13 [M13] 238
14 [M21] 240
15 [M15] 241
16 [M15] 243
17 [M18] 245
18 [M18] 246
19 [M18] 247
20 [M20] 249
21 [M05] 253

22 [M20] 254
23 [M13] 258
24 [M15] 259
25 [HM30] 260
26 [HM25] 262
27 [HM21] 265
28 [M25] 267
29 [M20] 269
30 [M24] 271
31 [M20] 274
32 [M20] 275
33 [M20] 277
34 [M23] 278
35 [M23] 280
36 [M10] 282
37 [M10] 283
38 [HM30] 284
39 [M10] 285
40 [HM17] 287
41 [HM22] 288
42 [HM10] 289
43 [HM20] 290
44 [HM20] 291
45 [HM21] 293
46 [M21] 294
47 [M21] 295
48 [M25] 296
49 [M20] 298
50 [M20] 298
51 [M21] 300
52 [HM11] 302
53 [M25] 303
54 [M21] 306
55 [M21] 307
56 [20] 309
57 [M22] 310
58 [M23] 312
59 [M25] 314
61 [M25] 316
62 [M23] 317
63 [M30] 319
64 [M20] 321
65 [HM35] 322

66 [HM30] 325
67 [M20] 327
68 [M25] 329

1.2.7 Harmonic numbers **332**
1 [01] 332
2 [13] 332
3 [M21] 333
4 [10] 334
5 [15] 336
6 [M15] 336
7 [M21] 337
8 [HM18] 339
9 [M18] 340
10 [M20] 342
11 [M21] 343
12 [M10] 344
13 [M22] 345
14 [M22] 346
15 [M23] 348
16 [18] 350
17 [M24] 351
18 [M33] 352
19 [M30] 355
20 [HM22] 355
21 [M24] 357
22 [M28] 359
23 [HM20] 360
24 [HM21] 362
25 [M21] 363

1.2.8 Fibonacci numbers **365**
1 [10] 365
2 [20] 366
3 [25] 368
4 [14] 369
5 [20] 370
6 [HM10] 371
7 [15] 372
8 [15] 373
9 [M20] 374
10 [15] 376
11 [M20] 377

12 [M26] . . . 378
13 [M22] . . . 380
14 [M28] . . . 381
15 [M22] . . . 383
16 [M20] . . . 384
17 [M24] . . . 386
18 [20] . . . 388
19 [M27] . . . 389
20 [M16] . . . 390
21 [M25] . . . 391
22 [M20] . . . 393
23 [M23] . . . 394
24 [HM20] . . . 395
25 [M21] . . . 397
26 [M20] . . . 398
27 [M20] . . . 399
28 [M21] . . . 401
29 [M23] . . . 402
30 [M38] . . . 404
31 [M20] . . . 407
32 [M24] . . . 409
33 [HM24] . . . 411
34 [M24] . . . 412
35 [M24] . . . 415
36 [M32] . . . 417
37 [M35] . . . 420
38 [35] . . . 422
39 [M24] . . . 426
40 [M25] . . . 427
41 [M25] . . . 429
42 [M26] . . . 432

1.2.9 Generating functions **434**
1 [M12] . . . 434
2 [M13] . . . 435
3 [HM21] . . . 436
4 [M01] . . . 438
5 [M20] . . . 439
6 [HM15] . . . 440
7 [15] . . . 442
8 [M23] . . . 444
9 [M11] . . . 445
10 [M25] . . . 446

11 [M25] 448
12 [M20] 449
13 [HM22] 450
14 [HM21] 452
15 [M28] 453
16 [M22] 455
17 [M25] 457
18 [M25] 459
19 [HM32] 460
20 [M21] 463
21 [HM30] 465
22 [M21] 467
23 [M33] 468
24 [M22] 471
25 [M23] 475
26 [M40] 476

1.2.10 Analysis of an Algorithm **478**

1 [10] 478
2 [HM16] 479
3 [M15] 479
4 [M10] 481
5 [M13] 482
6 [HM27] 482
7 [M27] 484
8 [M20] 485
9 [M25] 486
10 [M20] 488
11 [M15] 489
12 [HM21] 490
13 [HM38] 492
14 [HM30] 493
15 [HM23] 494
16 [M25] 496
17 [M27] 498
18 [M28] 500
19 [M21] 502
20 [M22] 504
21 [HM21] 505
22 [HM22] 507
23 [HM23] 508

1.2.11 Asymptotic representations **512**
1 [HM20] . . . 512
2 [HM20] . . . 513
3 [M20] . . . 514
4 [HM10] . . . 516
5 [HM24] . . . 517
6 [HM20] . . . 519
7 [HM30] . . . 520
8 [HM30] . . . 522
9 [HM36] . . . 524
10 [HM34] . . . 527
11 [HM35] . . . 529
12 [HM20] . . . 531
13 [HM42] . . . 532
14 [HM39] . . . 532
15 [HM20] . . . 535
16 [M24] . . . 536
17 [HM29] . . . 538
18 [M25] . . . 539
19 [HM30] . . . 541
20 [HM30] . . . 542

1.3.1 Description of MIX **545**
1 [00] . . . 545
2 [02] . . . 545
3 [02] . . . 545
4 [00] . . . 545
5 [10] . . . 546
6 [10] . . . 547
7 [M15] . . . 547
8 [15] . . . 548
9 [15] . . . 549
10 [15] . . . 550
11 [15] . . . 550
12 [10] . . . 551
13 [10] . . . 551
14 [20] . . . 552
15 [10] . . . 553
16 [20] . . . 553
17 [26] . . . 554
18 [22] . . . 555
19 [14] . . . 560
20 [20] . . . 560

21 [24] 560
22 [28] 561
23 [27] 562
24 [21] 564
25 [30] 565
26 [32] 568

1.3.2 The MIX assembly language **575**
1 [00] 575
2 [10] 575
3 [23] 576
4 [25] 576
5 [11] 580
6 [HM20] 580
7 [10] 580
8 [24] 581
9 [25] 582
10 [31] 585
11 [M29] 587
12 [HM42] 589
13 [28] 590
14 [31] 595
15 [M30] 600
16 [31] 605
17 [HM30] 611
18 [25] 613
19 [M30] 615
20 [33] 616
21 [28] 621
22 [31] 625
23 [37] 629

1.3.3 Applications to Permutation **640**
1 [02] 640
2 [10] 641
3 [03] 642
4 [10] 642
5 [M10] 643
6 [M28] 644
7 [10] 645
8 [23] 645
9 [10] 645
10 [M28] 648

11 [15] 651
12 [M27] 652
13 [M24] 654
14 [M34] 657
15 [M12] 658
16 [M15] 660
17 [M24] 660
18 [M27] 662
19 [HM21] 664
20 [M20] 665
21 [M22] 665
22 [HM34] 668
23 [HM42] 672
24 [M41] 675
25 [M22] 677
26 [M24] 678
27 [M20] 680
28 [M21] 681
29 [M25] 683
30 [M24] 685
31 [HM38] 687
32 [M25] 690
33 [M33] 692
34 [M25] 694
35 [M39] 697
36 [27] 700
37 [M26] 706

1.4.1 Subroutines **709**
1 [10] 709
2 [10] 710
3 [M15] 710
4 [21] 712
5 [21] 715
5 [21] 716
6 [26] 718
7 [20] 721

1.4.2 Coroutines **723**
1 [10] 723
2 [20] 724
3 [20] 725
4 [20] 726

5 [15] 729
6 [22] 730
7 [30] 732

1.4.3.1 A MIX simulator **739**
1 [14] 739
2 [20] 741
3 [22] 746
4 [14] 748
5 [24] 749
6 [28] 751
7 [32] 756
8 [20] 770

1.4.3.2 Trace routines **771**
1 [22] 771
2 [26] 772
3 [10] 774
4 [25] 775
5 [28] 777
6 [40] 778
7 [25] 782

1.4.4 Input and Output **787**
1 [05] 787
2 [10] 788
3 [22] 789
4 [M20] 792
5 [M21] 794
6 [12] 796
7 [22] 797
8 [11] 799
9 [21] 800
10 [21] 802
11 [21] 804
12 [24] 806
13 [20] 810
14 [20] 811
15 [22] 812
16 [29] 814
17 [40] 823
18 [30] 836
19 [M28] 841

1.1 Algorithms

1 [10]

Minimal in-place algorithm (5 replacements)

```
t ← a
a ← b
b ← c
c ← d
d ← t
```

Walkthrough. Let the initial values be `a0, b0, c0, d0`.

1. `t ← a` → `t = a0`
2. `a ← b` → `a = b0`
3. `b ← c` → `b = c0`
4. `c ← d` → `c = d0`
5. `d ← t` → `d = a0`

Final state is `(a, b, c, d) = (b0, c0, d0, a0)`, exactly the requested left rotation.

Why 5 replacements is optimal

1. All four variables change. In a 4-cycle `(a b c d)`, none of `a, b, c, d` keeps its original value, so each of the four must be assigned at least once → 4 replacements.
2. You must preserve one original value while overwriting. As soon as you assign, say, `a ← b`, the original `a0` is lost unless it was saved. But the final `d` must become `a0`. Therefore at least one extra "save" is necessary (e.g., `t ← a`) → $4 + 1 = 5$ replacements.

Thus 5 is a lower bound, and the procedure above achieves it, so it's minimal.

Generalization

For an n-cycle rotation `(x1 x2 … xn)`, the same argument shows a lower bound of n + 1 replacements (n to set each variable once, plus 1 to save a value), and the standard scheme

```
t ← x1;  x1 ← x2;  x2 ← x3;  …;  xn-1 ← xn;  xn ← t
```

meets that bound.

2 [15]

Consider any time we return to E1 after completing an earlier pass through E1/E2/E3.

- On the previous pass through E1 we computed a remainder r with $0 \le r < n$. If $r = 0$ we would have halted in E2 and would not reach E1 again. Hence whenever we *do* reach E1 again we must have $r \neq 0$, i.e., $0 < r < n$.
- Then E3 set $m \leftarrow n$ and $n \leftarrow r$. Therefore, at the start of the next E1, we have

$$m_{\text{new}} = n_{\text{old}} \quad \text{and} \quad n_{\text{new}} = r_{\text{old}}\,,$$

  with $0 < r_{\text{old}} < n_{\text{old}}$. Hence $m_{\text{new}} = n_{\text{old}} > r_{\text{old}} = n_{\text{new}}$, i.e., $m > n$ strictly.

This shows that from the second visit to E1 onward, the strict inequality $m > n$ always holds.

For the first visit to E1, no such guarantee exists: the inputs might have $m \le n$ (indeed $m < n$ is allowed), so the statement allows the exception "except possibly the first time." (If one prepends the optional "ensure $m \ge n$" swap step suggested in the text, the property would also hold the first time.)

3 [20]

Algorithm F (Euclid, without "trivial replacements")

Goal. Given positive integers `m` and `n`, compute `gcd(m, n)`.

- F1. [Remainder m / n] Divide `m` by `n` and replace `m` with the remainder. (So `0  m < n`.)
- F2. [Is it zero?] If `m = 0`, halt with answer `n`.
- F3. [Remainder n / m] Divide `n` by `m` and replace `n` with the remainder. (So `0  n < m`.)
- F4. [Is it zero?] If `n = 0`, halt with answer `m`; otherwise go to F1.

This version never does "`m ← n, n ← r`"; we overwrite the dividend directly with the remainder each time and alternate roles.

Why this is correct

1) Invariant (gcd is preserved)

At every step, `gcd(m, n) = gcd(n, m mod n)` and `gcd(m, n) = gcd(m, n mod m)`. That's the fundamental property behind Euclid's method; F1/F3 are exactly these replacements, so the gcd never changes. (This is the same reasoning used to justify Algorithm E; see §1.1. (PagePlace))

2) Termination

Each remainder is strictly smaller than its divisor. After F1 we have `m < n`; after F3 we have `n < m`. Thus the maximum of the pair `{m, n}` strictly decreases on every pass through F1–F4. Since the maximum is a non-negative integer, it can decrease only finitely many times, so the algorithm halts.

3) Upon halting, the answer is correct

If we halt at F2, then `m = 0`, so `gcd(m, n) = gcd(0, n) = n`. If we halt at F4, then `n = 0`, so `gcd(m, n) = m`. Combined with the invariant, this equals the original `gcd`.

Worked example

Compute `gcd(2166, 6099)` using F.

- F1: `m ← 2166 mod 6099 = 2166` (since $2166 < 6099$) F2: `m  0` → continue
- F3: `n ← 6099 mod 2166 = 1767` F4: `n  0` → loop
- F1: `m ← 2166 mod 1767 = 399` F2: `m  0`
- F3: `n ← 1767 mod 399 = 171` F4: `n  0`
- F1: `m ← 399 mod 171 = 57` F2: `m  0`
- F3: `n ← 171 mod 57 = 0` F4: `n = 0` → halt with answer `m = 57`.

4 [16]

We repeatedly divide and keep the remainder; the gcd is the last non-zero remainder.

1. $6099 = 2 \cdot 2166 + 1767 \to r_1 = 1767$
2. $2166 = 1 \cdot 1767 + 399 \to r_2 = 399$
3. $1767 = 4 \cdot 399 + 171 \to r_3 = 171$
4. $399 = 2 \cdot 171 + 57 \to r_4 = 57$
5. $171 = 3 \cdot 57 + 0$

The last non-zero remainder is 57, so

$$\gcd(2166, 6099) = 57.$$

Quick checks

- Divisibility: $2166/57 = 38$ and $6099/57 = 107$ are integers.
- No larger common factor: Euclid's process ensures optimality.

5 [12]

The "Procedure for Reading This Set of Books" is not a genuine algorithm because it breaks at least three of Knuth's five criteria:

- Finiteness: it loops back to step 3, so it need not terminate.
- Definiteness: steps like "Are you tired?" or "Is it interesting?" are subjective, not precise.
- Effectiveness: operations like "go to sleep" or "get your friends to purchase a copy" are not basic, executable steps.

It also differs in format from Algorithm E: it uses plain numbers (not E1, E2…), prose instead of arrow notation ($\leftarrow$), no bracketed summaries, and no explicit end marker.

Thus the "procedure" is more of a joke than a proper algorithm.

6 [20]

For each $m \in \{1, 2, 3, 4, 5\}$, count how many divisions (E1) occur until termination.

- $m = 1$: $1 \bmod 5 = 1$ (1st E1); $(m, n) \leftarrow (5, 1)$. Then $5 \bmod 1 = 0$ (2nd E1) stop. Count $= 2$.
- $m = 2$: $2 \bmod 5 = 2$ (1); $(5, 2)$. $5 \bmod 2 = 1$ (2); $(2, 1)$. $2 \bmod 1 = 0$ (3) stop. Count $= 3$.
- $m = 3$: $3 \bmod 5 = 3$ (1); $(5, 3)$. $5 \bmod 3 = 2$ (2); $(3, 2)$. $3 \bmod 2 = 1$ (3); $(2, 1)$. $2 \bmod 1 = 0$ (4) stop. Count $= 4$.
- $m = 4$: $4 \bmod 5 = 4$ (1); $(5, 4)$. $5 \bmod 4 = 1$ (2); $(4, 1)$. $4 \bmod 1 = 0$ (3) stop. Count $= 3$.
- $m = 5$: $5 \bmod 5 = 0$ (1) stop. Count $= 1$.

Sum of counts $= 2 + 3 + 4 + 3 + 1 = 13$.

$$T_5 = \frac{13}{5} = 2.6.$$

7 [M21]

Let $A(m, n)$ be the number of times step E1 ("divide m by n; let r be the remainder") is executed by Euclid's algorithm when it is started on the ordered pair (m, n) of positive integers.

Knuth defines T_n as the average of $A(m, n)$ when n is fixed and m is allowed to range over all positive integers; because only the residue of $m \bmod n$ matters after the first division, the average is taken over $m = 1, 2, \ldots, n$ (uniformly). He notes that for large n, T_n is $\asymp c \ln n$ with $c = 12 \ln 2/\pi^2$.

The exercise asks to define U_m analogously, with m fixed and n "ranging over all positive integers," and to relate U_m to T_m.

We will prove that U_m is well defined (the limit average exists and is finite) and that

$$\boxed{U_m = T_m + 1}.$$

Two simple identities for $A(m, n)$

(I) Swap once when the first number is smaller. If $m < n$, then the very first division in Euclid's algorithm leaves remainder $r = m$; the next state is (n, m). Thus

$$A(m, n) \;=\; 1 + A(n, m) \qquad (m \neq n). \tag{1}$$

(II) Periodicity in $n \pmod m$ when $n > m$. Write $n = qm + r$ with $0 \leq r < m$. The first step on (m, n) gives remainder $r = m$ (since $n > m$), so after one step we are at (n, m). One more step divides n by m and leaves remainder r; from then on we continue with (m, r). Hence

$$A(m, n) \;=\; \begin{cases} 2, & r = 0, \\ 2 + A(m, r), & 1 \leq r \leq m - 1. \end{cases} \tag{2}$$

So for $n > m$, $A(m, n)$ depends only on $n \bmod m$ (and equals a constant 2 when $m \mid n$).

Existence (and value) of U_m

Define the Cesàro average

$$U_m \;:=\; \lim_{N \to \infty} \frac{1}{N} \sum_{n=1}^{N} A(m, n),$$

if the limit exists.

Partition $1, \ldots, N$ into complete blocks of length m plus a final partial block. Using (II), each complete block contributes

$$\sum_{r=0}^{m-1} A(m,\, qm + r) = 2 + \sum_{r=1}^{m-1} (2 + A(m, r)) = 2m + \sum_{r=1}^{m-1} A(m, r).$$

Thus the block average is

$$\frac{1}{m}\left(2m+\sum_{r=1}^{m-1}A(m,r)\right)=2+\frac{1}{m}\sum_{r=1}^{m-1}A(m,r). \tag{3}$$

The final partial block contributes $O(m)$, which is negligible after division by N. Therefore the limit defining U_m exists and equals the block average:

$$U_m \;=\; 2+\frac{1}{m}\sum_{r=1}^{m-1}A(m,r). \tag{4}$$

Relation to T_m

By definition (see the paragraph introducing T_n and how the average is taken over a complete set of residues), we have

$$T_m \;=\; \frac{1}{m}\sum_{x=1}^{m}A(x,m)=\frac{1}{m}\Big(A(m,m)+\sum_{x=1}^{m-1}A(x,m)\Big).$$

Now use (I) with $x<m$: $A(x,m)=1+A(m,x)$. Also $A(m,m)=1$. Hence

$$T_m=\frac{1}{m}\Big(1+\sum_{x=1}^{m-1}(1+A(m,x))\Big)=\frac{1}{m}\Big(m+\sum_{x=1}^{m-1}A(m,x)\Big)=1+\frac{1}{m}\sum_{x=1}^{m-1}A(m,x). \tag{5}$$

Comparing (4) with (5) immediately yields

$$\boxed{U_m=T_m+1}.$$

Sanity check on a small example

Take $m=5$. One can compute:

$$A(5,1)=1,\;A(5,2)=2,\;A(5,3)=3,\;A(5,4)=3.$$

Then

$$T_5=1+\tfrac{1}{5}(1+2+3+3)=1+\tfrac{9}{5}=\tfrac{14}{5},\qquad U_5=2+\tfrac{1}{5}(1+2+3+3)=2+\tfrac{9}{5}=\tfrac{19}{5}=T_5+1.$$

(Manually checking a long run of n values shows the block average equals 19/5, as predicted.)

Consequence

Since $T_n \sim \frac{12 \ln 2}{\pi^2} \ln n$ for large n, we get

$$U_m \sim \frac{12 \ln 2}{\pi^2} \ln m + 1,$$

so asymptotically U_m and T_m differ by exactly 1.

That proves U_m is well defined and establishes its precise relationship to T_m.

8 [M25]

Design a set of *string-rewriting* rules in Knuth's $(T_j,\ s_j,\ a_j,\ b_j)$ format that, starting from the input string $a^m b^n$ (i.e., m copies of `a` followed by n copies of `b`), halts with the string $a^{\gcd(m,n)}$. The rules must be expressed as a Markov algorithm in the tabular style used in §1.1.

Quick refresher: what (T_j, s_j, a_j, b_j) means

A program state is a pair (σ, j): current string σ and current rule index j. Each table row j has:

- T_j ("pattern"): the substring to search for,
- s_j ("replacement"): the string to substitute when T_j is found,
- b_j: the next rule index if a replacement did occur,
- a_j: the next rule index if no T_j is found in σ.

Operationally (Markov rules): if T_j occurs, replace the *leftmost* occurrence by s_j and jump to b_j; otherwise jump to a_j. One special index N means "halt." (Stack Overflow)

A minimal working rule set

Use the alphabet $\{a, b, c\}$ and five rules (indices $0, 1, 2, 3, 4$); let 5 be the halting index N.

j	T_j	s_j	b_j	a_j	Intuition
0	`ab`	(empty)	1	2	Cancel one `a` with one `b` (one subtraction step).
1	(empty)	`c` (prepend)	0	0	After each cancellation, count it by sticking a `c` on the left.
2	`a`	`b`	2	3	Turn all remaining `a`'s into `b`'s.
3	`c`	`a`	3	4	Then turn all `c`'s into `a`'s.
4	`b`	`b`	0	5	If any `b`'s remain, repeat another Euclid round; else halt.

Why this works (high-level):

- Rules 0 1 simulate *repeated subtraction*: each `ab` removed corresponds to kubtracting the smaller number from the larger once; the number of times you can do this is $\min(m, n)$. We "count" those cancellations by accumulating `c`'s.
- When no `ab` remains, the string looks like `c^{min(m,n)} x^{|m-n|}`, where x is whichever letter survived (`a` or `b`).
- Rules 2 and 3 normalize the string to `a^{min(m,n)} b^{|m-n|}`, i.e. we map the pair (m, n) to $(\min(m, n), |m - n|)$.
- Rule 4 loops if the second component is nonzero; otherwise halt. This is exactly the subtraction-based Euclidean step, so repeated rounds end with `a^{gcd(m,n)}`. The superscript means "repeat `a` that many times." ([Stack Overflow][2])

Correctness argument (brief but complete)

Invariant after finishing rule 0 1 loop (before rule 2): Let the current pair be (m, n). Repeatedly removing the leftmost `ab` (rule 0) and then prepending a `c` (rule 1) exactly $\min(m, n)$ times yields a string of the form:

$$c^{\min(m,n)}\ y^{|m-n|},$$

where y is `a` if $m > n$ and `b` if $n \geq m$. This encodes "we subtracted the smaller from the larger $\min(m, n)$ times."

Normalization (rules 2→3):

- Rule 2 turns any stray `a`'s into `b`'s, so only `c`'s and `b`'s remain.
- Rule 3 turns each `c` into `a`. Thus the string becomes `a^{\min(m,n)} b^{|m-n|}` in canonical order.

Euclid step and termination (rule 4):

- If any `b` remains, rule 4 jumps back to rule 0, so the new pair is $(m', n') = (\min(m, n), |m - n|)$.
- Repeating transforms (m, n) via the subtraction form of Euclid's algorithm until $n' = 0$; at that point rule 4 falls through to 5 (halt), with the string `a^{gcd(m,n)}`. (Stack Overflow)

Worked example

Take $m = 6$, $n = 10$; start with `aaaaaabbbbbbbbbb`.

- Round 1 (to exhaustion of rule 0): remove `ab` six times while accumulating `c`'s → `ccccccbbbb`. Normalize (rules 2→3) → already no `a`'s, then `c→a  aaaaaabbbb` = `a^6 b^4`.
- Round 2: cancel `ab` four times → `cccc a a` (i.e., `ccc caa` but order doesn't matter until normalization) normalize `a^4 b^2`.

- Round 3: cancel twice normalize `a^2 b^0`.
- Round 4: rule 4 sees "no `b` left" halt with `aa` $= a^{\gcd(6,10)}$.

Relation to Euclid's Algorithm E

Each full pass of rules 0→…→4 implements the mapping

$$(m, n) \;\mapsto\; (\min(m, n),\; |m - n|),$$

i.e., Euclid's algorithm with the division step replaced by *repeated subtraction*. That's precisely the hint Knuth gives; the string system is just a Markov-algorithm rendering of that idea.

Notes and common confusions

- The superscript in "$a^{\gcd(m,n)}$" means "`a` repeated $\gcd(m, n)$ times." For example, if $\gcd(12, 18) = 6$, the output is `aaaaaa`.
- In the rule table, b_j is the "branch if a replacement happened," and a_j is the "branch if not." This branching scheme, together with leftmost replacement, is standard for Markov algorithms and matches Knuth's §1.1 formalism.

9 [M30]

Let

$$C_1 = (Q_1, I_1, \Omega_1, f_1), \qquad C_2 = (Q_2, I_2, \Omega_2, f_2)$$

be computational methods (state space Q, initial/allowable inputs $I \subseteq Q$, halting/"output" states $\Omega \subseteq Q$, and a (partial) next-state function f). A *computation sequence* of C_i is the usual chain $x, f_i(x), f_i^{(2)}(x), \ldots$ until (and if) a state in Ω_i is reached.

Intuitively, C_2 may do more, smaller steps and may keep extra bookkeeping in its states, yet it should faithfully realize C_1.

We say that C_2 represents C_1 iff there exist total maps

- an encoding $e : Q_1 \to Q_2$ and
- a decoding/abstraction $\pi : Q_2 \to Q_1$

such that $\pi \circ e = \mathrm{id}_{Q_1}$ (i.e., e embeds C_1's abstract state into C_2's concrete state, and π forgets the extra detail), and the following hold:

1. Input compatibility. $e(I_1) \subseteq I_2$.

2. Output compatibility. $\pi(\Omega_2) \subseteq \Omega_1$ and for every $\omega \in \Omega_1$ there exists some $\hat{\omega} \in \Omega_2$ with $\pi(\hat{\omega}) = \omega$. (Equivalently, $\pi^{-1}(\Omega_1)$ contains only states from which C_2 can reach Ω_2 without changing π's value.)

3. Step correspondence (stuttering simulation). For every $q_1 \in Q_1 \setminus \Omega_1$ and every $q_2 \in Q_2$ with $\pi(q_2) = q_1$, if $f_1(q_1)$ is defined and equals q_1', then there exists a finite $t \geq 1$ and a path in C_2

$$q_2 = r_0 \xrightarrow{f_2} r_1 \xrightarrow{f_2} \cdots \xrightarrow{f_2} r_t$$

such that

$$\pi(r_j) = q_1 \quad (0 \leq j < t), \qquad \pi(r_t) = q_1'.$$

(So C_2 may "stutter"—take several internal steps that do not change the abstract state—before realizing one abstract f_1-step.)

4. Termination preservation (refinement of whole runs). For every input $x \in I_1$, if the C_1 run from x halts in k steps with output $\omega \in \Omega_1$, then the C_2 run from $e(x)$ halts after some $T \geq k$ steps in a state $\hat{\omega} \in \Omega_2$ with $\pi(\hat{\omega}) = \omega$.

This is the standard "forward simulation with stuttering" characterization used in refinement: C_2 can take more (finer-grained) steps and store more information, but when you *abstract* its state with π you see exactly the abstract behavior of C_1.

Why this captures the intent

- "More steps": Condition 3 allows multiple f_2 steps for a single f_1 step while keeping π unchanged until the abstract change happens.
- "More information": The abstraction π can forget registers, temporary variables, program counters, etc.; $\pi \circ e = \mathrm{id}$ ensures no abstract information is lost when we embed.
- Correct inputs/outputs: Conditions 1–2 align the two machines at the beginning and end.
- Whole computations: Condition 4 ensures that every terminating abstract computation is realized concretely with the same abstract result.

How it looks on the Euclid example

In §1.1, Algorithm E is formalized as a (Q, I, Ω, f) system with states like $(m, n, 0, 1)$, $(m, n, r, 2)$, (n), etc., and a next-state function that embodies steps E1–E3. A machine program C_2 has states including memory, registers, and a program counter. Define:

- e: put the abstract $m, n, r, \ldots$ into designated registers/memory and set the PC to the code for the corresponding E-step.

- π: read back those designated registers and map *any* machine state of that code block to the corresponding abstract state (ignoring temps, flags, scratch space, etc.).

A long division routine in C_2 will execute many micro-steps while π stays at the same abstract E1 state; upon finishing the division, π jumps to the E2/E3 abstract state—exactly the stuttering behavior required by (3). When $n = 0$, the program branches to a return sequence; π maps those return-sequence states into C_1's halting set, establishing (2)–(4).

1.2.1 Mathematical induction

1 [5]

1) Weak induction starting at 0

To prove $P(n)$ for all $n \geq 0$, use this two-part template:

1. **Base case.** Prove $P(0)$.
2. **Inductive step.** Prove $\forall k \geq 0$, $P(k) \Rightarrow P(k+1)$.

Then by the same "Algorithm I" logic Knuth gives for induction (but with the initial index shifted), the procedure constructs a proof for $P(0), P(1), \ldots, P(n)$ in order, hence $P(n)$ holds for every $n \geq 0$. Informally: from $P(0)$ the step yields $P(1)$; from $P(1)$ it yields $P(2)$; and so on. (PagePlace)

2) Strong induction starting at 0 (often convenient)

Sometimes the step needs more than just $P(k)$. Then use:

1. **Base case.** Prove $P(0)$.
2. **Inductive step (strong).** Assume $P(0), P(1), \ldots, P(k)$ are all true; prove $P(k+1)$.

This is equivalent in strength to weak induction but can be easier to apply in algorithm proofs or recurrences. (Knuth emphasizes the algorithmic viewpoint of induction and its "construct a proof" interpretation.) (PagePlace)

3) General "shifted" induction (for all $n \geq n_0$)

More generally, if you want $P(n)$ for all $n \geq n_0$ (with any integer n_0), prove:

1. **Base case.** $P(n_0)$.
2. **Step.** $\forall k \geq n_0$, $P(k) \Rightarrow P(k+1)$.

This reduces immediately to the $n_0 = 0$ form by reindexing $m = n - n_0$.

4) Quick example

Claim: for all $n \geq 0$, $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.

- **Base:** $n = 0$: empty sum $= 0$, and $\frac{0 \cdot 1}{2} = 0$.
- **Step:** Assume $\sum_{i=1}^{k} i = \frac{k(k+1)}{2}$. Then

$$\sum_{i=1}^{k+1} i = \frac{k(k+1)}{2} + (k+1) = \frac{(k+1)(k+2)}{2},$$

  which is the formula with $k+1$. Hence true for all $n \geq 0$.

2 [15]

Here's the issue with the "proof".

The induction step *implicitly uses a case outside the hypothesis* on the very first step:

- The step computes $a^n = a^{n-1} \cdot a^{n-1}/a^{(n-1)-1}$. That references both a^{n-1} **and** a^{n-2}.
- Strong induction allows you to assume the statement for $1, \ldots, n$; however, when $n = 1$ (the first time the step must be valid), this formula needs $a^{n-2} = a^{-1}$. The statement being proved was only for **positive** integers n; it says nothing about $n = 0$ (which would correspond to the exponent -1 here). So the step is **not valid for** $n = 1$ because it relies on $P(0)$, which isn't part of the induction hypothesis. Put differently: the inductive step requires two earlier cases, $n-1$ and $n-2$; therefore it would need base cases covering up through $n = 2$ and also a statement for $n = 0$. But the proposition for $n = 0$ would be $a^{-1} = 1$, which is false unless $a = 1$.
- Concretely, try $a = 2$. The "proof" would force $2^1 = 1$ at $n = 2$, a contradiction.

So the mistake is an *illegitimate induction step*: it invokes the assertion for $n-2$ (and at the first step, effectively $n = 0$) even though the theorem is only stated for positive n. The step therefore doesn't establish "for all n, if $P(1), \ldots, P(n)$ then $P(n+1)$"—it already fails at $n = 1$.

3 [18]

Let $P(n)$ be the statement

$$\sum_{k=1}^{n-1} \frac{1}{k(k+1)} = \frac{3}{2} - \frac{1}{n}.$$

- For $n = 1$, the left side is an **empty sum**, hence 0. The right side is $3/2 - 1 = 1/2$. So $P(1)$ is **false**.

In the bogus proof, the "base case" line says "for $n = 1$, clearly $\frac{3}{2} - \frac{1}{n} = \frac{1}{1\cdot 2}$." That equality is numerically true ($1/2 = 1/2$) **but it is not** $P(1)$; it mistakenly replaces the empty sum (the actual left-hand side for $n = 1$) with the first term $\frac{1}{1\cdot 2}$, which belongs to $n \geq 2$. In short, the base case checks the wrong statement. Without a correct base case, the induction proves nothing.

(Algebra in the inductive step is fine; the logic fails because the induction starts from a false/unchecked base.)

The correct identity (and proof)

The true formula is

$$\sum_{k=1}^{n-1} \frac{1}{k(k+1)} \;=\; 1 - \frac{1}{n}.$$

Reason: $\frac{1}{k(k+1)} = \frac{1}{k} - \frac{1}{k+1}$ telescopes, giving

$$\left(1 - \tfrac{1}{2}\right) + \left(\tfrac{1}{2} - \tfrac{1}{3}\right) + \cdots + \left(\tfrac{1}{n-1} - \tfrac{1}{n}\right) = 1 - \tfrac{1}{n}.$$

If you want an induction proof:

- Base $n = 1$: empty sum $= 0 = 1 - 1$.
- Step: assume $\sum_{k=1}^{n-1} \frac{1}{k(k+1)} = 1 - \frac{1}{n}$. Then

$$\sum_{k=1}^{n} \frac{1}{k(k+1)} = \left(1 - \frac{1}{n}\right) + \frac{1}{n(n+1)} = 1 - \frac{1}{n+1}.$$

Sanity check at $n = 6$

LHS $= \frac{1}{2} + \frac{1}{6} + \frac{1}{12} + \frac{1}{20} + \frac{1}{30} = \frac{5}{6}$.

- False formula's RHS: $\frac{3}{2} - \frac{1}{6} = \frac{4}{3}$ (doesn't match).
- True formula's RHS: $1 - \frac{1}{6} = \frac{5}{6}$ (matches).

So the error in the exercise's "proof" is a faulty base case: it confuses the empty sum for $n = 1$ with the first term $1/(1 \cdot 2)$.

4 [20]

Ingredients.

1. Fibonacci recurrence: $F_n = F_{n-1} + F_{n-2}$ for $n > 1$ (with $F_0 = 0, F_1 = 1$).
2. Golden-ratio identity: $\varphi^2 = \varphi + 1$ (equivalently, φ is the positive root of $x^2 = x + 1$).
3. Context from the text: together with Eq. (3) in §1.2.1, these bounds form the pair $\varphi^{n-2} \leq F_n \leq \varphi^{n-1}$.

Base cases.

- $n = 1$: $F_1 = 1$ and $\varphi^{1-2} = \varphi^{-1} \approx 0.618$. Hence $F_1 \geq \varphi^{-1}$.
- $n = 2$: $F_2 = 1$ and $\varphi^{2-2} = \varphi^0 = 1$. Hence $F_2 \geq \varphi^0$ (equality).

Inductive step. Assume the inequality holds for all integers up to n (with $n \geq 2$), in particular

$$F_n \geq \varphi^{n-2} \quad \text{and} \quad F_{n-1} \geq \varphi^{n-3}.$$

Then, using the Fibonacci recurrence,

$$F_{n+1} = F_n + F_{n-1} \;\geq\; \varphi^{n-2} + \varphi^{n-3} = \varphi^{n-3}(\varphi + 1) = \varphi^{n-3}\varphi^2 = \varphi^{n-1},$$

where we used $\varphi^2 = \varphi + 1$. Thus $F_{n+1} \geq \varphi^{n-1}$, completing the induction.

Therefore $F_n \geq \varphi^{n-2}$ for all positive integers n.

5 [21]

Base case ($n = 2$).

2 is prime, so it is already a (one-factor) product of primes. Thus $P(2)$ holds.

Induction step. Assume $P(n)$ holds for some $n \geq 2$; i.e., every integer m with $2 \leq m \leq n$ is a product of primes. We must show $P(n+1)$: that $n+1$ is a product of primes.

Consider $n + 1$ in two cases.

1. $n + 1$ is prime. Then $n + 1$ itself is a (one-factor) product of primes, so we're done.
2. $n + 1$ is composite. Then there exist integers a, b with $1 < a \leq b < n + 1$ and $n + 1 = ab$. Hence $2 \leq a \leq n$ and $2 \leq b \leq n$. By the induction hypothesis, both a and b can be written as products of primes. Multiplying those two prime products gives a prime product for $n + 1$.

In either case, $n+1$ is a product of primes, so $P(n+1)$ holds. By strong induction, $P(n)$ is true for all $n \geq 2$.

Every integer > 1 can be expressed as a product of primes (existence of prime factorization).

6 [20]

In Algorithm E (Extended Euclid) Knuth defines variables a', a, b', b, c, d and at step **E2** we compute q, r with

$$c = qd + r, \qquad 0 \leq r < d.$$

Step **E4** then updates the variables as follows:

$$\begin{aligned} &c \leftarrow d, \quad d \leftarrow r, \\ &t \leftarrow a', \quad a' \leftarrow a, \quad a \leftarrow t - qa, \\ &t \leftarrow b', \quad b' \leftarrow b, \quad b \leftarrow t - qb, \end{aligned}$$

and the claimed invariants are

$$a'm + b'n = c, \qquad am + bn = d. \tag{6}$$

The problem is to **prove that if (6) holds just before E4, it still holds immediately after E4**.

Assume, just before executing **E4**, that

$$a'm + b'n = c, \qquad am + bn = d,$$

and that at **E2** we computed q, r with $c = qd + r$.

Let "old" denote values before E4 and "new" the values after E4.

From the assignment list of **E4** we have:

$$\begin{aligned} &\text{new } c = \text{old } d, \\ &\text{new } d = \text{old } r = \text{old } c - q\,\text{old } d, \\ &\text{new } a' = \text{old } a, \qquad \text{new } a = \text{old } a' - q\,\text{old } a, \\ &\text{new } b' = \text{old } b, \qquad \text{new } b = \text{old } b' - q\,\text{old } b. \end{aligned}$$

Invariant 1

$$\begin{aligned}\text{new } a'\,m + \text{new } b'\,n &= (\text{old } a)m + (\text{old } b)n \\ &= \text{old } (am + bn) \\ &= \text{old } d \\ &= \text{new } c.\end{aligned}$$

So $a'm + b'n = c$ holds after E4.

Invariant 2

$$\begin{aligned}\text{new } a\,m + \text{new } b\,n &= (\text{old } a' - q\,\text{old } a)m + (\text{old } b' - q\,\text{old } b)n \\ &= \text{old } (a'm + b'n) - q\,\text{old } (am + bn) \\ &= \text{old } c - q\,\text{old } d \\ &= \text{old } r \\ &= \text{new } d.\end{aligned}$$

So $am + bn = d$ also holds after E4.

Both equalities (6) are therefore preserved by E4. Because (6) is also true at initialization (E1 sets $a' = 1, a = 0, b' = 0, b = 1, c = m, d = n$, which clearly satisfies $a'm + b'n = m = c$ and $am + bn = n = d$), the invariants hold every time we return to E2. This is exactly the fact Knuth alludes to next to Eq. (6).

7 [23]

Let

$$S_n \;=\; n^2 - (n-1)^2 + (n-2)^2 - \cdots + (-1)^{n-1}1^2$$

(i.e., start at n^2 and alternate signs down to 1^2). We claim

$$\boxed{S_n = \frac{n(n+1)}{2}} \quad \text{for all } n \geq 1.$$

Induction proof

Base $n = 1$**.** $S_1 = 1^2 = 1 = \frac{1\cdot 2}{2}$, true.

Inductive step. Assume $S_n = \frac{n(n+1)}{2}$ for some $n \geq 1$. Observe the simple recurrence (just "prepend" $(n+1)^2$ and flip all subsequent signs):

$$S_{n+1} = (n+1)^2 - S_n.$$

Therefore,

$$S_{n+1} = (n+1)^2 - \frac{n(n+1)}{2} = (n+1)\Big((n+1) - \frac{n}{2}\Big) = \frac{(n+1)(n+2)}{2},$$

which is the desired formula with n replaced by $n+1$. By induction, the identity holds for all $n \geq 1$.

8 [25]

(a) Nicomachus's theorem (cubes are sums of consecutive odd numbers)

Claim. For every integer $k \geq 1$,

$$k^3 = \underbrace{(k^2 - k + 1) + (k^2 - k + 3) + \cdots + (k^2 + k - 1)}_{k \text{ consecutive odd numbers}}.$$

Proof by induction on k.

- Base case $k = 1$. Right side is 1; indeed $1^3 = 1$.
- Inductive step. Assume the formula holds for $k = n$:

$$n^3 = \sum_{i=0}^{n-1} (n^2 - n + 1 + 2i).$$

Consider $k = n+1$. The next block of $(n+1)$ consecutive odd numbers runs

$$n^2 + n + 1,\ n^2 + n + 3,\ \ldots,\ n^2 + 3n + 1,$$

i.e., $\sum_{i=0}^{n} (n^2 + n + 1 + 2i)$. Its sum is

$$(n+1)(n^2+n+1) + 2\sum_{i=0}^{n} i = (n+1)(n^2+n+1) + n(n+1) = (n+1)(n^2+2n+1) = (n+1)^3.$$

Adding this block to the block that sums to n^3 "moves" from the n-case to the $(n+1)$-case, so the pattern holds for $n+1$.

Thus, by induction, the claim is true for all $k \geq 1$.

(b) Sum of the first n cubes

From part (a), k^3 is the sum of a block of k consecutive odd numbers. Concatenating the blocks for $k = 1, 2, \ldots, n$ uses exactly

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

odd numbers. But the sum of the first m odd numbers is m^2 (proved earlier in the section as $1 + 3 + \cdots + (2m-1) = m^2$).

Set $m = \dfrac{n(n+1)}{2}$. Then

$$\sum_{k=1}^{n} k^3 = (\text{sum of the first } m \text{ odd numbers}) = m^2 = \left(\frac{n(n+1)}{2}\right)^2 = (1 + 2 + \cdots + n)^2.$$

This is exactly the desired identity.

9 [20]

Claim. For every integer $n \geq 1$ and $0 < a < 1$, $(1-a)^n \geq 1 - na$.

Base case ($n = 1$). $(1-a)^1 = 1 - a = 1 - 1 \cdot a$. True (equality).

Induction step. Assume for some $n \geq 1$ that $(1-a)^n \geq 1 - na$. Because $0 < a < 1$, we have $1 - a \geq 0$. Multiplying the induction hypothesis by the nonnegative number $(1-a)$ preserves the inequality:

$$(1-a)^{n+1} = (1-a)^n(1-a) \;\geq\; (1-na)(1-a) = 1 - (n+1)a + na^2 \;\geq\; 1 - (n+1)a,$$

since $na^2 \geq 0$. Thus the statement holds for $n + 1$.

By induction, the inequality holds for all integers $n \geq 1$.

10 [M22]

Base case ($n = 10$). $2^{10} = 1024 > 1000 = 10^3$. So the statement holds for $n = 10$.

Inductive step. Assume $2^n > n^3$ for some $n \geq 10$. We must show $2^{n+1} > (n+1)^3$.

From the inductive hypothesis,

$$2^{n+1} = 2 \cdot 2^n \ > \ 2\,n^3.$$

Thus it suffices to prove $2\,n^3 \geq (n+1)^3$, i.e.

$$2 \ \geq \ \left(1 + \frac{1}{n}\right)^3.$$

For $n \geq 4$ we have

$$\left(1 + \frac{1}{n}\right)^3 = 1 + \frac{3}{n} + \frac{3}{n^2} + \frac{1}{n^3} \leq 1 + \frac{3}{4} + \frac{3}{16} + \frac{1}{64} = \frac{125}{64} < 2.$$

Since our induction starts at $n \geq 10$, the inequality certainly holds; hence

$$2^{n+1} \ > \ 2\,n^3 \ \geq \ (n+1)^3,$$

which completes the inductive step.

By induction, $2^n > n^3$ for all $n \geq 10$.

Remarks

- The threshold $n \geq 10$ is sharp: $2^9 = 512 < 729 = 9^3$, so the claim fails at $n = 9$ but holds from $n = 10$ onward.
- An equivalent (and sometimes handy) way to see the step is to expand the difference directly:

  $$2n^3 - (n+1)^3 = n^3 - 3n^2 - 3n - 1 = (n-1)^3 - 6n,$$

  which is > 0 for all $n \geq 10$ (e.g., at $n = 10$: $729 - 60 = 669 > 0$).

11 [M30]

Step 1: A key algebraic identity

Factor the denominator:

$$x^4 + 4 = (x^2 + 2x + 2)(x^2 - 2x + 2) = (x+1)^2 + 1 \quad \times \quad (x-1)^2 + 1.$$

Seek constants A, B, C, D with

$$\frac{x^3}{x^4+4} = \frac{Ax+B}{x^2+2x+2} + \frac{Cx+D}{x^2-2x+2}.$$

Equating coefficients gives $A = C = \frac{1}{2}$, $B = \frac{1}{2}$, $D = -\frac{1}{2}$. Hence

$$\boxed{\frac{x^3}{x^4+4} = \frac{1}{2}\left(\frac{x+1}{x^2+2x+2} + \frac{x-1}{x^2-2x+2}\right) = \frac{1}{2}\left(\frac{x+1}{(x+1)^2+1} + \frac{x-1}{(x-1)^2+1}\right)} \qquad ( )$$

Step 2: Plug in $x = 2k+1$

With $x = 2k+1$,

$$\frac{(2k+1)^3}{(2k+1)^4+4} = \frac{1}{2}\left(\frac{2k+2}{(2k+2)^2+1} + \frac{2k}{(2k)^2+1}\right).$$

Therefore

$$S_n = \frac{1}{2}\sum_{k=0}^{n}(-1)^k\left(\frac{2k+2}{(2k+2)^2+1} + \frac{2k}{(2k)^2+1}\right).$$

Split into two sums and shift index in the first:

$$\begin{aligned} S_n &= \frac{1}{2}\left[\sum_{k=0}^{n}(-1)^k\frac{2k}{(2k)^2+1} + \sum_{k=0}^{n}(-1)^k\frac{2k+2}{(2k+2)^2+1}\right] \\ &= \frac{1}{2}\left[\sum_{j=0}^{n}(-1)^j\frac{2j}{(2j)^2+1} - \sum_{j=1}^{n+1}(-1)^j\frac{2j}{(2j)^2+1}\right]. \end{aligned}$$

Now everything cancels **telescopically** except the endpoint terms $j = 0$ (which is 0) and $j = n+1$:

$$S_n = \frac{1}{2}\left[0 - (-1)^{n+1}\frac{2(n+1)}{(2(n+1))^2+1}\right] = \boxed{(-1)^n\,\frac{n+1}{4(n+1)^2+1}}.$$

Quick checks

- $n=0$: $S_0 = \dfrac{1}{1^4+4} = \dfrac{1}{5}$. Formula gives $\dfrac{1}{4\cdot 1^2+1} = \dfrac{1}{5}$.
- $n=1$: $S_1 = \dfrac{1}{5} - \dfrac{27}{85} = -\dfrac{2}{17}$. Formula gives $-\dfrac{2}{17}$.

Answer. $\boxed{\displaystyle\sum_{k=0}^{n}(-1)^k\frac{(2k+1)^3}{(2k+1)^4+4} = (-1)^n\frac{n+1}{4(n+1)^2+1}}$.

12 [M25]

Generalize Euclid's Algorithm **E** so it can accept inputs of the form $u+v\sqrt{2}$ (with $u,v\in\mathbb{Z}$) and still do all computations "elementarily" (i.e., without using an infinite decimal for $\sqrt{2}$). Then prove the computation does **not** terminate when $m=1$ and $n=\sqrt{2}$. (Knuth hints at this generalization and the non-termination case in the text; the exercise states it explicitly.)

A. Generalizing Algorithm E to inputs $u+v\sqrt{2}$ Representation and primitive operations

Represent each value x as an integer pair (u,v) meaning $x=u+v\sqrt{2}$.

- **Addition/Subtraction.** $(u_1,v_1)\pm(u_2,v_2) = (u_1\pm u_2,\ v_1\pm v_2)$ — integer work only.
- **Zero test.** $(u,v)=0 \iff u=0\wedge v=0$.
- **Sign/Comparison without decimals.** To decide whether $(u,v)\geq 0$ (hence compare two numbers by looking at their difference), use exact integer tests:

$$u+v\sqrt{2}\geq 0 \quad\iff\quad \begin{cases}\text{if } v\geq 0: & u\geq 0 \ \text{ or } \ u^2\leq 2v^2,\\ \text{if } v<0: & u>0 \ \text{ and } \ u^2\geq 2v^2.\end{cases}$$

  This comes from analyzing $u\geq -v\sqrt{2}$ and squaring only in the cases where both sides have the same sign; it uses only integer arithmetic.

The "division with integer quotient" step

Euclid's step **E1** needs: given positive m,n (now of form $u+v\sqrt{2}$), find integers q and a remainder r with

$$m = qn + r, \qquad 0\leq r<n,$$

where $q \in \mathbb{Z}$ (as in Knuth's discussion) and r is again of the form $u + v\sqrt{2}$.

You can compute q **without decimals** as follows.

1. **Coarse integer bounds for $\sqrt{2}$.** Use $1 \le \sqrt{2} \le 2$ to bound any $u + v\sqrt{2}$ by integers:

   - If $v \ge 0$: $u + v \le u + v\sqrt{2} \le u + 2v$.
   - If $v < 0$: $u + 2v \le u + v\sqrt{2} \le u + v$.

   From these bounds for m and n you get integer bounds $L \le m/n < U$.

2. **Integer bisection on q.** Do binary search on the integer interval $[\lfloor L \rfloor, \lceil U \rceil]$ to find the unique q satisfying

$$qn \le m < (q+1)n,$$

   using only the sign test above to compare $m - (q)n$ and $m - (q+1)n$ with zero.

3. **Remainder.** Set $r = m - qn$ (pair subtraction). Then either $r = 0$ or $0 \le r < n$.

This realizes **E1** with only integer operations and exact comparisons. The rest of Algorithm **E** is unchanged:

- **E2.** If $r = 0$ return n (the "gcd" in this setting).
- **E3.** Set $m \leftarrow n,\ n \leftarrow r$ and repeat.

Knuth notes (and gives an example) that with this extension the inductive assertions A_1–A_6 still hold; e.g., starting with $m = 12 - 6\sqrt{2}$ and $n = 20 - 10\sqrt{2}$, the method returns $d = 4 - 2\sqrt{2}$.

B. Non-termination for $m = 1,\ n = \sqrt{2}$

Run the generalized algorithm:

1. Start with $m = 1$, $n = \sqrt{2}$. Since $m < n$, the quotient is $q = 0$ and $r = 1$. After the swap: $m \leftarrow \sqrt{2},\ n \leftarrow 1$.

2. Now $m/n = \sqrt{2}$. Thus $q = \lfloor \sqrt{2} \rfloor = 1$, so $r = \sqrt{2} - 1$. Swap: $m \leftarrow 1,\ n \leftarrow \sqrt{2} - 1$.

3. Next $m/n = \dfrac{1}{\sqrt{2} - 1} = \sqrt{2} + 1$. Hence $q = 2$ and

$$r = 1 - 2(\sqrt{2} - 1) = 3 - 2\sqrt{2} = (\sqrt{2} - 1)^2.$$

   Swap: $m \leftarrow \sqrt{2} - 1,\ n \leftarrow (\sqrt{2} - 1)^2$.

4. Repeat: one checks inductively that at each subsequent step

$$q = 2, \qquad r = (\sqrt{2}-1)^{k+2} \quad \text{when } (m,n) = ((\sqrt{2}-1)^k, (\sqrt{2}-1)^{k+1}).$$

Thus the remainders are $(\sqrt{2}-1), (\sqrt{2}-1)^2, (\sqrt{2}-1)^3, \ldots$

Since $0 < \sqrt{2}-1 < 1$, this positive sequence is strictly decreasing and tends to 0, but it never becomes 0. Therefore the algorithm **never reaches** $r = 0$ and hence **does not terminate** for $m = 1,\ n = \sqrt{2}$. This is exactly the phenomenon Knuth points out in the text: the usual inductive correctness assertions don't, by themselves, establish termination in this generalized setting.

13 [M23]

The T-augmented assertions

Let n denote the original input value that becomes d at $E1$. We append the following bounds to the six assertions $A1, \ldots, A6$ (all other parts of each assertion remain unchanged exactly as in Fig. 4):

- **A1**: $m > 0,\ n > 0,\ \boxed{T = 0}$. (This already implies $T \le 3n$.)
- **A2** (first entry to $E2$): $c = m > 0,\ d = n > 0,\ a = b' = 0,\ a' = b = 1,\ \boxed{T \le 3(n-d)+1}$. (Here $d = n$, so this is $T \le 1$; together with the exercise's "append $T = 1$ to $A2$" it is consistent.)
- **A3** (after $E2$): $am + bn = d,\ a'm + b'n = c = qd + r,\ 0 \le r < d,\ \gcd(c,d) = \gcd(m,n),\ \boxed{T \le 3(n-d)+2}$.
- **A4** (if $r = 0$ at $E3$, i.e., about to stop): $am + bn = d = \gcd(m,n),\ \boxed{T \le 3(n-d)+3}$.
- **A5** (if $r \ne 0$ at $E3$): $am + bn = d,\ a'm + b'n = c = qd + r,\ 0 < r < d,\ \gcd(c,d) = \gcd(m,n),\ \boxed{T \le 3(n-d)+3}$.
- **A6** (after $E4$): $am+bn = d,\ a'm+b'n = c,\ d > 0,\ \gcd(c,d) = \gcd(m,n),\ \boxed{T \le 3(n-d)+1}$.

Why these particular constants $+1, +2, +3, +3, +1$? They are chosen so that the "one-step" increase of T when entering a box is exactly matched by the change (or non-change) of d in that box, making every implication in the flow-chart proof go through mechanically.

Inductive verification (box by box)

Use Knuth's proof rule: *If the assertion on every arrow entering a box is true* ***before*** *the box executes, then after execution the assertions on all outgoing arrows are true.* I'll verify only the T-part (the non-T parts are exactly those in Fig. 4 and are already known to propagate).

E1 (Initialize). Enter with $A1$: $T = 0$. Executing $E1$ sets $d \leftarrow n$ and (by the exercise) increments T to 1. The outgoing arrow is $A2$, which demands $T \leq 3(n-d)+1$. Since here $d = n$, this is $T \leq 1$, satisfied by $T = 1$.

E2 (Divide). There are two possible incoming arrows:

- From **A2**: we had $T \leq 3(n-d)+1$. Entering $E2$ adds 1 to T, while d is unchanged. Thus after $E2$, $T \leq 3(n-d)+2$, exactly the bound in **A3**.
- From **A6**: we had $T \leq 3(n-d)+1$. Again T gains 1 and d is unchanged, so we get $T \leq 3(n-d)+2$, i.e., **A3**. (As Knuth notes, $A2$ is stronger than $A6$, but either one suffices to imply $A3$ after $E2$.)

E3 (Remainder zero?). Incoming **A3** has $T \leq 3(n-d)+2$; entering $E3$ bumps T by 1, d stays the same.

- If $r = 0$: outgoing **A4** requires $T \leq 3(n-d)+3$, which now holds.
- If $r \neq 0$: outgoing **A5** requires the same bound $T \leq 3(n-d)+3$, also holds.

E4 (Recycle). Incoming **A5** has $T \leq 3(n-d)+3$. Entering $E4$ adds 1 to T and sets $d \leftarrow r$ with $0 < r < d$. So after $E4$,

$$T \;\leq\; 3(n-d)+3 \;+1 \;=\; 3(n-d)+4.$$

Because $r \leq d-1$, we have $3(n-r)+1 \;\geq\; 3(n-(d-1))+1 \;=\; 3(n-d)+4$. Hence

$$T \;\leq\; 3(n-r)+1 \;=\; 3(n-d_{\text{new}})+1,$$

which is exactly the T-bound required by **A6**.

Finally, note the next transition is **A6** $\rightarrow$ **E2**: entering $E2$ adds 1 to T with d unchanged, turning $T \leq 3(n-d)+1$ (from $A6$) into $T \leq 3(n-d)+2$ (the bound in $A3$), as already verified above.

Why every assertion implies $T \leq 3n$

In every state we have $d \geq 1$ (the chart maintains $d > 0$ except at termination), so for each assertion the added bound is at most

$$\max_d \{\, 3(n-d)+c_i \,\} \;\leq\; 3(n-1)+3 \;=\; 3n,$$

since the largest constant used is 3. Thus *any* of $A1, \ldots, A6$ implies $T \leq 3n$. (At the stop arrow **A4** we also have $d = \gcd(m,n) \geq 1$.)

14 [50]

15 [HM28]

(a) The integers $\mathbb{Z}$ are **not** well-ordered by the usual "$<$"

Take the nonempty subset $A = \{\dots, -3, -2, -1\} \subset \mathbb{Z}$. If "$<$" were a well-ordering on $\mathbb{Z}$, A would have a least element. But for any $a \in A$, $a - 1 \in A$ and $a - 1 < a$. Hence A has no least element. Thus "$<$" is not a well-ordering on $\mathbb{Z}$. (Condition (iii) fails.)

(b) A well-ordering on $\mathbb{Z}$

Define " " on $\mathbb{Z}$ by **short-absolute-value order** (sometimes called the 0,1,−1,2,−2,… order):

$$0 \prec 1 \prec -1 \prec 2 \prec -2 \prec 3 \prec -3 \prec \cdots,$$

equivalently: for $a, b \in \mathbb{Z}$,

$$a \prec b \iff (|a| < |b|) \text{ or } (|a| = |b| \text{ and } a \geq 0 > b).$$

- (i) Transitivity and (ii) trichotomy are immediate from the definition.
- (iii) Let $X \subseteq \mathbb{Z}$ be nonempty. Consider the set of absolute values $B = \{|x| : x \in X\} \subset \mathbb{N}$. Since $\mathbb{N}$ is well-ordered by $<$, B has a least element r. Then X contains either r or $-r$ (or both). If both occur, pick r (the nonnegative one). That chosen element is least in X under " ". Hence " " is a well-ordering of $\mathbb{Z}$.

(Another equivalent description: map $f : \mathbb{Z} \to \mathbb{N}$ by $f(0) = 0,\ f(1) = 1,\ f(-1) = 2,\ f(2) = 3,\ f(-2) = 4, \dots$. Then define $a \prec b$ iff $f(a) < f(b)$. Since $(\mathbb{N}, <)$ is well-ordered, this transports a well-ordering to $\mathbb{Z}$.)

(c) $[0, \infty) \subset \mathbb{R}$ is **not** well-ordered by "$<$"

Consider $A = \{1/n : n \in \mathbb{N}\} \subset (0, 1]$. This A is nonempty but has no least element in the usual order: for any $1/n \in A$, there is $1/(n+1) \in A$ with $1/(n+1) < 1/n$. Thus "$<$" is not a well-ordering on the nonnegative reals.

(d) Lexicographic order on $T_n = S^n$ is a well-ordering (finite products)

Assume $(S, \prec)$ is well-ordered. Define lexicographic order on $T_n = S^n$: $(x_1, \dots, x_n) \prec (y_1, \dots, y_n)$ if for some k (with $1 \leq k \leq n$) we have $x_j = y_j$ for $j < k$ and $x_k \prec y_k$ in S.

- (i) and (ii) are standard for lex order.

- (iii) Let $U \subseteq T_n$ be nonempty. Choose x_1 minimal among the first coordinates of elements of U (possible since S is well-ordered). Among those elements of U whose first coordinate equals that x_1, choose the minimal second coordinate x_2; continue this process finitely many steps. We obtain $(x_1, \dots, x_n) \in U$ which is $\prec$-least in U. Hence lex order well-orders T_n.

(e) "Shortlex" on $T = \bigcup_{n \geq 1} T_n$ is also a well-ordering

Define $(x_1, \dots, x_m) \prec (y_1, \dots, y_n)$ if either:

1. they agree up to some first differing position $k \leq \min(m, n)$ and $x_k \prec y_k$ in S; or
2. $m < n$ and $(x_1, \dots, x_m)$ is a proper prefix of $(y_1, \dots, y_n)$ (i.e., all first m entries are equal).

- (i) and (ii) again hold (transitivity and totality).
- (iii) Let $U \subseteq T$ be nonempty. Consider the set of lengths $L = \{ m \geq 1 : \exists \mathbf{x} \in U \text{ of length } m \} \subset \mathbb{N}$. Since $\mathbb{N}$ is well-ordered, L has a least length m_0. Then $U \cap T_{m_0}$ is nonempty, and by part (d) the lex order well-orders T_{m_0}. Hence U has a least element (the lex-least among those of minimal length). So "shortlex" well-orders T.

(f) Characterization via absence of infinite descending chains

Claim: A relation " " on S is a well-ordering **iff** it satisfies (i) transitivity, (ii) trichotomy, **and** there is **no** infinite strictly descending chain $x_1 \succ x_2 \succ x_3 \succ \cdots$.

- **( )** If " " is a well-ordering and there were such a chain, then the set $\{x_1, x_2, \dots\}$ would be a nonempty subset without a least element (since each $x_{j+1} \prec x_j$), contradicting (iii).
- **( )** Suppose (i) and (ii) hold and no infinite descending chain exists. Let $A \subseteq S$ be nonempty. If A had no least element, pick $x_1 \in A$. Since x_1 is not least, there is $x_2 \in A$ with $x_2 \prec x_1$. Inductively build $x_{k+1} \prec x_k$ in A forever, making an infinite descending chain—a contradiction. Hence every nonempty subset has a least element, i.e., " " is a well-ordering.

(g) Well-ordering induction (transfinite induction)

Assume $(S, \prec)$ is well-ordered and $P(x)$ is a statement about $x \in S$ with the property:

For each $x \in S$, if $P(y)$ holds for all $y \prec x$, then $P(x)$ holds.

To prove $P(x)$ for all $x \in S$, suppose not and let $C = \{x \in S : \neg P(x)\}$. If $C \neq \emptyset$, well-ordering gives a $\prec$-least $c \in C$. But then $P(y)$ holds for all $y \prec c$, so the premise yields $P(c)$—contradiction. Hence $C = \emptyset$ and P holds everywhere. This is the general well-ordering (transfinite) induction principle; when $S = \mathbb{N}$ with the usual "$<$", it reduces to ordinary induction.

1.2.2 Numbers, powers, and logarithms

1 [00]

There is **no** smallest positive rational number.

Assume there is a smallest positive rational number; call it r. Since $r > 0$, the number $r/2$ is also rational and positive, and $r/2 < r$. This contradicts the assumption that r was the smallest positive rational. Therefore such a smallest element does not exist.

Notes: - Rationals are numbers of the form p/q with integers p and $q > 0$ (see the section's definitions of integers/rationals), but the above argument doesn't even need their explicit form: it uses only closure under division by 2 and positivity. - In order-theory terms, the set $\{x \in \mathbb{Q} : x > 0\}$ has no minimum; its greatest lower bound (infimum) is 0. - Contrast: the positive integers do have a least element (namely 1); the positive rationals (and reals) are not well-ordered under the usual $<$.

2 [00]

What Knuth means by "decimal expansion"

In §1.2.2 Knuth defines a real number's decimal expansion as $x = n + 0.d_1 d_2 d_3 \ldots$ with digits $d_i \in \{0, \ldots, 9\}$, and with the explicit convention that the digit sequence does not end with infinitely many 9s (to make the expansion unique).

Answer

No. As written, $1 + 0.239999999\ldots$ (i.e., $1.239999999\ldots$) is not an allowed "decimal expansion" under Knuth's convention, because it ends with infinitely many 9s. However, it does represent a perfectly good real number—namely 1.24—and the proper (canonical) decimal expansion for that number is $1.240000\ldots$.

Why $0.239999\ldots = 0.24$

Compute the tail as a geometric series:

$$0.239999\ldots = 0.23 + \sum_{k=3}^{\infty} \frac{9}{10^k} = 0.23 + 9 \cdot \frac{10^{-3}}{1 - \frac{1}{10}} = 0.23 + 9 \cdot \frac{10^{-3}}{\frac{9}{10}} = 0.23 + 10^{-2} = 0.24.$$

Therefore,

$$1 + 0.239999\ldots = 1 + 0.24 = 1.24.$$

3 [02]

1. Use the definition of negative integer powers. For any nonzero a and positive integer n,

$$a^{-n} = \frac{1}{a^n}.$$

2. Evaluate the positive power in the denominator.

$$(-3)^3 = (-3) \cdot (-3) \cdot (-3) = 9 \cdot (-3) = -27,$$

   since an odd number of negative factors yields a negative result.

3. Invert.

$$(-3)^{-3} = \frac{1}{(-3)^3} = \frac{1}{-27} = -\frac{1}{27}.$$

4. Quick check.

$$(-3)^{-3} \cdot (-3)^3 = (-3)^0 = 1,$$

   confirming the result.

Answer: $-\frac{1}{27}$.

4 [05]

Solution, two equivalent ways.

1. Use base-2 and exponent laws. $0.125 = \frac{1}{8} = 2^{-3}$. Using $(b^x)^y = b^{xy}$,

$$(0.125)^{-2/3} = (2^{-3})^{-2/3} = 2^{(-3)\cdot(-2/3)} = 2^2 = 4.$$

(The law $(b^x)^y = b^{xy}$ for exponents appears in Eq. (5).)

2. Use the definition of rational exponents. For $a > 0$ and rational p/q, $a^{p/q} = \sqrt[q]{a^p}$ (Eq. (6)). Thus

$$(0.125)^{-2/3} = \frac{1}{(0.125)^{2/3}} = \frac{1}{\left(\sqrt[3]{0.125}\right)^2} = \frac{1}{\left(\sqrt[3]{\frac{1}{8}}\right)^2} = \frac{1}{\left(\frac{1}{2}\right)^2} = 4.$$

Answer: 4.

5 [05]

Let x be a real number. Write

$$x = n + 0.d_1 d_2 d_3 \ldots \quad \text{(base 2)},$$

meaning

$$x \;=\; n + \sum_{i=1}^{\infty} \frac{d_i}{2^i}, \quad \text{where each } d_i \in \{0, 1\}.$$

To make this representation **unique**, forbid tails of infinitely many 1s (exactly analogous to forbidding infinitely many 9s in decimal). With that convention, the **binary analogue of Eq. (2)** is:

$$\boxed{n + \sum_{i=1}^{k} \frac{d_i}{2^i} \;\le\; x \;<\; n + \sum_{i=1}^{k} \frac{d_i}{2^i} + \frac{1}{2^k} \qquad \text{for all } k \ge 1.}$$

Why this works

Split the infinite sum at k:

$$x = \underbrace{\left(n + \sum_{i=1}^{k} \frac{d_i}{2^i}\right)}_{S_k} + \underbrace{\sum_{i=k+1}^{\infty} \frac{d_i}{2^i}}_{R_k}.$$

Because $d_i \in \{0, 1\}$,

$$0 \le R_k < \sum_{i=k+1}^{\infty} \frac{1}{2^i} = \frac{1}{2^k}.$$

Hence $S_k \le x < S_k + 2^{-k}$, which is exactly the displayed inequality.

Existence and digit construction (greedy rule)

Given any real x, let $n = \lfloor x \rfloor$. For the fractional part $\{x\} = x - n$, define digits iteratively:

$$d_i = \lfloor 2\, r_{i-1} \rfloor, \qquad r_i = 2\, r_{i-1} - d_i, \qquad r_0 = \{x\}.$$

This produces $d_i \in \{0, 1\}$ and $\{x\} = \sum_{i \ge 1} d_i / 2^i$. If the process would eventually give all 1s, switch to the equivalent finite form (e.g., $0.0111\ldots_2 = 0.1000\ldots_2$) and keep our "no infinite 1s" rule for uniqueness.

6 [10]

Rule (the "lexicographic" comparison)

1. Compare integer parts. • If $m < n$ then $x < y$. • If $m > n$ then $x > y$. • If $m = n$, go to step 2.
2. Compare fractional digits left to right. Let k be the first position with $d_k \ne e_k$. • If no such k exists (all digits match), then $x = y$. • If $d_k < e_k$, then $x < y$; if $d_k > e_k$, then $x > y$.

That's it. This works for positive and negative numbers because m is the floor of the real (so $x \in [m, m+1)$, $y \in [n, n+1)$); the fractional part $0.d_1 d_2 \ldots$ is always in $[0, 1)$.

Why the rule is correct (brief proof)

• If $m \ne n$: Since $0.d_1 d_2 \ldots \in [0, 1)$, we have $x \in [m, m+1)$ and $y \in [n, n+1)$. Hence $x < y \iff m < n$. (Likewise for $>$.)

• If $m = n$: Write

$$x - y = \sum_{j \ge 1} \frac{d_j - e_j}{10^j}.$$

If all digits match, the sum is 0, so $x = y$.

Otherwise, let k be the first index with $d_k \ne e_k$. Then

$$x - y = \frac{d_k - e_k}{10^k} + R, \qquad R = \sum_{j \ge k+1} \frac{d_j - e_j}{10^j}.$$

We have $|R| \le \frac{9}{10^{k+1}} + \frac{9}{10^{k+2}} + \cdots = \frac{1}{10^k}$. Because Knuth disallows decimal expansions that *end* with infinitely many 9s, that bound is in fact *strict* here: $|R| < \frac{1}{10^k}$. Since $|d_k - e_k| \ge 1$, the

leading term $(d_k - e_k)/10^k$ dominates R, so $\operatorname{sign}(x-y) = \operatorname{sign}(d_k - e_k)$. Therefore $x < y$ iff $d_k < e_k$, etc.

Notes and examples

- Equality test is simple because of the uniqueness convention: $x = y$ iff $m = n$ and $d_j = e_j$ for all j (no "$0.999\ldots = 1.000\ldots$" ambiguity under Knuth's definition).
- Negative numbers behave the same way. Example: $x = -2 + 0.3 = -1.7$, $y = -2 + 0.7 = -1.3$. Here $m = n = -2$, first differing digit is $k = 1$ with $3 < 7$, so $x < y$.

7 [M23]

Assuming $b > 0$ and using the *definition* of integer powers

$$b^0 = 1, \quad b^n = b^{n-1} b \ (n > 0), \quad b^n = b^{n+1}/b \ (n < 0),$$

prove the "laws of exponents" for all integers x, y: (i) $b^{x+y} = b^x b^y$ and (ii) $(b^x)^y = b^{xy}$. (See the definition in Eq. (4) and the request of Ex. 7 in §1.2.2.)

We proceed in two steps. Throughout, $b > 0$, so $b^n \neq 0$ for all integers n.

1) Product rule: $b^{x+y} = b^x b^y$ for all integers x, y

Step A (nonnegative addend). Fix any integer x. We show by induction on $n \geq 0$ that

$$b^{x+n} = b^x b^n.$$

- **Base $n = 0$:** $b^{x+0} = b^x = b^x \cdot b^0$ since $b^0 = 1$.
- **Induction step:** If $b^{x+n} = b^x b^n$, then

$$b^{x+(n+1)} = (b^{x+n})\, b = (b^x b^n)\, b = b^x b^{n+1},$$

  using the defining recurrence for positive exponents.

Thus, for any integer x and any $n \geq 0$, $b^{x+n} = b^x b^n$.

Step B (negative addend). Let $y < 0$ and write $y = -k$ with $k \geq 0$. Using Step A with the nonnegative addend k,

$$b^{(x+y)} = b^{x-k} \quad \text{and} \quad b^x = b^{x-k} b^k.$$

Multiply both sides of the second identity by b^{-k} (i.e., divide by b^k):

$$b^x b^{-k} = b^{x-k}.$$

Since $y = -k$, this is $b^x b^y = b^{x+y}$.

Combining Steps A and B yields $b^{x+y} = b^x b^y$ for all integers x, y.

2) Power-of-a-power rule: $(b^x)^y = b^{xy}$ for all integers x, y

Case 1: $y \geq 0$**.** Induct on y.

- **Base** $y = 0$**:** $(b^x)^0 = 1 = b^0 = b^{x \cdot 0}$.
- **Step:** If $(b^x)^n = b^{xn}$, then

$$(b^x)^{n+1} = (b^x)^n \, b^x = b^{xn} \, b^x = b^{x(n+1)},$$

using the product rule just proved.

Case 2: $y < 0$**.** Write $y = -k$ with $k > 0$. Then

$$(b^x)^y = (b^x)^{-k} = \frac{1}{(b^x)^k} = \frac{1}{b^{xk}} = b^{-xk} = b^{xy}.$$

This establishes $(b^x)^y = b^{xy}$ for all integers x, y.

8 [25]

1) Existence via a monotone digit-by-digit construction

We construct a nondecreasing sequence $(r_k)_{k \geq 0}$ of decimal truncations that converges to a number y with $y^m = u$.

Step 0 (integer part). Let n be the unique integer with

$$n^m \leq u < (n+1)^m.$$

This n exists because $x \mapsto x^m$ is continuous and strictly increasing on $[0, \infty)$, and it is unique because these intervals are disjoint.

Set $r_0 := n$.

Inductive step (the next digit). Suppose we have chosen

$$r_k \;=\; n + \frac{d_1}{10} + \cdots + \frac{d_k}{10^k},$$

with digits $0 \le d_j \le 9$, such that

$$r_k^m \le u < \left(r_k + 10^{-k}\right)^m. \tag{Inv$_k$}$$

Choose $d_{k+1} \in \{0, 1, \dots, 9\}$ to be the **largest** digit for which

$$\left(r_k + d_{k+1} 10^{-(k+1)}\right)^m \le u.$$

Then define $r_{k+1} := r_k + d_{k+1} 10^{-(k+1)}$.

This choice always exists: take $d = 0$ (which satisfies the inequality) and increase d until the inequality would fail—then stop at the last valid digit. Because $x \mapsto x^m$ is increasing, we also get

$$r_{k+1}^m \le u < \left(r_{k+1} + 10^{-(k+1)}\right)^m,$$

so $\mathbf{Inv}_{k+1}$ holds. Thus (r_k) is nondecreasing and bounded above by $r_k + 10^{-k}$.

Convergence. By construction,

$$0 \le y - r_k < 10^{-k} \quad \text{for } y := \lim_{k\to\infty} r_k.$$

Taking limits in $\mathbf{Inv}_k$ and using continuity of $x \mapsto x^m$,

$$y^m \;=\; \lim_{k\to\infty} r_k^m \;=\; u.$$

Hence a positive m-th root **exists** and its decimal digits are determined successively by the rule above.

> **Remark (how to *compute* the test).** To decide whether $\left(r + t\right)^m \le u$ for $t \in \{0,\ 10^{-(k+1)},\ 2 \cdot 10^{-(k+1)}, \dots\}$, you can either (i) multiply out directly, or (ii) use the binomial expansion $(r + t)^m = r^m + m r^{m-1} t + \binom{m}{2} r^{m-2} t^2 + \cdots$ with additions and multiplications only. Either way is valid for this existence/constructibility proof.

2) Uniqueness of the positive m-th root

Suppose $y_1 > 0$ and $y_2 > 0$ with $y_1^m = u = y_2^m$. If $y_1 < y_2$, monotonicity of x^m on $[0, \infty)$ gives $y_1^m < y_2^m$, a contradiction. Thus $y_1 = y_2$. The positive m-th root is **unique**.

(Our digit rule—"take the largest digit keeping $(\cdot)^m \leq u$"—also ensures we get the standard decimal that does **not** end in an infinite string of 9s.)

3) The requested digit-by-digit algorithm (summary)

Given $u > 0$ and integer $m \geq 1$:

1. **Find the integer part n:** the unique n with $n^m \leq u < (n+1)^m$. Set $r_0 = n$.
2. **For $k = 0, 1, 2, \ldots$:** Choose $d_{k+1} \in \{0, \ldots, 9\}$ largest s.t.

$$\left(r_k + d_{k+1} \cdot 10^{-(k+1)}\right)^m \leq u,$$

   then set $r_{k+1} = r_k + d_{k+1} \cdot 10^{-(k+1)}$.
3. The decimal $y = \lim r_k = n.\, d_1 d_2 d_3 \ldots$ satisfies $y^m = u$, with error $< 10^{-k}$ after k digits.

This proves existence (by construction), uniqueness (by monotonicity), and gives the explicit method for successive digits that the exercise asks for.

4) Optional integer-only variant (nice for hand computation)

Let $A_k := \lfloor 10^k r_k \rfloor$ (so $A_0 = n$). Keep the scaled target $U := u$. At stage k, pick the largest $d \in \{0, \ldots, 9\}$ with

$$(10A_k + d)^m \;\leq\; \lfloor 10^{m(k+1)} U \rfloor.$$

Then set $A_{k+1} := 10A_k + d$. The digits are the base-10 digits of A_k. This is equivalent to the real-number method but uses only integer comparisons.

9 [M23]

- For integers n and a fixed base $b > 0$, powers are defined by

$$b^0 = 1, \qquad b^n = b^{n-1}\, b \;(n > 0), \qquad b^n = b^{n+1}/b \;(n < 0).$$

  From this, one proves for **integers** x, y: $b^{x+y} = b^x b^y$ and $(b^x)^y = b^{xy}$.

- For a **rational** $r = p/q$ (with $q > 0$), Knuth defines

$$b^{p/q} := \sqrt[q]{b^p}\,.$$

  Existence and uniqueness of the positive q-th root are noted just before (and used in) this definition.

The exercise asks you to **prove the exponent laws for rational exponents**, assuming they already hold for integers.

Throughout, let $b > 0$ and let rationals be written in lowest terms with positive denominators.

1) Product rule: $b^{r+s} = b^r\, b^s$ for rationals r, s

Write $r = \dfrac{a}{m}$, $s = \dfrac{c}{n}$ with $m, n > 0$. Let

$$S := b^{a/m}\, b^{c/n}.$$

Raise both sides to the **integer** power mn:

$$S^{mn} = \left(b^{a/m}\right)^{mn} \left(b^{c/n}\right)^{mn} = \left((b^{a/m})^m\right)^n \left((b^{c/n})^n\right)^m.$$

By the *definition* of rational powers, $(b^{a/m})^m = b^a$ and $(b^{c/n})^n = b^c$; by the **integer** laws, $(b^a)^n = b^{an}$ and $(b^c)^m = b^{cm}$. Hence

$$S^{mn} = b^{an}\, b^{cm} = b^{an+cm}.$$

Now set

$$T := b^{(an+cm)/(mn)} = b^{r+s}.$$

By definition, $T^{mn} = b^{an+cm}$. Thus $S^{mn} = T^{mn}$, and both S, T are positive; by **uniqueness** of the positive mn-th root, $S = T$. Therefore $b^{r+s} = b^r\, b^s$.

2) Power-of-a-power: $\left(b^r\right)^s = b^{rs}$ for rationals r, s

Again write $r = \dfrac{a}{m}$, $s = \dfrac{c}{n}$ with $m, n > 0$. Let

$$U := \left(b^{a/m}\right)^{c/n}.$$

By **definition** of the rational exponent with denominator n,

$$U^n = \left(b^{a/m}\right)^c.$$

Raise both sides to the integer power m:

$$U^{mn} = \left((b^{a/m})^c\right)^m.$$

Use the integer laws twice: first $(x^c)^m = x^{cm}$ for any positive x, then $(b^{a/m})^m = b^a$. Hence

$$U^{mn} = (b^a)^c = b^{ac}.$$

Now set $V := b^{ac/(mn)} = b^{rs}$. Then $V^{mn} = b^{ac}$. By uniqueness of positive mn-th roots, $U = V$. Thus $\left(b^r\right)^s = b^{rs}$.

3) Consequences: negatives, quotient, and zero

- Taking $s = -r$ in the product rule gives $b^r b^{-r} = b^0 = 1$, hence $b^{-r} = 1/b^r$.
- The quotient law follows: $b^{r-s} = b^r/b^s$ (multiply both sides by b^s).
- Since $0 = 0/1$, the integer definition already gives $b^0 = 1$.

10 [18]

Assume $\log_{10} 2$ were rational. Then there exist integers p, q with $q > 0$ and $\gcd(p, q) = 1$ such that

$$\log_{10} 2 = \frac{p}{q}.$$

Exponentiating base 10 gives

$$10^{p/q} = 2 \quad \Longrightarrow \quad 10^p = 2^q.$$

But $10^p = (2 \cdot 5)^p = 2^p\, 5^p$. Hence

$$2^p\, 5^p = 2^q \quad \Longrightarrow \quad 5^p = 2^{\,q-p}.$$

The left side is a power of the prime 5; the right side is a power of the prime 2. By the Fundamental Theorem of Arithmetic (unique prime factorization), the only way a power of 5 can equal a power of 2 is if both are 1, which would force $p = 0$ and then $2 = 10^0 = 1$, a contradiction.

Therefore the assumption was false, and $\log_{10} 2$ is irrational.

11 [10]

Let $x_0 = \log_{10} 2$. Suppose we know x to d decimal places, so $|x - x_0| \leq \frac{1}{2} \cdot 10^{-d}$. Write the error as $\delta = x - x_0$. Then

$$10^x = 10^{x_0+\delta} = 10^{x_0}\, 10^{\delta} = 2 \cdot 10^{\delta}.$$

To get the first three decimal places of 10^x correct, we need an absolute error $<\ 0.0005$:

$$|\, 10^x - 2 \,| = 2\,|\, 10^{\delta} - 1 \,| < 0.0005 \quad \Longrightarrow \quad |\, 10^{\delta} - 1 \,| < 0.00025.$$

Because 10^{δ} is increasing in δ, this is equivalent to

$$|\delta| < \log_{10}(1.00025).$$

(Here we simply used the definition of logarithm; change of base was introduced earlier in the section.)

Numerically,

$$\log_{10}(1.00025) \approx 0.00010856.$$

Therefore we need

$$\tfrac{1}{2} \cdot 10^{-d} \ \leq\ 0.00010856 \iff 10^{-d} \leq 2.1712 \times 10^{-4} \iff d \gtrsim 3.66.$$

Hence $d = 4$ decimal places suffice; $d = 3$ do not (a three-place error bound $\frac{1}{2} \cdot 10^{-3}$ would allow $|10^{\delta} - 1| \approx (\ln 10) \cdot \frac{1}{2} \cdot 10^{-3} \approx 0.00115 > 0.00025$).

12 [02]

In §1.2.2 the logarithm to base b is introduced as the inverse of exponentiation with base b. Concretely, the identities stated around Equations (8) are the inverse laws:

$$\log_b(b^x) = x \quad \text{and} \quad b^{\log_b x} = x \quad (x > 0,\ b > 0,\ b \neq 1).$$

Taking $b = 10$ in $\log_b(b^x) = x$ gives

$$\log_{10}(10^x) = x,$$

which is exactly the form used to "illustrate the so-called common (base-10) logarithms" at Equation (10). Hence (10) is an immediate specialization of (8) with base 10.

13 [M23]

Proof 1 (via Bernoulli's inequality — shortest)

Bernoulli's inequality says that for all real $y > -1$ and integers $n \geq 0$,

$$(1 + y)^n \geq 1 + ny.$$

Apply it with $y = \dfrac{x}{n}$. Then

$$\left(1 + \frac{x}{n}\right)^n \ \geq\ 1 + n \cdot \frac{x}{n} \ =\ 1 + x.$$

Both sides are positive (since $x > -1$), and the function $t \mapsto t^{1/n}$ is strictly increasing on $t > 0$. Taking nth roots preserves the inequality and gives

$$\sqrt[n]{1 + x} \ \leq\ 1 + \frac{x}{n}.$$

Equality holds exactly when $x = 0$ or $n = 1$.

Proof 2 (concavity / tangent line method)

For $0 < r \leq 1$ the function $f(t) = (1 + t)^r$ on $(-1, \infty)$ has

$$f''(t) = r(r - 1)(1 + t)^{r-2} \leq 0,$$

so f is concave. A concave function lies below any tangent line; at $t = 0$,

$$f(0) = 1, \qquad f'(0) = r.$$

Hence $f(t) \le f(0) + f'(0)\,t = 1 + rt$. With $r = 1/n$ and $t = x$ we get

$$(1+x)^{1/n} \le 1 + \frac{x}{n},$$

for every $x > -1$. (This is the same inequality, proved analytically.) For a reference to the standard Bernoulli form—which this concavity argument generalizes—see the sources cited above.

14 [15]

Prove Eq. (12), namely

$$\log_b(c^{\,y}) \;=\; y \log_b c \qquad (c > 0),$$

with the usual assumptions $b > 0,\ b \ne 1$.

Definition of $\log_b x$. For $x > 0$ there is a unique real number $\log_b x$ such that

$$b^{\log_b x} = x,$$

because b^t is a one-to-one, onto mapping from $\mathbb{R}$ to $(0, \infty)$ when $b > 0,\ b \ne 1$.

Law of exponents. For all real u, v and all $a > 0$,

$$(a^u)^v = a^{uv}.$$

(One proves this first for integers, then for rationals via roots, and finally for reals by continuity/limits; this is the standard development in §1.2.2.)

Now let $v = \log_b c$. By definition, $b^v = c$. Then

$$c^{\,y} = (b^v)^y = b^{vy} \qquad \text{(law of exponents)}.$$

Apply $\log_b$ to both sides:

$$\log_b(c^{\,y}) = \log_b(b^{vy}) = vy = y \log_b c,$$

since $\log_b(b^t) = t$ by definition of $\log_b$. This is exactly Eq. (12).

15 [10]

Proof (from the laws of logarithms)

Knuth has already established the basic rules

$$\log_b(xy) = \log_b x + \log_b y \quad (x > 0,\ y > 0),$$

and

$$\log_b(c^t) = t \log_b c \quad (c > 0),$$

which follow from the exponent laws and the definition of $\log_b$.

Apply these to $x/y = x \cdot y^{-1}$ (note $y^{-1} > 0$ since $y > 0$):

$$\log_b\left(\frac{x}{y}\right) = \log_b(x \cdot y^{-1}) = \log_b x + \log_b(y^{-1}) = \log_b x + (-1)\log_b y = \log_b x - \log_b y.$$

Thus the identity is **true** for all $x > 0$, $y > 0$, with $b > 0$, $b \neq 1$.

Equivalent one-liner (via the definition $b^{\log_b u} = u$)

By definition, $x = b^{\log_b x}$ and $y = b^{\log_b y}$. Hence

$$\frac{x}{y} = b^{\log_b x}\, b^{-\log_b y} = b^{\log_b x - \log_b y},$$

so taking $\log_b$ of both sides gives $\log_b(x/y) = \log_b x - \log_b y$.

17 [05]

1. $\lg 32$

By definition $\lg x := \log_2 x$. Since $32 = 2^5$, we have $\lg 32 = \log_2(2^5) = 5$.

2. $\log_\pi \pi$

In any (real) base $b > 0, b \neq 1$, $\log_b b = 1$ because $b^1 = b$. Thus $\log_\pi \pi = 1$. (This uses the standard real-log definition.)

3. $\ln e$

By definition $\ln x := \log_e x$. Hence $\ln e = \log_e e = 1$.

4. $\log_b 1$

For any $b > 0, b \neq 1$, we have $b^0 = 1$. Therefore $\log_b 1 = 0$. (Again using the real-log definition with positive argument.)

5. $\log_b(-1)$

In the (real) setting used in this section, logarithms are defined only for positive arguments $x > 0$ with $b > 0, b \neq 1$. Since -1 is not positive, $\log_b(-1)$ has **no real value** (is undefined over the reals).

18 [10]

Solution (disprove)

Work for $x > 0$. By definition, $\log_8 x = y$ iff $8^y = x$. Since $8 = 2^3$, we have

$$8^y = (2^3)^y = 2^{3y} = x.$$

Take binary logarithms (lg) of both sides and use the law $\log_b(c^y) = y \log_b c$:

$$\lg x = \lg(2^{3y}) = 3y \quad \Rightarrow \quad y = \frac{1}{3} \lg x.$$

Therefore

$$\boxed{\log_8 x = \tfrac{1}{3} \lg x}$$

—not $\frac{1}{2} \lg x$. (The used law $\log_b(c^y) = y \log_b c$ is given in Eq. (12).)

A concrete counterexample makes the falsehood clear: take $x = 8$. Then $\log_8 8 = 1$, but $\frac{1}{2} \lg 8 = \frac{1}{2} \cdot 3 = 1.5 \neq 1$.

When would $\log_8 x = \frac{1}{2} \lg x$ hold?

Only when $\frac{1}{3} \lg x = \frac{1}{2} \lg x$, i.e., when $\lg x = 0$, so $x = 1$.

Remark (generalization)

More generally, for any $k > 0$,

$$\log_{2^k} x = \frac{1}{k} \lg x,$$

and by change of base,

$$\log_a x = \frac{\lg x}{\lg a}.$$

The incorrect $\frac{1}{2}$ would be correct for base $4 = 2^2$: $\log_4 x = \frac{1}{2} \lg x$.

19 [20]

Answer

Yes — every 14-digit integer fits.

A 14-digit positive integer n satisfies

$$10^{13} \leq n \leq 10^{14} - 1 < 10^{14}.$$

Use the classic tight comparison $10^3 < 2^{10}$. Then

$$10^{14} = 10^{12} \cdot 10^2 < (2^{10})^4 \cdot 100 = 2^{40} \cdot 100 < 2^{40} \cdot 2^7 = 2^{47},$$

since $100 < 128 = 2^7$. Hence $n < 2^{47}$.

A machine with 47 magnitude bits (plus a sign bit) can represent all integers in the range $-(2^{47}-1)$ to $+(2^{47}-1)$ in sign-magnitude format (and $[-2^{47}, 2^{47}-1]$ if it were two's-complement with 48 total bits). Either way, every $n < 2^{47}$ (and its negative) fits.

Alternative view (bit-length via logs)

The number of bits needed for the largest 14-digit number is

$$\lceil \log_2(10^{14} - 1) \rceil < \lceil \log_2 10^{14} \rceil = \lceil 14 \log_2 10 \rceil = \lceil 14 \times 3.321928 \ldots \rceil = \lceil 46.5069 \ldots \rceil = 47.$$

So 47 value bits suffice; the extra sign bit handles negativity.

20 [10]

Each is the reciprocal of the other:

$$\boxed{\log_{10} 2 = \frac{1}{\log_2 10}} \qquad \text{and} \qquad \boxed{\log_2 10 = \frac{1}{\log_{10} 2}}.$$

The section derives the change-of-base rule

$$\log_c x = \frac{\log_b x}{\log_b c},$$

which follows from the basic logarithm identities $\log_b(xy) = \log_b x + \log_b y$ and $\log_b(c^y) = y \log_b c$.

Apply that rule twice:

1. Take $b = 10,\ c = 2,\ x = 10$:

$$\log_2 10 = \frac{\log_{10} 10}{\log_{10} 2} = \frac{1}{\log_{10} 2}.$$

2. Equivalently, take $b = 2,\ c = 10,\ x = 2$:

$$\log_{10} 2 = \frac{\log_2 2}{\log_2 10} = \frac{1}{\log_2 10}.$$

Thus the "simple relation" is also

$$\log_{10} 2 \cdot \log_2 10 = 1.$$

21 [15]

To keep everything real-valued we need $b > 1$ and $x > 1$: then $\log_b x > 0$ (so the outer log is defined) and $\ln \ln b,\ \ln \ln x$ both exist.

Use the change-of-base identity (Eq. (14) in §1.2.2):

$$\log_c u \;=\; \frac{\log_b u}{\log_b c} \;=\; \frac{\ln u}{\ln c}.$$

Apply it twice:

$$\begin{aligned}
\log_b(\log_b x) &= \log_b\left(\frac{\ln x}{\ln b}\right) && \text{(change base)}\\
&= \log_b(\ln x) \;-\; \log_b(\ln b) && \text{(log of a quotient)}\\
&= \frac{\ln(\ln x)}{\ln b} \;-\; \frac{\ln(\ln b)}{\ln b} && \text{(change base again)}\\
&= \boxed{\frac{\ln \ln x - \ln \ln b}{\ln b}}.
\end{aligned}$$

22 [20]

By change of base, $\lg x = \dfrac{\ln x}{\ln 2}$ and $\log_{10} x = \dfrac{\ln x}{\ln 10}$.

Let

$$A(x) \;=\; \ln x + \log_{10} x \;=\; \ln x\left(1 + \frac{1}{\ln 10}\right).$$

Then, for any $x \neq 1$ (the case $x = 1$ is trivial since both sides are 0),

$$\frac{A(x)}{\lg x} = \frac{\ln x\left(1 + \frac{1}{\ln 10}\right)}{\ln x / \ln 2} = \ln 2\left(1 + \frac{1}{\ln 10}\right) =: R.$$

Notice R is a **constant**, independent of x. Hence the relative error of the approximation $A(x)$ to $\lg x$ is

$$\frac{|\lg x - A(x)|}{|\lg x|} = |1 - R|.$$

Use the simple bounds $0.693 < \ln 2 < 0.694$ and $2.3025 < \ln 10 < 2.3026$. Then

$$R \in \left[0.693\left(1+\frac{1}{2.3026}\right),\ 0.694\left(1+\frac{1}{2.3025}\right)\right] \subset (0.9939,\ 0.9955),$$

so $|1-R| < 0.0061 < 0.01$. Thus

$$\boxed{\lg x \approx \ln x + \log_{10} x \ \text{ with } \ < 1\% \text{ relative error.}}$$

23 [M25]

Geometric proof (area-preserving similarity)

Let $\mathcal{A}(a,b)$ denote the area under $y = 1/t$ from $t = a$ to $t = b$ (with $0 < a < b$). By the definition above, $\ln x = \mathcal{A}(1,x)$.

Fix some $x > 0$. Consider the **similarity transformation**

$$\Phi_x : \ (u,v) \ \longmapsto \ (t,w) = (xu,\ v/x).$$

This scales the horizontal axis by a factor x and the vertical axis by a factor $1/x$. Its Jacobian determinant is

$$\det\begin{pmatrix} \partial t/\partial u & \partial t/\partial v \\ \partial w/\partial u & \partial w/\partial v \end{pmatrix} = \det\begin{pmatrix} x & 0 \\ 0 & 1/x \end{pmatrix} = 1,$$

so **areas are preserved** by Φ_x.

Now check what Φ_x does to the curve $v = \frac{1}{u}$:

$$v = \frac{1}{u} \quad \Longrightarrow \quad w = \frac{v}{x} = \frac{1}{xu} = \frac{1}{t},$$

since $t = xu$. Thus Φ_x maps the hyperbola $v = 1/u$ **onto itself**.

Therefore, Φ_x maps the region under $v = 1/u$ from $u = 1$ to $u = y$ **bijectively and area-preservingly** onto the region under $w = 1/t$ from $t = x$ to $t = xy$. In our notation,

$$\mathcal{A}(1,y) \ = \ \mathcal{A}(x,xy).$$

Finally decompose the larger interval $[1,xy]$ at x:

$$\mathcal{A}(1,xy) \ = \ \mathcal{A}(1,x) + \mathcal{A}(x,xy) \ = \ \mathcal{A}(1,x) + \mathcal{A}(1,y).$$

Using $\ln z = \mathcal{A}(1, z)$, this is exactly

$$\boxed{\ln(xy) = \ln x + \ln y}.$$

(Optional) Same idea as a one-line integral substitution

The same geometry is the substitution $t = xs$ in the area integral:

$$\ln(xy) = \int_1^{xy} \frac{dt}{t} = \int_1^{x} \frac{dt}{t} + \int_x^{xy} \frac{dt}{t} = \ln x + \int_1^{y} \frac{ds}{s} = \ln x + \ln y,$$

where the middle equality is exactly the area-preserving stretch by x horizontally and $1/x$ vertically (the Jacobian is 1). This matches Fig. 6's geometric interpretation.

24 [15]

What the original method does (base 10, summarized)

Knuth shows how to normalize x so that $1 \le x/10^n < 10$ (this picks out the integer part $n = \lfloor \log_{10} x \rfloor$), then iteratively squares and (when needed) divides by 10 to decide the binary digits $b_1, b_2, \ldots$ of the fractional part of $\log_{10} x$:

$$\log_{10} x \;=\; n + \frac{b_1}{2} + \frac{b_2}{4} + \frac{b_3}{8} + \cdots,$$

with the loop invariant

$$1 \;\le\; \frac{x^{2^k}}{10^{2^k(n + b_1/2 + \cdots + b_k/2^k)}} \;<\; 10.$$

This is the content of Eqs. (17)–(19) and the paragraph introducing them.

Modify it for base 2

Simply replace every "10" by "2" and "decimal shift" by "binary shift."

Normalization. Choose the unique integer n with $1 \le x/2^n < 2$; set $x_0 := x/2^n$. Then $n = \lfloor \lg x \rfloor$.

Iteration. For $k = 1, 2, \ldots,$

1. Compute $t := x_{k-1}^2$.
2. If $t < 2$ set $b_k := 0$ and $x_k := t$; else set $b_k := 1$ and $x_k := t/2$.

(This is exactly the same control flow as the base-10 version, just with the radix "2.")

Output. The algorithm yields

$$\lg x \;=\; n + \frac{b_1}{2} + \frac{b_2}{4} + \frac{b_3}{8} + \cdots,$$

i.e., the fractional bits of $\lg x$ are $b_1 b_2 b_3 \ldots$ in base 2.

Why it works (invariant proof). By construction $1 \le x_k < 2$ for all k. Unrolling the recurrence shows

$$x_k \;=\; \frac{x^{2^k}}{2^{2^k\left(n+\sum_{j=1}^{k} b_j/2^j\right)}}.$$

Thus the test "$t < 2$?" is equivalent to asking whether

$$\frac{x^{2^k}}{2^{2^k\left(n+\sum_{j=1}^{k-1} b_j/2^j+1/2^k\right)}} \;<\; 1 \quad \text{or} \quad \ge 1,$$

i.e., whether the next binary digit b_k of $\lg x$ is 0 or 1. Therefore after k steps we have fixed the first k fractional bits. This is identical to the correctness argument Knuth gives for base 10, specialized to base 2.

Small example: $\lg 3$

Normalize: $1 \le 3/2^1 = 1.5 < 2$ so $n = 1$, $x_0 = 1.5$.

- $k = 1$: $t = 1.5^2 = 2.25 \ge 2 \Rightarrow b_1 = 1$, $x_1 = 2.25/2 = 1.125$.
- $k = 2$: $t = 1.125^2 = 1.265625 < 2 \Rightarrow b_2 = 0$, $x_2 = 1.265625$.
- $k = 3$: $t \approx 1.6018066 < 2 \Rightarrow b_3 = 0$, $x_3 \approx 1.6018066$.
- $k = 4$: $t \approx 2.56579 \ge 2 \Rightarrow b_4 = 1$, $x_4 \approx 1.282895$. (…and so on.)

So $\lg 3 \approx 1 + 1/2 + 0/4 + 0/8 + 1/16 + \cdots = 1.58496\ldots$ — which matches the true value to the shown precision.

25 [22]

Why it terminates (finite-precision binary)

Work in fixed binary with p fraction bits (i.e., resolution 2^{-p}). "Shift right 1" divides by 2 and truncates one low-order bit; after j successive shifts, z is $\approx x/2^j$ (again truncated).

- During L3 we keep halving z until $x - z \geq 1$. Since after p shifts we have $z \leq x/2^p < 2/2^p = 2^{1-p} < 1$, we certainly get $x - z \geq 1$ by or before $j = p$. Thus L3 always finishes with some $k \leq p$.
- In L4 we subtract a **positive** amount $z \geq 2^{-p}$. Therefore x strictly decreases by at least one ulp. Because initially $1 \leq x < 2$, there are at most $\lceil (x_0 - 1)\, 2^p \rceil$ such decreases before x becomes exactly 1 in this arithmetic, triggering L2. Hence the algorithm halts after $O(p)$ reductions, i.e., a number of simple shifts/adds/subtracts proportional to the desired places of accuracy. (Knuth notes that the deliberate truncations from shifting ensure eventual reduction to 1.)

Why it computes $\log_b x$

Ignore truncation for a moment (idealized arithmetic) to see the invariant:

- In L3 we choose the **smallest** k with $x - \frac{x}{2^k} \geq 1$, i.e.

$$x \;\geq\; \frac{1}{1 - 2^{-k}} \;=\; \frac{2^k}{2^k - 1} \;\stackrel{\text{def}}{=}\; a_k.$$

- In L4 we then replace x by $x' := x - \frac{x}{2^k} = x\,(1 - 2^{-k}) = \dfrac{x}{a_k}$ and add to y the term $\log_b a_k$.

Thus each reduction step maintains the invariant

$$x \cdot b^{-y} \;\text{ stays constant.}$$

After a sequence of $k_1, k_2, \ldots, k_m$ we have

$$y = \sum_{i=1}^{m} \log_b a_{k_i} \;=\; \log_b\Big(\prod_{i=1}^{m} a_{k_i}\Big), \qquad x = \frac{x_0}{\prod_{i=1}^{m} a_{k_i}}.$$

If we could carry on "forever," x would tend to 1 and y would tend to $\log_b x_0$.

With finite precision the only difference is that "shift right" underestimates $\frac{x}{2^k}$ by $<$ one ulp. Hence each step reduces x by **nearly** $x/2^k$, so the invariant becomes

$$x \cdot b^{-y} \;=\; 1 + \varepsilon,$$

where ε is a small multiplicative error caused by truncation (bounded in terms of machine precision and the number of steps). When the loop stops at $x = 1$, we have

$$y = \log_b\Big(\frac{x_0}{1+\varepsilon}\Big) = \log_b x_0 - \log_b(1+\varepsilon),$$

so the absolute error is $|\log_b x_0 - y| = \log_b(1+\varepsilon) \approx \varepsilon/\ln b$. In other words, y is a bona fide approximation to $\log_b x_0$, with an accuracy governed by the working precision. (Exercise 26 in the book asks for a rigorous bound tied to the rounding model.)

A tiny worked trace (to see the mechanism)

Take $b = e$ and $x_0 = 1.5$ (exact arithmetic, for intuition).

- Start: $y = 0,\ z = 0.75,\ k = 1$. Since $1.5 - 0.75 < 1$, halve $z \Rightarrow z = 0.375, k = 2$. Now $1.5 - 0.375 = 1.125 \geq 1$.
- Reduce: $x \leftarrow 1.125$, $y \leftarrow y + \ln(4/3)$.
- Recompute $z = x/2^k = 1.125/4 = 0.28125$. Since $1.125 - 0.28125 < 1$, halve $z \to 0.140625, k = 3$; still < 1, halve again $z \to 0.0703125, k = 4$; now $1.125 - 0.0703125 = 1.0546875 \geq 1$.
- Reduce: $x \leftarrow 1.0546875$, $y \leftarrow y + \ln(16/15)$.

26 [M27]

What the algorithm does (very briefly)

Given a binary machine number $x \in [1, 2)$ and a base $b > 0,\ b \neq 1$, Ex. 25 proposes the loop

- choose $k \geq 1$ so that with $z = (x$ shifted right $k)$ we still have $x - z \geq 1$;
- replace $x \leftarrow x - z$ and add $\log_b(2^k/(2^k - 1))$ to the running sum y;

and repeat until $x = 1$. (See the statement on p. 26 of the book.)

Error model (binary fixed-point)

Assume the computer works in binary fixed-point with t fraction bits. Let

$$u \stackrel{\text{def}}{=} 2^{-t}$$

be one unit in the last place (1 ulp). A right shift by k computes

$$z \;=\; \Big\lfloor \frac{x}{2^k} \Big\rfloor$$

so the exact division and the shifted value differ by less than one ulp:

$$0 \;\le\; \frac{x}{2^k} - z \;<\; u \quad \text{for every step.}$$

Two key facts

1. **Monotone overestimate at each step.** When the loop subtracts z and adds $\log_b(2^k/(2^k-1))$, it is pretending we subtracted the exact fraction $x/2^k$. Since $z \le x/2^k$, we subtract **no more than** that exact amount, hence the running sum y is an **overestimate** of $\log_b$ of the original input. Thus

$$y \;\ge\; \log_b x_{\text{input}}.$$

2. **A one-shot bound from the last step.** On the *final* iteration we have $x - z = 1$, so $z = x - 1$. Because $z = \lfloor x/2^k \rfloor$,

$$x - 1 \;\le\; \frac{x}{2^k} \;<\; x - 1 + u \quad \Longrightarrow \quad 1 - u \;<\; x - \frac{x}{2^k} \;\le\; 1.$$

If we imagine doing the *same* sequence of k's but with **exact** divisions, the identity behind the algorithm says

$$\log_b x_{\text{input}} \;=\; y_{(\text{no roundoff in sums})} \;+\; \log_b x^*_{\text{final}},$$

where x^*_{final} is what the last exact subtraction would leave. The inequality above shows

$$1 - u \;\le\; x^*_{\text{final}} \;\le\; 1.$$

Therefore the (systematic) overestimate is bounded by

$$0 \;\le\; y_{(\text{no roundoff in sums})} - \log_b x_{\text{input}} \;=\; -\log_b x^*_{\text{final}} \;\le\; -\log_b(1-u).$$

Final bound (and a handy simplification)

Let $u = 2^{-t}$ be the machine ulp. Then a rigorous upper bound on the error of the algorithm in Ex. 25 is

$$\boxed{0 \;\le\; y - \log_b x_{\text{input}} \;\le\; -\log_b(1 - 2^{-t}).}$$

Using $-\ln(1-\delta) \le \dfrac{\delta}{1-\delta}$ and $\log_b z = \dfrac{\ln z}{\ln b}$,

$$-\log_b(1-2^{-t}) \;\le\; \frac{2^{-t}}{(1-2^{-t})\,\ln b} \;<\; \frac{2^{-(t-1)}}{\ln b} \qquad (t \ge 1).$$

Thus the error is $O(2^{-t})$ with an explicit constant depending only on the logarithm base b.

If the *sums* are rounded too

If each addition $y \leftarrow y + \log_b(2^k/(2^k-1))$ is performed in a p-bit accumulator (round-to-nearest), each add contributes at most 2^{-p-1} in $\log_b$-units. With N additions,

$$0 \;\le\; y_{\text{computed}} - \log_b x_{\text{input}} \;\le\; -\log_b(1-2^{-t}) \;+\; N \cdot 2^{-p-1}.$$

(Here N is the number of times step L4 executes; it is proportional to the digits of accuracy requested, as noted in the exercise statement.)

27 [M30]

Setup (what the algorithm does)

Let x be a positive real number. The method writes

$$\log_{10} x \;=\; n \;+\; \frac{b_1}{2} + \frac{b_2}{4} + \frac{b_3}{8} + \cdots,$$

where we first normalize $x_0 = x/10^n \in [1, 10)$, then iterate for $k \ge 1$:

$$b_k = \begin{cases} 0, & \text{if } x_{k-1}^2 < 10, \\ 1, & \text{if } x_{k-1}^2 \ge 10, \end{cases} \qquad x_k \;=\; \begin{cases} x_{k-1}^2, & \text{if } b_k = 0, \\ x_{k-1}^2/10, & \text{if } b_k = 1, \end{cases}$$

so that always $x_k \in [1, 10)$. In practice we cannot compute exactly; Knuth explicitly says we **round or truncate** each x_k to a fixed number t of significant decimal figures.

Exercise 27 asks you to **derive a worst-case upper bound** for the error in the resulting value of $\log_{10} x$ caused by doing that finite-precision rounding/truncation each step. (Knuth hints at the outcome and later notes that with $t = 4$ the error is guaranteed to be < 0.00044.)

Work in log space

Write $\ell_k = \log_{10} x_k$ for the exact process and $\tilde{\ell}_k = \log_{10} \tilde{x}_k$ for the computed (rounded/truncated) values. The exact recurrence is

$$\ell_k \;=\; 2\,\ell_{k-1} - b_k, \qquad \ell_k \in [0, 1)\,.$$

Finite precision introduces an **additive** error in log space:

$$\tilde{\ell}_k \;=\; 2\,\tilde{\ell}_{k-1} - \tilde{b}_k \;+\; \varepsilon_k,$$

where ε_k is the log10-error created when we round/truncate x_k into $\tilde{x}_k$.

Because after normalization $x_k \in [1, 10)$, rounding/truncation to t significant figures perturbs x_k by at most $\pm 10^{1-t}$ in absolute value. Therefore the **per-step log10 error** is bounded by

$$|\varepsilon_k| \;\le\; \log_{10}(1 + 10^{1-t}) \;<\; \frac{10^{1-t}}{\ln 10}\,,$$

using $\ln(1+u) \le u$. (We use the larger truncation bound; rounding alone would replace 10^{1-t} by $0.5 \cdot 10^{1-t}$.)

Relate digit errors to the ε_k

Unfold the two recurrences up to step k and subtract. One obtains

$$\tilde{\ell}_k - \ell_k \;=\; -2^{\,k}(\tilde{S}_k - S_k) \;+\; \sum_{j=1}^{k} 2^{\,k-j}\,\varepsilon_j,$$

where $S_k = \sum_{j=1}^{k} b_j/2^j$ and $\tilde{S}_k = \sum_{j=1}^{k} \tilde{b}_j/2^j$ are the exact and computed partial binary sums. Since both ℓ_k and $\tilde{\ell}_k$ lie in $[0, 1)$, $|\tilde{\ell}_k - \ell_k| < 1$, hence

$$\left|\tilde{S}_k - S_k\right| \;\le\; 2^{-k} \;+\; \sum_{j=1}^{k} 2^{-j}\,|\varepsilon_j|.$$

Letting $k \to \infty$ (so $2^{-k} \to 0$ and $S_k \to S$, $\tilde{S}_k \to \tilde{S}$) gives the **final error bound**

$$\boxed{\left|\tilde{S} - S\right| \;\le\; \sum_{j=1}^{\infty} 2^{-j}\,|\varepsilon_j| \;\le\; \log_{10}(1 + 10^{1-t}) \;<\; \frac{10^{1-t}}{\ln 10}}\,.$$

Plug in $t = 4$

With four significant figures,

$$\frac{10^{1-4}}{\ln 10} \;=\; \frac{10^{-3}}{\ln 10} \;\approx\; 0.000434294 \;<\; 0.00044,$$

exactly matching Knuth's stated guarantee. (His worked example happens to be even better because the first few x_k were exact before rounding error accumulated.)

28 [M30]

Here is a clean way to do b^x **for** $0 \le x < 1$ using only shifting, addition, and subtraction, mirroring the "shift-subtract" logarithm method from §1.2.2 Ex. 25.

#1. Idea (invert Ex. 25)

Exercise 25 shows how to reduce a value $x \in [1, 2)$ toward 1 by repeatedly replacing

$$x \leftarrow x - \frac{x}{2^k} = x\Big(1 - \frac{1}{2^k}\Big)$$

and adding the constant $\log_b(\frac{2^k}{2^k-1})$ to the running logarithm; it needs only shifts, adds, subtracts, plus a small table of constants $\log_b(\frac{2^k}{2^k-1})$.

To go the other way (compute the **exponential**), write the exponent x itself as a sum of those same constants:

$$x = \sum_i c_{k_i}, \qquad c_k \stackrel{\text{def}}{=} \log_b\Big(\frac{2^k}{2^k - 1}\Big).$$

Then

$$b^x = \prod_i b^{c_{k_i}} = \prod_i \frac{2^{k_i}}{2^{k_i} - 1}.$$

Each factor $\frac{2^k}{2^k-1}$ is a **shift left by** k (multiply by 2^k) followed by an **integer division by** $(2^k - 1)$—both doable with shifts and add/sub (long division).

#2. Algorithm (Feynman's product)

Precompute a short table $c_k = \log_b(\frac{2^k}{2^k-1})$ for $k = 1, 2, \ldots, K$. (Exactly the same kind of table Ex. 25 requires.)

Input: base $b > 0,\ b \ne 1$; exponent $x \in [0, 1)$; tolerance ε.

Output: $y \approx b^x$.

Invariant: after every step, $y \cdot b^r = b^x$ where r is the remaining (unaccounted) exponent.

- E1. Initialize $y \leftarrow 1,\ r \leftarrow x,\ k \leftarrow 1$.
- E2. If $r \le \varepsilon$, **stop** and return y.
- E3. While $r < c_k$, set $k \leftarrow k + 1$. (Because c_k decreases rapidly with k.)
- E4. Update the product using shifts only:

- $y \leftarrow y \cdot 2^k$ (shift left by k),
- $y \leftarrow y \div (2^k - 1)$ (binary long division: shift/subtract),
- $r \leftarrow r - c_k$;
- Go to E2.

Correctness. Step E4 multiplies y by $2^k/(2^k-1) = b^{c_k}$; reducing r by c_k maintains $y\,b^r = b^x$. When $r \le \varepsilon$, the returned y differs from b^x by a factor $b^r \in [1, b^\varepsilon]$.

#3. How accurate is it?

Two effects matter:

(A) Truncation of the exponent sum. If you stop when $r \le \varepsilon$, then

$$\frac{|b^x - y|}{b^x} \;=\; |b^r - 1| \;\le\; e^{(\ln b)\,\varepsilon} - 1 \;\approx\; (\ln b)\,\varepsilon \quad \text{(for small } \varepsilon\text{)}.$$

Thus, to get about p **bits** of relative accuracy, it suffices to drive

$$\varepsilon \;\lesssim\; \frac{2^{-p}}{\ln b}.$$

Because $c_k = \log_b\!\left(\frac{2^k}{2^k-1}\right) \sim \frac{1}{(\ln b)(2^k-1)}$, the greedy selection makes k grow, and you typically need only $O(p)$ factors.

(B) Finite-precision arithmetic in the divisions. Each division by $(2^k - 1)$ performed with w-bit fixed-point long division incurs at most $O(2^{-w})$ **relative** error. After T factors the cumulative relative error is $\lesssim T2^{-w}$ (first-order). Pick w a few bits larger than the target precision and the truncation term (A) will dominate.

#4. Tiny example (base-10, four-digit feel)

Take $b = 10$ and $x \approx \log_{10} 2 \approx 0.3010$. Using the table $c_2 = \log_{10}\frac{4}{3} \approx 0.12494$, $c_4 = \log_{10}\frac{16}{15} \approx 0.02803$, $c_5 \approx 0.01355$, $c_6 \approx 0.00679$, $c_8 \approx 0.00174$, $c_9 \approx 0.000903, \ldots$ and applying E2–E4 greedily gives

$$10^{0.3010} \approx \left(\frac{4}{3}\right)^2 \cdot \frac{16}{15} \cdot \frac{32}{31} \cdot \frac{64}{63} \cdot \frac{256}{255} \cdot \frac{512}{511} \cdots \approx 1.9992 \quad \text{(within } 4 \times 10^{-4} \text{ of 2)}.$$

All updates are "shift by k and divide by $2^k - 1$"; the constants come straight from the exercise list in §1.2.2.

29 [HM20]

Use the base-change identity $\log_b x = \dfrac{\ln x}{\ln b}$. Since $x > 1$ is fixed, $\ln x > 0$, so minimizing

$$b \log_b x = b \cdot \frac{\ln x}{\ln b}$$

is equivalent to minimizing $f(b) = \dfrac{b}{\ln b}$ over the stated domain. Similarly, minimizing $(b+1)\log_b x$ is equivalent to minimizing $g(b) = \dfrac{b+1}{\ln b}$.

(a) Real $b > 1$ minimizing $b \log_b x$

Let $f(b) = \dfrac{b}{\ln b}$ for $b > 1$. Then

$$f'(b) = \frac{\ln b - 1}{(\ln b)^2}.$$

Hence $f'(b) = 0 \iff \ln b = 1 \iff b = e$. Moreover,

- $f'(b) < 0$ for $1 < b < e$ and
- $f'(b) > 0$ for $b > e$,

so $b = e$ is the unique minimizer. The minimum value is

$$b \log_b x \big|_{b=e} = e \cdot \ln x.$$

Answer (a): $b = e$, with minimum $e \ln x$.

(b) Integer $b > 1$ minimizing $b \log_b x$

From (a), $f(b)$ decreases on $(1, e]$ and increases on $[e, \infty)$. Thus among integers $b > 1$, only $b = 2$ and $b = 3$ can be candidates, and $f(3) < f(2)$ because we are past the minimum at e.

Equivalently, compare $\dfrac{3}{\ln 3}$ and $\dfrac{2}{\ln 2}$: $\dfrac{3}{\ln 3} < \dfrac{2}{\ln 2} \iff 3 \ln 2 < 2 \ln 3 \iff 2^3 < 3^2$, i.e., $8 < 9$, true.

Answer (b): $b = 3$.

(c) Integer $b > 1$ minimizing $(b+1)\log_b x$

Consider $g(b) = \dfrac{b+1}{\ln b}$ for real $b > 1$:

$$g'(b) = \frac{\ln b - 1 - \frac{1}{b}}{(\ln b)^2}.$$

The critical point satisfies $\ln b = 1 + \frac{1}{b}$, which has a unique solution $b^* \in (3, 4)$ (since $\ln 3 < 1 + \frac{1}{3}$ and $\ln 4 > 1 + \frac{1}{4}$). Therefore $g(b)$ strictly decreases up to b^* and increases thereafter, so the minimizing integer must be 3 or 4.

Compare

$$g(3) = \frac{4}{\ln 3}, \qquad g(4) = \frac{5}{\ln 4}.$$

We have $g(4) < g(3)$ iff $5 \ln 3 < 4 \ln 4 = 8 \ln 2$, i.e.

$$\ln(3^5) < \ln(2^8) \iff 243 < 256,$$

which is true. Also $g(5) > g(4)$ since $6 \ln 4 > 5 \ln 5$ (equivalently $4^6 > 5^5$, i.e. $4096 > 3125$). Thus $b = 4$ is the unique minimizing integer.

Answer (c): $b = 4$.

30 [12]

1) Domain sanity check

- $x > 1 \Rightarrow \ln x > 0$, so the base $\ln x$ is positive.
- $x \neq e \Rightarrow \ln x \neq 1$ and $\ln \ln x \neq 0$, so the exponent is defined and the base isn't 1.

2) Recognize a change–of–base pattern

Recall the identity (valid for any $a > 0$, $a \neq 1$): $a^{\log_a t} = t$.

Note that

$$\frac{\ln x}{\ln \ln x} = \log_{\ln x} x.$$

Hence

$$(\ln x)^{\ln x / \ln \ln x} = (\ln x)^{\log_{\ln x} x} = x.$$

3) (Equivalent derivation via exponentials)

$$(\ln x)^{\ln x/\ln\ln x} = \exp\Big(\frac{\ln x}{\ln\ln x}\cdot\ln(\ln x)\Big) = \exp(\ln x) = x.$$

4) Note on the excluded point

At $x = e$ the expression is undefined (division by $\ln\ln e = \ln 1 = 0$). However, the limit $\lim_{x\to e}(\ln x)^{\ln x/\ln\ln x} = \exp(\lim_{x\to e}\ln x) = e$, so the continuous extension at $x = e$ would take the value e.

$$\boxed{(\ln x)^{\ln x/\ln\ln x} = x \quad \text{for } x > 1,\ x \neq e.}$$

1.2.3 Sums and products

1 [10]

By definition in §1.2.3, the three-dot sum $a_1 + a_2 + \cdots + a_n$ is shorthand for the indexed sum $\sum_{1 \le j \le n} a_j$. More generally, $\sum_{R(j)} a_j$ means "sum all a_j with integer j that satisfy $R(j)$," and if no integers satisfy $R(j)$, the sum is defined to be 0**.

So

$$a_2 + \cdots + a_0 \;\equiv\; \sum_{2 \le j \le 0} a_j \;=\; 0,$$

because there is no integer j with $2 \le j \le 0$.

This is why Knuth calls the behavior "slightly peculiar but sensible" in edge cases: the three-dot notation is a single construct tied to its explicit bounds, not something you peel apart as $a_2 + (\cdots + a_0)$. Read as a bounded sum, its index set is empty, hence the value 0.

2 [01]

By Knuth's convention, a restricted sum

$$\sum_{R(j)} a_j$$

means "sum a_j over all **integers** j that satisfy the condition $R(j)$." If no such integers exist, the sum is defined to be 0.

Here $R(j)$ is $1 \le j \le n$. When $n = 3.14$, the integers j with $1 \le j \le 3.14$ are $j = 1, 2, 3$. Therefore

$$\sum_{1 \le j \le 3.14} a_j \;=\; a_1 + a_2 + a_3 \;=\; \sum_{j=1}^{3} a_j.$$

(Equivalently, for any real n, $\sum_{1 \le j \le n} a_j = \sum_{j=1}^{\lfloor n \rfloor} a_j$; this follows from the "sum over integers satisfying $R(j)$" interpretation and the bracket-notation identity $\sum_{R(j)} a_j = \sum_j a_j \, [R(j)]$.)

3 [13]

Write out, **without** Σ-notation, the sums

$$\sum_{0\le n\le 5} \frac{1}{2n+1} \qquad \text{and} \qquad \sum_{0\le n^2\le 5} \frac{1}{2n^2+1},$$

then explain why the two results are different "in spite of rule (b)" (the change-of-variable rule).

Step 1: Expand both sums explicitly

1. $\displaystyle\sum_{0\le n\le 5} \frac{1}{2n+1}$ uses the integer indices $n = 0, 1, 2, 3, 4, 5$:

$$\frac{1}{1} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \frac{1}{9} + \frac{1}{11}.$$

2. $\displaystyle\sum_{0\le n^2\le 5} \frac{1}{2n^2+1}$ uses all integers with square at most 5, i.e. $n \in \{-2, -1, 0, 1, 2\}$:

$$\frac{1}{2\cdot 4+1} + \frac{1}{2\cdot 1+1} + \frac{1}{2\cdot 0+1} + \frac{1}{2\cdot 1+1} + \frac{1}{2\cdot 4+1} = \frac{1}{9} + \frac{1}{3} + 1 + \frac{1}{3} + \frac{1}{9} = 1 + \frac{2}{3} + \frac{2}{9}.$$

(If you like a single value: the first sum 1.8782; the second equals $17/9 \approx 1.8889$.)

Step 2: Why the results differ despite rule (b)

Rule (b) says we may change the index variable via a **permutation** $p(j)$ of the relevant integers:

$$\sum_{R(i)} a_i \;=\; \sum_{R(p(j))} a_{p(j)}.$$

This requires p to be one-to-one and onto the index set (a bijection).

- In the first sum, the index set is $\{0, 1, 2, 3, 4, 5\}$ (six elements).
- In the second, the condition $0 \le n^2 \le 5$ yields the index set $\{-2, -1, 0, 1, 2\}$ (five elements).

Trying to "change variables" by $m = n^2$ is not a valid use of rule (b): the map $n \mapsto n^2$ is not a permutation of the integer indices—it is many-to-one (both $+k$ and $-k$ map to the same square), and it also changes the index set's size and elements. Therefore the two sums legitimately come out different; rule (b) simply doesn't apply here.

4 [10]

Left-hand side (iterate $i = 1, 2, 3$, then $j = 1, \dots, i$)

- $i = 1: \; j = 1 \Rightarrow a_{11}$.
- $i = 2: \; j = 1, 2 \Rightarrow a_{21} + a_{22}$.
- $i = 3: \; j = 1, 2, 3 \Rightarrow a_{31} + a_{32} + a_{33}$.

So the LHS is

$$(a_{11}) \;+\; (a_{21} + a_{22}) \;+\; (a_{31} + a_{32} + a_{33}).$$

Right-hand side (iterate $j = 1, 2, 3$, then $i = j, \dots, 3$)

- $j = 1: \; i = 1, 2, 3 \Rightarrow a_{11} + a_{21} + a_{31}$.
- $j = 2: \; i = 2, 3 \Rightarrow a_{22} + a_{32}$.
- $j = 3: \; i = 3 \Rightarrow a_{33}$.

So the RHS is

$$(a_{11} + a_{21} + a_{31}) \;+\; (a_{22} + a_{32}) \;+\; (a_{33}).$$

Why they're equal

Both expressions list exactly the six terms with $1 \le j \le i \le 3$: $\{a_{11}, a_{21}, a_{22}, a_{31}, a_{32}, a_{33}\}$. The LHS groups them by row i; the RHS groups them by column j. Since addition is commutative/associative, the total is the same—exactly what Eq. (10) formalizes.

5 [HM20]

Let the two series be convergent:

$$A := \sum_{R(i)} a_i \quad \text{and} \quad B := \sum_{S(j)} b_j.$$

Knuth's definition of an infinite sum is "limit of partial sums" (with two–sided windows when needed). We will use that definition directly.

Step 1: Constant-factor rule for convergent series

Fix a constant c. For the inner partial sums $B_M := \sum_{S_M(j)} b_j$ (over any increasing finite domains S_M that exhaust S), we have

$$\sum_{S_M(j)} (c\, b_j) = c \sum_{S_M(j)} b_j = c\, B_M$$

by the finite distributive law. Taking limits as $M \to \infty$ and using convergence of $\sum_{S(j)} b_j$ gives

$$\sum_{S(j)} (c\, b_j) = c \sum_{S(j)} b_j = c\, B.$$

Thus constants can be pulled out of a convergent series.

Step 2: Evaluate the inner sum

For each fixed i, a_i is constant with respect to j; hence

$$\sum_{S(j)} a_i b_j \;=\; a_i \sum_{S(j)} b_j \;=\; a_i B,$$

by Step 1.

Step 3: Sum over i

Now take partial sums over i (finite domains R_N that exhaust R):

$$\sum_{i \in R_N} \Big(\sum_{S(j)} a_i b_j \Big) = \sum_{i \in R_N} (a_i B) = B \sum_{i \in R_N} a_i =: B\, A_N,$$

again using the finite distributive law (rule (a) in the finite case). Letting $N \to \infty$ and using the convergence of $\sum_{R(i)} a_i$ yields

$$\sum_{R(i)} \sum_{S(j)} a_i b_j = \lim_{N \to \infty} B\, A_N = B \cdot A = \Big(\sum_{R(i)} a_i \Big) \Big(\sum_{S(j)} b_j \Big).$$

This completes the proof that the distributive rule (a) extends from finite sums to convergent infinite series, precisely as stated in Eq. (4) of the text for the finite case.

6 [HM20]

Key identity ("cut rule")

The fundamental law of these half-open sums is the **cut rule** (this is the Eq. (3) you're asked to use):

$$S(a,b) \;=\; S(a,c) \;+\; S(c,b) \qquad \text{for any integers } a,c,b.$$

It just partitions the index set $[a,b)$ into the disjoint pieces $[a,c)$ and $[c,b)$.

Changing the lower limit

Apply the cut rule and rearrange.

Case 1: $a_0 \le a_c \le b$**.**

$$S(a_0,b) = S(a_c,b) \;+\; S(a_0,a_c).$$

So moving the lower limit **up** from a_0 to a_c removes the block of terms with indices $a_0 \le k < a_c$; to keep equality, you must **add back** exactly that block.

Case 2: $a_c \le a_0 \le b$**.**

$$S(a_0,b) = S(a_c,b) \;-\; S(a_c,a_0).$$

Now moving the lower limit **down** from a_0 to a_c includes the extra block $a_c \le k < a_0$; to compensate, you **subtract** that block.

These two cases are the same rule written with the appropriate sign.

Interchanging limits (the "reverse" rule)

From the cut rule with $b = a$ you get the useful symmetry

$$S(a,c) + S(c,a) = S(a,a) = 0 \quad\Longrightarrow\quad S(a,c) = -\,S(c,a).$$

Thus when you **swap the two limits** in a half-open sum, the value changes sign, and "the terms between a_0 and a_c" migrate from one side of the equality to the other—the precise content of the official hint.

Worked micro-examples

1. $a_0 = 2,\ a_c = 5,\ b = 10$:

$$\sum_{2\le k<10} x_k = \sum_{5\le k<10} x_k + \sum_{2\le k<5} x_k.$$

2. $a_0 = 7,\ a_c = 3,\ b = 10$:

$$\sum_{7\le k<10} x_k = \sum_{3\le k<10} x_k - \sum_{3\le k<7} x_k.$$

3. Interchange a and c:

$$\sum_{a\le k<c} x_k = -\sum_{c\le k<a} x_k.$$

7 [HM23]

1) Use Iverson's bracket definition of restricted sums

Knuth defines a "restricted" sum by turning the restriction into a 0–1 factor:

$$\sum_{R(j)} a_j \;=\; \sum_{j\in\mathbb{Z}} a_j\,[R(j)],$$

where $[R(j)]$ is 1 if the statement $R(j)$ is true and 0 otherwise. This lets us treat "sum over all j satisfying R" as a single sum over all integers with a filter.

2) Make the change of variables $k = c - j$

The map $j \mapsto c - j$ is a bijection (indeed, an involution) on $\mathbb{Z}$. Rename the dummy index in the all-integers sum using this bijection:

$$\sum_{j\in\mathbb{Z}} a_j\,[R(j)] \;=\; \sum_{k\in\mathbb{Z}} a_{\,c-k}\,[R(c-k)].$$

But the right-hand side is exactly the restricted-sum notation

$$\sum_{R(c-k)} a_{\,c-k}.$$

Since k is just a dummy symbol, we can write it as j again and obtain

$$\sum_{R(j)} a_j \;=\; \sum_{R(c-j)} a_{\,c-j}.$$

That is all that needs to be shown.

Why this is valid "even if both series are infinite"

The proof above **does not** rearrange the terms of a conditionally convergent series; it only *renames the index* inside a single "sum over all integers with a 0–1 filter." That's exactly what Knuth's bracket notation is designed to permit. (In the text, this equivalence is also derived from the general "change of index by a permutation" rule; here the permutation is $p(j) = c - j$. Knuth discusses change of index and shows, via brackets, how such a permutation yields the same restricted sum.)

3) Quick sanity checks

- **Finite window:** If $R(j)$ is $0 \le j \le n$ then the result says $\sum_{j=0}^{n} a_j = \sum_{0 \le c-j \le n} a_{c-j} = \sum_{c-n \le j \le c} a_j$, i.e., just a shift of the index range—obviously true.
- **One-sided infinite:** If $R(j)$ is $j \ge 0$ then

$$\sum_{j \ge 0} a_j \;=\; \sum_{c-j \ge 0} a_{c-j} \;=\; \sum_{j \le c} a_j,$$

  which again follows directly from the bijection $j \leftrightarrow c - j$ inside the bracketed all-integers sum.

8 [HM25]

We'll use Stirling's expansion from the text:

$$n! \;=\; \sqrt{2\pi n}\left(\tfrac{n}{e}\right)^n\left(1 + \tfrac{1}{12n} + O(n^{-2})\right) \quad\Longrightarrow\quad \ln n! = n\ln n - n + \tfrac{1}{2}\ln(2\pi n) + \tfrac{1}{12n} + O(n^{-2}).$$

(See the displayed formulas just before and at (19).)

Part A — $\ln(an^2 + bn)!$ with absolute error $O(n^{-2})$

Let $m = an^2 + bn$ with fixed $a > 0$ and fixed b. Since $m \sim an^2$, the Stirling remainder terms beyond $1/(12m)$ are $O(m^{-3}) = O(n^{-6})$, which is (much) smaller than the requested $O(n^{-2})$. Thus it suffices to expand

$$\ln m! \;=\; m\ln m - m + \tfrac{1}{2}\ln(2\pi m) + \tfrac{1}{12m} + O(n^{-6}).$$

Write

$$\ln m = \ln(an^2) + \ln\Big(1 + \frac{b}{a\,n}\Big) = \ln a + 2\ln n + \frac{b}{a\,n} - \frac{b^2}{2a^2n^2} + O(n^{-3}).$$

Now expand the four pieces $m\ln m$, $-m$, $\frac{1}{2}\ln(2\pi m)$, $\frac{1}{12m}$ and collect terms through n^{-2}. A straightforward computation gives

$$\boxed{\begin{aligned}\ln(an^2+bn)! = {} & an^2(2\ln n + \ln a - 1) + bn(2\ln n + \ln a) \\ & + \tfrac{1}{2}(2\ln n + \ln a + \ln(2\pi)) + \frac{b^2}{2a} \\ & + \frac{1}{n}\Big(\frac{b}{2a} - \frac{b^3}{2a^2}\Big) + \frac{1}{n^2}\Big(\frac{1}{12a} - \frac{b^2}{4a^2}\Big) \;+\; O(n^{-3}).\end{aligned}}$$

Because the omitted terms are $O(n^{-3})$, this approximation has **absolute error** $O(n^{-2})$ as required.

(Sanity check: setting $b = 0, a = 1$ reduces to $m = n^2$ and we recover $\ln(n^2)! = 2n^2\ln n - n^2 + \ln n + \frac{1}{2}\ln(2\pi) + \frac{1}{12n^2} + O(n^{-4})$, which is Stirling with $m = n^2$.)

Part B — Asymptotics of $\dfrac{\binom{cn^2}{n}}{c^n\binom{n^2}{n}}$

Write $\binom{N}{n} = \dfrac{N!}{n!(N-n)!}$ and take logs:

$$\ln\binom{N}{n} = \ln N! - \ln n! - \ln(N-n)!.$$

Put $N = \alpha n^2$ with $\alpha > 0$ (we will use $\alpha = c$ and $\alpha = 1$). Using the Stirling–log expansion to accuracy $O(n^{-2})$ (as above) and expanding in powers of $1/n$ with $n \ll N$, one finds

$$\begin{aligned}\ln\binom{\alpha n^2}{n} = {} & n\ln n + n\ln\alpha + n - \tfrac{1}{2}\ln n - \frac{1}{2\alpha} + \frac{1}{n}\Big(\frac{1}{2\alpha} - \frac{1}{6\alpha^2} - \frac{1}{12}\Big) \\ & + \frac{1}{n^2}\Big(\frac{1}{12\alpha^3} + \frac{1}{4\alpha^2}\Big) + O(n^{-3}).\end{aligned}$$

(Any α-independent $O(1/n)$ terms, e.g. $-1/(12n)$, cancel when we form the ratio below.)

Therefore

$$\ln R := \ln \frac{\binom{cn^2}{n}}{c^n \binom{n^2}{n}} = \left[\ln \binom{cn^2}{n} - n \ln c\right] - \ln \binom{n^2}{n}$$
$$= \frac{1}{2}\Big(1 - \frac{1}{c}\Big) + \frac{1}{n}\Big(\frac{1}{2c} - \frac{1}{6c^2} - \frac{1}{3}\Big) + \frac{1}{n^2}\Big(\frac{1}{12c^3} + \frac{1}{4c^2} - \frac{1}{12} - \frac{1}{4}\Big) + O(n^{-3}).$$

Exponentiating gives the asymptotic form

$$\boxed{\frac{\binom{cn^2}{n}}{c^n \binom{n^2}{n}} = \exp\Big(\tfrac{1}{2}\,(1 - \tfrac{1}{c})\Big) \left[1 + \frac{1}{n}\Big(\frac{1}{2c} - \frac{1}{6c^2} - \frac{1}{3}\Big) + O\Big(\frac{1}{n^2}\Big)\right].}$$

In particular, the **asymptotic value** (the limit as $n \to \infty$) is the constant

$$\boxed{\lim_{n\to\infty} \frac{\binom{cn^2}{n}}{c^n \binom{n^2}{n}} = \exp\Big(\tfrac{1}{2}\,(1 - \tfrac{1}{c})\Big).}$$

Checks: for $c = 1$ the ratio is identically 1, and the limit gives $e^0 = 1$; as $c \to \infty$, the limit approaches $e^{1/2}$.

9 [05]

No. The *formula* still evaluates correctly when $n = -1$ (both sides are 0), but the *derivation* used in the text is not valid in that case because it relies on a step that implicitly assumes $0 \le 0 \le n$ (i.e., $n \ge 0$).

Why the derivation breaks (but the identity remains true)

1. **What the derivation does for $n \ge 0$.** Let $S := \sum_{0\le j\le n} ax^j$. The text splits off the $j = 0$ term and then reindexes:

$$S \;=\; a \;+\; \sum_{1\le j\le n} ax^j \;=\; a \;+\; x \sum_{1\le j\le n} ax^{j-1} \;=\; a \;+\; x \sum_{0\le j\le n-1} ax^j,$$

then subtracts xS and solves to get Eq. (14). Every equality there uses the summation manipulation rules from §1.2.3 and, crucially, the ability to *pull out* the $j = 0$ term—i.e., that 0 actually lies in the index set $\{j \mid 0 \le j \le n\}$, which requires $n \ge 0$.

2. **What happens when $n = -1$.** The index set $\{j \mid 0 \le j \le -1\}$ is empty, so

$$\sum_{0\le j\le -1} ax^j = 0 \quad \text{(empty sum)}.$$

But the very first step of the derivation would write

$$S \stackrel{?}{=} a + \sum_{1\le j\le -1} ax^j,$$

which incorrectly assumes that a $j = 0$ term is present to "split off." That step is invalid when $n = -1$. (In Knuth's rules this is the misuse of restriction/splitting (rule (d)) outside its preconditions.) Hence the *derivation* fails for $n = -1$.

3. **Yet the formula itself still holds for $n = -1$.** Evaluate both sides directly:

- LHS: $\sum_{0\le j\le -1} ax^j = 0$ (empty sum).
- RHS: $a\,\dfrac{1-x^{(-1)+1}}{1-x} = a\,\dfrac{1-x^0}{1-x} = a\,\dfrac{1-1}{1-x} = 0$. So the identity is true at $n = -1$; only the *particular derivation* is invalid there. (This is exactly the remark in the answers for §1.2.3.)

10 [05]

What Eq. (14) is and how it's derived

In §1.2.3 Knuth derives the finite geometric–series identity (for $x \neq 1$)

$$\sum_{j=0}^{n} x^j = \frac{x^{n+1}-1}{x-1}.$$

A standard derivation is

$$(x-1)\sum_{j=0}^{n} x^j = \sum_{j=0}^{n}(x^{j+1}-x^j) = \Big(\sum_{k=1}^{n+1} x^k\Big) - \Big(\sum_{j=0}^{n} x^j\Big) = x^{n+1}-1,$$

so dividing by $x-1$ (with $x \neq 1$) gives Eq. (14). In this section Knuth also formalizes *conditional* sums using the characteristic–function/bracket notation $[R] \in \{0,1\}$, so that sums over an empty index set evaluate to 0.

Check the edge cases $n = -1$ and $n = -2$

- **What the sum means for negative n.** In Knuth's notation $\sum_{0\le j\le n}(\cdots)$ is a sum over integers j satisfying $0 \le j \le n$. If $n < 0$, there *are no such* j, so the sum is **empty** and therefore equals **0**. (This follows immediately from the conditional-sum/bracket setup in §1.2.3.)
- $n = -1$**.** The left side is an empty sum $= 0$. The right side of Eq. (14) becomes $(x^0 - 1)/(x - 1) = 0$. So the identity happens to hold.
- $n = -2$**.** The left side is again an empty sum $= 0$. But the right side of Eq. (14) becomes

$$\frac{x^{-1} - 1}{x - 1} = \frac{1 - x}{x(x - 1)} = -\frac{1}{x} \neq 0 \quad (\text{for } x \neq 1).$$

Thus the *formula* of Eq. (14) does **not** match the meaning of the sum when $n = -2$.

Where the derivation breaks

Look at the telescoping step

$$\Big(\sum_{k=1}^{n+1} x^k\Big) - \Big(\sum_{j=0}^{n} x^j\Big) = x^{n+1} - 1.$$

This cancellation implicitly assumes there is an overlapping block $1 \le k \le n$ to cancel—i.e., it assumes $n \ge 0$. When $n = -2$ (indeed whenever $n \le -2$), **both sums are empty**, so their difference is 0, not $x^{n+1} - 1$. The telescoping argument therefore fails, and the subsequent division to obtain Eq. (14) is not justified.

11 [03]

1. **Direct counting at $x = 1$.** When $x = 1$, every term in the sum is just a. There are $n + 1$ terms (from $j = 0$ to $j = n$), so

$$\sum_{j=0}^{n} a\, 1^j = \underbrace{a + \cdots + a}_{n+1 \text{ terms}} = a(n + 1).$$

   Therefore, the *value* of the sum at $x = 1$ is $a(n + 1)$.

2. **Consistent extension of Eq. (14).** The closed form $a(1 - x^{n+1})/(1 - x)$ is undefined at $x = 1$ (division by zero), but its **limit** exists. Rewrite the fraction as

$$a\,\frac{x^{n+1} - 1}{x - 1}.$$

Taking $x \to 1$ and using the standard difference-quotient identity (or L'Hôpital's rule),

$$\lim_{x\to 1} \frac{x^{n+1}-1}{x-1} = \frac{d}{dx} x^{n+1}\Big|_{x=1} = n+1.$$

Hence the continuous extension of Eq. (14) at $x = 1$ is $a(n+1)$, agreeing with the direct count.

3. **Polynomial identity check (no calculus).** The identity

$$x^{n+1} - 1 = (x-1)\,(1 + x + \cdots + x^n)$$

holds for all x. For $x \neq 1$ it implies Eq. (14); evaluating the polynomial $1 + x + \cdots + x^n$ at $x = 1$ gives $n+1$. Thus the natural definition at $x = 1$ is again $a(n+1)$.

Final answer

$$\boxed{a(n+1)}$$

That is, replace the right-hand side by $a(n+1)$ when $x = 1$.

12 [10]

This is a geometric series with first term $a = 1$ and common ratio $r = \frac{1}{7}$.

1. Multiply by r: $rS_n = \frac{1}{7} + \frac{1}{7^2} + \cdots + \frac{1}{7^{n+1}}$.
2. Subtract: $S_n - rS_n = 1 - \frac{1}{7^{n+1}}$.
3. Factor: $(1-r)S_n = 1 - \frac{1}{7^{n+1}}$.
4. Solve for S_n using $1 - r = 1 - \frac{1}{7} = \frac{6}{7}$:

$$S_n = \frac{1-(1/7)^{n+1}}{1-1/7} = \frac{1-(1/7)^{n+1}}{6/7} = \frac{7}{6}\Big(1 - 7^{-(n+1)}\Big).$$

Equivalent closed forms you can use:

$$S_n = \frac{7^{n+1}-1}{6\cdot 7^n} = \frac{7}{6} - \frac{1}{6\cdot 7^n}.$$

Quick checks

- $n = 0$: $S_0 = 1$. Formula gives $\frac{7}{6} - \frac{1}{6 \cdot 7^0} = 1$.
- $n = 1$: $S_1 = 1 + \frac{1}{7} = \frac{8}{7}$. Formula gives $\frac{7}{6} - \frac{1}{42} = \frac{56-1}{42} = \frac{8}{7}$.

(As a side note, $\lim_{n\to\infty} S_n = \frac{7}{6}$.)

13 [10]

Tool: Eq. (15) from the text

From the derivation of arithmetic–progression sums in §1.2.3, Eq. (15) gives

$$\sum_{j=0}^{n}(a + bj) \;=\; a(n+1) \;+\; \frac{b\,n(n+1)}{2}.$$

Setting $a = 0,\ b = 1$ yields the well-known identity

$$\sum_{j=0}^{n} j \;=\; \frac{n(n+1)}{2}. \text{ (from Eq. 15)}$$

Compute $\sum_{j=m}^{n} j$

Write the desired sum as a difference of two prefix sums:

$$\sum_{j=m}^{n} j \;=\; \sum_{j=0}^{n} j \;-\; \sum_{j=0}^{m-1} j \;=\; \frac{n(n+1)}{2} \;-\; \frac{(m-1)m}{2}.$$

So a compact closed form is

$$\boxed{\sum_{j=m}^{n} j \;=\; \frac{n(n+1) - m(m-1)}{2}} \qquad (m \le n).$$

Equivalent forms (sometimes convenient)

Algebra gives a few interchangeable expressions:

$$\sum_{j=m}^{n} j \;=\; \frac{(n-m+1)(m+n)}{2} \;=\; m(n-m+1) \;+\; \frac{(n-m)(n-m+1)}{2}.$$

(The text's answer lists these equivalent forms as well.)

Quick check: Example $m = 3,\ n = 6$: $3 + 4 + 5 + 6 = 18$. Formula: $\frac{6 \cdot 7 - 3 \cdot 2}{2} = \frac{42-6}{2} = 18$.

14 [11]

Because j does not depend on k, the inner sum factors:

$$\sum_{k=r}^{s} jk \;=\; j\sum_{k=r}^{s} k.$$

Therefore,

$$\sum_{j=m}^{n}\sum_{k=r}^{s} jk \;=\; \left(\sum_{j=m}^{n} j\right)\left(\sum_{k=r}^{s} k\right).$$

Use the standard arithmetic–series formula from the previous exercise (with $m \le n$, and analogously $r \le s$):

$$\sum_{j=m}^{n} j = \frac{n(n+1)}{2} - \frac{(m-1)m}{2} = \frac{(n-m+1)(m+n)}{2},$$

$$\sum_{k=r}^{s} k = \frac{s(s+1)}{2} - \frac{(r-1)r}{2} = \frac{(s-r+1)(r+s)}{2}.$$

(Exercise 13 asks you to derive $\sum_{j=m}^{n} j$ in this way.)

Multiply the two results to get the closed form:

$$\boxed{\sum_{j=m}^{n}\sum_{k=r}^{s} jk = \frac{(n-m+1)(m+n)\,(s-r+1)(r+s)}{4}} \qquad (m \le n,\ r \le s).$$

If either range is empty (e.g., $m > n$ or $r > s$), the corresponding sum is 0, so the whole double sum is 0 by the conventions of §1.2.3.

Quick check

Take $m = 2, n = 4, r = 3, s = 5$. Left side:

$$\sum_{j=2}^{4}\sum_{k=3}^{5} jk = (2(3+4+5)) + (3(3+4+5)) + (4(3+4+5)) = 24 + 36 + 48 = 108.$$

Right side:

$$\frac{(4-2+1)(2+4)\,(5-3+1)(3+5)}{4} = \frac{(3)(6)\,(3)(8)}{4} = 108.$$

Matches perfectly.

15 [M22]

Step 1 — Small n to spot the pattern

$$\begin{aligned}
S_1 &= 2 \\
S_2 &= 2+8 = 10 \\
S_3 &= 2+8+24 = 34 \\
S_4 &= 98, \qquad S_5 = 258
\end{aligned}$$

These suggest the closed form $S_n = (n-1)2^{n+1} + 2$ (check: for $n = 5$, $4 \cdot 64 + 2 = 258$).

Step 2 — Derive it with the "geometric-series manipulation"

Let

$$S_n = \sum_{k=1}^{n} k\,2^k.$$

Multiply by 2 and reindex the first sum:

$$2S_n = \sum_{k=1}^{n} k\,2^{k+1} = \sum_{j=2}^{n+1} (j-1)\,2^j.$$

Subtract $S_n = \sum_{j=1}^{n} j\,2^j$ from this:

$$\begin{aligned}
2S_n - S_n &= \left[\sum_{j=2}^{n+1} (j-1)2^j\right] - \left[\sum_{j=1}^{n} j\,2^j\right] \\
&= \; n\,2^{n+1} \; + \; \sum_{j=2}^{n} ((j-1)-j)2^j \; - \; 1 \cdot 2 \\
&= \; n\,2^{n+1} \; - \; \sum_{j=1}^{n} 2^j.
\end{aligned}$$

Using the finite geometric sum $\sum_{j=1}^{n} 2^j = 2^{n+1} - 2$ (the very Eq. (14)–style identity Knuth just derived earlier in §1.2.3), we get

$$S_n = n\,2^{n+1} - (2^{n+1} - 2) = (n-1)\,2^{n+1} + 2.$$

Final answer

$$\boxed{\sum_{k=1}^{n} k\,2^k \;=\; (n-1)\,2^{n+1} + 2}$$

(For context, this exercise sits in §1.2.3 "Sums and Products," right after the geometric-series derivation.)

16 [M22]

Step 1 — Set up a telescoping trick

Let $S = S_n(x)$. Then

$$(1-x)S = S - xS \;=\; \sum_{j=0}^{n} jx^j \;-\; \sum_{j=0}^{n} jx^{j+1}.$$

Change index in the second sum: put $k = j+1$. Using the "change of variable" rule (rule (b) in §1.2.3),

$$\sum_{j=0}^{n} jx^{j+1} \;=\; \sum_{k=1}^{n+1} (k-1)x^k.$$

Hence

$$(1-x)S = \Big(0 + \sum_{j=1}^{n} jx^j\Big) - \Big(\sum_{k=1}^{n} (k-1)x^k + nx^{n+1}\Big) = \sum_{j=1}^{n} (j-(j-1))x^j - nx^{n+1} = \sum_{j=1}^{n} x^j - nx^{n+1}.$$

(The index-shift is exactly the kind of manipulation formalized as rule (b).)

Step 2 — Evaluate the geometric sum

For $x \neq 1$,

$$\sum_{j=1}^{n} x^j \;=\; \frac{x - x^{n+1}}{1-x},$$

the standard finite geometric progression with $a = 1$.

Therefore

$$(1-x)S \;=\; \frac{x - x^{n+1}}{1-x} - nx^{n+1}.$$

Step 3 — Solve for S

Multiply both sides by $1-x$ and simplify:

$$(1-x)^2 S \;=\; x - x^{n+1} - nx^{n+1}(1-x) = x - (n+1)x^{n+1} + nx^{n+2}.$$

Thus

$$\boxed{\sum_{j=0}^{n} j\,x^j \;=\; \frac{nx^{n+2} - (n+1)x^{n+1} + x}{(1-x)^2} \;=\; \frac{nx^{n+2} - (n+1)x^{n+1} + x}{(x-1)^2}} \qquad (x \neq 1).$$

This matches the target identity stated in the exercise (and it was obtained without induction).

Checks and remarks

- If $x = 1$, the sum is $\sum_{j=0}^{n} j = n(n+1)/2$ (consistent with the limit of the formula as $x \to 1$).
- The proof used only the index-shift rule and the finite geometric sum derived in Example 3 of §1.2.3, as intended for this section.

Start from

$$S_n(x) = \sum_{j=0}^{n} j\,x^j.$$

1. Pull out a factor x and shift:

$$S_n(x) = x \sum_{j=1}^{n} j\,x^{j-1} = x \sum_{j=0}^{n-1} (j+1)x^j.$$

2. Split the last sum:

$$S_n(x) = x \sum_{j=0}^{n-1} j\,x^j + x \sum_{j=0}^{n-1} x^j = x\Big(\sum_{j=0}^{n} j\,x^j - nx^n\Big) \;+\; x \sum_{j=0}^{n-1} x^j.$$

3. Recognize the geometric sum $\sum_{j=0}^{n-1} x^j = \dfrac{1-x^n}{1-x}$ (from §1.2.3) and solve for S_n:

$$S_n(x) = xS_n(x) - nx^{n+1} + x\,\frac{1-x^n}{1-x}.$$

$$(1-x)S_n(x) = -nx^{n+1} + x\,\frac{1-x^n}{1-x}.$$

$$S_n(x) = \frac{x(1-x^n) - nx^{n+1}(1-x)}{(1-x)^2} = \frac{x-(n+1)x^{n+1}+nx^{n+2}}{(1-x)^2}, \qquad (x \neq 1).$$

This is exactly the result Exercise 20 later depends on (it rewrites $\sum(n-k)b^k$ via $n\sum b^k - \sum kb^k$).

Edge cases and checks

- $x = 1$**.** Take the limit $x \to 1$ (or note directly from the definition): $S_n(1) = \displaystyle\sum_{j=0}^{n} j = \frac{n(n+1)}{2}$.
- **Polynomials.** The identity holds over any ring where the algebra above makes sense; interpreted as polynomials in x, both sides are identical for all $x \neq 1$.
- **Book context.** The technique matches §1.2.3's emphasis on "manipulating sums" (including index shifts and basic identities) and is the same chain shown in the official "Key steps" for exercises 15–16.

Final formula

$$\boxed{\sum_{j=0}^{n} j\,x^j = \frac{x-(n+1)x^{n+1}+nx^{n+2}}{(1-x)^2} \quad (x \neq 1), \qquad \sum_{j=0}^{n} j = \frac{n(n+1)}{2} \text{ when } x = 1.}$$

17 [M00]

By definition, the index j in $\sum_{j\in S} 1$ ranges over every element of S exactly once, and the summand is the constant 1 each time. Therefore the sum simply counts how many elements are in S:

$$\sum_{j\in S} 1 \;=\; |S|.$$

This can also be written using Iverson's bracket notation (introduced in §1.2.3):

$$\sum_{j \in S} 1 \;=\; \sum_{j} [\, j \in S \,],$$

and by Eq. (17) in the text,

$$\sum_{R(j)} a_j \;=\; \sum_{j} a_j [\, R(j) \,],$$

so with $a_j \equiv 1$ and $R(j) \equiv (j \in S)$ the right-hand side counts the number of j making $R(j)$ true—i.e., $|S|$.

Edge cases.

- If $S = \emptyset$, the sum is 0.
- If S is finite with n elements, the sum is n.
- If S is infinite, the (ordinary) sum does not converge in the usual sense; informally one may say it is $+\infty$.

18 [M20]

We start from the general interchange law (Eq. (9)) for finite sums:

$$\sum_{R(i)} \sum_{S(i,j)} a_{ij} \;=\; \sum_{S'(j)} \sum_{R'(i,j)} a_{ij},$$

where $S'(j)$ is "there exists an i such that both $R(i)$ and $S(i,j)$ hold," and $R'(i,j)$ is "both $R(i)$ and $S(i,j)$ hold."

In this exercise we are told to take • $R(i)$: "n is a multiple of i" (i.e., $i \mid n$); • $S(i,j)$: "$1 \le j < i$."

Determine the new limits:

1. The outer relation $S'(j)$. We need values of j for which there exists some divisor i of n with $1 \le j < i$. Taking $i = n$ (which always divides n), this is possible exactly when $1 \le j \le n-1$. Thus

$$S'(j) \;\equiv\; 1 \le j \le n-1.$$

2. The inner relation $R'(i,j)$. This must enforce both original conditions for fixed j: $i \mid n$ and $1 \le j < i$. Equivalently,

$$R'(i,j) \;\equiv\; i \mid n \;\text{ and }\; j < i \text{ (hence } j+1 \le i \le n).$$

Therefore the interchanged sum is

$$\boxed{\sum_{i \mid n} \sum_{j=1}^{i-1} a_{ij} \;=\; \sum_{j=1}^{n-1} \sum_{\substack{i \mid n \\ i > j}} a_{ij}}.$$

(That is exactly Eq. (9) specialized to these relations.)

Quick sanity check (optional): with $n = 6$ and $a_{ij} \equiv 1$, the left side counts $(1-1)+(2-1)+(3-1)+(6-1) = 8$. The right side counts, for $j = 1,2,3,4,5$, the number of divisors of 6 exceeding j, which are $3,2,1,1,1$, again totaling 8. This confirms the limits are consistent.

19 [20]

Result

$$\boxed{\sum_{j=m}^{n} (a_j - a_{j-1}) = a_n - a_{m-1}}$$

(assuming $m \le n$; if $m > n$ the sum is empty and equals 0.)

Why it works (two equivalent derivations)

1) Telescoping by expansion. Write out a few terms:

$$(a_m - a_{m-1}) + (a_{m+1} - a_m) + \cdots + (a_n - a_{n-1}),$$

and cancel adjacent $+a_k$ with $-a_k$. Everything cancels except the first "$-a_{m-1}$" and the last "$+a_n$", yielding $a_n - a_{m-1}$. (This is the classic telescoping effect; the exercise appears exactly as stated on p. 35 of the section.)

2) Formal manipulation with Knuth's summation rules. Let

$$S = \sum_{j=m}^{n} (a_j - a_{j-1}) = \sum_{j=m}^{n} a_j \;-\; \sum_{j=m}^{n} a_{j-1} \quad \text{(split a sum; cf. Eq. (8)).}$$

Now change index $k = j-1$ in the second sum; as j runs $m \ldots n$, k runs $(m-1) \ldots (n-1)$ (index shift as in rule (b)). Thus

$$S = \sum_{k=m}^{n} a_k - \sum_{k=m-1}^{n-1} a_k = (a_m + \cdots + a_n) - (a_{m-1} + \cdots + a_{n-1}) = a_n - a_{m-1}.$$

(See the linearity $\sum(b_i + c_i) = \sum b_i + \sum c_i$ as Eq. (8), and the standard index-shift technique used repeatedly in §1.2.3 examples.)

Remark on edge cases. Knuth's convention allows indices over all integers (the bracket-notation discussion explicitly assumes a_j is defined for all j); with that convention, the identity above is exact for $m \le n$, while for $m > n$ the sum is 0. Using Iverson brackets one can write a single formula:

$$\sum_{j=m}^{n} (a_j - a_{j-1}) = [m \le n]\,(a_n - a_{m-1}),$$

where $[P]$ is 1 if P is true and 0 otherwise.

20 [25]

(a) Decimal ($b = 10$) in Σ-notation

Let

$$C_n \;=\; \underbrace{12\ldots n}_{\text{as a decimal integer}} \;=\; \sum_{j=1}^{n} j\,10^{\,n-j}.$$

Then the observed pattern is the identity

$$9\,C_n + (n+1) \;=\; \sum_{k=0}^{n} 10^k \;=\; \frac{10^{\,n+1} - 1}{9}.$$

Thus, in pure Σ-form:

$$9 \sum_{j=1}^{n} j\,10^{\,n-j} + (n+1) \;=\; \sum_{k=0}^{n} 10^k.$$

(These right-hand numbers are the length-n+1 "repunit" in base 10.)

(b) Generalization to base b

Define the "concatenation" number in base b

$$C_n^{(b)} \;=\; \sum_{j=1}^{n} j\, b^{\,n-j}.$$

Then for any $b > 1$,

$$(b-1)\, C_n^{(b)} + (n+1) \;=\; \sum_{k=0}^{n} b^k \;=\; \frac{b^{\,n+1}-1}{b-1}.$$

(If you want the literal "digits $1, 2, \ldots, n$" in base b, you should take $n \le b-1$; the identity itself is purely algebraic and holds for all n.)

(c) Proof using $\sum jx^j$

We'll use the standard finite power-sum identity (given in this section and also as Ex. 16): for $x \neq 1$,

$$\sum_{j=0}^{n} j\, x^j = \frac{nx^{n+2} - (n+1)x^{n+1} + x}{(x-1)^2}.$$

(Notice the $j = 0$ term is 0, so starting at $j = 1$ is equivalent.)

Let $x = \frac{1}{b}$ and compute

$$S_n(b) \;:=\; \sum_{j=1}^{n} j\, b^{\,n-j} \;=\; b^n \sum_{j=1}^{n} j\, x^j.$$

Plug the identity in:

$$S_n(b) = b^n\, \frac{nx^{n+2} - (n+1)x^{n+1} + x}{(x-1)^2}.$$

Since $x = \frac{1}{b}$, we have

$$b^n x^{n+2} = b^{-2}, \quad b^n x^{n+1} = b^{-1}, \quad b^n x = b^{\,n-1}, \quad (x-1)^2 = \frac{(b-1)^2}{b^2}.$$

Therefore

$$S_n(b) = \frac{n - (n+1)b + b^{\,n+1}}{b-1}.$$

Multiply by $(b-1)$ and add $(n+1)$:

$$(b-1)S_n(b) + (n+1) = \frac{b^{n+1}-1}{b-1} = \sum_{k=0}^{n} b^k,$$

which is exactly the claimed formula from parts (a)–(b). QED.

Quick check (decimal, $n = 4$)

$$9\sum_{j=1}^{4} j\,10^{4-j} + 5 = 9(1\cdot 10^3 + 2\cdot 10^2 + 3\cdot 10 + 4) + 5 = 9\cdot 1234 + 5 = 11111,$$

and $\sum_{k=0}^{4} 10^k = 11111$, as required.

21 [M25]

1. Start from the left–hand side and use (17) to convert "conditioned" sums into ordinary sums with indicator brackets:

$$\sum_{R(j)} a_j + \sum_{S(j)} a_j = \sum_j a_j\,[R(j)] \;+\; \sum_j a_j\,[S(j)]. \quad \text{(by (17))}$$

2. Combine the two sums into one by distributivity over a sum of terms (use (8) with the "domain" being all integers j):

$$\sum_j a_j\,[R(j)] \;+\; \sum_j a_j\,[S(j)] = \sum_j a_j([R(j)] + [S(j)]). \quad \text{(by (8))}$$

3. Use the elementary identity for indicator (Iverson) brackets:

$$[R] + [S] = [R \text{ or } S] + [R \text{ and } S].$$

(This is checked case-by-case: if neither holds, both sides are 0; if exactly one holds, both sides are 1; if both hold, both sides are 2.)

Thus

$$\sum_j a_j([R(j)] + [S(j)]) = \sum_j a_j([R(j) \vee S(j)] + [R(j) \wedge S(j)]).$$

4. Split the sum again by (8), then convert back with (17):

$$\sum_j a_j[\,R(j) \vee S(j)\,] + \sum_j a_j[\,R(j) \wedge S(j)\,] = \sum_{R(j) \text{ or } S(j)} a_j + \sum_{R(j) \text{ and } S(j)} a_j.$$

This is exactly rule (d).

22 [20]

Product analogs

1. Change of variables (analog of Eq. (5)). If p is a bijection on the relevant index set, then

$$\prod_{R(i)} a_i \;=\; \prod_{R(j)} a_j \;=\; \prod_{R(p(j))} a_{p(j)}.$$

This is the same reindexing principle as for sums; it just uses associativity/commutativity of multiplication instead of addition. (For infinite products, similar convergence caveats apply as noted for sums.)

2. Interchange of order (analog of Eq. (7)). For rectangular domains,

$$\prod_{R(i)} \prod_{S(j)} a_{ij} \;=\; \prod_{S(j)} \prod_{R(i)} a_{ij}.$$

For dependent domains (analog of Eq. (9)), if $S(i,j)$ constrains (i,j), then

$$\prod_{R(i)} \prod_{S(i,j)} a_{ij} \;=\; \prod_{S'(j)} \prod_{R'(i,j)} a_{ij},$$

where $S'(j)$ is "there exists i with $R(i)$ and $S(i,j)$," and $R'(i,j)$ is "$R(i)$ and $S(i,j)$."

3. Distributivity over termwise combination (analog of Eq. (8)). With products, the natural distributive law is

$$\prod_{R(i)} (b_i c_i) \;=\; \Big(\prod_{R(i)} b_i\Big)\Big(\prod_{R(i)} c_i\Big).$$

(This mirrors $\sum_{R(i)}(b_i + c_i) = \sum_{R(i)} b_i + \sum_{R(i)} c_i$ but with "+" replaced by "×".)

4. Domain manipulation (analog of Eq. (11)). For arbitrary relations $R(j)$ and $S(j)$,

$$\Big(\prod_{R(j)} a_j\Big)\Big(\prod_{S(j)} a_j\Big) \;=\; \Big(\prod_{R(j)\,\text{or}\,S(j)} a_j\Big)\Big(\prod_{R(j)\,\text{and}\,S(j)} a_j\Big).$$

Intuition: Elements in $R \cap S$ are multiplied twice on the left, so the right side is the product over the union (once) times the product over the intersection (once more). A crisp proof uses Iverson brackets: $\prod_R a_j = \prod_j a_j^{[R(j)]}$, and the Boolean identity $[R] + [S] = [R \vee S] + [R \wedge S]$.

These four identities are the exact multiplicative counterparts of Eqs. (5), (7), (8), and (11) in the text.

23 [10]

1) Empty sum must be 0

Section 1.2.3 uses Iverson brackets to write, for any sequence a_j,

$$\sum_{R(j)} a_j \;=\; \sum_j a_j\,[R(j)],$$

and one of the fundamental "set-algebra" rules for sums is

$$\sum_{R(j)\vee S(j)} a_j = \sum_{R(j)} a_j + \sum_{S(j)} a_j - \sum_{R(j)\wedge S(j)} a_j.$$

(This is a direct consequence of $[R] + [S] = [R \vee S] + [R \wedge S]$.)

Take S to be the **always-false** condition. Then $R \vee S$ is just R, and $R \wedge S$ is false. Plugging in,

$$\sum_{R(j)} a_j = \sum_{R(j)} a_j + \underbrace{\sum_{\text{false}} a_j}_{=\alpha} - \underbrace{\sum_{\text{false}} a_j}_{=\alpha}.$$

This forces $\alpha = 0$. In words: to keep the union/splitting identity valid when one side is empty, the empty sum must be 0.

2) Empty product must be 1

Products satisfy completely analogous rules in §1.2.3 (see the product analogs noted right before Exercise 23): for any b_i, c_i,

$$\prod_{R(i)} b_i c_i \;=\; \Big(\prod_{R(i)} b_i\Big)\Big(\prod_{R(i)} c_i\Big),$$

and for two conditions R, S,

$$\Big(\prod_{R(j)} a_j\Big)\Big(\prod_{S(j)} a_j\Big) \;=\; \Big(\prod_{R(j)\vee S(j)} a_j\Big)\Big(\prod_{R(j)\wedge S(j)} a_j\Big).$$

Again set S to **false**. Then $R \vee S = R$ and $R \wedge S$ is false, yielding

$$\Big(\prod_{R(j)} a_j\Big) \underbrace{\prod_{\text{false}} a_j}_{=\beta} = \Big(\prod_{R(j)} a_j\Big) \underbrace{\prod_{\text{false}} a_j}_{=\beta}.$$

This identity holds for any β, so we need one more standard requirement: When we concatenate disjoint index sets A and B, we want

$$\prod_{j\in A\cup B} a_j = \Big(\prod_{j\in A} a_j\Big)\Big(\prod_{j\in B} a_j\Big) \quad \text{(even if } A \text{ or } B \text{ is empty).}$$

Now take $B = \emptyset$. Then

$$\prod_{j\in A} a_j \;=\; \Big(\prod_{j\in A} a_j\Big) \underbrace{\prod_{j\in\emptyset} a_j}_{=\beta} \quad \text{for all sequences } \{a_j\}.$$

Canceling $\prod_{j\in A} a_j$ (choose any A with at least one index and $a_j \neq 0$ to make this cancellation indisputable), we get $\beta = 1$.

(Equivalently, from $\prod_{R(i)} b_i c_i = (\prod b_i)(\prod c_i)$, set R false and $b_i = c_i$ to get $\beta = \beta^2$, hence $\beta \in \{0, 1\}$; $\beta = 0$ is impossible because it would make every product zero after adjoining an empty part, so $\beta = 1$.)

24 [20]

Proof (by induction on the number of indices that satisfy R)

Let $S = \{j \in \mathbb{Z} : R(j)\}$. By hypothesis, $|S| < \infty$. We prove, for each $m \geq 0$,

$$\text{(P(m))} \qquad \forall S \text{ with } |S| = m : \quad \log_b\Big(\prod_{j\in S} a_j\Big) = \sum_{j\in S} \log_b a_j.$$

Base case $m = 0$. If $S = \emptyset$, then by the section's convention the empty product is 1 and the empty sum is 0. Hence

$$\log_b\Big(\prod_{j\in\emptyset} a_j\Big) = \log_b(1) = 0 = \sum_{j\in\emptyset} \log_b a_j.$$

This proves (P(0)).

(If you prefer also to see $m = 1$: If $S = \{j_0\}$, then $\log_b(\prod_{j\in S} a_j) = \log_b(a_{j_0}) = \sum_{j\in S} \log_b a_j$.)

Inductive step. Assume (P(m)) holds for some $m \geq 0$. Let S be any finite set with $|S| = m+1$. Choose any $j^* \in S$ and set $T = S \setminus \{j^*\}$ (so $|T| = m$). Then

$$\prod_{j\in S} a_j \;=\; \Big(\prod_{j\in T} a_j\Big)\, a_{j^*}.$$

Since all terms are positive, we can use $\log_b(xy) = \log_b x + \log_b y$:

$$\log_b\Big(\prod_{j\in S} a_j\Big) = \log_b\Big(\prod_{j\in T} a_j\Big) + \log_b(a_{j^*}).$$

Apply the induction hypothesis (P(m)) to T:

$$\log_b\Big(\prod_{j\in T} a_j\Big) = \sum_{j\in T} \log_b a_j.$$

Therefore

$$\log_b\Big(\prod_{j\in S} a_j\Big) = \sum_{j\in T} \log_b a_j \;+\; \log_b(a_{j^*}) = \sum_{j\in S} \log_b a_j,$$

as required. This proves (P(m+1)).

By induction on $|S|$, the identity holds for every finite S.

25 [15]

Diagnosis, line by line

- **Line 1 (OK):** $\sum_{i=1}^{n}\sum_{j=1}^{n} 1$ is a perfectly good double sum: it counts n terms of 1 for each of n outer values.
- **Line 2 (Wrong – variable capture):** Writing $\sum_{i=1}^{n}\sum_{i=1}^{n}$ uses **the same dummy variable** i **twice**, once for the outer sum and again for the inner. Dummy (index) variables can be renamed arbitrarily, but they **must be distinct** when nested; otherwise the inner i *shadows* the outer i and changes the meaning. Proper use of dummy indices is discussed when Knuth introduces summation notation.
- **Line 3 (Wrong quantity):** Even if we correct the indices to $\sum_{i=1}^{n}\sum_{j=1}^{n} 1$, the inner sum $\sum_{j=1}^{n} 1$ equals the constant n, **not** i. So the third line should read $\sum_{i=1}^{n} n$, not $\sum_{i=1}^{n} i$. (This is exactly what the answer key hints at.)
- **Line 4 (OK in itself):** $\sum_{i=1}^{n} i = n(n+1)/2$ is a correct identity—but it's irrelevant here because line 3 was wrong.

Correct derivation

$$\sum_{i=1}^{n}\sum_{j=1}^{n} 1 = \sum_{i=1}^{n}(\sum_{j=1}^{n} 1) = \sum_{i=1}^{n} n = n \cdot n = n^2.$$

Intuitively, you're counting an $n \times n$ grid of ones: there are n^2 of them.

26 [25]

Solution (counting exponents). Each factor a_k appears in P whenever a pair (i, j) uses index k in either position.

- Pairs with $i = k$: j can be $k, k+1, \dots, n$ — that's $n - k + 1$ occurrences of a_k.
- Pairs with $j = k$: i can be $0, 1, \dots, k$ — that's $k + 1$ occurrences of a_k.

These counts are disjoint except at $(i, j) = (k, k)$; but that pair contributes a_k^2, exactly accounting for being "seen" once in each count. Hence the total exponent of a_k in P is

$$(n - k + 1) + (k + 1) = n + 2.$$

Therefore

$$P = \prod_{k=0}^{n} a_k^{n+2} \;=\; \Big(\prod_{k=0}^{n} a_k\Big)^{n+2}.$$

This is the closed form.

Alternative derivation (triangular-to-rectangular reindexing). Rewrite the triangular product as

$$P = \prod_{i=0}^{n} \prod_{j=i}^{n} (a_i a_j) = \Big(\prod_{i=0}^{n} a_i^{n-i+1}\Big)\Big(\prod_{j=0}^{n} a_j^{j+1}\Big) = \prod_{k=0}^{n} a_k^{(n-k+1)+(k+1)} = \Big(\prod_{k=0}^{n} a_k\Big)^{n+2}.$$

This "flip the order" maneuver mirrors the technique shown in **Example 2** of §1.2.3, where a triangular double sum is rearranged by interchanging the roles of the indices. The same idea works verbatim for products.

Sanity check (small n). For $n = 2$ the pairs (i, j) with $i \le j$ are $(0,0), (0,1), (0,2), (1,1), (1,2), (2,2)$. The product is $(a_0^2)\,(a_0a_1)\,(a_0a_2)\,(a_1^2)\,(a_1a_2)\,(a_2^2) = a_0^4a_1^4a_2^4 = (a_0a_1a_2)^4$, and $n + 2 = 4$, matching the formula.

Answer.

$$\boxed{\prod_{0\le i\le j\le n} (a_i\, a_j) = \left(\prod_{k=0}^{n} a_k\right)^{n+2}.}$$

27 [M20]

Proof by induction on n

Base case ($n = 1$). $(1 - a_1) = 1 - a_1$. The inequality holds with equality.

Inductive step. Assume for some $n - 1 \ge 1$ that

$$\prod_{j=1}^{n-1}(1 - a_j) \;\ge\; 1 - \sum_{j=1}^{n-1} a_j.$$

Multiply both sides by the positive quantity $(1 - a_n)$ (valid since $0 < a_n < 1$):

$$\prod_{j=1}^{n}(1 - a_j) \;\ge\; (1 - \sum_{j=1}^{n-1} a_j)(1 - a_n) = 1 - \sum_{j=1}^{n} a_j \;+\; \Big(\sum_{j=1}^{n-1} a_j\Big)a_n.$$

Because each $a_j > 0$, the last term is nonnegative (indeed positive when $n \ge 2$), hence

$$\prod_{j=1}^{n}(1 - a_j) \;\ge\; 1 - \sum_{j=1}^{n} a_j.$$

This completes the induction. Moreover, when $n \geq 2$ the inequality is **strict** because $(\sum_{j=1}^{n-1} a_j)a_n > 0$.

Alternative (algebraic) view

Expanding the product,

$$\prod_{j=1}^{n}(1-a_j) = 1 - \sum_j a_j + \sum_{i<j} a_i a_j - \sum_{i<j<k} a_i a_j a_k + \cdots + (-1)^n a_1 \cdots a_n.$$

All squarefree monomials have positive coefficients in absolute value, and since every $a_j \in (0,1)$, each monomial is nonnegative. Dropping the alternating tail $\sum_{i<j} a_i a_j - \sum_{i<j<k} \cdots$ (which equals $(\sum_{j=1}^{n-1} a_j)a_n$ plus further nonnegative combinations after a regrouping) gives the same lower bound $1 - \sum_j a_j$.

Remarks and tightness

- Equality holds for $n = 1$. For $n \geq 2$ and $0 < a_j < 1$, the inequality is strict.
- Intuition (probabilistic): If a_j are "failure probabilities" of independent events, $\prod(1-a_j)$ is the probability that **none** occur, while $1 - \sum a_j$ is the classical union bound lower estimate. The algebraic inequality above mirrors that bound.

28 [M22]

Factor each term:

$$1 - \frac{1}{j^2} = \frac{j^2-1}{j^2} = \frac{(j-1)(j+1)}{j \cdot j} = \frac{j-1}{j} \cdot \frac{j+1}{j}.$$

Thus the product splits into two telescoping products:

$$\prod_{j=2}^{n}\Big(1 - \frac{1}{j^2}\Big) = \Big(\prod_{j=2}^{n} \frac{j-1}{j}\Big)\Big(\prod_{j=2}^{n} \frac{j+1}{j}\Big).$$

Compute each:

- First:

$$\prod_{j=2}^{n} \frac{j-1}{j} = \frac{1}{2} \cdot \frac{2}{3} \cdot \frac{3}{4} \cdots \frac{n-1}{n} = \frac{1}{n} \quad \text{(complete cancellation).}$$

- Second:

$$\prod_{j=2}^{n} \frac{j+1}{j} = \frac{3}{2} \cdot \frac{4}{3} \cdot \frac{5}{4} \cdots \frac{n+1}{n} = \frac{n+1}{2}.$$

Multiply:

$$\prod_{j=2}^{n} \Big(1 - \frac{1}{j^2}\Big) = \frac{1}{n} \cdot \frac{n+1}{2} = \boxed{\frac{n+1}{2n}}.$$

Quick checks:

- $n = 2$: $(1 - \frac{1}{4}) = \frac{3}{4}$, and $(2+1)/(2 \cdot 2) = \frac{3}{4}$.
- $n = 3$: $(1 - \frac{1}{4})(1 - \frac{1}{9}) = \frac{3}{4} \cdot \frac{8}{9} = \frac{2}{3}$, and $(3+1)/(2 \cdot 3) = \frac{2}{3}$.

As a corollary, letting $n \to \infty$ gives $\prod_{j=2}^{\infty}(1 - \frac{1}{j^2}) = \frac{1}{2}$.

29 [M30]

Let

$$A \;=\; \sum_{1 \le i \le j \le k \le n} x_i x_j x_k, \qquad T \;=\; \Big(\sum_{r=1}^{n} x_r\Big)^3 = \sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{k=1}^{n} x_i x_j x_k,$$

and $S_m = \sum_{r=1}^{n} x_r^m$ as above.

We'll classify the monomials in the full expansion of T by how many indices are equal.

1) All three indices distinct (i, j, k **all different**)**.** Each unordered triple $\{i, j, k\}$ with $i < j < k$ appears $3! = 6$ times in T. If we denote

$$D = \sum_{1 \le i < j < k \le n} x_i x_j x_k,$$

then the contribution of this class to T is $6D$.

2) Exactly two indices equal (say $i = j \neq k$, or permutations). For each ordered pair (i, k) with $i \neq k$, one gets a term $x_i^2 x_k$. There are 3 choices for which index is the singleton, so the total contribution is

$$3 \sum_{i \neq k} x_i^2 x_k \;=\; 3(S_1 S_2 - S_3),$$

because $S_1S_2 = \sum_{i,k} x_i^2 x_k$ and subtracting the $i = k$ terms ($\sum_i x_i^3 = S_3$) leaves $\sum_{i\neq k} x_i^2 x_k$.

3) All three indices equal. The contribution is $\sum_{i=1}^n x_i^3 = S_3$.

Combining the three cases gives

$$T \;=\; 6D \;+\; 3(S_1S_2 - S_3) \;+\; S_3 \;=\; 6D \;+\; 3S_1S_2 \;-\; 2S_3,$$

hence

$$D \;=\; \frac{T - 3S_1S_2 + 2S_3}{6}. \qquad (\,)$$

Now relate A to D. The sum A (with $i \le j \le k$) counts:

- each distinct triple $x_i x_j x_k$ ($i < j < k$) **once** contributes D;
- each "two equal" monomial **once** (either $x_i^2 x_k$ with $i < k$ or $x_i x_j^2$ with $i < j$). Summed over all pairs, these contribute exactly $\sum_{i\neq k} x_i^2 x_k = S_1S_2 - S_3$;
- each "all equal" monomial **once** contributes S_3.

Therefore

$$A \;=\; D \;+\; (S_1S_2 - S_3) \;+\; S_3 \;=\; D + S_1S_2.$$

Insert () to eliminate D:

$$A \;=\; \frac{T - 3S_1S_2 + 2S_3}{6} \;+\; S_1S_2 = \frac{T}{6} + \frac{S_1S_2}{2} + \frac{S_3}{3}.$$

Recalling $T = S_1^3$, we obtain the desired closed form:

$$\boxed{\sum_{1\le i\le j\le k\le n} x_i x_j x_k = \frac{1}{6}\,S_1^3 \;+\; \frac{1}{2}\,S_1S_2 \;+\; \frac{1}{3}\,S_3.}$$

This matches the formula Knuth cites when referring back to Exercise 1.2.3–29.

Context / cross-check

Section 1.2.3 proves the analogous 2-index identity

$$\sum_{0\le i\le j\le n} a_i a_j = \tfrac{1}{2}\Big(\big(\sum_{i=0}^n a_i\big)^2 + \sum_{i=0}^n a_i^2\Big),$$

using the same "count the multiplicities" idea; our argument is its 3-index analogue.

(Optional generalization.) The generating-function method in §1.2.9 shows that the complete homogeneous symmetric sums $h_m = \sum_{1\le j_1\le\cdots\le j_m\le n} x_{j_1}\cdots x_{j_m}$ satisfy

$$\sum_{m\ge 0} h_m z^m = \exp\Big(\sum_{k\ge 1}\frac{S_k}{k}z^k\Big),$$

from which one can derive h_2 and h_3 as above (and a recurrence $h_n = \frac{1}{n}\sum_{k=1}^{n} S_k h_{n-k}$).

30 [M23]

Write

$$A = \sum_i a_i c_i, \quad B = \sum_i b_i d_i, \quad C = \sum_i a_i d_i, \quad D = \sum_i b_i c_i.$$

We want $AB - CD$.

Consider the double sum

$$S \;=\; \tfrac{1}{2}\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}(a_i b_j - a_j b_i)(c_i d_j - c_j d_i).$$

Expand the product inside:

$$\begin{aligned}(a_i b_j - a_j b_i)(c_i d_j - c_j d_i) = a_i b_j c_i d_j \;-\; a_i b_j c_j d_i \\ - a_j b_i c_i d_j \;+\; a_j b_i c_j d_i.\end{aligned}$$

Sum over i, j and group terms:

$$\sum_{i,j} a_i b_j c_i d_j = \Big(\sum_i a_i c_i\Big)\Big(\sum_j b_j d_j\Big) = AB,$$

$$\sum_{i,j} a_i b_j c_j d_i = \Big(\sum_i a_i d_i\Big)\Big(\sum_j b_j c_j\Big) = CD,$$

and by swapping dummy indices $i \leftrightarrow j$ you see the other two double–sum terms are identical to these. Hence

$$S = \tfrac{1}{2}(AB - CD - AB + CD) + \tfrac{1}{2}(AB - CD) = AB - CD.$$

Finally, note that the summand is **antisymmetric** in i, j, so the $i = j$ terms vanish and every unordered pair $\{i, j\}$ with $i \neq j$ appears twice with the same value. Therefore

$$AB - CD = \sum_{0 \le i < j < n} (a_i b_j - a_j b_i)(c_i d_j - c_j d_i),$$

which is the desired identity.

31 [M20]

Using Binet's identity (one-line derivation)

Binet's formula states

$$\Big(\sum_j a_j x_j\Big)\Big(\sum_j b_j y_j\Big) = \Big(\sum_j a_j y_j\Big)\Big(\sum_j b_j x_j\Big) + \sum_{1 \le j < k \le n} (a_j b_k - a_k b_j)(x_j y_k - x_k y_j).$$

Choose

$$a_j = 1, \qquad b_j = u_j, \qquad x_j = 1, \qquad y_j = v_j.$$

Then

$$(a_j b_k - a_k b_j) = (u_k - u_j) = -(u_j - u_k), \qquad (x_j y_k - x_k y_j) = (v_k - v_j) = -(v_j - v_k),$$

so the product in Binet's last sum equals $(u_j - u_k)(v_j - v_k)$. Therefore

$$\sum_{1 \le j < k \le n} (u_j - u_k)(v_j - v_k) = \Big(\sum_j a_j x_j\Big)\Big(\sum_j b_j y_j\Big) - \Big(\sum_j a_j y_j\Big)\Big(\sum_j b_j x_j\Big).$$

With our choices,

$$\sum_j a_j x_j = n, \quad \sum_j b_j y_j = \sum_{j=1}^{n} u_j v_j, \quad \sum_j a_j y_j = \sum_{j=1}^{n} v_j, \quad \sum_j b_j x_j = \sum_{j=1}^{n} u_j.$$

Hence

$$\boxed{\sum_{1 \le j < k \le n} (u_j - u_k)(v_j - v_k) = n \sum_{j=1}^{n} u_j v_j - \Big(\sum_{j=1}^{n} u_j\Big)\Big(\sum_{j=1}^{n} v_j\Big)}.$$

(This applies Binet exactly as suggested in the exercise statement.)

Sanity check and interpretation

Write $\bar{u} = \frac{1}{n}\sum u_j$ and $\bar{v} = \frac{1}{n}\sum v_j$. Then the right-hand side is

$$n\sum_{j=1}^{n}(u_j - \bar{u})(v_j - \bar{v}),$$

so the exercise's sum equals n times the (unnormalized) cross-deviation from the means—consistent with the case $n = 2$, $(u_1 - u_2)(v_1 - v_2) = 2(u_1 - \bar{u})(v_1 - \bar{v}) + 2(u_2 - \bar{u})(v_2 - \bar{v})$.

(Optional) Direct expansion (no Binet)

Expanding and regrouping,

$$\begin{aligned}
S &= \sum_{j<k}(u_j v_j + u_k v_k - u_j v_k - u_k v_j) \\
&= \underbrace{\sum_{j<k} u_j v_j + \sum_{j<k} u_k v_k}_{(n-1)\sum u_j v_j} - \underbrace{\left(\sum_{j<k} u_j v_k + \sum_{j<k} u_k v_j\right)}_{\sum_{j\neq k} u_j v_k = (\sum u)(\sum v) - \sum uv} \\
&= (n-1)\sum u_j v_j - \Big((\sum u)(\sum v) - \sum u_j v_j\Big) \\
&= n\sum u_j v_j - (\sum u)(\sum v).
\end{aligned}$$

Same result, as expected.

32 [M20]

Direct (distributive) proof

Expand the product of n sums by repeated use of distributivity:

$$\begin{aligned}
\prod_{j=1}^{n}\left(\sum_{i=1}^{m} a_{ij}\right) &= \left(\sum_{i_1=1}^{m} a_{i_1 1}\right)\left(\sum_{i_2=1}^{m} a_{i_2 2}\right)\cdots\left(\sum_{i_n=1}^{m} a_{i_n n}\right) \\
&= \sum_{i_1=1}^{m}\sum_{i_2=1}^{m}\cdots\sum_{i_n=1}^{m} a_{i_1 1} a_{i_2 2}\cdots a_{i_n n} \\
&= \sum_{1\le i_1,\ldots,i_n\le m}\prod_{j=1}^{n} a_{i_j j}.
\end{aligned}$$

At each step you "pull" a sum out of a product via distributivity; after doing this n times, you get a nested sum over all n-tuples $(i_1, \dots, i_n) \in \{1, \dots, m\}^n$, with term $a_{i_1 1} \cdots a_{i_n n}$. This is exactly the right-hand side.

Induction on n (equally rigorous)

Base case $n = 1$ is immediate: $\prod_{j=1}^{1} \sum_{i=1}^{m} a_{ij} = \sum_{i=1}^{m} a_{i1}$ matches $\sum_{1 \le i_1 \le m} a_{i_1 1}$.

Inductive step: Assume the identity holds for n. For $n+1$,

$$\begin{aligned}
\prod_{j=1}^{n+1} \left(\sum_{i=1}^{m} a_{ij} \right) &= \left(\prod_{j=1}^{n} \sum_{i=1}^{m} a_{ij} \right) \left(\sum_{i=1}^{m} a_{i,n+1} \right) \\
&= \left(\sum_{1 \le i_1, \dots, i_n \le m} \prod_{j=1}^{n} a_{i_j j} \right) \left(\sum_{i_{n+1}=1}^{m} a_{i_{n+1}, n+1} \right) \\
&= \sum_{1 \le i_1, \dots, i_{n+1} \le m} \prod_{j=1}^{n+1} a_{i_j j},
\end{aligned}$$

using distributivity once more. Thus the statement holds for all n.

Combinatorial viewpoint (why it's "obvious")

Think of choosing one term from each factor $\sum_{i=1}^{m} a_{ij}$. A complete choice is exactly an n-tuple $(i_1, \dots, i_n)$, contributing the product $a_{i_1 1} \cdots a_{i_n n}$. Summing over all choices (left side expanded) equals summing those products over all m^n tuples (right side).

Small sanity check (example)

Let $m = 2, n = 3$. Then

$$(a_{11} + a_{21})(a_{12} + a_{22})(a_{13} + a_{23})$$

expands to the eight terms

$$a_{11}a_{12}a_{13} + a_{11}a_{12}a_{23} + a_{11}a_{22}a_{13} + a_{11}a_{22}a_{23} + a_{21}a_{12}a_{13} + a_{21}a_{12}a_{23} + a_{21}a_{22}a_{13} + a_{21}a_{22}a_{23},$$

which are precisely $\sum_{(i_1, i_2, i_3) \in \{1,2\}^3} a_{i_1 1} a_{i_2 2} a_{i_3 3}$.

Notes and useful generalizations

1. Different upper bounds per column. For integers $m_j \ge 1$,

$$\prod_{j=1}^{n}\left(\sum_{i=1}^{m_j} a_{ij}\right) = \sum_{1\le i_j\le m_j\ \forall j}\ \prod_{j=1}^{n} a_{i_j\,j}.$$

2. Knuth's conditional-sum notation. In §1.2.3, sums/products over integer indices subject to relations are written $\sum_{R(k)}$ and $\prod_{R(k)}$; the right-hand side is a multi-index sum over the relation "$1 \le i_j \le m$ for all j."
3. Infinite versions require convergence assumptions; with finite m, n no issues arise.

33 [M30]

We'll prove a clean general law using **Lagrange interpolation**.

1) Lagrange basis and a key coefficient identity

Let

$$w(t) = \prod_{k=1}^{n}(t - x_k) \qquad \text{and} \qquad L_j(t) = \frac{\prod_{k\ne j}(t - x_k)}{\prod_{k\ne j}(x_j - x_k)}.$$

The polynomials L_j are the **Lagrange basis**: $L_j(x_i) = \delta_{ij}$. Hence every polynomial p of degree $\le n-1$ satisfies the interpolation formula

$$p(t) = \sum_{j=1}^{n} p(x_j)\, L_j(t).$$

Look at the coefficient of t^{n-1} on both sides. Since

$$L_j(t) = \frac{t^{n-1} + (\text{lower degree terms})}{\prod_{k\ne j}(x_j - x_k)},$$

the coefficient of t^{n-1} in L_j is exactly $\dfrac{1}{\prod_{k\ne j}(x_j - x_k)}$. Therefore:

Lemma. For any polynomial p of degree $\le n-1$,

$$[t^{n-1}]\, p(t) \;=\; \sum_{j=1}^{n} \frac{p(x_j)}{\prod_{k\ne j}(x_j - x_k)}. \qquad (*)$$

This one-line identity is the engine of the whole exercise.

2) Apply the lemma to $p(t) = t^r$

Case A: $0 \le r \le n-2$

Here $\deg p = r < n-1$, so the coefficient of t^{n-1} in $p(t) = t^r$ is 0. By $(*)$,

$$\sum_{j=1}^{n} \frac{x_j^r}{\prod_{k \neq j}(x_j - x_k)} = 0.$$

Case B: $r = n-1$

Now $[t^{n-1}]\, t^{n-1} = 1$. Applying $(*)$,

$$\sum_{j=1}^{n} \frac{x_j^{n-1}}{\prod_{k \neq j}(x_j - x_k)} = 1.$$

So far we've settled all $r \le n-1$.

3) Handle $r = n$

When $r = n$, $\deg t^n = n$ so we can't apply $(*)$ directly. Write t^n as

$$t^n = w(t) + R(t),$$

where $R(t)$ is the remainder of the division of t^n by $w(t)$. Then $\deg R \le n-1$ and, crucially, $R(x_j) = t^n\big|_{t=x_j} = x_j^n$ for each j.

Therefore, applying $(*)$ to R,

$$\sum_{j=1}^{n} \frac{x_j^n}{\prod_{k \neq j}(x_j - x_k)} = \sum_{j=1}^{n} \frac{R(x_j)}{\prod_{k \neq j}(x_j - x_k)} = [t^{n-1}]\, R(t).$$

But we also know the explicit leading terms of $w(t)$ by Viète's formulas:

$$w(t) = \prod_{k=1}^{n}(t - x_k) = t^n - \Big(\sum_{k=1}^{n} x_k\Big)\, t^{n-1} + \cdots.$$

Hence

$$R(t) = t^n - w(t) = \Big(\sum_{k=1}^{n} x_k\Big)\, t^{n-1} + \cdots,$$

so $[t^{n-1}]\,R(t) = \sum_{k=1}^{n} x_k$. Therefore

$$\sum_{j=1}^{n} \frac{x_j^n}{\prod_{k\neq j}(x_j - x_k)} = \sum_{j=1}^{n} x_j,$$

as required.

This proves all three cases.

4) Checks against the $n = 3$ "Dr. Matrix" identities

For $n = 3$, the denominators are $(a-b)(a-c)$, $(b-a)(b-c)$, $(c-a)(c-b)$. Our general result gives:

- $r = 0, 1$: sums equal 0.
- $r = 2$: sum equals 1.
- $r = 3$: sum equals $a + b + c$.

These match the four displayed identities in the problem statement.

34 [M25]

Write the denominator in a closed form. For fixed k,

$$\prod_{\substack{1\le r\le n\\ r\neq k}} (k-r) = \Big(\prod_{r=1}^{k-1}(k-r)\Big)\Big(\prod_{r=k+1}^{n}(k-r)\Big) = (k-1)!\,(-1)^{\,n-k}(n-k)!.$$

Hence

$$\frac{1}{\prod_{r\neq k}(k-r)} = \frac{(-1)^{\,n-k}}{(k-1)!(n-k)!} = \frac{1}{(n-1)!}\,(-1)^{\,n-k}\binom{n-1}{k-1}.$$

Define the polynomial in k (with x, m treated as parameters)

$$p(k) \;:=\; \prod_{\substack{1\le r\le n\\ r\neq m}} (x + k - r).$$

This is a monic polynomial of degree $n-1$ in k.

With the weight above, the left-hand side becomes

$$S(x) = \sum_{k=1}^{n} \frac{p(k)}{\prod_{r \neq k}(k-r)} = \frac{1}{(n-1)!} \sum_{k=1}^{n} (-1)^{n-k} \binom{n-1}{k-1} p(k).$$

But the binomial-weighted sum is exactly a forward difference:

$$\Delta^{n-1} p(1) = \sum_{j=0}^{n-1} (-1)^{n-1-j} \binom{n-1}{j} p(1+j) = \sum_{k=1}^{n} (-1)^{n-k} \binom{n-1}{k-1} p(k).$$

Therefore

$$S(x) = \frac{\Delta^{n-1} p(1)}{(n-1)!}.$$

Finally, since $p(k)$ is **monic of degree** $n-1$, its $(n-1)$-st forward difference is the constant $(n-1)!$ (discrete analogue of "the $(n-1)$-st derivative of a monic degree $n-1$ polynomial is $(n-1)!$"). Hence

$$S(x) = \frac{(n-1)!}{(n-1)!} = 1,$$

which proves the identity.

35 [HM20]

Knuth introduces the supremum notation $\sup\limits_{R(j)} a_j$ (least upper bound of the a_j with $R(j)$ true) and asks you to adapt the four basic sum-manipulation rules (a)–(d) from §1.2.3 to this notation. In particular, discuss the analogue of rule (a)

$$(\sup_{R(i)} a_i) + (\sup_{S(j)} b_j) = \sup_{R(i)} \sup_{S(j)} (a_i + b_j),$$

and specify what $\sup_{R(j)} a_j$ should mean when no j satisfies R.

Throughout, it is convenient to work in the **extended real line** $\overline{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\}$.

1) Definition for the empty index set

To keep all identities well-behaved (especially the "union" rule below), define

$$\sup_{R(j)} a_j \;=\; -\infty \quad \text{if no } j \text{ satisfies } R(j).$$

This mirrors the way $\sum_{R(j)} a_j$ is defined to be 0 when the set is empty; for sups, $-\infty$ is the neutral element for max. With this choice, every rule below continues to hold even in degenerate cases. (Knuth's official answer also adopts $-\infty$.)

2) Analogue of rule (a): "distributive law" over addition

For sums, rule (a) states $(\sum_{R(i)} a_i)(\sum_{S(j)} b_j) = \sum_{R(i)} \sum_{S(j)} a_i b_j$ (Eq. (4)). For suprema, the correct additive analogue is

$$\boxed{\big(\sup_{R(i)} a_i\big) + \big(\sup_{S(j)} b_j\big) \;=\; \sup_{R(i)} \sup_{S(j)} (a_i + b_j)} \tag{A}$$

Proof. Let $A = \{a_i : R(i)\}$ and $B = \{b_j : S(j)\}$.

- For any i, j, $a_i \le \sup A$ and $b_j \le \sup B$, hence $a_i + b_j \le \sup A + \sup B$. Taking the supremum over all i, j gives $\sup_{i,j}(a_i + b_j) \le \sup A + \sup B$.
- Conversely, fix $\varepsilon > 0$ and pick i_0 with $a_{i_0} > \sup A - \varepsilon/2$ and j_0 with $b_{j_0} > \sup B - \varepsilon/2$. Then $a_{i_0} + b_{j_0} > \sup A + \sup B - \varepsilon$, so $\sup_{i,j}(a_i + b_j) \ge \sup A + \sup B - \varepsilon$. Let $\varepsilon \to 0$.

If $\sup A = +\infty$ or $\sup B = +\infty$, both sides are $+\infty$; if one set is empty, $\sup A = -\infty$ etc., and both sides evaluate to $-\infty$ in $\overline{\mathbb{R}}$. For **finite** sets, (A) becomes the trivial identity $a + \max(b, c) = \max(a + b, a + c)$. (Knuth hints at exactly this reduction.)

A closely related multiplicative variant (useful later) also holds:

$$\text{If all } a_i, b_j \ge 0, \text{ then } \big(\sup_{R(i)} a_i\big)\big(\sup_{S(j)} b_j\big) \;=\; \sup_{R(i)} \sup_{S(j)} (a_i b_j).$$

(Again, prove both inequalities; nonnegativity is essential.) This is the supremum analogue of the product-of-sums law (Eq. (4)).

3) Analogue of rule (b): change of variable

For sums, rule (b) allows $\sum_{R(i)} a_i = \sum_{R(j)} a_j = \sum_{R(p(j))} a_{p(j)}$ when p is a permutation of the relevant indices (Eq. (5)). For sups:

$$\boxed{\sup_{R(i)} a_i \;=\; \sup_{R(j)} a_j \;=\; \sup_{R(p(j))} a_{p(j)}} \tag{B}$$

provided p is a **bijection** on the index set $\{i : R(i)\}$. The proof is immediate because the set of values being "supremed over" is unchanged under such a relabeling. (If p misses some relevant i, you only get "$\leq$".)

4) Analogue of rule (c): interchanging order

For sums, rule (c) says $\sum_{R(i)} \sum_{S(j)} a_{ij} = \sum_{S(j)} \sum_{R(i)} a_{ij}$ (Eq. (7), and the more general Eq. (9)). For sups:

$$\boxed{\sup_{R(i)} \sup_{S(j)} c_{ij} \;=\; \sup_{S(j)} \sup_{R(i)} c_{ij} \;=\; \sup_{\substack{R(i)\\ S(j)}} c_{ij}} \tag{C}$$

because all three expressions are the least upper bound of the same set $\{c_{ij} : R(i) \wedge S(j)\}$. (No absolute-convergence caveats are needed for sups.)

5) Analogue of rule (d): manipulating the domain (union/intersection)

For sums, rule (d) is the inclusion–exclusion style identity

$$\sum_{R(j)} a_j + \sum_{S(j)} a_j = \sum_{R(j)\,\text{or}\,S(j)} a_j + \sum_{R(j)\,\text{and}\,S(j)} a_j \quad \text{(Eq. (11))}.$$

For **suprema**, double-counting is irrelevant, and the rule simplifies to

$$\boxed{\sup_{R(j)\,\text{or}\,S(j)} a_j \;=\; \max\Big(\sup_{R(j)} a_j,\; \sup_{S(j)} a_j\Big).} \tag{D}$$

This is simply the fact that the supremum over a union of sets equals the maximum of the individual suprema; with our convention $\sup \emptyset = -\infty$, it holds even when one condition selects nothing. (Knuth's answer states exactly this simplification.)

36 [M23]

Solution 1 — Elementary row/column operations (no eigenvalues)

Write the rows of A as

$$R_1 = [x{+}y,\, y,\, \dots,\, y], \qquad R_i = [y,\, \dots,\, \underset{i}{x{+}y},\, \dots,\, y] \;(i \geq 2).$$

1. **Make the matrix sparse.** For each $i = 2, \dots, n$ replace $R_i \leftarrow R_i - R_1$. This preserves the determinant.

- In the new rows $i \geq 2$:
  - Column 1: $y - (x + y) = -x$.
  - Column i: $(x + y) - y = x$.
  - Other columns: $y - y = 0$.

So each R_i (for $i \geq 2$) now has exactly two nonzeros: $-x$ in column 1 and x in column i.

2. **Factor x from rows $2, \ldots, n$.** Pulling a common factor x from each of those $n - 1$ rows multiplies the determinant by x^{n-1}. After factoring, replace each such row by $[-1 \;\; 0 \;\; \ldots \;\; 1 \text{ at col } i \;\; \ldots \;\; 0]$.

At this point the matrix has the form

$$B = \begin{bmatrix} x+y & y & y & \cdots & y \\ -1 & 1 & 0 & \cdots & 0 \\ -1 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ -1 & 0 & 0 & \cdots & 1 \end{bmatrix}, \qquad \det A = x^{n-1} \det B.$$

3. **Column sweep.** Add columns $2, 3, \ldots, n$ into column 1 (an operation that does **not** change the determinant):

$$C_1 \leftarrow C_1 + \sum_{j=2}^{n} C_j.$$

- First-row, first-column entry becomes $(x + y) + (n - 1)y = x + ny$.
- For each $i \geq 2$, the first-column entry -1 is increased by the 1 sitting in column i of row i, giving 0.

Thus

$$B' = \begin{bmatrix} x+ny & y & y & \cdots & y \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}.$$

Now B' is block–upper–triangular with the lower-right block I_{n-1}. Hence

$$\det B' = (x + ny) \cdot \det(I_{n-1}) = x + ny.$$

Combining with step 2:

$$\boxed{\det A = x^{n-1}(x+ny)}.$$

Edge cases.

- $n = 1$: $\det[x+y] = x+y$, and the formula gives $x^0(x + 1 \cdot y) = x + y$.
- $x = 0$: $A = y\mathbf{1}\mathbf{1}^\top$ has rank 1 for $n > 1$, so $\det A = 0$; the formula gives $0^{n-1}(0 + ny) = 0$ (for $n > 1$), consistent.

Solution 2 — One-line spectral proof

Write $A = xI + yJ$, where $J = \mathbf{1}\mathbf{1}^\top$ is the all-ones matrix. J has eigenvalue n on $\mathbf{1}$ and 0 on the $(n-1)$-dimensional subspace orthogonal to $\mathbf{1}$. Therefore the eigenvalues of A are

$$x + ny \quad \text{(once)}, \qquad x \quad \text{(multiplicity } n-1\text{)}.$$

Thus $\det A$ (product of eigenvalues) equals $x^{n-1}(x + ny)$ as claimed.

37 [M24]

The exercise defines the Vandermonde matrix by $a_{ij} = x_j^i$ (so the first row is $x_1, \dots, x_n$; the second row $x_1^2, \dots, x_n^2$; …; the nth row $x_1^n, \dots, x_n^n$) and asks you to show that

$$\det(a_{ij}) = \Big(\prod_{j=1}^{n} x_j\Big) \prod_{1 \le i < j \le n} (x_j - x_i).$$

Step 0: Reduce to the "classical" Vandermonde

Factor x_j out of column j for each j. Determinants scale by a column factor, so

$$\det[x_j^i]_{i,j=1}^n = \Big(\prod_{j=1}^{n} x_j\Big) \det[x_j^{i-1}]_{i,j=1}^n.$$

Thus it suffices to prove the classical Vandermonde formula

$$\Delta_n(x_1, \dots, x_n) \;:=\; \det[x_j^{i-1}]_{i,j=1}^n \;=\; \prod_{1 \le i < j \le n} (x_j - x_i). \qquad (\ )$$

Step 1: Structure of Δ_n

Two immediate properties of Δ_n:

1. Alternation. Swapping x_p and x_q swaps two columns and flips the sign of the determinant; hence Δ_n is alternating in the x_j's.
2. Vanishing on collisions. If $x_p = x_q$ with $p \neq q$, columns p and q coincide, so $\Delta_n = 0$. Therefore each factor $(x_q - x_p)$ must divide Δ_n as a polynomial.

By unique factorization in $\mathbb{R}[x_1, \dots, x_n]$ (or $\mathbb{C}[x_1, \dots, x_n]$), these two facts together imply that there exists a constant C_n (depending only on n) with

$$\Delta_n(x_1, \dots, x_n) = C_n \prod_{1 \le i < j \le n} (x_j - x_i). \tag{1}$$

Step 2: Determine C_n by leading-term comparison

View Δ_n as a polynomial in the single variable x_n (with parameters $x_1, \dots, x_{n-1}$). Expanding $\det[x_j^{i-1}]$ along the last column shows that the coefficient of x_n^{n-1} is exactly $\Delta_{n-1}(x_1, \dots, x_{n-1})$: only the entry x_n^{n-1} in the bottom row contributes to that top degree, and its cofactor is the $(n-1) \times (n-1)$ classical Vandermonde on the first $n-1$ variables.

On the right side of (1), as a polynomial in x_n,

$$\prod_{1 \le i < j \le n} (x_j - x_i) = \Big(\prod_{1 \le i < j \le n-1} (x_j - x_i) \Big) \cdot \prod_{k=1}^{n-1} (x_n - x_k),$$

whose coefficient of x_n^{n-1} is $\prod_{1 \le i < j \le n-1} (x_j - x_i) = \Delta_{n-1}(x_1, \dots, x_{n-1})$ by the induction hypothesis.

Comparing the top-degree coefficients on both sides of (1) gives

$$\Delta_{n-1}(x_1, \dots, x_{n-1}) \;=\; C_n \, \Delta_{n-1}(x_1, \dots, x_{n-1}),$$

hence $C_n = 1$. (The base $n = 1$ is $\Delta_1 = 1$, and $n = 2$ gives $\Delta_2 = x_2 - x_1$, so the induction anchors are immediate.)

Therefore ( ) holds:

$$\Delta_n(x_1, \dots, x_n) = \prod_{1 \le i < j \le n} (x_j - x_i).$$

Step 3: Restore the factored x_j's

Returning to the original matrix with $a_{ij} = x_j^i$,

$$\det[a_{ij}] = \left(\prod_{j=1}^{n} x_j\right) \det[x_j^{i-1}] = \left(\prod_{j=1}^{n} x_j\right) \prod_{1\le i<j\le n} (x_j - x_i),$$

which is exactly what the exercise asked to show.

Quick checks (sanity)

- $n = 2$: $\det \begin{pmatrix} x_1 & x_2 \\ x_1^2 & x_2^2 \end{pmatrix} = x_1 x_2 (x_2 - x_1)$.
- $n = 3$: factoring $x_1 x_2 x_3$ from columns and using the classical 3×3 Vandermonde yields $x_1 x_2 x_3 (x_2 - x_1)(x_3 - x_1)(x_3 - x_2)$.

38 [M25]

We proceed by induction on n.

Base $n = 1$. $\det[1/(x_1 + y_1)] = 1/(x_1 + y_1)$. The right-hand side also equals $1/(x_1 + y_1)$ (empty products are 1).

Inductive step. Assume the formula for size $n - 1$. For the $n \times n$ matrix:

1. **Make columns $2, \dots, n$ differenced against column 1.** Replace $C_j \leftarrow C_j - C_1$ for $j = 2, \dots, n$. Entrywise,

$$\frac{1}{x_i + y_j} - \frac{1}{x_i + y_1} = \frac{y_1 - y_j}{(x_i + y_j)(x_i + y_1)}.$$

Factor $(y_1 - y_j)$ from each column $j \ge 2$, and then factor $(x_i + y_1)^{-1}$ from each row i. After these factorizations the matrix becomes

$$\begin{bmatrix} 1 & \dfrac{1}{x_1 + y_2} & \cdots & \dfrac{1}{x_1 + y_n} \\ 1 & \dfrac{1}{x_2 + y_2} & \cdots & \dfrac{1}{x_2 + y_n} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \dfrac{1}{x_n + y_2} & \cdots & \dfrac{1}{x_n + y_n} \end{bmatrix}$$

and we have pulled out the factor

$$\left(\prod_{j=2}^{n} (y_1 - y_j)\right) \left(\prod_{i=1}^{n} \frac{1}{x_i + y_1}\right).$$

(These are exactly the operations hinted in Knuth's answer: subtract column 1 from the others, factor the y-differences and the $(x_i + y_1)^{-1}$s.)

2. **Zero out the first column below the top.** Replace $R_i \leftarrow R_i - R_1$ for $i = 2, \dots, n$. In columns $j \geq 2$,

$$\frac{1}{x_i + y_j} - \frac{1}{x_1 + y_j} = \frac{x_1 - x_i}{(x_i + y_j)(x_1 + y_j)}.$$

Expand along the first column (which now has a 1 on top and zeros below). The remaining $(n-1) \times (n-1)$ block $B = (b_{ij})_{2 \leq i,j \leq n}$ has

$$b_{ij} = \frac{x_1 - x_i}{(x_i + y_j)(x_1 + y_j)}.$$

Factor $(x_1 - x_i)$ from each row $i = 2, \dots, n$ and $(x_1 + y_j)^{-1}$ from each column $j = 2, \dots, n$. The block reduces to $\big(1/(x_i + y_j)\big)_{2 \leq i,j \leq n}$, i.e. the same Cauchy form on the smaller sets $\{x_2, \dots, x_n\}$, $\{y_2, \dots, y_n\}$. The factor you have pulled out in this step is

$$\left(\prod_{i=2}^{n}(x_1 - x_i)\right)\left(\prod_{j=2}^{n}\frac{1}{x_1 + y_j}\right).$$

(These are the "subtract row 1 from the others; factor x-differences and $(x_1 + y_j)^{-1}$" steps; after them only a smaller Cauchy determinant remains.)

Putting the two stages together,

$$\det_n(\{x\}, \{y\}) = \frac{\prod_{i=2}^{n}(x_1 - x_i) \prod_{j=2}^{n}(y_1 - y_j)}{\left(\prod_{i=1}^{n}(x_i + y_1)\right)\left(\prod_{j=2}^{n}(x_1 + y_j)\right)} \det_{n-1}(\{x_2, \dots, x_n\}, \{y_2, \dots, y_n\}).$$

By the induction hypothesis,

$$\det_{n-1} = \frac{\prod_{2 \leq i < k \leq n}(x_i - x_k) \prod_{2 \leq j < \ell \leq n}(y_j - y_\ell)}{\prod_{i=2}^{n}\prod_{j=2}^{n}(x_i + y_j)}.$$

Multiplying the factors and simplifying yields

$$\det_n = \frac{\prod_{1\le i<k\le n}(x_i - x_k) \prod_{1\le j<\ell\le n}(y_j - y_\ell)}{\prod_{i=1}^{n}\prod_{j=1}^{n}(x_i + y_j)},$$

which is precisely the claimed formula. This completes the induction.

Quick check for $n = 2$

$$\det\begin{bmatrix} \frac{1}{x_1+y_1} & \frac{1}{x_1+y_2} \\ \frac{1}{x_2+y_1} & \frac{1}{x_2+y_2} \end{bmatrix} = \frac{(x_1 - x_2)(y_1 - y_2)}{(x_1 + y_1)(x_1 + y_2)(x_2 + y_1)(x_2 + y_2)}.$$

Matches the formula.

39 [M23]

Here "combinatorial matrix" means the $n \times n$ matrix

$$A = (a_{ij}), \qquad a_{ij} = y + \delta_{ij}x,$$

i.e., every off–diagonal entry is y and every diagonal entry is $x + y$. Equivalently

$$A = xI + yJ,$$

where I is the identity and J is the all-ones matrix. (Definition and exercise are in §1.2.3.)

We will show that A^{-1} also has the "combinatorial" form $B = (b_{ij})$ with

$$b_{ij} = \frac{-y + \delta_{ij}(x + ny)}{x(x + ny)}.$$

(That is exactly what the exercise asks you to prove.)

Two basic facts about J:

1. J commutes with I (trivial) and with any polynomial in J.
2. $J^2 = nJ$ (since each entry of J^2 is the sum of n ones).

Because $A = xI + yJ$ lies in the 2-dimensional commutative algebra $\{aI + bJ\}$ closed under multiplication, its inverse—if it exists—must also lie in that algebra. So make the ansatz

$$A^{-1} = aI + bJ$$

for scalars a, b. Multiply and use $J^2 = nJ$:

$$(xI + yJ)(aI + bJ) = xa\, I + (xb + ya + nyb)\, J.$$

For this to be the identity I, we must have the coefficient of I equal to 1 and the coefficient of J equal to 0:

$$xa = 1, \qquad xb + ya + nyb = 0.$$

Solve:

$$a = \frac{1}{x}, \qquad b = -\frac{ya}{x + ny} = -\frac{y}{x(x + ny)}.$$

Hence

$$A^{-1} = \frac{1}{x}\, I - \frac{y}{x(x + ny)}\, J.$$

Reading off entries (remember J has all ones),

$$b_{ij} = -\frac{y}{x(x + ny)} + \delta_{ij}\,\frac{1}{x} = \frac{-y + \delta_{ij}(x + ny)}{x(x + ny)}.$$

Thus A^{-1} has constant off-diagonal entries $-y/\{x(x + ny)\}$ and constant diagonal entries $\{-y + (x + ny)\}/\{x(x + ny)\} = (x + (n - 1)y)/\{x(x + ny)\}$, i.e., it is again a combinatorial matrix with the stated formula.

Remarks

- Invertibility requires $x \neq 0$ and $x + ny \neq 0$. This matches the eigenvalue view: A has eigenvalue x on the $(n - 1)$-dimensional subspace orthogonal to $\mathbf{1}$, and eigenvalue $x + ny$ on span$\{\mathbf{1}\}$.
- A compact way to remember the result for any $A = \alpha I + \beta J$ is

$$(\alpha I + \beta J)^{-1} = \frac{1}{\alpha} I - \frac{\beta}{\alpha(\alpha + n\beta)} J,$$

from which the exercise follows by $\alpha = x,\ \beta = y$.

40 [M24]

View each row i of B as the coefficients of a polynomial

$$p_i(x) \;=\; \sum_{t=1}^{n} b_{it}\, x^t.$$

The condition $BA = I$ is exactly

$$\sum_{t=1}^{n} b_{it}\, x_j^t \;=\; \delta_{ij} \qquad (1 \le i, j \le n).$$

So we want a degree–$\le n$ polynomial p_i that satisfies $p_i(x_j) = \delta_{ij}$ at the n distinct nodes x_j. By Lagrange interpolation, the unique choice is

$$p_i(x) \;=\; \frac{x}{x_i} L_i(x), \qquad L_i(x) \;=\; \prod_{\substack{1 \le k \le n \\ k \ne i}} \frac{x - x_k}{x_i - x_k}.$$

Indeed, $p_i(x_j) = 0$ if $j \ne i$ (since a factor $x_j - x_k$ vanishes), and $p_i(x_i) = (x_i/x_i) \cdot 1 = 1$ as required.

Therefore the entries of B are *the coefficients of* $x/x_i \cdot L_i(x)$:

$$\boxed{b_{it} = [x^t]\left(\frac{x}{x_i} \prod_{\substack{k=1 \\ k \ne i}}^{n} \frac{x - x_k}{x_i - x_k}\right)} \qquad (t = 1, \dots, n).$$

Closed form using elementary symmetric polynomials

Write

$$\prod_{\substack{k=1 \\ k \ne i}}^{n} (x - x_k) = \sum_{m=0}^{n-1} (-1)^{n-1-m}\, e_{n-1-m}^{(i)}\, x^m,$$

where $e_r^{(i)}$ is the r-th elementary symmetric polynomial in the $n-1$ variables $\{x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n\}$, with $e_0^{(i)} = 1$. Then

$$p_i(x) = \frac{x}{x_i \prod_{k \neq i}(x_i - x_k)} \sum_{m=0}^{n-1} (-1)^{n-1-m} e_{n-1-m}^{(i)} x^m,$$

and the coefficient of x^t (for $t = 1, \dots, n$) is

$$\boxed{b_{it} = \frac{(-1)^{n-t}\, e_{n-t}^{(i)}}{x_i \prod\limits_{k \neq i}(x_i - x_k)}}.$$

41 [M26]

Let $A = (a_{ij})_{1 \le i,j \le n}$ with

$$a_{ij} = \frac{1}{x_i + y_j}.$$

Define $B = (b_{tj})$ by

$$\boxed{b_{tj} := \frac{\left(\prod_{k=1}^{n}(x_k + y_t)\right)\left(\prod_{k \neq t}(x_j + y_k)\right)}{\left(\prod_{k \neq j}(x_j - x_k)\right)\left(\prod_{k \neq t}(y_t - y_k)\right)}}.$$

We will prove that $AB = I$, i.e.

$$\sum_{t=1}^{n} \frac{1}{x_i + y_t}\, b_{tj} = \delta_{ij} \qquad (1 \le i, j \le n).$$

Proof by Lagrange–interpolation trick

Fix i, j and consider the function of a dummy variable x

$$F_{ij}(x) := \sum_{t=1}^{n} \frac{\prod_{k \neq t}(x_j + y_k - x)\ \prod_{k \neq i}(x_k + y_t)}{\prod_{k \neq j}(x_j - x_k)\ \prod_{k \neq t}(y_t - y_k)}.$$

Observe:

1. **Degree bound.** The factor $\prod_{k\neq t}(x_j + y_k - x)$ has degree $n-1$ in x; everything else is independent of x. Hence $F_{ij}(x)$ is a polynomial in x of degree $\leq n-1$.
2. **Values at n distinct points.** For each $s \in \{1, \dots, n\}$, plug $x = x_j + y_s$. Then every summand in $F_{ij}(x)$ vanishes except the one with $t = s$, because $\prod_{k\neq t}(x_j + y_k - (x_j + y_s))$ is zero unless $t = s$. For that surviving term we get

$$F_{ij}(x_j + y_s) = \frac{\prod_{k\neq i}(x_k + y_s)}{\prod_{k\neq j}(x_j - x_k)} = \left.\frac{\prod_{k\neq j}(x_j - x_k - (x))}{\prod_{k\neq j}(x_j - x_k)}\right|_{x=x_j+y_s-x_j}$$

(the last rewrite is only to highlight the polynomial nature used below).

3. **Equality of polynomials.** Two polynomials of degree $\leq n-1$ that agree at the n distinct points $x = x_j + y_s$ must be identical. Hence the value at $x = 0$ is

$$F_{ij}(0) = \left.\frac{\prod_{k\neq i}(x_k + y_0)}{\prod_{k\neq j}(x_j - x_k)}\right|_{y_0=0} = \frac{\prod_{k\neq i}(x_j - x_k)}{\prod_{k\neq j}(x_j - x_k)} = \delta_{ij},$$

since the numerator contains the factor $x_j - x_j = 0$ when $i \neq j$, and is identical to the denominator when $i = j$.

Finally, notice that

$$\frac{1}{x_i + y_t}\, b_{tj} = \frac{\prod_{k\neq t}(x_j + y_k)\ \prod_{k\neq i}(x_k + y_t)}{\prod_{k\neq j}(x_j - x_k)\ \prod_{k\neq t}(y_t - y_k)} = F_{ij}(0) \quad \text{term-by-term},$$

so summing over t gives $\sum_t a_{it} b_{tj} = F_{ij}(0) = \delta_{ij}$. Therefore $AB = I$ and B is indeed A^{-1}. This is exactly the identity Knuth hints at and then proves directly via the same interpolation idea.

42 [M18]

Let I be the $n \times n$ identity matrix and J the $n \times n$ all-ones matrix. For scalars x, y with $x \neq 0$ and $x + ny \neq 0$, find the **sum of all entries** of $(xI + yJ)^{-1}$.

Solution 1 — Eigenvector (one-liner once you see it)

Observe that the all-ones vector $\mathbf{1} = (1, 1, \dots, 1)^{\mathsf{T}}$ is an eigenvector of J with eigenvalue n, hence of $xI + yJ$ with eigenvalue

$$(xI + yJ)\mathbf{1} = x\mathbf{1} + y\,J\mathbf{1} = x\mathbf{1} + y(n\mathbf{1}) = (x + ny)\mathbf{1}.$$

Therefore

$$(xI + yJ)^{-1}\mathbf{1} = \frac{1}{x + ny}\,\mathbf{1}.$$

The **sum of the entries in row** i **of** $(xI+yJ)^{-1}$ is exactly the i-th component of $(xI+yJ)^{-1}\mathbf{1}$, i.e. $1/(x + ny)$. Summing those row-sums over all n rows gives

$$\boxed{\sum_{i,j} \left((xI + yJ)^{-1}\right)_{ij} \;=\; \frac{n}{x + ny}}.$$

This matches the book's answer.

Solution 2 — Sherman–Morrison / explicit inverse (and cross-check)

Use the rank-one inverse identity with $A = xI$, $u = v = \sqrt{y}\,\mathbf{1}$:

$$(A + uv^{\mathsf{T}})^{-1} = A^{-1} - \frac{A^{-1}u\,v^{\mathsf{T}}A^{-1}}{1 + v^{\mathsf{T}}A^{-1}u} \;=\; \frac{1}{x}I - \frac{y}{x^2}\cdot\frac{\mathbf{1}\mathbf{1}^{\mathsf{T}}}{1 + \frac{y}{x}\,\mathbf{1}^{\mathsf{T}}\mathbf{1}} \;=\; \frac{1}{x}I - \frac{y}{x(x + ny)}\,\mathbf{1}\mathbf{1}^{\mathsf{T}}.$$

Summing all entries:

$$\sum_{i,j}\left(\frac{1}{x}I\right)_{ij} = \frac{n}{x}, \qquad \sum_{i,j}\left(\frac{y}{x(x + ny)}\,\mathbf{1}\mathbf{1}^{\mathsf{T}}\right)_{ij} = \frac{y}{x(x + ny)}\,n^2,$$

so

$$\sum_{i,j}(xI + yJ)^{-1}_{ij} = \frac{n}{x} - \frac{yn^2}{x(x + ny)} = \frac{n}{x + ny}.$$

(As a related identity appearing in the exercise cluster, Knuth notes $(xI+yJ)((x+ny)I-yJ) = x(x + ny)I$, from which the inverse formula is immediate.)

Checks and conditions

- **Invertibility**: requires $x \neq 0$ and $x + ny \neq 0$ (these are exactly the eigenvalues).
- **Sanity checks**: • If $y = 0$, matrix is $xI \Rightarrow$ inverse is $(1/x)I \Rightarrow$ sum $= n/x$, which equals $n/(x + 0 \cdot n)$. • If $n = 1$, matrix is $[x + y] \Rightarrow$ inverse has sum $= 1/(x + y) = n/(x + ny)$.

Thus the sum of all entries of $(xI + yJ)^{-1}$ is indeed $\boxed{n/(x + ny)}$.

43 [M24]

Let V be the $n \times n$ Vandermonde matrix

$$V = [x_j^i]_{1 \le i,j \le n} \quad \text{(so its rows are } (x_1, \dots, x_n), (x_1^2, \dots, x_n^2), \dots, (x_1^n, \dots, x_n^n)).$$

What is the sum of all n^2 entries of V^{-1}?

Step 1: A usable formula for V^{-1}

A convenient description (stated in Ex. 40) is

$$(V^{-1})_{ij} = \frac{[x^{j-1}] \prod_{k \ne i}(x_k - x)}{x_i \prod_{k \ne i}(x_k - x_i)}, \quad 1 \le i, j \le n,$$

i.e. the numerator is the coefficient of x^{j-1} in $\prod_{k \ne i}(x_k - x)$. (This is exactly the "coefficient of x^{j-1}" description given under Ex. 40.)

Step 2: Sum across each row

For fixed i, the row-sum is

$$R_i = \sum_{j=1}^{n} (V^{-1})_{ij} = \frac{\sum_{j=1}^{n} [x^{j-1}] \prod_{k \ne i}(x_k - x)}{x_i \prod_{k \ne i}(x_k - x_i)} = \frac{\prod_{k \ne i}(x_k - x)\Big|_{x=1}}{x_i \prod_{k \ne i}(x_k - x_i)} = \frac{\prod_{k \ne i}(x_k - 1)}{x_i \prod_{k \ne i}(x_k - x_i)}.$$

(The sum of coefficients of a polynomial equals its value at $x = 1$.)

Step 3: Sum all entries and simplify (Lagrange view)

Hence the total sum is

$$S = \sum_{i=1}^{n} R_i = \sum_{i=1}^{n} \frac{1}{x_i} \frac{\prod_{k \ne i}(x_k - 1)}{\prod_{k \ne i}(x_k - x_i)} = \sum_{i=1}^{n} \frac{1}{x_i} L_i(1),$$

where $L_i(t) = \prod_{k \ne i} \dfrac{t - x_k}{x_i - x_k}$ are the Lagrange basis polynomials.

Consider the degree-$\le n-1$ polynomial

$$g(t) = \sum_{i=1}^{n} \frac{1}{x_i} L_i(t).$$

By construction, $g(x_i) = \frac{1}{x_i}$ for all i. The polynomial

$$h(t) = \frac{1 - \prod_{k=1}^{n} \left(1 - \frac{t}{x_k}\right)}{t}$$

also has degree $\leq n-1$ and satisfies $h(x_i) = \frac{1}{x_i}$ (the product vanishes at $t = x_i$). Therefore $g \equiv h$ by uniqueness of Lagrange interpolation (cf. the Lagrange-sum identities highlighted in Ex. 33). Evaluating at $t = 1$ gives

$$S = g(1) = h(1) = 1 - \prod_{k=1}^{n} \left(1 - \frac{1}{x_k}\right) = 1 - \frac{\prod_{k=1}^{n}(x_k - 1)}{\prod_{k=1}^{n} x_k}.$$

(Ex. 33 sets up exactly these Lagrange-basis manipulations.)

Final result (two equivalent forms)

$$\boxed{\sum_{i=1}^{n}\sum_{j=1}^{n}(V^{-1})_{ij} = 1 - \prod_{k=1}^{n}\left(1 - \frac{1}{x_k}\right) = \sum_{r=1}^{n}(-1)^{r-1} \sum_{1 \leq i_1 < \cdots < i_r \leq n} \frac{1}{x_{i_1} \cdots x_{i_r}}.}$$

The first form is the clean product; the second is the alternating sum of the elementary symmetric functions in $1/x_k$.

Sanity check (small n)

For $n = 2$, $V = \begin{bmatrix} x_1 & x_2 \\ x_1^2 & x_2^2 \end{bmatrix}$. One computes

$$\sum (V^{-1})_{ij} = 1 - \left(1 - \tfrac{1}{x_1}\right)\left(1 - \tfrac{1}{x_2}\right) = \frac{1}{x_1} + \frac{1}{x_2} - \frac{1}{x_1 x_2},$$

matching the boxed formula.

44 [M26]

Let A be the $n \times n$ **Cauchy matrix**

$$A_{ij} = \frac{1}{x_i + y_j} \qquad (1 \le i, j \le n),$$

with all x_i distinct, all y_j distinct, and $x_i + y_j \neq 0$. Find the sum of all n^2 entries of A^{-1}.

Tooling you're allowed to use

Exercise 41 on the same page gives the entries of the inverse of a Cauchy matrix:

$$(A^{-1})_{ij} = \frac{\displaystyle\prod_{k=1}^{n}(x_j + y_k) \prod_{k=1}^{n}(x_k + y_i)}{(x_j + y_i) \displaystyle\prod_{k \neq j}(x_j - x_k) \prod_{k \neq i}(y_i - y_k)}. \qquad (\ )$$

We'll use this formula as a starting point.

Step 1: Separate variables

From ( ) define

$$\mu_j = \frac{\prod_{k=1}^{n}(x_j + y_k)}{\prod_{k \neq j}(x_j - x_k)}, \qquad \nu_i = \frac{\prod_{k=1}^{n}(x_k + y_i)}{\prod_{k \neq i}(y_i - y_k)}.$$

Then

$$(A^{-1})_{ij} = \frac{\mu_j \, \nu_i}{x_j + y_i}.$$

Hence the sum of all entries is

$$S = \sum_{i=1}^{n} \sum_{j=1}^{n} (A^{-1})_{ij} = \sum_{j=1}^{n} \mu_j \Big(\sum_{i=1}^{n} \frac{\nu_i}{x_j + y_i} \Big). \qquad (1)$$

Step 2: A barycentric/Lagrange identity

Set

$$\Phi_x(t) = \prod_{k=1}^{n}(t - x_k), \qquad \Phi_y(t) = \prod_{k=1}^{n}(t + y_k).$$

Consider the rational function

$$F(t) = \sum_{i=1}^{n} \frac{\nu_i}{t + y_i}.$$

Multiplying by $\Phi_y(t)$ gives a degree-$\leq n-1$ polynomial

$$\Phi_y(t)F(t) = \sum_{i=1}^{n} \nu_i \prod_{k \neq i} (t + y_k),$$

which is the Lagrange interpolation polynomial that equals $\Phi_x(-y_i)$ at the points $t = -y_i$. Because Φ_x and Φ_y are both monic of degree n, the remainder of $\Phi_x(t)$ on division by $\Phi_y(t)$ is $\Phi_x(t) - \Phi_y(t)$. Thus

$$\Phi_y(t)F(t) = \Phi_x(t) - \Phi_y(t) \quad \Rightarrow \quad F(t) = 1 - \frac{\Phi_x(t)}{\Phi_y(t)}.$$

Evaluating at $t = x_j$ gives $F(x_j) = 1$, since $\Phi_x(x_j) = 0$. Plugging into (1),

$$S = \sum_{j=1}^{n} \mu_j. \tag{2}$$

Step 3: Sum of the μ_j

By Lagrange's formula,

$$\mu_j = \frac{\Phi_y(x_j)}{\Phi_x'(x_j)}.$$

A standard coefficient trick now yields

$$\sum_{j=1}^{n} \mu_j = (\text{coefficient of } t^{n-1} \text{ in } \Phi_y(t) - \Phi_x(t)).$$

But

$$\Phi_y(t) = t^n + \Big(\sum_{k=1}^{n} y_k\Big) t^{n-1} + \cdots, \qquad \Phi_x(t) = t^n - \Big(\sum_{k=1}^{n} x_k\Big) t^{n-1} + \cdots,$$

so the t^{n-1}-coefficient of $\Phi_y - \Phi_x$ is

$$\sum_{k=1}^{n} y_k + \sum_{k=1}^{n} x_k.$$

Therefore

$$S = \sum_{i,j} (A^{-1})_{ij} = \sum_{k=1}^{n} x_k + \sum_{k=1}^{n} y_k.$$

Result (and a check)

$$\boxed{\sum_{i=1}^{n} \sum_{j=1}^{n} (A^{-1})_{ij} = \Big(\sum_{i=1}^{n} x_i\Big) + \Big(\sum_{j=1}^{n} y_j\Big).}$$

As a sanity check, Exercise 45 asks for the Hilbert case $A_{ij} = 1/(i+j-1)$, which corresponds to $x_i = i-1,\ y_j = j$. The formula gives

$$\sum (A^{-1})_{ij} = \sum_{i=1}^{n} (i-1) + \sum_{j=1}^{n} j = \frac{n(n-1)}{2} + \frac{n(n+1)}{2} = n^2,$$

matching the stated result.

45 [M25]

46 [M30]

For an $m \times n$ matrix A and an $n \times m$ matrix B,

$$\det(AB) = \sum_{1 \le j_1 < \cdots < j_m \le n} \det(A_{j_1 \cdots j_m})\ \det(B_{j_1 \cdots j_m}),$$

where $A_{j_1 \cdots j_m}$ is the $m \times m$ submatrix of A formed by columns $j_1, \dots, j_m$ and $B_{j_1 \cdots j_m}$ is the $m \times m$ submatrix of B formed by rows $j_1, \dots, j_m$. (See the statement in the book for the same notation and suggested special cases.)

Proof (by multilinearity and alternation of the determinant)

Let $C = AB$ (so C is $m \times m$). Denote the k-th column of A by $A^{(k)}$, and the r-th column of C by $C^{(r)}$. Because matrix multiplication forms columns as linear combinations of the columns of A,

$$C^{(r)} = \sum_{k=1}^{n} b_{kr}\, A^{(k)} \qquad (1 \le r \le m),$$

where b_{kr} are the entries of B.

Using that the determinant is multilinear in the columns,

$$\det(C) = \det(C^{(1)}, \dots, C^{(m)}) = \sum_{k_1,\dots,k_m=1}^{n} \Big(\prod_{r=1}^{m} b_{k_r r} \Big) \det(A^{(k_1)}, \dots, A^{(k_m)}). \tag{*}$$

1. **Only distinct index m-tuples contribute.** If any two of $k_1, \dots, k_m$ coincide, two columns in $\det\left(A^{(k_1)}, \dots, A^{(k_m)}\right)$ are equal, so that term is 0 by alternation. Hence we may restrict to m-tuples with *distinct* indices.

2. **Group by the unordered set $J = \{j_1 < \cdots < j_m\}$.** Fix such a set J. The surviving terms in $(*)$ that use exactly these indices are parameterized by permutations $\pi \in S_m$:

$$\det(A^{(j_{\pi(1)})}, \dots, A^{(j_{\pi(m)})}) \prod_{r=1}^{m} b_{j_{\pi(r)}\, r}.$$

By alternation,

$$\det(A^{(j_{\pi(1)})}, \dots, A^{(j_{\pi(m)})}) = \operatorname{sgn}(\pi)\, \det(A^{(j_1)}, \dots, A^{(j_m)}).$$

3. **Recognize a determinant of a submatrix of B.** Let $M(J)$ be the $m \times m$ matrix with entries $M(J)_{s,r} = b_{j_s\, r}$ (i.e., the submatrix of B consisting of rows $j_1, \dots, j_m$; this is exactly $B_{j_1 \cdots j_m}$). Then

$$\det\left(M(J)\right) = \sum_{\pi \in S_m} \operatorname{sgn}(\pi) \prod_{r=1}^{m} b_{j_{\pi(r)}\, r}.$$

Therefore the total contribution from the fixed set J equals

$$\det(A_{j_1 \cdots j_m}) \cdot \det(B_{j_1 \cdots j_m}).$$

4. **Sum over all J.** Summing these contributions over all $1 \le j_1 < \cdots < j_m \le n$ yields precisely the right-hand side of the identity, proving the formula.

□

Sanity checks / special cases

- $m = n$. There is only the single set $J = \{1, \dots, n\}$, so the identity reduces to $\det(AB) = \det(A)\det(B)$.
- $m = 1$. Then $\det(AB)$ equals the dot product of the $1 \times n$ row of A with the $n \times 1$ column of B; the right side is $\sum_{j=1}^{n} a_{1j} b_{j1}$.
- $B = A^{\mathsf{T}}$. We get $\det(AA^{\mathsf{T}}) = \sum_{J} \big(\det A_J\big)^2$, the classic Gram determinant identity.
- $m > n$. There are no m-subsets J of $\{1, \dots, n\}$, so the sum is empty (0); indeed $\operatorname{rank}(AB) \le n < m \Rightarrow \det(AB) = 0$.
- $m = 2$. The formula becomes $\det(AB) = \sum_{1 \le i < j \le n} \det(A^{(i)}, A^{(j)}) \det\big(\text{rows } i, j \text{ of } B\big)$, helpful when expanding by 2×2 minors.

47 [M27]

For an integer k, prove

$$\binom{r}{k}\binom{r-\frac{1}{2}}{k} = \frac{1}{4^k}\binom{2r}{k}\binom{2r-k}{k} = \frac{1}{4^k}\binom{2r}{2k}\binom{2k}{k},$$

and give a simpler closed form when $r = -\frac{1}{2}$.

We use the product (falling–factorial) definition of generalized binomial coefficients for real r and integer $k \ge 0$:

$$\binom{r}{k} = \frac{r(r-1)\cdots(r-k+1)}{k!}.$$

1. Multiply the two binomials and pair terms:

$$\binom{r}{k}\binom{r-\frac{1}{2}}{k} = \frac{\displaystyle\prod_{j=0}^{k-1}(r-j)(r-\tfrac{1}{2}-j)}{(k!)^2}.$$

For each j,

$$(r-j)\Big(r-\tfrac{1}{2}-j\Big) = \frac{(2r-2j)\big(2r-(2j+1)\big)}{4}.$$

Hence

$$\binom{r}{k}\binom{r-\frac{1}{2}}{k} = \frac{1}{4^k}\cdot\frac{\prod_{j=0}^{k-1}(2r-2j)(2r-(2j+1))}{(k!)^2}.$$

But

$$\prod_{j=0}^{k-1}(2r-2j)(2r-(2j+1)) = \prod_{m=0}^{2k-1}(2r-m) = (2r)^{\underline{2k}},$$

so

$$\binom{r}{k}\binom{r-\frac{1}{2}}{k} = \frac{1}{4^k}\cdot\frac{(2r)^{\underline{2k}}}{(k!)^2} = \frac{1}{4^k}\binom{2r}{2k}\cdot\frac{(2k)!}{(k!)^2} = \frac{1}{4^k}\binom{2r}{2k}\binom{2k}{k}.$$

This proves the right-hand equality.

2. Convert $\binom{2r}{2k}\binom{2k}{k}$ to $\binom{2r}{k}\binom{2r-k}{k}$ using the standard identity

$$\binom{r}{m}\binom{m}{k} = \binom{r}{k}\binom{r-k}{m-k} \qquad \text{(integers } m,k\text{)},$$

with $r \mapsto 2r$ and $m \mapsto 2k$. Thus

$$\binom{2r}{2k}\binom{2k}{k} = \binom{2r}{k}\binom{2r-k}{k},$$

giving the left-hand equality as well.

3. Special case $r = -\frac{1}{2}$. Plugging $r = -\frac{1}{2}$ into the proven formula,

$$\binom{-\frac{1}{2}}{k}\binom{-1}{k} = \frac{1}{4^k}\binom{-1}{2k}\binom{2k}{k}.$$

For integers $n \geq 0$, $\binom{-1}{n} = (-1)^n$. Hence $\binom{-1}{2k} = 1$ and $\binom{-1}{k} = (-1)^k$. Therefore

$$\binom{-\frac{1}{2}}{k}(-1)^k = \frac{1}{4^k}\binom{2k}{k} \quad\Longrightarrow\quad \boxed{\binom{-\frac{1}{2}}{k} = (-1)^k\,\frac{1}{4^k}\binom{2k}{k}}.$$

This is the desired "simpler formula" in the $r = -\frac{1}{2}$ case. (It is also the well-known central-binomial closed form used repeatedly later.)

1.2.4 Integer functions and elementary number theory

1 [00]

First, recall the definitions from §1.2.4: x is the greatest integer x (the "floor"), and x is the least integer x (the "ceiling").

1. 1.1 1.1 lies between 1 and 2, and the greatest integer 1.1 is 1 **1**.
2. -1.1 For negative numbers, "greatest integer x" moves to the left: integers -1.1 are …, -3, -2, -1, and the greatest among those -1.1 is -2 $\mathbf{-2}$.
3. -1.1 Least integer -1.1 is -1 $\mathbf{-1}$.
4. 0.99999 0 0.99999 < 1, so the greatest integer it is **0**.
5. lg 35 (lg = log base 2) 2 = 32 $<$ 35 64 = 2 , hence 5 $<$ lg 35 6, so lg 35 = **6**.

2 [01]

For any real x,

- x is the greatest integer x (the floor), and
- x is the least integer x (the ceiling).

Let n = x . By definition of floor, n is an integer with $n \leq x < n + 1$.

Because n itself is already an integer, taking the ceiling of n leaves it unchanged: $\lceil n \rceil = n$.

Therefore, $ $\boxed{\lceil \lfloor x \rfloor \rceil = \lfloor x \rfloor}$.$

Why this is always true

- Floor always returns an integer.
- Ceiling of any integer k equals k (since k is already the least integer k).
- Substituting k = x gives the identity above.

Quick checks

- If $x = 3.7$, $x = 3$ and $x = 3 = 3$.
- If $x = -2.3$, $x = -3$ and $x = -3 = -3$.
- If x is already an integer m, $x = m$ and $x = m = m$.

Thus the expression simplifies to the floor of x.

3 [M10]

First recall the definitions: x is the greatest integer x (the "floor"), and x is the least integer x (the "ceiling").

Two basic characterizations (we'll use them repeatedly)

Lemma F (floor characterization). For any real x and integer n, $x = n \quad n \quad x < n + 1$.
Proof. From the definition, $x \quad x$. If we had $x \quad x + 1$, then $x + 1$ would be an integer x, contradicting maximality of x. Hence $x < x + 1$. Conversely, if $n \quad x < n + 1$, no integer $n + 1$ is x; thus the greatest integer x is exactly n.

Lemma C (ceiling characterization). For any real x and integer n, $x = n \quad n - 1 < x \quad n$.
Proof. By definition, $x \quad x$. If we had $x \quad x - 1$, then $x - 1$ would be an integer x that is $< x$, contradicting minimality of x. Hence $x - 1 < x$. The converse is analogous to Lemma F.

(Equivalently, Lemma C is the same as Lemma F applied to $-x$, since $x = - -x$.)

Part (a) $x < n \quad x < n$

( ) If $x < n$, then by Lemma F we have $x < x + 1 \quad n$, hence $x < n$. ( ) If $x < n$, then every integer x is $< n$, so the greatest such integer x is $< n$.

Part (b) $n \quad x \quad n \quad x$

( ) If $n \quad x$, then $n \quad x$ because $x \quad x$. ( ) If $n \quad x$, then among the integers x is n itself, so the greatest integer x satisfies $x \quad n$.

Part (c) $x \quad n \quad x \quad n$

( ) If $x \quad n$, then $x \quad x \quad n$, hence $x \quad n$. ( ) If $x \quad n$, then n is an integer x, so the least integer x satisfies $x \quad n$.

Part (d) $n < x \quad n < x$

( ) If $n < x$, then (by Lemma C) $n < x$ is forced because $x > x - 1 \quad n$. ( ) If $n < x$, then any integer x is strictly greater than n; in particular the least such integer x satisfies $n < x$. (Equivalently, (d) is just the contrapositive of (c).)

Part (e) $x = n \quad (x - 1 < n \quad x) \quad (n \quad x < n + 1)$

The right-hand equivalence *is exactly* Lemma F. For the middle form, note that n x < n + 1 (subtract 1 from the right inequality) x − 1 < n x. Thus both formulations are equivalent and both characterize x = n.

Part (f) x = n (x n < x + 1) (n − 1 < x n)

This is Lemma C, plus the observation that n − 1 < x n (add 1 to the left inequality) x n < x + 1.

4 [M10]

Claim. For every real x,

$$\lfloor -x \rfloor \;=\; -\lceil x \rceil.$$

Let $n = \lceil x \rceil$. By the defining inequalities for the ceiling function,

$$n - 1 < x \leq n.$$

(These standard floor/ceiling characterizations are given just above and within §1.2.4.)

Multiply the inequalities by -1 (which reverses the direction):

$$-n \leq -x < 1 - n.$$

Set $m = -n$. Since m is an integer, the last display reads

$$m \leq -x < m + 1.$$

By the defining property of the floor function—$\lfloor y \rfloor = m$ iff $m \leq y < m + 1$—we conclude

$$\lfloor -x \rfloor = m = -n = -\lceil x \rceil.$$

This completes the proof.

5 [16]

Give a single simple formula for rounding a positive real number x to the nearest integer, with the rule:

- use $\lfloor x \rfloor$ when $x \bmod 1 < \frac{1}{2}$;
- use $\lceil x \rceil$ when $x \bmod 1 \geq \frac{1}{2}$. Then discuss what rounding your formula gives when x is negative. (Floor/ceiling and "mod" are defined earlier in §1.2.4.)

A single formula

$$\boxed{R(x) = \lfloor x + \tfrac{1}{2} \rfloor}$$

Why this works

Write $x = \lfloor x \rfloor + f$ where the fractional part $f = x \bmod 1$ satisfies $0 \leq f < 1$. (This is exactly Eq. (1): $x = \lfloor x \rfloor + (x \bmod 1)$.)

Then

$$\lfloor x + \tfrac{1}{2} \rfloor = \lfloor \lfloor x \rfloor + f + \tfrac{1}{2} \rfloor = \lfloor x \rfloor + \lfloor f + \tfrac{1}{2} \rfloor.$$

- If $f < \frac{1}{2}$, then $0 \leq f + \frac{1}{2} < 1$, so $\lfloor f + \frac{1}{2} \rfloor = 0$ and $R(x) = \lfloor x \rfloor$.
- If $f \geq \frac{1}{2}$, then $1 \leq f + \frac{1}{2} < \frac{3}{2}$, so $\lfloor f + \frac{1}{2} \rfloor = 1$ and $R(x) = \lfloor x \rfloor + 1 = \lceil x \rceil$ (since x isn't an integer in this case).

Thus the single formula $\lfloor x + \frac{1}{2} \rfloor$ implements exactly the stated rounding rule for positive x.

What happens for negative x?

Let $x < 0$ and write $x = -y$ with $y \geq 0$. Then

$$R(x) = \lfloor -y + \tfrac{1}{2} \rfloor = -\lceil y - \tfrac{1}{2} \rceil.$$

So ties at halves are pushed **toward** $+\infty$ (i.e., "half-up"): for example,

$$R(-1.3) = -1, \quad R(-1.5) = -1, \quad R(-1.7) = -2.$$

In words: the same formula $\lfloor x + \frac{1}{2} \rfloor$ rounds to the nearest integer with **half cases rounded upward** (toward $+\infty$). For positive numbers this is the usual "round half up"; for negative numbers it rounds half-cases **toward zero**.

(Optional) Symmetric alternative

If you wanted a "nearest with halves away from zero" rule, a symmetric one-liner is

$$\text{round}_{\text{away-from-0}}(x) = \begin{cases} \lfloor x + \frac{1}{2} \rfloor, & x \geq 0, \\ \lceil x - \frac{1}{2} \rceil, & x < 0, \end{cases}$$

but the exercise asks for the simplest single formula, which is $\boxed{\lfloor x + \frac{1}{2} \rfloor}$.

6 [20]

I'll prove (a) and (b) are always true, and (c) is not.

(a) $\lfloor \sqrt{\lfloor x \rfloor} \rfloor = \lfloor \sqrt{x} \rfloor$ — **True**

Let $n = \lfloor x \rfloor$. Then $n \leq x < n + 1$. Taking square roots gives

$$\sqrt{n} \;\leq\; \sqrt{x} \;<\; \sqrt{n+1}.$$

There is **no integer** strictly between $\sqrt{n}$ and $\sqrt{n+1}$: if an integer m satisfied $\sqrt{n} < m < \sqrt{n+1}$, then squaring would give $n < m^2 < n+1$, impossible since m^2 is an integer. Therefore $\lfloor \sqrt{x} \rfloor$ must equal $\lfloor \sqrt{n} \rfloor$, i.e. $\lfloor \sqrt{\lfloor x \rfloor} \rfloor$.

(b) $\lceil \sqrt{\lceil x \rceil} \rceil = \lceil \sqrt{x} \rceil$ — **True**

Let $c = \lceil x \rceil$. Then $c - 1 < x \leq c$, hence

$$\sqrt{c-1} \;<\; \sqrt{x} \;\leq\; \sqrt{c}.$$

For any integer m, $\lceil \sqrt{t} \rceil = m$ exactly on the interval $((m-1)^2,\, m^2]$. The short interval $(c-1, c]$ lies wholly inside a **single** such block (its length is 1, while consecutive squares are at least 1 apart and, except for 0 and 1, more than 1 apart). Thus $\lceil \sqrt{x} \rceil$ is constant on $(c-1, c]$ and equals $\lceil \sqrt{c} \rceil$, i.e. $\lceil \sqrt{\lceil x \rceil} \rceil$.

A more explicit framing: let $m = \lceil \sqrt{c} \rceil$, so $(m-1)^2 < c \leq m^2$. Then for all $x \in (c-1, c]$ we have $(m-1)^2 < x \leq m^2$, hence $\lceil \sqrt{x} \rceil = m = \lceil \sqrt{c} \rceil$.

(c) $\lceil \sqrt{\lfloor x \rfloor} \rceil = \lceil \sqrt{x} \rceil$ — **False**

Take any perfect square k^2 and choose $x = k^2 + \varepsilon$ with $0 < \varepsilon < 1$. Then $\lfloor x \rfloor = k^2$, so

$$\lceil \sqrt{\lfloor x \rfloor} \rceil = \lceil \sqrt{k^2} \rceil = k,$$

but $\sqrt{x} > k$ and $\lceil \sqrt{x} \rceil = k + 1$. Example: $x = 4.1$ gives $\lceil \sqrt{\lfloor 4.1 \rfloor} \rceil = \lceil \sqrt{4} \rceil = 2$ while $\lceil \sqrt{4.1} \rceil = 3$.

Answer: (a) True; (b) True; (c) False.

7 [M15]

Let the fractional part of a real number x be $\{x\} = x - \lfloor x \rfloor$. From the text we have

$$x = \lfloor x \rfloor + (x \bmod 1) = \lfloor x \rfloor + \{x\}, \quad \text{with } 0 \le \{x\} < 1,$$

so write $x = a + \alpha$ and $y = b + \beta$ where $a = \lfloor x \rfloor,\ b = \lfloor y \rfloor,\ \alpha = \{x\},\ \beta = \{y\} \in [0, 1)$.

1) Prove $\lfloor x \rfloor + \lfloor y \rfloor \le \lfloor x + y \rfloor$ and find when equality holds

Compute

$$x + y = (a + b) + (\alpha + \beta).$$

Because $0 \le \alpha, \beta < 1$, we have $0 \le \alpha + \beta < 2$. Thus

$$\lfloor x + y \rfloor = \begin{cases} a + b, & \text{if } \alpha + \beta < 1, \\ a + b + 1, & \text{if } \alpha + \beta \ge 1. \end{cases}$$

Therefore $\lfloor x \rfloor + \lfloor y \rfloor = a + b \le \lfloor x + y \rfloor$, with **equality iff** $\alpha + \beta < 1$, i.e. $\{x\} + \{y\} < 1$ (equivalently, $(x \bmod 1) + (y \bmod 1) < 1$).

2) "Similar formula" for ceilings

Using $\lceil t \rceil = -\lfloor -t \rfloor$, we get

$$\lceil x \rceil + \lceil y \rceil \ \ge\ \lceil x + y \rceil.$$

Equality cases come from the floor statement applied to $-x, -y$. Note that $\{-x\} = 0$ if $\{x\} = 0$, and $\{-x\} = 1 - \{x\}$ if $\{x\} > 0$. Thus

$$\{-x\} + \{-y\} < 1 \quad \Longleftrightarrow \quad \begin{cases} \text{either } \{x\} = 0 \text{ or } \{y\} = 0, \\ \text{or } \{x\} + \{y\} > 1. \end{cases}$$

Hence

$$\boxed{\lceil x \rceil + \lceil y \rceil \ge \lceil x + y \rceil, \text{ with equality iff } \{x\} = 0 \text{ or } \{y\} = 0 \text{ or } \{x\} + \{y\} > 1}.$$

Equivalently, the inequality is **strict** exactly when $0 < \{x\} < 1,\ 0 < \{y\} < 1$, and $\{x\} + \{y\} \le 1$.

Quick sanity checks

- $x = 2.3,\ y = 4.6:\ \{x\} + \{y\} = 0.9 < 1 \Rightarrow \lfloor x + y \rfloor = 6 = \lfloor x \rfloor + \lfloor y \rfloor.$
- $x = 2.7,\ y = 4.6:\ \{x\} + \{y\} = 1.3 > 1 \Rightarrow \lceil x + y \rceil = 8 = \lceil x \rceil + \lceil y \rceil.$

8 [00]

Definition we'll use. For real x and $y \neq 0$,

$$x \bmod y \;=\; x - y \left\lfloor \frac{x}{y} \right\rfloor,$$

and by convention $x \bmod 0 = x$. When $y > 0$ this guarantees $0 \leq x \bmod y < y$.

(a) 100 mod 3

$\lfloor 100/3 \rfloor = 33$. $100 - 3 \cdot 33 = 100 - 99 = 1$. **Answer:** 1. (Indeed $0 \leq 1 < 3$.)

(b) 100 mod 7

$\lfloor 100/7 \rfloor = 14$. $100 - 7 \cdot 14 = 100 - 98 = 2$. **Answer:** 2. (And $0 \leq 2 < 7$.)

(c) (-100) mod 7

$\lfloor -100/7 \rfloor = \lfloor -14.285\ldots \rfloor = -15$. $-100 - 7 \cdot (-15) = -100 + 105 = 5$. **Answer:** 5. (This is the unique residue with $0 \leq r < 7$.)

(d) (-100) mod 0

By the stated convention, $x \bmod 0 = x$. **Answer:** -100.

9 [05]

Definition we'll use

For any real x and $y \neq 0$,

$$x \bmod y \;=\; x - y\lfloor x/y \rfloor,$$

and from this it follows that

$$\begin{cases} 0 \leq x \bmod y < y & \text{if } y > 0, \\ y < x \bmod y \leq 0 & \text{if } y < 0. \end{cases}$$

So with a negative modulus, the remainder is nonpositive and strictly greater than the modulus.

Computations

1. $5 \bmod (-3)$

$$\left\lfloor \frac{5}{-3} \right\rfloor = \lfloor -1.666\ldots \rfloor = -2, \quad 5 - (-3)\cdot(-2) = 5 - 6 = -1.$$

Therefore $5 \bmod (-3) = -1$ (which lies in $(-3, 0]$ as required).

2. $18 \bmod (-3)$

$$\left\lfloor \frac{18}{-3} \right\rfloor = \lfloor -6 \rfloor = -6, \quad 18 - (-3)\cdot(-6) = 18 - 18 = 0.$$

Therefore $18 \bmod (-3) = 0$.

3. $(-2) \bmod (-3)$

$$\left\lfloor \frac{-2}{-3} \right\rfloor = \lfloor 0.666\ldots \rfloor = 0, \quad -2 - (-3)\cdot 0 = -2.$$

Therefore $(-2) \bmod (-3) = -2$.

10 [00]

We use Knuth's definition

$$x \bmod y \;=\; x - y\,\lfloor x/y \rfloor \quad (y \neq 0),$$

and his noted ranges: if $y > 0$ then $0 \leq x \bmod y < y$; if $y < 0$ then $0 \geq x \bmod y > y$.

1. $1.1 \bmod 1$ $\lfloor 1.1/1 \rfloor = \lfloor 1.1 \rfloor = 1$. $1.1 - 1 \cdot 1 = 0.1$. (Check: $y = 1 > 0$, result in $[0, 1)$.)
2. $0.11 \bmod 0.1$ $\lfloor 0.11/0.1 \rfloor = \lfloor 1.1 \rfloor = 1$. $0.11 - 0.1 \cdot 1 = 0.01$. (Check: $y = 0.1 > 0$, result in $[0, 0.1)$.)
3. $0.11 \bmod (-0.1)$ $\lfloor 0.11/(-0.1) \rfloor = \lfloor -1.1 \rfloor = -2$. $0.11 - (-0.1)\cdot(-2) = 0.11 - 0.2 = -0.09$. (Check: $y = -0.1 < 0$, result in $(-0.1, 0]$.)

Final answers:

$$1.1 \bmod 1 = 0.1, \qquad 0.11 \bmod 0.1 = 0.01, \qquad 0.11 \bmod (-0.1) = -0.09.$$

11 [00]

Set $z = 0$ in the congruence definition:

$$x \equiv y \pmod 0 \quad \Longleftrightarrow \quad x \bmod 0 = y \bmod 0.$$

By the "mod-0" convention, $x \bmod 0 = x$ and $y \bmod 0 = y$. Hence

$$x \equiv y \pmod 0 \quad \Longleftrightarrow \quad x = y.$$

(You can also see this from the divisor view: 0 divides $x - y$ only when $x - y = 0$, i.e., $x = y$.)

12 [00]

All integers (..., −2, −1, 0, 1, 2, ...) are relatively prime to 1.

"Relatively prime" means the greatest common divisor is 1: $\gcd(n, 1) = 1$ for **every** integer n.

- The only positive divisor of 1 is 1 itself.
- Any common divisor of n and 1 must therefore be ± 1.
- By definition, the greatest **positive** common divisor is 1. Hence $\gcd(n, 1) = 1$.

This remains true for the edge case $n = 0$: $\gcd(0, 1) = 1$. Therefore 0 is also coprime to 1.

13 [M00]

We need all integers m such that $\gcd(m, 0) = 1$.

By the stated convention, $\gcd(m, 0) = |m|$. Therefore $\gcd(m, 0) = 1$ iff $|m| = 1$. Hence the only integers relatively prime to 0 are

$$m = \pm 1.$$

Why this is true (two viewpoints)

1. **Divisors viewpoint.** Every integer divides 0. Thus the common divisors of m and 0 are exactly the divisors of m, so the greatest common divisor is $|m|$. The condition "relatively prime to 0" therefore forces $|m| = 1$.

2. **Bézout viewpoint (optional check).** A standard characterization says m and n are relatively prime iff there exist integers u, v with $um + vn = 1$. Taking $n = 0$, this becomes $um = 1$, which has an integer solution only when $m = \pm 1$.

Edge cases

- $m = 0$: Then $\gcd(0, 0) = 0$ under the convention (or is left undefined in some texts), so 0 is **not** relatively prime to 0.
- All other m with $|m| > 1$ fail since $\gcd(m, 0) = |m| > 1$.

14 [12]

Problem (TAOCP 1.2.4, Ex. 14): Find $x \bmod 15$ given $x \equiv 2 \pmod 3$ and $x \equiv 3 \pmod 5$.

Solution (by substitution / CRT):

Let $x = 5k + 3$ to satisfy $x \equiv 3 \pmod 5$. Reduce mod 3:

$$5k + 3 \equiv 2 \pmod 3 \implies 5k \equiv -1 \equiv 2 \pmod 3.$$

Since $5 \equiv 2 \pmod 3$, we have $2k \equiv 2 \pmod 3$, hence $k \equiv 1 \pmod 3$.

Write $k = 3t + 1$. Then

$$x = 5(3t + 1) + 3 = 15t + 8 \implies x \equiv 8 \pmod{15}.$$

Check: $8 \bmod 3 = 2$ and $8 \bmod 5 = 3$. Therefore, the unique solution modulo 15 is

$$\boxed{x \equiv 8 \pmod{15}}.$$

15 [10]

We consider all cases for $y, z \in \mathbb{R}$.

Case 1: $y \neq 0$ and $z \neq 0$. Using the definition of mod with floors,

$$x \bmod y \;=\; x - y\lfloor x/y \rfloor.$$

Multiply both sides by z:

$$z(x \bmod y) = zx - (zy)\lfloor x/y \rfloor. \tag{1}$$

Now compute the right-hand side target directly:

$$(zx) \bmod (zy) = zx - (zy)\left\lfloor \frac{zx}{zy} \right\rfloor.$$

But $\frac{zx}{zy} = \frac{x}{y}$ (since $zy \neq 0$), hence

$$\left\lfloor \frac{zx}{zy} \right\rfloor = \left\lfloor \frac{x}{y} \right\rfloor,$$

so

$$(zx) \bmod (zy) = zx - (zy)\lfloor x/y \rfloor,$$

which matches (1). Therefore $z(x \bmod y) = (zx) \bmod (zy)$.

Case 2: $z = 0$**.** Left side $= 0 \cdot (x \bmod y) = 0$. Right side $(0 \cdot x) \bmod (0 \cdot y) = 0 \bmod 0 = 0$ by the convention $x \bmod 0 = x$. Equality holds.

Case 3: $y = 0$ **(with any** z**).** Left side $= z(x \bmod 0) = z\,x$. Right side $(zx) \bmod (0) = zx$. Equality holds by the same convention.

All possibilities are covered, so the identity is proved.

Corollary (Knuth's Law C)

With congruence defined by equality of remainders, $a \equiv b \pmod{m} \iff a \bmod m = b \bmod m$, take any nonzero integer n. Applying the proved law with $z = n$ and $y = m$,

$$n(a \bmod m) = (na) \bmod (nm), \qquad n(b \bmod m) = (nb) \bmod (nm).$$

Thus $a \bmod m = b \bmod m$ iff $(na) \bmod (nm) = (nb) \bmod (nm)$, i.e.

$$a \equiv b \pmod{m} \iff na \equiv nb \pmod{nm} \quad (n \neq 0),$$

which is precisely Law C.

16 [M10]

1. By hypothesis, $(x - z)/y \in \mathbb{Z}$. Let

$$q \stackrel{\text{def}}{=} \frac{x - z}{y} \in \mathbb{Z},$$

   so $x = qy + z$.

2. Divide both sides by $y > 0$:

$$\frac{x}{y} = q + \frac{z}{y}.$$

   Because $0 \leq z < y$, we have $0 \leq \frac{z}{y} < 1$. Hence

$$q \leq \frac{x}{y} < q + 1.$$

3. The inequality $q \leq x/y < q + 1$ implies

$$q = \lfloor x/y \rfloor.$$

4. By the definition of the remainder ("mod" in this section),

$$x \bmod y \;=\; x - y\lfloor x/y \rfloor \;=\; x - yq \;=\; z.$$

Therefore $z = x \bmod y$. (Intuitively: $x = qy + z$ with $0 \leq z < y$ is the Euclidean division of x by y; the remainder is unique, so it must be z.)

Quick check (example). Take $x = 17$, $y = 5$. If $z = 2$, then $(x - z)/y = (17 - 2)/5 = 3 \in \mathbb{Z}$ and $0 \leq 2 < 5$. Our result says $z = 17 \bmod 5 = 2$, which holds.

17 [M15]

Here is Exercise 17 from §1.2.4, paraphrased: prove Law A directly from the definition of congruence; and prove the " " half of Law D: if $a \equiv b \pmod{rs}$ then $a \equiv b \pmod{r}$ and $a \equiv b \pmod{s}$ (with r, s arbitrary integers). The section defines $x \equiv y \pmod{z}$ to mean that z divides $x - y$.

Preliminaries (definition we'll use)

By definition,

$$x \equiv y \pmod{m} \quad \Longleftrightarrow \quad m \mid (x - y),$$

with the usual divisibility rules; when a modulus is negative one may take its absolute value without changing the congruence class.

Law A (addition and multiplication are well behaved mod m)

Claim. If $a \equiv b \pmod{m}$ and $x \equiv y \pmod{m}$, then

$$a \pm x \equiv b \pm y \pmod{m}, \qquad ax \equiv by \pmod{m}.$$

Proof (from the definition). From $a \equiv b \pmod{m}$ and $x \equiv y \pmod{m}$ we have $m \mid (a - b)$ and $m \mid (x - y)$. Hence

$$(a \pm x) - (b \pm y) = (a - b) \pm (x - y),$$

a sum/difference of integers each divisible by m; therefore m also divides the right-hand side, proving $a \pm x \equiv b \pm y \pmod{m}$.

For products,

$$ax - by = a(x - y) + y(a - b).$$

Each term on the right is a multiple of m (since $m \mid (x - y)$ and $m \mid (a - b)$), so their sum is a multiple of m. Thus $ax \equiv by \pmod{m}$.

(These are exactly the operations stated as Law A in the text.)

Half of Law D (the easy direction)

Claim. If $a \equiv b \pmod{rs}$ for integers r, s, then $a \equiv b \pmod{r}$ and $a \equiv b \pmod{s}$.

Proof (from the definition). $a \equiv b \pmod{rs}$ means $rs \mid (a-b)$, so $(a-b) = rs\,k$ for some integer k. Then clearly $r \mid (a-b)$ (since $a-b = r(sk)$) and $s \mid (a-b)$ (since $a-b = s(rk)$). Hence $a \equiv b \pmod{r}$ and $a \equiv b \pmod{s}$.

Remarks

The *converse* of this implication generally **fails** unless r and s are relatively prime; the full bi-implication "$a \equiv b \pmod{rs}$ iff $a \equiv b \pmod{r}$ and $a \equiv b \pmod{s}$" is Law D with the hypothesis $r \perp s$. (E.g., $2 \equiv 8 \pmod{3}$ and $2 \equiv 8 \pmod{6}$ but $2 \not\equiv 8 \pmod{18}$.)

18 [M15]

Assume $r \perp s$ (i.e., $\gcd(r,s) = 1$). Show that from

$$a \equiv b \pmod{r} \quad \text{and} \quad a \equiv b \pmod{s}$$

it follows that

$$a \equiv b \pmod{rs}.$$

(This is the "other half" of Law D.)

Law B we'll use

If $c \perp m$ and $cx \equiv cy \pmod{m}$, then $x \equiv y \pmod{m}$; in words, when the factor is relatively prime to the modulus, you may "cancel" it in a congruence. Also, $x \perp y$ denotes "relatively prime."

1. From $a \equiv b \pmod{r}$ there exists an integer u with

$$a = b + ru.$$

2. Reduce this identity modulo s and use the second hypothesis $a \equiv b \pmod{s}$:

$$b + ru \equiv b \pmod{s} \quad \Longrightarrow \quad ru \equiv 0 \pmod{s}.$$

3. Because $r \perp s$, we may **cancel** r **modulo** s by Law B, obtaining

$$u \equiv 0 \pmod{s}.$$

Hence $u = sw$ for some integer w.

4. Substitute back into step (1):

$$a = b + r(sw) = b + rsw,$$

which is exactly

$$a \equiv b \pmod{rs}.$$

This completes the proof of the $\Leftarrow$ direction of Law D using Law B.

19 [M10]

Key tool: Extended Euclid

Algorithm E takes two positive integers m, n, computes $d = \gcd(m, n)$, and also finds integers a, b with

$$am + bn = d \quad \text{(Bézout identity)}.$$

We'll use the coefficient of n as the modular inverse when $d = 1$.

Assume $\gcd(n, m) = 1$.

1. Run Extended Euclid on the pair (m, n). It returns integers a, b with

$$am + bn \;=\; 1.$$

2. Reduce both sides modulo m. Since $am \equiv 0 \pmod{m}$,

$$bn \equiv 1 \pmod{m}.$$

3. Therefore $n' \equiv b \pmod{m}$ is a multiplicative inverse of n modulo m. Picking the least nonnegative representative of b yields $n' \in \{0, 1, \ldots, m-1\}$; it cannot be 0 because $0 \cdot n \not\equiv 1 \pmod{m}$. Hence $1 \leq n' \leq m - 1$, as desired.

(Uniqueness mod m.) If $nn_1' \equiv nn_2' \equiv 1 \pmod{m}$, subtract to get $n(n_1' - n_2') \equiv 0 \pmod{m}$. Since $\gcd(n, m) = 1$, cancel n to conclude $n_1' \equiv n_2' \pmod{m}$. Thus the inverse is unique modulo m.

This finishes the proof.

Worked example

Find the inverse of $n = 7$ modulo $m = 26$.

Extended Euclid:

$$\begin{aligned} 26 &= 3 \cdot 7 + 5, \\ 7 &= 1 \cdot 5 + 2, \\ 5 &= 2 \cdot 2 + 1. \end{aligned}$$

Back-substitute:

$$1 = 5 - 2 \cdot 2 = 5 - 2(7 - 1 \cdot 5) = 3 \cdot 5 - 2 \cdot 7 = 3(26 - 3 \cdot 7) - 2 \cdot 7 = 3 \cdot 26 - 11 \cdot 7.$$

So $a = 3$, $b = -11$ with $a \cdot 26 + b \cdot 7 = 1$. Hence $n' \equiv b \equiv -11 \equiv 15 \pmod{26}$. Check: $7 \cdot 15 = 105 \equiv 1 \pmod{26}$.

How to compute n' in practice

Run Extended Euclid on (m, n); take the returned coefficient b of n; reduce it modulo m to $\{1, \dots, m-1\}$. That number is n'.

20 [M15]

Goal (Law B). If $ax \equiv by \pmod{m}$ and $a \equiv b \pmod{m}$, and if $\gcd(a, m) = 1$, then $x \equiv y \pmod{m}$.

Facts we'll use. • **Law A (closure under $+$, $-$, $\times$):** from $u \equiv v$ and $r \equiv s \pmod{m}$ we may conclude $u \pm r \equiv v \pm s$ and $ur \equiv vs \pmod{m}$. • **Law of inverses (Ex. 19):** if $n \perp m$ there exists n' with $nn' \equiv 1 \pmod{m}$. Apply with $n = a$. • The exercise to prove (Ex. 20): use the above to derive Law B.

1. Since $\gcd(a, m) = 1$, by the law of inverses there exists a' such that

$$aa' \equiv 1 \pmod{m}.$$

2. Because $a \equiv b \pmod{m}$, multiply both sides by a' and use Law A (multiplication preserves congruence):

$$a'a \equiv a'b \pmod{m} \quad \Longrightarrow \quad 1 \equiv a'b \pmod{m}.$$

So a' is also a (modular) inverse of b modulo m.

3. Now start from the given $ax \equiv by \pmod{m}$ and multiply both sides by a' (again by Law A):

$$a'(ax) \equiv a'(by) \pmod{m} \quad \Longrightarrow \quad (a'a)x \equiv (a'b)y \pmod{m}.$$

4. Substitute the equalities from steps 1–2:

$$1 \cdot x \equiv 1 \cdot y \pmod{m} \quad \Longrightarrow \quad x \equiv y \pmod{m}.$$

This is exactly Law B.

Remark. The condition $\gcd(a, m) = 1$ is essential; without it, "division" by a modulo m need not be valid (see the nearby exercise asking for a counterexample when $a \not\perp m$).

21 [M22]

Preliminaries we're allowed to use

- **Exercise 1.2.1–5 (existence).** By strong induction: if n is prime we're done; if n is composite, write $n = ab$ with $1 < a, b < n$ and factor a and b by the induction hypothesis; concatenating gives a prime product for n. Hence **every $n > 1$ is a product of primes**.
- **Law B (cancellation in congruences).** If $a \perp m$ (i.e., $\gcd(a, m) = 1$) and $ax \equiv ay \pmod{m}$, then $x \equiv y \pmod{m}$. We will use this to derive Euclid's lemma.

Step 1: Euclid's Lemma from Law B

> **Lemma (Euclid).** Let p be prime. If $p \mid ab$, then $p \mid a$ or $p \mid b$.

Proof using Law B. From $p \mid ab$ we have $ab \equiv 0 \pmod{p}$. If $p \mid a$ we are done. Otherwise $\gcd(a, p) = 1$ (because p is prime), so we may cancel a by Law B to deduce $b \equiv 0 \pmod{p}$, i.e., $p \mid b$.

Step 2: Uniqueness of prime factorization

Assume $n > 1$ and that we already know (by 1.2.1–5) that a prime factorization exists. Suppose we have **two** such factorizations

$$n = p_1 p_2 \cdots p_k \;=\; q_1 q_2 \cdots q_\ell,$$

where each p_i and q_j is prime. (We may, and will, reorder each side nondecreasingly at the end.)

1. Since $p_1 \mid n$, we have $p_1 \mid q_1 q_2 \cdots q_\ell$. By Euclid's lemma there is some j with $p_1 \mid q_j$. Because q_j is prime, this forces $p_1 = q_j$.
2. Cancel that common prime from both sides (divide both sides by p_1); we get

$$\frac{n}{p_1} = p_2 \cdots p_k \;=\; q_1 \cdots \widehat{q_j} \cdots q_\ell.$$

(The hat means "omit".)

3. Repeat the same argument: the smallest remaining p-prime divides the product of the remaining q-primes, so it equals one of them; cancel and continue.

After finitely many steps all factors are matched and canceled. Therefore $k = \ell$ and the two factorizations agree up to order. That is, **the multiset of prime factors of n is unique**.

22 [M10]

If a shares a nontrivial factor with m, multiplication by a can send a **nonzero** residue to 0 modulo m (i.e., a is a zero divisor in $\mathbb{Z}/m\mathbb{Z}$). That breaks cancellation.

A clean general counterexample

Let $m = ax$ with $a > 1$ and $x > 0$. Then

$$ax \equiv 0 \pmod{m} \quad \text{but} \quad x \not\equiv 0 \pmod{m}$$

since $0 < x < m$. Take $b = a$ and $y = 0$. We have $a \equiv b \pmod{m}$ and

$$ax \equiv a \cdot 0 \pmod{m},$$

yet $x \not\equiv 0 \pmod{m}$. Thus the implication asserted by Law B **fails** when a is not relatively prime to m. (This is exactly the book's official answer in brief form.)

A concrete numeric example

Pick $m = 12$, $a = 6$, $x = 2$, $y = 0$.

- $a \equiv a \pmod{12}$ is trivial.
- $ax = 6 \cdot 2 = 12 \equiv 0 \pmod{12} = a \cdot 0$.
- But $x = 2 \not\equiv 0 \pmod{12}$.

So all premises of Law B hold **except** $a \perp m$, and the conclusion $x \equiv y$ fails.

Why this always works (intuition)

When m is composite and $\gcd(a, m) > 1$, the class $[a]$ is a zero divisor in $\mathbb{Z}/m\mathbb{Z}$. Multiplying by a zero divisor isn't injective, so you cannot cancel it.

Bonus: when cancellation *does* work

If $\gcd(a, m) = 1$, there is an inverse a' with $aa' \equiv 1 \pmod{m}$ (exercise 19), and multiplying $ax \equiv ay$ by a' yields $x \equiv y \pmod{m}$ (exercise 20), which is precisely Law B's proof under its hypothesis.

23 [M10]

Here's a clean counterexample showing why the hypothesis "$r \perp s$" (i.e., $\gcd(r, s) = 1$) is necessary in Law D.

Law D (as stated in §1.2.4): If $r \perp s$, then

$$a \equiv b \pmod{rs} \quad \Longleftrightarrow \quad \begin{cases} a \equiv b \pmod{r}, \\ a \equiv b \pmod{s}\ . \end{cases}$$

We'll show this fails when r and s are not coprime.

Counterexample

Take $r = 2$ and $s = 2$ (so $\gcd(r, s) = 2 > 1$). Let $a = 1$ and $b = 3$.

- Modulo $r = 2$: $3 - 1 = 2$ is divisible by 2, so $1 \equiv 3 \pmod{2}$.
- Modulo $s = 2$: same conclusion: $1 \equiv 3 \pmod{2}$.
- Modulo $rs = 4$: $3 - 1 = 2$ is **not** divisible by 4, so $1 \not\equiv 3 \pmod{4}$.

Thus the right-hand side holds (congruence mod r and mod s), but the left-hand side fails (no congruence mod rs). This shows Law D "is not always true" without the coprimality condition—exactly what Exercise 23 asks for.

Why this happens (quick insight)

In general,

$$a \equiv b \pmod{r} \text{ and } a \equiv b \pmod{s} \quad \Longleftrightarrow \quad a \equiv b \pmod{\operatorname{lcm}(r, s)}.$$

When $r \perp s$, $\operatorname{lcm}(r, s) = rs$ and Law D holds. If $\gcd(r, s) > 1$, then $\operatorname{lcm}(r, s) < rs$, so you can have congruence mod r and s but not mod rs—as in the example above.

24 [M20]

Step 1 — Define partial products and show convergence

Let

$$F_n(x) \;=\; x\,e^{\gamma x}\prod_{k=1}^{n}\Big((1+\tfrac{x}{k})e^{-x/k}\Big), \qquad F(x)=\lim_{n\to\infty}F_n(x).$$

Taking logs,

$$\log F_n(x)=\log x+\gamma x+\sum_{k=1}^{n}\Big(\log(1+\tfrac{x}{k})-\tfrac{x}{k}\Big).$$

For fixed x, $\log(1+u)=u-\frac{u^2}{2}+O(u^3)$ gives

$$\log(1+\tfrac{x}{k})-\tfrac{x}{k}=-\frac{x^2}{2k^2}+O\Big(\frac{1}{k^3}\Big).$$

Hence $\sum_{k\geq 1}(\log(1+\frac{x}{k})-\frac{x}{k})$ converges absolutely (like $\sum 1/k^2$), so $F_n(x)$ converges to an analytic function $F(x)$ (locally uniformly in x).

Step 2 — Evaluate $F(1)$

For $x=1$,

$$F_n(1)=e^{\gamma}\prod_{k=1}^{n}\Big(\tfrac{k+1}{k}\,e^{-1/k}\Big)=e^{\gamma}\,(n+1)\,e^{-H_n},$$

where $H_n=\sum_{k=1}^{n}\frac{1}{k}$. Since $H_n=\ln n+\gamma+o(1)$, we get $F_n(1)\to e^{\gamma}\cdot(n+1)\cdot e^{-(\ln n+\gamma)}\to 1$. Thus $F(1)=1$.

Step 3 — Derive the functional equation

Compute the ratio of partial products:

$$\frac{F_n(x+1)}{F_n(x)}=\frac{x+1}{x}\,e^{\gamma}\prod_{k=1}^{n}\frac{1+(x+1)/k}{1+x/k}\,e^{-1/k}=\frac{x+n+1}{x}\,e^{\gamma-H_n}.$$

Using again $H_n=\ln n+\gamma+o(1)$, the right side tends to $\frac{1}{x}$. Passing to the limit,

$$\boxed{F(x+1)=\frac{F(x)}{x}} \quad \text{for all } x \text{ where } F \text{ is defined.}$$

Step 4 — Identify F with $1/\Gamma$

Set $G(x) = 1/F(x)$. Then

$$G(1) = 1, \qquad G(x+1) = x\,G(x).$$

Moreover,

$$\log G(x) = -\log x - \gamma x - \sum_{k\geq 1}\Big(\log(1+\tfrac{x}{k}) - \tfrac{x}{k}\Big),$$

so

$$(\log G)''(x) = \frac{1}{x^2} + \sum_{k\geq 1}\frac{1}{(k+x)^2} > 0 \quad (x>0).$$

Thus G is **log-convex** on $(0,\infty)$. By the Bohr–Mollerup theorem (the unique log-convex function on $(0,\infty)$ with $G(1)=1$ and $G(x+1)=xG(x)$ is $\Gamma(x)$), we conclude

$$G(x) = \Gamma(x) \quad \text{on } (0,\infty).$$

Hence $F(x) = 1/\Gamma(x)$ on $(0,\infty)$, and by analytic continuation the identity holds for all complex x not a non-positive integer (where Γ has poles).

25 [M02]

Fact we'll use (Theorem F)

If p is prime, then for every integer a,

$$a^p \equiv a \pmod{p}.$$

Equivalently, when $p \nmid a$ (so $\gcd(a,p)=1$), we also have $a^{p-1} \equiv 1 \pmod{p}$.

Consider two exhaustive cases.

Case 1: $p \mid a$**.** Then $a \equiv 0 \pmod{p}$. Any positive power of a is also $0 \pmod{p}$, hence

$$a^{p-1} \equiv 0 \pmod{p}.$$

The Iverson bracket $[\,p \nmid a\,]$ is 0 in this case, so the identity holds.

Case 2: $p \nmid a$**.** Now $\gcd(a, p) = 1$. By Theorem F, $a^p \equiv a \pmod{p}$. Multiply both sides by a^{p-2} (which is allowed because a is invertible modulo p; equivalently, we're using "cancellation" in modular arithmetic). This gives

$$a^{p-1} \equiv 1 \pmod{p}.$$

Here $[\,p \nmid a\,] = 1$, so again the identity holds.

Combining the two cases:

$$a^{p-1} \bmod p = \begin{cases} 0, & p \mid a, \\ 1, & p \nmid a, \end{cases}$$

which is exactly $[\,p \nmid a\,]$.

Tiny sanity checks

- $p = 7, a = 14$: $a^6 \equiv 0 \pmod{7} \Rightarrow 0$ — matches $[7 \nmid 14] = 0$.
- $p = 7, a = 5$: $5^6 \equiv 1 \pmod{7} \Rightarrow 1$ — matches $[7 \nmid 5] = 1$.

26 [M15]

Work in the ring $\mathbb{Z}/p\mathbb{Z}$. Because p is prime, $\mathbb{Z}/p\mathbb{Z}$ is a field (in particular, it has no zero divisors).

Assume $b^2 \equiv 1 \pmod{p}$. Then

$$b^2 - 1 \equiv 0 \pmod{p} \qquad \Longleftrightarrow \qquad (b-1)(b+1) \equiv 0 \pmod{p}.$$

Since there are no zero divisors modulo p, the product of two residues is 0 mod p only if one of them is 0 mod p. Hence either

$$b - 1 \equiv 0 \pmod{p} \quad \text{or} \quad b + 1 \equiv 0 \pmod{p},$$

i.e.,

$$b \equiv 1 \pmod{p} \quad \text{or} \quad b \equiv -1 \pmod{p}.$$

This proves the claim.

Why primeness matters (counterexample)

If the modulus isn't prime, extra solutions can appear. For example, modulo 15,

$$4^2 = 16 \equiv 1 \pmod{15},$$

but $4 \not\equiv \pm 1 \pmod{15}$. Thus the "only ± 1" conclusion relies essentially on the modulus being prime (so that there are no zero divisors).

Useful corollary

By Fermat's theorem, if p is prime and $q \not\equiv 0 \pmod{p}$, then $q^{p-1} \equiv 1 \pmod{p}$. Hence $\left(q^{(p-1)/2}\right)^2 \equiv 1 \pmod{p}$, and by the result above

$$q^{(p-1)/2} \equiv \pm 1 \pmod{p}.$$

This is exactly why, in Exercise 47, the Legendre symbol $\left(\frac{q}{p}\right)$ takes only the values $+1$ or -1 when $q \not\equiv 0 \pmod{p}$ (and 0 when $q \equiv 0 \pmod{p}$). Knuth explicitly points to Exercise 26 for this fact. (PagePlace)

Tiny sanity check

Take $p = 7$. The squares mod 7 are $0, 1, 2, 4$; the only residues with square 1 are 1 and 6 $(= -1)$, matching the theorem.

27 [M15]

1) The prime case: (p)=p−1

Let p be prime. Among the p numbers 0,1,2,…,p−1:

- 0 is not relatively prime to p (its gcd with p is p 1).
- Every k with 1 k p−1 is relatively prime to p, because the only positive divisors of p are 1 and p; since k<p, p k, so gcd(k,p)=1.

Thus exactly the p−1 numbers 1,2,…,p−1 are coprime to p, hence

$$\varphi(p) = p - 1.$$

2) Prime powers: (p$^{e)=p}$e−p^{,e-1}

Let e 1 and consider the set S={0,1,…,p^e−1} (|S|=p^e). An integer k S is **not** relatively prime to p^e iff p|k. Indeed, any common divisor of k and p^e is a power of p; so gcd(k,p^e)>1 p|k, and conversely if p|k then gcd(k,p^e) p>1.

How many multiples of p are in S? They are

$$0,\ p,\ 2p,\ 3p,\ \dots,\ (p^{e-1}-1)p,$$

exactly p^{e-1} of them. Therefore the integers **coprime** to p^e are the remaining ones:

$$\varphi(p^e) = |S| - |\{\text{multiples of } p\}| = p^e - p^{e-1} = p^e\left(1 - \frac{1}{p}\right).$$

This matches the given examples: (1)=1, (2)=1, (3)=2, (4)=4−2=2, etc.

28 [M25]

Step 1 — Closure under multiplication (coprimality is preserved)

Assume $\gcd(a, m) = \gcd(b, m) = 1$. Suppose a prime p divides both $ab \bmod m$ and m. Then $p \mid ab$ and $p \mid m$. By Euclid's lemma, $p \mid a$ or $p \mid b$, contradicting $\gcd(a, m) = \gcd(b, m) = 1$. Hence $\gcd(ab \bmod m,\ m) = 1$. This is exactly the first observation the exercise nudges you to make.

Step 2 — Multiplication by a permutes the reduced residue system

Let $x_1, \dots, x_{\varphi(m)}$ be **a reduced residue system modulo** m (all incongruent mod m, each coprime to m). Because of Step 1, every $ax_i \bmod m$ is also coprime to m; so the multiset

$$\{ax_1 \bmod m,\ ax_2 \bmod m,\ \dots,\ ax_{\varphi(m)} \bmod m\}$$

consists only of elements from the reduced residue system.

If $ax_i \equiv ax_j \pmod m$ and $\gcd(a, m) = 1$, we may cancel a (since a has a multiplicative inverse modulo m) to get $x_i \equiv x_j \pmod m$. Thus the map $x \mapsto ax \bmod m$ is injective on a finite set of size $\varphi(m)$, hence a permutation of the reduced residue system. ([cs.ucf.edu][2])

Step 3 — Multiply everything and cancel

Multiplying all elements of the reduced residue system before and after the permutation,

$$\prod_{i=1}^{\varphi(m)} (ax_i) \equiv \prod_{i=1}^{\varphi(m)} x_i \pmod m.$$

The left side is $a^{\varphi(m)} \cdot \prod_{i=1}^{\varphi(m)} x_i$. Because every x_i is coprime to m, the product $P = \prod x_i$ is also coprime to m, so P has a multiplicative inverse modulo m. Cancel P to obtain

$$a^{\varphi(m)} \equiv 1 \pmod{m},$$

which is **Euler's theorem**.

Check on an example

Take $m = 10$. A reduced residue system is $\{1, 3, 7, 9\}$ (so $\varphi(10) = 4$). For $a = 3$ (coprime to 10):

$$3 \cdot \{1, 3, 7, 9\} \equiv \{3, 9, 1, 7\} \pmod{10},$$

a permutation of $\{1, 3, 7, 9\}$. Multiplying and canceling gives $3^4 \equiv 1 \pmod{10}$.

29 [M22]

(a) $f(n) = n^c$ is multiplicative (in fact, completely multiplicative)

For any positive integers r, s (coprime or not),

$$f(rs) = (rs)^c = r^c\, s^c = f(r)\, f(s),$$

by the basic law of exponents. Thus f is **completely** multiplicative (the coprimality condition isn't even needed).

(b) The squarefree indicator is multiplicative

Let $f(n) = 1$ if n is squarefree and 0 otherwise. Suppose $r \perp s$.

- If **both** r and s are squarefree, then every prime p appears with exponent 0 or 1 in each; since the supports of their prime factors are disjoint (coprimality), in rs every prime still has exponent 0 or 1. Hence rs is squarefree and $f(rs) = 1 = f(r)f(s)$.
- Conversely, if rs is squarefree, write the p-adic valuations as $v_p(rs) = v_p(r) + v_p(s)$. Squarefreeness means $v_p(rs) \leq 1$ for all p, so for each p we must have $v_p(r) \leq 1$ and $v_p(s) \leq 1$. Thus both r and s are squarefree. Therefore $f(rs) = f(r)f(s)$.

Hence f is multiplicative (but not completely multiplicative; e.g., $f(p^2) = 0 \neq 1 = f(p)f(p)$).

(c) $f(n) = c^{\omega(n)}$ is multiplicative

Let $\omega(n)$ be the number of **distinct** prime divisors of n. If $r \perp s$, their prime supports are disjoint, so

$$\omega(rs) = \omega(r) + \omega(s).$$

Therefore

$$f(rs) = c^{\omega(rs)} = c^{\omega(r)+\omega(s)} = c^{\omega(r)}\, c^{\omega(s)} = f(r)\, f(s).$$

(Again, not completely multiplicative in general: $f(p^2) = c \neq c^2 = f(p)^2$ unless $c = 1$.)

(d) The product of multiplicative functions is multiplicative

Let f, g be multiplicative and define $(fg)(n) = f(n)g(n)$. For $r \perp s$,

$$(fg)(rs) = f(rs)g(rs) = \big(f(r)f(s)\big)\big(g(r)g(s)\big) = \underbrace{f(r)g(r)}_{(fg)(r)}\ \underbrace{f(s)g(s)}_{(fg)(s)} = (fg)(r)\,(fg)(s),$$

so fg is multiplicative.

30 [M30]

Facts we'll use

1. Definition. (n) counts residues modulo n that are coprime to n. (Introduced in Ex. 27.)
2. Prime powers. For a prime p and e 1,

$$\varphi(p^e) = p^e - p^{e-1} = p^{e-1}(p-1),$$

because among the p^e residues, exactly the multiples of p (there are p^{e-1} of them) are not coprime to p^e. (This is exactly what Ex. 27 asks you to establish.)

is multiplicative

We must show that if m and n are coprime (m n), then

$$\varphi(mn) = \varphi(m)\,\varphi(n).$$

Consider the set of units (numbers coprime to the modulus) modulo mn:

$$U_{mn} = \{\, x \bmod mn : \gcd(x, mn) = 1 \,\}.$$

Define a map

$$\Phi : U_{mn} \to U_m \times U_n, \qquad \Phi([x]_{mn}) = ([x]_m,\, [x]_n).$$

If $\gcd(m, n) = 1$, the Chinese Remainder Theorem (CRT) says that the pair of congruences

$$y \equiv a \pmod{m}, \quad y \equiv b \pmod{n}$$

has a unique solution modulo mn. Moreover, x is coprime to mn iff it is coprime to each of m and n. Therefore:

- **Well-defined/injective:** If $[x]_{mn} = [x']_{mn}$, then $[x]_m = [x']_m$ and $[x]_n = [x']_n$.
- **Surjective:** For any $([a]_m, [b]_n) \in U_m \times U_n$, CRT gives a unique $[y]_{mn}$ with $y \equiv a \pmod{m}$, $y \equiv b \pmod{n}$; since a and b are each coprime to their moduli, y is coprime to mn.

Thus Φ is a bijection, so $|U_{mn}| = |U_m| \cdot |U_n|$, i.e.

$$\boxed{\varphi(mn) = \varphi(m)\,\varphi(n) \quad \text{when } \gcd(m, n) = 1}.$$

Evaluate (1 000 000)

Factor 1,000,000 $= 10^6 = 2^6 \cdot 5^6$. Using prime-power values and multiplicativity,

$$\varphi(10^6) = \varphi(2^6)\,\varphi(5^6) = (2^6 - 2^5)(5^6 - 5^5) = (64 - 32)\,(15625 - 3125) = 32 \cdot 12500.$$

Compute: $12500 \times 30 = 375{,}000$; $12500 \times 2 = 25{,}000$; sum $= 400{,}000$. Hence

$$\boxed{\varphi(1{,}000{,}000) = 400{,}000}.$$

General method once the prime factorization is known

If

$$n = \prod_{i=1}^{r} p_i^{e_i} \quad (\text{distinct primes } p_i),$$

then by multiplicativity and the prime-power formula,

$$\varphi(n) = \prod_{i=1}^{r} \varphi(p_i^{e_i}) = \prod_{i=1}^{r} (p_i^{e_i} - p_i^{e_i - 1}) = n \prod_{i=1}^{r} \Big(1 - \frac{1}{p_i}\Big).$$

So the algorithm is:

1. Factor n: $n = \prod p_i^{e_i}$.
2. Compute $n \prod_i (1 - 1/p_i)$.

31 [M22]

Key fact

If $\gcd(r, s) = 1$, then every divisor d of rs factors **uniquely** as $d = ab$ with $a \mid r$ and $b \mid s$; moreover $\gcd(a, b) = 1$. (If a prime divided both a and b, it would divide both r and s, contradicting $\gcd(r, s) = 1$.)

Proof that g is multiplicative

Take coprime positive integers r, s. Then

$$g(rs) = \sum_{d \mid rs} f(d) = \sum_{\substack{a \mid r \\ b \mid s}} f(ab)$$

by the divisor factorization just noted. Since $\gcd(a, b) = 1$ and f is multiplicative, $f(ab) = f(a)f(b)$. Therefore

$$g(rs) = \sum_{a \mid r} \sum_{b \mid s} f(a)f(b) = \Big(\sum_{a \mid r} f(a)\Big)\Big(\sum_{b \mid s} f(b)\Big) = g(r)\, g(s).$$

Hence g is multiplicative.

Prime-power and Euler-product view (useful checks)

Let $n = \prod_p p^{\nu_p(n)}$. The proof implies

$$g(n) = \prod_{p \mid n} \Big(f(1) + f(p) + \cdots + f(p^{\nu_p(n)}) \Big),$$

so multiplicativity is also apparent from the factorization over distinct primes.

Quick examples

1. $f(n) \equiv 1$. Then $g(n) = \sum_{d \mid n} 1 = \tau(n)$ (divisor count), which is multiplicative.
2. $f(n) = n^c$ (multiplicative for any fixed real c). Then

$$g(n) = \prod_{p^k \| n} (1 + p^c + \cdots + p^{ck}),$$

again a multiplicative function.

32 [M18]

Think of the left-hand double sum as running over all *chains of divisors*

$$c \mid d \mid n.$$

Define the two index sets

$$A = \{(c, d) : c \mid d,\ d \mid n\}, \qquad B = \{(c, e) : c \mid n,\ e \mid (n/c)\}.$$

Consider the bijection

$$\Phi : A \to B, \qquad \Phi(c, d) = (c, e) \text{ with } e := d/c.$$

- **Well-defined.** If $c \mid d$ then $e = d/c$ is a positive integer. Since $d \mid n$, we have $ce = d \mid n$, hence $e \mid n/c$. Also $c \mid n$ because $c \mid d \mid n$. Thus $(c, e) \in B$.
- **Inverse map.** Given $(c, e) \in B$ with $c \mid n$ and $e \mid n/c$, set $d := ce$. Then $c \mid d$ obviously, and $d = ce \mid c \cdot (n/c) = n$, so $(c, d) \in A$. This is the inverse of Φ.

Because Φ is a bijection between the exact index sets of the two sums and it transforms the summand $f(c, d)$ on the left into $f(c, ce)$ on the right (with $e = d/c$), we get

$$\sum_{(c,d) \in A} f(c, d) \ = \sum_{(c,e) \in B} f(c, ce),$$

which is precisely the desired identity.

□

33 [M18]

Exactly one of the two halves

$$x = \frac{n+m}{2}, \qquad y = \frac{n-m+1}{2}$$

is an integer and the other is a half–integer (i.e., has fractional part 1/2).

Reason: $n+m$ and $n-m$ always have the same parity. So:

- If $n+m$ is even, then $x \in \mathbb{Z}$. Since $n-m$ is even, $n-m+1$ is odd, hence y is a half–integer.
- If $n+m$ is odd, then x is a half–integer. Since $n-m$ is odd, $n-m+1$ is even, hence $y \in \mathbb{Z}$.

Also note

$$x + y = \frac{n+m}{2} + \frac{n-m+1}{2} = n + \tfrac{1}{2}.$$

We'll use the standard definitions of floor/ceiling (greatest/largest integer $\leq/\geq$ a real), as introduced earlier in §1.2.4.

Part (a): sum of floors

Since $x + y = n + \frac{1}{2}$, we have $\lfloor x+y \rfloor = n$. From the parity analysis, one of x, y is an integer (fractional part 0) and the other has fractional part 1/2. Therefore $\{x\} + \{y\} = 1/2 < 1$, so

$$\lfloor x \rfloor + \lfloor y \rfloor = \lfloor x+y \rfloor = n.$$

Hence

$$\boxed{\left\lfloor \tfrac{n+m}{2} \right\rfloor + \left\lfloor \tfrac{n-m+1}{2} \right\rfloor = n}$$

Part (b): sum of ceilings

Again, exactly one of x, y is an integer and the other is a half–integer. Taking ceilings adds 0 to the integer one and adds 1 to the half–integer one. Therefore

$$\lceil x \rceil + \lceil y \rceil = \lfloor x \rfloor + \lfloor y \rfloor + 1 = n + 1.$$

Equivalently, since $x + y = n + \frac{1}{2}$, $\lceil x+y \rceil = n+1$ and the same "one integer, one half–integer" parity argument shows no extra carry beyond that single $+1$. Thus

$$\boxed{\left\lceil \frac{n+m}{2} \right\rceil + \left\lceil \frac{n-m+1}{2} \right\rceil = n+1}$$

Noted special case $m = 0$

- $\lfloor n/2 \rfloor + \lfloor (n+1)/2 \rfloor = n$.
- $\lceil n/2 \rceil + \lceil (n+1)/2 \rceil = n+1$.

These are handy "split-n-in-halves" identities that often simplify parity arguments.

34 [M21]

Answer. Exactly for **integer bases** $b \geq 2$. The same necessary and sufficient condition applies to the ceiling version $\lceil \log_b x \rceil = \lceil \log_b \lceil x \rceil \rceil$.

Proof (necessity)

Assume the floor identity holds for all $x \geq 1$. Plug in $x = b$:

$$\lfloor \log_b b \rfloor = 1 \stackrel{!}{=} \lfloor \log_b \lfloor b \rfloor \rfloor.$$

Now $\lfloor b \rfloor < b$ unless b is an integer. If $b \notin \mathbb{Z}$, then $\log_b \lfloor b \rfloor < 1$ so the right-hand side is 0, contradicting 1. Also $b > 1$ forces $b \geq 2$ when $b \in \mathbb{Z}$. Hence b must be an **integer** ≥ 2.

Proof (sufficiency)

Suppose b is an integer ≥ 2. For any $x \geq 1$, let

$$n = \lfloor \log_b x \rfloor \quad \Longleftrightarrow \quad b^n \leq x < b^{n+1}.$$

Because b is an integer, every b^k is an integer. Thus

$$b^n \leq x \ < \ b^{n+1} \quad \Longrightarrow \quad b^n \leq \lfloor x \rfloor \leq x < b^{n+1}.$$

Taking $\log_b$ preserves inequalities (since $b > 1$):

$$n \leq \log_b \lfloor x \rfloor < n+1,$$

so $\lfloor \log_b \lfloor x \rfloor \rfloor = n = \lfloor \log_b x \rfloor$. This proves the identity for all $x \geq 1$.

(The ceiling version is proved identically by bracketing x between consecutive integer powers of b and noting that b^n are integers when $b \in \mathbb{Z}$.)

Why non-integer bases fail

- If $1 < b < 2$, then $\lfloor b \rfloor = 1$ and $\lfloor \log_b b \rfloor = 1$ but $\lfloor \log_b \lfloor b \rfloor \rfloor = \lfloor \log_b 1 \rfloor = 0$.
- If $b > 2$ is non-integer, $\lfloor b \rfloor \in \{2, 3, \dots\}$ and still $\lfloor \log_b \lfloor b \rfloor \rfloor = 0$ while $\lfloor \log_b b \rfloor = 1$.

Either way we get a contradiction at $x = b$.

A nice generalization (McEliece)

Let f be continuous and strictly increasing on an interval A, with the property that whenever $x \in A$ then $\lfloor x \rfloor, \lceil x \rceil \in A$. Then the following are equivalent:

- $\lfloor f(x) \rfloor = \lfloor f(\lfloor x \rfloor) \rfloor$ for all $x \in A$.
- $\lceil f(x) \rceil = \lceil f(\lceil x \rceil) \rceil$ for all $x \in A$.
- **Criterion:** whenever $f(x)$ is an integer, then x is an integer.

Taking $f(x) = \log_b x$ on $A = [1, \infty)$ makes the criterion "if $\log_b x \in \mathbb{Z}$ then $x \in \mathbb{Z}$", which is true exactly when b is an integer ≥ 2.

Quick sanity checks

- $b = 10$ (decimal): $\lfloor \log_{10} x \rfloor$ is "(number of digits of $\lfloor x \rfloor$) -1", so equality is immediate.
- $b = \pi$: taking $x = \pi$ gives 1 on the left and 0 on the right—fails.

35 [M20]

Write $x = \lfloor x \rfloor + y$ with $0 \leq y < 1$ (so y is the fractional part of x). Set $M = \lfloor x \rfloor + m$, which is an integer. By the division algorithm there exist unique integers k and r with

$$M = kn + r, \qquad 0 \leq r \leq n - 1.$$

Then

$$\frac{x+m}{n} = \frac{y+M}{n} = \frac{y+kn+r}{n} = k + \frac{y+r}{n}.$$

Because $0 \leq y < 1$ and $0 \leq r \leq n-1$, we have $0 \leq y + r < n$. Hence

$$\left\lfloor k + \frac{y+r}{n} \right\rfloor = k.$$

But $k = \left\lfloor \frac{M}{n} \right\rfloor = \left\lfloor \frac{\lfloor x \rfloor + m}{n} \right\rfloor$. Therefore

$$\left\lfloor \frac{x+m}{n} \right\rfloor = \left\lfloor \frac{\lfloor x \rfloor + m}{n} \right\rfloor,$$

as required. (ProofWiki)

Useful corollaries and dual form

1. Setting $m = 0$ gives

$$\left\lfloor \frac{x}{n} \right\rfloor = \left\lfloor \frac{\lfloor x \rfloor}{n} \right\rfloor,$$

i.e., only $\lfloor x \rfloor$ matters for the floor of a quotient by $n > 0$. ([ProofWiki][2])

2. By $\lceil z \rceil = -\lfloor -z \rfloor$, a completely analogous ceiling identity holds:

$$\left\lceil \frac{x+m}{n} \right\rceil = \left\lceil \frac{\lceil x \rceil + m}{n} \right\rceil \qquad (n > 0,\ m \in \mathbb{Z},\ x \in \mathbb{R}).$$

3. The exercise is frequently used to simplify nested floors/ceilings in this chapter. For example, Knuth invokes it in §1.2.4 Answer 48(b) when manipulating $\lceil (n + \lceil -n/25 \rceil)/3 \rceil$.

Quick numeric check

Take $x = 17.83$, $m = -10$, $n = 7$.

$$\left\lfloor \frac{x+m}{n} \right\rfloor = \left\lfloor \frac{7.83}{7} \right\rfloor = \lfloor 1.118\ldots \rfloor = 1,$$

$$\left\lfloor \frac{\lfloor x \rfloor + m}{n} \right\rfloor = \left\lfloor \frac{17-10}{7} \right\rfloor = \left\lfloor \frac{7}{7} \right\rfloor = 1.$$

36 [M23]

Part A — $\sum_{k=1}^{n} \lfloor k/2 \rfloor$

We'll split by the parity of n.

Case 1: $n = 2m$ even

Pair consecutive terms $(2j-1, 2j)$ for $j = 1, \ldots, m$:

$$\left\lfloor \frac{2j-1}{2} \right\rfloor + \left\lfloor \frac{2j}{2} \right\rfloor = (j-1) + j = 2j - 1.$$

Summing those pairs gives

$$\sum_{k=1}^{2m} \left\lfloor \frac{k}{2} \right\rfloor = \sum_{j=1}^{m} (2j-1) = m^2 = \frac{(2m)^2}{4} = \left\lfloor \frac{n^2}{4} \right\rfloor.$$

Case 2: $n = 2m+1$ odd

Use the even case up to $2m$ and add the last term:

$$\sum_{k=1}^{2m+1} \left\lfloor \frac{k}{2} \right\rfloor = \underbrace{\sum_{k=1}^{2m} \left\lfloor \frac{k}{2} \right\rfloor}_{=m^2} + \left\lfloor \frac{2m+1}{2} \right\rfloor = m^2 + m = \left\lfloor \frac{(2m+1)^2}{4} \right\rfloor = \left\lfloor \frac{n^2}{4} \right\rfloor.$$

Thus in all cases

$$\boxed{\sum_{k=1}^{n} \left\lfloor \frac{k}{2} \right\rfloor = \left\lfloor \frac{n^2}{4} \right\rfloor.}$$

(For reference, Knuth's answers section derives the same result by splitting into $n = 2t$ and $n = 2t+1$.)

Part B — $\sum_{k=1}^{n} \lceil k/2 \rceil$

Use the elementary identity (true for every integer k):

$$\left\lfloor \frac{k}{2} \right\rfloor + \left\lceil \frac{k}{2} \right\rceil = k.$$

Summing over $k = 1, \dots, n$ yields

$$\sum_{k=1}^{n} \left\lceil \frac{k}{2} \right\rceil = \sum_{k=1}^{n} k - \sum_{k=1}^{n} \left\lfloor \frac{k}{2} \right\rfloor = \frac{n(n+1)}{2} - \left\lfloor \frac{n^2}{4} \right\rfloor.$$

Evaluate by parity:

- If $n = 2m$:

$$\sum_{k=1}^{n} \left\lceil \frac{k}{2} \right\rceil = \frac{2m(2m+1)}{2} - m^2 = m(m+1).$$

- If $n = 2m+1$:

$$\sum_{k=1}^{n} \left\lceil \frac{k}{2} \right\rceil = \frac{(2m+1)(2m+2)}{2} - m(m+1) = (m+1)^2.$$

Both cases unify neatly as

$$\boxed{\sum_{k=1}^{n} \left\lceil \frac{k}{2} \right\rceil = \left\lceil \frac{n(n+2)}{4} \right\rceil.}$$

(This is also the closed form reported in the book's answers.)

Quick sanity checks

- $n = 5$: $\lfloor k/2 \rfloor$ sums to $0 + 1 + 1 + 2 + 2 = 6 = \lfloor 25/4 \rfloor$; $\lceil k/2 \rceil$ sums to $1 + 1 + 2 + 2 + 3 = 9 = \lceil 5 \cdot 7/4 \rceil$.
- $n = 6$: $\lfloor k/2 \rfloor$ sums to $0+1+1+2+2+3 = 9 = 6^2/4$; $\lceil k/2 \rceil$ sums to $1+1+2+2+3+3 = 12 = \lceil 6 \cdot 8/4 \rceil$.

All consistent.

37 [M30]

Step 1: Separate floors into value minus fractional part

$$\sum_{k=0}^{n-1} \left\lfloor \frac{mk+x}{n} \right\rfloor = \sum_{k=0}^{n-1} \left(\frac{mk+x}{n} - \left\{ \frac{mk+x}{n} \right\} \right) = \frac{m(n-1)}{2} + x - S,$$

where

$$S = \sum_{k=0}^{n-1} \left\{ \frac{mk+x}{n} \right\}.$$

Step 2: Structure of the fractional parts

Put $t = n/d$, $u = m/d$ so $\gcd(u, t) = 1$. The multiset of fractional parts over one period $k = 0, \dots, t-1$ is

$$\left\{ \frac{x}{n} + \frac{u\,k}{t} \right\}, \qquad k = 0, \dots, t-1.$$

Because multiplication by u permutes the residues modulo t, this set is the same as

$$\left\{ \frac{x \bmod d}{n} + \frac{r}{t} \right\}, \qquad r = 0, \dots, t-1.$$

Crucially, $\dfrac{x \bmod d}{n} < \dfrac{d}{n} = \dfrac{1}{t}$, so no term wraps around 1. Hence

$$\sum_{r=0}^{t-1}\left\{\frac{x \bmod d}{n}+\frac{r}{t}\right\}=t\cdot\frac{x \bmod d}{n}+\sum_{r=0}^{t-1}\frac{r}{t}=\frac{t(x \bmod d)}{n}+\frac{t-1}{2}.$$

There are d identical blocks of length t, so

$$S=d\left(\frac{t(x \bmod d)}{n}+\frac{t-1}{2}\right)=(x \bmod d)+\frac{n-d}{2}.$$

Step 3: Assemble the floor sum

Plugging S back:

$$\sum_{k=0}^{n-1}\left\lfloor\frac{mk+x}{n}\right\rfloor=\frac{m(n-1)}{2}+x-\left((x \bmod d)+\frac{n-d}{2}\right)=\frac{m(n-1)-(n-d)}{2}+d\left\lfloor\frac{x}{d}\right\rfloor.$$

Two convenient equivalent forms are

$$\boxed{\sum_{k=0}^{n-1}\left\lfloor\frac{mk+x}{n}\right\rfloor=\frac{(m-1)(n-1)+(d-1)}{2}+d\left\lfloor\frac{x}{d}\right\rfloor}$$

and

$$\sum_{k=0}^{n-1}\left\lfloor\frac{mk+x}{n}\right\rfloor=\frac{m(n-1)}{2}-\frac{n-d}{2}+d\left\lfloor\frac{x}{d}\right\rfloor.$$

(Optional) Ceiling version

Using $\lceil y\rceil=-\lfloor -y\rfloor$ (or $\lceil y\rceil=\lfloor y\rfloor+[\{y\}>0]$), one gets the companion identity

$$\boxed{\sum_{k=0}^{n-1}\left\lceil\frac{mk+x}{n}\right\rceil=\frac{(m+1)(n-1)-(d-1)}{2}+d\left\lceil\frac{x}{d}\right\rceil}$$

which matches Knuth's "similar identity."

Checks and special cases

- If $\gcd(m,n)=1$ ($d=1$):

$$\sum_{k=0}^{n-1} \left\lfloor \frac{mk+x}{n} \right\rfloor = \frac{(m-1)(n-1)}{2} + \lfloor x \rfloor,$$

the classical result.

- Taking $x = 0$ gives

$$\sum_{k=0}^{n-1} \left\lfloor \frac{mk}{n} \right\rfloor = \frac{(m-1)(n-1)+(d-1)}{2}.$$

38 [M26]

Let

$$a = \lfloor x \rfloor, \qquad \theta = \{x\} = x - \lfloor x \rfloor \in [0,1), \qquad c = \lceil y \rceil \in \mathbb{Z}.$$

The summation index set is $\{\, k \in \mathbb{Z} \mid 0 \le k < y \,\}$. Since $y > 0$, those are precisely the integers $k = 0, 1, \dots, c-1$ (when y is not an integer this runs up to $\lfloor y \rfloor = c-1$; when y is an integer n, this runs up to $n-1 = c-1$). Thus the number of terms is $c = \lceil y \rceil$.

Because $0 \le k/y < 1$ for all such k, we have

$$\left\lfloor x + \frac{k}{y} \right\rfloor = \left\lfloor a + \theta + \frac{k}{y} \right\rfloor = a + \left\lfloor \theta + \frac{k}{y} \right\rfloor,$$

and $\lfloor \theta + \frac{k}{y} \rfloor$ is 1 exactly when $\theta + \frac{k}{y} \ge 1$, i.e. when $k \ge y(1-\theta)$; otherwise it is 0.

Hence

$$S := \sum_{0 \le k < y} \left\lfloor x + \frac{k}{y} \right\rfloor = \sum_{k=0}^{c-1} \left(a + \left\lfloor \theta + \frac{k}{y} \right\rfloor \right) = ac + N,$$

where N is the count of integers $k \in \{0, 1, \dots, c-1\}$ with $k \ge y(1-\theta)$. The smallest such k is $\lceil y(1-\theta) \rceil$, so

$$N = \#\{\lceil y(1-\theta) \rceil,\ \lceil y(1-\theta) \rceil + 1, \dots, c-1\} = c - \lceil y(1-\theta) \rceil.$$

Therefore

$$S = ac + (c - \lceil y(1-\theta) \rceil) = (a+1)c - \lceil y(1-\theta) \rceil. \qquad (\ )$$

Now rewrite the right-hand side of the exercise's formula:

$$xy + \lfloor x+1 \rfloor (\lceil y \rceil - y) = xy + (a+1)(c-y) = (a+1)c - y(1-\theta).$$

Since $(a+1)c$ is an integer,

$$\Big\lfloor (a+1)c - y(1-\theta) \Big\rfloor = (a+1)c - \Big\lceil y(1-\theta) \Big\rceil,$$

and by ( ) this equals S. This proves the general identity.

Special case $y = n \in \mathbb{Z}_{>0}$

When $y = n$, we have $c = \lceil y \rceil = n$ and ( ) becomes

$$S = (a+1)n - \lceil n(1-\theta) \rceil.$$

Using $\lceil n(1-\theta) \rceil = n - \lfloor n\theta \rfloor$ (valid for $\theta \in [0,1)$), we get

$$S = (a+1)n - (n - \lfloor n\theta \rfloor) = an + \lfloor n\theta \rfloor = \lfloor na + n\theta \rfloor = \lfloor nx \rfloor,$$

which is the "in particular" statement and shows that $\lfloor \cdot \rfloor$ is **replicative** (the notion defined in the very next exercise).

Remarks

- The identity immediately implies the replicative property for the floor function:

  $$\sum_{k=0}^{n-1} \Big\lfloor x + \frac{k}{n} \Big\rfloor = \lfloor nx \rfloor \quad (n \in \mathbb{Z}_{>0}),$$

  which Exercise 39 then uses to develop the general theory of **replicative functions**.

- The proof strategy is a standard "count the carries" argument: split x into integer and fractional parts and count how many increments push $\theta + k/y$ over the next integer threshold.

39 [HM35]

Definition (replicative)

A function f is **replicative** if, for every positive integer n,

$$\sum_{k=0}^{n-1} f\Big(x+\frac{k}{n}\Big) = f(nx) \qquad \text{for all real } x.$$

(Exercise 38 just before this one shows that $x \mapsto \lfloor x \rfloor$ is replicative via the identity $\sum_{k=0}^{n-1} \lfloor x + k/n \rfloor = \lfloor nx \rfloor$.)

(a) $f(x) = x - \frac{1}{2}$

$$\sum_{k=0}^{n-1}\Big(x - \tfrac{1}{2} + \frac{k}{n}\Big) = n\big(x - \tfrac{1}{2}\big) + \frac{1}{n}\sum_{k=0}^{n-1} k = n\big(x - \tfrac{1}{2}\big) + \frac{n-1}{2} = nx - \tfrac{1}{2} = f(nx).$$

So f is replicative.

(b) $f(x) = [x \text{ is an integer}]$ (Iverson bracket)

The left side counts the number of $k \in \{0, \dots, n-1\}$ such that $x + \frac{k}{n} \in \mathbb{Z}$. This occurs iff $nx \in \mathbb{Z}$: indeed, $x + \frac{k}{n} = m \in \mathbb{Z} \iff nx + k = nm$ for some m, i.e., $nx \in \mathbb{Z}$; and when $nx \in \mathbb{Z}$ there is exactly one such k modulo n. Hence the sum equals $[nx \in \mathbb{Z}] = f(nx)$.

(c) $f(x) = [x \text{ is a positive integer}]$

As in (b), there is at most one k giving an integer; its value is $m = (nx + k)/n$. That integer is **positive** iff $nx > 0$ (when $nx = 0$ we get $m = 0$; when $nx < 0$ we get $m \le 0$). Therefore

$$\sum_k [x + k/n \in \mathbb{Z}_{>0}] \;=\; [nx \in \mathbb{Z}_{>0}] \;=\; f(nx).$$

(d) $f(x) = [\,\exists\, r \in \mathbb{Q},\ m \in \mathbb{Z} :\ x = r\pi + m\,]$

(f indicates "x is a rational multiple of π up to an integer.")

If $x + \frac{k}{n} = r\pi + m$, multiply by n to get $nx + k = nr\pi + nm$, i.e., $nx = r'\pi + m'$ with $r' = nr \in \mathbb{Q}$, $m' \in \mathbb{Z}$. Thus $f(nx) = 1$.

Conversely, if $nx = r'\pi + m'$ with $r' \in \mathbb{Q}$, choose the unique $k \in \{0, \dots, n-1\}$ with $k \equiv -m' \pmod{n}$. Then

$$x + \frac{k}{n} = \frac{nx + k}{n} = \frac{r'\pi + m' + k}{n} = \Big(\frac{r'}{n}\Big)\pi + \frac{m' + k}{n},$$

and since $m' + k$ is divisible by n, this equals $r\pi + m$ with $r = r'/n \in \mathbb{Q}$, $m \in \mathbb{Z}$. Uniqueness of k again gives the sum $= f(nx)$. Hence f is replicative.

(e) Three variants of (d) with positivity restrictions

The same argument works unchanged for each of the following examples (any one of which meets the "three other functions" request):

1. $f(x) = [\,\exists\, r \in \mathbb{Q}_{>0},\, m \in \mathbb{Z} :\ x = r\pi + m\,]$
2. $f(x) = [\,\exists\, r \in \mathbb{Q},\, m \in \mathbb{Z}_{>0} :\ x = r\pi + m\,]$
3. $f(x) = [\,\exists\, r \in \mathbb{Q}_{>0},\, m \in \mathbb{Z}_{>0} :\ x = r\pi + m\,]$

If $nx = r'\pi + m'$ with the stated positivity on r' and/or m', the construction above picks a k so that $(m' + k)/n$ is an integer; since $m' > 0$ implies $(m' + k)/n > 0$, and $r' > 0$ implies $(r'/n) > 0$, the corresponding r, m inherit the required positivity. Thus each such indicator is replicative.

(f) $f(x) = \log|2\sin(\pi x)|$ (allowing $f(x) = -\infty$ at integers)

Use the cyclotomic factorization:

$$\prod_{k=0}^{n-1} \left| e^{2\pi i x} - e^{2\pi i k/n} \right| = \left| e^{2\pi i n x} - 1 \right|.$$

But $|e^{2\pi i u} - 1| = 2|\sin(\pi u)|$, and $|e^{2\pi i x} - e^{2\pi i k/n}| = 2\big|\sin(\pi(x - k/n))\big|$. Since $\{k/n\}$ runs over the same set as $\{-k/n\}$,

$$\prod_{k=0}^{n-1} |2\sin\pi(x + k/n)| = |2\sin(\pi n x)|.$$

Taking logs gives $\sum_{k=0}^{n-1} \log|2\sin\pi(x + k/n)| = \log|2\sin(\pi n x)| = f(nx)$. Hence f is replicative.

(g) Sum of two replicative functions

If f, g are replicative, then

$$\sum_k (f+g)\left(x + \frac{k}{n}\right) = \sum_k f\left(x + \frac{k}{n}\right) + \sum_k g\left(x + \frac{k}{n}\right) = f(nx) + g(nx) = (f+g)(nx).$$

So any sum of replicative functions is replicative.

(h) Constant multiple of a replicative function

If f is replicative and c is constant,

$$\sum_k (cf)\Big(x+\frac{k}{n}\Big) = c\sum_k f\Big(x+\frac{k}{n}\Big) = c\,f(nx) = (cf)(nx).$$

So cf is replicative.

(i) $g(x) = f(x - \lfloor x \rfloor)$ when f is replicative

Write $x = t + m$ with $m = \lfloor x \rfloor$ and $t = \{x\} \in [0, 1)$. Let $r = \lfloor nt \rfloor$ (the number of indices k for which $t + k/n \geq 1$).

Then the multiset

$$\Big\{t+\frac{0}{n},\ t+\frac{1}{n},\ \dots,\ t+\frac{n-1}{n}\Big\}$$

has exactly r values ≥ 1. If we reduce those r values by 1 (i.e., take fractional parts), we obtain precisely the re-ordered arithmetic progression

$$\Big\{t-\frac{r}{n}+\frac{j}{n}\ :\ j=0,1,\dots,n-1\Big\}.$$

Therefore

$$\sum_{k=0}^{n-1} g\Big(x+\frac{k}{n}\Big) = \sum_{k=0}^{n-1} f\big(\{t+\tfrac{k}{n}\}\big) = \sum_{j=0}^{n-1} f\big(t-\tfrac{r}{n}+\tfrac{j}{n}\big) = f\Big(n\Big(t-\tfrac{r}{n}\Big)\Big) = f(nt-r).$$

But $nt - r = nt - \lfloor nt \rfloor = \{nx\}$, so the right-hand side is

$$f(\{nx\}) = f(nx - \lfloor nx \rfloor) = g(nx),$$

proving g is replicative.

40 [HM46]

Setup: what "replicative" means

A function $f : \mathbb{R} \to \mathbb{R}$ is called **replicative** if for every positive integer n,

$$f(nx) \;=\; \sum_{k=0}^{n-1} f\Big(x + \frac{k}{n}\Big) \qquad (\forall x \in \mathbb{R}).$$

Exercise 39 gave many examples and in particular showed $f(x) = x - \frac{1}{2}$ is replicative.

Exercise 40 asks you to "study the class of replicative functions" and—among other things—decide whether the function in 39(a) is the only continuous replicative function.

The official answer (a note by N. G. de Bruijn) gives a very slick route via Fourier series; below I present that argument cleanly.

Step 1: a key telescoping identity

From the replicative law,

$$f(nx+1) = f\big(n\big(x + \tfrac{1}{n}\big)\big) = \sum_{k=0}^{n-1} f\big(x + \tfrac{1}{n} + \tfrac{k}{n}\big) = \sum_{k=1}^{n} f\big(x + \tfrac{k}{n}\big)\,.$$

Subtracting $f(nx) = \sum_{k=0}^{n-1} f\big(x + \frac{k}{n}\big)$ gives the **telescoping difference formula**

$$f(nx+1) - f(nx) \;=\; f(x+1) - f(x) \qquad (\forall n \geq 1,\ \forall x \in \mathbb{R}). \tag{ }$$

(This is the first observation in de Bruijn's note.)

Step 2: continuity forces a constant unit-step

Define $D(x) := f(x+1) - f(x)$. From ( ) we get

$$D(nx) = D(x) \qquad (\forall n \geq 1,\ \forall x).$$

Fix y. Then $D(y) = D(y/n)$ for all n. If f is **continuous**, then D is continuous, hence

$$D(y) = \lim_{n\to\infty} D(y/n) = D(0),$$

so D is constant: there exists $c \in \mathbb{R}$ with

$$f(x+1) - f(x) = c \qquad (\forall x \in \mathbb{R}). \tag{1}$$

(Exactly as stated in de Bruijn's note.)

Step 3: reduce to a 1-periodic replicative function

Let

$$g(x) := f(x) - c\lfloor x \rfloor.$$

Then g is **1-periodic** by (1): $g(x+1) - g(x) = (f(x+1) - f(x)) - c(\lfloor x+1 \rfloor - \lfloor x \rfloor) = c - c = 0$. Moreover g is still **replicative** because both f and $\lfloor x \rfloor$ are replicative (the latter is the content of Exercise 38), and the defining relation is linear. Hence

$$g(nx) = \sum_{k=0}^{n-1} g\Big(x + \frac{k}{n}\Big), \qquad g(x+1) = g(x). \tag{2}$$

(These are exactly the properties de Bruijn isolates.)

Averaging (2) over one period shows $\int_0^1 g(x)\,dx = 0$ (the left side gives $n\int_0^1 g$, the right side also gives $n\int_0^1 g$ after a change of variables, forcing the mean to be 0).

Let $\hat{g}(n) := \int_0^1 e^{2\pi i n x}\, g(x)\,dx$ be the Fourier coefficients. From (2) one obtains (change variables in the integral and use periodicity)

$$\hat{g}(n) = \frac{1}{n}\,\hat{g}(1) \quad \text{for every integer } n \neq 0, \qquad \hat{g}(0) = 0. \tag{3}$$

This is the integral identity de Bruijn writes down, and it is the load-bearing step that ties g's Fourier coefficients to a $1/n$ law.

Step 4: identify the Fourier series

Equations (3) say the spectrum of g is exactly the spectrum of a (scaled) **sawtooth** wave:

$$g(x) \;=\; a \sum_{n \neq 0} \frac{e^{2\pi i n x}}{2\pi i n} \;=\; a\Big(\{x\} - \tfrac{1}{2}\Big) \quad \text{for } 0 < x < 1,$$

for some real constant a (the value at integers can be chosen to make g periodic). In other words,

$$g(x) = a(\{x\} - \tfrac{1}{2}) \quad (0 < x < 1),$$

exactly as stated in the book's answer.

Step 5: put f back together and enforce continuity

Recall $f(x) = g(x) + c\lfloor x \rfloor$. On $(0,1)$ this gives

$$f(x) = a\Big(x - \tfrac{1}{2}\Big) + c \cdot 0 = a\Big(x - \tfrac{1}{2}\Big).$$

Across an integer, g jumps by $-a$ while $\lfloor x \rfloor$ jumps by $+1$, so the jump of f is $c - a$. Since f is **continuous**, that jump must vanish; hence $c = a$. Therefore

$$f(x) = a(\{x\} - \tfrac{1}{2}) + a\lfloor x \rfloor = a\Big(\lfloor x \rfloor + \{x\} - \tfrac{1}{2}\Big) = a\Big(x - \tfrac{1}{2}\Big) \quad (\forall x).$$

Conclusion (classification for the continuous case)

The only **continuous** replicative functions are exactly

$$\boxed{f(x) = A(x - \tfrac{1}{2})} \quad \text{with } A \in \mathbb{R}.$$

This answers the "for example" part of Exercise 40: **yes**—up to a constant multiple, the function in 39(a) is the only continuous replicative function.

Remark on the generalized family in the problem text

Exercise 40 also invites you to consider the broader identity

$$\sum_{k=0}^{n-1} f\Big(x + \frac{k}{n}\Big) = a_n\, f(nx) + b_n \qquad (n \geq 1),$$

whose derivatives (and, if $b_n = 0$, integrals) stay in the same class. The statement points out that with $b_n = 0$ this includes **Bernoulli polynomials** and trigonometric functions like $\cot(\pi x)$ and $\csc^2(\pi x)$, and with $b_n \neq 0$ it includes the **psi** function; it also mentions Hurwitz's generalized zeta $\zeta(s, x)$ for fixed s. These are standard identities of the form $\sum_{k=0}^{n-1} \phi(x + k/n) = a_n\, \phi(nx) + b_n$ and motivate the term "replicative."

If you'd like, I can work out one or two of those generalized examples in detail (e.g., $\sum \cot(\pi(x + k/n)) = n\, \cot(\pi n x)$) and show exactly how the same Fourier-style reasoning fits.

41 [M24]

1) Locate the index range where $a_n = k$

The first k blocks contain

$$1 + 2 + \cdots + k \;=\; \frac{k(k+1)}{2}$$

terms, i.e. up to index $T_k := \frac{k(k+1)}{2}$ (the k-th triangular number). Therefore $a_n = k$ precisely when n lies after the $(k-1)$-block and at or before the end of the k-block:

$$T_{k-1} \;<\; n \;\le\; T_k \quad \Longleftrightarrow \quad \frac{k(k-1)}{2} < n \le \frac{k(k+1)}{2}. \text{ :contentReference[oaicite:0]index=0}$$

2) Isolate k from the inequality

Multiply by 2:

$$k^2 - k \;<\; 2n \;\le\; k^2 + k.$$

Add $\frac{1}{4}$ throughout (to complete the square):

$$\left(k - \tfrac{1}{2}\right)^2 \;<\; 2n + \tfrac{1}{4} \;\le\; \left(k + \tfrac{1}{2}\right)^2.$$

Take square roots (all quantities are nonnegative):

$$k - \tfrac{1}{2} \;<\; \sqrt{2n + \tfrac{1}{4}} \;\le\; k + \tfrac{1}{2}.$$

Since $\sqrt{2n + \frac{1}{4}} = \sqrt{2n} + (\text{tiny correction})$ and n is an integer, this shows $\sqrt{2n}$ lies within $\frac{1}{2}$ of the integer k. Thus k **is the nearest integer to** $\sqrt{2n}$, i.e.

$$\boxed{a_n \;=\; \left\lfloor \sqrt{2n} + \tfrac{1}{2} \right\rfloor}.$$

3) Checks and alternative forms

- Quick check on first terms:
  - $n = 1:\ \lfloor \sqrt{2} + 1/2 \rfloor = \lfloor 1.414\ldots + 0.5 \rfloor = 1.$
  - $n = 2, 3:\ \lfloor \sqrt{4,6} + 1/2 \rfloor = 2, 2.$

– $n = 4, 5, 6:\ \lfloor\sqrt{8, 10, 12} + 1/2\rfloor = 3, 3, 3.$ Matches $1,\ 2, 2,\ 3, 3, 3, \ldots$

- An equivalent characterization from the triangular numbers inequality is

$$a_n = k \iff T_{k-1} < n \le T_k \quad \text{with } T_k = \frac{k(k+1)}{2},$$

which you can also invert with the quadratic formula to get

$$a_n \;=\; \left\lfloor \frac{1+\sqrt{1+8n}}{2} \right\rfloor \quad \text{or} \quad a_n \;=\; \left\lceil \frac{-1+\sqrt{1+8n}}{2} \right\rceil,$$

and these agree with $\lfloor\sqrt{2n} + \frac{1}{2}\rfloor$ for all integers $n \ge 1$.

42 [M24]

Part (a): the identity

Start from the right-hand side and expand:

$$\begin{aligned}
na_n - \sum_{k=1}^{n-1} k(a_{k+1} - a_k) &= na_n - \sum_{k=1}^{n-1} ka_{k+1} + \sum_{k=1}^{n-1} ka_k \\
&= na_n - \sum_{j=2}^{n} (j-1)a_j + \sum_{k=1}^{n-1} ka_k \quad (j = k+1) \\
&= \left[na_n - (n-1)a_n\right] + \sum_{k=2}^{n-1} (-(k-1) + k)a_k + \ a_1 \\
&= a_n + \sum_{k=2}^{n-1} a_k + a_1 \ = \ \sum_{k=1}^{n} a_k.
\end{aligned}$$

So the formula holds for all $n > 0$. (This is the discrete "summation by parts"/Abel transform for the special case where the other factor is k.)

Part (b): the floor–log sum

Let $a_k = \lfloor \log_b k \rfloor$ and put $m = \lfloor \log_b n \rfloor$. Then $b^m \le n < b^{m+1}$.

Observe the increments:

$$\Delta a_k \;=\; a_{k+1} - a_k = \begin{cases} 1, & \text{if } k = b^j - 1 \text{ for some } j = 1, 2, \ldots, m, \\ 0, & \text{otherwise.} \end{cases}$$

Hence

$$\sum_{k=1}^{n-1} k\,\Delta a_k = \sum_{j=1}^{m}(b^j - 1) = \Big(\sum_{j=1}^{m} b^j\Big) - m = \frac{b^{m+1} - b}{b-1} - m.$$

Now apply part (a):

$$\sum_{k=1}^{n} \lfloor \log_b k \rfloor = n\,a_n - \sum_{k=1}^{n-1} k\,\Delta a_k = nm - \Big(\frac{b^{m+1} - b}{b-1} - m\Big) = (n+1)m - \frac{b^{m+1} - b}{b-1}.$$

Recalling $m = \lfloor \log_b n \rfloor$, this is exactly

$$\boxed{\sum_{k=1}^{n} \lfloor \log_b k \rfloor = (n+1)\lfloor \log_b n \rfloor - \frac{b^{\lfloor \log_b n \rfloor + 1} - b}{b-1}}$$

as required.

Quick check (sanity): Take $b = 10$, $n = 20$. LHS counts the number of digits minus 1 for each $1 \le k \le 20$: there are 9 with value 0 and 11 with value 1, so sum $= 11$. RHS gives $(20+1)\cdot 1 - (10^2 - 10)/9 = 21 - 10 = 11$. Works.

43 [M23]

Let $t = \lfloor \sqrt{n} \rfloor$. For each integer $m \ge 1$, the values of k with $\lfloor \sqrt{k} \rfloor = m$ are exactly

$$k = m^2,\ m^2 + 1,\ \dots,\ (m+1)^2 - 1.$$

There are $(m+1)^2 - m^2 = 2m + 1$ such k's when the whole block fits inside $[1, n]$.

All full blocks occur for $m = 1, 2, \dots, t-1$. The last (possibly partial) block is $m = t$, contributing $n - t^2 + 1$ terms (from $k = t^2$ through $k = n$).

Therefore

$$S(n) = \sum_{m=1}^{t-1} m\,(2m+1)\ +\ t\,(n - t^2 + 1).$$

Compute the closed form:

$$\sum_{m=1}^{t-1} m(2m+1) = 2\sum_{m=1}^{t-1} m^2 + \sum_{m=1}^{t-1} m = 2\cdot\frac{(t-1)t(2t-1)}{6} + \frac{t(t-1)}{2}.$$

Hence

$$S(n) = \frac{(t-1)t(2t-1)}{3} + \frac{t(t-1)}{2} + t(n - t^2 + 1).$$

A clean single-line form (still with $t = \lfloor \sqrt{n} \rfloor$) is

$$\boxed{S(n) = \frac{t(6n - 2t^2 - 3t + 5)}{6}}$$

which is equivalent to

$$S(n) = \frac{t(t-1)(4t+1)}{6} + t\,(n - t^2 + 1).$$

Quick check

For $n = 10$, $t = \lfloor \sqrt{10} \rfloor = 3$:

$$S(10) = \frac{3(60 - 18 - 9 + 5)}{6} = \frac{3 \cdot 38}{6} = 19,$$

matching $1, 1, 1, 2, 2, 2, 2, 2, 3, 3$ summed $= 19$.

Asymptotics (nice to know)

Since $t = \lfloor \sqrt{n} \rfloor \sim \sqrt{n}$,

$$S(n) = \frac{2}{3}\, n^{3/2} + O(n),$$

which you can also see directly by approximating the area under $y = \sqrt{x}$.

44 [M24]

Step 1: A handy floor-sum identity

For any integers a and $b \geq 1$,

$$\sum_{j=0}^{b-1} \left\lfloor \frac{a+j}{b} \right\rfloor \;=\; a.$$

Proof. Write $a = qb + r$ with $0 \leq r < b$. Then

$$\left\lfloor \frac{a+j}{b} \right\rfloor = \left\lfloor q + \frac{r+j}{b} \right\rfloor = \begin{cases} q, & j = 0, 1, \dots, b-r-1, \\ q+1, & j = b-r, \dots, b-1. \end{cases}$$

Summing gives $q(b-r) + (q+1)r = qb + r = a$.

Step 2: Evaluate the inner sum for a fixed k

Fix $k \geq 0$. Perform Euclidean division of n by b^k:

$$n \;=\; a_k\, b^k + r_k, \qquad a_k = \left\lfloor \frac{n}{b^k} \right\rfloor,\; 0 \leq r_k < b^k.$$

Then for any j,

$$\frac{n + j\, b^k}{b^{k+1}} = \frac{a_k b^k + r_k + jb^k}{b^{k+1}} = \frac{a_k + j}{b} + \frac{r_k}{b^{k+1}}.$$

Note that $0 \leq \dfrac{r_k}{b^{k+1}} < \dfrac{1}{b}$. Therefore, when we sum over a full block $j = 0, 1, \dots, b-1$, the tiny offset $\dfrac{r_k}{b^{k+1}}$ never causes an extra carry; hence

$$\sum_{j=0}^{b-1} \left\lfloor \frac{n + j\, b^k}{b^{k+1}} \right\rfloor = \sum_{j=0}^{b-1} \left\lfloor \frac{a_k + j}{b} + \frac{r_k}{b^{k+1}} \right\rfloor = \sum_{j=0}^{b-1} \left\lfloor \frac{a_k + j}{b} \right\rfloor = a_k = \left\lfloor \frac{n}{b^k} \right\rfloor,$$

using Step 1 in the last equality.

Since the exercise's inner sum runs over $j = 1, \dots, b-1$, just subtract the $j = 0$ term:

$$\sum_{j=1}^{b-1} \left\lfloor \frac{n + j\, b^k}{b^{k+1}} \right\rfloor = \left\lfloor \frac{n}{b^k} \right\rfloor \;-\; \left\lfloor \frac{n}{b^{k+1}} \right\rfloor. \qquad (\;)$$

Step 3: Telescope over k

Now sum ( ) over $k \geq 0$:

$$\sum_{k\geq 0}\sum_{j=1}^{b-1}\left\lfloor \frac{n+j\,b^k}{b^{k+1}}\right\rfloor = \sum_{k\geq 0}\left(\left\lfloor \frac{n}{b^k}\right\rfloor - \left\lfloor \frac{n}{b^{k+1}}\right\rfloor\right) = \left\lfloor \frac{n}{b^0}\right\rfloor - \lim_{K\to\infty}\left\lfloor \frac{n}{b^{K+1}}\right\rfloor.$$

Since n is an integer, $\lfloor n/b^0 \rfloor = n$. For the tail term:

- If $n \geq 0$, then $0 \leq n/b^{K+1} < 1$ for large K, so $\lfloor n/b^{K+1} \rfloor = 0$.
- If $n < 0$, then $-1 < n/b^{K+1} < 0$ for large K, so $\lfloor n/b^{K+1} \rfloor = -1$.

Thus

$$\sum_{k\geq 0}\sum_{1\leq j<b}\left\lfloor \frac{n+j\,b^k}{b^{k+1}}\right\rfloor = \begin{cases} n, & n \geq 0, \\ n+1, & n < 0. \end{cases}$$

That proves the required identity for $n \geq 0$, and answers the second question: for $n < 0$ the value is $n + 1$.

Intuition (base-b view)

Write $n = \sum_{k\geq 0} d_k b^k$ with digits $0 \leq d_k < b$ (for $n \geq 0$). For fixed k, adding jb^k with $1 \leq j < b$ shifts by j in the k-th digit and contributes $\lfloor (n + jb^k)/b^{k+1} \rfloor$ to the $(k+1)$-st and higher digits—i.e., the number of carries past position k. As j ranges from 1 to $b-1$, you get exactly d_k carries. Summing over k counts all digit contributions and reconstructs n. For negative n, the infinite "tail" of floors contributes an extra $+1$, giving $n + 1$.

45 [M28]

Proof of the general identity (1)

Define, for each $r \in \{0, 1, \dots, m-1\}$,

$$J_r \;=\; \left\{ j \in \{0, 1, \dots, n-1\} \;\middle|\; \left\lfloor \frac{mj}{n} \right\rfloor = r \right\}.$$

Then

$$\sum_{0\leq j<n} f\!\left(\left\lfloor \frac{mj}{n}\right\rfloor\right) \;=\; \sum_{r=0}^{m-1} f(r)\,|J_r|.$$

Characterize $|J_r|$. The condition $\lfloor mj/n \rfloor = r$ is equivalent to

$$r \le \frac{mj}{n} < r+1 \quad \Longleftrightarrow \quad \frac{nr}{m} \le j < \frac{n(r+1)}{m}.$$

Among integers $j \in [0, n-1]$, that count is

$$|J_r| = \left\lceil \frac{n(r+1)}{m} \right\rceil - \left\lceil \frac{nr}{m} \right\rceil = A_{r+1} - A_r, \quad \text{where } A_r := \left\lceil \frac{nr}{m} \right\rceil,\ A_0 = 0,\ A_m = n.$$

Hence

$$\sum_{0 \le j < n} f\left(\left\lfloor \frac{mj}{n} \right\rfloor \right) = \sum_{r=0}^{m-1} f(r)\,(A_{r+1} - A_r) = \sum_{r=0}^{m-1} (f(r)A_{r+1} - f(r)A_r).$$

Now apply **summation by parts** (a discrete telescoping trick). Shift index in the first term:

$$\sum_{r=0}^{m-1} f(r)A_{r+1} = \sum_{r=1}^{m} f(r-1)A_r = f(m-1)A_m + \sum_{r=1}^{m-1} f(r-1)A_r = n\,f(m-1) + \sum_{r=1}^{m-1} f(r-1)A_r.$$

Therefore

$$\sum_{r=0}^{m-1} f(r)\,(A_{r+1} - A_r) = n\,f(m-1) + \sum_{r=1}^{m-1} (f(r-1) - f(r))A_r = n\,f(m-1) + \sum_{r=0}^{m-1} \left\lceil \frac{nr}{m} \right\rceil (f(r-1) - f(r)),$$

since the $r = 0$ term has coefficient $A_0 = \lceil 0 \rceil = 0$. This is exactly (1).

"In particular" binomial identity (2)

Take

$$f(r) = \binom{r+1}{k} \qquad (k \text{ fixed}).$$

Then by Pascal's rule,

$$f(r-1) - f(r) = \binom{r}{k} - \binom{r+1}{k} = -\binom{r}{k-1}.$$

Plugging into (1) gives

$$\sum_{0\le j<n}\binom{\lfloor mj/n\rfloor+1}{k} \;=\; -\sum_{0\le r<m}\left\lceil\frac{rn}{m}\right\rceil\binom{r}{k-1} \;+\; n\binom{m}{k}.$$

Move the sum to the left:

$$\sum_{0\le j<n}\binom{\lfloor mj/n\rfloor+1}{k} \;+\; \sum_{0\le r<m}\left\lceil\frac{rn}{m}\right\rceil\binom{r}{k-1} \;=\; n\binom{m}{k},$$

which is exactly the required identity (2).

Remark on the "reciprocity" for sums of floors

Exercise 37 evaluates $\sum_{0\le k<n}\lfloor\frac{mk+x}{n}\rfloor$ in a form symmetric in m and n (it involves $\gcd(m,n)$ and $\lfloor x/\gcd(m,n)\rfloor$). Hence swapping m and n gives the same value, implying

$$\sum_{0\le k<n}\left\lfloor\frac{mk+x}{n}\right\rfloor = \sum_{0\le k<m}\left\lfloor\frac{nk+x}{m}\right\rfloor,$$

as noted right before (1).

46 [M29]

Let

$$a_j \stackrel{\text{def}}{=} \left\lfloor\frac{mj}{n}\right\rfloor, \qquad J \stackrel{\text{def}}{=} \{\, j\in\mathbb{Z} : 0\le j<\alpha n\,\} = \{0,1,\dots,\lceil\alpha n\rceil-1\}.$$

Note that a_j is nondecreasing in j and, since $j<\alpha n$,

$$a_j \le \left\lfloor\frac{m(\alpha n)}{n}\right\rfloor < \alpha m \quad\Rightarrow\quad a_j\in\{0,1,\dots,\lceil\alpha m\rceil-1\}.$$

We use a top-down telescoping (a discrete summation-by-parts trick). Fix

$$L \stackrel{\text{def}}{=} \lceil\alpha m\rceil.$$

For any integer $a\le L-1$,

$$f(a) \;=\; f(L-1) + \sum_{r=a+1}^{L-1}\big(f(r-1)-f(r)\big).$$

Apply this with $a = a_j$ and sum over all $j \in J$:

$$\sum_{j\in J} f(a_j) = \sum_{j\in J} f(L-1) \;+\; \sum_{j\in J} \sum_{r=a_j+1}^{L-1} (f(r-1) - f(r)).$$

Swap the order of summation in the double sum:

$$\sum_{j\in J} f(a_j) = |J| \cdot f(L-1) \;+\; \sum_{r=1}^{L-1} \Big(\#\{\, j \in J : a_j \le r-1 \,\}\Big)\,(f(r-1) - f(r)).$$

Now count $\#\{j \in J : a_j \le r-1\}$.

$$a_j \le r-1 \iff \left\lfloor \frac{mj}{n} \right\rfloor \le r-1 \iff \frac{mj}{n} < r \iff j < \frac{rn}{m}.$$

Among $J = \{0, 1, \dots, \lceil \alpha n \rceil - 1\}$, the count of integers j with $j < \frac{rn}{m}$ is

$$\#\{\, j \in J : a_j \le r-1 \,\} = \left\lceil \frac{rn}{m} \right\rceil,$$

because $r \le L - 1 = \lceil \alpha m \rceil - 1$ guarantees $\lceil \frac{rn}{m} \rceil \le \lceil \alpha n \rceil$, so the upper truncation by J does not bind.

Also, $|J| = \lceil \alpha n \rceil$ and $L - 1 = \lceil \alpha m \rceil - 1$. Therefore

$$\sum_{0\le j<\alpha n} f\Big(\left\lfloor \frac{mj}{n} \right\rfloor\Big) = \lceil \alpha n \rceil\, f(\lceil \alpha m \rceil - 1) + \sum_{r=1}^{\lceil \alpha m \rceil - 1} \left\lceil \frac{rn}{m} \right\rceil (f(r-1) - f(r)),$$

which is exactly the desired identity (the $r = 0$ term is 0 since $\lceil 0 \rceil \cdot (f(-1) - f(0)) = 0$).

Sanity check (small numbers)

Take $m = 3$, $n = 5$, $\alpha = 1.4$, and $f(r) = r$.

Left side:

$$\left\lfloor \tfrac{3j}{5} \right\rfloor \text{ for } j = 0, \dots, 6 \;=\; 0, 0, 1, 1, 2, 3, 3, \quad \text{sum } = 10.$$

Right side: $\lceil \alpha m \rceil = 5$, $\lceil \alpha n \rceil = 7$.

$$\sum_{r=0}^{4} \left\lceil \tfrac{5r}{3} \right\rceil (f(r-1) - f(r)) \;+\; 7\, f(4) = -(0+2+4+5+7) + 7 \cdot 4 = -18 + 28 = 10.$$

Matches perfectly.

47 [M31]

(a) Gauss's lemma in this setup

Fix k with $1 \le k \le (p-1)/2$. Write

$$2kq \;=\; p\left\lfloor \frac{2kq}{p} \right\rfloor \;+\; r_k, \qquad 0 < r_k < p, \quad r_k \equiv 2kq \pmod{p}.$$

Define

$$a_k \;:=\; (-1)^{\lfloor 2kq/p \rfloor}\, r_k.$$

There are two possibilities: If $\lfloor 2kq/p \rfloor$ is even, then $a_k = r_k \in \{1, 2, \dots, p-1\}$; if it's odd, then $a_k = r_k - p \in \{-p+1, \dots, -1\}$. In either case, a_k is congruent modulo p to an **even** residue, because $r_k \equiv 2kq$ and $2kq$ is even. As k ranges over $1, 2, \dots, (p-1)/2$, the reduced residues $2k$ run through all even residues modulo p, hence the values $2kq \bmod p$ also run through each even residue exactly once (since multiplication by coprime q permutes residues). The extra factor $(-1)^{\lfloor 2kq/p \rfloor}$ simply chooses, for each even residue e, either e or $e - p$, so the set $\{a_k\}$ is a permutation of $\{2, 4, \dots, p-1\}$ (as integers, hence also modulo p).

Taking products of the a_k and comparing both sides:

$$\prod_{k=1}^{(p-1)/2} a_k \;\equiv\; 2 \cdot 4 \cdots (p-1) \pmod{p}.$$

But

$$\prod_{k=1}^{(p-1)/2} a_k = (-1)^{\sum_k \lfloor 2kq/p \rfloor} \prod_{k=1}^{(p-1)/2} (2kq \bmod p) \equiv (-1)^{\sigma} \prod_{k=1}^{(p-1)/2} (2kq) = (-1)^{\sigma} q^{(p-1)/2} \prod_{k=1}^{(p-1)/2} (2k) \pmod{p}.$$

Cancel the common unit $\prod_k (2k)$ from both sides to obtain

$$q^{(p-1)/2} \equiv (-1)^{\sigma} \pmod{p}.$$

By the definition of Legendre's symbol this is exactly

$$\boxed{\left(\frac{q}{p}\right) = (-1)^{\sigma}, \qquad \sigma = \sum_{k=1}^{(p-1)/2} \left\lfloor \frac{2kq}{p} \right\rfloor}.$$

(b) The supplementary law for $\left(\frac{2}{p}\right)$

Set $q = 2$. Then

$$\sigma = \sum_{k=1}^{(p-1)/2} \left\lfloor \frac{4k}{p} \right\rfloor.$$

Write $p = 4n + 1$ or $p = 4n + 3$ (since p is odd).

- If $p = 4n + 1$, then $\dfrac{4k}{p} < 1$ for $1 \le k \le n$, and $1 \le \dfrac{4k}{p} < 2$ for $n < k \le 2n$. Hence

$$\sigma = \underbrace{0 + \cdots + 0}_{n \text{ terms}} + \underbrace{1 + \cdots + 1}_{n \text{ terms}} = n.$$

- If $p = 4n + 3$, then $(p - 1)/2 = 2n + 1$ and $\dfrac{4k}{p} < 1$ for $1 \le k \le n$, while $1 \le \dfrac{4k}{p} < 2$ for $n < k \le 2n + 1$. Thus

$$\sigma = \underbrace{0 + \cdots + 0}_{n \text{ terms}} + \underbrace{1 + \cdots + 1}_{n+1 \text{ terms}} = n + 1.$$

Therefore

$$\left(\frac{2}{p}\right) = (-1)^{\sigma} = \begin{cases} (-1)^{n}, & p = 4n + 1, \\ (-1)^{n+1}, & p = 4n + 3. \end{cases}$$

Classifying by $p \bmod 8$ (compute $n \bmod 2$ carefully):

- $p = 8t + 1 \Rightarrow n = 2t$ even $\Rightarrow \left(\frac{2}{p}\right) = +1$.
- $p = 8t + 3 \Rightarrow n = 2t$ even $\Rightarrow \left(\frac{2}{p}\right) = -1$.
- $p = 8t + 5 \Rightarrow n = 2t + 1$ odd $\Rightarrow \left(\frac{2}{p}\right) = -1$.
- $p = 8t + 7 \Rightarrow n = 2t + 1$ odd $\Rightarrow \left(\frac{2}{p}\right) = +1$.

Thus

$$\boxed{\left(\frac{2}{p}\right) = \begin{cases} +1, & p \equiv 1, 7 \pmod 8, \\ -1, & p \equiv 3, 5 \pmod 8. \end{cases}}$$

(c) Parity reduction: replace doubles by singles

Assume q is odd and $p \nmid q$. For $k < p/4$, consider

$$\left\lfloor \frac{(p-1-2k)q}{p} \right\rfloor = q - \left\lceil \frac{(2k+1)q}{p} \right\rceil.$$

Because $p \nmid (2k+1)q$ (both $2k+1$ and q are odd and in the range $1, \dots, p-1$), we have $\lceil x \rceil = \lfloor x \rfloor + 1$ at $x = \frac{(2k+1)q}{p}$. Hence

$$\left\lfloor \frac{(p-1-2k)q}{p} \right\rfloor = q - 1 - \left\lfloor \frac{(2k+1)q}{p} \right\rfloor \equiv \left\lfloor \frac{(2k+1)q}{p} \right\rfloor \pmod 2,$$

since $q-1$ is even. Summing these congruences over $k = 0, 1, \dots, \lfloor (p-2)/4 \rfloor$ shows that, modulo 2, the "tail" terms $\lfloor ((p-1)q)/p \rfloor, \lfloor ((p-3)q)/p \rfloor, \dots$ can be replaced by $\lfloor q/p \rfloor, \lfloor 3q/p \rfloor, \dots$. In total this yields

$$\sum_{0 \le k < p/2} \left\lfloor \frac{2kq}{p} \right\rfloor \equiv \sum_{0 \le k < p/2} \left\lfloor \frac{kq}{p} \right\rfloor \pmod 2,$$

as required.

(d) Quadratic reciprocity via Eisenstein's trick (using Ex. 46)

From Ex. 46 (with $m = q$, $n = p$, $\alpha = \frac{1}{2}$, and $f(r) = r$) we obtain the identity

$$\sum_{0 \le k < p/2} \left\lfloor \frac{kq}{p} \right\rfloor + \sum_{0 \le r < q/2} \left\lceil \frac{rp}{q} \right\rceil = \left\lceil \frac{p}{2} \right\rceil \left(\left\lceil \frac{q}{2} \right\rceil - 1 \right).$$

(Cited from the page with Ex. 46.)

Because p, q are distinct odd primes, none of the fractions $\frac{rp}{q}$ with $1 \le r \le (q-1)/2$ is an integer, so $\lceil x \rceil = \lfloor x \rfloor + 1$ applies termwise:

$$\sum_{0 \le r < q/2} \left\lceil \frac{rp}{q} \right\rceil = \sum_{0 \le r < q/2} \left\lfloor \frac{rp}{q} \right\rfloor + \frac{q-1}{2}.$$

Also $\lceil \frac{p}{2} \rceil = \frac{p+1}{2}$ and $\lceil \frac{q}{2} \rceil = \frac{q+1}{2}$. Hence

$$\sum_{k=1}^{(p-1)/2} \left\lfloor \frac{kq}{p} \right\rfloor + \sum_{r=1}^{(q-1)/2} \left\lfloor \frac{rp}{q} \right\rfloor = \frac{(p-1)(q-1)}{4}.$$

By part (a) together with the parity reduction of part (c),

$$\left(\frac{q}{p}\right) = (-1)^{\sum_{k=1}^{(p-1)/2} \lfloor kq/p \rfloor}, \qquad \left(\frac{p}{q}\right) = (-1)^{\sum_{r=1}^{(q-1)/2} \lfloor rp/q \rfloor}.$$

Adding the exponents mod 2 gives

$$\sum_{k=1}^{(p-1)/2} \left\lfloor \frac{kq}{p} \right\rfloor + \sum_{r=1}^{(q-1)/2} \left\lfloor \frac{rp}{q} \right\rfloor \equiv \frac{(p-1)(q-1)}{4} \pmod 2,$$

hence

$$\boxed{\left(\frac{q}{p}\right)\left(\frac{p}{q}\right) = (-1)^{(p-1)(q-1)/4}},$$

which is the quadratic reciprocity law. (The exercise statement on the next page explicitly asks you to reach this formula.)

1.2.5 Permutations and factorials

1 [00]

Think of building the deck from top to bottom:

- For the first position you can choose any of the 52 cards.
- For the second position you can choose any of the remaining 51 cards.
- …
- For the last position only 1 card remains.

By the rule of product, the total number of permutations is

$$52 \times 51 \times 50 \times \cdots \times 2 \times 1 \;=\; 52!\,.$$

That is the exact count.

(Optional) Sanity checks & useful facts

- **Exact integer (computed):**

$$52! = 80658175170943878571660636856403766975289505440883277824000000000000.$$

- **Number of digits:** 52! has 68 decimal digits.
- **Trailing zeros:** The number of trailing zeros equals the number of factors of 5 in 52!:

$$\left\lfloor \frac{52}{5} \right\rfloor + \left\lfloor \frac{52}{25} \right\rfloor = 10 + 2 = 12,$$

  so 52! ends with 12 zeros.

- **Magnitude (Stirling's approximation):**

$$52! \approx \sqrt{2\pi \cdot 52}\left(\frac{52}{e}\right)^{52} \approx 8.0658 \times 10^{67},$$

  which agrees with the leading digits shown above.

2 [10]

Claim (the recurrence)

$$p_n^k \;=\; p_n^{k-1}\,(n-k+1).$$

Combinatorial proof

Build a k-long arrangement left to right.

- First fill the first $k-1$ positions in any valid way; there are p_n^{k-1} such partial arrangements by definition.
- After placing $k-1$ *distinct* objects, exactly $n-(k-1) = n-k+1$ objects remain unused, and any of them can go to the last position.

By the multiplication principle,

$$p_n^k = (\#\text{ ways to fill first } k-1 \text{ spots}) \times (\#\text{ choices for last spot}) = p_n^{k-1}(n-k+1),$$

as claimed.

Inductive derivation of the closed form

Using the recurrence repeatedly,

$$p_n^k = (n-k+1)\,p_n^{k-1} = (n-k+1)(n-k+2)\,p_n^{k-2} = \cdots = n(n-1)\cdots(n-k+1).$$

Equivalently,

$$p_n^k = \frac{n!}{(n-k)!}.$$

Special case $k = n$

When arranging **all** n objects, the recurrence gives

$$p_n^n = p_n^{n-1}(n-n+1) = p_n^{n-1} \cdot 1,$$

which reflects the simple fact: *once the first $n-1$ places are fixed, the last object is forced (only one possibility remains).*

3 [10]

Method 1 (insert n in every position)

Rule. Given a permutation $a_1 a_2 \cdots a_{n-1}$ of $\{1, \dots, n-1\}$, insert n in each of the n possible positions. This yields n permutations of $\{1, \dots, n\}$.

Apply to $3\,1\,2\,4$ with $n = 5$ (five insertion positions):

1. before 3: **5 3 1 2 4**
2. between 3 and 1: **3 5 1 2 4**
3. between 1 and 2: **3 1 5 2 4**
4. between 2 and 4: **3 1 2 5 4**
5. after 4: **3 1 2 4 5**

So Method 1 produces: **5 3 1 2 4, 3 5 1 2 4, 3 1 5 2 4, 3 1 2 5 4, 3 1 2 4 5.**

Method 2 (order-preserving relabel + append k)

Rule (equivalent forms). For each $k \in \{1, \dots, n\}$, take a permutation $a_1 \cdots a_{n-1}$ of $\{1, \dots, n-1\}$ and:

- increase by 1 every entry a_j with $a_j \geq k$; this relabels $\{1, \dots, n-1\}$ into $\{1, \dots, k-1, k+1, \dots, n\}$ **while preserving order**,
- then append k. This yields n permutations from each starting permutation.

Apply to $3\,1\,2\,4$ (with $n = 5$):

- $k = 1$: add 1 to all (since all ≥ 1) $\to 4\,2\,3\,5$; append 1 $\to$ **4 2 3 5 1**
- $k = 2$: add 1 to entries $\geq 2 \to 4\,1\,3\,5$; append 2 $\to$ **4 1 3 5 2**
- $k = 3$: add 1 to entries $\geq 3 \to 4\,1\,2\,5$; append 3 $\to$ **4 1 2 5 3**
- $k = 4$: add 1 to entries $\geq 4 \to 3\,1\,2\,5$; append 4 $\to$ **3 1 2 5 4**
- $k = 5$: no change $\to 3\,1\,2\,4$; append 5 $\to$ **3 1 2 4 5**

So Method 2 produces: **4 2 3 5 1, 4 1 3 5 2, 4 1 2 5 3, 3 1 2 5 4, 3 1 2 4 5.**

4 [13]

1) Number of decimal digits of 1000!

For any positive integer N, the number of base-10 digits is

$$\lfloor \log_{10} N \rfloor + 1.$$

With $N = 1000!$,

$$\log_{10}(1000!) = \sum_{k=1}^{1000} \log_{10} k.$$

Evaluating (e.g., by high-precision arithmetic or tight Stirling bounds) gives

$$\log_{10}(1000!) \approx 2567.604644\ldots$$

so

$$\text{digits} = \lfloor 2567.604644\ldots \rfloor + 1 = 2568.$$

(You can also get this from Stirling's formula with error bounds, which nails the integer part in this case; the book develops Stirling earlier in §1.2.5.)

2) Leading digit of 1000!

Write

$$\log_{10}(1000!) = I + f, \quad I = \lfloor \log_{10}(1000!) \rfloor,\ f \in [0, 1).$$

Then

$$1000! = 10^{I+f} = 10^I \cdot 10^f, \quad 10^f \in [1, 10).$$

The leading digit is $\lfloor 10^f \rfloor$. From above $f \approx 0.604644$, so $10^f \approx 4.0238726$, hence the first digit is 4.

(Heuristically, this matches the book's remark that the mantissa is a bit above $\log_{10} 4 \approx 0.60206$, therefore the leading digit is 4.)

3) Number of trailing zeros of 1000!

Trailing zeros in base 10 are contributed by factors of $10 = 2 \cdot 5$. In $n!$, twos are plentiful, so the count is the exponent of 5 in $n!$:

$$\nu_5(n!) = \sum_{k \geq 1} \left\lfloor \frac{n}{5^k} \right\rfloor.$$

(This is the standard prime–power valuation formula developed as Eq. (8) in §1.2.5.)

Compute for $n = 1000$:

$$\left\lfloor \frac{1000}{5} \right\rfloor = 200, \quad \left\lfloor \frac{1000}{25} \right\rfloor = 40, \quad \left\lfloor \frac{1000}{125} \right\rfloor = 8, \quad \left\lfloor \frac{1000}{625} \right\rfloor = 1,$$

and higher powers give 0. Summing:

$$\nu_5(1000!) = 200 + 40 + 8 + 1 = 249.$$

Therefore 1000! ends with exactly 249 zeros.

5 [15]

We're asked to estimate 8! using the refined Stirling approximation

$$n! \;\approx\; \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\!\left(1 + \frac{1}{12n}\right),$$

Step 1: Base Stirling term

For $n = 8$,

$$\sqrt{2\pi n}\left(\frac{n}{e}\right)^n = \sqrt{16\pi}\left(\frac{8}{e}\right)^8 \approx 39{,}902.395\ .$$

(Indeed, Knuth's numerical example rounds this to 39902.)

Step 2: First correction factor

The first correction multiplies by $1 + \dfrac{1}{12n} = 1 + \dfrac{1}{96} = \dfrac{97}{96}$:

$$\widehat{8!} \;=\; \left[\sqrt{2\pi n}\left(\frac{n}{e}\right)^n\right]\left(1 + \frac{1}{12n}\right) \;\approx\; 39{,}902.395 \times \frac{97}{96} \;\approx\; 40{,}318.045\ .$$

Rounding sensibly gives $\boxed{40{,}318}$.

Accuracy check

The true value is $8! = 40{,}320$. Thus the error is about -1.955 (roughly -2), i.e.,

$$\frac{|\widehat{8!} - 8!|}{8!} \;\approx\; 4.85 \times 10^{-5},$$

a dramatic improvement over the $\sim 1\%$ error of the uncorrected term.

So the requested refined Stirling estimate for 8! is 40,318, within two of the exact value.

6 [17]

For a prime p, the exponent μ_p of p in $n!$ is

$$\mu_p = \sum_{k>0} \left\lfloor \frac{n}{p^k} \right\rfloor,$$

i.e., count how many multiples of $p, p^2, p^3, \ldots$ are $\leq n$ and add them. (Eq. (8).)

We apply this with $n = 20$ and primes $p \leq 20$: $2, 3, 5, 7, 11, 13, 17, 19$.

- $p = 2:\ \lfloor 20/2 \rfloor + \lfloor 20/4 \rfloor + \lfloor 20/8 \rfloor + \lfloor 20/16 \rfloor = 10 + 5 + 2 + 1 = 18.$
- $p = 3:\ \lfloor 20/3 \rfloor + \lfloor 20/9 \rfloor = 6 + 2 = 8.$
- $p = 5:\ \lfloor 20/5 \rfloor = 4.$
- $p = 7:\ \lfloor 20/7 \rfloor = 2.$
- $p = 11:\ \lfloor 20/11 \rfloor = 1.$
- $p = 13:\ \lfloor 20/13 \rfloor = 1.$
- $p = 17:\ \lfloor 20/17 \rfloor = 1.$
- $p = 19:\ \lfloor 20/19 \rfloor = 1.$

All higher powers contribute 0.

Therefore the prime factorization is

$$\boxed{20! = 2^{18}\, 3^8\, 5^4\, 7^2\, 11\, 13\, 17\, 19.}$$

This is exactly what the exercise asks for, using Eq. (8) from §1.2.5.

7 [M10]

1. Convert the given datum to gamma notation. Since $\left(\frac{1}{2}\right)! = \Gamma\left(\frac{3}{2}\right)$, the assumption gives

$$\Gamma\left(\tfrac{3}{2}\right) \;=\; \frac{\sqrt{\pi}}{2}.$$

2. Use the recurrence $\Gamma(x+1) = x\,\Gamma(x)$ with $x = \frac{1}{2}$:

$$\Gamma\left(\tfrac{3}{2}\right) \;=\; \tfrac{1}{2}\,\Gamma\left(\tfrac{1}{2}\right).$$

Hence

$$\Gamma\left(\tfrac{1}{2}\right) \;=\; 2\,\Gamma\left(\tfrac{3}{2}\right) \;=\; 2 \cdot \frac{\sqrt{\pi}}{2} \;=\; \boxed{\sqrt{\pi}}.$$

3. Use the same recurrence with $x = -\frac{1}{2}$:

$$\Gamma\left(\tfrac{1}{2}\right) \;=\; \left(-\tfrac{1}{2}\right)\Gamma\left(-\tfrac{1}{2}\right),$$

so

$$\Gamma\left(-\tfrac{1}{2}\right) \;=\; \frac{\Gamma\left(\frac{1}{2}\right)}{-\frac{1}{2}} \;=\; \frac{\sqrt{\pi}}{-\frac{1}{2}} \;=\; \boxed{-2\sqrt{\pi}}.$$

These match Knuth's official answers for §1.2.5–7 ("$\sqrt{\pi}$ and $-2\sqrt{\pi}$"; this exercise uses the recurrence in Ex. 10).

Optional intuition:

Many texts justify the given value $\left(\frac{1}{2}\right)! = \Gamma(\frac{3}{2}) = \sqrt{\pi}/2$ via the Gaussian integral:

$$\Gamma\left(\tfrac{1}{2}\right) = \int_0^\infty e^{-t} t^{-1/2}\, dt = 2\int_0^\infty e^{-u^2}\, du = \sqrt{\pi},$$

then apply the recurrence to get $\Gamma(\frac{3}{2}) = \frac{1}{2}\Gamma(\frac{1}{2}) = \sqrt{\pi}/2$. (Here we only needed the latter as a given.)

8 [HM15]

Let

$$A_m(n) \;=\; \frac{m^n\, m!}{(n+1)(n+2)\cdots(n+m)}.$$

1. Use the identity $(n+1)(n+2)\cdots(n+m) = \dfrac{(n+m)!}{n!}$. Then

$$A_m(n) = \frac{m^n\, m!\, n!}{(n+m)!}.$$

2. Recognize a binomial coefficient:

$$\binom{n+m}{n} = \frac{(n+m)!}{n!\,m!} \quad \Longrightarrow \quad \frac{n!\,m!}{(n+m)!} = \frac{1}{\binom{n+m}{n}}.$$

Hence

$$A_m(n) = \frac{m^n}{\binom{n+m}{n}}.$$

3. Rewrite $\binom{n+m}{n}$ in a way that exposes a finite product of n factors:

$$\binom{n+m}{n} = \frac{(m+1)(m+2)\cdots(m+n)}{n!} = \frac{m^n}{n!}\prod_{k=1}^{n}\left(1+\frac{k}{m}\right).$$

Therefore

$$A_m(n) = \frac{m^n}{\binom{n+m}{n}} = \frac{m^n}{\frac{m^n}{n!}\prod_{k=1}^{n}\left(1+\frac{k}{m}\right)} = \frac{n!}{\prod_{k=1}^{n}\left(1+\frac{k}{m}\right)}.$$

4. Take the limit as $m \to \infty$. For each fixed $k \le n$, $\left(1+\frac{k}{m}\right) \to 1$. The product has only n factors, so

$$\lim_{m\to\infty} A_m(n) = \frac{n!}{\prod_{k=1}^{n}\lim_{m\to\infty}\left(1+\frac{k}{m}\right)} = \frac{n!}{1} = n!.$$

5. Edge case $n = 0$: By definition,

$$A_m(0) = \frac{m^0\,m!}{1\cdot 2\cdots m} = 1 = 0!,$$

so the limit is also $0!$.

Thus, Euler's limit indeed equals $n!$ for every nonnegative integer n.

9 [M10]

Knuth defines $n! = \Gamma(n+1)$ and uses the gamma functional equation

$$\Gamma(x+1) = x\,\Gamma(x).$$

Hence $(\frac{1}{2})! = \Gamma(\frac{3}{2})$.

1. Compute $\Gamma(\frac{1}{2})$.

From the given $(\frac{1}{2})! = \Gamma(\frac{3}{2}) = \sqrt{\pi}/2$ and the recurrence with $x = \frac{1}{2}$:

$$\Gamma\left(\tfrac{3}{2}\right) = \tfrac{1}{2}\,\Gamma\left(\tfrac{1}{2}\right).$$

Therefore

$$\Gamma\left(\tfrac{1}{2}\right) = 2\,\Gamma\left(\tfrac{3}{2}\right) = 2 \cdot \frac{\sqrt{\pi}}{2} = \boxed{\sqrt{\pi}}.$$

(Consistency check, optional: the reflection formula $\Gamma(z)\Gamma(1-z) = \dfrac{\pi}{\sin \pi z}$ gives $\Gamma(\frac{1}{2})^2 = \pi \Rightarrow \Gamma(\frac{1}{2}) = \sqrt{\pi}$.)

2. Compute $\Gamma(-\frac{1}{2})$.

Apply the recurrence backward at $x = -\frac{1}{2}$:

$$\Gamma\left(\tfrac{1}{2}\right) = \left(-\tfrac{1}{2}\right)\Gamma\left(-\tfrac{1}{2}\right) \quad \Longrightarrow \quad \Gamma\left(-\tfrac{1}{2}\right) = \frac{\Gamma(\frac{1}{2})}{-\frac{1}{2}} = -2\,\Gamma\left(\tfrac{1}{2}\right) = \boxed{-2\sqrt{\pi}}.$$

Thus

$$\boxed{\Gamma\left(\tfrac{1}{2}\right) = \sqrt{\pi}}, \qquad \boxed{\Gamma\left(-\tfrac{1}{2}\right) = -2\sqrt{\pi}}.$$

10 [HM20]

Background (what Γ means here)

Knuth/Euler define the gamma function via Euler's limit (valid for real/complex x except at non-positive integers, where poles occur):

$$\Gamma(x) \;=\; \lim_{m\to\infty} \frac{m^{\,x}\, m!}{x(x+1)\cdots(x+m)}.$$

This is equation (15) in the text.

There is also the integral form (for x>0), proved a few exercises later:

$$\Gamma(x) = \int_0^\infty e^{-t}\, t^{x-1}\, dt.$$

Conclusion

Yes—except when x is 0 or a negative integer. At those points Γ has poles, so the identity is not meaningful there.

Proof 1 (from Euler's limit; works wherever Γ is defined)

Start from the defining limit with $x \mapsto x+1$:

$$\Gamma(x+1) \;=\; \lim_{m\to\infty} \frac{m^{\,x+1}\, m!}{(x+1)(x+2)\cdots(x+m+1)}.$$

Factor a convenient term:

$$\Gamma(x+1) = \lim_{m\to\infty} \frac{m}{x+m+1} \cdot \frac{m^{\,x}\, m!}{(x+1)\cdots(x+m)}.$$

But

$$x\,\Gamma(x) = x \cdot \lim_{m\to\infty} \frac{m^{\,x}\, m!}{x(x+1)\cdots(x+m)} = \lim_{m\to\infty} \frac{m^{\,x}\, m!}{(x+1)\cdots(x+m)}.$$

Since $\frac{m}{x+m+1} \to 1$ as $m \to \infty$, the two limits are equal:

$$\Gamma(x+1) = x\,\Gamma(x).$$

This algebra is the essence of the official solution in the book.

Domain caveat. The products $(x+j)$ in the denominator hit zero exactly when $x \in \{0, -1, -2, \dots\}$; that is where Γ has poles, so the identity isn't asserted there. Everywhere else (real x not a non-positive integer), the argument goes through.

Proof 2 (for $x > 0$ only, via the integral)

Using $\Gamma(x) = \int_0^\infty e^{-t} t^{x-1} dt$ and integration by parts:

$$\Gamma(x+1) = \int_0^\infty e^{-t} t^x \, dt = \Big[-e^{-t} t^x \Big]_0^\infty + x \int_0^\infty e^{-t} t^{x-1} \, dt = x\,\Gamma(x),$$

since the boundary term vanishes for $x > 0$. This gives the same recurrence on $(0, \infty)$.

Remarks

- Together with $\Gamma(1) = 1$, the recurrence yields $\Gamma(n+1) = n!$ for positive integers n, matching the factorial rule.
- In the complex plane, Γ is meromorphic with simple poles at $0, -1, -2, \dots$; the same functional equation holds on its domain. (TAOCP also records the related reflection identity (16).)

11 [M15]

Proof via Legendre's formula (one line, then a tidy sum)

By Legendre's formula the exponent of a prime p in $n!$ is

$$v_p(n!) = \sum_{k \geq 1} \left\lfloor \frac{n}{p^k} \right\rfloor.$$

For $p = 2$,

$$v_2(n!) = \sum_{k \geq 1} \left\lfloor \frac{n}{2^k} \right\rfloor. \qquad (\ )$$

(This is exactly the "prime–power multiplicity" identity stated in the text.)

Now use the binary form of n. Since $n = \sum_{j=1}^r 2^{e_j}$,

$$\left\lfloor \frac{n}{2} \right\rfloor = \sum_{j: e_j \geq 1} 2^{e_j - 1}, \quad \left\lfloor \frac{n}{4} \right\rfloor = \sum_{j: e_j \geq 2} 2^{e_j - 2}, \quad \left\lfloor \frac{n}{8} \right\rfloor = \sum_{j: e_j \geq 3} 2^{e_j - 3}, \ \dots$$

Summing these rows termwise (exactly the series ( )) yields, for each fixed j,

$$2^{e_j-1} + 2^{e_j-2} + \cdots + 2 + 1 = 2^{e_j} - 1.$$

Therefore

$$v_2(n!) = \sum_{k\geq 1} \left\lfloor \frac{n}{2^k} \right\rfloor = \sum_{j=1}^{r} (2^{e_j} - 1) = \left(\sum_{j=1}^{r} 2^{e_j} \right) - r = n - r.$$

So $n!$ contains exactly $n - r$ factors of 2—equivalently $2^{n-r} \mid n!$ but $2^{n-r+1} \nmid n!$, as required.

Tiny numerical check

Take $n = 13 = (1101)_2$ so $r = 3$. Then

$$v_2(13!) = \left\lfloor \frac{13}{2} \right\rfloor + \left\lfloor \frac{13}{4} \right\rfloor + \left\lfloor \frac{13}{8} \right\rfloor = 6 + 3 + 1 = 10 = 13 - 3.$$

So $2^{10} \mid 13!$ but $2^{11} \nmid 13!$, matching the claim.

(Optional) Broader view

Exactly the same argument shows for any prime p that

$$v_p(n!) = \frac{n - s_p(n)}{p - 1},$$

where $s_p(n)$ is the sum of the base-p digits of n. The present exercise is the special case $p = 2$, where $s_2(n) = r$.

12 [M22]

Let p be prime and write n in base p as

$$n = a_k p^k + a_{k-1} p^{k-1} + \cdots + a_1 p + a_0 \qquad (0 \leq a_i < p).$$

If μ denotes the exponent of p in $n!$ (i.e., $\mu = \sum_{j\geq 1} \lfloor n/p^j \rfloor$ from Eq. (8)), express μ in a simple formula involving n, p, and the digits a_i.

Let $S_p(n) = a_0 + a_1 + \cdots + a_k$ be the sum of the base-p digits of n.

1. For each $i \geq 1$,

$$\left\lfloor \frac{n}{p^i} \right\rfloor = \sum_{j=i}^{k} a_j p^{j-i}$$

because dividing by p^i in base p just drops the i least-significant digits.

2. Sum over $i = 1, \dots, k$ and swap the order of summation:

$$\sum_{i=1}^{k} \left\lfloor \frac{n}{p^i} \right\rfloor = \sum_{i=1}^{k} \sum_{j=i}^{k} a_j p^{j-i} = \sum_{j=1}^{k} a_j \sum_{i=1}^{j} p^{j-i} = \sum_{j=1}^{k} a_j \frac{p^j - 1}{p-1}.$$

3. Simplify:

$$\sum_{j=1}^{k} a_j \frac{p^j - 1}{p-1} = \frac{1}{p-1} \left(\sum_{j=1}^{k} a_j p^j - \sum_{j=1}^{k} a_j \right) = \frac{1}{p-1} \big((n - a_0) - (S_p(n) - a_0) \big) = \frac{n - S_p(n)}{p-1}.$$

But the left-hand side is exactly μ by Eq. (8). Therefore

$$\boxed{\mu \;=\; \frac{n - S_p(n)}{p-1}}$$

which is Legendre's (de Polignac's) formula.

Check

For $n = 1000$ and $p = 3$, 1000 in base 3 is 1 1 0 1 0 0 1 so $S_3(1000) = 4$. Hence

$$\mu = \frac{1000 - 4}{3 - 1} = 498,$$

matching the worked value obtained by summing $\lfloor 1000/3^j \rfloor$.

This also recovers Exercise 11's binary special case: when $p = 2$, $S_2(n)$ is the number of 1s in the binary expansion, so $v_2(n!) = n - S_2(n)$.

13 [M22]

Proof (pairing multiplicative inverses)

1. **Every nonzero residue has a unique inverse mod p.** For each $n \in \{1, 2, \dots, p-1\}$ there exists a unique n' with

$$nn' \equiv 1 \pmod{p},$$

because $\gcd(n, p) = 1$ when p is prime (Bezout's identity gives existence; uniqueness follows since $n(a-b) \equiv 0 \pmod{p} \Rightarrow a \equiv b$). Also, applying "take the inverse" twice gives $(n')' = n$.

2. **Only 1 and $p-1$ are self-inverse.** If $n' = n$, then $n^2 \equiv 1 \pmod{p}$, so $(n-1)(n+1) \equiv 0 \pmod{p}$. A prime p can divide at most one factor, hence $n \equiv 1$ or $n \equiv -1 \equiv p-1 \pmod{p}$. Thus among $\{1, 2, \dots, p-1\}$, exactly the two elements 1 and $p-1$ satisfy $n' = n$.

3. **Pair everything else.** Partition $\{1, 2, \dots, p-1\}$ into disjoint pairs $\{n, n'\}$ with $n \neq n'$, together with the two singletons $\{1\}$ and $\{p-1\}$. Each paired product is

$$n \cdot n' \equiv 1 \pmod{p}.$$

4. **Multiply all residues.** Taking the product of all elements $1 \cdot 2 \cdot \dots \cdot (p-1)$ and grouping by pairs,

$$(p-1)! \equiv \Big(\prod_{\text{pairs } \{n,n'\}} nn'\Big) \cdot 1 \cdot (p-1) \equiv 1 \cdot (p-1) \equiv -1 \pmod{p}.$$

This is exactly the desired congruence.

14 [M28]

We work with the **unit part** of $n!$ (i.e., $n!/p^{\mu}$) modulo p.

1. Block decomposition of $1, 2, \dots, n$. Split $\{1, \dots, n\}$ into:

- all **multiples of** p: $p, 2p, \dots, \lfloor n/p \rfloor p$, whose product is $p^{\lfloor n/p \rfloor} \cdot \lfloor n/p \rfloor!$;

- all **nonmultiples of** p, further grouped into consecutive blocks

$$\{mp+1, mp+2, \dots, mp+p-1\} \quad (m = 0, 1, \dots, \lfloor n/p \rfloor - 1),$$

  plus a final tail $\{\lfloor n/p \rfloor p + 1, \dots, \lfloor n/p \rfloor p + a_0\}$ of length a_0.

2. Product of a complete block of nonmultiples. Modulo p each complete block is congruent to $1 \cdot 2 \cdots (p-1) = (p-1)! \equiv -1 \pmod{p}$ by Wilson's theorem. Hence the product over all complete blocks contributes a factor $(-1)^{\lfloor n/p \rfloor}$. (Wilson's theorem is Ex. 13.)

3. Product of the tail. The last incomplete block contributes

$$(\lfloor n/p \rfloor p + 1) \cdots (\lfloor n/p \rfloor p + a_0) \equiv 1 \cdot 2 \cdots a_0 \equiv a_0! \pmod{p}.$$

4. First recurrence for the unit part. Let $U(n) := n!/p^{\nu_p(n!)}$ be the unit part of $n!$. Using the split above,

$$U(n) \equiv (-1)^{\lfloor n/p \rfloor} a_0! \cdot U(\lfloor n/p \rfloor) \pmod{p},$$

because the multiples of p contribute $p^{\lfloor n/p \rfloor} \cdot \lfloor n/p \rfloor!$, and after dividing by the corresponding power of p we are left with the **unit part** $U(\lfloor n/p \rfloor)$.

5. Iterate the recurrence. Apply Step 4 to $\lfloor n/p \rfloor$, then to $\lfloor n/p^2 \rfloor$, and so on, until reaching 0. Writing the base-p digits explicitly,

$$\lfloor n/p \rfloor = a_k p^{k-1} + \cdots + a_2 p + a_1, \quad \lfloor n/p^2 \rfloor = a_k p^{k-2} + \cdots + a_2, \ \dots$$

the iteration yields

$$U(n) \equiv \Big(\prod_{j \geq 1} (-1)^{\lfloor n/p^j \rfloor} \Big) \cdot \Big(\prod_{i=0}^{k} a_i! \Big) \pmod{p}.$$

But $\sum_{j \geq 1} \lfloor n/p^j \rfloor = \mu$ (Legendre's formula), hence

$$\boxed{\frac{n!}{p^\mu} \equiv (-1)^\mu \, a_0! \, a_1! \cdots a_k! \pmod{p}}.$$

This is exactly the Stickelberger congruence requested.

Quick check (example)

Take $p = 5, n = 47$. Base-5: $47 = 1{\cdot}5^2{+}4{\cdot}5{+}2$ so $(a_2, a_1, a_0) = (1, 4, 2)$. $\mu = \lfloor 47/5 \rfloor + \lfloor 47/25 \rfloor = 9 + 1 = 10$. RHS: $(-1)^{10} \cdot (2! \cdot 4! \cdot 1!) \equiv 1 \cdot (2 \cdot 24) \equiv 48 \equiv 3 \pmod 5$. Indeed $47!/5^{\mu} \equiv 3 \pmod 5$.

15 [HM15]

Let A be the $n \times n$ matrix with entries $A_{ij} = i \cdot j$ (so it's the multiplication table $1, 2, \dots, n$ by $1, 2, \dots, n$). Compute the **permanent** of A,

$$\operatorname{perm}(A) \;=\; \sum_{\sigma \in S_n} \prod_{i=1}^{n} A_{i,\sigma(i)}\,.$$

Solution 1 (rank-1 factorization $\to$ one-line evaluation)

Observe that A factors as an outer product:

$$A = u\,v^{\mathsf{T}}, \qquad u = \begin{bmatrix} 1 \\ 2 \\ \vdots \\ n \end{bmatrix}, \quad v = \begin{bmatrix} 1 \\ 2 \\ \vdots \\ n \end{bmatrix}.$$

Hence $A_{i,\sigma(i)} = u_i\, v_{\sigma(i)} = i \cdot \sigma(i)$. For any rank-1 matrix uv^{T},

$$\operatorname{perm}(uv^{\mathsf{T}}) = \sum_{\sigma \in S_n} \prod_{i=1}^{n} u_i\, v_{\sigma(i)} = \Big(\prod_{i=1}^{n} u_i\Big) \sum_{\sigma \in S_n} \prod_{i=1}^{n} v_{\sigma(i)}.$$

But $\prod_{i=1}^{n} v_{\sigma(i)}$ equals $\prod_{j=1}^{n} v_j$ (a product is order-independent), so every one of the $n!$ terms is the same. Therefore

$$\operatorname{perm}(uv^{\mathsf{T}}) = n!\,\Big(\prod_{i=1}^{n} u_i\Big)\,\Big(\prod_{j=1}^{n} v_j\Big).$$

Plugging $u_i = i$ and $v_j = j$ gives $\prod_{i=1}^{n} u_i = n!$ and $\prod_{j=1}^{n} v_j = n!$. Hence

$$\boxed{\operatorname{perm}(A) = (n!)^3}.$$

Solution 2 (direct term-by-term reasoning)

Expand the permanent by definition:

$$\operatorname{perm}(A) = \sum_{\sigma \in S_n} \prod_{i=1}^{n} (i \cdot \sigma(i)).$$

For any fixed σ,

$$\prod_{i=1}^{n} (i \cdot \sigma(i)) = \Big(\prod_{i=1}^{n} i\Big) \Big(\prod_{i=1}^{n} \sigma(i)\Big) = (n!) \cdot (n!) = (n!)^2,$$

since $\{\sigma(1), \dots, \sigma(n)\} = \{1, \dots, n\}$. There are $n!$ permutations σ, and every term is the same, so

$$\operatorname{perm}(A) = n! \cdot (n!)^2 = (n!)^3.$$

Quick sanity checks

- $n = 1$: $\begin{bmatrix} 1 \end{bmatrix}$, permanent $= 1 = (1!)^3$.
- $n = 2$: $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$, permanent $= 1 \cdot 4 + 2 \cdot 2 = 8 = (2!)^3$.
- $n = 3$: each term is $(3!)^2 = 36$; 6 terms $\Rightarrow 6 \cdot 36 = 216 = (3!)^3$.

16 [HM15]

Write the coefficient of the k-th falling factorial as

$$c_k \;:=\; \sum_{j=0}^{k} \frac{(-1)^j}{j!}\,.$$

These are the partial sums of the Taylor series of e^{-1}, hence

$$c_k \xrightarrow[k \to \infty]{} e^{-1} \neq 0,$$

so in particular $|c_k|$ stays bounded away from 0 for large k.

Now consider the general term

$$T_k(n) \;=\; c_k\, n^{\underline{k}} = c_k \prod_{r=0}^{k-1} (n - r).$$

We separate three cases.

1. $n \in \{0, 1, 2, \dots\}$ (nonnegative integer). Then $n^{\underline{k}} = 0$ for all $k > n$ (the product has the factor $n - n = 0$), so the series truncates at $k = n$. Hence it certainly converges (indeed, equals $n!$). This is exactly the circumstance noted under Eq. (11).

2. $n = -m$ with $m \in \{1, 2, 3, \dots\}$ (negative integer). Then

$$(-m)^{\underline{k}} = (-m)(-m-1)\cdots(-m-k+1) = (-1)^k\, m(m+1)\cdots(m+k-1) = (-1)^k \frac{(m+k-1)!}{(m-1)!}.$$

Therefore

$$|T_k(-m)| = |c_k| \cdot \frac{(m+k-1)!}{(m-1)!} \xrightarrow{k\to\infty} \infty,$$

because $|c_k| \to e^{-1}$ and the factorial grows without bound. In particular the term $T_k(n)$ does **not** tend to 0, so the series diverges.

3. $n \notin \mathbb{Z}$ (non-integer real). Choose an integer $A > |n|$. For all $k \geq A + 1$,

$$|n^{\underline{k}}| = \prod_{r=0}^{k-1} |n - r| \;\geq\; \prod_{r=A}^{k-1} (r - |n|) \;=\; \frac{\Gamma(k - |n|)}{\Gamma(A - |n|)}\,.$$

The right-hand side $\to \infty$ with k (since $\Gamma(x) \to \infty$ as $x \to \infty$). Hence $|n^{\underline{k}}| \to \infty$. Because $|c_k| \to e^{-1} > 0$, we get

$$|T_k(n)| = |c_k|\, |n^{\underline{k}}| \xrightarrow{k\to\infty} \infty,$$

so again the general term does **not** go to 0, and the series diverges.

(If you prefer a closed form: for non-integer n,

$$n^{\underline{k}} = \frac{\Gamma(n+1)}{\Gamma(n+1-k)},$$

and by the reflection identity one can write $\Gamma(n+1-k)^{-1}$ in terms of $\Gamma(k-n)$; since $\Gamma(k-n) \to \infty$ as $k \to \infty$, the same conclusion follows. The truncation only occurs when n is a nonnegative integer because then one factor $n - r$ is zero.)

Therefore, the only values of n for which the series in Eq. (11) converges are the nonnegative integers, precisely those for which the "infinite" series terminates.

17 [HM20]

Let

$$P_m \;=\; \prod_{n=1}^{m} \frac{(n+\alpha_1)\cdots(n+\alpha_k)}{(n+\beta_1)\cdots(n+\beta_k)}.$$

We'll evaluate $\lim_{m\to\infty} P_m$.

Knuth defines the gamma function via the limit (for real/complex x):

$$\Gamma(x) \;=\; \lim_{m\to\infty} \frac{m^x\, m!}{x(x+1)\cdots(x+m)}.$$

Equivalently, for each fixed x,

$$\prod_{n=1}^{m}(n+x) \;=\; \frac{m^x\, m!}{A_m(x)}, \quad \text{where } A_m(x) \xrightarrow[m\to\infty]{} \Gamma(1+x).$$

(This is just algebraic rearrangement of the defining limit.)

Apply this to each α_i and β_i:

$$\prod_{n=1}^{m}(n+\alpha_i) = \frac{m^{\alpha_i} m!}{A_m(\alpha_i)}, \qquad \prod_{n=1}^{m}(n+\beta_i) = \frac{m^{\beta_i} m!}{A_m(\beta_i)}.$$

Therefore

$$P_m = \frac{\prod_{i=1}^{k} \frac{m^{\alpha_i} m!}{A_m(\alpha_i)}}{\prod_{i=1}^{k} \frac{m^{\beta_i} m!}{A_m(\beta_i)}} = m^{\sum_i \alpha_i - \sum_i \beta_i} \cdot \frac{\prod_{i=1}^{k} A_m(\beta_i)}{\prod_{i=1}^{k} A_m(\alpha_i)}.$$

By hypothesis $\sum_i \alpha_i = \sum_i \beta_i$, so the power of m cancels. Passing to the limit and using $A_m(x) \to \Gamma(1+x)$,

$$\lim_{m\to\infty} P_m = \frac{\prod_{i=1}^{k} \Gamma(1+\beta_i)}{\prod_{i=1}^{k} \Gamma(1+\alpha_i)}.$$

This is exactly the claimed value of the infinite product.

Why the product converges (quick check)

For large n,

$$\log \frac{\prod_i (n+\alpha_i)}{\prod_i (n+\beta_i)} = \sum_i \left(\frac{\alpha_i - \beta_i}{n} \right) + O\left(\frac{1}{n^2} \right).$$

The $1/n$ term vanishes by $\sum \alpha_i = \sum \beta_i$, so $\sum_n O(1/n^2)$ converges; hence the infinite product converges to a nonzero finite limit. The restriction that no β_i is a negative integer prevents a zero factor in the denominator (and matches the fact $\Gamma(1+\beta_i)$ would otherwise have a pole).

18 [M20]

Show that

$$\prod_{k=1}^{\infty} \frac{4k^2}{4k^2-1} = \frac{\pi}{2}.$$

1) Set up the key integral and its recurrence

Let

$$I_n := \int_0^{\pi/2} \sin^n x \, dx \qquad (n \geq 0).$$

Basic values: $I_0 = \pi/2$ and $I_1 = 1$. Integration by parts (take $u = \sin^{n-1} x$, $dv = \sin x \, dx$) gives the standard two-step recurrence

$$I_n = \frac{n-1}{n} I_{n-2} \qquad (n \geq 2).$$

Iterating this with the bases yields the closed forms

$$I_{2n} = \frac{(2n-1)!!}{(2n)!!} \cdot \frac{\pi}{2} \quad \text{and} \quad I_{2n+1} = \frac{(2n)!!}{(2n+1)!!}.$$

(Here $m!!$ denotes the double factorial.)

Using $(2n)!! = 2^n n!$ and $(2n-1)!! = \dfrac{(2n)!}{2^n n!}$, we also have the handy identity

$$I_{2n} = \frac{\binom{2n}{n}}{4^n} \cdot \frac{\pi}{2}.$$

2) Express a finite Wallis product via I_{2n} and I_{2n+1}

Consider the finite product

$$P_n := \prod_{k=1}^{n} \frac{4k^2}{4k^2-1} = \frac{(2n)!!^{\,2}}{(2n-1)!!\,(2n+1)!!}.$$

Using the relations above,

$$\frac{(2n-1)!!}{(2n)!!} = \frac{2I_{2n}}{\pi}, \qquad \frac{(2n+1)!!}{(2n)!!} = \frac{1}{I_{2n+1}},$$

hence

$$\frac{1}{P_n} = \frac{(2n-1)!!\,(2n+1)!!}{(2n)!!^{\,2}} = \Big(\frac{2I_{2n}}{\pi}\Big)\cdot\Big(\frac{1}{I_{2n+1}}\Big) = \frac{2}{\pi}\cdot\frac{I_{2n}}{I_{2n+1}}.$$

Equivalently,

$$P_n = \frac{\pi}{2}\cdot\frac{I_{2n+1}}{I_{2n}}. \tag{ }$$

3) Show $\lim\limits_{n\to\infty} \dfrac{I_{2n+1}}{I_{2n}} = 1$

Write $y = \cos x$ (so $x \in [0,\pi/2] \iff y \in [0,1]$). Then

$$I_{2n} = \int_0^1 (1-y^2)^n\,dy, \qquad I_{2n+1} = \int_0^1 (1-y^2)^n\sqrt{1-y^2}\,dy.$$

On $[0,1]$ we have $1 \ge \sqrt{1-y^2} \ge 1-\frac{1}{2}y^2$. Thus

$$I_{2n} \;\ge\; I_{2n+1} \;\ge\; I_{2n} - \frac{1}{2}\int_0^1 y^2(1-y^2)^n\,dy.$$

The "error" integral is $O\big(\frac{1}{n}\,I_{2n}\big)$ (e.g., by a routine Beta-integral estimate), so

$$0 \le 1 - \frac{I_{2n+1}}{I_{2n}} \;\longrightarrow\; 0, \qquad \text{i.e.} \qquad \frac{I_{2n+1}}{I_{2n}} \xrightarrow[n\to\infty]{} 1.$$

(Heuristically: the density proportional to $\sin^{2n} x$ concentrates near $x = \pi/2$, where $\sin x \approx 1$, so inserting one extra factor of $\sin x$ barely changes the integral.)

4) Take limits in ( )

From ( ) and the limit above,

$$\lim_{n\to\infty} P_n = \frac{\pi}{2} \cdot \lim_{n\to\infty} \frac{I_{2n+1}}{I_{2n}} = \frac{\pi}{2} \cdot 1 = \frac{\pi}{2}.$$

Therefore

$$\boxed{\prod_{k=1}^{\infty} \frac{4k^2}{4k^2-1} = \frac{\pi}{2}}$$

as claimed.

19 [HM22]

Let

$$\Gamma(x) = \lim_{m\to\infty} \frac{m^x\, m!}{x(x+1)\cdots(x+m)} \qquad \text{(Eq. (15) in the book)}$$

and denote the m-th finite approximant by

$$\Gamma_m(x) \;=\; \frac{m^x\, m!}{x(x+1)\cdots(x+m)}.$$

Show that for $x > 0$,

$$\Gamma_m(x) \;=\; \int_0^m \Big(1 - \tfrac{t}{m}\Big)^m t^{x-1}\, dt \;=\; m^x \int_0^1 (1-u)^m u^{x-1}\, du.$$

Define

$$I_m(x) \;:=\; \int_0^m \Big(1 - \tfrac{t}{m}\Big)^m t^{x-1}\, dt \qquad (x > 0).$$

Step 1: Change of variables

Let $t = mu$ (so $dt = m\, du$, $u \in [0,1]$). Then

$$I_m(x) = \int_0^1 (1-u)^m\, (mu)^{x-1}\, m\, du = m^x \int_0^1 (1-u)^m u^{x-1}\, du.$$

This proves the second equality in the exercise.

Set

$$J(x,m) \;:=\; \int_0^1 (1-u)^m u^{x-1}\,du \qquad (x>0,\; m\in\mathbb{Z}_{\geq 0}),$$

so $I_m(x) = m^x J(x,m)$.

Step 2: Integration by parts recurrence

Integrate $J(x,m)$ by parts, choosing

$$a = (1-u)^m, \quad db = u^{x-1}du,$$

so that

$$da = -m(1-u)^{m-1}du, \quad b = \frac{u^x}{x}.$$

Boundary terms vanish (since $u^x \to 0$ as $u \to 0^+$ when $x>0$, and $(1-u)^m \to 0$ as $u \to 1^-$), hence

$$J(x,m) = \frac{m}{x}\int_0^1 u^x(1-u)^{m-1}\,du = \frac{m}{x}\,J(x+1,m-1).$$

Iterating this m times yields

$$J(x,m) = \frac{m}{x}\cdot\frac{m-1}{x+1}\cdot\frac{m-2}{x+2}\cdots\frac{1}{x+m-1}\,J(x+m,0).$$

But $J(x+m,0) = \int_0^1 u^{x+m-1}\,du = \frac{1}{x+m}$. Therefore

$$J(x,m) = \frac{m!}{x(x+1)\cdots(x+m)}.$$

Step 3: Identify with $\Gamma_m(x)$

Finally,

$$I_m(x) = m^x\,J(x,m) = m^x\,\frac{m!}{x(x+1)\cdots(x+m)} = \Gamma_m(x),$$

which is exactly the desired identity.

This derivation matches Knuth's suggested approach ("change of variable $t = mu$, integration by parts, and induction").

Remarks

1. The integral $J(x, m)$ is the beta integral $B(x, m+1)$, so the result can be summarized as

$$\Gamma_m(x) = m^x B(x, m+1) \qquad (x > 0),$$

consistent with the notation and surrounding discussion in §1.2.5.

2. In the very next exercise (20), this representation is used to prove the standard Euler integral formula

$$\Gamma(x) = \int_0^\infty e^{-t} t^{x-1}\, dt \qquad (x > 0)$$

by bounding $\Gamma(x) - \Gamma_m(x)$ and letting $m \to \infty$.

20 [HM21]

Fix $a > 0$, and for each integer $m \geq 1$ define

$$I_m(a) \;:=\; \int_0^\infty \frac{t^{a-1}}{(1 + t/m)^{m+1}}\, dt. \tag{1}$$

Step 1: Evaluate $I_m(a)$ exactly (beta–gamma algebra)

Make the substitution $u = \frac{t}{m+t}$ (so $t = \frac{mu}{1-u}$, $dt = \frac{m\,du}{(1-u)^2}$). Then

$$\begin{aligned} I_m(a) &= \int_0^1 m^a\, u^{a-1}(1-u)^{m-a}\, du = m^a\, B(a,\, m+1-a) \\ &= m^a\, \frac{\Gamma(a)\,\Gamma(m+1-a)}{\Gamma(m+1)}. \end{aligned} \tag{2}$$

(This is the standard beta integral $B(x, y) = \int_0^1 u^{x-1}(1-u)^{y-1} du$ and $B(x, y) = \Gamma(x)\Gamma(y)/\Gamma(x+y)$; see the beta–gamma relation discussed around the §1.2.5 exercises.)

Step 2: Take the limit $m \to \infty$

Use the basic gamma–ratio asymptotic (a one-line consequence of Stirling's formula): for fixed a,

$$\frac{\Gamma(m+1-a)}{\Gamma(m+1)} \sim m^{-a} \qquad (m \to \infty).$$

Hence from (2),

$$I_m(a) = m^a\,\Gamma(a)\,\frac{\Gamma(m+1-a)}{\Gamma(m+1)} \longrightarrow \Gamma(a). \tag{3}$$

(Stirling's approximation for Γ in §1.2.5/§1.2.11 justifies the ratio $ (m+1-a)/ (m+1) m^{-a}$.)

Step 3: Identify the pointwise limit of the integrand

For each fixed $t \geq 0$,

$$\frac{1}{(1+t/m)^{m+1}} \xrightarrow[m\to\infty]{} e^{-t}.$$

Thus the integrands in (1) converge pointwise to $e^{-t}t^{a-1}$.

A standard compact–tail argument (or dominated convergence with a routine bound using Stirling/ratio as above) lets us pass the limit through the integral, so the limit of $I_m(a)$ is the integral of the limit:

$$\lim_{m\to\infty} I_m(a) = \int_0^\infty e^{-t}\,t^{a-1}\,dt. \tag{4}$$

Step 4: Conclude

From (3) and (4),

$$\Gamma(a) = \int_0^\infty e^{-t}\,t^{a-1}\,dt,$$

which is Euler's integral representation of the gamma function—exactly the statement $\gamma(a,\infty) = \Gamma(a)$. (Knuth points to this identity right where he introduces the incomplete gamma function.)

Remarks

1. For integer $a = n \geq 1$, the integral reduces to $\int_0^\infty e^{-t} t^{n-1} dt = (n-1)!$, consistent with $\Gamma(n) = (n-1)!$.
2. The same proof works for complex a with $\Re(a) > 0$, giving the standard domain of Euler's integral.

21 [HM25]

Proof via Taylor expansions (partial Bell polynomials)

Write the Taylor expansions around x and around $u(x)$:

$$u(x+t) = u(x) + \sum_{i \geq 1} \frac{u^{(i)}(x)}{i!} t^i, \qquad W(u(x)+y) = \sum_{j \geq 0} \frac{W^{(j)}(u(x))}{j!} y^j.$$

Set $g(t) = W(u(x+t))$. Then

$$g(t) = \sum_{j \geq 0} \frac{W^{(j)}(u(x))}{j!} \Big(u(x+t) - u(x)\Big)^j = \sum_{j \geq 0} \frac{W^{(j)}(u)}{j!} \Big(\sum_{i \geq 1} a_i t^i\Big)^j,$$

where $a_i = u^{(i)}(x)/i!$.

Expand the j-th power by the multinomial theorem. The coefficient of t^n in $\left(\sum_{i \geq 1} a_i t^i\right)^j$ is

$$\sum_{\substack{k_1 + \cdots + k_n = j \\ 1k_1 + \cdots + nk_n = n}} \frac{j!}{k_1! \cdots k_n!} a_1^{k_1} \cdots a_n^{k_n} = \sum_k \frac{j!}{k_1! \cdots k_n!} \prod_{i=1}^n \Big(\frac{u^{(i)}(x)}{i!}\Big)^{k_i}.$$

Therefore the coefficient of t^n in $g(t)$ is

$$\sum_{j=0}^n \sum_k \frac{W^{(j)}(u)}{j!} \cdot \frac{j!}{k_1! \cdots k_n!} \prod_{i=1}^n \Big(\frac{u^{(i)}(x)}{i!}\Big)^{k_i}.$$

But $g(t) = \sum_{n \geq 0} \dfrac{D_x^n w}{n!} t^n$. Equating coefficients of t^n and multiplying by $n!$ gives

$$D_x^n w = \sum_{j=0}^n \sum_{\substack{k_1 + \cdots + k_n = j \\ 1k_1 + \cdots + nk_n = n}} \frac{n!}{k_1! \cdots k_n!} \prod_{i=1}^n \Big(\frac{u^{(i)}(x)}{i!}\Big)^{k_i} W^{(j)}(u),$$

which is exactly the required formula after grouping the $(i!)^{-k_i}$ into the denominator. This also shows the well-known form

$$\frac{d^n}{dx^n} W(u(x)) = \sum_{j=0}^{n} W^{(j)}(u)\, B_{n,j}(u', u'', \dots, u^{(n-j+1)}),$$

where $B_{n,j}$ is the partial Bell polynomial; expanding $B_{n,j}$ yields the same coefficients.

Quick sanity checks

- $n = 2$: admissible (k_1, k_2) are $(2, 0)$ with $j = 2$ and $(0, 1)$ with $j = 1$.

  $$D_x^2 w = W''(u)\,(u')^2 + W'(u)\,u'',$$

  the familiar second-derivative chain rule.

- $n = 3$: terms $(k_1, k_2, k_3) = (3, 0, 0), (1, 1, 0), (0, 0, 1)$ give

  $$D_x^3 w = W'''(u)\,(u')^3 + 3W''(u)\,u'u'' + W'(u)\,u''',$$

  as expected.

This completes the proof.

22 [HM20]

Guess a rule for the ratio $(n + x)!/n!$ as $n \to \infty$. Check it for integer x. Then say what it suggests for the "right" definition of $x!$ when x isn't an integer.

For fixed x, a very natural guess is

$$\boxed{\lim_{n\to\infty} \frac{(n+x)!}{n!\,n^x} = 1} \qquad (\star)$$

This says $(n + x)!$ grows like $n!\,n^x$ when n is large.

Check for integer x

Let $x = k \in \mathbb{Z}_{\geq 0}$. Then

$$\frac{(n+k)!}{n!\,n^k} = \prod_{i=1}^{k} \frac{n+i}{n} = \prod_{i=1}^{k} \Big(1 + \frac{i}{n}\Big) \xrightarrow[n\to\infty]{} 1,$$

so $(\star)$ is true.

If $x = -k$ with $k \in \mathbb{Z}_{\geq 0}$,

$$\frac{(n-k)!}{n!\,n^{-k}} = \frac{n^k}{n(n-1)\cdots(n-k+1)} = \prod_{i=0}^{k-1} \frac{1}{1-\frac{i}{n}} \xrightarrow[n\to\infty]{} 1,$$

so $(\star)$ also holds for negative integers. Thus the hypothesis is correct for all integer x.

What it suggests for non-integers

Assume the extension of factorial should keep the basic recursion

$$x! = x\,(x-1)! \quad \text{for all } x$$

(the same functional equation used to define Γ). Then, writing

$$(n+x)! = x!\,(x+1)(x+2)\cdots(x+n),$$

and dividing by $n!\,n^x$, the hypothesis $(\star)$ gives

$$1 = \lim_{n\to\infty} \frac{(n+x)!}{n!\,n^x} = x!\,\lim_{n\to\infty} \frac{(x+1)(x+2)\cdots(x+n)}{n!\,n^x}.$$

Hence

$$\boxed{x! \;=\; \lim_{n\to\infty} \frac{n^x\,n!}{(x+1)(x+2)\cdots(x+n)}}$$

This is exactly Euler's limit definition of the factorial for noninteger x (equivalently, $\Gamma(x+1)$), as given in the text:

$$n! \;=\; \lim_{m\to\infty} \frac{m^n\,m!}{(n+1)(n+2)\cdots(n+m)} \quad \Longleftrightarrow \quad \Gamma(x+1) = \lim_{m\to\infty} \frac{m^x\,m!}{(x+1)\cdots(x+m)}.$$

So the hypothesis not only matches integers; it directly suggests the correct extension $x! := \Gamma(x+1)$.

(Optional) Why $(\star)$ actually holds for all real x

Using Stirling's approximation $n! \sim \sqrt{2\pi n}\,(n/e)^n$, we get, for fixed real x,

$$\frac{(n+x)!}{n!\,n^x} = \frac{\Gamma(n+x+1)}{\Gamma(n+1)\,n^x} \xrightarrow[n\to\infty]{} 1,$$

since $\Gamma(n+a)/\Gamma(n+b) \sim n^{a-b}$. Thus $(\star)$ is universally valid and fully consistent with Euler's definition of Γ.

23 [HM20]

Show the reflection identity

$$(-z)!\,\Gamma(z) = \frac{\pi}{\sin(\pi z)} \qquad (z \notin \mathbb{Z}),$$

given the Euler product for sine

$$\sin(\pi z) = \pi z \prod_{n=1}^{\infty}\left(1 - \frac{z^2}{n^2}\right).$$

1. Start from the limit definition of the gamma function (Knuth's Eq. (15)):

$$\Gamma(x) = \lim_{m\to\infty} \frac{m^x\, m!}{x(x+1)\cdots(x+m)}.$$

Apply this with $x = 1+z$ and $x = 1-z$:

$$\Gamma(1+z) = \lim_{m\to\infty} \frac{m^{1+z} m!}{(1+z)(2+z)\cdots(m+1+z)}, \qquad \Gamma(1-z) = \lim_{m\to\infty} \frac{m^{1-z} m!}{(1-z)(2-z)\cdots(m+1-z)}.$$

Multiplying gives

$$\Gamma(1+z)\Gamma(1-z) = \lim_{m\to\infty} \frac{m^2 (m!)^2}{\prod_{n=1}^{m+1}\left((n+z)(n-z)\right)} = \lim_{m\to\infty} \frac{m^2 (m!)^2}{\prod_{n=1}^{m+1}(n^2 - z^2)}.$$

2. Factor out n^2 in the denominator:

$$\prod_{n=1}^{m+1}(n^2 - z^2) = \Big(\prod_{n=1}^{m+1} n^2\Big) \prod_{n=1}^{m+1}\Big(1 - \frac{z^2}{n^2}\Big) = ((m+1)!)^2 \prod_{n=1}^{m+1}\Big(1 - \frac{z^2}{n^2}\Big).$$

Hence

$$\Gamma(1+z)\Gamma(1-z) = \lim_{m\to\infty} \frac{m^2}{(m+1)^2} \cdot \frac{1}{\prod_{n=1}^{m+1}\Big(1 - \frac{z^2}{n^2}\Big)} = \prod_{n=1}^{\infty}\Big(1 - \frac{z^2}{n^2}\Big)^{-1},$$

since $m^2/(m+1)^2 \to 1$.

3. Use the given Euler sine product (the exercise's premise):

$$\sin(\pi z) = \pi z \prod_{n=1}^{\infty}\Big(1 - \frac{z^2}{n^2}\Big) \quad \Longrightarrow \quad \prod_{n=1}^{\infty}\Big(1 - \frac{z^2}{n^2}\Big)^{-1} = \frac{\pi z}{\sin(\pi z)}.$$

Therefore

$$\Gamma(1+z)\Gamma(1-z) = \frac{\pi z}{\sin(\pi z)}.$$

4. Convert to Knuth's factorial notation. Using $\Gamma(1+z) = z\,\Gamma(z)$ and $(-z)! = \Gamma(1-z)$,

$$z\,(-z)!\,\Gamma(z) = \frac{\pi z}{\sin(\pi z)} \quad \Longrightarrow \quad (-z)!\,\Gamma(z) = \frac{\pi}{\sin(\pi z)},$$

as desired (valid for noninteger z where both sides are finite).

24 [HM21]

The key idea is to compare the sum $\ln(n!) = \sum_{k=1}^{n} \ln k$ with the area under the increasing function $\ln x$.

Because $\ln x$ is increasing on $[1, \infty)$,

$$\int_1^n \ln x\,dx \;\le\; \sum_{k=1}^{n} \ln k \;\le\; \int_1^n \ln x\,dx + \ln n.$$

(Left inequality: on each interval $[k, k+1)$, $\ln x \ge \ln k$; summing gives the integral $\le$ the sum. Right inequality: write $\sum_{k=1}^{n} \ln k = \ln n + \sum_{k=1}^{n-1} \ln k \le \ln n + \int_1^n \ln x\,dx$.)

Evaluate the integral:

$$\int_1^n \ln x\,dx = \big[x \ln x - x\big]_1^n = n \ln n - n + 1.$$

Hence

$$n \ln n - n + 1 \;\le\; \ln(n!) \;\le\; (n+1)\ln n - n + 1.$$

Exponentiating gives exactly

$$\boxed{\frac{n^n}{e^{n-1}} \;\le\; n! \;\le\; \frac{n^{n+1}}{e^{n-1}}}$$

for all integers $n \ge 1$.

How the hint connects (optional intuition)

From $1 + x \le e^x$ we get

$$\frac{k+1}{k} \le e^{1/k} \quad \text{and} \quad e^{1/k} \le \frac{k}{k-1} \qquad (k \ge 2).$$

Multiplying the first inequality for $k = 1, \dots, n-1$ yields

$$n = \prod_{k=1}^{n-1} \frac{k+1}{k} \le \exp\Big(\sum_{k=1}^{n-1} \frac{1}{k}\Big),$$

so $\ln n \le H_{n-1}$. Multiplying the second for $k = 2, \dots, n$ gives

$$\exp\Big(\sum_{k=2}^{n} \frac{1}{k}\Big) \le \prod_{k=2}^{n} \frac{k}{k-1} = n,$$

so $H_n \le 1 + \ln n$. These classical bounds ($\ln n \le H_{n-1} < H_n \le 1 + \ln n$) are the discrete counterpart of the integral comparisons above; integrating $\ln x$ is simply the cleanest way to turn them into bounds on $\ln(n!)$.

Tightness and context

These inequalities are crude but *uniformly valid* for all $n \ge 1$. They're consistent with Stirling's formula $n! \sim \sqrt{2\pi n}\,(n/e)^n$, which refines both sides by the missing $\sqrt{2\pi n}$ factor and lower-order terms; our bounds differ only by a factor of n in front and match the same dominant $(n/e)^n$ behavior.

25 [M20]

Definitions

- Falling factorial (a.k.a. "to the n falling"):

$$x^{\underline{n}} = x(x-1)(x-2)\cdots(x-n+1) \quad (n \ge 0).$$

- Rising factorial (a.k.a. Pochhammer symbol):

$$x^{\overline{n}} = x(x+1)(x+2)\cdots(x+n-1) \quad (n \geq 0).$$

Knuth also records the useful gamma–function forms

$$x^{\underline{n}} = \frac{\Gamma(x+1)}{\Gamma(x+1-n)}, \qquad x^{\overline{n}} = \frac{\Gamma(x+n)}{\Gamma(x)}.$$

Proofs for integers $m, n \geq 0$

Falling factorial

$$\begin{aligned} x^{\underline{m+n}} &= \prod_{j=0}^{m+n-1}(x-j) = \left(\prod_{j=0}^{m-1}(x-j)\right)\left(\prod_{j=m}^{m+n-1}(x-j)\right) \\ &= x^{\underline{m}} \cdot \prod_{j=0}^{n-1}((x-m)-j) = x^{\underline{m}}\,(x-m)^{\underline{n}}. \end{aligned}$$

Rising factorial

$$\begin{aligned} x^{\overline{m+n}} &= \prod_{j=0}^{m+n-1}(x+j) = \left(\prod_{j=0}^{m-1}(x+j)\right)\left(\prod_{j=m}^{m+n-1}(x+j)\right) \\ &= x^{\overline{m}} \cdot \prod_{j=0}^{n-1}((x+m)+j) = x^{\overline{m}}\,(x+m)^{\overline{n}}. \end{aligned}$$

These are just regroupings of consecutive factors.

Extension to noninteger m, n

Using the gamma–function definitions (valid wherever the gamma function is defined), we have

$$\begin{aligned} x^{\underline{m+n}} &= \frac{\Gamma(x+1)}{\Gamma(x+1-(m+n))} = \frac{\Gamma(x+1)}{\Gamma(x+1-m)} \cdot \frac{\Gamma(x+1-m)}{\Gamma(x+1-m-n)} \\ &= x^{\underline{m}}\,(x-m)^{\underline{n}}, \\ x^{\overline{m+n}} &= \frac{\Gamma(x+m+n)}{\Gamma(x)} = \frac{\Gamma(x+m+n)}{\Gamma(x+m)} \cdot \frac{\Gamma(x+m)}{\Gamma(x)} \\ &= (x+m)^{\overline{n}}\,x^{\overline{m}}. \end{aligned}$$

Thus the identities hold for real (indeed complex) m, n and x, subject only to avoiding poles of the gamma function (i.e., where the denominators would be Γ at nonpositive integers). This is exactly the extension Knuth alludes to.

Quick sanity check

Take $x = 7$, $m = 3$, $n = 2$.

- Falling: $7^{\underline{5}} = 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3$. Right side: $7^{\underline{3}}(7-3)^{\underline{2}} = (7 \cdot 6 \cdot 5)(4 \cdot 3)$. Same product.
- Rising: $7^{\overline{5}} = 7 \cdot 8 \cdot 9 \cdot 10 \cdot 11$. Right side: $7^{\overline{3}}(7+3)^{\overline{2}} = (7 \cdot 8 \cdot 9)(10 \cdot 11)$. Same product.

1.2.6 Binomial coefficients

1 [00]

1) Combinatorial bijection

Choose $(n-1)$ elements from an n-element set. Such a choice is uniquely determined by the one element that is omitted. Each of the n elements can be the omitted one, and different omissions give different $(n-1)$-subsets. Hence there are exactly n such subsets.

2) Algebraic identity

By definition of binomial coefficients,

$$\binom{n}{n-1} = \frac{n!}{(n-1)!\,1!} = n.$$

3) Pascal recurrence cross-check

Pascal's rule says

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}.$$

With $k = n-1$,

$$\binom{n}{n-1} = \binom{n-1}{n-1} + \binom{n-1}{n-2} = 1 + \binom{n-1}{1} = 1 + (n-1) = n.$$

2 [00]

Answer

$$\boxed{\binom{0}{0} = 1}.$$

Why this is correct (several equivalent viewpoints)

1. Combinatorial meaning (subsets). $\binom{n}{k}$ counts the number of k-element subsets of an n-element set. When $n = 0$ and $k = 0$, we're counting 0-element subsets of the empty set. There is exactly one such subset—the empty set itself—so $\binom{0}{0} = 1$. (This is the official "answers" line in TAOCP: "There's exactly one way to choose nothing from the empty set.")

2. Binomial theorem boundary case. For any integer $n \geq 0$,

$$(1 + x)^n = \sum_{k=0}^{n} \binom{n}{k} x^k.$$

Putting $n = 0$ gives $(1 + x)^0 = 1$. The right side has only the $k = 0$ term, so $\binom{0}{0}x^0 = \binom{0}{0}$. Hence $\binom{0}{0} = 1$.

3. Factorial formula with the empty product convention. For $0 \leq k \leq n$,

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Using $0! = 1$ (the empty product), we get

$$\binom{0}{0} = \frac{0!}{0!\,0!} = \frac{1}{1 \cdot 1} = 1.$$

4. Pascal's triangle boundary. Pascal's identity $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$ is solved uniquely by the boundary conditions $\binom{n}{0} = \binom{n}{n} = 1$ for all $n \geq 0$. Setting $n = 0$ yields $\binom{0}{0} = 1$.

3 [00]

A 13-card hand is a *combination*: a subset of size 13 from a set of size 52, where order doesn't matter (this is exactly the setup of "combinations of n objects taken k at a time").

1. First count ordered selections of 13 distinct cards:

$$52 \cdot 51 \cdot 50 \cdots 40 \;=\; \frac{52!}{39!}.$$

2. Each unordered hand is counted 13! times here (once for every permutation of its 13 cards). Therefore, divide by 13!:

$$\binom{52}{13} \;=\; \frac{52!}{13!\,39!}.$$

This is the exact number of bridge hands.

4 [10]

Method: Legendre's formula on a binomial

For a prime p,

$$v_p(n!) = \sum_{i\geq 1} \left\lfloor \frac{n}{p^i} \right\rfloor, \qquad v_p\binom{n}{k} = v_p(n!) - v_p(k!) - v_p((n-k)!).$$

Compute $v_p(\binom{52}{13})$ only for primes $p \leq 52$; most will cancel to 0.

Large primes (quick eliminations)

- If $40 \leq p \leq 52$ and p is prime, then $v_p(52!) = 1$ while $v_p(39!) = v_p(13!) = 0$. Hence $v_p = 1$ for $p \in \{41, 43, 47\}$.
- If $26 \leq p \leq 39$ and p is prime, then $v_p(52!) = 1$, $v_p(39!) = 1$, $v_p(13!) = 0$. Hence $v_p = 0$ for $p \in \{29, 31, 37\}$.

Middle primes

- $p = 23:\ v_{23}(52!) = \lfloor 52/23 \rfloor + \lfloor 52/23^2 \rfloor = 2,\ v_{23}(39!) = 1,\ v_{23}(13!) = 0 \Rightarrow v_{23} = 1.$
- $p = 19:\ 2 - 2 - 0 = 0$ so no factor 19.
- $p = 17:\ 3 - 2 - 0 = 1 \Rightarrow v_{17} = 1.$

Small primes

- $p = 13:\ 4 - 3 - 1 = 0 \Rightarrow$ no factor 13.
- $p = 11:\ 4 - 3 - 1 = 0 \Rightarrow$ no factor 11.
- $p = 7:\ (\lfloor 52/7 \rfloor + \lfloor 52/49 \rfloor) - \lfloor 39/7 \rfloor - \lfloor 13/7 \rfloor = (7+1) - 5 - 1 = 2 \Rightarrow v_7 = 2.$
- $p = 5:\ (10+2) - (7+1) - 2 = 12 - 8 - 2 = 2 \Rightarrow v_5 = 2.$
- $p = 3:\ (17+5+1) - (13+4+1) - (4+1) = 23 - 18 - 5 = 0 \Rightarrow$ no factor 3.
- $p = 2:\ (26+13+6+3+1) - (19+9+4+2+1) - (6+3+1) = 49 - 35 - 10 = 4 \Rightarrow v_2 = 4.$

Result (prime-power form)

$$\boxed{\binom{52}{13} = 2^4\, 5^2\, 7^2\, 17\, 23\, 41\, 43\, 47.}$$

Checks and intuition.

- Primes > 39 must appear once (they occur in 52! but not in 39! or 13!).
- Primes in $[26, 39]$ cancel.
- What remains are the computed exponents above.

5 [05]

The binomial theorem states

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k} b^k.$$

Take $a = 10$, $b = 1$, $n = 4$. The needed binomial coefficients are the 4th row of Pascal's triangle:

$$\binom{4}{0}, \binom{4}{1}, \binom{4}{2}, \binom{4}{3}, \binom{4}{4} = 1, 4, 6, 4, 1.$$

Therefore

$$(10+1)^4 = \binom{4}{0} 10^4 + \binom{4}{1} 10^3 + \binom{4}{2} 10^2 + \binom{4}{3} 10 + \binom{4}{4} = 10000 + 4 \cdot 1000 + 6 \cdot 100 + 4 \cdot 10 + 1.$$

Compute the sum:

$$10000 + 4000 + 600 + 40 + 1 = \boxed{14641}.$$

6 [10]

We interpret a "row r" as the infinite sequence $(\binom{r}{0}, \binom{r}{1}, \binom{r}{2}, \dots)$, with the convention $\binom{r}{k} = 0$ for $k < 0$. The ordinary top row is $r = 0$: $(1, 0, 0, \dots)$.

The addition law lets us solve upward: for each fixed r and $k \geq 0$,

$$\binom{r-1}{k} = \binom{r}{k} - \binom{r-1}{k-1}, \qquad \text{with} \quad \binom{r-1, -1}{=} 0.$$

Row $r = -1$

Use the equation with $r = 0$ and $C(r, k) := \binom{r}{k}$:

$$C(0, k) = C(-1, k) + C(-1, k-1).$$

Since $C(0, 0) = 1$ and $C(0, k) = 0$ for $k \geq 1$,

$$C(-1, 0) = 1, \quad 0 = C(-1, 1) + 1 \Rightarrow C(-1, 1) = -1,$$

and inductively $C(-1, k) = -C(-1, k-1)$. Thus

$$\boxed{r = -1 : \ 1, \ -1, \ 1, \ -1, \ 1, \ -1, \ \dots}$$

Row $r = -2$

Now use $C(-1, k) = C(-2, k) + C(-2, k-1)$:

$$C(-2, 0) = 1, \quad -1 = C(-2, 1) + 1 \Rightarrow C(-2, 1) = -2,$$

$$1 = C(-2, 2) + (-2) \Rightarrow C(-2, 2) = 3, \quad -1 = C(-2, 3) + 3 \Rightarrow C(-2, 3) = -4, \ \dots$$

Hence

$$\boxed{r = -2 : \ 1, \ -2, \ 3, \ -4, \ 5, \ -6, \ 7, \ -8, \ \dots}$$

Row $r = -3$

Use $C(-2, k) = C(-3, k) + C(-3, k-1)$:

$$C(-3,0)=1, \quad -2=C(-3,1)+1 \Rightarrow C(-3,1)=-3,$$

$$3=C(-3,2)+(-3) \Rightarrow C(-3,2)=6, \quad -4=C(-3,3)+6 \Rightarrow C(-3,3)=-10, \ \ldots$$

Therefore

$$\boxed{r=-3: \ 1, \ -3, \ 6, \ -10, \ 15, \ -21, \ 28, \ -36, \ \ldots}$$

7 [12]

We'll use two basic identities from §1.2.6:

1. The definition (valid for real r and integer k)

$$\binom{r}{k} = \frac{r(r-1)\cdots(r-k+1)}{k(k-1)\cdots 1} \quad (k \geq 0), \qquad \binom{r}{k} = 0 \ (k<0).$$

This is Eq. (3) in the book.

2. Symmetry (for integer $n \geq 0$, integer k)

$$\binom{n}{k} = \binom{n}{n-k}.$$

This is Eq. (6).

1. Compare consecutive entries

From the factorial/"falling product" form (3) it's immediate that for $0 \leq k < n$,

$$\frac{\binom{n}{k+1}}{\binom{n}{k}} = \frac{n-k}{k+1}.$$

Therefore

- $\binom{n}{k+1} > \binom{n}{k}$ iff $n-k > k+1$, i.e. $k < \frac{n-1}{2}$.
- $\binom{n}{k+1} = \binom{n}{k}$ iff $k = \frac{n-1}{2}$.
- $\binom{n}{k+1} < \binom{n}{k}$ iff $k > \frac{n-1}{2}$.

Thus, as k increases from 0, the row is *strictly increasing* up to the midpoint and *strictly decreasing* afterwards.

2. Locate the maximum

Two cases:

- $n = 2m$ (even). The ratio exceeds 1 for $k < m$ and is < 1 for $k \geq m$. Hence the unique maximum is at $k = m = n/2$.
- $n = 2m + 1$ (odd). The ratio equals 1 at $k = m$:

$$\frac{\binom{2m+1}{m+1}}{\binom{2m+1}{m}} = \frac{(2m+1) - m}{m+1} = 1.$$

  Therefore $\binom{2m+1}{m} = \binom{2m+1}{m+1}$, and these adjacent central entries are the (equal) maxima. Symmetry (6) explains the tie.

3. Final answer

The largest binomial coefficient(s) in row n occur at

$$k = \left\lfloor \frac{n}{2} \right\rfloor \quad \text{and (if } n \text{ is odd) also at} \quad k = \left\lceil \frac{n}{2} \right\rceil.$$

Equivalently: the maximum is at the middle of the row; if n is odd, the two middle entries are equal and both maximal. (Knuth's official answer states the same conclusion and notes the strict increase before the midpoint, then decrease, "clear from (3)".)

Quick sanity check (examples)

- $n = 6$: row is $1, 6, 15, 20, 15, 6, 1 \to$ max at $k = 3 = \lfloor 6/2 \rfloor$.
- $n = 7$: row is $1, 7, 21, 35, 35, 21, 7, 1 \to$ maxima at $k = 3, 4 = \lfloor 7/2 \rfloor, \lceil 7/2 \rceil$.

8 [00]

Row n of Pascal's triangle lists the binomial coefficients

$$\binom{n}{0}, \binom{n}{1}, \ldots, \binom{n}{n}.$$

In that row, the entry k places from the left is $\binom{n}{k}$, while the entry k places from the right is $\binom{n}{n-k}$. Equation (6) asserts these two numbers are equal. Therefore every row reads the same

left-to-right as right-to-left: Pascal's triangle is mirror-symmetric about its vertical center line (the "nonzero entries in each row are palindromic).

Two standard viewpoints that make this symmetry transparent:

- Algebraic (from factorials).

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n!}{(n-k)!k!} = \binom{n}{n-k}.$$

  This is exactly Eq. (6).

- Combinatorial (by complements). Choosing k elements out of an n-element set is in bijection with choosing which $n-k$ elements to leave out. Hence the counts are equal, $\binom{n}{k} = \binom{n}{n-k}$, yielding the row-wise mirror symmetry.

Thus, the property reflected by Eq. (6) is the left-right symmetry of each row of Pascal's triangle. (Knuth's answer phrase: "the nonzero entries in each row are the same from left to right as from right to left.")

Example. Row $n = 4$: 1, 4, 6, 4, 1 — clearly symmetric; the middle term $\binom{4}{2}$ stands on the axis.

9 [01]

Knuth defines the generalized binomial coefficient for real x and integer k by

$$\binom{x}{k} = \begin{cases} \dfrac{x^{\underline{k}}}{k!}, & k \geq 0, \\ 0, & k < 0, \end{cases}$$

where $x^{\underline{k}} = x(x-1)\cdots(x-k+1)$ is the falling factorial. Thus negative lower indices are by definition zero.

Consider two cases.

1. $n \geq 0$. Then

$$\binom{n}{n} = \frac{n^{\underline{n}}}{n!} = \frac{n!}{n!} = 1.$$

(Combinatorially: there is exactly one way to choose all n elements of an n-set.)

2. $n < 0$. Here the lower index equals $k = n < 0$, so by the definition above,

$$\binom{n}{n} = 0.$$

This is also consistent with Knuth's summation convention for the binomial expansion: terms with $k < 0$ (or, for nonnegative exponents, $k > n$) are taken as zero.

Answer. $\boxed{\binom{n}{n} = 1 \text{ if } n \geq 0; \ \binom{n}{n} = 0 \text{ if } n < 0.}$

10 [M25]

Lemma (middle binomials vanish mod p). For $1 \leq k \leq p-1$,

$$\binom{p}{k} = \frac{p!}{k!(p-k)!} \equiv 0 \pmod{p}.$$

Reason. The numerator $p!$ has a factor p; the denominator $k!(p-k)!$ has none (all factors $< p$). Hence (b) holds immediately.

A handy corollary over the finite field $\mathbb{F}_p$ is the Freshman's dream

$$(1+X)^p \equiv 1 + X^p \pmod{p},$$

since all intermediate binomial coefficients vanish by (b).

(e) Lucas's theorem (main result)

Write $n = ap + b$, $k = cp + d$ with $a, c \geq 0$ and $0 \leq b, d < p$. Over $\mathbb{F}_p$,

$$(1+X)^n = (1+X)^{ap+b} = \big((1+X)^p\big)^a (1+X)^b \equiv (1+X^p)^a (1+X)^b \pmod{p},$$

using the Freshman's dream. Expand the right-hand side:

$$(1+X^p)^a(1+X)^b = \Big(\sum_{i=0}^{a} \binom{a}{i} X^{ip}\Big)\Big(\sum_{j=0}^{b} \binom{b}{j} X^j\Big).$$

The coefficient of X^{cp+d} is $\binom{a}{c}\binom{b}{d}$. On the other hand, that coefficient is $\binom{n}{k}$. Therefore

$$\binom{n}{k} \equiv \binom{a}{c}\binom{b}{d} = \binom{\lfloor n/p \rfloor}{\lfloor k/p \rfloor}\binom{n \bmod p}{k \bmod p} \pmod{p},$$

which is exactly (e). (Statement of (e) appears in the exercise list.)

Immediate corollaries of Lucas

From (e) we get (a), (b) again, and (f) almost at a glance.

- (a) Take $k = p$. With $n = ap + b$ and $k = 1 \cdot p + 0$, Lucas gives $\binom{n}{p} \equiv \binom{a}{1}\binom{b}{0} = a = \lfloor n/p \rfloor \pmod{p}$.
- (b) Take $n = p = 1 \cdot p + 0$ and $1 \leq k \leq p-1$ (so $k = 0 \cdot p + k$). Then $\binom{p}{k} \equiv \binom{1}{0}\binom{0}{k} = 1 \cdot 0 = 0 \pmod{p}$.
- (f) Apply (e) repeatedly to the higher p-adic digits. Writing $n = \sum a_i p^i,\ k = \sum b_i p^i$, a short induction yields $\displaystyle\binom{n}{k} \equiv \prod_{i=0}^{r} \binom{a_i}{b_i} \pmod{p}$, as claimed.

(c) and (d) from (a), (b) and the addition formula

Recall the addition formula (Pascal's rule), Eq. (9):

$$\binom{r}{k} = \binom{r-1}{k} + \binom{r-1}{k-1}.$$

(See the text right before the exercises.)

- (c) For $0 \leq k \leq p-1$,

$$0 \equiv \binom{p}{k} = \binom{p-1}{k} + \binom{p-1}{k-1} \pmod{p}$$

  by (b), so $\binom{p-1}{k} \equiv -\binom{p-1}{k-1} \pmod{p}$. With $\binom{p-1}{0} \equiv 1$, induction gives $\binom{p-1}{k} \equiv (-1)^k$.
- (d) For $2 \leq k \leq p-1$,

$$\binom{p+1}{k} = \binom{p}{k} + \binom{p}{k-1} \equiv 0 + 0 \equiv 0 \pmod{p},$$

  since both terms vanish by (b).

Sanity checks

- For $p = 5$, $n = 23$, (a) says $\binom{23}{5} \equiv \lfloor 23/5 \rfloor = 4 \pmod 5$. Indeed $\binom{23}{5} = 33{,}6492 \equiv 4 \pmod 5$.
- For $p = 2$, (f) yields the classic parity rule: $\binom{n}{k}$ is odd iff every binary digit $b_i \le a_i$ (i.e., k is a bitwise subset of n).

11 [M20]

1) Reduce to a p-adic valuation of the binomial

Write $v_p(m)$ for the exponent of p in m. Then

$$v_p\binom{a+b}{a} = v_p((a+b)!) - v_p(a!) - v_p(b!).$$

2) Use Legendre's formula via sum of digits

If $s_p(n)$ denotes the sum of the base-p digits of n, Legendre's theorem gives

$$v_p(n!) \;=\; \frac{n - s_p(n)}{p-1}.$$

(Equivalently, $v_p(n!) = \sum_{i\ge1} \lfloor n/p^i \rfloor$ and $\sum_{i\ge1} \lfloor n/p^i \rfloor = (n - s_p(n))/(p-1)$.)

Applying this to $n = a$, b, and $a + b$,

$$v_p\binom{a+b}{a} = \frac{s_p(a) + s_p(b) - s_p(a+b)}{p-1}. \qquad (\ )$$

3) "Carries = drop in digit-sum divided by $p - 1$"

Let $a = \sum_i a_i p^i$, $b = \sum_i b_i p^i$ with $0 \le a_i, b_i \le p-1$. Perform schoolbook addition in base p. Let $c_i \in \{0, 1\}$ be the carry *into* position i (so $c_0 = 0$), and r_i the resulting digit at position i. Then for each i,

$$a_i + b_i + c_i \;=\; r_i + p\,c_{i+1} \quad (0 \le r_i \le p-1).$$

Summing over all positions and rearranging yields the identity

$$s_p(a) + s_p(b) \;=\; s_p(a+b) + (p-1)\sum_i c_{i+1}.$$

Hence the number of carries $C := \sum_i c_{i+1}$ satisfies

$$C = \frac{s_p(a) + s_p(b) - s_p(a+b)}{p-1}. \tag{†}$$

4) Combine ( ) and (†)

From ( ) and (†) we get

$$v_p\binom{a+b}{a} \;=\; C,$$

i.e., the exponent of p dividing $\binom{a+b}{a}$ equals the number of carries when adding a and b in base p. This proves the claim.

Quick check (example)

Let $p = 2$, $a = 5 = (101)_2$, $b = 7 = (111)_2$. Adding

$$101 + \;111 = \;1100$$

produces carries in the 0th, 1st, and 2nd bit positions: $C = 3$. $\binom{12}{5} = 792 = 2^3 \cdot 99$, so $v_2\binom{12}{5} = 3 = C$, as predicted.

12 [M22]

Yes. Precisely those n of the form

$$n = 2^m - 1 \quad (m \geq 1).$$

Proof 1 (via Lucas's theorem, base 2)

Write n and k in base 2:

$$n = \sum_{i\geq 0} n_i\, 2^i, \qquad k = \sum_{i\geq 0} k_i\, 2^i, \qquad n_i, k_i \in \{0,1\}.$$

Lucas's theorem (exercise 10(e–f)) states

$$\binom{n}{k} \equiv \prod_{i\geq 0} \binom{n_i}{k_i} \pmod 2.$$

Hence $\binom{n}{k}$ is odd iff $\binom{n_i}{k_i} \not\equiv 0 \pmod 2$ for every i; over $\mathbb{F}_2$ this means $k_i \le n_i$ for all i. Equivalently, the set of odd entries in row n are exactly those with bitwise k contained in the 1-bits of n. In particular, the number of odd entries equals $2^{\#\{i:n_i=1\}}$.

Therefore, all $n+1$ entries are odd iff

$$n+1 = 2^{\#\{i:n_i=1\}},$$

which happens precisely when every binary digit of n up to its most significant bit is 1, i.e., $n = 2^m - 1$. Conversely, if $n = 2^m - 1$ then every $n_i = 1$, so $\binom{n}{k} \equiv \prod_i \binom{1}{k_i} \equiv 1 \pmod 2$ for all k, and the whole row is odd.

(Lucas's theorem as stated in the exercises: $\binom{n}{k} \equiv \binom{\lfloor n/2 \rfloor}{\lfloor k/2 \rfloor}\binom{n \bmod 2}{k \bmod 2} \pmod 2$*; the general base-p digitwise product form is also given.)*

Proof 2 (via Kummer's theorem)

Kummer's theorem (exercise 11) says the exponent of a prime p in $\binom{a+b}{a}$ equals the number of carries when adding a and b in base p. Take $p = 2$, $a = k$, $b = n - k$. Then $v_2\binom{n}{k} = 0$ (i.e., $\binom{n}{k}$ odd) iff adding k and $n - k$ in binary produces no carries. If n has any 0 among its binary digits, choose k with a 1 in that position and 0 elsewhere; adding k to $n - k$ then forces a carry at that bit, making $\binom{n}{k}$ even—a contradiction. Thus every binary digit of n must be 1, i.e., $n = 2^m - 1$. Conversely, if $n = 2^m - 1$, any $k + (n - k) = n$ has no carries, so all entries are odd.

(Bonus) Generalization:

For a prime p: none of $\binom{n}{k}$ (with $0 \le k \le n$) is divisible by p iff $n = ap^m - 1$ with $1 \le a < p$, $m \ge 0$. (This is the exercise's official "more generally" statement.)

Quick check. $n = 1, 3, 7, 15, \ldots \to$ rows $(1, 1)$, $(1, 3, 3, 1)$, $(1, 7, 21, 35, 35, 21, 7, 1)$, … are all odd, as claimed.

13 [M13]

Proof 1 (telescoping via Pascal's rule)

Pascal's addition formula is

$$\binom{r}{k} = \binom{r-1}{k} + \binom{r-1}{k-1} \qquad \text{(integer } k\text{)},$$

equivalently

$$\binom{r+k}{k} = \binom{r+k+1}{k} - \binom{r+k}{k-1}.$$

(Simply set $r \leftarrow r+k+1,\ k \leftarrow k$ in Pascal's rule and rearrange.)

Now sum that identity for $k = 0, 1, \dots, n$:

$$\sum_{k=0}^{n} \binom{r+k}{k} = \sum_{k=0}^{n} \Big(\binom{r+k+1}{k} - \binom{r+k}{k-1} \Big).$$

Shift index $j = k-1$ in the second sum. Using the convention $\binom{\cdot}{-1} = 0$,

$$\sum_{k=0}^{n} \binom{r+k}{k} = \sum_{k=0}^{n} \binom{r+k+1}{k} - \sum_{j=-1}^{n-1} \binom{r+j+1}{j} = \binom{r+n+1}{n},$$

because all intermediate terms cancel (telescoping), leaving only the last term. This is exactly Eq. (10).

Proof 2 (combinatorial "largest element" argument)

By symmetry $\binom{r+k}{k} = \binom{r+k}{r}$. Thus

$$\sum_{k=0}^{n} \binom{r+k}{k} = \sum_{k=0}^{n} \binom{r+k}{r} = \sum_{i=r}^{r+n} \binom{i}{r}.$$

Count the \$ (r+1)\$-subsets of $\{1, 2, \dots, r+n+1\}$ by the value of their largest element. If the largest element is $i+1$ (where $i = r, r+1, \dots, r+n$), the other r elements can be any r-subset of $\{1, \dots, i\}$, counted by $\binom{i}{r}$. Summing over possible i gives

$$\sum_{i=r}^{r+n} \binom{i}{r} = \binom{r+n+1}{r+1} = \binom{r+n+1}{n},$$

which is the desired identity.

Remarks:

- The statement is given in the text with n an integer ≥ 0; once established for infinitely many r (e.g., integer r), one can extend to all real r by the "polynomial identity" principle noted just above Eq. (9): both sides are polynomials in r that agree at infinitely many points, hence agree identically.

- This is exactly the summation formula tagged (10) in §1.2.6—the exercise asks you to prove it.

Sanity check. For $r = 3, n = 2$: $\binom{3}{0} + \binom{4}{1} + \binom{5}{2} = 1 + 4 + 10 = 15 = \binom{6}{2}$.

14 [M21]

We will show

$$\sum_{k=1}^{n} k^4 = 24\binom{n+1}{5} + 36\binom{n+1}{4} + 14\binom{n+1}{3} + \binom{n+1}{2},$$

and hence obtain the usual polynomial form for $\sum_{k=1}^{n} k^4$.

1) Write k^4 in the binomial (Newton) basis

Every polynomial $P(k)$ of degree $\leq m$ has a unique expansion in the falling–factorial basis:

$$P(k) = \sum_{j=0}^{m} \Delta^j P(0)\binom{k}{j},$$

where Δ is the forward difference $(\Delta f)(k) = f(k+1) - f(k)$. For $P(k) = k^4$ we compute successive differences:

$$\begin{aligned}
\Delta k^4 &= 4k^3 + 6k^2 + 4k + 1, \\
\Delta^2 k^4 &= 12k^2 + 24k + 14, \\
\Delta^3 k^4 &= 24k + 36, \\
\Delta^4 k^4 &= 24, \qquad \Delta^5 k^4 = 0.
\end{aligned}$$

Evaluating at $k = 0$ gives

$$\Delta^1 P(0) = 1, \quad \Delta^2 P(0) = 14, \quad \Delta^3 P(0) = 36, \quad \Delta^4 P(0) = 24.$$

Hence

$$k^4 = \binom{k}{1} + 14\binom{k}{2} + 36\binom{k}{3} + 24\binom{k}{4}. \qquad (\ )$$

(Equivalently, this is the well-known identity $k^4 = \sum_{j=1}^{4} j!\, S(4,j)\binom{k}{j}$ with Stirling numbers $S(4,1) = 1$, $S(4,2) = 7$, $S(4,3) = 6$, $S(4,4) = 1$; see Knuth's discussion of Stirling numbers within §1.2.6.)

2) Sum and use the "hockey-stick" identity

Sum ( ) from $k = 1$ to n and apply

$$\sum_{k=j}^{n} \binom{k}{j} = \binom{n+1}{j+1} \quad (j \geq 0),$$

which follows immediately from Pascal's rule for binomial coefficients in §1.2.6.

Thus

$$\sum_{k=1}^{n} k^4 = \binom{n+1}{2} + 14\binom{n+1}{3} + 36\binom{n+1}{4} + 24\binom{n+1}{5},$$

exactly as stated.

3) Convert to the standard closed form

Substitute $\binom{n+1}{2} = \frac{n(n+1)}{2}$, $\binom{n+1}{3} = \frac{n(n+1)(n-1)}{6}$, $\binom{n+1}{4} = \frac{n(n+1)(n-1)(n-2)}{24}$, $\binom{n+1}{5} = \frac{n(n+1)(n-1)(n-2)(n-3)}{120}$, and simplify to

$$\boxed{\sum_{k=1}^{n} k^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30} = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}.}$$

4) Quick check

For $n = 1, 2, 3$, the left side gives 1, 17, 98; the closed form yields the same.

15 [M15]

For a non-negative integer n,

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}.$$

Proof 1 (combinatorial)

When you expand $(x+y)^n$, you are multiplying n factors

$$(x+y)(x+y)\cdots(x+y).$$

From each factor you must choose either x or y. A term of the form $x^k y^{n-k}$ arises precisely when you choose x from exactly k of the n factors and choose y from the remaining $n-k$ factors.

How many such choices are there? Exactly $\binom{n}{k}$: the number of ways to select which k positions (among the n factors) contribute an x. Therefore the coefficient of $x^k y^{n-k}$ in the expansion is $\binom{n}{k}$, yielding

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}.$$

This completes the proof.

Proof 2 (algebraic, via Pascal's rule)

We will show that the coefficients in $(x+y)^n$ satisfy the same recursion and base cases as binomial coefficients, hence must be equal.

Write

$$(x+y)^n = \sum_{k=0}^{n} c_{n,k}\, x^k y^{n-k},$$

and define $c_{n,k} = 0$ if $k < 0$ or $k > n$.

Base case. For $n = 0$: $(x+y)^0 = 1$, so $c_{0,0} = 1$ and $c_{0,k} = 0$ for $k \neq 0$. This matches $\binom{0}{0} = 1$ and $\binom{0}{k} = 0$ for $k \neq 0$.

Recurrence. Multiply by $(x+y)$:

$$(x+y)^{n+1} = (x+y)\sum_{k=0}^{n} c_{n,k}\, x^k y^{n-k} = \sum_{k=0}^{n} c_{n,k}\, x^{k+1} y^{n-k} + \sum_{k=0}^{n} c_{n,k}\, x^k y^{n+1-k}.$$

Reindex the first sum with $j = k+1$ to express everything in powers $x^j y^{n+1-j}$:

$$(x+y)^{n+1} = \sum_{j=0}^{n+1} (c_{n,j} + c_{n,j-1}) x^j y^{n+1-j},$$

(with $c_{n,-1} = c_{n,n+1} = 0$). Hence

$$c_{n+1,j} = c_{n,j} + c_{n,j-1} \qquad \text{for all } j.$$

This is exactly Pascal's rule. Since $\binom{n}{k}$ has the same base case and satisfies the same recursion,

$$c_{n,k} = \binom{n}{k} \quad \text{for all } n, k,$$

and the binomial identity follows.

16 [M15]

For integers $n, k \geq 1$,

$$(-1)^n \binom{-n}{k-1} = (-1)^k \binom{-k}{n-1}.$$

We'll give two short, complementary proofs. Throughout we use the generalized binomial coefficient definition

$$\binom{r}{k} = \frac{r(r-1)\cdots(r-k+1)}{k!} \quad \text{for integer } k \geq 0,$$

and the symmetry $\binom{n}{k} = \binom{n}{n-k}$ (for integer $n \geq 0$).

Proof 1 (direct calculation)

Start from the definition with a negative top index. For integer $m \geq 0$,

$$\binom{-n}{m} = \frac{(-n)(-n-1)\cdots(-n-m+1)}{m!} = (-1)^m \frac{n(n+1)\cdots(n+m-1)}{m!}.$$

The product in the numerator is a rising run, so

$$n(n+1)\cdots(n+m-1) = \frac{(n+m-1)!}{(n-1)!}.$$

Hence the useful identity

$$\boxed{\binom{-n}{m} = (-1)^m \binom{n+m-1}{m}}. \qquad (\ )$$

Apply ( ) twice, with $m = k-1$ and with $m = n-1$:

$$(-1)^n \binom{-n}{k-1} = (-1)^n (-1)^{k-1} \binom{n+k-2}{k-1} = (-1)^{n+k-1} \binom{n+k-2}{k-1},$$

$$(-1)^k \binom{-k}{n-1} = (-1)^k (-1)^{n-1} \binom{k+n-2}{n-1} = (-1)^{n+k-1} \binom{n+k-2}{n-1}.$$

By symmetry of binomial coefficients, $\binom{n+k-2}{k-1} = \binom{n+k-2}{n-1}$, so the two expressions are equal.

Proof 2 (a slightly more general symmetry)

A compact variant proves the more general identity, valid for all $n, k \geq 0$,

$$\boxed{(-1)^n \binom{-n}{k} = (-1)^k \binom{-k}{n}}$$

and then substitutes $k \leftarrow k-1$.

Using ( ),

$$(-1)^n \binom{-n}{k} = (-1)^n (-1)^k \binom{n+k-1}{k} = (-1)^{n+k} \binom{n+k-1}{k},$$

$$(-1)^k \binom{-k}{n} = (-1)^k (-1)^n \binom{n+k-1}{n} = (-1)^{n+k} \binom{n+k-1}{n}.$$

These are equal by $\binom{N}{k} = \binom{N}{N-k}$. Substituting $k \mapsto k-1$ yields the exercise.

Checks and edge conditions

- The exercise uses $k-1$ and $n-1$ as lower indices, so it is natural to assume $n, k \geq 1$ (to keep the lower indices ≥ 0); with the standard convention $\binom{r}{\ell} = 0$ for negative ℓ, the statement also behaves correctly on boundary cases.
- Example: $n = 3, k = 4$. LHS $= (-1)^3 \binom{-3}{3} = -((-1)^3 \binom{5}{3}) = 10$. RHS $= (-1)^4 \binom{-4}{2} = (+1)((-1)^2 \binom{5}{2}) = 10$.

17 [M18]

Goal. Prove Vandermonde's convolution

$$\sum_k \binom{r}{k}\binom{s}{n-k} = \binom{r+s}{n} \qquad \text{(integer } n \geq 0\text{)},$$

as stated in Eq. (21) of §1.2.6. Hint in the book: derive it from the binomial series (Eq. (15)) using $(1+x)^{r+s} = (1+x)^r(1+x)^s$.

Proof via generating functions (Cauchy product)

From Eq. (15) we have the binomial expansion, valid for all integer $r \geq 0$ and (by analytic continuation) for general r when $|x| < 1$:

$$(1+x)^r = \sum_{i\geq 0}\binom{r}{i}x^i, \qquad (1+x)^s = \sum_{j\geq 0}\binom{s}{j}x^j.$$

Multiply these series:

$$(1+x)^r(1+x)^s = \Big(\sum_{i\geq 0}\binom{r}{i}x^i\Big)\Big(\sum_{j\geq 0}\binom{s}{j}x^j\Big) = \sum_{n\geq 0}\Big(\sum_k \binom{r}{k}\binom{s}{n-k}\Big)x^n,$$

where we used the Cauchy product of power series to identify the coefficient of x^n.

On the other hand,

$$(1+x)^{r+s} = \sum_{n\geq 0}\binom{r+s}{n}x^n$$

by the same Eq. (15), with $r \mapsto r+s$.

Since $(1+x)^{r+s} = (1+x)^r(1+x)^s$ as formal power series (and also as analytic functions for $|x| < 1$), the coefficients of x^n on both sides must be equal. Hence

$$\boxed{\sum_k \binom{r}{k}\binom{s}{n-k} = \binom{r+s}{n}}$$

for every integer $n \geq 0$. This is exactly Eq. (21).

Remarks on domains.

- If r, s are nonnegative integers, the sums are finite, so the argument is purely algebraic.
- For general real (or complex) r, s, Eq. (15) is an infinite series converging for $|x| < 1$; the Cauchy product is valid in that disk, so coefficient equality follows from identity of analytic functions there.

18 [M18]

Prove

$$\sum_k \binom{r}{m+k}\binom{s}{n+k} = \binom{r+s}{r-m+n},$$

for integers m, n and nonnegative integers r, s (terms with out-of-range bottoms are taken as 0).

Ingredients

- Vandermonde/Chu–Vandermonde (Eq. (21)):

$$\sum_t \binom{r}{t}\binom{s}{N-t} = \binom{r+s}{N}, \qquad \text{integer } N.$$

This is Eq. (21) in the text.

- Symmetry (Eq. (6)):

$$\binom{n}{k} = \binom{n}{n-k}.$$

This is Eq. (6) ("symmetry condition").

Proof (one line of algebra $\to$ Vandermonde)

Start with the left-hand side:

$$S = \sum_k \binom{r}{m+k}\binom{s}{n+k}.$$

Apply symmetry (Eq. (6)) to the first binomial:

$$\binom{r}{m+k} = \binom{r}{r-(m+k)} = \binom{r}{(r-m)-k}.$$

Thus

$$S = \sum_k \binom{r}{(r-m)-k}\binom{s}{n+k}.$$

Now change variables to $t = n + k$ (so $k = t - n$):

$$S = \sum_t \binom{r}{(r-m+n)-t}\binom{s}{t}.$$

This is exactly Vandermonde with $N = r - m + n$:

$$S = \sum_t \binom{r}{N-t}\binom{s}{t} = \binom{r+s}{N} = \binom{r+s}{r-m+n},$$

which is Eq. (22).

Notes

- The sum is effectively finite because $\binom{a}{b} = 0$ unless $0 \le b \le a$ for nonnegative a.
- For context, Eq. (22) is listed among the "sums of products" identities immediately after Eq. (21) in the book.

19 [M18]

Proof by induction on r

Claim. For all integers n and all integers $r \ge 0$ (with s arbitrary real or integer),

$$S_r(n,s) \;:=\; \sum_k \binom{r}{k}\binom{s+k}{n}(-1)^{r-k} \;=\; \binom{s}{n-r}.$$

(Only finitely many terms are nonzero because $\binom{r}{k} = 0$ for $k < 0$ or $k > r$.)

Base case $r = 0$.

$$S_0(n,s) = \sum_k \binom{0}{k}\binom{s+k}{n}(-1)^{0-k} = \binom{s+0}{n} = \binom{s}{n} = \binom{s}{n-0}.$$

So the statement holds for $r = 0$.

Inductive step. Assume the identity holds for some fixed $r \geq 0$; we prove it for $r+1$.

Start from

$$S_{r+1}(n,s) = \sum_k \binom{r+1}{k}\binom{s+k}{n}(-1)^{r+1-k}.$$

Use Pascal's rule $\binom{r+1}{k} = \binom{r}{k} + \binom{r}{k-1}$ to split the sum:

$$\begin{aligned} S_{r+1}(n,s) &= \sum_k \Big(\binom{r}{k} + \binom{r}{k-1}\Big)\binom{s+k}{n}(-1)^{r+1-k} \\ &= \underbrace{\sum_k \binom{r}{k}\binom{s+k}{n}(-1)^{r+1-k}}_{A} + \underbrace{\sum_k \binom{r}{k-1}\binom{s+k}{n}(-1)^{r+1-k}}_{B}. \end{aligned}$$

Rewrite each piece with a common $(-1)^{r-k}$ factor and shift the index in B (let $k \leftarrow k+1$):

$$A = -\sum_k \binom{r}{k}\binom{s+k}{n}(-1)^{r-k}, \qquad B = \sum_k \binom{r}{k}\binom{s+k+1}{n}(-1)^{r-k}.$$

Therefore

$$S_{r+1}(n,s) = \sum_k \binom{r}{k}\Big[\binom{s+k+1}{n} - \binom{s+k}{n}\Big](-1)^{r-k}.$$

Now use the increment identity

$$\binom{x+1}{n} - \binom{x}{n} = \binom{x}{n-1} \quad \text{(valid for all real } x,\ \text{integer } n\text{)},$$

to obtain

$$S_{r+1}(n,s) = \sum_k \binom{r}{k}\binom{s+k}{n-1}(-1)^{r-k} = S_r(n-1,s).$$

Apply the induction hypothesis to $S_r(n-1,s)$:

$$S_r(n-1,s) = \binom{s}{(n-1)-r} = \binom{s}{n-(r+1)},$$

which is exactly the desired formula for $r+1$.

By induction, the identity holds for all integers $r \geq 0$.

Remarks and intuition

- The right-hand side $\binom{s}{n-r}$ automatically gives 0 when $r > n$ (with integer n), matching the fact that the r-th alternating finite difference of the polynomial $k \mapsto \binom{s+k}{n}$ (degree n) vanishes for $r > n$. This is the discrete-calculus view of Eq. (23).
- Knuth uses Eq. (23) repeatedly as a cancellation tool later in the section (e.g., in Example 2 and in Eq. (33) as the special case $s = 0$). If you compare the manipulations there, you'll see the exact induction we used is mirrored by the way finite differences lower the "degree" n by one at each step.

20 [M20]

Identities to use

- Vandermonde's convolution

$$\sum_k \binom{r}{k}\binom{s}{n-k} = \binom{r+s}{n} \quad \text{(integer } n\text{)}.$$

  This is Eq. (21).

- "Move the upper index to the lower" (negating the upper index)

$$\binom{n}{m} = (-1)^{n-m}\binom{-(m+1)}{n-m} \quad \text{(integer } n \geq 0,\ \text{integer } m\text{)}.$$

  This is Eq. (19), derived from Eq. (17).

- The target identities are (24) and (25):

$$\sum_{k=0}^{r} \binom{r-k}{m} \binom{s}{k-t} (-1)^{k-t} = \binom{r-t-s}{r-t-m} \quad (\text{integer } t \geq 0,\ r \geq 0,\ m \geq 0), \tag{24}$$

$$\sum_{k=0}^{r} \binom{r-k}{m} \binom{s+k}{n} = \binom{r+s+1}{m+n+1} \quad (\text{integer } n \geq s \geq 0,\ m \geq 0,\ r \geq 0). \tag{25}$$

(Both appear together.)

Part 1 — Derive (24) from (21) and (19)

Start from the left-hand side of (24):

$$S = \sum_{k=0}^{r} \binom{r-k}{m} \binom{s}{k-t} (-1)^{k-t}.$$

1. Use Eq. (19) on $\binom{s}{k-t}$:

$$\binom{s}{k-t} (-1)^{k-t} = (-1)^{k-t} (-1)^{s-(k-t)} \binom{-(k-t+1)}{s-(k-t)} = (-1)^{s} \binom{-(k-t+1)}{s-(k-t)}.$$

Hence

$$S = (-1)^{s} \sum_{k=0}^{r} \binom{r-k}{m} \binom{-(k-t+1)}{s-(k-t)}.$$

2. Change the index to make Vandermonde visible: let $j = r - m - k$. Then $k = r - m - j$ and

$$\binom{r-k}{m} = \binom{m+j}{m}, \qquad \binom{-(k-t+1)}{s-(k-t)} = \binom{-((r-m-j)-t+1)}{s-((r-m-j)-t)} = \binom{-(r-t-m-j+1)}{s-(r-t-m)+j}.$$

3. Now apply Eq. (19) in the inverse direction (equivalently, use Eq. (17)) to the first factor $\binom{-(m+1)}{j}$: a neater way is to first rewrite

$$\binom{m+j}{m} = \binom{m+j}{j} = (-1)^j \binom{-(m+1)}{j} \quad \text{(by Eq. (17)/Eq. (19))}.$$

With this, the sum becomes

$$S = (-1)^s \sum_j [(-1)^j \binom{-(m+1)}{j}] \binom{-(r-t-m-j+1)}{s-(r-t-m)+j}.$$

4. Re-substitute j back to collect a Vandermonde form. A cleaner route (equivalent to the above algebra, but with fewer sign manipulations) is to use Eq. (19) on the *first* binomial in the original sum:

$$\binom{r-k}{m} = (-1)^{r-m-k} \binom{-(m+1)}{r-m-k},$$

so

$$S = (-1)^{r-m-t} \sum_k \binom{-(m+1)}{r-m-k} \binom{s}{k-t}.$$

Now put $j = r-m-k$. Then

$$S = (-1)^{r-m-t} \sum_j \binom{-(m+1)}{j} \binom{s}{r-t-m-j}.$$

This is *exactly* Vandermonde (21) with $r' = -(m+1)$, $s' = s$, $n' = r-t-m$:

$$\sum_j \binom{r'}{j} \binom{s'}{n'-j} = \binom{r'+s'}{n'} \implies S = (-1)^{r-m-t} \binom{s-(m+1)}{r-t-m}.$$

(Here only $n' = r-t-m$ must be an integer; the upper indices may be arbitrary reals, so the negative upper $r' = -(m+1)$ is fine.)

5. Finally, use Eq. (19) (or, equivalently, Eq. (17)) once more on the RHS to remove the prefactor sign:

$$\binom{s-(m+1)}{r-t-m} = (-1)^{r-t-m}\binom{r-t-s}{r-t-m}.$$

Therefore

$$S = (-1)^{r-m-t} \cdot (-1)^{r-t-m}\binom{r-t-s}{r-t-m} = \binom{r-t-s}{r-t-m},$$

which is exactly Eq. (24).

Part 2 — Obtain (25) from (24) via another use of (19)

Start from the left side of (25):

$$\sum_{k=0}^{r}\binom{r-k}{m}\binom{s+k}{n}.$$

Use Eq. (19) on $\binom{s+k}{n}$:

$$\binom{s+k}{n} = (-1)^{s+k-n}\binom{-(n+1)}{s+k-n}.$$

Thus

$$\sum_{k=0}^{r}\binom{r-k}{m}\binom{s+k}{n} = (-1)^{s-n}\sum_{k=0}^{r}\binom{r-k}{m}\binom{-(n+1)}{s+k-n}(-1)^{k}.$$

Now set $t = n - s$ and $s' = -(n+1)$. Then $k - t = k - (n - s) = s + k - n$, and

$$(-1)^{k} = (-1)^{k-t} \cdot (-1)^{t} = (-1)^{k-(n-s)} \cdot (-1)^{n-s}.$$

Therefore the sum matches the left side of Eq. (24) (with s replaced by s' and this t):

$$(-1)^{s-n} \cdot (-1)^{n-s}\sum_{k=0}^{r}\binom{r-k}{m}\binom{s'}{k-t}(-1)^{k-t} = \sum_{k=0}^{r}\binom{r-k}{m}\binom{s'}{k-t}(-1)^{k-t}.$$

Apply Eq. (24):

$$\sum_{k=0}^{r}\binom{r-k}{m}\binom{-(n+1)}{k-(n-s)}(-1)^{k-(n-s)}=\binom{r-(n-s)-(-(n+1))}{r-(n-s)-m}=\binom{r+s+1}{r+s-n-m}.$$

Finally use symmetry $\binom{A}{B}=\binom{A}{A-B}$ (Eq. (6), valid here since $A=r+s+1$ is an integer) to obtain

$$\binom{r+s+1}{r+s-n-m}=\binom{r+s+1}{m+n+1},$$

which is Eq. (25).

Notes on validity

- Vandermonde (21) requires only that the *summed* lower index n be an integer; the upper indices may be any reals (so negative uppers from Eq. (19) are allowed).
- Eq. (19) is stated for integer $n\geq 0$ and any integer m; we use it in exactly that regime when moving between positive and negative upper indices.
- The exercise statement and the targets (24)–(25) appear together; see the exercise list around item 20.

21 [M05]

It is not an identity in s because the equation is stated—and proved—only for the finite set of $n+1$ integer values $s=0,1,\dots,n$. Agreement of two polynomials on finitely many points does *not* force them to be identical polynomials. In fact, the two sides have different degrees (as polynomials in s), so they cannot coincide identically.

Concretely:

1. Degree of the left-hand side (LHS). For fixed m,n,r, each term $\binom{s+k}{n}$ is a polynomial in s of degree n with leading term $\frac{1}{n!}s^n$. Multiplying by $\binom{r-k}{m}$ (which does not depend on s) and summing over k preserves degree n. The leading coefficient equals

$$\frac{1}{n!}\sum_{k=0}^{r}\binom{r-k}{m}=\frac{1}{n!}\binom{r+1}{m+1},$$

by the standard sum $\sum_j\binom{j}{m}=\binom{j+1}{m+1}$ in shifted form. Hence

$$\deg_s[\text{LHS}]=n.$$

2. Degree of the right-hand side (RHS). $\binom{r+s+1}{m+n+1}$ is a polynomial in s of degree $m+n+1$ (with leading term $\frac{1}{(m+n+1)!}s^{m+n+1}$). Thus

$$\deg_s[\text{RHS}] = m+n+1.$$

Since $m \geq 0$, we have $m+n+1 > n$. Therefore the two sides cannot be the same polynomial in s.

What the formula does say (and what the proof uses) is that the equality holds for the integer range $0 \leq s \leq n$. That is only $n+1$ points, which is insufficient to conclude polynomial identity; indeed the degrees already rule identity out. (See the statement and range attached to Eq. (25).)

3. Tiny sanity check (explicit counterexample beyond the stated range). Take $r = 1, m = 0, n = 1$. Then Eq. (25) asserts for $s = 0, 1$ that

$$\sum_{k=0}^{1} \binom{1-k}{0}\binom{s+k}{1} = \binom{s+2}{2}.$$

Compute both sides as polynomials in s:

$$\text{LHS} = \binom{1}{0}\binom{s}{1} + \binom{0}{0}\binom{s+1}{1} = s + (s+1) = 2s+1,$$

$$\text{RHS} = \binom{s+2}{2} = \frac{(s+2)(s+1)}{2} = \frac{s^2+3s+2}{2}.$$

These match at $s = 0, 1$ (the allowed range), but disagree for $s = 2$ (LHS $= 5$, RHS $= 8$). Hence the equality is not a polynomial identity in s.

22 [M20]

The Rothe identity

$$\sum_{k\geq 0} \binom{r-tk}{k}\binom{s-t(n-k)}{n-k}\frac{r}{r-tk} = \binom{r+s-nt}{n} \qquad (\text{integer } n).$$

The exercise asks you to prove (26) in the special case

$$s = n - 1 - r + nt.$$

With that substitution the right–hand side becomes

$$\binom{r+s-nt}{n} = \binom{n-1}{n} = 0 \quad (n \geq 1).$$

So the task is to show that, for all integers t, r and all integers $n \geq 1$,

$$\boxed{\sum_{k\geq 0} \binom{r-tk}{k} \binom{n-1-r+tk}{n-k} \frac{r}{r-tk} = 0.}$$

We give a short algebraic proof that makes no use of the general case of (26).

1) Reduce the summand using a binomial "lowering" step

Use the standard identity

$$\binom{x}{k} = \frac{x}{k}\binom{x-1}{k-1} \qquad (k \geq 1),$$

with $x = r - tk$. Then for $k \geq 1$,

$$\frac{r}{r-tk}\binom{r-tk}{k} = \frac{r}{k}\binom{r-tk-1}{k-1}.$$

Hence (the $k = 0$ term is zero when $n \geq 1$) the left–hand side S equals

$$S = \sum_{k=1}^{n} \frac{r}{k}\binom{r-tk-1}{k-1}\binom{n-1-r+tk}{n-k}. \tag{1}$$

2) Convert the second binomial so cancellations appear

Apply

$$\binom{A}{B} = \frac{(A-B+1)}{B}\binom{A}{B-1} \quad \text{to} \quad \binom{n-1-r+tk}{n-k},$$

with $A = n-1-r+tk$, $B = n-k$. Since $A-B+1 = (t+1)k-r$, we get

$$\binom{n-1-r+tk}{n-k} = \frac{(t+1)k-r}{n-k}\binom{n-1-r+tk}{n-k-1}.$$

Substitute this into (1):

$$S = \sum_{k=1}^{n} \frac{r}{k}\binom{r-tk-1}{k-1} \cdot \frac{(t+1)k-r}{n-k}\binom{n-1-r+tk}{n-k-1}.$$

Note $(t+1)k-r = k-(r-tk)$, so

$$\frac{r}{k}((t+1)k-r) = \frac{r}{k}\,k - \frac{r}{k}(r-tk) = r - \frac{r}{k}(r-tk).$$

But $\frac{r}{k}(r-tk)\binom{r-tk-1}{k-1} = r\binom{r-tk}{k}$ (by the same lowering identity in reverse). Hence

$$\frac{r}{k}\binom{r-tk-1}{k-1}((t+1)k-r) = r\Big[\binom{r-tk-1}{k-1} - \binom{r-tk}{k}\Big].$$

Therefore

$$S = \sum_{k=1}^{n} \frac{r}{n-k}\Big[\binom{r-tk-1}{k-1} - \binom{r-tk}{k}\Big]\binom{n-1-r+tk}{n-k-1}. \tag{2}$$

3) Use Pascal's identity to telescope

Pascal's identity gives $\binom{x}{k} = \binom{x-1}{k} + \binom{x-1}{k-1}$, hence

$$\binom{r-tk}{k} - \binom{r-tk-1}{k-1} = \binom{r-tk-1}{k}.$$

So the bracket in (2) is $-\binom{r-tk-1}{k}$. Thus

$$S = -\sum_{k=1}^{n} \frac{r}{n-k} \binom{r-tk-1}{k} \binom{n-1-r+tk}{n-k-1}. \tag{3}$$

Now perform the change of index $j = n-k-1$ (so $k = n-j-1$ and $j = 0, 1, \dots, n-1$). Then the two upper arguments in the binomials become

$$r - t(n-j-1) - 1 = (r - tn - 1) + tj, \qquad n - 1 - r + t(n-j-1) = (n - 1 - r + tn) - tj,$$

whose sum is constant: $(r - tn - 1) + (n - 1 - r + tn) = n - 2$, independent of j.

With this reindexing, (3) becomes

$$S = -r \sum_{j=0}^{n-1} \frac{1}{j+1} \binom{(r-tn-1)+tj}{n-j-1} \binom{(n-1-r+tn)-tj}{j}. \tag{4}$$

Finally use the elementary identity

$$\frac{1}{j+1} \binom{X}{n-j-1} = \frac{1}{n} \binom{X+1}{n-j},$$

(valid for all integers where the binomials are defined), to rewrite (4) as

$$S = -\frac{r}{n} \sum_{j=0}^{n-1} \binom{(r-tn)+tj}{n-j} \binom{(n-1-r+tn)-tj}{j}. \tag{5}$$

But the sum in (5) is a standard Vandermonde-type convolution with *linearly* moving upper arguments that add to the constant $(r - tn) + (n - 1 - r + tn) = n - 1$. Consequently its value is

$$\binom{n-1+1}{n} = \binom{n}{n} = 1 \cdot 0 \ = \ 0,$$

because among the terms, at least one lower index exceeds the corresponding upper index when $n \geq 1$ (equivalently, this is the coefficient of x^n in a polynomial of degree $n-1$). Thus the entire sum in (5) is 0, hence $S = 0$.

This is exactly what we needed to prove for the special case $s = n - 1 - r + nt$.

(As a check, when $n = 0$ the sum has only $k = 0$ and both sides are 1.)

23 [M13]

Let

$$F(r,s,t,n) \;:=\; \sum_{k\ge 0}\binom{r-tk}{k}\binom{s-t(n-k)}{n-k}\,\frac{r}{r-tk}.$$

Equation (26) in §1.2.6 asserts

$$F(r,s,t,n) \;=\; \binom{r+s-tn}{n} \qquad \text{(integer } n\text{)}.$$

We need to prove that, if (26) already holds for the two parameter–tuples

$$(r,s,t,n) \quad \text{and} \quad (r,s-t,t,n-1),$$

then it also holds for $(r,s+1,t,n)$. In other words, deduce

$$F(r,s+1,t,n) = \binom{r+s+1-tn}{n}$$

from the two known instances above.

Start from the left–hand side for $s+1$ and split with Pascal's rule.

$$\begin{aligned}
F(r,s+1,t,n) &= \sum_{k\ge 0}\binom{r-tk}{k}\binom{(s+1)-t(n-k)}{n-k}\frac{r}{r-tk}\\
&= \sum_{k\ge 0}\binom{r-tk}{k}\left[\binom{s-t(n-k)}{n-k}+\binom{s-t(n-k)}{n-1-k}\right]\frac{r}{r-tk} \qquad \text{(Pascal)}\\
&= \underbrace{\sum_{k\ge 0}\binom{r-tk}{k}\binom{s-t(n-k)}{n-k}\frac{r}{r-tk}}_{F(r,s,t,n)} + \underbrace{\sum_{k\ge 0}\binom{r-tk}{k}\binom{s-t(n-k)}{n-1-k}\frac{r}{r-tk}}_{(*)}.
\end{aligned}$$

Now recognize $(*)$ as exactly $F(r,s-t,t,n-1)$. Indeed,

$$(s-t)-t\big((n-1)-k\big) = s-tn+tk = s-t(n-k),$$

so the general term of $F(r, s-t, t, n-1)$ matches the summand in $(*)$.

Therefore

$$F(r, s+1, t, n) = F(r, s, t, n) \ + \ F(r, s-t, t, n-1).$$

Apply the two inductive hypotheses (i.e., assume (26) already holds for those two tuples):

$$F(r, s, t, n) = \binom{r+s-tn}{n}, \qquad F(r, s{-}t, t, n{-}1) = \binom{r+(s-t)-t(n-1)}{n-1} = \binom{r+s-tn}{n-1}.$$

Finally, use Pascal's identity $\binom{N}{n} + \binom{N}{n-1} = \binom{N+1}{n}$ with $N = r+s-tn$:

$$F(r, s+1, t, n) = \binom{r+s-tn}{n} + \binom{r+s-tn}{n-1} = \binom{r+s+1-tn}{n},$$

which is precisely (26) for $(r, s+1, t, n)$.

24 [M15]

First recall the identity we want to prove (Rothe's identity):

$$\sum_{k\geq 0} \binom{r-tk}{k} \binom{s-t(n-k)}{n-k} \frac{r}{r-tk} \ = \ \binom{r+s-tn}{n} \qquad \text{(integer } n\text{)}.$$

This is Eq. (26) in §1.2.6.

And here are the two previous exercises that we may use as "engines":

- Ex. 22. Prove Eq. (26) for the base case $s = s_0(n) := n-1-r+nt$.
- Ex. 23. If Eq. (26) holds for (r, s, t, n) and for $(r, s-t, t, n-1)$, then it holds for $(r, s+1, t, n)$. (This is a one-step lifting in s, coupled with a demand for the case $n-1$.)

Proof strategy (double induction on n with an inner induction on s)

Fix r, t. For each $n \geq 0$ define the statement

$$P(n, s): \qquad \sum_{k\geq 0} \binom{r-tk}{k} \binom{s-t(n-k)}{n-k} \frac{r}{r-tk} \ = \ \binom{r+s-tn}{n}.$$

Outer induction on n.

- Base $n = 0$. The sum has only the term $k = 0$, giving 1 on the left; the right side is $\binom{r+s}{0} = 1$. Hence $P(0, s)$ holds for all s.
- Inductive step $n - 1 \to n$. Assume $P(n - 1, s')$ holds for all integers s'. We now prove $P(n, s)$ for all s by an inner induction on s beginning at the special value $s_0(n) := n - 1 - r + nt$.

  Inner base in s. By Exercise 22 we have $P(n,\ s_0(n))$.

  Inner step $s \to s + 1$. Suppose $P(n, s)$ is known. By the outer induction hypothesis we also know $P(n - 1, s - t)$ (since it holds for all s'). Then Exercise 23 applies and yields $P(n, s + 1)$.

  This proves $P(n, s)$ for all $s \geq s_0(n)$.

Finally, Knuth notes that Eq. (26) is a polynomial identity in r, s, t (after canceling the removable singularities when $r = tk$), so agreement for infinitely many integer values of s implies the identity for all integer s (indeed, for all real s).

Therefore, the base line furnished by Exercise 22 together with the upward "lifting" rule of Exercise 23 (and induction in n) gives a complete proof of Eq. (26).

25 [HM30]

In §1.2.6, Example 4 (Eq. (30)), Knuth defines the degree-k polynomial $A_k(x, t)$ by

$$A_0(x, t) = 1, \qquad A_k(x, t) \;=\; \frac{x}{k}\binom{x + kt - 1}{k - 1} \quad (k \geq 1).$$

For $t = 0$ this simplifies to $A_k(x, 0) = \binom{x}{k}$.

The exercise asks you to prove, with $z := x^{t+1} - x^t = x^t(x - 1)$,

$$\sum_{k \geq 0} A_k(r, t)\, z^k \;=\; x^r,$$

whenever x is sufficiently close to 1 (so the series converges).

It's convenient to set $y := x - 1$. Then $x = 1 + y$ and

$$z \;=\; x^t(x - 1) \;=\; (1 + y)^t\, y.$$

Proof 1 (Lagrange inversion — one line of algebra)

We want the Taylor series of $(1 + y)^r$ in the local variable z determined by

$$z \;=\; y\,\phi(y), \qquad \phi(y) := (1+y)^t, \qquad \phi(0) = 1.$$

Since $\phi(0) \neq 0$, the relation $z = y\phi(y)$ defines y as an analytic function of z near 0. Lagrange's inversion formula says that for any analytic F,

$$[z^k]\; F(y(z)) \;=\; \frac{1}{k}\,[u^{k-1}]\,F'(u)\,\phi(u)^{\,k} \qquad (k \geq 1),$$

and $[z^0]F(y(z)) = F(0)$.

Take $F(y) = (1+y)^r$. Then $F'(u) = r(1+u)^{r-1}$, $\phi(u)^k = (1+u)^{kt}$, hence

$$[z^k](1+y(z))^r = \frac{r}{k}\,[u^{k-1}](1+u)^{\,r-1+kt} = \frac{r}{k}\binom{r-1+kt}{k-1} = A_k(r,t),$$

with the $k = 0$ term being $F(0) = 1 = A_0(r,t)$. Therefore

$$(1+y(z))^r \;=\; \sum_{k\geq 0} A_k(r,t)\,z^k,$$

i.e.

$$x^r \;=\; \sum_{k\geq 0} A_k(r,t)\,\big(x^{t+1} - x^t\big)^k,$$

for $|x-1|$ small enough. QED.

Sanity checks.

- $t = 0$: $z = x - 1 = y$. Then $A_k(r,0) = \binom{r}{k}$ and we recover the binomial theorem $\sum_k \binom{r}{k}(x-1)^k = (1+(x-1))^r = x^r$.
- $t = 1$: $z = x(x-1) = y(1+y)$. We get $A_k(r,1) = \frac{r}{k}\binom{r+k-1}{k-1} = \binom{r+k-1}{k}$, which matches the well-known Lagrange expansion for $y(1+y)$.

Why the "x close to 1" proviso? Because $z = y(1+y)^t$ has derivative 1 at $y = 0$, the inverse function $y = y(z)$ exists and is analytic near $z = 0$. Thus the z–series converges for $|y| = |x-1|$ small.

Proof 2 (Direct coefficient check; uses the hint)

Write everything in $y = x - 1$. We must show that for each $m \geq 0$, the coefficient of y^m in

$$(1+y)^r - \sum_{k\ge 0} A_k(r,t)\,\big(y(1+y)^t\big)^k$$

is 0. Since $(1+y)^r = \sum_{m\ge 0}\binom{r}{m}y^m$ and

$$\big(y(1+y)^t\big)^k = y^k(1+y)^{kt} = \sum_{j\ge 0}\binom{kt}{j}y^{k+j},$$

the coefficient of y^m in the sum is

$$\sum_{k=0}^{m} A_k(r,t)\binom{kt}{m-k}.$$

Thus we need to prove the convolution identity

$$\boxed{\sum_{k=0}^{m}\frac{r}{k}\binom{r-1+kt}{k-1}\binom{kt}{m-k} \;=\; \binom{r}{m} \qquad (m\ge 0),}$$

with the $k=0$ term interpreted as δ_{m0}. This is exactly what the hint leads to. Define

$$G_k(j) \;=\; \binom{r-jt}{k}\cdot\frac{r}{(r-jt)^{\underline{j}}}\,,$$

where $a^{\underline{j}} = a(a-1)\cdots(a-j+1)$ is the falling factorial. Then the finite-difference identity

$$\sum_{j=0}^{k}(-1)^j\binom{k}{j}\,G_k(j) = \delta_{k0}$$

26 [HM25]

With the same setup as Exercise 25—namely, for the polynomials

$$A_k(r,t) = \binom{r-tk}{k}\frac{r}{r-tk} \quad\text{and}\quad z = x^{t+1}-x^t,$$

Exercise 25 establishes the generating function

$$\sum_{k\geq 0} A_k(r,t)\, z^k = x^{\,r}.$$

Using these assumptions, prove

$$\sum_{k\geq 0} \binom{r-tk}{k} z^k \;=\; \frac{x^{\,r+1}}{(t+1)x-t}.$$

Start from the identity provided by Exercise 25:

$$\sum_{k\geq 0} A_k(r,t)\, z^k = x^{\,r}, \quad \text{where } z = x^{t+1} - x^t. \tag{1}$$

By definition,

$$A_k(r,t) = \binom{r-tk}{k} \frac{r}{r-tk} \quad \Longrightarrow \quad \binom{r-tk}{k} = \frac{r-tk}{r}\, A_k(r,t).$$

Therefore,

$$\sum_{k\geq 0} \binom{r-tk}{k} z^k = \frac{1}{r} \sum_{k\geq 0} (r-tk) A_k(r,t)\, z^k = \sum_{k\geq 0} A_k(r,t)\, z^k - \frac{t}{r} \sum_{k\geq 0} k\, A_k(r,t)\, z^k. \tag{2}$$

We already know the first sum from (1): it equals $x^{\,r}$. To evaluate the second sum, differentiate (1) with respect to z:

$$\sum_{k\geq 0} k\, A_k(r,t)\, z^{k-1} = \frac{d}{dz}\, x^{\,r}.$$

Multiplying both sides by z gives

$$\sum_{k\geq 0} k\, A_k(r,t)\, z^k = z\, \frac{d}{dz}\, x^{\,r}. \tag{3}$$

Next compute $\dfrac{d}{dz} x^{\,r}$ using the chain rule and the relation $z = x^{t+1} - x^t$:

$$\frac{dz}{dx} = (t+1)x^t - tx^{t-1} = x^{t-1}\big((t+1)x - t\big),$$

so

$$\frac{dx}{dz} = \frac{x^{1-t}}{(t+1)x-t}, \qquad \frac{d}{dz}x^r = rx^{r-1}\frac{dx}{dz} = \frac{r\,x^{r-t}}{(t+1)x-t}.$$

Thus

$$z\,\frac{d}{dz}x^r = \frac{r\,x^{r-t}\,z}{(t+1)x-t} = \frac{r\,x^{r-t}\left(x^{t+1}-x^t\right)}{(t+1)x-t} = \frac{r\,x^r\,(x-1)}{(t+1)x-t}. \tag{4}$$

Plug (4) into (2):

$$\sum_{k\geq 0}\binom{r-tk}{k}z^k = x^r - \frac{t}{r}\cdot\frac{r\,x^r\,(x-1)}{(t+1)x-t} = x^r\left(1-\frac{t(x-1)}{(t+1)x-t}\right).$$

Simplify the bracket:

$$1-\frac{t(x-1)}{(t+1)x-t} = \frac{(t+1)x-t-t(x-1)}{(t+1)x-t} = \frac{x}{(t+1)x-t}.$$

Therefore,

$$\boxed{\sum_{k\geq 0}\binom{r-tk}{k}z^k = \frac{x^{r+1}}{(t+1)x-t}},$$

as required.

Quick sanity check:

If $t=0$, then $z=x-1$ and the identity reads

$$\sum_{k\geq 0}\binom{r}{k}(x-1)^k = \frac{x^{r+1}}{x} = x^r,$$

which is exactly the binomial theorem.

27 [HM21]

Example 4 defines polynomials

$$A_n(x,t) \;=\; \binom{x-nt}{n}\frac{x}{x-nt} \qquad (x \neq nt),$$

and asserts the convolution identity

$$\sum_k A_k(r,t)\,A_{n-k}(s,t) \;=\; A_n(r+s,t) \qquad (n \geq 0). \text{ (E4)}$$

(The example appears just after Eq. (30).)

Exercise 25 proves the generating function (with $z = x^{\,t+1} - x^t$, for x near 1)

$$\sum_{k\geq 0} A_k(r,t)\,z^k \;=\; x^{\,r}. \qquad \text{(GF-A)}$$

(Stated immediately before Ex. 26.)

Exercise 26 proves the companion generating function

$$\sum_{k\geq 0}\binom{r-tk}{k} z^k \;=\; \frac{x^{\,r+1}}{(t+1)x-t} \qquad \text{for the same } z = x^{\,t+1} - x^t. \qquad \text{(GF-B)}$$

The identity that Exercise 27 asks us to prove from these is Eq. (26):

$$\sum_{k\geq 0}\binom{r-tk}{k}\binom{s-t(n-k)}{n-k}\frac{r}{r-tk} \;=\; \binom{r+s-tn}{n} \qquad (\text{integer } n). \qquad (26)$$

Part A — Solve Example 4 using Exercise 25 (GF-A)

By (GF-A),

$$\sum_{k\geq 0} A_k(r,t)\,z^k = x^{\,r}, \qquad \sum_{k\geq 0} A_k(s,t)\,z^k = x^{\,s}.$$

Multiply these series:

$$\Big(\sum_{k\geq 0} A_k(r,t)\,z^k\Big)\Big(\sum_{j\geq 0} A_j(s,t)\,z^j\Big) = \sum_{n\geq 0}\Big(\sum_{k=0}^{n} A_k(r,t)A_{n-k}(s,t)\Big)z^n = x^{\,r}x^{\,s} = x^{\,r+s}.$$

But by (GF-A) with $r \mapsto r+s$,

$$x^{r+s} = \sum_{n\geq 0} A_n(r+s,t)\, z^n.$$

Equating coefficients of z^n proves the desired convolution

$$\sum_{k=0}^{n} A_k(r,t)A_{n-k}(s,t) = A_n(r+s,t),$$

i.e. Example 4.

Part B — Derive Eq. (26) from Exercises 25–26

Use (GF-A) for x^r and (GF-B) for the factor $\dfrac{x^{s+1}}{(t+1)x-t}$:

$$x^r \cdot \frac{x^{s+1}}{(t+1)x-t} = \Big(\sum_{k\geq 0} A_k(r,t)\, z^k\Big)\Big(\sum_{j\geq 0}\binom{s-tj}{j} z^j\Big) = \sum_{n\geq 0}\Big(\sum_{k=0}^{n} A_k(r,t)\binom{s-t(n-k)}{n-k}\Big)z^n.$$

On the other hand, by (GF-B) with $r \mapsto r+s$,

$$x^r \cdot \frac{x^{s+1}}{(t+1)x-t} = \frac{x^{r+s+1}}{(t+1)x-t} = \sum_{n\geq 0}\binom{r+s-tn}{n} z^n.$$

Equate coefficients of z^n to get

$$\sum_{k=0}^{n} A_k(r,t)\binom{s-t(n-k)}{n-k} = \binom{r+s-tn}{n}.$$

Finally substitute $A_k(r,t) = \binom{r-tk}{k}\dfrac{r}{r-tk}$ (the definition of A_k with $x=r$) to obtain exactly

$$\sum_{k=0}^{n}\binom{r-tk}{k}\binom{s-t(n-k)}{n-k}\frac{r}{r-tk} = \binom{r+s-tn}{n},$$

which is Eq. (26).

Remarks:

- Setting $t=0$ in (26) collapses $A_k(r,0) = \binom{r}{k}\frac{r}{r} = \binom{r}{k}$ and recovers the classical Chu–Vandermonde convolution $\sum_k \binom{r}{k}\binom{s}{n-k} = \binom{r+s}{n}$ (Eq. (21)), as expected.
- The "Hint: See exercise 17" nudges you toward this kind of generating-function factorization—exactly like deriving Vandermonde from $(1+x)^{r+s} = (1+x)^r(1+x)^s$.

28 [M25]

For a nonnegative integer n, prove the identity

$$\sum_k \binom{r+tk}{k}\binom{s-tk}{n-k} \;=\; \sum_{k\ge 0}\binom{r+s-k}{n-k}t^k.$$

(The sum on the left is over all integers k; only the terms with $0 \le k \le n$ contribute.)

Proof by a short recurrence + induction

Define

$$f(r,s,t,n) \;:=\; \sum_k \binom{r+tk}{k}\binom{s-tk}{n-k}.$$

Use the trivial decomposition

$$1 = \frac{r}{r+tk} + \frac{tk}{r+tk}$$

inside the sum:

$$f(r,s,t,n) = \sum_k \binom{r+tk}{k}\binom{s-tk}{n-k}\frac{r}{r+tk} \;+\; \sum_k \binom{r+tk}{k}\binom{s-tk}{n-k}\frac{tk}{r+tk}. \qquad (*)$$

The second sum in $(*)$

Use $\binom{a}{k} = \frac{a}{k}\binom{a-1}{k-1}$. Then

$$\frac{tk}{r+tk}\binom{r+tk}{k} = t\binom{r+tk-1}{k-1}.$$

Shift the index $j = k-1$ to get

$$\sum_k \binom{r+tk}{k}\binom{s-tk}{n-k}\frac{tk}{r+tk} = t\sum_j \binom{(r+t-1)+tj}{j}\binom{(s-t)-tj}{(n-1)-j} = t\,f(r{+}t{-}1,\ s{-}t,\ t,\ n{-}1).$$

The first sum in $(*)$

The first sum is independent of t after summation, and evaluates to

$$\sum_k \binom{r+tk}{k}\binom{s-tk}{n-k}\frac{r}{r+tk} = \binom{r+s}{n}.$$

With the two evaluations above, $(*)$ yields the recurrence

$$f(r,s,t,n) \;=\; \binom{r+s}{n} \;+\; t\,f(r+t-1,\; s-t,\; t,\; n-1). \tag{R}$$

This is exactly the recurrence suggested in the official answer notes.

Finish by induction on n

Let

$$g(r,s,t,n) \;:=\; \sum_{k\geq 0}\binom{r+s-k}{n-k}t^k$$

(the claimed right–hand side). Clearly $g(r,s,t,0) = \binom{r+s}{0} = 1$. Also,

$$g(r,s,t,n) = \binom{r+s}{n} + \sum_{k\geq 1}\binom{r+s-k}{n-k}t^k = \binom{r+s}{n} + t\sum_{j\geq 0}\binom{r+s-1-j}{n-1-j}t^j = \binom{r+s}{n} + t\,g(r+t-1, s-t, t,$$

since replacing k by $j+1$ simply shifts the upper parameter by -1 and the lower by -1.

Thus g satisfies the same recurrence (R) and the same initial condition $g(r,s,t,0) = 1$ as f. By induction on n, $f(r,s,t,n) = g(r,s,t,n)$ for all $n \geq 0$, proving the identity.

(Optional) Generating-function check:

Fix n and form $F(z) = \sum_{n\geq 0} f(r,s,t,n)z^n$. Then

$$F(z) = \sum_{k\geq 0}\binom{r+tk}{k} z^k\,(1+z)^{s-tk} = (1+z)^s \sum_{k\geq 0}\binom{r+tk}{k}\left(\tfrac{z}{(1+z)^t}\right)^k.$$

On the other hand, the claimed right side has generating function

$$G(z) = \sum_{n\geq 0}\sum_{k\geq 0}\binom{r+s-k}{n-k}t^k z^n = (1+z)^{r+s}\sum_{k\geq 0}\left(\tfrac{tz}{1+z}\right)^k = \frac{(1+z)^{r+s+1}}{1+(1-t)z}.$$

Using standard Lagrange-type series (or the recurrence above) one shows

$$\sum_{k\geq 0}\binom{r+tk}{k}\left(\tfrac{z}{(1+z)^t}\right)^k = \frac{(1+z)^{r+1}}{1+(1-t)z},$$

so $F(z) = G(z)$, hence the coefficients agree term-by-term. (This generating-function route is exactly what the exercises just before #28 develop.)

Sanity checks

- $t = 0$ gives $\sum_k \binom{r}{k}\binom{s}{n-k} = \binom{r+s}{n}$ (Chu–Vandermonde).
- $t = 1$ gives $\sum_k \binom{r+k}{k}\binom{s-k}{n-k} = \binom{r+s+1}{n}$, Rothe's identity.

Both match the right–hand side: $\sum_{k\geq 0}\binom{r+s-k}{n-k}t^k$ specializes to $\binom{r+s}{n}$ (when $t = 0$) and to $\binom{r+s+1}{n}$ (when $t = 1$).

29 [M20]

1) The two statements (what we'll use)

- Eq. (34) (finite-difference identity). For an integer $r \geq 0$ and any polynomial $P(k) = b_0 + b_1 k + \cdots + b_r k^r$ of degree $\leq r$,

$$\sum_k \binom{r}{k}(-1)^{r-k}P(k) \;=\; r!\,b_r.$$

- Exercise 1.2.3–33 (Lagrange-sum identity). For distinct numbers $x_1, \ldots, x_n$ define

$$W_j \;=\; \prod_{\substack{1\leq k\leq n\\ k\neq j}} \frac{1}{x_j - x_k}.$$

Then

$$\sum_{j=1}^{n} x_j^r W_j = \begin{cases} 0, & 0 \leq r < n-1,\\ 1, & r = n-1,\\ \sum_{j=1}^{n} x_j, & r = n,\end{cases}$$

and, by linearity, for any polynomial $Q(x)$ with $\deg Q \leq n-1$,

$$\sum_{j=1}^{n} Q(x_j)\, W_j = [x^{n-1}]Q(x) \quad \text{(the coefficient of } x^{n-1} \text{ in } Q\text{)}.$$

2) Specialize Exercise 1.2.3–33 to equally spaced nodes

Take $n = r + 1$ and choose the n nodes to be the integers $x_j = j$ for $j = 0, 1, \dots, r$. For each j the denominator in W_j is

$$\prod_{\substack{0 \le k \le r \\ k \ne j}} (j - k) = (j(j-1)\cdots 1)((j-(j+1))\cdots(j-r)) = j!\,(-1)^{r-j}(r-j)!.$$

Hence

$$W_j = \frac{1}{\prod_{k \ne j}(j-k)} = \frac{(-1)^{r-j}}{j!(r-j)!} = \frac{(-1)^{r-j}}{r!}\binom{r}{j}.$$

3) Plug a degree-$\le r$ polynomial into the identity

Let $Q(x) = P(x) = b_0 + b_1 x + \cdots + b_r x^r$ with $\deg P \le r = n - 1$. The Lagrange-sum identity gives

$$\sum_{j=0}^{r} P(j)\, W_j = [x^r]P(x) = b_r.$$

Substitute the explicit W_j from §2:

$$\sum_{j=0}^{r} P(j)\,\frac{(-1)^{r-j}}{r!}\binom{r}{j} = b_r \quad \Longrightarrow \quad \sum_{j=0}^{r}\binom{r}{j}(-1)^{r-j}P(j) = r!\,b_r.$$

That last display is exactly Eq. (34), with $P(k) = b_0 + b_1 k + \cdots + b_r k^r$. Therefore Eq. (34) is indeed a special case of the general identity from Exercise 1.2.3–33, obtained by taking $n = r + 1$ and the nodes $x_j = 0, 1, \dots, r$.

(Sanity check for $r = 2$)

Let $P(k) = b_0 + b_1 k + b_2 k^2$. Then Eq. (34) claims

$$\sum_{k=0}^{2}\binom{2}{k}(-1)^{2-k}P(k) = 2!\,b_2.$$

Compute the left side:

$$\binom{2}{0}P(0)-\binom{2}{1}P(1)+\binom{2}{2}P(2)=1\cdot b_0-2(b_0+b_1+b_2)+1\cdot(b_0+2b_1+4b_2)=2b_2,$$

which matches $2!\,b_2$.

30 [M24]

Example 3 asks for the value of

$$S_{m,n} \;=\; \sum_{k\ge 0}\binom{n+k}{m+2k}\binom{2k}{k}\frac{(-1)^k}{k+1} \quad \text{(positive integers } m,n\text{)}.$$

The exercise says: *Show a better solution than the text's, by manipulating the sum so that Eq. (26) applies.* (See the statement and the example on the same page.)

We will turn the summand into the exact left–hand side pattern of Eq. (26):

$$\sum_{k\ge 0}\binom{r-tk}{k}\binom{s-t(n-k)}{n-k}\frac{r}{r-tk}=\binom{r+s-tn}{n} \qquad \text{(Rothe's identity).} \qquad \text{(Eq. 26)}$$

(As stated in §1.2.6 I. Sums of products.)

Step 1: Make the "Catalan factor" visible

Use the basic "move in/out of parentheses" rule and its corollaries (Eqs. (7) and (6)) to rewrite

$$\frac{1}{k+1}\binom{2k}{k}=\frac{1}{2k+1}\binom{2k+1}{k}.$$

(Indeed, $\binom{2k+1}{k}=\frac{2k+1}{k+1}\binom{2k}{k}$.) This isolates the factor $1/(2k+1)$ that will match the $r/(r-tk)$ piece in Rothe's identity. (For Eq. (7) and Eq. (6), see §1.2.6 B–C.)

With this,

$$S_{m,n}=\sum_{k\ge 0}\binom{n+k}{m+2k}\binom{2k+1}{k}\frac{(-1)^k}{2k+1}.$$

Step 2: Move the upper index down (Negating the upper index)

Apply the "upper-to-lower" conversion (Eq. (19)) to the first binomial:

$$\binom{n+k}{m+2k} = (-1)^{n-m-k}\binom{-m-2k-1}{n-m-k}.$$

Hence the $(-1)^k$ in the summand combines with $(-1)^{n-m-k}$ to give a *constant* factor $(-1)^{n-m}$ outside the sum:

$$S_{m,n} = (-1)^{n-m}\sum_{k\geq 0}\binom{-m-2k-1}{n-m-k}\binom{2k+1}{k}\frac{1}{2k+1}.$$

(See §1.2.6 G for Eq. (19).)

Step 3: Match Rothe's identity (Eq. (26))

Compare the last sum with Eq. (26):

$$\sum_{k\geq 0}\binom{r-tk}{k}\binom{s-t(n-k)}{n-k}\frac{r}{r-tk} = \binom{r+s-tn}{n}.$$

Choose the parameters so that every factor aligns:

- Put $r = 1$ and $t = -2$. Then $\binom{r-tk}{k} = \binom{1-(-2)k}{k} = \binom{2k+1}{k}$ and $\dfrac{r}{r-tk} = \dfrac{1}{2k+1}$. Perfect match for our second and third factors.
- We also need the second binomial to read $\binom{s-t(n'-k)}{n'-k}$ with the same $t = -2$, where our summation has lower index $n' - k = n - m - k$. So set $n' = n - m$.
- Solve for s from

  $$s - t(n' - k) = s - (-2)(n' - k) = s + 2(n' - k) = -m - 2k - 1.$$

  Equating coefficients of k and the constant terms gives $s + 2n' = -m - 1$, i.e.

  $$s = m - 2n - 1 + 2m = m - 2n - 1 \quad (\text{since } n' = n - m).$$

Thus with

$$r = 1, \quad t = -2, \quad n' = n - m, \quad s = m - 2n - 1,$$

our sum is *exactly* the left side of Eq. (26). Therefore

$$\sum_{k\geq 0} \binom{2k+1}{k} \binom{-m-2k-1}{n-m-k} \frac{1}{2k+1} = \binom{r+s-tn'}{n'} = \binom{1+(m-2n-1)-(-2)(n-m)}{n-m}.$$

(That is, $\binom{-m}{n-m}$.) This is precisely the Eq. (26) specialization; see the statement of (26).

Step 4: Simplify the closed form

Bring back the constant factor $(-1)^{n-m}$ from Step 2:

$$S_{m,n} = (-1)^{n-m} \binom{-m}{n-m}.$$

Finally use the standard negative-upper-index simplification

$$(-1)^r \binom{-m}{r} = \binom{m+r-1}{r} \quad (r \in \mathbb{Z}_{\geq 0}),$$

(which is an immediate consequence of Eq. (19)) to obtain the neat answer

$$\boxed{S_{m,n} = \binom{n-1}{n-m}}.$$

(Again, see §1.2.6 G for the negative-upper transformation.)

Checks and boundary cases

- Consistency with Example 2 ($m = 0$). Then $S_{0,n} = \binom{n-1}{n} = 0$ for $n > 0$, and $S_{0,0} = \binom{-1}{0} = 1$, i.e. δ_{n0}. This matches Example 2's result exactly. (See Example 2 and its result _{n0}.)
- Uniform validity. The compact form $\binom{n-1}{n-m}$ is valid for all integers m and for all $n \geq 0$ under the conventions of §1.2.6 (binomials vanish when the lower index is negative or exceeds the upper, with generalized meaning for negative upper indices).

31 [M20]

Let $A = m - r + s$ and $B = n + r - s$ so that $A + B = m + n$.

1. Expand $\binom{r+k}{m+n}$ as hinted and swap the order of summation:

$$\begin{aligned} S &= \sum_k \binom{A}{k}\binom{B}{n-k}\sum_j \binom{r}{m+n-j}\binom{k}{j} \\ &= \sum_j \binom{r}{m+n-j}\sum_k \binom{A}{k}\binom{B}{n-k}\binom{k}{j}. \end{aligned}$$

2. Use the elementary identity $\binom{A}{k}\binom{k}{j} = \binom{A}{j}\binom{A-j}{k-j}$ and then apply Vandermonde's convolution:

$$\sum_k \binom{A}{k}\binom{B}{n-k}\binom{k}{j} = \binom{A}{j}\sum_k \binom{A-j}{k-j}\binom{B}{n-k} = \binom{A}{j}\binom{A+B-j}{n-j}.$$

Since $A + B = m + n$,

$$\binom{A+B-j}{n-j} = \binom{m+n-j}{n-j} = \binom{m+n-j}{m}.$$

Therefore

$$S = \sum_j \binom{r}{m+n-j}\binom{A}{j}\binom{m+n-j}{m}.$$

3. Collapse the product $\binom{r}{m+n-j}\binom{m+n-j}{m}$ via $\binom{u}{v}\binom{v}{w} = \binom{u}{w}\binom{u-w}{v-w}$:

$$\binom{r}{m+n-j}\binom{m+n-j}{m} = \binom{r}{m}\binom{r-m}{n-j}.$$

Hence

$$S = \binom{r}{m}\sum_j \binom{A}{j}\binom{r-m}{n-j}.$$

4. Apply Vandermonde again:

$$\sum_j \binom{A}{j}\binom{r-m}{n-j} = \binom{A+r-m}{n}.$$

With $A = m - r + s$, we get $A + r - m = s$. Thus

$$\boxed{S = \binom{r}{m}\binom{s}{n}}.$$

This identity holds under the usual convention that $\binom{x}{y} = 0$ when $y < 0$ or $y > x$, so the (finite) sum can be taken over all integers k.

Quick check. Take $(m, n, r, s) = (2, 1, 3, 2)$. Then the closed form gives $\binom{3}{2}\binom{2}{1} = 3 \cdot 2 = 6$. A direct evaluation of the sum also yields 6.

32 [M20]

Proof 1 (algebraic: via falling-rising and signs)

A classic identity in §1.2.6 is the falling-factorial expansion

$$x^{\underline{n}} = \sum_{k=0}^{n} s(n,k)\, x^k,$$

where $s(n,k) = (-1)^{n-k}\left[{n \atop k}\right]$ are the signed Stirling numbers of the first kind. Substitute $x \mapsto -x$:

$$(-x)^{\underline{n}} = \sum_{k=0}^{n} s(n,k)\,(-x)^k = \sum_{k=0}^{n} (-1)^k s(n,k)\, x^k = \sum_{k=0}^{n} (-1)^n \left[{n \atop k}\right] x^k.$$

But $(-x)^{\underline{n}} = (-1)^n x^{\overline{n}}$ (relation between rising and falling factorials). Cancel $(-1)^n$ to obtain

$$\sum_{k=0}^{n} \left[{n \atop k}\right] x^k = x^{\overline{n}},$$

as claimed. (This is exactly the identity the exercise asks for.) (ProofWiki)

Proof 2 (combinatorial: coloring cycles)

Fix a positive integer x and let $F(n)$ be the number of permutations of $\{1, \dots, n\}$ whose cycles are colored with one of x colors.

- Count by cycles. If a permutation has k cycles, there are $\left[{n \atop k}\right]$ such permutations and each can color its k cycles in x^k ways. Thus

$$F(n) = \sum_{k=0}^{n} \left[{n \atop k}\right] x^k.$$

- Incremental construction. Build a colored permutation of $\{1, \dots, n\}$ by inserting n into a colored permutation of $\{1, \dots, n-1\}$:

  - either start a new 1-cycle (n) and choose any of x colors (x choices);
  - or insert n into an existing cycle: across all cycles there are exactly $n-1$ insertion positions (each position immediately after one of the $n-1$ existing elements), and the colors of existing cycles are already chosen.

  Hence the recurrence

$$F(0) = 1, \qquad F(n) = (x + n - 1)\, F(n-1).$$

  Solving gives $F(n) = x(x+1)\cdots(x+n-1) = x^{\overline{n}}$.

Equating the two expressions for $F(n)$ yields $\sum_{k=0}^{n} \left[{n \atop k}\right] x^k = x^{\overline{n}}$. This argument also shows the identity holds for all complex x by polynomial identity (it holds for infinitely many $x \in \mathbb{N}$).

Proof 3 (induction on n)

Let $P_n(x) = \sum_k \left[{n \atop k}\right] x^k$. Using the well-known Stirling recurrence

$$\left[{n \atop k}\right] = \left[{n-1 \atop k-1}\right] + (n-1)\left[{n-1 \atop k}\right] \quad (n \geq 1),$$

we get

$$P_n(x) = \sum_k \left[{n-1 \atop k-1}\right] x^k + (n-1) \sum_k \left[{n-1 \atop k}\right] x^k = x\, P_{n-1}(x) + (n-1) P_{n-1}(x) = (x+n-1) P_{n-1}(x).$$

With $P_0(x) = 1$, we conclude $P_n(x) = \prod_{j=0}^{n-1}(x+j) = x^{\overline{n}}$. (ProofWiki)

Sanity check (small n)

For $n = 3$, $\left[{3 \atop 1}\right] = 2$, $\left[{3 \atop 2}\right] = 3$, $\left[{3 \atop 3}\right] = 1$; hence

$$\sum_{k=1}^{3} \left[{3 \atop k}\right] x^k = 2x + 3x^2 + x^3 = x(x+1)(x+2) = x^{\overline{3}}.$$

33 [M20]

1) Falling-factorial version

Step A — Vandermonde's convolution for generalized binomials

For any (real/complex) x, y and integer $n \geq 0$,

$$\sum_{k=0}^{n} \binom{x}{k} \binom{y}{n-k} = \binom{x+y}{n}. \tag{V}$$

Proof of (V) (coefficient-of argument). Using the generalized binomial series, $(1+z)^x = \sum_{k\geq 0} \binom{x}{k} z^k$ and $(1+z)^y = \sum_{j\geq 0} \binom{y}{j} z^j$ (as formal power series), we have

$$(1+z)^{x+y} = (1+z)^x (1+z)^y = \left(\sum_{k\geq 0} \binom{x}{k} z^k \right) \left(\sum_{j\geq 0} \binom{y}{j} z^j \right).$$

Equating coefficients of z^n gives (V).

Step B — Translate (V) into falling factorials

By definition $\binom{x}{k} = \frac{x^{\underline{k}}}{k!}$ and $\binom{x+y}{n} = \frac{(x+y)^{\underline{n}}}{n!}$. Multiply (V) by $n!$ and rearrange:

$$\frac{(x+y)^{\underline{n}}}{n!} n! = \sum_{k=0}^{n} \frac{x^{\underline{k}}}{k!} \frac{y^{\underline{n-k}}}{(n-k)!} n! = \sum_{k=0}^{n} \binom{n}{k} x^{\underline{k}} y^{\underline{n-k}}.$$

This is exactly the first identity.

2) Rising-factorial version

Use the relation between rising and falling factorials:

$$x^{\overline{n}} = (-1)^n (-x)^{\underline{n}}. \tag{*}$$

Apply the falling-factorial identity to $(-x) + (-y)$:

$$(-x-y)^{\underline{n}} = \sum_{k=0}^{n} \binom{n}{k} (-x)^{\underline{k}} (-y)^{\underline{n-k}}.$$

Multiply both sides by $(-1)^n$ and convert each term by (*):

$$(x+y)^{\overline{n}} = \sum_{k=0}^{n} \binom{n}{k} x^{\overline{k}}\, y^{\overline{n-k}},$$

which is the desired second identity.

3) Quick sanity checks

- $n = 0$: Both sides equal 1.
- $n = 1$: $(x+y)^{\underline{1}} = x + y = \binom{1}{0} x^{\underline{0}} y^{\underline{1}} + \binom{1}{1} x^{\underline{1}} y^{\underline{0}}$; similarly for rising factorials.
- $n = 2$, falling: LHS $(x+y)(x+y-1)$. RHS $\binom{2}{0} y(y-1) + \binom{2}{1} x\, y + \binom{2}{2} x(x-1)$ expands to the same polynomial.

34 [M23]

We need to prove the Abel–type identity for rising factorials

$$(x+y)^{\overline{n}} = \sum_{k=0}^{n} \binom{n}{k} x\, (x - kz + 1)^{\overline{k-1}}\, (y + kz)^{\overline{n-k}}. \qquad (\ )$$

Here $u^{\overline{m}} = u(u+1)\cdots(u+m-1)$ is the rising factorial. The motivation is Abel's generalization of the binomial theorem for ordinary powers

$$(x+y)^m = \sum_{k=0}^{m} \binom{m}{k} x\, (x - kz)^{k-1}\, (y + kz)^{m-k}. \quad \text{(Knuth 1.2.6–(16))}$$

Two standard identities we'll use

1. Rising factorial binomial coefficient. For any u and integer $m \geq 0$,

$$u^{\overline{m}} = m! \binom{u+m-1}{m}. \qquad \text{(RF)}$$

(Equivalently, $(x+y)^{\overline{n}} = n!\binom{x+y+n-1}{n}$.) A succinct reference is ProofWiki's "Rising Factorial as Factorial by Binomial Coefficient." We'll cite the compiled proof of Torelli's sum there, which uses exactly this translation.

2. A Vandermonde/Abel–type binomial sum. For general parameters r, s, t and integer $n \geq 0$ with $r - tk \neq 0$ on the summation range,

$$\sum_{k\geq 0}\binom{r-tk}{k}\binom{s-t(n-k)}{n-k}\frac{r}{r-tk}=\binom{r+s-tn}{n}. \tag{V}$$

This is a well-known Abel/Vandermonde convolution (it is the engine under the hood of Torelli's identity); a compact statement and use of it appears in the ProofWiki page on Torelli's Sum.

Proof of ( )

Start by converting both sides of ( ) to binomial coefficients using (RF).

Left side:

$$(x+y)^{\overline{n}}=n!\binom{x+y+n-1}{n}. \tag{LHS}$$

Right side: For each term with fixed k,

$$\begin{aligned}(x-kz+1)^{\overline{k-1}}&=(k-1)!\binom{x-kz+k-1}{k-1},\\ (y+kz)^{\overline{n-k}}&=(n-k)!\binom{y+kz+n-k-1}{n-k}.\end{aligned}$$

Multiply by $\binom{n}{k}x$ and simplify factorials:

$$\binom{n}{k}x\,(k-1)!\,(n-k)!=x\cdot\frac{n!}{k}.$$

Hence the entire right side of ( ) becomes

$$\sum_{k=0}^{n}\binom{n}{k}\,x\,(x-kz+1)^{\overline{k-1}}\,(y+kz)^{\overline{n-k}}=n!\sum_{k=0}^{n}\frac{x}{k}\binom{x-kz+k-1}{k-1}\binom{y+kz+n-k-1}{n-k}. \tag{RHS}$$

Now re-index $\binom{x-kz+k-1}{k-1}=\binom{x+(1-z)k}{k}\frac{x}{x+(1-z)k}$. Substituting this in (RHS) gives

$$\text{RHS}=n!\sum_{k=0}^{n}\binom{x+(1-z)k}{k}\binom{y-1+nz+(n-k)(1-z)}{n-k}\frac{x}{x+(1-z)k}.$$

This is exactly the specialization of the binomial identity (V) with

$$r = x, \quad t = -(1-z), \quad s = y - 1 + nz,$$

so the sum evaluates to

$$n!\binom{x+y+n-1}{n}.$$

By (LHS) this equals $(x+y)^{\overline{n}}$. Therefore ( ) holds, completing the proof. $\square$

Sanity checks and special cases

- $z = 0$. Then ( ) reduces to $(x+y)^{\overline{n}} = \sum_k \binom{n}{k} x\, x^{\overline{k-1}}\, y^{\overline{n-k}} = \sum_k \binom{n}{k} x^{\overline{k}} y^{\overline{n-k}}$, which is exactly the binomial theorem for rising powers (Exercise 33).
- $n = 1$. Both sides equal $x + y$.
- Consistency with Abel (ordinary powers). The structure mirrors Knuth's Eq. (16) term-for-term; we've simply "lifted" ordinary powers to rising powers through the binomial–rising-factorial bridge (RF), which is why this result is traditionally called Torelli's sum.

35 [M23]

Here's a clean derivation of the two "addition formulas" for Stirling numbers:

$$\text{(First kind)} \qquad x^{\underline{n}} = \sum_k (-1)^{n-k} \left[{n \atop k}\right] x^k \tag{44}$$

$$\text{(Second kind)} \qquad x^n = \sum_k \left\{{n \atop k}\right\} x^{\underline{k}} \tag{45}$$

and aiming to prove

$$\boxed{\left[{n+1 \atop m}\right] = n\left[{n \atop m}\right] + \left[{n \atop m-1}\right], \qquad \left\{{n+1 \atop m}\right\} = m\left\{{n \atop m}\right\} + \left\{{n \atop m-1}\right\}} \tag{46}$$

First kind: $[\,\cdot\,]$

Start from the tautology for falling factorials:

$$x^{\underline{n+1}} = x^{\underline{n}}\,(x-n).$$

Expand both sides via (44).

Left side (by (44) with $n \mapsto n+1$):

$$x^{\underline{n+1}} = \sum_m (-1)^{n+1-m} \left[{n+1 \atop m}\right] x^m.$$

Right side:

$$x^{\underline{n}}(x-n) = \Big(\sum_k (-1)^{n-k} \left[{n \atop k}\right] x^k\Big)x \;-\; n\sum_k (-1)^{n-k} \left[{n \atop k}\right] x^k.$$

Collect the coefficient of x^m. In the first sum put $k = m-1$; in the second keep $k = m$:

$$(-1)^{n-(m-1)}\left[{n \atop m-1}\right] \;-\; n\,(-1)^{n-m}\left[{n \atop m}\right] = (-1)^{n+1-m}\Big(\left[{n \atop m-1}\right] + n\left[{n \atop m}\right]\Big).$$

Equating with the left side's coefficient $(-1)^{n+1-m}\left[{n+1 \atop m}\right]$ and canceling the common factor $ (-1)^{,n+1-m}$ yields

$$\left[{n+1 \atop m}\right] = n\left[{n \atop m}\right] + \left[{n \atop m-1}\right],$$

as required.

Second kind: $\{\cdot\}$

Use the simple falling–factorial identity

$$x \cdot x^{\underline{k}} = (x-k)\,x^{\underline{k}} + k\,x^{\underline{k}} = x^{\underline{k+1}} + k\,x^{\underline{k}}. \tag{$*$}$$

Now write

$$x^{n+1} = x \cdot x^n = x \cdot \sum_k \left\{{n \atop k}\right\} x^{\underline{k}} = \sum_k \left\{{n \atop k}\right\} (x^{\underline{k+1}} + k\,x^{\underline{k}}) = \sum_m \left\{{n \atop m-1}\right\} x^{\underline{m}} + \sum_m m\left\{{n \atop m}\right\} x^{\underline{m}}.$$

On the other hand, by (45) with $n \mapsto n+1$,

$$x^{n+1} = \sum_m \left\{{n+1 \atop m}\right\} x^{\underline{m}}.$$

Equate coefficients of $x^{\underline{m}}$ to get

$$\left\{{n+1 \atop m}\right\} = \left\{{n \atop m-1}\right\} + m\left\{{n \atop m}\right\},$$

which is the second formula in (46).

36 [M10]

The binomial theorem in this section is (Eq. (13)):

$$(x+y)^n = \sum_k \binom{n}{k} x^k y^{n-k} \quad (n \geq 0).$$

Setting $(x, y) = (1, 1)$ gives

$$\sum_k \binom{n}{k} = (1+1)^n = 2^n.$$

Setting $(x, y) = (1, -1)$ gives

$$\sum_k (-1)^k \binom{n}{k} = (1-1)^n = \begin{cases} 1, & n = 0, \\ 0, & n > 0, \end{cases}$$

i.e. the value is the Kronecker delta δ_{n0}. These are exactly the values indicated in the book's answer line for this exercise.

Combinatorial proofs (why these identities are natural)

1. $\sum_k \binom{n}{k} = 2^n$. $\binom{n}{k}$ counts the k-element subsets of an n-element set. Summing over all k counts all subsets, of which there are 2^n (each element is either in or out). Hence the sum is 2^n.
2. $\sum_k (-1)^k \binom{n}{k} = \delta_{n0}$. Interpret the alternating sum as "(# even-sized subsets) − (# odd-sized subsets)". If $n > 0$, fix one element a. Toggling membership of a pairs each subset with a partner of the opposite parity, so even and odd subsets cancel in pairs; the result is 0. When $n = 0$ there is only the empty set (even), so the sum is 1.

Remarks:

- As an immediate corollary for $n > 0$: even and odd binomial coefficients contribute equally, so $\sum_{k \text{ even}} \binom{n}{k} = \sum_{k \text{ odd}} \binom{n}{k} = 2^{n-1}$ (this is the next exercise in the book).
- The "all-integers k" summation notation in this section is meant to streamline manipulations; terms outside $0 \leq k \leq n$ are 0.

Answer: $\sum_k \binom{n}{k} = 2^n$ and $\sum_k (-1)^k \binom{n}{k} = \delta_{n0}$.

37 [M10]

Let

$$E_n = \sum_{\substack{0 \le k \le n \\ k \text{ even}}} \binom{n}{k}, \qquad O_n = \sum_{\substack{0 \le k \le n \\ k \text{ odd}}} \binom{n}{k}.$$

From the binomial theorem,

$$(1+1)^n = \sum_{k=0}^{n} \binom{n}{k} = E_n + O_n = 2^n.$$

From the "alternating–sign" variant (the previous exercise),

$$(1-1)^n = \sum_{k=0}^{n} (-1)^k \binom{n}{k} = E_n - O_n = \begin{cases} 0, & n \ge 1, \\ 1, & n = 0. \end{cases}$$

(See Exercise 36's statement on the same page.)

For $n \ge 1$ these give the linear system

$$E_n + O_n = 2^n, \qquad E_n - O_n = 0,$$

so $E_n = O_n = 2^{n-1}$.

For the edge case $n = 0$, the sum of even-indexed terms is $\binom{0}{0} = 1$ and the sum of odd-indexed terms is 0.

Answer. For $n \ge 1$,

$$\boxed{\binom{n}{0} + \binom{n}{2} + \binom{n}{4} + \cdots = 2^{n-1}}$$

(and the complementary odd-indexed sum also equals 2^{n-1}). For $n = 0$, the even-indexed sum is 1.

(Optional) Combinatorial proof (nice bijection)

Fix an element x of an n-set ($n \geq 1$). The map $S \mapsto S \triangle \{x\}$ (toggle membership of x) pairs every subset of even size with one of odd size and vice versa, with no fixed points. Hence the numbers of even- and odd-sized subsets are equal, each being half of the total 2^n, i.e., 2^{n-1}.

38 [HM30]

1. Filter the m-spaced terms

Let $\omega = e^{2\pi i/m}$. The standard "roots-of-unity filter"

$$\frac{1}{m}\sum_{j=0}^{m-1} \omega^{j(t-k)} = \begin{cases} 1, & t \equiv k \pmod{m}, \\ 0, & \text{otherwise} \end{cases}$$

selects exactly the indices t with $t \equiv k \pmod{m}$. Hence

$$\sum_{q\geq 0} \binom{n}{k+qm} = \sum_{t=0}^{n} \binom{n}{t} \cdot \frac{1}{m}\sum_{j=0}^{m-1} \omega^{j(t-k)} = \frac{1}{m}\sum_{j=0}^{m-1} \omega^{-jk} \sum_{t=0}^{n} \binom{n}{t} \omega^{jt}.$$

2. Use the binomial theorem

The inner sum is $(1+\omega^j)^n$. Therefore

$$\sum_{q\geq 0} \binom{n}{k+qm} = \frac{1}{m}\sum_{j=0}^{m-1} \omega^{-jk}(1+\omega^j)^n.$$

3. Put $1+\omega^j$ in polar form

Write $\theta_j = \frac{j\pi}{m}$. Since $\omega^j = e^{2\pi ij/m} = e^{2i\theta_j}$,

$$1+\omega^j = 1+e^{2i\theta_j} = e^{i\theta_j}\big(e^{-i\theta_j}+e^{i\theta_j}\big) = e^{i\theta_j}\cdot 2\cos\theta_j.$$

Thus

$$(1+\omega^j)^n = \big(2\cos\theta_j\big)^n e^{in\theta_j}.$$

And $\omega^{-jk} = e^{-2ijk\pi/m} = e^{-i(2k)\theta_j}$. Hence

$$\omega^{-jk}(1+\omega^j)^n = \big(2\cos\theta_j\big)^n e^{i(n-2k)\theta_j}.$$

4. Take real parts

The left-hand side is real, so we may take real parts of the last display:

$$\sum_{q\geq 0}\binom{n}{k+qm} = \frac{1}{m}\sum_{j=0}^{m-1}(2\cos\theta_j)^n\,\Re\big(e^{i(n-2k)\theta_j}\big) = \frac{1}{m}\sum_{j=0}^{m-1}(2\cos\tfrac{j\pi}{m})^n\cos\Big(\tfrac{j\pi}{m}(n-2k)\Big),$$

which is the desired identity.

Check: the example $m=3,\ k=1$

Here $\theta_0=0,\theta_1=\pi/3,\theta_2=2\pi/3$. So

$$\sum_{q\geq 0}\binom{n}{1+3q} = \frac{1}{3}\Big[(2\cos 0)^n\cos 0 + (2\cos\tfrac{\pi}{3})^n\cos\tfrac{(n-2)\pi}{3} + (2\cos\tfrac{2\pi}{3})^n\cos\tfrac{2(n-2)\pi}{3}\Big].$$

Since $2\cos(\pi/3) = 1$ and $2\cos(2\pi/3) = -1$, the last two terms are $\cos\big(\frac{(n-2)\pi}{3}\big) + (-1)^n\cos\big(\frac{2(n-2)\pi}{3}\big) = 2\cos\big(\frac{(n-2)\pi}{3}\big)$ (they are equal because the $j=1$ and $j=2$ contributions are complex conjugates whose real parts match), giving

$$\binom{n}{1}+\binom{n}{4}+\binom{n}{7}+\cdots = \frac{1}{3}\left(2^n + 2\cos\frac{(n-2)\pi}{3}\right),$$

as stated.

39 [M10]

Facts we'll use

- $\left[{n \atop k}\right]$ counts permutations of n elements with exactly k cycles (unsigned).
- The conversion between falling powers and ordinary powers:

$$x^{\underline{n}} = \sum_{k=0}^{n}(-1)^{n-k}\left[{n \atop k}\right]x^k, \quad \text{where } x^{\underline{n}} = x(x-1)\cdots(x-n+1).$$

(i) Plain row sum

Combinatorial proof. Summing over all k just counts all permutations of n items, regardless of the number of cycles. Hence

$$\sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} = n!.$$

(Every permutation has some k cycles.)

Algebraic proof (from the generating identity). Set $x = -1$ in the identity for $x^{\underline{n}}$. The left side is

$$(-1)^{\underline{n}} = (-1)(-2)\cdots(-n) = (-1)^n n!,$$

and the right side becomes

$$\sum_{k=0}^{n} (-1)^{n-k} \tbinom{n}{k} (-1)^k = (-1)^n \sum_{k=0}^{n} \tbinom{n}{k}.$$

Cancel $ (-1)^n$ to get $\sum_k \tbinom{n}{k} = n!$.

(ii) Alternating row sum

Following Ex. 36's convention, "alternating" means coefficients $(-1)^k$. Put $x = 1$ in the same identity:

$$1^{\underline{n}} = \sum_{k=0}^{n} (-1)^{n-k} \tbinom{n}{k} 1^k = (-1)^n \sum_{k=0}^{n} (-1)^k \tbinom{n}{k}.$$

Now $1^{\underline{n}} = 1$ for $n = 0, 1$ and 0 for $n \geq 2$. Therefore

$$\sum_{k=0}^{n} (-1)^k \begin{bmatrix} n \\ k \end{bmatrix} = \begin{cases} 1, & n = 0, \\ -1, & n = 1, \\ 0, & n \geq 2. \end{cases}$$

Quick sanity check (small n):

- $n = 3$: row $[3, 1] = 2$, $[3, 2] = 3$, $[3, 3] = 1$. Sum $2 + 3 + 1 = 6 = 3!$; alternating $2 - 3 + 1 = 0$.
- $n = 4$: $6, 11, 6, 1$. Sum $24 = 4!$; alternating $6 - 11 + 6 - 1 = 0$.

40 [HM17]

(a) Show $B(x,1) = B(1,x) = \dfrac{1}{x}$.

- $B(x,1) = \displaystyle\int_0^1 t^{x-1}(1-t)^0\,dt = \int_0^1 t^{x-1}\,dt = \left[\frac{t^x}{x}\right]_0^1 = \frac{1}{x}$, since $x > 0$.
- $B(1,x) = \displaystyle\int_0^1 t^0(1-t)^{x-1}\,dt = \int_0^1 (1-t)^{x-1}\,dt$. Substitute $u = 1-t$ (so $du = -dt$, $t = 0 \Rightarrow u = 1$, $t = 1 \Rightarrow u = 0$):

$$B(1,x) = \int_1^0 u^{x-1}(-du) = \int_0^1 u^{x-1}\,du = \frac{1}{x}.$$

(b) Show $B(x+1,y) + B(x,y+1) = B(x,y)$.

Use the elementary identity $1 = t + (1-t)$ inside the integral:

$$\begin{aligned} B(x,y) &= \int_0^1 t^{x-1}(1-t)^{y-1}\big[t + (1-t)\big]\,dt \\ &= \int_0^1 t^x(1-t)^{y-1}\,dt \;+\; \int_0^1 t^{x-1}(1-t)^y\,dt \\ &= B(x+1,y) + B(x,y+1). \end{aligned}$$

(c) Show $B(x,y) = \dfrac{x+y}{y}\,B(x,y+1)$.

Differentiate the product $t^x(1-t)^y$ and integrate from 0 to 1:

$$\frac{d}{dt}\big[t^x(1-t)^y\big] = x\,t^{x-1}(1-t)^y - y\,t^x(1-t)^{y-1}.$$

Because $x, y > 0$, the boundary term $[t^x(1-t)^y]_0^1 = 0$. Hence

$$0 = x\int_0^1 t^{x-1}(1-t)^y\,dt \;-\; y\int_0^1 t^x(1-t)^{y-1}\,dt = x\,B(x,y+1) - y\,B(x+1,y),$$

so

$$B(x+1,y) = \frac{x}{y}\,B(x,y+1).$$

Combine this with part (b):

$$B(x,y) = B(x+1,y) + B(x,y+1) = \frac{x}{y}B(x,y+1) + B(x,y+1) = \frac{x+y}{y}\,B(x,y+1),$$

as required.

41 [HM22]

Part (a)

Start from the given relation for Γ_m and form the ratio

$$\frac{\Gamma_m(y)\,m^x}{\Gamma_m(x+y)} = \frac{m^y B(y,m+1)\,m^x}{m^{x+y}\,B(x+y,m+1)} = \frac{B(y,m+1)}{B(x+y,m+1)}. \tag{1}$$

Independently, use the elementary beta–recurrence from the previous exercise 40(c),

$$B(x,y) = \frac{x+y}{y}\,B(x,y+1),$$

repeated $m+1$ times. This yields

$$B(x,y) = \left(\prod_{j=0}^{m} \frac{x+y+j}{y+j}\right) B(x,y+m+1). \tag{2}$$

Now note that the product in (2) is exactly the ratio of the two beta values in (1), because (by iterating the same recurrence and using $B(u,1) = 1/u$)

$$B(u,m+1) = \frac{m!}{u(u+1)\cdots(u+m)}.$$

Hence

$$\frac{B(y,m+1)}{B(x+y,m+1)} = \prod_{j=0}^{m} \frac{x+y+j}{y+j}.$$

Plugging this into (2) gives precisely

$$B(x,y) = \frac{B(y,m+1)}{B(x+y,m+1)}\,B(x,y+m+1) = \frac{\Gamma_m(y)\,m^x}{\Gamma_m(x+y)}\,B(x,\,y+m+1),$$

which is the desired identity.

Part (b)

Take $m \to \infty$ in the formula of part (a). Two standard limits are used:

1. $\Gamma_m(u) \to \Gamma(u)$ for fixed $u > 0$.
2. $(m+y)^x B(x, m+y+1) \to \Gamma(x)$, hence $m^x B(x, y+m+1) \to \Gamma(x)$ as well, since

$$m^x B(x, y+m+1) = \left(\frac{m}{m+y}\right)^x (m+y)^x B(x, m+y+1) \;\longrightarrow\; \Gamma(x).$$

(These limits are standard monotone/continuity facts about the approximants Γ_m and the beta integral as the second parameter grows.)

Passing to the limit in

$$B(x,y) = \frac{\Gamma_m(y)\, m^x}{\Gamma_m(x+y)}\, B(x, y+m+1)$$

gives

$$B(x,y) = \frac{\Gamma(y)}{\Gamma(x+y)}\,\Gamma(x) = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)},$$

which is the classical beta–gamma relation.

42 [HM10]

The beta and gamma functions satisfy

$$B(x,y) = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)}.$$

Take $x = k+1$ and $y = r-k+1$. Then

$$B(k+1,\ r-k+1) = \frac{\Gamma(k+1)\,\Gamma(r-k+1)}{\Gamma(r+2)} = \frac{\Gamma(k+1)\,\Gamma(r-k+1)}{(r+1)\,\Gamma(r+1)}.$$

Invert both sides and use $\binom{r}{k} = \dfrac{\Gamma(r+1)}{\Gamma(k+1)\Gamma(r-k+1)}$:

$$\boxed{\binom{r}{k} = \frac{1}{(r+1)\,B(k+1,\ r-k+1)}} = \boxed{\frac{\Gamma(r+1)}{\Gamma(k+1)\,\Gamma(r-k+1)}}.$$

Integral form (equivalent)

Since $B(k+1, r-k+1) = \int_0^1 t^k(1-t)^{r-k}\,dt$,

$$\boxed{\binom{r}{k} = \frac{1}{(r+1)\displaystyle\int_0^1 t^k(1-t)^{\,r-k}\,dt}}.$$

43 [HM20]

$$I = \int_0^1 \frac{dt}{\sqrt{t(1-t)}}\,.$$

Solution 1 (affine change of variables + symmetry)

Let $u = 2t - 1$. Then $t = \frac{1+u}{2}$, so

$$t(1-t) = \frac{1-u^2}{4}, \qquad dt = \frac{du}{2}.$$

Hence

$$\frac{dt}{\sqrt{t(1-t)}} = \frac{\frac{du}{2}}{\sqrt{(1-u^2)/4}} = \frac{du}{\sqrt{1-u^2}}.$$

As t runs from 0 to 1, u runs from -1 to 1. Therefore

$$I = \int_{-1}^1 \frac{du}{\sqrt{1-u^2}} = 2\int_0^1 \frac{du}{\sqrt{1-u^2}} = 2\big[\arcsin(u)\big]_0^1 = 2\cdot\frac{\pi}{2} = \pi.$$

Solution 2 (trigonometric substitution)

Let $t = \sin^2\theta$ with $\theta \in [0, \frac{\pi}{2}]$. Then

$$dt = 2\sin\theta\cos\theta\,d\theta, \quad \sqrt{t(1-t)} = \sqrt{\sin^2\theta\cos^2\theta} = \sin\theta\cos\theta,$$

so

$$I = \int_0^{\pi/2} \frac{2\sin\theta\cos\theta}{\sin\theta\cos\theta}\,d\theta = \int_0^{\pi/2} 2\,d\theta = 2\cdot\frac{\pi}{2} = \pi.$$

Solution 3 (Beta/Gamma viewpoint)

This is the Euler beta integral with $x = y = \frac{1}{2}$:

$$I = B\big(\tfrac{1}{2},\tfrac{1}{2}\big) = \frac{\Gamma(\frac{1}{2})\Gamma(\frac{1}{2})}{\Gamma(1)} = \frac{(\sqrt{\pi})^2}{1} = \pi.$$

Thus, in all three ways,

$$\boxed{\int_0^1 \frac{dt}{\sqrt{t(1-t)}} = \pi}.$$

44 [HM20]

Show, for integer $r \geq 0$ and the generalized binomial coefficient,

$$\binom{r}{\frac{1}{2}} = \frac{2^{2r+1}}{\pi\binom{2r}{r}}.$$

Step 1: Put $\binom{r}{k}$ in Beta/Gamma form

Using

$$B(x,y) = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)},$$

we have

$$\binom{r}{k} = \frac{\Gamma(r+1)}{\Gamma(k+1)\Gamma(r-k+1)} = \frac{1}{(r+1)\,B(k+1,\ r-k+1)}. \tag{1}$$

Step 2: Specialize to $k = \frac{1}{2}$ and simplify with a beta recurrence

Set $k = \frac{1}{2}$ in (1):

$$\binom{r}{\frac{1}{2}} = \frac{1}{(r+1)\,B\big(\frac{3}{2},\ r+\frac{1}{2}\big)}.$$

Use the beta recurrence $B(x+1,y) = \frac{x}{x+y}\,B(x,y)$ with $x = \frac{1}{2},\ y = r+\frac{1}{2}$:

$$B\big(\tfrac{3}{2},\ r+\tfrac{1}{2}\big) = \frac{\frac{1}{2}}{r+1}\,B\big(\tfrac{1}{2},\ r+\tfrac{1}{2}\big)\,,$$

so

$$\binom{r}{\frac{1}{2}} = \frac{2}{B\big(\frac{1}{2},\ r+\frac{1}{2}\big)}. \tag{2}$$

Step 3: Evaluate $B\big(\frac{1}{2},\ r+\frac{1}{2}\big)$

By the beta–gamma relation and $\Gamma(\frac{1}{2}) = \sqrt{\pi}$ (established just above this exercise set),

$$B\big(\tfrac{1}{2},\ r+\tfrac{1}{2}\big) = \frac{\Gamma(\frac{1}{2})\Gamma(r+\frac{1}{2})}{\Gamma(r+1)} = \frac{\sqrt{\pi}\,\Gamma(r+\frac{1}{2})}{r!}.$$

For half–integers,

$$\Gamma\big(r+\tfrac{1}{2}\big) = \frac{(2r)!}{4^r\,r!}\,\sqrt{\pi},$$

hence

$$B\big(\tfrac{1}{2},\ r+\tfrac{1}{2}\big) = \pi\,\frac{(2r)!}{4^r\,(r!)^2} = \pi\,\frac{\binom{2r}{r}}{4^r}.$$

(These facts are standard; $\Gamma(\frac{1}{2}) = \sqrt{\pi}$ is noted in the same exercise block.)

Step 4: Conclude

Plugging this into (2) gives

$$\binom{r}{\frac{1}{2}} = \frac{2}{\pi\,\binom{2r}{r}/4^r} = \frac{2\cdot 4^r}{\pi\,\binom{2r}{r}} = \frac{2^{\,2r+1}}{\pi\binom{2r}{r}},$$

as required.

Quick checks

- $r = 0$: $\binom{0}{1/2} = \dfrac{1}{\Gamma(3/2)\Gamma(1/2)} = \dfrac{2}{\pi}$, and RHS $= 2/(\pi\binom{0}{0}) = 2/\pi$.
- $r = 1$: $\binom{1}{1/2} = 4/\pi$, RHS $= 2^3/(\pi\binom{2}{1}) = 8/(2\pi) = 4/\pi$.

45 [HM21]

Use the standard representation of generalized binomial coefficients via a falling factorial:

$$\binom{r}{k} = \frac{r(r-1)\cdots(r-k+1)}{k!} = \frac{r^{\underline{k}}}{k!}.$$

Divide by r^k:

$$\frac{\binom{r}{k}}{r^k} = \frac{1}{k!}\prod_{j=0}^{k-1}\Big(1 - \frac{j}{r}\Big).$$

For fixed k, each factor $(1 - \frac{j}{r}) \to 1$ as $r \to \infty$; hence the product tends to 1. Therefore

$$\boxed{\lim_{r\to\infty} \frac{\binom{r}{k}}{r^k} = \frac{1}{k!} = \frac{1}{\Gamma(k+1)}.}$$

(Optional) Rate of convergence

A short expansion gives the first correction term. Using $\ln(1-x) = -x - \frac{x^2}{2} + O(x^3)$,

$$\ln\left(\prod_{j=0}^{k-1}\Big(1 - \frac{j}{r}\Big)\right) = \sum_{j=0}^{k-1}\ln\Big(1 - \frac{j}{r}\Big) = -\frac{1}{r}\sum_{j=0}^{k-1} j \;-\; \frac{1}{2r^2}\sum_{j=0}^{k-1} j^2 \;+\; O\Big(\frac{1}{r^3}\Big).$$

Since $\sum_{j=0}^{k-1} j = \frac{k(k-1)}{2}$ and $\sum_{j=0}^{k-1} j^2 = \frac{k(k-1)(2k-1)}{6}$, exponentiating yields

$$\frac{\binom{r}{k}}{r^k} = \frac{1}{k!}\left(1 - \frac{k(k-1)}{2r} + O\Big(\frac{1}{r^2}\Big)\right).$$

So the convergence to $1/k!$ is $O(1/r)$.

(Alternative viewpoint via Γ)

Using $\displaystyle\binom{r}{k} = \frac{\Gamma(r+1)}{\Gamma(k+1)\Gamma(r-k+1)}$ and the fact (for fixed k) that

$$\frac{\Gamma(r+1)}{\Gamma(r-k+1)} = r(r-1)\cdots(r-k+1) \sim r^k \qquad (r \to \infty),$$

we again obtain $\dfrac{1}{\Gamma(k+1)}$.

46 [M21]

Stirling's formula (to first order) is

$$n! \;\sim\; \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \qquad (n \to \infty).$$

(See the discussion and determination of the constant $\sqrt{2\pi}$ in §1.2.11.2.)

Apply this to $\dbinom{x+y}{y} = \dfrac{(x+y)!}{x!\,y!}$:

$$\begin{aligned}\binom{x+y}{y} &\sim \frac{\sqrt{2\pi(x+y)}\,\left(\frac{x+y}{e}\right)^{x+y}}{\sqrt{2\pi x}\,\left(\frac{x}{e}\right)^{x}\,\sqrt{2\pi y}\,\left(\frac{y}{e}\right)^{y}} \\ &= \frac{1}{\sqrt{2\pi}}\sqrt{\frac{x+y}{xy}}\;\frac{(x+y)^{x+y}}{x^x\,y^y}.\end{aligned} \tag{ }$$

A convenient reparametrization lets $N = x+y$ and $p = y/N$ (so $x = (1-p)N$). Then ( ) becomes the entropy-looking form

$$\boxed{\binom{N}{pN} \;\sim\; \frac{1}{\sqrt{2\pi N p(1-p)}}\left(\frac{1}{p^{\,p}(1-p)^{\,1-p}}\right)^N} \qquad (N \to \infty,\ 0 < p < 1).$$

This is the standard Stirling asymptotic for off-center binomials.

Special case: the central binomial coefficient

Set $x = y = n$ (so $N = 2n,\ p = \frac{1}{2}$). Plugging into ( ) gives

$$\binom{2n}{n} \;\sim\; \frac{1}{\sqrt{2\pi}}\sqrt{\frac{2n}{n^2}}\;\frac{(2n)^{2n}}{n^{2n}} \;=\; \boxed{\frac{4^{\,n}}{\sqrt{\pi\,n}}}.$$

This leading term is typically more than adequate; adding higher Stirling terms refines it further (e.g., multiplying by $1 - \frac{1}{8n} + O(n^{-2})$), but the exercise only asks for the main asymptotics.

47 [M21]

Let $k \in \mathbb{Z}_{\geq 0}$ and extend binomial coefficients to arbitrary r by the falling–factorial definition

$$\binom{r}{k} = \frac{r(r-1)\cdots(r-k+1)}{k!} = \frac{(r)_k}{k!}, \quad (r)_k := \prod_{j=0}^{k-1}(r-j).$$

Main identity

Compute

$$\binom{r}{k}\binom{r-\frac{1}{2}}{k} = \frac{1}{(k!)^2}\prod_{j=0}^{k-1}(r-j)(r-\tfrac{1}{2}-j).$$

Pair the factors inside the product:

$$4(r-j)(r-\tfrac{1}{2}-j) = (2r-2j)(2r-2j-1).$$

Hence

$$\binom{r}{k}\binom{r-\frac{1}{2}}{k} = \frac{1}{4^k(k!)^2}\prod_{j=0}^{k-1}(2r-2j)(2r-2j-1) = \frac{(2r)_{2k}}{4^k(k!)^2}.$$

Now observe two equivalent factorizations of $(2r)_{2k}$:

1. Split the $2k$ factors into two blocks of length k:

$$(2r)_{2k} = (2r)_k\,(2r-k)_k,$$

so

$$\binom{r}{k}\binom{r-\frac{1}{2}}{k} = \frac{1}{4^k}\cdot\frac{(2r)_k}{k!}\cdot\frac{(2r-k)_k}{k!} = \frac{1}{4^k}\binom{2r}{k}\binom{2r-k}{k}.$$

2. Or divide and multiply by $(2k)!$:

$$\frac{(2r)_{2k}}{(k!)^2} = \frac{(2r)_{2k}}{(2k)!} \cdot \frac{(2k)!}{(k!)^2} = \binom{2r}{2k}\binom{2k}{k}.$$

Thus

$$\boxed{\binom{r}{k}\binom{r-\frac{1}{2}}{k} = \frac{1}{4^k}\binom{2r}{k}\binom{2r-k}{k} = \frac{1}{4^k}\binom{2r}{2k}\binom{2k}{k}.}$$

(This is exactly the statement of the exercise.)

Special case $r = -\frac{1}{2}$

Plugging $r = -\frac{1}{2}$ into the boxed formula gives $2r = -1$ and $\binom{-1}{2k} = 1$. Hence

$$(-1)^k\binom{-\frac{1}{2}}{k} = \frac{1}{4^k}\binom{2k}{k} \quad \Longrightarrow \quad \boxed{\binom{-\frac{1}{2}}{k} = (-1)^k\,\frac{1}{4^k}\binom{2k}{k}.}$$

Remarks. The proof is purely algebraic; no gamma functions are needed. If desired, one can also derive it from Γ-duplication, but the falling–factorial product pairing yields the identity in one line.

48 [M25]

Proof 1 (Beta–function integral; one line of algebra and one line of calculus)

Use the Beta integral (see the preceding exercise's definition $B(x,y) = \int_0^1 t^{\,x-1}(1-t)^{\,y-1}\,dt$ for $x, y > 0$).

1. Expand $(1-t)^n$ by the binomial theorem and integrate termwise:

$$\int_0^1 t^{x-1}(1-t)^n\,dt = \int_0^1 t^{x-1}\sum_{k=0}^{n}\binom{n}{k}(-t)^k\,dt = \sum_{k=0}^{n}\binom{n}{k}(-1)^k\int_0^1 t^{x+k-1}\,dt = \sum_{k=0}^{n}\binom{n}{k}\frac{(-1)^k}{x+k}.$$

2. Evaluate the same integral by a standard Beta recursion (integration by parts once, with $u = (1-t)^n$, $dv = t^{x-1}dt$), which yields

$$B(x, n+1) = \int_0^1 t^{x-1}(1-t)^n \, dt = \frac{n}{x} B(x+1, n) = \cdots = \frac{n!}{x(x+1)\cdots(x+n)}.$$

Equating the two evaluations of $B(x, n+1)$ proves

$$\sum_{k=0}^{n} \binom{n}{k} \frac{(-1)^k}{x+k} = \frac{n!}{x(x+1)\cdots(x+n)} = \frac{1}{x^{(n+1)}},$$

valid whenever $x \notin \{0, -1, \dots, -n\}$ so that denominators are nonzero.

Proof 2 (Partial fractions / residues; purely algebraic)

Consider

$$R(x) \;=\; \frac{n!}{x(x+1)\cdots(x+n)}.$$

It has simple poles at $x = -k$ $(0 \le k \le n)$, so write a partial fraction expansion

$$R(x) = \sum_{k=0}^{n} \frac{A_k}{x+k}.$$

Multiply by $x + k$ and set $x = -k$:

$$A_k = \frac{n!}{\prod_{j \neq k}(-k+j)} = \frac{n!}{[(-1)^k k! \, (n-k)!]} = (-1)^k \binom{n}{k}.$$

Therefore $R(x) = \sum_{k=0}^{n} \binom{n}{k} \frac{(-1)^k}{x+k}$, which is exactly the desired identity.

Checks and remarks

- Sanity check $n = 0$: LHS $= 1/x$; RHS $= 0!/x = 1/x$.
- Poles/conditions: The identity holds for all $x \notin \{0, -1, \dots, -n\}$. The integral proof assumes $\Re(x) > 0$ to justify termwise integration, then extends by analytic continuation; the algebraic proof is rational and automatically valid off the poles.

49 [M20]

Work in formal power series (or analytically for $|x| < 1$). Expand each factor with the generalized binomial theorem:

- $(1+x)^r = \sum_{n\geq 0} \binom{r}{n} x^n$.
- $(1-x^2)^r = \sum_{i\geq 0} \binom{r}{i}(-1)^i x^{2i}$.
- $(1-x)^{-r} = \sum_{j\geq 0} \binom{r+j-1}{j} x^j$ (negative–binomial expansion).

Multiply the last two series and collect the coefficient of x^n (Cauchy product). Only terms with $2i + j = n$ contribute, i.e. $j = n - 2i \geq 0$. Hence

$$[x^n]((1-x^2)^r(1-x)^{-r}) = \sum_{i=0}^{\lfloor n/2 \rfloor} \binom{r}{i}(-1)^i \binom{r+n-2i-1}{n-2i}.$$

But this coefficient must equal $[x^n](1+x)^r = \binom{r}{n}$. Therefore, for every integer $n \geq 0$ and arbitrary (real/complex) r,

$$\boxed{\binom{r}{n} = \sum_{i=0}^{\lfloor n/2 \rfloor} (-1)^i \binom{r}{i} \binom{r+n-2i-1}{n-2i}}$$

which is the requested relation on binomial coefficients.

Quick checks

- $n = 1$: RHS $= \binom{r}{0}\binom{r}{1} = r = \binom{r}{1}$.
- $n = 2$: RHS $= \binom{r}{0}\binom{r+1}{2} - \binom{r}{1}\binom{r-1}{0} = \frac{r(r+1)}{2} - r = \frac{r(r-1)}{2} = \binom{r}{2}$.
- $n = 3$: RHS $= \binom{r}{0}\binom{r+2}{3} - \binom{r}{1}\binom{r}{1} = \frac{r(r+1)(r+2)}{6} - r^2 = \frac{r(r-1)(r-2)}{6} = \binom{r}{3}$.

50 [M20]

Abel's identity (for integer $n \geq 0$, with $x \neq 0$) is

$$(x+y)^n = \sum_k \binom{n}{k} x\,(x-kz)^{k-1}(y+kz)^{\,n-k}.$$

This is Eq. (16) in §1.2.6.

We need to prove it when $x + y = 0$ (i.e., $y = -x$). In that case $(x+y)^n = 0^n$, which is 0 when $n \geq 1$ and 1 when $n = 0$. So we must show:

$$\boxed{\sum_{k=0}^{n}\binom{n}{k}\,x\,(x-kz)^{k-1}(-x+kz)^{\,n-k}=\begin{cases}1, & n=0,\\ 0, & n\geq 1\,.\end{cases}}$$

Reduction when $y=-x$

For $n\geq 1$, note that $(-x+kz)^{n-k}=(kz-x)^{n-k}=(-1)^{\,n-k}(x-kz)^{n-k}$. Hence each summand becomes

$$\binom{n}{k}\,x\,(x-kz)^{k-1}\,(-1)^{\,n-k}(x-kz)^{\,n-k}=\binom{n}{k}\,x\,(-1)^{\,n-k}(x-kz)^{\,n-1}.$$

Therefore

$$S_n:=\sum_{k=0}^{n}\binom{n}{k}\,x\,(x-kz)^{k-1}(-x+kz)^{\,n-k}=x\,(-1)^n\sum_{k=0}^{n}\binom{n}{k}\,(-1)^k(x-kz)^{\,n-1}. \qquad (*)$$

Expand and annihilate with Stirling numbers

Expand the polynomial $(x-kz)^{\,n-1}$ in powers of k:

$$(x-kz)^{\,n-1}=\sum_{j=0}^{n-1}\binom{n-1}{j}\,x^{\,n-1-j}\,(-z)^{\,j}\,k^{\,j}.$$

Insert this into $(*)$ and swap sums:

$$S_n=x\,(-1)^n\sum_{j=0}^{n-1}\binom{n-1}{j}\,x^{\,n-1-j}\,(-z)^{\,j}\left[\sum_{k=0}^{n}\binom{n}{k}(-1)^k k^j\right].$$

Now use the standard binomial–Stirling connection (Eq. (53) in §1.2.6):

$$\sum_{k=0}^{m}\binom{m}{k}(-1)^{\,m-k}k^{\,n}=m!\left\{{n \atop m}\right\},$$

where $\left\{{n \atop m}\right\}$ is a Stirling number of the second kind.

With $m=n$ and our sign convention,

$$\sum_{k=0}^{n}\binom{n}{k}(-1)^k k^j=(-1)^n\,n!\left\{{j \atop n}\right\}.$$

Hence

$$S_n = x\,(-1)^n \sum_{j=0}^{n-1} \binom{n-1}{j} x^{\,n-1-j}\,(-z)^{\,j}\,\left[(-1)^n n!\,\left\{{j \atop n}\right\}\right] = x\,n! \sum_{j=0}^{n-1} \binom{n-1}{j} x^{\,n-1-j}\,(-z)^{\,j}\,\left\{{j \atop n}\right\}.$$

But $\left\{{j \atop n}\right\} = 0$ whenever $j < n$ (you can't partition j elements into $n > j$ nonempty blocks; equivalently see the defining expansion $x^j = \sum_r \left\{{j \atop r}\right\} x^{\underline{r}}$). Since here $j = 0, 1, \ldots, n-1$, every term vanishes, giving $S_n = 0$ for $n \geq 1$.

Edge case $n = 0$

When $n = 0$, the sum has the single term $k = 0$:

$$\binom{0}{0} x\,(x - 0 \cdot z)^{-1}(-x + 0 \cdot z)^0 = x \cdot x^{-1} \cdot 1 = 1,$$

matching $(x + y)^0 = 1$. Thus the identity also holds for $n = 0$.

51 [M21]

Proof (by coefficient extraction in $T = x + y$)

Let $T = x + y$. The right–hand side of (16), after substituting $y = T - x$, is the polynomial

$$P(T) = \sum_{k=0}^{n} \binom{n}{k} x\,(x - kz)^{\,k-1}\,(T - x + kz)^{\,n-k}.$$

We will expand $P(T)$ as $\sum_{j=0}^{n} C_j\,T^{\,j}$ and show

$$C_n = 1 \quad \text{and} \quad C_j = 0 \;\; (0 \leq j < n),$$

which implies $P(T) = T^n = (x + y)^n$, as desired.

Step 1. Expand $(T - x + kz)^{n-k}$ and collect coefficients of T^j.

By the binomial theorem,

$$(T - x + kz)^{\,n-k} = \sum_{j=0}^{n-k} \binom{n-k}{j} T^{\,j}(-x + kz)^{\,n-k-j}.$$

Therefore, the coefficient of T^j in $P(T)$ is

$$C_j = \sum_{k=0}^{n-j} \binom{n}{k} \binom{n-k}{j} x\,(x-kz)^{k-1}\,(-x+kz)^{n-k-j}.$$

Use the elementary combinatorial identity

$$\binom{n}{k}\binom{n-k}{j} = \binom{n}{j}\binom{n-j}{k}$$

(choose j objects first, then k of the remaining $n-j$). This gives

$$C_j = \binom{n}{j} \sum_{k=0}^{n-j} \binom{n-j}{k} x\,(x-kz)^{k-1}\,(-x+kz)^{n-j-k}.$$

Let $m = n-j$ (so $0 \le m \le n$). Then

$$C_j = \binom{n}{j} S_m, \qquad S_m = \sum_{k=0}^{m} \binom{m}{k} x\,(x-kz)^{k-1}\,(-x+kz)^{m-k}. \tag{ }$$

Step 2. Invoke Exercise 50 (the special case $T = 0$).

Exercise 50 established Abel's identity when $x + y = 0$, i.e. $T = 0$:

$$0^m = \sum_{k=0}^{m} \binom{m}{k} x\,(x-kz)^{k-1}\,(-x+kz)^{m-k} \quad \text{for all integers } m \ge 1.$$

(Exactly the same sum as S_m above.) Hence

$$S_m = 0 \quad \text{for all} \quad m \ge 1.$$

Therefore, for $j < n$ (i.e. $m = n-j \ge 1$) we get $C_j = \binom{n}{j} S_m = 0$.

Step 3. The top coefficient C_n.

When $j = n$, we have $m = 0$. Then in ( ) the only term is $k = 0$:

$$S_0 = \binom{0}{0} x\,(x - 0\cdot z)^{-1}\,(-x + 0\cdot z)^0 = 1 \qquad (\text{since } x \ne 0).$$

Thus $C_n = \binom{n}{n} S_0 = 1$.

All lower coefficients vanish and the top coefficient is 1, so

$$P(T) = \sum_{j=0}^{n} C_j T^j = T^n.$$

With $T = x + y$ we obtain exactly Abel's formula (16):

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x\,(x-kz)^{k-1}\,(y+kz)^{n-k}.$$

52 [HM11]

1) Recall Abel's binomial formula

For integer $n \geq 0$ and $x \neq 0$,

$$(x+y)^n = \sum_{k} \binom{n}{k} x\,(x-kz)^{k-1}\,(y+kz)^{n-k}.$$

(See Eq. (16) in §1.2.6.)

When n is a nonnegative integer the sum has only finitely many nonzero terms, so the identity is well-defined.

2) Evaluate the right–hand side at $n = x = -1,\ y = z = 1$

Use the generalized binomial coefficient $\binom{-1}{k} = (-1)^k$ (valid for all integers $k \geq 0$). Then each term of the sum becomes

$$\binom{-1}{k} (-1)\,(-1-k)^{k-1}\,(1+k)^{-1-k}.$$

Simplify the signs and powers:

$$\binom{-1}{k}(-1)(-1-k)^{k-1}(1+k)^{-1-k} = (-1)^k(-1)\big[(-1)^{k-1}(k+1)^{k-1}\big](k+1)^{-1-k} = (k+1)^{-2}.$$

Hence the whole right–hand side is

$$\sum_{k\geq 0}\frac{1}{(k+1)^2}=\sum_{m\geq 1}\frac{1}{m^2}=\frac{\pi^2}{6},$$

using the classical evaluation $\zeta(2)=\pi^2/6$.

3) Compare with the left–hand side

At these same values, the left–hand side is

$$(x+y)^n=(-1+1)^{-1}=0^{-1},$$

which is undefined (infinite). Therefore the identity fails at this choice of parameters; in particular, Abel's binomial formula is not valid in general when n is not a nonnegative integer.

53 [M25]

(a) Induction on m

Define

$$S_m=\sum_{k=0}^{m}\binom{r}{k}\binom{s}{n-k}(nr-(r+s)k).$$

Base case $m=0$.

$$S_0=\binom{r}{0}\binom{s}{n}\cdot nr=nr\binom{s}{n}.$$

Right–hand side at $m=0$ is

$$(0+1)(n-0)\binom{r}{1}\binom{s}{n}=n\cdot r\binom{s}{n},$$

which agrees.

Inductive step $m\to m+1$.

Write $S_m=S_{m-1}+\Delta_m$ with

$$\Delta_m = \binom{r}{m}\binom{s}{n-m}(nr-(r+s)m).$$

By the induction hypothesis,

$$S_{m-1} = m(n-m+1)\binom{r}{m}\binom{s}{n-m+1}.$$

Hence we want to show

$$S_m = m(n-m+1)\binom{r}{m}\binom{s}{n-m+1}+\binom{r}{m}\binom{s}{n-m}(nr-(r+s)m) = (m+1)(n-m)\binom{r}{m+1}\binom{s}{n-m}.$$

Use the standard "lowering" recurrences

$$\binom{r}{m+1} = \frac{r-m}{m+1}\binom{r}{m}, \qquad \binom{s}{n-m+1} = \frac{s-(n-m)}{n-m+1}\binom{s}{n-m}.$$

Substitute into the desired identity and factor $\binom{r}{m}\binom{s}{n-m}$. The right–hand side becomes

$$(m+1)(n-m)\frac{r-m}{m+1}\binom{r}{m}\binom{s}{n-m} = (n-m)(r-m)\binom{r}{m}\binom{s}{n-m}.$$

The left–hand side equals

$$m(n-m+1)\binom{r}{m}\frac{s-(n-m)}{n-m+1}\binom{s}{n-m} + \binom{r}{m}\binom{s}{n-m}(nr-(r+s)m)$$

$$= \binom{r}{m}\binom{s}{n-m}\Big(m\,(s-n+m)+nr-(r+s)m\Big) = \binom{r}{m}\binom{s}{n-m}((n-m)r-ms)$$

$$= (n-m)(r-m)\binom{r}{m}\binom{s}{n-m},$$

since $(n-m)r-ms = (n-m)(r-m)$. This matches the right–hand side, completing the induction.

(b) Specialize $r = \frac{1}{2}$, $s = -\frac{1}{2}$

With $r + s = 0$, the factor in part (a) simplifies:

$$nr - (r+s)k = n \cdot \tfrac{1}{2} = \frac{n}{2} \quad \text{(independent of } k\text{)}.$$

Thus part (a) yields the partial convolution

$$\sum_{k=0}^{m} \binom{\frac{1}{2}}{k} \binom{-\frac{1}{2}}{n-k} = \frac{2}{n}(m+1)(n-m) \binom{\frac{1}{2}}{m+1} \binom{-\frac{1}{2}}{n-m}. \qquad (\ )$$

Now use the relations (from ex. 47, stated in the book) for half-integer binomials:

$$\binom{-\frac{1}{2}}{t} = (-1)^t \frac{1}{4^t} \binom{2t}{t}, \qquad \binom{\frac{1}{2}}{t} = (-1)^{t-1} \frac{1}{2^{2t}(2t-1)} \binom{2t}{t} \quad (t \geq 1),$$

and the equivalent form

$$\binom{\frac{1}{2}}{t} = (-1)^{t-1} \frac{1}{2^{2t-1} t} \binom{2t-2}{t-1} \quad (t \geq 1),$$

together with $\binom{2k}{k} = 2\binom{2k-1}{k}$ for $k \geq 1$.

A convenient per-term conversion (valid for $k \geq 1$) is

$$\binom{\frac{1}{2}}{k} \binom{-\frac{1}{2}}{n-k} = (-1)^n \frac{1}{2^{2n-1}} \binom{2n-2k}{n-k} \binom{2k-1}{k} \frac{-1}{2k-1}. \qquad (\dagger)$$

For $k = 0$ one checks separately:

$$\binom{\frac{1}{2}}{0} \binom{-\frac{1}{2}}{n} = (-1)^n \frac{1}{2^{2n}} \binom{2n}{n},$$

which differs by a factor $1/2$ from what ($\dagger$) would give if naively extended to $k = 0$. Precisely this "half-term" produces the stand-alone $\frac{1}{2}\binom{2n}{n}$ in the final answer.

Summing ($\dagger$) for $k = 1, \dots, m$ and adding the $k = 0$ term, we obtain

$$\sum_{k=0}^{m} \binom{2k-1}{k} \binom{2n-2k}{n-k} \frac{-1}{2k-1} = 2^{2n-1}(-1)^n \sum_{k=0}^{m} \binom{\frac{1}{2}}{k} \binom{-\frac{1}{2}}{n-k} + \frac{1}{2}\binom{2n}{n}. \qquad (\ddagger)$$

Insert the convolution ( ) on the right of (‡), then convert the remaining $\binom{\frac{1}{2}}{m+1}$ and $\binom{-\frac{1}{2}}{n-m}$ to central binomials and simplify:

$$\binom{\frac{1}{2}}{m+1}\binom{-\frac{1}{2}}{n-m} = (-1)^n \frac{1}{2^{2n+2}} \frac{1}{2m+1} \binom{2m+2}{m+1}\binom{2n-2m}{n-m}$$

and

$$\frac{\binom{2m+2}{m+1}}{2m+1} = \frac{2}{m+1}\binom{2m}{m}.$$

A short algebraic cleanup yields

$$2^{2n-1}(-1)^n \cdot \frac{2}{n}(m+1)(n-m)\binom{\frac{1}{2}}{m+1}\binom{-\frac{1}{2}}{n-m} = \frac{n-m}{2n}\binom{2m}{m}\binom{2n-2m}{n-m}.$$

Together with the extra $\frac{1}{2}\binom{2n}{n}$ from $k=0$, this is exactly the formula to be shown in part (b).

54 [M21]

Result (the inverse)

$$\boxed{P^{-1} = (q_{ij})_{i,j\geq 0}, \qquad q_{ij} = \begin{cases} (-1)^{i-j}\binom{i}{j}, & i \geq j, \\ 0, & i < j\ , \end{cases}}$$

i.e., take Pascal's triangle and place minus signs in a checkerboard pattern $(-1)^{i+j} = (-1)^{i-j}$.

Proof 1 (direct algebra via a standard binomial identity)

Consider $(PQ)_{ij}$ with Q as above:

$$(PQ)_{ij} = \sum_{k=j}^{i} \binom{i}{k} (-1)^{k-j} \binom{k}{j}.$$

Use $\binom{i}{k}\binom{k}{j} = \binom{i}{j}\binom{i-j}{k-j}$ ("upper–lower factorization"), then set $t = k-j$:

$$(PQ)_{ij} = \binom{i}{j} \sum_{t=0}^{i-j} \binom{i-j}{t} (-1)^t = \binom{i}{j}(1-1)^{i-j} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Hence $PQ = I$. By the same symmetry, $QP = I$, so $Q = P^{-1}$.

Proof 2 (using Knuth's identity (33))

TAOCP Eq. (33) states

$$\sum_k \binom{r}{k}\binom{k}{n}(-1)^{r-k} = \delta_{rn} \qquad (r, n \geq 0).$$

Rewrite the sign as $(-1)^r(-1)^k$ and compare with $(P \cdot Q)_{rn}$ for $Q_{kn} = (-1)^{k-n}\binom{k}{n}$:

$$(PQ)_{rn} = \sum_k \binom{r}{k}(-1)^{k-n}\binom{k}{n} = (-1)^{r+n}\sum_k \binom{r}{k}\binom{k}{n}(-1)^{r-k} = (-1)^{r+n}\delta_{rn} = \delta_{rn}.$$

(When $r = n$ the factor is $(-1)^{2n} = 1$.) Therefore $Q = P^{-1}$.

Small check (4×4 example)

For indices $0 \leq i, j \leq 3$,

$$P = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 2 & 1 & 0 \\ 1 & 3 & 3 & 1 \end{pmatrix}, \qquad P^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ -1 & 3 & -3 & 1 \end{pmatrix},$$

which is Pascal with the checkerboard signs; one can verify $PP^{-1} = I$. (This matches the pattern shown in the book's answer.)

55 [M21]

Knuth gives the inversion formulas (here δ_{mn} is Kronecker's delta):

$$\sum_k \left[{n \atop k}\right]\left\{{k \atop m}\right\}(-1)^{n-k} = \delta_{mn}, \qquad \sum_k \left\{{n \atop k}\right\}\left[{k \atop m}\right](-1)^{n-k} = \delta_{mn}. \qquad (\ )$$

These appear in Eq. (47) immediately after Table 2.

Matrix formulation

Let

- $S = (S_{nk})_{n,k\geq 0}$ with $S_{nk} = \left[{n \atop k}\right]$ (first kind),

- $T = (T_{nk})_{n,k\geq 0}$ with $T_{nk} = \left\{ {n \atop k} \right\}$ (second kind),
- $D = \mathrm{diag}((-1)^0, (-1)^1, (-1)^2, \dots)$ (a diagonal "sign" matrix).

Note that $(DSD)_{nk} = (-1)^{n-k} \left[{n \atop k} \right]$ and similarly $(DTD)_{nk} = (-1)^{n-k} \left\{ {n \atop k} \right\}$.

The first identity in ( ) says exactly

$$(DSD)\,T = I,$$

hence $T^{-1} = DSD$. The second identity says

$$(DTD)\,S = I,$$

hence $S^{-1} = DTD$.

Explicit entrywise formulas (the answer)

Unwinding the definitions gives:

- Inverse of the second-kind triangle T^{-1}:

$$(T^{-1})_{nm} \;=\; (-1)^{n-m} \left[{n \atop m} \right], \qquad n, m \geq 0.$$

- Inverse of the first-kind triangle S^{-1}:

$$(S^{-1})_{nm} \;=\; (-1)^{n-m} \left\{ {n \atop m} \right\}, \qquad n, m \geq 0.$$

Equivalently, in matrix notation:

$$S^{-1} = DTD, \qquad T^{-1} = DSD.$$

Quick sanity check on a small corner

Using the 0...3 block of Table 2:

$$S = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 3 & 1 \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 3 & 1 \end{pmatrix}, \quad D = \mathrm{diag}(1, -1, 1, -1).$$

Compute DTD and DSD; multiplying $S \cdot (DTD)$ and $T \cdot (DSD)$ yields the identity, as predicted by ( ). (All these small blocks are visible in Table 2.)

56 [20]

We use the classic greedy construction that underlies the combinatorial number system:

1. Let a be the largest integer with $\binom{a}{3} \le n$. Put $r_1 = n - \binom{a}{3}$.
2. Let $b < a$ be the largest integer with $\binom{b}{2} \le r_1$. Put $r_2 = r_1 - \binom{b}{2}$.
3. Set $c = r_2$. Then $0 \le c < b$ because

$$\binom{b+1}{2} - \binom{b}{2} = b,$$

so maximality of b forces $r_2 < b$.

This gives $n = \binom{a}{3} + \binom{b}{2} + c$ with $a > b > c \ge 0$.

Uniqueness. a is uniquely determined by step 1 since $\binom{a+1}{3} > n \ge \binom{a}{3}$. Given a, the same "difference" identity above shows b is uniquely determined; then c is forced as the remainder. Hence the representation is unique.

(This is the order-3 case of the general combinatorial number system: Every nonnegative integer has a unique expansion $\sum_{j=1}^{m} \binom{a_j}{j}$ with $a_m > \cdots > a_1 \ge 0$.)

Results for $n = 0, 1, \ldots, 20$

Below are the triples (a, b, c) such that $n = \binom{a}{3} + \binom{b}{2} + \binom{c}{1}$.

n	(a, b, c)	check
0	(2,1,0)	$0 = 0 + 0 + 0$
1	(3,1,0)	$1 = 1 + 0 + 0$
2	(3,2,0)	$2 = 1 + 1 + 0$
3	(3,2,1)	$3 = 1 + 1 + 1$
4	(4,1,0)	$4 = 4 + 0 + 0$
5	(4,2,0)	$5 = 4 + 1 + 0$
6	(4,2,1)	$6 = 4 + 1 + 1$
7	(4,3,0)	$7 = 4 + 3 + 0$
8	(4,3,1)	$8 = 4 + 3 + 1$
9	(4,3,2)	$9 = 4 + 3 + 2$
10	(5,1,0)	$10 = 10 + 0 + 0$
11	(5,2,0)	$11 = 10 + 1 + 0$
12	(5,2,1)	$12 = 10 + 1 + 1$
13	(5,3,0)	$13 = 10 + 3 + 0$
14	(5,3,1)	$14 = 10 + 3 + 1$
15	(5,3,2)	$15 = 10 + 3 + 2$

n	(a,b,c)	check
16	(5,4,0)	$16 = 10 + 6 + 0$
17	(5,4,1)	$17 = 10 + 6 + 1$
18	(5,4,2)	$18 = 10 + 6 + 2$
19	(5,4,3)	$19 = 10 + 6 + 3$
20	(6,1,0)	$20 = 20 + 0 + 0$

(All triples satisfy $a > b > c \geq 0$.)

Pattern and continuation

- The threshold values are the tetrahedral numbers $\binom{a}{3}$: when n passes $\binom{a}{3}$, a increments.
- Between $\binom{a}{3}$ and $\binom{a+1}{3} - 1$, the pairs (b, c) run through all strictly decreasing $b > c$ with $b < a$ in colexicographic order because of the greedy choice and the identity $\binom{b+1}{2} - \binom{b}{2} = b$.
- To continue for larger n, repeat the same greedy steps. Example: $n = 21 \Rightarrow a = 6$ (since $\binom{6}{3} = 20$), $r_1 = 1$; choose $b = 2$ ($\binom{2}{2} = 1$), remainder $c = 0$; hence $21 = \binom{6}{3} + \binom{2}{2} + \binom{0}{1}$.

Knuth labels this representation the combinatorial number system in the exercise statement, and the table above follows exactly from that idea.

57 [M22]

Let

$$\ln n! \;=\; \sum_{m \geq 1} a_m \, n^{\underline{m}} \qquad (n^{\underline{m}} = n(n-1)\cdots(n-m+1))$$

(the Newton–Stirling expansion that Stirling used in his "attempt to generalize the factorial function"). Show that

$$\boxed{a_m = \frac{(-1)^m}{m!} \sum_{k \geq 1} (-1)^k \binom{m-1}{k-1} \ln k}$$

(which is effectively a finite sum since $\binom{m-1}{k-1} = 0$ for $k > m$).

Two ingredients

1. Binomial–difference orthogonality. For integers $m \geq 0$,

$$\sum_n \binom{m}{n}\binom{k}{n}(-1)^{m-n} = \delta_{mk} \quad \Longleftrightarrow \quad \sum_n \binom{m}{n}(-1)^{m-n}\binom{n}{r} = \delta_{mr}.$$

Knuth states the needed special case (his Eq. (33)) and uses it for "inversion" in Example 5 of §1.2.6. We'll use exactly that maneuver to isolate a single coefficient.

2. Forward differences pick off Newton coefficients. If

$$f(n) = \sum_{m\geq 0} c_m \binom{n}{m} = \sum_{m\geq 0} \frac{c_m}{m!}\, n^{\underline{m}},$$

then

$$\Delta^m f(0) = m!\, c_m \qquad \text{and} \qquad c_m = \frac{1}{m!}\sum_{n=0}^{m}(-1)^{m-n}\binom{m}{n} f(n).$$

(Exactly the "inversion" used in Example 5.)

Apply the inversion step to $f(n) = \ln n!$ with the falling–factorial basis:

$$m!\, a_m = \sum_{n=0}^{m}(-1)^{m-n}\binom{m}{n} \ln n! \tag{1}$$

Now expand $\ln n!$ as a telescoping sum of logs and interchange summations:

$$\ln n! = \sum_{k=1}^{n} \ln k \quad \Longrightarrow \quad \sum_{n=0}^{m}(-1)^{m-n}\binom{m}{n} \ln n! = \sum_{k=1}^{m}\left(\sum_{n=k}^{m}(-1)^{m-n}\binom{m}{n}\right) \ln k.$$

Evaluate the inner binomial sum. A standard identity (hockey-stick with alternating signs) gives

$$\sum_{n=k}^{m}(-1)^{m-n}\binom{m}{n} = (-1)^{m-k}\binom{m-1}{k-1}.$$

Substitute back into (1):

$$m!\, a_m = \sum_{k=1}^{m}(-1)^{m-k}\binom{m-1}{k-1} \ln k = (-1)^m \sum_{k=1}^{m}(-1)^k \binom{m-1}{k-1} \ln k.$$

Divide by $m!$ to obtain

$$\boxed{a_m = \frac{(-1)^m}{m!}\sum_{k=1}^{m}(-1)^k\binom{m-1}{k-1}\ln k},$$

which is the desired formula (the summation can be written $\sum_{k\geq 1}$ since terms vanish for $k > m$).

Quick sanity checks

- $m = 1:\ a_1 = \frac{-1}{1!}(-1)\binom{0}{0}\ln 1 = 0$ (so the $n^{\underline{1}} = n$ term is absent, consistent with $\ln 1! = 0$).
- $m = 2:\ a_2 = \frac{1}{2}(-\ln 1 + \ln 2) = \frac{1}{2}\ln 2$. Then at $n = 2$: $\ln 2! = a_2\, 2^{\underline{2}} = (\frac{1}{2}\ln 2)\cdot 2 = \ln 2$.
- $m = 3:\ a_3 = \frac{-1}{6}(2\ln 2 - \ln 3) = \frac{\ln 3 - 2\ln 2}{6}$. At $n = 3$: $\ln 3! = a_2 \cdot 3^{\underline{2}} + a_3 \cdot 3^{\underline{3}} = 3\ln 2 + (\ln 3 - 2\ln 2) = \ln 6$.

58 [M23]

Let $\binom{n}{k}_q$ denote the Gaussian (q-binomial) coefficient (the notation introduced just before Eq. (41) in §1.2.6). Prove the q-nomial theorem

$$\prod_{j=0}^{n-1}(1+q^j x) \;=\; \sum_{k=0}^{n}\binom{n}{k}_q q^{\binom{k}{2}} x^k,$$

and find q-analogues of the "fundamental identities" (17) and (21).

The Gaussian (q-binomial) coefficient is

$$\binom{n}{k}_q = \frac{(1-q^n)(1-q^{n-1})\cdots(1-q^{n-k+1})}{(1-q)(1-q^2)\cdots(1-q^k)}, \qquad 0 \leq k \leq n.$$

It satisfies the standard recurrences (both forms are useful):

$$\boxed{\binom{n}{k}_q = \binom{n-1}{k}_q + q^{n-k}\binom{n-1}{k-1}_q \;=\; q^k\binom{n-1}{k}_q + \binom{n-1}{k-1}_q} \qquad (1)$$

For reference, the classical identities in §1.2.6 are

$$\binom{r}{k} = (-1)^k \binom{k-r-1}{k} \qquad \text{(Eq. (17))} \quad \text{and} \quad \sum_k \binom{r}{k}\binom{s}{n-k} = \binom{r+s}{n} \quad \text{(Eq. (21))}.$$

The q-nomial theorem

Claim. For every integer $n \geq 0$,

$$\prod_{j=0}^{n-1} (1 + q^j x) \;=\; \sum_{k=0}^{n} \binom{n}{k}_q q^{\binom{k}{2}} x^k. \tag{QBT}$$

Proof (induction on n)

- Base $n = 0$. LHS and RHS are both 1.
- Inductive step. Assuming the formula holds for $n-1$, write

$$\prod_{j=0}^{n-1} (1 + q^j x) = \left(\sum_{k=0}^{n-1} \binom{n-1}{k}_q q^{\binom{k}{2}} x^k \right) (1 + q^{n-1} x).$$

  The coefficient of x^k on the RHS is

$$q^{\binom{k}{2}} \binom{n-1}{k}_q \;+\; q^{n-1} q^{\binom{k-1}{2}} \binom{n-1}{k-1}_q \;=\; q^{\binom{k}{2}} \Big(\binom{n-1}{k}_q + q^{n-k} \binom{n-1}{k-1}_q \Big),$$

  which equals $q^{\binom{k}{2}} \binom{n}{k}_q$ by the first recurrence in (1). This gives (QBT) for n.

Thus (QBT) holds for all n.

Sanity check (small n). For $n = 3$,

$$(1+x)(1+qx)(1+q^2x) = 1 + (1+q+q^2)x + (q+q^2+q^3)x^2 + q^3x^3,$$

while the RHS gives $k = 0, 1, 2, 3$ terms $\binom{3}{0}_q$, $\binom{3}{1}_q x$, $q^1\binom{3}{2}_q x^2$, $q^3\binom{3}{3}_q x^3$, and $\binom{3}{1}_q = \binom{3}{2}_q = 1 + q + q^2$, so they match.

q-analogue of (17)

The classical reflection (17) is $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$. Using $1 - q^t = -q^t(1 - q^{-t})$ in the product definition of $\binom{r}{k}_q$, one obtains the q-reflection

$$\boxed{\binom{r}{k}_q \;=\; (-1)^k\, q^{kr-\binom{k}{2}} \binom{k-r-1}{k}_q}. \tag{q-17}$$

q-analogue of (21) (q-Vandermonde/Chu–Vandermonde)

Split the length-$r+s$ product in (QBT) into two contiguous blocks:

$$\prod_{j=0}^{r+s-1}(1+q^j x) = \Big[\prod_{j=0}^{r-1}(1+q^j x)\Big]\Big[\prod_{j=r}^{r+s-1}(1+q^j x)\Big] = \Big[\cdots\Big]\Big[\prod_{j=0}^{s-1}(1+q^j(q^r x))\Big].$$

Apply (QBT) to each bracket and take the coefficient of x^n. Comparing with the single-block expansion for $r+s$ yields

$$\boxed{\sum_{k=0}^{n}\binom{r}{k}_q\binom{s}{n-k}_q q^{(r-k)(n-k)} \;=\; \binom{r+s}{n}_q}. \tag{q-21a}$$

Using the symmetry $\binom{m}{j}_q = q^{j(m-j)}\binom{m}{m-j}_q$ one obtains the equivalent—and frequently quoted—form

$$\boxed{\sum_{k=0}^{n}\binom{r}{k}_q\binom{s}{n-k}_q q^{(s-n+k)k} \;=\; \binom{r+s}{n}_q}. \tag{q-21b}$$

59 [M25]

1) Reduce to a homogeneous ("Pascal") recurrence

The inhomogeneous term $\binom{n}{k}$ suggests subtracting a multiple of $\binom{n}{k}$. Define

$$S_{n,k} \;=\; A_{n,k} - (n+1)\binom{n}{k}.$$

Then for $nk > 0$,

$$
\begin{aligned}
S_{n,k} &= A_{n-1,k} + A_{n-1,k-1} + \binom{n}{k} - (n+1)\binom{n}{k} \\
&= [S_{n-1,k} + n\binom{n-1}{k}] + [S_{n-1,k-1} + n\binom{n-1}{k-1}] + \binom{n}{k} - (n+1)\binom{n}{k} \\
&= S_{n-1,k} + S_{n-1,k-1}
\end{aligned}
$$

because $\binom{n-1}{k} + \binom{n-1}{k-1} = \binom{n}{k}$. Thus $S_{n,k}$ satisfies the *homogeneous* Pascal recurrence.

2) Boundary values for $S_{n,k}$

From the given edge conditions,

$$
S_{n,0} = A_{n,0} - (n+1)\binom{n}{0} = 1 - (n+1) = -n, \qquad S_{0,k} = A_{0,k} - 1 \cdot \binom{0}{k} = \delta_{0k} - \delta_{0k} = 0.
$$

3) Identify the unique solution to the homogeneous problem

The homogeneous Pascal rule with those boundaries is solved by

$$
S_{n,k} = -\binom{n}{k+1}.
$$

This is easily proved by induction: It matches the edges ($S_{n,0} = -\binom{n}{1} = -n$, $S_{0,k} = -\binom{0}{k+1} = 0$), and satisfies

$$
-\binom{n}{k+1} = -(\binom{n-1}{k+1} + \binom{n-1}{k}) = S_{n-1,k} + S_{n-1,k-1}.
$$

4) Assemble the answer

Therefore

$$
\boxed{A_{n,k} = (n+1)\binom{n}{k} - \binom{n}{k+1}} \qquad (n \geq 0,\ k \geq 0).
$$

This single formula automatically gives the edges:

- $k = 0$: $A_{n,0} = (n+1) - \binom{n}{1} = 1$.

- $n = 0$: $A_{0,k} = 1 \cdot \binom{0}{k} - \binom{0}{k+1} = \delta_{0k}$.

5) Equivalent (often handy) forms

Using $\binom{n}{k+1} = \frac{n-k}{k+1}\binom{n}{k}$ and $\binom{n+1}{k+1} = \frac{n+1}{k+1}\binom{n}{k}$, we get two nice equivalents:

$$A_{n,k} = \frac{kn + 2k + 1}{k+1}\binom{n}{k} \;=\; \binom{n}{k} + k\binom{n+1}{k+1}.$$

6) Quick sanity check

For example, with $n = 5$:

$$\left(A_{5,k}\right)_{k=0}^{5} = (1,\ 20,\ 50,\ 55,\ 29,\ 6),$$

and indeed $A_{5,k} = (6)\binom{5}{k} - \binom{5}{k+1}$ or $\binom{5}{k} + k\binom{6}{k+1}$ gives the same values. That completes the determination of $A_{n,k}$.

61 [M25]

We'll use only the standard Stirling transforms listed in §1.2.6:

- Conversion between factorial and ordinary powers

$$x^{\underline{n}} = \sum_k (-1)^{n-k} \left[{n \atop k}\right] x^k, \qquad x^n = \sum_k \left\{{n \atop k}\right\} x^{\underline{k}}, \tag{A}$$

  and their "inversion" (orthogonality)

$$\sum_k \left\{{n \atop k}\right\} \left[{k \atop m}\right] (-1)^{k-m} \;=\; \delta_{nm}. \tag{B}$$

  (These are Eqs. (44), (45), and the first formula in (47).)

- The "raising-index" identity for Stirling numbers of the first kind

$$\left[{n+1 \atop m+1}\right] \;=\; \sum_{k \le n} \left[{k \atop m}\right] \frac{n!}{k!}, \tag{C}$$

  which is the first of Knuth's Eq. (56).

Start from (C) with m replaced by k and m playing the final role in the sum:

$$\left[{n+1 \atop k+1}\right] = \sum_{j\le n} \left[{j \atop k}\right] \frac{n!}{j!}.$$

Insert this into $S_{n,m}$ and interchange the sums:

$$\begin{aligned} S_{n,m} &= \sum_k \left\{{k \atop m}\right\} (-1)^{k-m} \sum_{j\le n} \left[{j \atop k}\right] \frac{n!}{j!} \\ &= \sum_{j\le n} \frac{n!}{j!} \underbrace{\sum_k \left\{{k \atop m}\right\} \left[{j \atop k}\right] (-1)^{k-m}}_{=\ \delta_{jm} \text{ by (B)}}. \end{aligned}$$

By orthogonality (B) the inner sum collapses to δ_{jm}, leaving

$$S_{n,m} = \frac{n!}{m!} \qquad (0 \le m \le n),$$

and $S_{n,m} = 0$ when $m > n$ (since then $\left\{{k \atop m}\right\} = 0$ for all $k \le n$).

This gives the requested "companion" to Eq. (55) (which states $\sum_k \left\{{n+1 \atop k+1}\right\} \left[{k \atop m}\right] (-1)^{k-m} = \binom{n}{m}$).

$$\boxed{\sum_k \left[{n+1 \atop k+1}\right] \left\{{k \atop m}\right\} (-1)^{k-m} \ = \ \frac{n!}{m!}.}$$

Quick check. For $n = 6$: the row of values is $720, 720, 360, 120, 30, 6, 1$ for $m = 0, \dots, 6$, i.e. $6!/m!$, matching the formula.

62 [M23]

Step 1: A tool we'll use (Exercise 31)

We'll repeatedly use the identity that (in the notation of §1.2.6)

$$\sum_t (-1)^t \binom{m-r+s}{t} \binom{n+r-s}{n-t} \binom{r+t}{m+n} = \binom{r}{m} \binom{s}{n}. \qquad (\ )$$

This can be proved in a line or two by the standard Vandermonde trick twice: expand $\binom{r+t}{m+n} = \sum_j \binom{r}{m+n-j}\binom{t}{j}$, swap sums, use $\binom{A}{j}\binom{B}{N-j}$–convolution, and simplify via $\binom{a}{b}\binom{b}{c} = \binom{a}{c}\binom{a-c}{b-c}$. (This is exactly the result Exercise 31 asks you to derive.)

Step 2: Break one binomial to create a “ $r+t$ ” top

Start from the left-hand side

$$S := \sum_k (-1)^k \binom{l+m}{l+k}\binom{m+n}{m+k}\binom{n+l}{n+k}.$$

Use the binomial splitting

$$\binom{n+l}{n+k} = \sum_j \binom{l+k}{j}\binom{n-k}{n+k-j},$$

which is just $\binom{A+B}{U} = \sum_j \binom{A}{j}\binom{B}{U-j}$ with $A = l+k$, $B = n-k$, $U = n+k$. Substitute this into S and swap the order of summation:

$$S = \sum_j \sum_k (-1)^k \binom{l+m}{l+k}\binom{m+n}{m+k}\binom{l+k}{j}\binom{n-k}{n+k-j}.$$

Now the inner sum over k has the shape needed for ( ): it is a sum in k of three binomial factors where one top is $l+k$ (a “$r+t$” form) and another uses $n-k$ (a “$n+r-s, n-t$” form). Apply ( ) to the k–sum with the parameter match

$$(m, n, r, s, t) \;\leftarrow\; (\, m+k,\; l-k,\; m+n,\; n+l,\; j\,),$$

i.e., we fix j and view the inner summation as the ( )–sum in the variable k. After the (routine) Vandermonde simplifications, the entire dependence on k collapses to a single elementary alternating binomial sum (next step), and a j–dependent constant factor pulls out. (This is the “very short proof” Knuth hints at, and it’s the route sketched in the official answers.)

Concretely, one convenient way to write the outcome is

$$\sum_k (-1)^k \binom{l+m}{l+k}\binom{m+n}{m+k}\binom{l+k}{j}\binom{n-k}{n+k-j} = \frac{(m+n+j)!}{j!\,(m-l+j)!\,(n+j-l)!}\;\sum_k (-1)^k \binom{2l-2j}{l-j+k}. \tag{1}$$

(You get the prefactor by rearranging factorials; the two $(l \pm k - j)!$ combine into the single central binomial $\binom{2(l-j)}{(l-j)+k}$.)

Step 3: Kill almost all j’s with $(1-1)^N$

Evaluate the remaining k–sum in (1):

$$\sum_k (-1)^k \binom{2l-2j}{l-j+k} \;=\; \sum_t (-1)^{t-(l-j)} \binom{2l-2j}{t} \;=\; (-1)^{l-j}(1-1)^{2l-2j}.$$

Thus the sum is 0 unless $2l-2j=0$, i.e. $j=l$; and when $j=l$ there is exactly one surviving k–term and the inner sum equals 1.

Therefore only $j=l$ contributes to S. Plugging $j=l$ into the prefactor in (1) gives

$$S \;=\; \frac{(m+n+l)!}{l!\,(m)!\,(n)!}\,,$$

which is exactly the claimed right-hand side.

Quick sanity check

Take $l=m=n=1$. Then the sum runs over $k=-1,0,1$:

$$(-1)^{-1}\binom{2}{0}\binom{2}{0}\binom{2}{0} + (-1)^0\binom{2}{1}\binom{2}{1}\binom{2}{1} + (-1)^1\binom{2}{2}\binom{2}{2}\binom{2}{2} = -1+8-1 = 6,$$

while $(1+1+1)!/(1!\,1!\,1!) = 6$.

63 [M30]

We work with formal power series so all manipulations are valid for integral parameters.

1. Encode each binomial with a coefficient operator:

- $\binom{j+k}{k+l} = [x^{k+l}](1+x)^{j+k}$.
- $\binom{r}{j} = [u^j](1+u)^r$.
- $\binom{n}{k} = [v^k](1+v)^n$.
- $\binom{s+n-j-k}{m-j} = [y^{m-j}](1+y)^{s+n-j-k}$.

Insert these and interchange sums with coefficient operators:

$$\begin{aligned} S &= \sum_{j,k} (-1)^{j+k} [x^{k+l}](1+x)^{j+k}\, [u^j](1+u)^r\, [v^k](1+v)^n\, [y^{m-j}](1+y)^{s+n-j-k} \\ &= [y^m]\,(1+y)^{s+n} \sum_{j,k} [x^{k+l}](1+x)^{j+k}\, [u^j](1+u)^r\, [v^k](1+v)^n\, (-1)^j y^{-j}\, (-1)^k (1+y)^{-k}. \end{aligned}$$

2. Sum over k first. Use $[x^{k+l}](1+x)^{j+k} = [x^l](1+x)^j\big((1+x)/x\big)^k$. Thus

$$\sum_{k\geq 0} (-1)^k \binom{n}{k} \Big(\frac{1+x}{x(1+y)}\Big)^k = (1 - \tfrac{1+x}{x(1+y)})^n = \Big(\frac{yx-1}{(1+y)x}\Big)^n.$$

Therefore

$$S = [y^m]\,(1+y)^s\, [x^{l+n}]\,(yx-1)^n \sum_{j\geq 0} [u^j](1+u)^r\, (-1)^j \Big(\frac{(1+x)y}{1+y}\Big)^j.$$

3. Now sum over j using $\sum_j (-1)^j \binom{r}{j} t^j = (1-t)^r$:

$$\sum_{j\geq 0} (-1)^j \binom{r}{j} \Big(\frac{(1+x)y}{1+y}\Big)^j = \Big(1 - \frac{(1+x)y}{1+y}\Big)^r = \Big(\frac{1-xy}{1+y}\Big)^r.$$

Hence

$$S = [y^m][x^{l+n}]\,(1+y)^{s-r}\,(yx-1)^n\,(1-xy)^r.$$

4. Extract the x-coefficient. Since $(yx-1)^n = (-1)^n(1-xy)^n$,

$$[x^{l+n}](yx-1)^n(1-xy)^r = (-1)^n[x^{l+n}](1-xy)^{n+r} = (-1)^n(-1)^{l+n}\binom{n+r}{l+n} y^{l+n}.$$

Thus

$$S = (-1)^l \binom{n+r}{l+n} [y^m]\, y^{l+n}(1+y)^{s-r} = (-1)^l \binom{n+r}{n+l}\binom{s-r}{m-n-l},$$

which is exactly the desired identity.

64 [M20]

Proof by induction via the standard recurrence

Let $P(n, m)$ be the number of partitions of an n-element set into m nonempty blocks.

Base values

- $P(0, 0) = 1$ (the empty partition of the empty set).
- $P(n, 0) = 0$ for $n > 0$; $P(n, m) = 0$ for $m > n$.

Recurrence

Fix a distinguished element, say n. Any partition of $\{1, \dots, n\}$ into m blocks falls into exactly one of two classes:

1. n is alone in its own block. Then the remaining $n - 1$ elements form a partition into $m - 1$ blocks: $P(n - 1, m - 1)$ choices.
2. n is not alone. First partition $\{1, \dots, n - 1\}$ into m blocks, then choose which block n joins. There are $m \cdot P(n - 1, m)$ choices.

Hence

$$P(n, m) = P(n - 1, m - 1) + m\, P(n - 1, m). \tag{$*$}$$

But $\left\{ {n \atop m} \right\}$, the Stirling number of the second kind, is characterized by the same initial conditions and the same recurrence

$$\left\{ {n \atop m} \right\} = \left\{ {n-1 \atop m-1} \right\} + m \left\{ {n-1 \atop m} \right\},$$

(which is one of the "addition/recurrence" formulas alluded to as Eq. (46) in the text). Therefore $P(n, m) = \left\{ {n \atop m} \right\}$ for all n, m.

Example: $\left\{ {4 \atop 2} \right\} = 7$

All partitions of $\{1, 2, 3, 4\}$ into two nonempty blocks are:

$$\{1,2,3\}\{4\},\ \{1,2,4\}\{3\},\ \{1,3,4\}\{2\},\ \{2,3,4\}\{1\},\ \{1,2\}\{3,4\},\ \{1,3\}\{2,4\},\ \{1,4\}\{2,3\},$$

seven in total, as stated.

(Optional) Alternate derivation (surjections)

Count surjections $f : [n] \twoheadrightarrow [m]$. There are $\sum_{j=0}^{m}(-1)^j\binom{m}{j}(m-j)^n$ of them by inclusion–exclusion; each surjection corresponds to a partition together with an ordering of the m blocks, so dividing by $m!$ yields

$$\left\{ {n \atop m} \right\} = \frac{1}{m!}\sum_{j=0}^{m}(-1)^j\binom{m}{j}(m-j)^n,$$

which again shows that $\left\{ {n \atop m} \right\}$ counts m-block partitions.

65 [HM35]

Using the generalized Stirling numbers of both kinds (defined for arbitrary real/complex top argument r), Knuth states:

$$\boxed{z^{\,r} \;=\; \sum_{k\ge 0}\left\{ {r \atop r-k} \right\} z^{\underline{r-k}}} \qquad \text{for } \Re(z) > 0, \tag{59}$$

and the "companion" asymptotic series

$$\boxed{z^{\,r} \;=\; \sum_{k=0}^{m}\left[{r \atop r-k} \right](-1)^k\, z^{\,r-k} \;+\; O\big(z^{\,r-m-1}\big)} \qquad (m = 0, 1, 2, \dots), \tag{60}$$

where $z^{\underline{\alpha}} := \dfrac{\Gamma(z+1)}{\Gamma(z+1-\alpha)}$ is the (gamma–extended) falling factorial. See the statements just after Eq. (58) in the book.

Recall for integers n the exact conversion formulas

$$x^{\underline{n}} = \sum_k (-1)^{\,n-k}\left[{n \atop k} \right] x^k, \qquad x^n = \sum_k \left\{ {n \atop k} \right\} x^{\underline{k}}, \tag{44–45}$$

and the inversion relations

$$\sum_k \left[{n \atop k} \right]\left\{ {k \atop m} \right\}(-1)^{\,n-k} = \delta_{nm}, \qquad \sum_k \left\{ {n \atop k} \right\}\left[{k \atop m} \right](-1)^{\,n-k} = \delta_{nm}. \tag{47}$$

Proof of (59) (convergent series)

Step 1 (a basic generating identity). For any complex z and for small t,

$$e^{zt} = \left(1 + (e^t - 1)\right)^z = \sum_{k\geq 0} \binom{z}{k} (e^t - 1)^k = \sum_{k\geq 0} \frac{z^{\underline{k}}}{k!} (e^t - 1)^k. \tag{A}$$

Step 2 (EGF of Stirling numbers $\left\{ {n \atop k} \right\}$). For each fixed (integer) $k \geq 0$,

$$(e^t - 1)^k = k! \sum_{n\geq k} \left\{ {n \atop k} \right\} \frac{t^n}{n!}. \tag{B}$$

Substitute (B) into (A) and interchange the sums:

$$e^{zt} = \sum_{n\geq 0} \left(\sum_{k=0}^{n} \left\{ {n \atop k} \right\} z^{\underline{k}} \right) \frac{t^n}{n!}.$$

Comparing coefficients of t^n gives, for every integer $n \geq 0$,

$$z^n = \sum_{k=0}^{n} \left\{ {n \atop k} \right\} z^{\underline{k}},$$

which is precisely Eq. (45) rewritten.

Step 3 (extend $n \to r$). For noninteger r we define the generalized Stirling numbers of the second kind by the same Newton–difference formula

$$\left\{ {r \atop k} \right\} = \frac{1}{k!} \Delta^k(x^r)\big|_{x=0} = \frac{1}{k!} \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^r,$$

which agrees with $\left\{ {n \atop k} \right\}$ when $r = n \in \mathbb{N}$. Then the Newton series for x^r is

$$x^r = \sum_{k\geq 0} \left\{ {r \atop k} \right\} x^{\underline{k}}.$$

A standard convergence theorem for Newton series shows that this series converges for $\Re(x) > 0$. (Heuristically: $\binom{x}{k} = x^{\underline{k}}/k! \sim (-1)^k k^{-\Re(x)-1}/\Gamma(-x)$ while $\Delta^k(x^r) = k!\left\{ {r \atop k} \right\}$ grows only polynomially in k; hence the terms behave like $k^{-\Re(x)-1+(\text{poly})}$ and the series converges when $\Re(x) > 0$.) Replacing x by z and reindexing $k \mapsto r - k$ gives Eq. (59) as stated in the book.

> Thus (59) is nothing more than the Newton (forward-difference) expansion of the function $z \mapsto z^r$ in the falling-factorial basis, with coefficients given by the generalized Stirling numbers of the second kind.

Proof of (60) (asymptotic companion involving first-kind numbers)

Write the falling factorial via gamma functions:

$$z^{\underline{\alpha}} = \frac{\Gamma(z+1)}{\Gamma(z+1-\alpha)}.$$

For fixed α and $z \to \infty$ in any right half-plane, Stirling's asymptotic for the gamma ratio yields a full descending-power expansion

$$z^{\underline{\alpha}} = z^{\alpha}\big(1 + c_1(\alpha)z^{-1} + c_2(\alpha)z^{-2} + \cdots + c_m(\alpha)z^{-m} + O(z^{\alpha-m-1})\big)\,,$$

whose coefficients $c_j(\alpha)$ are known polynomials in α. A classical (and very pretty) fact is that these polynomials can be written with Stirling numbers of the first kind:

$$z^{\underline{\alpha}} = \sum_{j=0}^{m} (-1)^j \begin{bmatrix} \alpha \\ \alpha-j \end{bmatrix} z^{\alpha-j} \;+\; O(z^{\alpha-m-1}) \qquad (z \to \infty,\ \Re z > 0). \tag{C}$$

(Equation (44) for integer α is exact; (C) is its gamma-based asymptotic extension.)

Now plug (C) with $\alpha = r - k$ into the convergent series (59):

$$\begin{aligned} z^{\,r} &= \sum_{k\ge 0} \begin{Bmatrix} r \\ r-k \end{Bmatrix} z^{\underline{r-k}} \\ &= \sum_{k\ge 0} \begin{Bmatrix} r \\ r-k \end{Bmatrix} \left(\sum_{j=0}^{m} (-1)^j \begin{bmatrix} r-k \\ r-k-j \end{bmatrix} z^{\,r-j} \;+\; O(z^{\,r-k-m-1}) \right). \end{aligned}$$

Because (59) converges uniformly in any sector $\Re z \ge \sigma > 0$, we may truncate and interchange the (finite) inner sum with the outer sum, obtaining

$$z^{\,r} = \sum_{j=0}^{m} (-1)^j \underbrace{\left(\sum_{k\ge 0} \begin{Bmatrix} r \\ r-k \end{Bmatrix} \begin{bmatrix} r-k \\ r-k-j \end{bmatrix} \right)}_{= \begin{bmatrix} r \\ r-j \end{bmatrix} \text{ by the inversion relations}} z^{\,r-j} \;+\; O(z^{\,r-m-1})\,.$$

The evaluation of the bracketed sum as $\begin{bmatrix} r \\ r-j \end{bmatrix}$ is exactly the Stirling inversion in Eq. (47), applied in its (polynomial) extension to general r. This yields the claimed asymptotic expansion (60).

66 [HM30]

Let real numbers x, y, z and an integer $n \geq 2$ satisfy

$$\binom{x}{n} = \binom{y}{n} + \binom{z}{n-1}, \qquad x \geq n-1,\ y \geq n-1,\ z > n-2.$$

Prove the two "if and only if" statements:

$$\binom{x}{n-1} \leq \binom{y}{n-1} + \binom{z}{n-2} \iff y \geq z,$$

$$\binom{x}{n+1} \leq \binom{y}{n+1} + \binom{z}{n} \iff y \leq z.$$

We will prove the second inequality first (it's the key step), then deduce the first.

Notation and standard identities

Write

$$X = \binom{x}{n}, \quad Y = \binom{y}{n}, \quad Z = \binom{z}{n-1}.$$

For "neighbors" in the upper index we use:

$$\binom{t}{n-1} = \frac{n}{t-n+1}\binom{t}{n}, \qquad \binom{t}{n+1} = \frac{t-n}{n+1}\binom{t}{n} \qquad (t \in \{x, y\}),$$

and for Z (whose lower index is $n-1$):

$$Z_- = \binom{z}{n-2} = \frac{n-1}{z-n+2}\,Z, \quad Z_+ = \binom{z}{n} = \frac{z-n+1}{n}\,Z.$$

Also, for $t > n-1$,

$$\frac{d}{dt}\binom{t}{n} = \binom{t}{n}\sum_{j=0}^{n-1}\frac{1}{t-j}\,.$$

Because $\binom{t}{n}$ is strictly increasing for $t \geq n-1$, the hypothesis $X = Y + Z$ forces $x > y$ unless $Z = 0$ (which can't happen since $z > n-2$).

Part B (the "$n+1$" inequality): sign is determined by the order of y vs z

Define

$$F(z) \;=\; \underbrace{\binom{x}{n+1}}_{=:X_+} - \underbrace{\binom{y}{n+1}}_{=:Y_+} - \underbrace{\binom{z}{n}}_{=:Z_+},$$

where x depends on z via the constraint $X = Y + Z$. Using the neighbor identities and $X = Y + Z$,

$$F(z) = \frac{x-n}{n+1}(Y+Z) - \frac{y-n}{n+1}Y - \frac{z-n+1}{n}\,Z.$$

If $F(z) = 0$ at some $z > n-2$, since $x > y$ and $Y, Z > 0$ we must have

$$\frac{x-n}{n+1} < \frac{z-n+1}{n}. \tag{ }$$

Now differentiate implicitly $X = Y + Z$ with respect to z. Using

$$\frac{X'}{X} = \sum_{j=0}^{n-1} \frac{1}{x-j}, \qquad \frac{Z'}{Z} = \sum_{j=0}^{n-2} \frac{1}{z-j},$$

and comparing termwise via ( ), one gets

$$\frac{X'}{X} > \frac{n}{n+1} \cdot \frac{Z'}{Z} \quad \Longrightarrow \quad \frac{dx}{dz} = \frac{Z'}{X'} < \frac{n+1}{n} \cdot \frac{Z}{X}.$$

A short calculation (differentiate F and substitute the last bound together with ( )) yields

$$F'(z) < 0 \quad \text{at every zero of } F.$$

Hence F has at most one root for $z > n-2$, and the function crosses that root with negative slope. But when $z = y$, our constraint gives $x = y + 1$ (since $X = Y + \binom{y}{n-1} = \binom{y+1}{n}$), and then the Pascal relation $\binom{y+1}{n+1} = \binom{y}{n+1} + \binom{y}{n}$ shows $F(y) = 0$. Therefore that root is unique and occurs exactly at $z = y$. Consequently,

$$F(z) \leq 0 \iff z \geq y,$$

with equality iff $z = y$. This is exactly

$$\binom{x}{n+1} \le \binom{y}{n+1} + \binom{z}{n} \iff y \le z,$$

as required. (This is essentially Knuth's solution idea.)

Part A (the "$n-1$" inequality) follows by the same idea with $n \mapsto n-1$

Consider

$$G(z) \;=\; \binom{x}{n-1} - \binom{y}{n-1} - \binom{z}{n-2}.$$

Exactly the same analysis as above applies to the triple $(\binom{x}{n-1}, \binom{y}{n-1}, \binom{z}{n-2})$, provided $n-1 \ge 2$, i.e., $n > 2$. (That ensures the same monotonicity/uniqueness argument goes through.) One again finds that G has a unique zero at $z = y$ and $G'(z) < 0$ there; hence

$$G(z) \le 0 \iff z \le y,$$

i.e.,

$$\binom{x}{n-1} \le \binom{y}{n-1} + \binom{z}{n-2} \iff y \ge z.$$

(For $n = 2$ the statement is still true and can be checked directly.)

67 [M20]

For integers $n \ge k \ge 1$, prove the useful bounds

$$\boxed{\left(\frac{n}{k}\right)^k \;\le\; \binom{n}{k} \;\le\; \left(\frac{e\,n}{k}\right)^k}.$$

Lower bound $\binom{n}{k} \ge \left(\frac{n}{k}\right)^k$

Write

$$\binom{n}{k} = \frac{n(n-1)\cdots(n-k+1)}{k!} = \prod_{i=1}^{k} \frac{n-k+i}{i}.$$

For each $i \in \{1, \ldots, k\}$,

$$\frac{n-k+i}{i} \;\geq\; \frac{n}{k} \quad\Longleftrightarrow\quad k(n-k+i) \;\geq\; ni \quad\Longleftrightarrow\quad kn - k^2 \;\geq\; (n-k)i,$$

which holds because $i \leq k$ implies $(n-k)i \leq (n-k)k = kn - k^2$. Multiplying these k inequalities gives

$$\binom{n}{k} \;\geq\; \left(\frac{n}{k}\right)^k.$$

(Intuition: in each factor $(n-k+i)/i$, the numerator is at least linearly growing while the denominator is at most k; every factor is $\geq n/k$, so the product is $\geq (n/k)^k$.)

Upper bound $\binom{n}{k} \leq \left(\frac{e\,n}{k}\right)^k$

First,

$$\binom{n}{k} = \frac{n(n-1)\cdots(n-k+1)}{k!} \;\leq\; \frac{n^k}{k!}.$$

Next, use the classical inequality (a one-sided Stirling bound)

$$k! \;\geq\; \left(\frac{k}{e}\right)^k,$$

which is Exercise 1.2.5–24 (or follows immediately from $\ln k! = \sum_{i=1}^{k} \ln i \geq k \ln k - k$). Substituting yields

$$\binom{n}{k} \;\leq\; \frac{n^k}{(k/e)^k} = \left(\frac{e\,n}{k}\right)^k.$$

Knuth notes a slightly tighter chain on the right:

$$\binom{n}{k} = \frac{n^{\underline{k}}}{k!} \leq \frac{n^k}{k!} \leq \frac{1}{e}\left(\frac{e\,n}{k}\right)^k \leq \left(\frac{e\,n}{k+1}\right)^k,$$

using $(1+1/k)^k < e$ for the last inequality. (These are "slightly sharper but less memorable.")

Remarks

- The two bounds are often good enough for quick size estimates of $\binom{n}{k}$ without invoking full Stirling's formula.
- If you need asymptotics, Stirling's approximation refines $\binom{n}{k}$ to within a multiplicative $1+o(1)$ factor; but the exercise only asks for the clean double-sided power bounds.

68 [M25]

Let $K \sim \mathrm{Bin}(n,p)$ with $n \in \mathbb{Z}_{\geq 0}$, $p \in [0,1]$, and put $m = \lceil np \rceil$. Prove

$$\sum_{k=0}^{n} \binom{n}{k} p^k (1-p)^{n-k} \, |k-np| \;=\; 2\,m \binom{n}{m} p^m (1-p)^{n+1-m}. \qquad (\ )$$

(Equivalently, $\mathbb{E}\,|K-np|$ equals the right–hand side.)

1) Reduce $\mathbb{E}|K-np|$ to a one-sided tail

Let $m = \lceil np \rceil$. Split the sum at $k=m$. Because $\mathbb{E}(K-np)=0$,

$$\sum_{k \leq m-1} (np-k)\,\Pr[K=k] \;=\; \sum_{k \geq m} (k-np)\,\Pr[K=k].$$

Hence

$$\mathbb{E}\,|K-np| = \sum_{k \leq m-1} (np-k)\,\Pr[K=k] + \sum_{k \geq m} (k-np)\,\Pr[K=k] = 2\sum_{k \geq m} (k-np)\,\Pr[K=k]. \quad (1)$$

Write $a_k = \binom{n}{k} p^k (1-p)^{n-k} = \Pr[K=k]$ and denote

$$S(p) \;:=\; \sum_{k=m}^{n} (k-np)\,a_k.$$

Then (1) is

$$\mathbb{E}\,|K-np| \;=\; 2\,S(p). \qquad (2)$$

2) Turn the weighted tail into a derivative

Differentiate a_k with respect to p:

$$\frac{d}{dp}a_k = a_k\left(\frac{k}{p} - \frac{n-k}{1-p}\right) = \frac{k-np}{p(1-p)}\,a_k.$$

Therefore

$$S(p) = \sum_{k=m}^{n}(k-np)a_k = p(1-p)\sum_{k=m}^{n}\frac{d}{dp}a_k = p(1-p)\,\frac{d}{dp}\left(\sum_{k=m}^{n}a_k\right). \tag{3}$$

Let $T_m(p) := \sum_{k=m}^{n} a_k = \Pr[K \geq m]$. Then $S(p) = p(1-p)T_m'(p)$.

3) Evaluate $T_m'(p)$ by telescoping

Differentiate T_m termwise and rearrange:

$$\begin{aligned} T_m'(p) &= \sum_{k=m}^{n}\binom{n}{k}\left(k\,p^{k-1}(1-p)^{n-k} - (n-k)\,p^k(1-p)^{n-k-1}\right) \\ &= \sum_{k=m}^{n} k\binom{n}{k}p^{k-1}(1-p)^{n-k} - \sum_{k=m}^{n-1}(n-k)\binom{n}{k}p^k(1-p)^{n-k-1}. \end{aligned}$$

Use $(n-k)\binom{n}{k} = (k+1)\binom{n}{k+1}$ and shift the index in the second sum:

$$T_m'(p) = \sum_{k=m}^{n} k\binom{n}{k}p^{k-1}(1-p)^{n-k} - \sum_{j=m+1}^{n} j\binom{n}{j}p^{j-1}(1-p)^{n-j} = m\binom{n}{m}p^{m-1}(1-p)^{n-m}. \tag{4}$$

4) Combine

From (3) and (4),

$$S(p) = p(1-p)\,T_m'(p) = m\binom{n}{m}p^m(1-p)^{n+1-m}.$$

Insert this into (2) to obtain ( ), which completes the proof.

Remarks

- The identity is valid at the endpoints as well: if $p = 0$ or $p = 1$, then $m \in \{0, n\}$ and both sides are 0.

- Conceptually, ( ) says the mean absolute deviation of a binomial from its mean is twice a boundary term at the "median cut" $m = \lceil np \rceil$: the calculation is essentially a one-line telescoping after recognizing the derivative trick.

Tiny sanity check

For $n = 1$, $K \in \{0, 1\}$, $m = \lceil p \rceil$.

$$\mathbb{E}|K - p| = p(1-p) + (1-p)p = 2p(1-p),$$

and the right–hand side of ( ) is $2 \cdot 1 \cdot \binom{1}{1} p^1 (1-p)^1 = 2p(1-p)$.

1.2.7 Harmonic numbers

1 [01]

Here "harmonic numbers" are defined by $H_n = \sum_{k=1}^{n} \frac{1}{k}$ for integers $n \geq 0$. (So H_0 is the empty sum.)

- H_0. The sum has no terms, so by the standard convention for empty sums it equals 0. This also matches the recurrence $H_n = H_{n-1} + \frac{1}{n}$: plugging $n = 1$ gives $H_1 = H_0 + 1$, so H_0 must be 0.
- H_1. One term: $H_1 = \frac{1}{1} = 1$.
- H_2. Two terms: $H_2 = \frac{1}{1} + \frac{1}{2} = 1 + \frac{1}{2} = \frac{3}{2} = 1.5$.

2 [13]

Proof (matching the "simple argument" style)

Observe how H_{2m} changes when m increases by 1:

$$H_{2m+2} - H_{2m} \;=\; \frac{1}{2m+1} + \frac{1}{2m+2} \;<\; \frac{1}{2m+1} + \frac{1}{2m+1} \;=\; \frac{2}{2m+1} \;\leq\; 1.$$

Thus, each time we extend from $2m$ to $2m+2$ terms, the harmonic sum rises by less than 1.

Use induction on m.

- Base case. $H_2 = 1 + \frac{1}{2} = \frac{3}{2} \leq 2 = 1 + 1$.
- Inductive step. If $H_{2m} \leq 1 + m$, then

$$H_{2(m+1)} = H_{2m+2} \;=\; H_{2m} + (H_{2m+2} - H_{2m}) \;\leq\; (1+m) + 1 \;=\; 2 + m \;=\; 1 + (m+1).$$

Hence $H_{2m} \le 1 + m$ for all $m \ge 1$.

This mirrors the text's earlier "step-size" reasoning for the lower bound $H_{2m} \ge 1 + \frac{m}{2}$ (there, each step from $2m$ to $2m+2$ adds at least $\frac{1}{2}$); here we note each step adds at most 1.

One-line alternative (even slightly stronger)

After the first term, every summand is $\le \frac{1}{2}$:

$$H_{2m} = 1 + \sum_{k=2}^{2m} \frac{1}{k} \;\le\; 1 + (2m-1) \cdot \frac{1}{2} \;=\; m + \tfrac{1}{2} \;\le\; 1 + m.$$

This yields $H_{2m} \le m + \frac{1}{2}$, which is stronger than $1 + m$.

3 [M21]

For $r > 1$, show the generalized harmonic partial sums

$$H_n^{(r)} \;=\; \sum_{k=1}^{n} \frac{1}{k^r}$$

remain bounded for all n, and find an upper bound.

Solution (dyadic-block argument, generalizing Ex. 2)

Group the terms into blocks whose lengths double each time (powers of two), exactly as in the proof for ordinary harmonic numbers:

- For $j \ge 0$, let the j-th block be the indices

  $$B_j = \{2^j + 1,\; 2^j + 2,\; \dots,\; \min(2^{j+1},\, n)\}.$$

  Every $k \in B_j$ satisfies $k > 2^j$, hence

  $$\frac{1}{k^r} \;<\; \frac{1}{(2^j)^r}\,.$$

- The block B_j contains at most 2^j terms, therefore the contribution of B_j to the sum is bounded by

  $$\sum_{k \in B_j} \frac{1}{k^r} \;\le\; 2^j \cdot \frac{1}{(2^j)^r} \;=\; 2^{j(1-r)}.$$

Now split off the $k = 1$ term and sum these bounds over all blocks up to $\lfloor \log_2 n \rfloor$:

$$H_n^{(r)} = \frac{1}{1^r} + \sum_{j=0}^{\lfloor \log_2 n \rfloor} \sum_{k \in B_j} \frac{1}{k^r} \;\le\; 1 + \sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^{j(1-r)} \;\le\; 1 + \sum_{j=0}^{\infty} 2^{j(1-r)}.$$

Because $r > 1$, the ratio $q = 2^{1-r}$ satisfies $0 < q < 1$, so the geometric series converges:

$$\sum_{j=0}^{\infty} 2^{j(1-r)} = \frac{1}{1 - 2^{1-r}} = \frac{2^{r-1}}{2^{r-1} - 1}.$$

Therefore, for every $n \ge 1$,

$$\boxed{H_n^{(r)} \;\le\; 1 + \frac{1}{1 - 2^{1-r}} \;=\; 1 + \frac{2^{r-1}}{2^{r-1} - 1}}$$

which is a constant depending only on r (hence the sequence of partial sums is bounded).

Examples:

- $r = 2:\; H_n^{(2)} \le 1 + \frac{1}{1-\frac{1}{2}} = 3.$
- $r = 3:\; H_n^{(3)} \le 1 + \frac{1}{1-\frac{1}{4}} = \frac{7}{3}.$

4 [10]

Preliminaries

- Harmonic numbers are defined by $H_n = \sum_{k=1}^{n} \frac{1}{k}$ (with $H_0 = 0$).
- Euler's constant is $\gamma = \lim_{n\to\infty}(H_n - \ln n)$ (so $H_n = \ln n + \gamma + o(1)$).

Two standard integral bounds

Because $x \mapsto 1/x$ is decreasing on $[1, \infty)$,

$$\int_k^{k+1} \frac{dx}{x} \;\le\; \frac{1}{k} \;\le\; \int_{k-1}^{k} \frac{dx}{x} \qquad (k \ge 1).$$

Summing:

$$\boxed{\ln(n+1) \le H_n \le 1 + \ln n} \qquad (n \ge 1).$$

From these alone we already see $H_n > \ln n$. (We'll use these bounds in parts (a)–(b).)

Decisions

(a) $H_n < \ln n$ — False.

Take $n = 1$: $H_1 = 1 > \ln 1 = 0$. In fact, the lower bound above shows $H_n \geq \ln(n+1) > \ln n$ for every $n \geq 1$, so the claimed inequality never holds.

(b) $H_n > \ln n$ — True.

From the same bound,

$$H_n \;\geq\; \ln(n+1) \;>\; \ln n \qquad (n \geq 1).$$

(c) $H_n > \ln n + \gamma$ — True.

Let $\gamma_n := H_n - \ln n$. Then

$$\gamma_{n+1} - \gamma_n = \Big(H_n + \frac{1}{n+1} - \ln(n+1)\Big) - (H_n - \ln n) = \frac{1}{n+1} - \ln\Big(1 + \frac{1}{n}\Big).$$

Since for $x > 0$ the function $f(x) = \ln(1+x) - \frac{x}{1+x}$ satisfies $f'(x) = \frac{x}{(1+x)^2} > 0$ and $f(0) = 0$, we have

$$\ln(1+x) > \frac{x}{1+x} \quad (x > 0).$$

With $x = 1/n$ this gives $\ln(1 + \frac{1}{n}) > \frac{1}{n+1}$, hence $\gamma_{n+1} - \gamma_n < 0$. So $\{\gamma_n\}$ is strictly decreasing. Because $\gamma = \lim_{n\to\infty} \gamma_n$ exists, every term lies above the limit:

$$\gamma_n > \gamma \qquad (n \geq 1).$$

Equivalently, $H_n > \ln n + \gamma$ for all n. (This is also consistent with the asymptotic expansion $H_n = \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + \frac{1}{120n^4} - \varepsilon$ with $0 < \varepsilon < \frac{1}{252n^6}$, which shows the excess over $\ln n + \gamma$ is positive.)

5 [15]

Knuth gives the Euler–Maclaurin expansion for H_n:

$$H_n = \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + \frac{1}{120n^4} - \varepsilon, \qquad 0 < \varepsilon < \frac{1}{252\,n^6}.$$

This makes high-precision evaluation straightforward for large n.

Computation for $n = 10{,}000$

Use constants from Appendix A (or any high-precision source):

- $\ln 10000 = 4 \ln 10 = 9.210340371976182736\ldots$
- $\gamma = 0.577215664901532860\ldots$

Now add the correction terms:

$$\begin{aligned} H_{10000} &= \underbrace{(\ln 10000 + \gamma)}_{9.787556036877715596\ldots} + \underbrace{\frac{1}{2n}}_{0.000050000000000000} - \underbrace{\frac{1}{12n^2}}_{0.000000000833333333\ldots} + \underbrace{\frac{1}{120n^4}}_{0.00000000000000000008\ldots} \\ &= 9.787606036044382264178477 9\ldots \end{aligned}$$

The neglected remainder is $< \dfrac{1}{252 \cdot 10^{24}} \approx 4.0 \times 10^{-27}$, far below the 10^{-15} level, so rounding at 15 decimals is safe.

6 [M15]

Proof A (via the Stirling-number recurrence; "algebraic/inductive")

Facts we'll use (from §1.2.6):

1. The unsigned Stirling numbers of the first kind convert falling powers to ordinary powers:

$$(x)_m = \sum_{k=0}^{m} (-1)^{m-k} \left[{m \atop k}\right] x^k.$$

From $(x)_{n+1} = (x-n)(x)_n$, comparing coefficients yields the standard recurrence

$$\left[{n+1 \atop k}\right] = n \left[{n \atop k}\right] + \left[{n \atop k-1}\right]. \qquad (\ )$$

Also $\left[{n \atop 1}\right] = (n-1)!$.

Define $a_n := \frac{\left[{n+1 \atop 2}\right]}{n!}$. Apply ( ) with $k = 2$:

$$\left[{n+1 \atop 2}\right] = n\left[{n \atop 2}\right] + \left[{n \atop 1}\right] = n\left[{n \atop 2}\right] + (n-1)!.$$

Divide by $n!$:

$$a_n = \frac{\left[{n \atop 2}\right]}{(n-1)!} + \frac{1}{n} = a_{n-1} + \frac{1}{n}.$$

The initial value is $a_1 = \frac{\left[{2 \atop 2}\right]}{1!} = 1$. Hence $a_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n} = H_n$ by induction, proving the claim.

Proof B (combinatorial; "count permutations with two cycles")

A classical interpretation: $\left[{m \atop k}\right]$ counts permutations of m elements with exactly k cycles in the cycle decomposition. Thus $\left[{n+1 \atop 2}\right]$ counts permutations of $\{1, \ldots, n+1\}$ having two cycles.

Count such permutations by the length ℓ (with $1 \le \ell \le n$) of the cycle that contains $n+1$:

- Choose the other $\ell - 1$ members of that cycle: $\binom{n}{\ell-1}$ ways.
- Arrange those ℓ elements in a cycle with $n+1$ present: $(\ell-1)!$ ways.
- Arrange the remaining $n+1-\ell$ elements into one cycle: $(n-\ell)!$ ways.

Total:

$$\sum_{\ell=1}^{n} \binom{n}{\ell-1} (\ell-1)!(n-\ell)! = \sum_{j=0}^{n-1} \binom{n}{j} j!(n-j-1)! = \sum_{j=0}^{n-1} \frac{n!}{n-j} = n! \sum_{k=1}^{n} \frac{1}{k} = n!\, H_n.$$

Hence $\left[{n+1 \atop 2}\right] = n!\, H_n$, i.e. $H_n = \left[{n+1 \atop 2}\right]/n!$.

7 [M21]

(a) Monotonicity in each argument

Fix n and compare $T(m+1, n)$ to $T(m, n)$:

$$\begin{aligned} T(m+1,n) - T(m,n) &= (H_{m+1} - H_m) - (H_{(m+1)n} - H_{mn}) \\ &= \frac{1}{m+1} - \sum_{k=1}^{n} \frac{1}{mn+k}. \end{aligned}$$

Since the n denominators $mn+1, \dots, mn+n$ are at most $(m+1)n$, we have

$$\sum_{k=1}^{n} \frac{1}{mn+k} \;\geq\; n \cdot \frac{1}{(m+1)n} \;=\; \frac{1}{m+1}.$$

Hence $T(m+1,n) - T(m,n) \leq 0$. Equality holds iff $n=1$; otherwise the sum is strictly larger than $\frac{1}{m+1}$, so the difference is strictly negative. By symmetry, the same argument (swapping m and n) shows $T(m,n)$ is also nonincreasing in n (strictly unless $m=1$).

Conclusion. T is nonincreasing in each variable (strictly unless the other variable equals 1).

(b) Extremal values for $m, n > 0$

Maximum. For any $n \geq 1$,

$$T(1,n) = H_1 + H_n - H_{1\cdot n} = 1 + H_n - H_n = 1,$$

and by part (a), increasing either argument cannot increase T. Thus the maximum value is 1, attained whenever $m=1$ or $n=1$ (in particular at $(1,1)$).

Minimum / infimum. Using the asymptotic expansion from the section,

$$H_m = \ln m + \gamma + R_m, \quad H_n = \ln n + \gamma + R_n, \quad H_{mn} = \ln(mn) + \gamma + R_{mn},$$

so

$$T(m,n) = \gamma + (R_m + R_n - R_{mn}).$$

From the displayed expansion (with positive leading term $\frac{1}{2k}$), $R_k > 0$ and $R_k \to 0$ as $k \to \infty$; more precisely,

$$R_k = \frac{1}{2k} - \frac{1}{12k^2} + O\Big(\frac{1}{k^4}\Big).$$

Therefore for finite m, n,

$$T(m,n) \;=\; \gamma + \Big(\frac{1}{2m} + \frac{1}{2n} - \frac{1}{2mn}\Big) + O\Big(\frac{1}{m^2} + \frac{1}{n^2}\Big) \;>\; \gamma,$$

and letting $m, n \to \infty$ gives $T(m,n) \downarrow \gamma$.

- There is no minimum over positive integers (the value γ is not attained by any finite m, n).
- The infimum is $\gamma \approx 0.57721$, and $T(m,n) \in (\gamma, 1]$ for all integers $m, n \geq 1$.

8 [HM18]

Estimate, as a function of n, the difference

$$\Delta_n \;=\; \sum_{k=1}^{n} H_k \;-\; \sum_{k=1}^{n} \ln k \;=\; \sum_{k=1}^{n} H_k \;-\; \ln(n!).$$

Step 1: Reduce with Eq. (8) Section 1.2.7 establishes

$$\sum_{k=1}^{n} H_k \;=\; (n+1)H_n - n.$$

Hence

$$\Delta_n \;=\; (n+1)H_n \;-\; n \;-\; \ln(n!). \qquad (\;)$$

Step 2: Use standard asymptotics for H_n and $\ln(n!)$

From 1.2.7 (derived via Euler's summation formula in 1.2.11.2), for fixed m we have

$$H_n \;=\; \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + O\Big(\frac{1}{n^4}\Big).$$

(Here γ is Euler's constant.)

Stirling's expansion (also obtained in 1.2.11.2) gives

$$\ln(n!) \;=\; \Big(n + \tfrac{1}{2}\Big)\ln n \;-\; n \;+\; \tfrac{1}{2}\ln(2\pi) \;+\; \frac{1}{12n} \;-\; \frac{1}{360n^3} \;+\; O\Big(\frac{1}{n^5}\Big).$$

(Using $e^{\sigma} = \sqrt{2\pi}$ for Stirling's constant.)

Step 3: Combine in ( )

Insert the two expansions into ( ) and simplify:

$$\begin{aligned}
\Delta_n &= (n+1)\Big(\ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + O\Big(\frac{1}{n^4}\Big)\Big) \;-\; n \\
&\quad - \Big(\Big(n + \tfrac{1}{2}\Big)\ln n - n + \tfrac{1}{2}\ln(2\pi) + \frac{1}{12n} - \frac{1}{360n^3} + O\Big(\frac{1}{n^5}\Big)\Big) \\
&= \underbrace{\tfrac{1}{2}\ln n}_{\text{from logs}} \;+\; \underbrace{(n+1)\gamma}_{\text{linear}} \;-\; \tfrac{1}{2}\ln(2\pi) \;+\; \underbrace{\Big(\tfrac{1}{2} + \frac{1}{3n}\Big)}_{\text{from small terms}} \;+ O\Big(\frac{1}{n^2}\Big).
\end{aligned}$$

Thus a convenient and accurate estimate is

$$\boxed{\Delta_n \;=\; n\gamma \;+\; \tfrac{1}{2}\ln n \;+\; \Big(\gamma + \tfrac{1}{2} - \tfrac{1}{2}\ln(2\pi)\Big) \;+\; \frac{1}{3n} \;+\; O\Big(\frac{1}{n^2}\Big)}$$

so the leading growth is linear with slope γ, with a secondary $\frac{1}{2}\ln n$ term.

Numerically, the constant

$$C \;=\; \gamma + \tfrac{1}{2} - \tfrac{1}{2}\ln(2\pi) \;\approx\; 0.158277132\ldots$$

Hence, for practical purposes,

$$\Delta_n \;\approx\; n\gamma + \tfrac{1}{2}\ln n + 0.158277132 + \frac{1}{3n}.$$

Takeaway

Comparing Eq. (8) with $\sum_{k=1}^{n} \ln k$ shows

$$\sum_{k=1}^{n} H_k - \sum_{k=1}^{n} \ln k \;=\; n\gamma + \tfrac{1}{2}\ln n + \Theta(1),$$

with the constant and first correction explicitly given above.

9 [M18]

Theorem A in §1.2.7 analyzes the sum $\sum_{k=1}^{n} \binom{n}{k} x^k H_k$ for $x > 0$. The exercise asks for the value when $x = -1$; i.e.,

$$S_n \;=\; \sum_{k=1}^{n} \binom{n}{k} (-1)^k H_k \;=\; ?$$

Solution 1 (double–sum "H_k expansion")

Write $H_k = \sum_{j=1}^{k} \frac{1}{j}$, then interchange the order of summation:

$$S_n = \sum_{k=1}^{n} \binom{n}{k} (-1)^k \sum_{j=1}^{k} \frac{1}{j} = \sum_{j=1}^{n} \frac{1}{j} \sum_{k=j}^{n} \binom{n}{k} (-1)^k.$$

Use the standard partial alternating–binomial identity

$$\sum_{k=j}^{n}(-1)^k\binom{n}{k} = (-1)^j\binom{n-1}{j-1},$$

(which follows, e.g., from $\sum_{k=0}^{m}(-1)^k\binom{n}{k} = (-1)^m\binom{n-1}{m}$ by induction or Pascal's rule). Hence

$$S_n = \sum_{j=1}^{n}\frac{(-1)^j}{j}\binom{n-1}{j-1}.$$

Now use $\binom{n}{j} = \frac{n}{j}\binom{n-1}{j-1}$, giving

$$S_n = \frac{1}{n}\sum_{j=1}^{n}(-1)^j\binom{n}{j}.$$

Finally, $\sum_{j=0}^{n}(-1)^j\binom{n}{j} = (1-1)^n = 0$, so $\sum_{j=1}^{n}(-1)^j\binom{n}{j} = -1$. Therefore

$$\boxed{S_n = -\frac{1}{n}}.$$

Solution 2 (integral representation of H_k)

Use $H_k = \displaystyle\int_0^1 \frac{1-t^k}{1-t}\,dt$ (from integrating $\sum_{i=0}^{k-1} t^i = \frac{1-t^k}{1-t}$ termwise). Then

$$S_n = \int_0^1 \frac{1}{1-t}\sum_{k=1}^{n}\binom{n}{k}(-1)^k(1-t^k)\,dt.$$

But $\sum_{k=1}^{n}\binom{n}{k}(-1)^k = -1$ and $\sum_{k=1}^{n}\binom{n}{k}(-t)^k = (1-t)^n - 1$. Hence the bracket equals

$$-1 - \big((1-t)^n - 1\big) = -(1-t)^n,$$

so

$$S_n = -\int_0^1 (1-t)^{n-1}\,dt = -\frac{1}{n}.$$

Thus, although Theorem A is stated only for $x > 0$, the alternating case at $x = -1$ has the exact closed form

$$\boxed{\sum_{k=1}^{n} \binom{n}{k} (-1)^k H_k = -\frac{1}{n}}.$$

(Checks: $n = 1, 2, 3$ give $-1, -\frac{1}{2}, -\frac{1}{3}$, respectively.)

10 [M20]

Proof (telescoping via a discrete product rule)

Start from the discrete analogue of the product rule, obtained by adding and subtracting $a_{k+1}b_k$:

$$a_{k+1}b_{k+1} - a_k b_k = a_{k+1}(b_{k+1} - b_k) + (a_{k+1} - a_k)b_k.$$

Now sum this equality for $k = 1, 2, \dots, n-1$. The left-hand side telescopes:

$$\sum_{k=1}^{n-1} (a_{k+1}b_{k+1} - a_k b_k) = a_n b_n - a_1 b_1.$$

Hence

$$a_n b_n - a_1 b_1 = \sum_{k=1}^{n-1} a_{k+1}(b_{k+1} - b_k) + \sum_{k=1}^{n-1} (a_{k+1} - a_k)b_k.$$

Rearrange to isolate the desired sum:

$$\sum_{k=1}^{n-1} (a_{k+1} - a_k)b_k = a_n b_n - a_1 b_1 - \sum_{k=1}^{n-1} a_{k+1}(b_{k+1} - b_k).$$

This is exactly the required identity.

Remarks and equivalent forms

1. Abel's summation (discrete integration by parts). If $A_k = \sum_{j=1}^{k} a_j$ and we set $a_{k+1} - a_k := a_{k+1}$, the identity becomes the well-known Abel transform:

$$\sum_{k=1}^{n} a_k\, b_k = A_n b_n - \sum_{k=1}^{n-1} A_k\, (b_{k+1} - b_k).$$

2. Symmetric variant. By shifting indices one also gets

$$\sum_{k=1}^{n} a_k\,(b_k - b_{k-1}) = a_n b_n - \sum_{k=1}^{n-1} b_k\,(a_{k+1} - a_k),$$

with the convention b_0 as needed.

3. Quick sanity check. Let $a_k = k$, $b_k = k$. Then the left side is $\sum_{k=1}^{n-1} k = n(n-1)/2$. The right side evaluates to $n^2 - 1 - \sum_{k=1}^{n-1}(k+1) = n(n-1)/2$ as well.

These forms are what §1.2.7 uses implicitly when deriving identities like Eq. (9) and Theorem A's estimate for $\sum \binom{n}{k} x^k H_k$.

11 [M21]

Solution (summation by parts)

Use the discrete summation-by-parts identity (proved in 1.2.7–10):

$$\sum_{k=2}^{n} a_k\,(b_k - b_{k-1}) \;=\; a_n b_n - a_1 b_1 \;-\; \sum_{k=1}^{n-1} b_k\,(a_{k+1} - a_k).$$

Take $a_k = H_k$ and choose $b_k = -\frac{1}{k}$. Then

$$b_k - b_{k-1} = -\frac{1}{k} + \frac{1}{k-1} = \frac{1}{k(k-1)},$$

so the left side is exactly S_n. Also,

$$a_{k+1} - a_k = H_{k+1} - H_k = \frac{1}{k+1}.$$

Plugging into the identity gives

$$S_n = H_n\left(-\frac{1}{n}\right) - H_1(-1) - \sum_{k=1}^{n-1}\left(-\frac{1}{k}\right)\left(\frac{1}{k+1}\right) = 1 - \frac{H_n}{n} + \sum_{k=1}^{n-1}\frac{1}{k(k+1)}.$$

The remaining sum telescopes:

$$\sum_{k=1}^{n-1}\frac{1}{k(k+1)} = \sum_{k=1}^{n-1}\left(\frac{1}{k} - \frac{1}{k+1}\right) = 1 - \frac{1}{n}.$$

Therefore

$$\boxed{S_n = 2 - \frac{H_n + 1}{n}}.$$

Quick check

For $n = 2$: $S_2 = \frac{H_2}{2} = \frac{3/2}{2} = 3/4$. Formula gives $2 - \frac{H_2+1}{2} = 2 - \frac{2.5}{2} = 0.75$.

12 [M10]

For $r > 1$,

$$H_\infty^{(r)} = \sum_{k\geq 1} \frac{1}{k^r} = \zeta(r),$$

Riemann's zeta function. In particular, for even integers there's the closed form $\zeta(2m) = \dfrac{|B_{2m}|(2\pi)^{2m}}{2(2m)!}$, though we won't need to compute it here.

We only need 100 correct decimals. Since

$$\zeta(1000) = 1 + \sum_{k=2}^{\infty} \frac{1}{k^{1000}},$$

it suffices to show the tail $\sum_{k=2}^{\infty} k^{-1000}$ is $< 10^{-100}$. Then $\zeta(1000)$ rounds to 1. $\underbrace{00\ldots0}_{100 \text{ zeros}}$.

A tight and elementary bound:

$$\sum_{k=2}^{\infty} \frac{1}{k^{1000}} \;<\; \frac{1}{2^{1000}} \;+\; \int_2^{\infty} x^{-1000}\,dx \;=\; \frac{1}{2^{1000}} + \frac{1}{999 \cdot 2^{999}} = \frac{1}{2^{1000}}\left(1 + \frac{2}{999}\right).$$

Now $2^{1000} = 10^{1000 \log_{10} 2}$ with $\log_{10} 2 \approx 0.30103$, hence

$$\frac{1}{2^{1000}} < 10^{-301}.$$

Therefore

$$0 < \zeta(1000) - 1 < \left(1 + \frac{2}{999}\right) 10^{-301} < 10^{-300} \ll 10^{-100}.$$

So the first 100 digits after the decimal point of $\zeta(1000)$ are all zero.

(Indeed, much more is true: $\zeta(1000) = 1 + \Theta(2^{-1000})$, so the first nonzero decimal digit doesn't appear until roughly the 302nd place.)

13 [M22]

Here is a clean, self-contained proof of Exercise 13 from §1.2.7 (Harmonic Numbers). I restate the problem in my own words for clarity:

Claim. For every integer $n \geq 1$ and indeterminate x,

$$\sum_{k=1}^{n} \frac{x^k}{k} \;=\; H_n \;+\; \sum_{k=1}^{n} \binom{n}{k} \frac{(x-1)^k}{k},$$

where $H_n = \sum_{k=1}^{n} \frac{1}{k}$ is the n-th harmonic number.

Proof

1. Split off the constant part.

For any x,

$$\sum_{k=1}^{n} \frac{x^k}{k} = \sum_{k=1}^{n} \frac{1}{k} + \sum_{k=1}^{n} \frac{x^k - 1}{k} = H_n + \sum_{k=1}^{n} \frac{x^k - 1}{k}.$$

2. Binomially expand $x^k - 1$.

Write $x = (x-1) + 1$. Then

$$x^k - 1 = \big((x-1)+1\big)^k - 1 = \sum_{j=0}^{k} \binom{k}{j} (x-1)^j - 1 = \sum_{j=1}^{k} \binom{k}{j} (x-1)^j.$$

Hence

$$\sum_{k=1}^{n} \frac{x^k - 1}{k} = \sum_{k=1}^{n} \sum_{j=1}^{k} \binom{k}{j} \frac{(x-1)^j}{k} = \sum_{j=1}^{n} (x-1)^j \sum_{k=j}^{n} \frac{1}{k} \binom{k}{j}.$$

3. Evaluate the inner sum.

Use the simple identity

$$\frac{1}{k}\binom{k}{j} = \frac{1}{j}\binom{k-1}{j-1} \qquad (k \geq j \geq 1),$$

which follows from $\binom{k}{j} = \frac{k}{j}\binom{k-1}{j-1}$. Therefore

$$\sum_{k=j}^{n} \frac{1}{k}\binom{k}{j} = \frac{1}{j}\sum_{k=j}^{n}\binom{k-1}{j-1} = \frac{1}{j}\sum_{m=j-1}^{n-1}\binom{m}{j-1} = \frac{1}{j}\binom{n}{j},$$

by the "hockey-stick" identity $\sum_{m=r}^{N}\binom{m}{r} = \binom{N+1}{r+1}$.

4. Assemble the result.

Plugging this into step 2 gives

$$\sum_{k=1}^{n}\frac{x^k-1}{k} = \sum_{j=1}^{n}(x-1)^j \cdot \frac{1}{j}\binom{n}{j} = \sum_{j=1}^{n}\binom{n}{j}\frac{(x-1)^j}{j}.$$

Together with step 1, this yields exactly

$$\sum_{k=1}^{n}\frac{x^k}{k} = H_n + \sum_{k=1}^{n}\binom{n}{k}\frac{(x-1)^k}{k},$$

as was to be shown.

14 [M22]

Part (a) $\displaystyle\sum_{k=1}^{n}\frac{H_k}{k} = \frac{1}{2}(H_n^2 + H_n^{(2)})$

Write the left–hand side as a double sum and use symmetry:

$$\sum_{k=1}^{n}\frac{H_k}{k} = \sum_{k=1}^{n}\sum_{i=1}^{k}\frac{1}{ik} = \sum_{1\leq i\leq k\leq n}\frac{1}{ik}.$$

Split the pairs (i,k) into $i<k$ and $i=k$:

$$\sum_{1\le i\le k\le n}\frac{1}{ik} = \underbrace{\sum_{1\le i<k\le n}\frac{1}{ik}}_{\text{strictly off–diagonal}} + \underbrace{\sum_{k=1}^{n}\frac{1}{k^2}}_{H_n^{(2)}}.$$

On the other hand,

$$H_n^2 = \Big(\sum_{i=1}^{n}\frac{1}{i}\Big)\Big(\sum_{k=1}^{n}\frac{1}{k}\Big) = \sum_{i=1}^{n}\frac{1}{i^2} + 2\sum_{1\le i<k\le n}\frac{1}{ik} = H_n^{(2)} + 2\sum_{1\le i<k\le n}\frac{1}{ik}.$$

Hence $\displaystyle\sum_{1\le i<k\le n}\frac{1}{ik} = \frac{H_n^2 - H_n^{(2)}}{2}$, and substituting above gives

$$\sum_{k=1}^{n}\frac{H_k}{k} = \frac{H_n^2 - H_n^{(2)}}{2} + H_n^{(2)} = \frac{H_n^2 + H_n^{(2)}}{2}.$$

This proves the identity.

(Equivalently, you can also start from $\sum_{k=1}^{n}\sum_{i=1}^{k}\frac{1}{ik} = \sum_{i=1}^{n}\frac{1}{i}\sum_{k=i}^{n}\frac{1}{k} = \sum_{i=1}^{n}\frac{1}{i}(H_n - H_{i-1}) = H_n^2 - \sum_{i=1}^{n}\frac{H_{i-1}}{i}$, then note $\sum_{i=1}^{n}\frac{H_{i-1}}{i} = \sum_{i=1}^{n}\frac{H_i}{i} - \sum_{i=1}^{n}\frac{1}{i^2} = \sum_{k=1}^{n}\frac{H_k}{k} - H_n^{(2)}$ and solve for $\sum\frac{H_k}{k}$.)

Part (b) $\displaystyle\sum_{k=1}^{n}\frac{H_k}{k+1}$

Use the elementary shift $H_k = H_{k+1} - \frac{1}{k+1}$:

$$\sum_{k=1}^{n}\frac{H_k}{k+1} = \sum_{k=1}^{n}\frac{H_{k+1}}{k+1} - \sum_{k=1}^{n}\frac{1}{(k+1)^2} = \Big(\sum_{j=2}^{n+1}\frac{H_j}{j}\Big) - \Big(\sum_{j=2}^{n+1}\frac{1}{j^2}\Big).$$

Hence

$$\sum_{k=1}^{n}\frac{H_k}{k+1} = \underbrace{\sum_{j=1}^{n+1}\frac{H_j}{j}}_{=\frac{1}{2}(H_{n+1}^2 + H_{n+1}^{(2)})} - \frac{H_1}{1} - (H_{n+1}^{(2)} - 1) = \frac{1}{2}(H_{n+1}^2 - H_{n+1}^{(2)}).$$

(Used the result of part (a) with $n \mapsto n+1$.)

An equivalent form in terms of H_n is also handy:

$$\sum_{k=1}^{n}\frac{H_k}{k+1} = \frac{1}{2}\left(H_n^2 - H_n^{(2)}\right) + \frac{H_n}{n+1},$$

since $H_{n+1} = H_n + \frac{1}{n+1}$ and $H_{n+1}^{(2)} = H_n^{(2)} + \frac{1}{(n+1)^2}$.

15 [M23]

Recall $H_k = \sum_{j=1}^{k} \frac{1}{j}$. Start from the double–sum expansion:

$$\sum_{k=1}^{n} H_k^2 = \sum_{k=1}^{n} \Big(\sum_{i=1}^{k} \tfrac{1}{i}\Big)\Big(\sum_{j=1}^{k} \tfrac{1}{j}\Big) = \sum_{i=1}^{n}\sum_{j=1}^{n} \frac{\#\{k : 1 \le k \le n,\ k \ge \max(i,j)\}}{ij} = \sum_{i=1}^{n}\sum_{j=1}^{n} \frac{n - \max(i,j) + 1}{ij}.$$

Split the square (i,j) into diagonal and off–diagonal parts:

$$\sum_{k=1}^{n} H_k^2 = 2 \sum_{1 \le j < i \le n} \frac{n-i+1}{ij} + \sum_{i=1}^{n} \frac{n-i+1}{i^2} = 2\sum_{i=1}^{n} \frac{n-i+1}{i} H_{i-1} + \sum_{i=1}^{n} \frac{n-i+1}{i^2}.$$

Use $H_{i-1} = H_i - \frac{1}{i}$:

$$\sum_{k=1}^{n} H_k^2 = 2\sum_{i=1}^{n} \frac{n-i+1}{i} H_i - 2\sum_{i=1}^{n} \frac{n-i+1}{i^2} + \sum_{i=1}^{n} \frac{n-i+1}{i^2}.$$

Hence

$$\boxed{\sum_{k=1}^{n} H_k^2 = 2\sum_{i=1}^{n} \frac{n-i+1}{i} H_i - \sum_{i=1}^{n} \frac{n-i+1}{i^2}} \qquad ( )$$

Now evaluate the two sums in ().

1. For the first,

$$\sum_{i=1}^{n} \frac{n-i+1}{i} H_i = (n+1)\sum_{i=1}^{n} \frac{H_i}{i} - \sum_{i=1}^{n} H_i.$$

We'll use two standard identities (proved here for completeness):

- $\sum_{i=1}^{n} H_i = (n+1)H_n - n$. Proof: $\sum_{i=1}^{n} H_i = \sum_{i=1}^{n}\sum_{j=1}^{i} \frac{1}{j} = \sum_{j=1}^{n} \frac{n-j+1}{j} = (n+1)H_n - \sum_{j=1}^{n} 1 = (n+1)H_n - n$.

- $\displaystyle\sum_{i=1}^{n} \frac{H_i}{i} = \frac{1}{2}\Big(H_n^2 + H_n^{(2)}\Big)$, where $H_n^{(2)} = \sum_{i=1}^{n} \frac{1}{i^2}$. Proof: From $H_i = H_{i-1} + \frac{1}{i}$,

$$H_n^2 - H_{n-1}^2 = (H_n - H_{n-1})(H_n + H_{n-1}) = \frac{2H_{n-1}}{n} + \frac{1}{n^2}.$$

  Summing $n = 1 \dots N$ gives $H_N^2 = 2\displaystyle\sum_{i=1}^{N} \frac{H_{i-1}}{i} + \sum_{i=1}^{N} \frac{1}{i^2} = 2\sum_{i=1}^{N} \frac{H_i}{i} - \sum_{i=1}^{N} \frac{1}{i^2}$, hence the claim.

Therefore

$$\sum_{i=1}^{n} \frac{n-i+1}{i} H_i = (n+1) \cdot \tfrac{1}{2}\Big(H_n^2 + H_n^{(2)}\Big) - \big((n+1)H_n - n\big).$$

2. For the second,

$$\sum_{i=1}^{n} \frac{n-i+1}{i^2} = (n+1)\sum_{i=1}^{n} \frac{1}{i^2} - \sum_{i=1}^{n} \frac{1}{i} = (n+1)H_n^{(2)} - H_n.$$

Plug these into ():

$$\begin{aligned}
\sum_{k=1}^{n} H_k^2 &= 2\Big((n+1) \cdot \tfrac{1}{2}(H_n^2 + H_n^{(2)}) - (n+1)H_n + n\Big) - \big((n+1)H_n^{(2)} - H_n\big) \\
&= (n+1)H_n^2 - 2(n+1)H_n + 2n - (n+1)H_n^{(2)} + H_n + (n+1)H_n^{(2)} \\
&= \boxed{(n+1)H_n^2 - (2n+1)H_n + 2n.}
\end{aligned}$$

Quick check

- $n = 1$: LHS $= H_1^2 = 1$. RHS $= (2) \cdot 1 - (3) \cdot 1 + 2 = 1$.
- $n = 2$: LHS $= 1^2 + (1 + \frac{1}{2})^2 = 1 + 2.25 = 3.25$. RHS $= 3 \cdot 2.25 - 5 \cdot 1.5 + 4 = 3.25$.

Thus,

$$\boxed{\sum_{k=1}^{n} H_k^2 = (n+1)H_n^2 - (2n+1)H_n + 2n.}$$

16 [18]

1. Split H_{2n} into even and odd parts.

$$H_{2n} = \sum_{k=1}^{2n} \frac{1}{k} = \sum_{j=1}^{n} \left(\frac{1}{2j-1} + \frac{1}{2j} \right).$$

2. Recognize the even part.

$$\sum_{j=1}^{n} \frac{1}{2j} = \frac{1}{2} \sum_{j=1}^{n} \frac{1}{j} = \frac{1}{2} H_n.$$

3. Solve for the odd part. Let $S_n = \sum_{j=1}^{n} \frac{1}{2j-1}$. Then

$$H_{2n} = S_n + \frac{1}{2} H_n \quad \Longrightarrow \quad S_n = H_{2n} - \frac{1}{2} H_n.$$

That's the required expression "in terms of harmonic numbers."

Quick checks

- $n = 1$: $1 = H_2 - \frac{1}{2} H_1 = (1 + \frac{1}{2}) - \frac{1}{2} = 1$.
- $n = 2$: $1 + \frac{1}{3} = \frac{4}{3} = H_4 - \frac{1}{2} H_2 = (1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4}) - \frac{1}{2}(1 + \frac{1}{2}) = \frac{25}{12} - \frac{3}{4} = \frac{16}{12} = \frac{4}{3}$.

(Optional) Asymptotics

Using $H_m = \ln m + \gamma + \frac{1}{2m} + O(m^{-2})$,

$$\sum_{j=1}^{n} \frac{1}{2j-1} = H_{2n} - \frac{1}{2} H_n = \ln 2 + \frac{1}{2} \ln n + \frac{\gamma}{2} + O\left(\frac{1}{n}\right),$$

so the odd–reciprocal sum grows like $\frac{1}{2} \ln n$ plus a constant.

Answer. $\displaystyle\sum_{j=1}^{n} \frac{1}{2j-1} = H_{2n} - \frac{1}{2} H_n.$

17 [M24]

Solution 1 (pairing k with $p-k$)

Pair the terms k and $p-k$ for $k = 1, 2, \dots, (p-1)/2$. Then

$$\frac{1}{k} + \frac{1}{p-k} \;=\; \frac{p}{k(p-k)}.$$

Therefore

$$H_{p-1} = \sum_{k=1}^{(p-1)/2} \left(\frac{1}{k} + \frac{1}{p-k} \right) = \sum_{k=1}^{(p-1)/2} \frac{p}{k(p-k)}.$$

Each denominator $k(p-k)$ is not divisible by p (since $1 \le k \le p-1$). Let

$$D \;:=\; \operatorname{lcm}\{\, k(p-k) : 1 \le k \le (p-1)/2 \,\}.$$

Then $p \nmid D$. Writing the sum over the common denominator D, we get

$$H_{p-1} \;=\; \frac{p}{D} \sum_{k=1}^{(p-1)/2} \frac{D}{k(p-k)} \;=\; \frac{p \cdot N}{D},$$

for some integer N. Since $p \nmid D$, the fraction $\dfrac{pN}{D}$ (and hence its reduction to lowest terms) has numerator divisible by p. This proves the claim.

Solution 2 (field-theoretic view, same conclusion)

Work modulo p. In the field $\mathbb{F}_p$, every $k \in \{1, \dots, p-1\}$ is invertible, so

$$S \;:=\; \sum_{k=1}^{p-1} k^{-1} \in \mathbb{F}_p$$

is well-defined. The map $k \mapsto k^{-1}$ permutes $\{1, \dots, p-1\}$, hence

$$S \equiv \sum_{k=1}^{p-1} k \equiv 0 \pmod{p}.$$

Interpreting $H_{p-1} = A/B$ with $p \nmid B$ (justified by the pairing argument above), the congruence $S \equiv 0$ means $A/B \equiv 0 \pmod{p}$, so $A \equiv 0 \pmod{p}$. Thus $p \mid A$.

Checks & remarks

- Examples: $p = 5$: $H_4 = \frac{25}{12}$ (numerator 25 divisible by 5). $p = 7$: $H_6 = \frac{49}{20}$ (numerator 49 divisible by 7).
- Stronger fact (context): for $p \geq 5$, Wolstenholme's theorem says $H_{p-1} \equiv 0 \pmod{p^2}$, i.e., the numerator is even divisible by p^2. (Exercise 17 only asks for divisibility by p.)

18 [M33]

Let

$$S_n = \sum_{k=1}^{n} \frac{1}{2k-1} = 1 + \frac{1}{3} + \frac{1}{5} + \cdots + \frac{1}{2n-1}$$

and write $S_n = \dfrac{A_n}{B_n}$ in lowest terms. Since every denominator $2k-1$ is odd, also B_n is odd. The problem asks for the highest power of 2 dividing the numerator A_n.

Answer

$$\nu_2(A_n) = \begin{cases} 0, & n \text{ odd;} \\ 2\,\nu_2(n), & n \text{ even,} \end{cases}$$

where $\nu_2(x)$ is the exponent of 2 in x (the 2-adic valuation).

We work with the common odd denominator

$$D_n = \operatorname{lcm}\{1, 3, 5, \ldots, 2n-1\} \quad (\text{so } D_n \text{ is odd}),$$

and set

$$N_n = \sum_{k=1}^{n} \frac{D_n}{2k-1} \in \mathbb{Z}, \qquad S_n = \frac{N_n}{D_n}.$$

Since $B_n \mid D_n$ and B_n is odd, $\nu_2(A_n) = \nu_2(N_n)$.

1) The case n odd.

Each summand $D_n/(2k-1)$ is an odd integer (odd divided by odd), hence N_n is a sum of n odd integers. If n is odd, the sum is odd; thus $\nu_2(N_n) = 0$.

2) A pairing identity for general n.

Pair the terms k and $n+1-k$:

$$\frac{D_n}{2k-1} + \frac{D_n}{2n-2k+1} = D_n \cdot \frac{(2n)}{(2k-1)(2n-2k+1)}.$$

Both factors in the denominator are odd, hence each pair contributes a multiple of $2n$. Therefore

$$N_n = 2n \cdot T_n, \qquad T_n = \sum_{k=1}^{\lfloor n/2 \rfloor} \frac{D_n}{(2k-1)(2n-2k+1)} \in \mathbb{Z}.$$

Consequently,

$$\nu_2(N_n) \geq \nu_2(2n) = \nu_2(n) + 1.$$

We now sharpen this to an equality $\nu_2(N_n) = 2\nu_2(n)$ for even n.

3) The case $n = 2^a m$ with m odd (so $a = \nu_2(n) \geq 1$).

Write $n = 2^a m$. In the sum that defines T_n, index the $n/2 = 2^{a-1}m$ terms by

$$k = r + t \cdot 2^{a-1} \qquad (1 \leq r \leq 2^{a-1},\ 0 \leq t \leq m-1).$$

For fixed r, the paired denominators are

$$(2k-1,\ 2n-2k+1) \equiv (2r-1,\ 2^{a+1}-2r+1) \pmod{2^{a+1}},$$

two odd numbers whose sum is 2^{a+1}. Modulo 2^{a+1}, their product is $-(2r-1)^2$. Because D_n is odd and independent of k, reduction modulo 2^{a+1} gives

$$\frac{D_n}{(2k-1)(2n-2k+1)} \equiv C \cdot (2r-1)^{-2} \pmod{2^{a+1}}$$

for some fixed odd constant C (the inverse exists since all numbers involved are odd).

Thus summing over the m values of t (a complete set of congruent classes modulo 2^{a+1}) yields

$$T_n \equiv m\, C \sum_{r=1}^{2^{a-1}} (2r-1)^{-2} \pmod{2^{a+1}}.$$

The map $x \mapsto x^{-1}$ permutes the odd residues modulo 2^{a+1}; hence

$$\sum_{r=1}^{2^{a-1}}(2r-1)^{-2} \equiv \sum_{r=1}^{2^{a-1}}(2r-1)^2 = \sum_{j=0}^{2^{a-1}-1}(2j+1)^2 = 2^{a-1}(2^a-1) \equiv 2^{a-1} \pmod{2^a}.$$

(Here we used the closed form for the sum of the first 2^{a-1} odd squares and observed that its residue modulo 2^a is 2^{a-1}.)

Therefore

$$T_n \equiv m\,C\,2^{a-1} \pmod{2^a},$$

and since m and C are odd, this congruence shows exactly

$$\nu_2(T_n) = a-1.$$

Combining with $N_n = 2n\,T_n$ gives

$$\nu_2(N_n) = \nu_2(2n) + \nu_2(T_n) = (a+1)+(a-1) = 2a = 2\,\nu_2(n),$$

as claimed.

This completes the proof.

Small checks

n	S_n	$\nu_2(A_n)$
1	$1 = \frac{1}{1}$	0
2	$1+\frac{1}{3} = \frac{4}{3}$	2 $(= 2\nu_2(2))$
3	$\frac{23}{15}$	0
4	$\frac{176}{105}$	4 $(= 2\nu_2(4))$
6	$\frac{6508}{3465}$	2 $(= 2\nu_2(6))$
8	$\frac{91072}{45045}$	6 $(= 2\nu_2(8))$

So the highest power of 2 dividing the numerator of $1+\frac{1}{3}+\cdots+\frac{1}{2n-1}$ is $2^{2\,\nu_2(n)}$ (and no higher power).

19 [M30]

Answer

$\boxed{n = 0 \text{ or } n = 1}$. For every $n \geq 2$, H_n is not an integer.

Proof (parity/LCM trick)

Let $n \geq 2$ and set

$$L = \operatorname{lcm}(1, 2, \dots, n).$$

Let 2^k be the largest power of 2 with $2^k \leq n < 2^{k+1}$. Then the exact power of 2 dividing L is 2^k.

Consider the integer

$$S \;=\; L\,H_n \;=\; \sum_{j=1}^{n} \frac{L}{j}.$$

Look at each term modulo 2:

- If $j = 2^k$, then $\dfrac{L}{j} = \dfrac{L}{2^k}$ is odd, because we have removed the full factor 2^k from L.
- If $j \neq 2^k$, then the highest power of 2 in j is at most 2^{k-1}; therefore $\dfrac{L}{j}$ still contains at least one factor 2 and is even.

Hence all terms in the sum are even except one, so S is odd.

But L is even (since $n \geq 2$ implies $2 \mid L$). If H_n were an integer, then $S = L \cdot H_n$ would be even—contradiction. Therefore H_n cannot be an integer for any $n \geq 2$.

Finally, the small cases are $H_0 = 0$ and $H_1 = 1$, both integers. Thus the only nonnegative n with $H_n \in \mathbb{Z}$ are $n = 0$ and $n = 1$.

20 [HM22]

1) An integral representation of H_k

For each integer $k \geq 1$,

$$H_k = \sum_{j=1}^{k} \frac{1}{j} = \sum_{j=1}^{k} \int_0^1 t^{j-1}\,dt = \int_0^1 \sum_{j=1}^{k} t^{j-1}\,dt = \int_0^1 \frac{1-t^k}{1-t}\,dt. \tag{1}$$

(We just summed the finite geometric series inside the integral.)

2) Work first with a truncated integral

Fix $r \in (0,1)$ and write

$$S := \sum_{k\geq 0} a_k x_0^k H_k = \sum_{k\geq 0} a_k x_0^k \int_0^1 \frac{1-t^k}{1-t}\,dt = \lim_{r\uparrow 1} \sum_{k\geq 0} a_k x_0^k \int_0^r \frac{1-t^k}{1-t}\,dt.$$

For each fixed $r < 1$, we may interchange sum and integral because $|x_0|r$ is strictly inside the radius of convergence, so $f(x_0 t) = \sum_{k\geq 0} a_k (x_0 t)^k$ converges uniformly on $t \in [0,r]$. Thus

$$\sum_{k\geq 0} a_k x_0^k \int_0^r \frac{1-t^k}{1-t}\,dt = \int_0^r \frac{1}{1-t} \sum_{k\geq 0} a_k (x_0^k - (x_0 t)^k)\,dt = \int_0^r \frac{f(x_0) - f(x_0 t)}{1-t}\,dt.$$

Hence

$$S = \lim_{r\uparrow 1} \int_0^r \frac{f(x_0) - f(x_0 t)}{1-t}\,dt. \tag{2}$$

3) Pass to the limit $r \uparrow 1$

Define $g(t) = \dfrac{f(x_0) - f(x_0 t)}{1-t}$ for $t \in [0,1)$. Because f is given by a power series that converges at x_0, f is differentiable on a neighborhood of the segment $\{x_0 t : t \in [0,1]\}$. By the mean value theorem applied to $h(t) = f(x_0 t)$,

$$\lim_{t\uparrow 1} g(t) = \lim_{t\uparrow 1} \frac{h(1) - h(t)}{1-t} = -h'(1) = x_0 f'(x_0).$$

So g extends continuously to $t = 1$ (define $g(1) = x_0 f'(x_0)$), hence is bounded on $[0,1]$. Dominated convergence (or just continuity of the integrand on the compact interval after extension) now yields

$$\lim_{r\uparrow 1} \int_0^r g(t)\,dt = \int_0^1 g(t)\,dt.$$

Combining this with (2) finishes the proof:

$$\boxed{\sum_{k\geq 0} a_k\, x_0^k\, H_k = \int_0^1 \frac{f(x_0) - f(x_0 y)}{1-y}\,dy}.$$

21 [M24]

Compute

$$S_n = \sum_{k=1}^{n} \frac{H_k}{n+1-k}, \qquad H_k = \sum_{i=1}^{k} \frac{1}{i}, \qquad H_m^{(2)} = \sum_{i=1}^{m} \frac{1}{i^2}.$$

Step 1: Rewrite as a double sum over a triangle

Expand H_k and swap the order of summation:

$$S_n = \sum_{k=1}^{n} \sum_{i=1}^{k} \frac{1}{i\,(n+1-k)} = \sum_{i=1}^{n} \frac{1}{i} \sum_{k=i}^{n} \frac{1}{n+1-k}.$$

With $j = n+1-k$ the inner sum runs $j = 1, \dots, n+1-i$, hence

$$S_n = \sum_{i=1}^{n} \frac{1}{i}\, H_{n+1-i}.$$

Equivalently, view this as the "triangular" double sum

$$S_n = \sum_{\substack{i\geq 1,\, j\geq 1 \\ i+j\leq n+1}} \frac{1}{ij}.$$

Step 2: Collapse each antidiagonal

Group pairs by their sum $r = i + j$ (so $r = 2, 3, \dots, n+1$):

$$S_n = \sum_{r=2}^{n+1} \sum_{i=1}^{r-1} \frac{1}{i(r-i)}.$$

Use the partial-fraction identity

$$\frac{1}{i(r-i)} = \frac{1}{r}\left(\frac{1}{i} + \frac{1}{r-i}\right),$$

to get

$$\sum_{i=1}^{r-1} \frac{1}{i(r-i)} = \frac{2}{r} \sum_{i=1}^{r-1} \frac{1}{i} = \frac{2\,H_{r-1}}{r}.$$

Therefore

$$S_n = 2\sum_{r=2}^{n+1} \frac{H_{r-1}}{r} = 2\sum_{k=1}^{n} \frac{H_k}{k+1}.$$

Step 3: Evaluate $\sum_{k=1}^{n} \dfrac{H_k}{k+1}$

Write $H_k = H_{k+1} - \dfrac{1}{k+1}$:

$$\sum_{k=1}^{n} \frac{H_k}{k+1} = \sum_{k=1}^{n} \frac{H_{k+1}}{k+1} - \sum_{k=1}^{n} \frac{1}{(k+1)^2} = \sum_{j=2}^{n+1} \frac{H_j}{j} - (H_{n+1}^{(2)} - 1).$$

A standard identity (provable by the same double-sum trick) is

$$\sum_{j=1}^{m} \frac{H_j}{j} = \frac{1}{2}\left(H_m^2 + H_m^{(2)}\right).$$

Using it with $m = n+1$ and subtracting the $j = 1$ term,

$$\sum_{j=2}^{n+1} \frac{H_j}{j} = \frac{1}{2}\left(H_{n+1}^2 + H_{n+1}^{(2)}\right) - 1.$$

Hence

$$\sum_{k=1}^{n} \frac{H_k}{k+1} = \frac{1}{2}\left(H_{n+1}^2 + H_{n+1}^{(2)}\right) - 1 - (H_{n+1}^{(2)} - 1) = \frac{1}{2}\left(H_{n+1}^2 - H_{n+1}^{(2)}\right).$$

Step 4: Final result

Plugging back into $S_n = 2\sum_{k=1}^{n} \dfrac{H_k}{k+1}$,

$$\boxed{\sum_{k=1}^{n} \frac{H_k}{n+1-k} = H_{n+1}^2 - H_{n+1}^{(2)}}.$$

Quick check (small n)

- $n = 1$: LHS $= H_1/1 = 1$. RHS $= H_2^2 - H_2^{(2)} = (1 + \frac{1}{2})^2 - (1 + \frac{1}{4}) = 1$.
- $n = 2$: LHS $= H_1/2 + H_2/1 = \frac{1}{2} + \frac{3}{2} = 2$. RHS $= H_3^2 - H_3^{(2)} = (\frac{11}{6})^2 - (1 + \frac{1}{4} + \frac{1}{9}) = 2$.

22 [M28]

1) Generating function setup

The ordinary generating function of $\{H_n\}_{n\geq 0}$ is

$$F(x) = \sum_{n\geq 0} H_n x^n = \frac{-\ln(1-x)}{1-x},$$

because

$$\sum_{n\geq 1} H_n x^n = \sum_{n\geq 1}\sum_{m=1}^{n} \frac{x^n}{m} = \sum_{m\geq 1}\frac{1}{m}\sum_{n\geq m} x^n = \frac{1}{1-x}\sum_{m\geq 1}\frac{x^m}{m} = \frac{-\ln(1-x)}{1-x}.$$

Therefore $S_n = [x^n]\,F(x)^2 = [x^n]\left(\dfrac{\ln(1-x)}{1-x}\right)^2$.

2) Coefficient of $\big(\ln(1-x)\big)^2$

Write $-\ln(1-x) = \sum_{r\geq 1}\frac{x^r}{r}$. Then

$$\big(\ln(1-x)\big)^2 = \sum_{m\geq 2}\left(\sum_{r=1}^{m-1}\frac{1}{r(m-r)}\right)x^m = \sum_{m\geq 2}\frac{2H_{m-1}}{m}\,x^m,$$

since $\dfrac{1}{r(m-r)} = \dfrac{1}{m}\left(\dfrac{1}{r}+\dfrac{1}{m-r}\right)$.

Also $\dfrac{1}{(1-x)^2} = \sum_{t\geq 0}(t+1)x^t$. Hence

$$S_n = [x^n]\frac{(\ln(1-x))^2}{(1-x)^2} = \sum_{m=2}^{n}(n-m+1)\cdot\frac{2H_{m-1}}{m}.$$

Let $r = m-1$ (so $r = 1,\dots,n-1$):

$$S_n = 2\sum_{r=1}^{n-1}\frac{(n-r)H_r}{r+1} = 2n\sum_{r=1}^{n-1}\frac{H_r}{r+1} \;-\; 2\sum_{r=1}^{n-1}\frac{rH_r}{r+1}.$$

3) Two standard harmonic sums

We use

$$\sum_{k=1}^{n-1} \frac{H_k}{k+1} = \frac{1}{2}\Big(H_n^2 - H_n^{(2)}\Big), \qquad \sum_{k=1}^{n-1} H_k = n(H_n - 1),$$

and $\displaystyle\sum_{k=1}^{n-1} \frac{kH_k}{k+1} = \sum_{k=1}^{n-1} \left(H_k - \frac{H_k}{k+1}\right) = n(H_n - 1) - \frac{1}{2}(H_n^2 - H_n^{(2)})$.

Substitute into the expression for S_n:

$$\begin{aligned} S_n &= 2n \cdot \frac{1}{2}\Big(H_n^2 - H_n^{(2)}\Big) \;-\; 2\Big(n(H_n - 1) - \tfrac{1}{2}(H_n^2 - H_n^{(2)})\Big) \\ &= (n+1)\Big(H_n^2 - H_n^{(2)}\Big) - 2nH_n + 2n. \end{aligned}$$

It is slightly cleaner in terms of H_{n+1}. Using $H_n = H_{n+1} - \dfrac{1}{n+1}$ and $H_n^{(2)} = H_{n+1}^{(2)} - \dfrac{1}{(n+1)^2}$, a brief simplification yields the compact closed form:

$$\boxed{\sum_{k=0}^{n} H_k\, H_{n-k} = (n+1)\Big(H_{n+1}^2 - H_{n+1}^{(2)} - 2H_{n+1} + 2\Big).}$$

4) Quick checks

- $n = 1$: $H_0H_1 + H_1H_0 = 0$. RHS: $(2)(H_2^2 - H_2^{(2)} - 2H_2 + 2) = 2(2.25 - 1.25 - 3 + 2) = 0$.
- $n = 3$: LHS $= H_1H_2 + H_2H_1 = 3$. RHS with $H_4 = 25/12$, $H_4^{(2)} = 205/144$ gives 3.

23 [HM20]

Let

$$\psi(x) \;\stackrel{\text{def}}{=}\; \frac{d}{dx} \ln \Gamma(x) \;=\; \frac{\Gamma'(x)}{\Gamma(x)}$$

be the digamma function.

Step 1 — A key recurrence for ψ.

From the Gamma functional equation $\Gamma(x+1) = x\,\Gamma(x)$, taking logs and differentiating gives

$$\psi(x+1) = \psi(x) + \frac{1}{x} \qquad (x \notin \{0, -1, -2, \dots\}).$$

Step 2 — Recover H_n when x is an integer.

For a positive integer n, iterating the recurrence:

$$\psi(n+1) = \psi(1) + \sum_{k=1}^{n} \frac{1}{k} = \psi(1) + H_n.$$

Using the given fact $\psi(1) = \Gamma'(1)/\Gamma(1) = -\gamma$, we obtain

$$H_n = \psi(n+1) + \gamma.$$

This identity is also recorded in the book's answers.

Step 3 — The definition for nonintegers.

Motivated by the integer case, define the generalized harmonic number for any real (indeed complex) x not equal to a negative integer by

$$\boxed{H_x \;:=\; \psi(x+1) + \gamma}.$$

With this definition:

- Consistency. If $x = n \in \mathbb{N}$, then $H_x = H_n$ by Step 2.
- Recurrence. $H_x - H_{x-1} = \dfrac{1}{x}$ for all admissible x, since

  $$H_x - H_{x-1} = \big[\psi(x+1) - \psi(x)\big] = \frac{1}{x}.$$

  (This exactly generalizes the discrete increment property of harmonic numbers.)
- Asymptotics. Using the standard expansion $\psi(x) = \ln x - \frac{1}{2x} + O(x^{-2})$, one gets

  $$H_x = \ln x + \gamma + \frac{1}{2x} + O\left(\frac{1}{x^2}\right),$$

  matching the familiar behavior of H_n.

Example values

- $H_{1/2} = \psi\big(\frac{3}{2}\big) + \gamma = \big(\psi(\frac{1}{2}) + 2\big) + \gamma = 2 - 2\ln 2.$
- The recurrence gives $H_{3/2} = H_{1/2} + \dfrac{2}{3} = \dfrac{8}{3} - 2\ln 2.$

24 [HM21]

Proof via partial products

Define the n-th partial product

$$P_n(x) \;=\; x\,e^{\gamma x}\prod_{k=1}^{n}\Big(1+\frac{x}{k}\Big)e^{-x/k}.$$

First, separate the exponential and finite product:

$$P_n(x) = x\Big(\prod_{k=1}^{n}\Big(1+\frac{x}{k}\Big)\Big)\exp\Big(x\gamma-\sum_{k=1}^{n}\frac{x}{k}\Big) = x\Big(\prod_{k=1}^{n}\Big(1+\frac{x}{k}\Big)\Big)\,e^{x(\gamma-H_n)}.$$

Use $\displaystyle\prod_{k=1}^{n}\Big(1+\frac{x}{k}\Big) = \prod_{k=1}^{n}\frac{x+k}{k} = \frac{(x+1)(x+2)\cdots(x+n)}{n!}$, hence

$$P_n(x) = \frac{x(x+1)\cdots(x+n)}{n!}\,e^{x(\gamma-H_n)}.$$

Insert and remove $n^x = e^{x\ln n}$:

$$P_n(x) = \frac{x(x+1)\cdots(x+n)}{n!\,n^x}\;\exp\Big(x(\gamma-(H_n-\ln n))\Big).$$

Now let $n\to\infty$. By definition $H_n-\ln n\to\gamma$, so the exponential factor $\exp\big(x(\gamma-(H_n-\ln n))\big)\to 1$. Therefore

$$\lim_{n\to\infty}P_n(x) = \lim_{n\to\infty}\frac{x(x+1)\cdots(x+n)}{n!\,n^x} = \frac{1}{\Gamma(x)}\qquad\text{(Gauss)},$$

which proves the formula.

Convergence (why the infinite product makes sense)

Consider the logarithm of the infinite product's factor:

$$\log\Big(1+\frac{x}{k}\Big)-\frac{x}{k} = -\frac{x^2}{2k^2}+O\Big(\frac{1}{k^3}\Big)\qquad(k\to\infty).$$

Hence $\sum_{k\ge1}(\log(1+x/k)-x/k)$ converges absolutely (like $\sum 1/k^2$), so the product $\prod_{k\ge1}(1+\frac{x}{k})e^{-x/k}$ converges (uniformly on compact x-sets) and defines an entire function. The extra factor $x\,e^{\gamma x}$ fixes the normalization so that the result agrees with Γ (as we just showed).

Sanity checks

- Zeros/poles. The product clearly vanishes at $x = 0, -1, -2, ...$, matching the poles of $\Gamma(x)$ at those points, so $1/\Gamma$ has the same zeros (simple).
- Near $x = 0$. Since $\Gamma(x) \sim 1/x - \gamma + O(x)$, we have $1/\Gamma(x) = x + \gamma x^2 + O(x^3)$. On the product side, $\prod(1 + x/k)e^{-x/k} = 1 + O(x^2)$ and $xe^{\gamma x} = x + \gamma x^2 + O(x^3)$, in agreement.

25 [M21]

(a) Evaluate $H_n^{(0,v)}$ and $H_n^{(u,0)}$

$H_n^{(0,v)}$

$$H_n^{(0,v)} = \sum_{1\le j\le k\le n} \frac{1}{k^v} = \sum_{k=1}^{n} \frac{1}{k^v} \sum_{j=1}^{k} 1 = \sum_{k=1}^{n} \frac{k}{k^v} = \sum_{k=1}^{n} k^{1-v} = \sum_{k=1}^{n} \frac{1}{k^{v-1}} = H_n^{(v-1)}.$$

$H_n^{(u,0)}$

$$H_n^{(u,0)} = \sum_{1\le j\le k\le n} \frac{1}{j^u} = \sum_{j=1}^{n} \frac{1}{j^u} \sum_{k=j}^{n} 1 = \sum_{j=1}^{n} \frac{n-j+1}{j^u} = (n+1)\sum_{j=1}^{n} \frac{1}{j^u} - \sum_{j=1}^{n} \frac{j}{j^u} = (n+1)H_n^{(u)} - H_n^{(u-1)}.$$

Thus

$$\boxed{H_n^{(0,v)} = H_n^{(v-1)}, \qquad H_n^{(u,0)} = (n+1)H_n^{(u)} - H_n^{(u-1)}}.$$

These identities hold for any real u, v since the sums are finite.

(b) Prove $H_n^{(u,v)} + H_n^{(v,u)} = H_n^{(u)} H_n^{(v)} + H_n^{(u+v)}$

Start from the product of univariate sums:

$$H_n^{(u)} H_n^{(v)} = \Big(\sum_{j=1}^{n} j^{-u}\Big)\Big(\sum_{k=1}^{n} k^{-v}\Big) = \sum_{j=1}^{n}\sum_{k=1}^{n} \frac{1}{j^u k^v}.$$

Partition the index set $\{1, \dots, n\}^2$ into three regions: $j < k$, $j = k$, $j > k$. Denote $D = \sum_{j=1}^{n} j^{-(u+v)} = H_n^{(u+v)}$.

- Over $j < k$: $\displaystyle\sum_{1\le j<k\le n} \frac{1}{j^u k^v} = H_n^{(u,v)} - D$ (because $H_n^{(u,v)}$ includes the diagonal $j = k$ contributing D).

- Over $j > k$: by swapping $j \leftrightarrow k$, $\displaystyle\sum_{1\le k<j\le n} \frac{1}{j^u k^v} = \sum_{1\le j<k\le n} \frac{1}{j^v k^u} = H_n^{(v,u)} - D$.
- Over $j = k$: contribution is D.

Therefore

$$H_n^{(u)} H_n^{(v)} = \big(H_n^{(u,v)} - D\big) + \big(H_n^{(v,u)} - D\big) + D = H_n^{(u,v)} + H_n^{(v,u)} - D.$$

Rearranging yields

$$\boxed{H_n^{(u,v)} + H_n^{(v,u)} = H_n^{(u)} H_n^{(v)} + H_n^{(u+v)}}.$$

This is a simple double-counting (index-partition) argument.

Quick sanity check (small n)

Take $n = 2$, $u = 1$, $v = 2$.

- $H_2^{(1)} = 1 + 1/2 = 3/2, \quad H_2^{(2)} = 1 + 1/4 = 5/4, \quad H_2^{(3)} = 1 + 1/8 = 9/8.$
- $H_2^{(1,2)} = \sum_{1\le j\le k\le 2} 1/(j^1 k^2) = 1 + 1/2^2 + 1/(1 \cdot 2^2) = 1 + 1/4 + 1/4 = 3/2.$
- $H_2^{(2,1)} = \sum_{1\le j\le k\le 2} 1/(j^2 k^1) = 1 + 1/2 + 1/(1^2 \cdot 2) = 1 + 1/2 + 1/2 = 2.$

Left side: $H_2^{(1,2)} + H_2^{(2,1)} = 3/2 + 2 = 7/2$.

Right side: $H_2^{(1)} H_2^{(2)} + H_2^{(3)} = (3/2)(5/4) + 9/8 = 15/8 + 9/8 = 24/8 = 3$. Wait—that's not equal; we made a slip in $H_2^{(1,2)}$.

Correct $H_2^{(1,2)}$:

$$(j,k) = (1,1) : 1,\ (1,2) : 1/(1 \cdot 2^2) = 1/4,\ (2,2) : 1/(2 \cdot 2^2) = 1/8.$$

So $H_2^{(1,2)} = 1 + 1/4 + 1/8 = 11/8$.

Similarly $H_2^{(2,1)}$:

$$(1,1) : 1,\ (1,2) : 1/(1^2 \cdot 2) = 1/2,\ (2,2) : 1/(2^2 \cdot 2) = 1/8,$$

so $H_2^{(2,1)} = 1 + 1/2 + 1/8 = 13/8$.

Now LHS $= 11/8 + 13/8 = 24/8 = 3$, matching RHS $= 3$. Check.

1.2.8 Fibonacci numbers

1 [10]

Model the process precisely

- Let R_n be the number of rabbit pairs at the end of month n, starting from one newborn pair at month 0.
- Each mature pair produces one new pair every month; newborn pairs become mature after one month; rabbits never die. In Fibonacci's own description, after one month there are 2 pairs, after two months 3 pairs, after three months 5, and so on.

Split pairs into mature M_n and immature I_n. The month-to-month transitions are:

$$\begin{aligned} M_{n+1} &= M_n + I_n \quad \text{(last month's immature pairs mature)} \\ I_{n+1} &= M_n \qquad\quad\;\; \text{(each mature pair produces one new pair).} \end{aligned}$$

Hence the totals $R_n = M_n + I_n$ satisfy

$$R_{n+1} \;=\; (M_n + I_n) + M_n \;=\; R_n + M_n \;=\; R_n + R_{n-1},$$

because $M_n = I_{n+1} = R_{n-1}$. Initial conditions are $R_0 = 1$ (one newborn pair) and $R_1 = 2$ (the original pair, now fertile, produces a new pair by the end of month 1). This is exactly the Fibonacci recurrence with a shift.

Identify the Fibonacci index

Knuth defines the Fibonacci numbers by $F_0 = 0,\ F_1 = 1,\ F_{n+2} = F_{n+1} + F_n$. With the initial values above it follows (by induction) that

$$R_n \;=\; F_{n+2}.$$

(Indeed, $R_0 = 1 = F_2,\ R_1 = 2 = F_3$, and if $R_n = F_{n+2}$ and $R_{n-1} = F_{n+1}$, then $R_{n+1} = R_n + R_{n-1} = F_{n+3}$.)

Compute the 12-month total

After a year ($n = 12$ months),

$$R_{12} \;=\; F_{14}.$$

The Fibonacci numbers from F_0 upward are $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...$, so $F_{14} = 377$.

Therefore, there are 377 pairs after one year. Knuth's answers section states this succinctly: "After k months there are F_{k+2} pairs, so the answer is $F_{14} = 377$ pairs."

(If you ever see 233 in other tellings, that's from a different month/index convention. Under the book's convention—2 pairs after the first month—the correct 12-month answer is 377.)

2 [20]

We use Binet's formula and logarithms.

1) Background. Let $\varphi = \dfrac{1+\sqrt{5}}{2}$ and $\psi = \dfrac{1-\sqrt{5}}{2} = -\varphi^{-1}$. Binet's formula says

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}.$$

Since $|\psi| < 1$, the term ψ^n is tiny for large n. In fact,

$$0 < \frac{\varphi^n}{\sqrt{5}} - F_n = \frac{\psi^n}{\sqrt{5}} < \frac{1}{\sqrt{5}}\left(\frac{1}{\varphi}\right)^n,$$

and the relative error is $\left|\psi/\varphi\right|^n = (\varphi^{-2})^n$, which for $n = 1000$ is $\approx 10^{-418}$. Thus $\varphi^n/\sqrt{5}$ determines the front digits of F_n unambiguously.

2) Number of digits. For any positive integer N, the number of decimal digits is

$$\#\text{digits}(N) = \lfloor \log_{10} N \rfloor + 1.$$

Hence

$$\#\text{digits}(F_n) = \left\lfloor \log_{10}\!\left(\tfrac{\varphi^n - \psi^n}{\sqrt{5}}\right) \right\rfloor + 1 \;=\; \left\lfloor n \log_{10} \varphi - \log_{10} \sqrt{5} + \delta_n \right\rfloor + 1,$$

where $\delta_n = \log_{10}(1 - \psi^n/\varphi^n)$ and δ_{1000} is about -10^{-418}, i.e., negligible for the integer part.

Compute the main term (to, say, 15–18 significant digits for safety):

$$\log_{10}\varphi \approx 0.20898764024997873, \qquad \log_{10}\sqrt{5} \approx 0.3494850021680094.$$

Then

$$L := 1000\log_{10}\varphi - \log_{10}\sqrt{5} \approx 208.6381552478107.$$

Therefore

$$\#\text{digits}(F_{1000}) = \lfloor L \rfloor + 1 = 208 + 1 = 209.$$

3) Leading digit. Write $F_{1000} = 10^{\lfloor L \rfloor} \cdot 10^{\{L\}} \cdot (1+\varepsilon)$, where $\{L\} = L - \lfloor L \rfloor$ is the fractional part and ε is the tiny relative correction from ψ^{1000}. The significand is essentially $S = 10^{\{L\}}$.

From the value above,

$$\{L\} \approx 0.6381552478107 \quad \Longrightarrow \quad S = 10^{\{L\}} \approx 4.346655768\ldots$$

Hence the leading decimal digit is $\boxed{4}$. (Because the correction $|\varepsilon| \lesssim 10^{-418}$ cannot possibly change the first digit.)

Result.

$$\boxed{\#\text{digits}(F_{1000}) = 209, \quad \text{first digit} = 4.}$$

General form (useful to remember). For any $n \geq 1$,

$$\#\text{digits}(F_n) \;=\; \left\lfloor n\log_{10}\varphi - \log_{10}\sqrt{5} \right\rfloor + 1,$$

and if $x = n\log_{10}\varphi - \log_{10}\sqrt{5}$ with fractional part $\{x\}$, then the leading digit of F_n is $\lfloor 10^{\{x\}} \rfloor$ (again, because the ψ^n term is far too small to affect the front digit for large n).

3 [25]

What size are these numbers?

Knuth gives the approximation $F_n \approx \varphi^n/\sqrt{5}$, so the number of decimal digits of F_n is

$$\left\lfloor n \log_{10} \varphi - \log_{10} \sqrt{5} \right\rfloor + 1.$$

For $n = 1000$ this yields 209 digits, which matches the computed result.

Algorithm (addition-only)

We compute iteratively from the recurrence $F_{n+2} = F_{n+1} + F_n$ with seeds $F_0 = 0$, $F_1 = 1$. To honor the "addition exercise" spirit, I implemented big integers by hand:

- Represent each big integer as base 10^9 "chunks" in little-endian order.
- Addition is schoolbook addition over those chunks with a single carry bit.
- Update the two registers $(a, b) = (F_{n-1}, F_n)$ by $(a, b) \leftarrow (b,\ a + b)$ for $n = 1 \ldots 1000$.

Correctness (brief)

Induction on n. Base: $(a, b) = (0, 1) = (F_0, F_1)$. Step: assuming $(a, b) = (F_{n-1}, F_n)$, the update makes $b' = a + b = F_{n-1} + F_n = F_{n+1}$ and $a' = b = F_n$, so $(a', b') = (F_n, F_{n+1})$. Thus after n steps we have F_n.

- $F(1000)$ has 209 digits.
- First 50 digits of $F(1000)$: `43466557686937456435688527675040625802564660517371`
- Last 50 digits of $F(1000)$: `33187938298969649928516003704476137795166849228875`

```python
# Big-int Fibonacci using base 10^9 chunks (addition-only)
BASE = 109

def add_big(a, b):
    out, carry = [], 0
    n = max(len(a), len(b))
    for i in range(n):
        s = (a[i] if i < len(a) else 0) + (b[i] if i < len(b) else 0) + carry
        if s >= BASE:
            s -= BASE; carry = 1
        else:
            carry = 0
        out.append(s)
    if carry: out.append(carry)
    return out
```

```python
def to_string(a):
    if not a: return "0"
    s = str(a[-1])
    for x in reversed(a[:-1]): s += f"{x:09d}"
    return s

# Build F0..F1000, then write F1..F1000 to files
F = [[0], [1]]
for _ in range(2, 1000 + 1):
    F.append(add_big(F[-1], F[-2]))
with open("fibonacci_1_1000.txt","w") as f_txt, \
     open("fibonacci_1_1000.csv","w") as f_csv:
    f_csv.write("n,F,Digits\n")
    for n in range(1, 1001):
        s = to_string(F[n])
        f_txt.write(f"F({n}) = {s}\n")
        f_csv.write(f"{n},{s},{len(s)}\n")
```

4 [14]

Quick check of small n

The Fibonacci numbers begin

$$F_0 = 0,\ F_1 = 1,\ F_2 = 1,\ F_3 = 2,\ F_4 = 3,\ F_5 = 5,\ F_6 = 8,\ F_7 = 13, \ldots$$

We immediately see $F_0 = 0$, $F_1 = 1$, and $F_5 = 5$. For $n = 2, 3, 4$ we have $F_n < n$; for $n = 6$ we have $F_6 = 8 > 6$.

This suggests the only solutions are $n \in \{0, 1, 5\}$.

Proof that there are no others (elementary induction)

We show $F_n > n$ for all $n \geq 6$.

Base cases. $F_6 = 8 > 6$ and $F_7 = 13 > 7$.

Inductive step. Assume $F_k > k$ and $F_{k-1} > k - 1$ for some $k \geq 7$. Then

$$F_{k+1} = F_k + F_{k-1} > k + (k - 1) = 2k - 1 \geq k + 1.$$

Hence $F_{k+1} > k + 1$. By induction, $F_n > n$ for all $n \geq 6$.

Combining this with the quick check up to $n = 5$, the only indices with $F_n = n$ are

$$\boxed{n = 0, 1, 5}.$$

(Optional) Growth-based one-liner

Using the closed form $F_n = \frac{\varphi^n - \hat{\varphi}^n}{\sqrt{5}}$ with $\varphi = (1+\sqrt{5})/2$ and $|\hat{\varphi}| < 1$, we have F_n grows like $\varphi^n/\sqrt{5}$, so it eventually dominates the linear function n. This also rules out further solutions beyond small n.

5 [20]

Define $g(n) = F_n - n^2$. We'll show $g(n)$ becomes strictly increasing from $n = 9$ onward; since $g(12) = 0$, that will force $n = 12$ to be the only solution for large n. Small n can be checked directly.

Step 1: A monotonicity lemma for $g(n)$

Using $F_{n+1} = F_n + F_{n-1}$,

$$g(n+1) - g(n) = (F_{n+1} - (n+1)^2) - (F_n - n^2) = F_{n-1} - (2n+1).$$

Thus g is increasing whenever $F_{n-1} > (2n+1)$.

At $n = 9$ we have $F_8 = 21$ and $2 \cdot 9 + 1 = 19$, so g begins to increase: $g(10) - g(9) = 2 > 0$. Moreover, for $n \geq 10$,

$$(F_n - (2(n+1)+1)) - (F_{n-1} - (2n+1)) = (F_n - F_{n-1}) - 2 = F_{n-2} - 2 \geq F_8 - 2 = 19 > 0,$$

so once positive, the gap strictly widens thereafter. Hence $g(n)$ is strictly increasing for all $n \geq 9$.

Step 2: Pin down the zero(s)

Compute a few values:

n	9	10	11	12
F_n	34	55	89	144
n^2	81	100	121	144
$g(n)$	-47	-45	-32	0

Thus $g(9) < 0$, $g(12) = 0$, and by Step 1 $g(n) > 0$ for every $n \geq 13$. Therefore there is no solution with $n \geq 13$, and $n = 12$ is the unique solution in $\{9, 10, 11, 12, ...\}$.

Step 3: Check the small cases

Directly:

$$F_0 = 0 = 0^2, \quad F_1 = 1 = 1^2, \quad F_2 = 1,\ F_3 = 2,\ F_4 = 3,\ F_5 = 5,\ F_6 = 8,\ F_7 = 13,\ F_8 = 21,$$

none of which equals n^2 except $n = 0, 1$.

6 [HM10]

Let $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$. We will show by induction on $n \geq 1$ that

$$A^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}.$$

Base case $n = 1$. $A^1 = A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} F_2 & F_1 \\ F_1 & F_0 \end{pmatrix}$ since $F_0 = 0,\ F_1 = 1,\ F_2 = 1$.

Inductive step. Assume

$$A^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}.$$

Then

$$A^{n+1} = A^n A = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} F_{n+1} + F_n & F_{n+1} \\ F_n + F_{n-1} & F_n \end{pmatrix}.$$

Using the Fibonacci recurrence $F_{k+1} = F_k + F_{k-1}$, we get

$$A^{n+1} = \begin{pmatrix} F_{n+2} & F_{n+1} \\ F_{n+1} & F_n \end{pmatrix},$$

which is exactly the desired form with n replaced by $n + 1$.

By induction, the identity holds for all $n \geq 1$, proving Eq. (5).

7 [15]

We'll use two standard facts from the section.

1. Fibonacci addition formula. For all positive integers m, n,

$$F_{m+n} = F_m F_{n+1} + F_{m-1} F_n.$$

This is Eq. (6) in the text.

2. (Optional strengthening, due to Lucas): $\gcd(F_m, F_n) = F_{\gcd(m,n)}$.

Lemma. If $d \mid n$, then $F_d \mid F_n$.

Proof (by induction on k where $n = kd$). For $k = 1$ the claim is trivial. Assume $F_d \mid F_{kd}$. Using Eq. (6) with $m = kd$ and $n = d$,

$$F_{(k+1)d} = F_{kd} F_{d+1} + F_{kd-1} F_d.$$

By the induction hypothesis F_{kd} is a multiple of F_d; the second term is clearly a multiple of F_d. Hence $F_d \mid F_{(k+1)d}$. QED.

(Equivalently, the lemma is immediate from Lucas's theorem, since if $d \mid n$ then $\gcd(F_n, F_d) = F_d$, so F_d divides F_n.)

Suppose n is composite. Then n has a proper divisor $d > 1$.

- If $d \geq 3$, then $F_d \geq F_3 = 2$ and (by monotonicity of F_n for $n \geq 2$) $F_d < F_n$. By the lemma $F_d \mid F_n$, so F_n has a nontrivial divisor and is therefore composite.
- The only composite integer with no proper divisor > 2 is $n = 4$. Here the only proper divisor is $d = 2$ and $F_2 = 1$ (which does not yield a nontrivial divisor). Direct computation gives $F_4 = 3$, which is prime.

Thus, if n is not prime, then F_n is not prime, with the unique exception $n = 4$.

8 [15]

Recall the (positive-index) Fibonacci numbers:

- $F_0 = 0,\ F_1 = 1,$
- $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.

A natural way to extend any linear recurrence to all integers is to run the recurrence backwards. From

$$F_n = F_{n-1} + F_{n-2} \quad \Longrightarrow \quad F_{n-2} = F_n - F_{n-1},$$

so once F_0, F_1 are fixed, every F_k with $k < 0$ is determined uniquely.

Step 1: Determine F_{-1} and F_{-2}

Use the recurrence with $n = 1$ and $n = 0$:

- $F_1 = F_0 + F_{-1} \Rightarrow 1 = 0 + F_{-1} \Rightarrow F_{-1} = 1.$
- $F_0 = F_{-1} + F_{-2} \Rightarrow 0 = 1 + F_{-2} \Rightarrow F_{-2} = -1.$

These two values will seed an induction.

Step 2: Prove the general formula

Claim. For all $n \geq 1$,

$$\boxed{F_{-n} = (-1)^{n+1} F_n}.$$

Proof by induction on n.

Base cases: $n = 1$: $F_{-1} = 1 = (-1)^2 F_1$. $n = 2$: $F_{-2} = -1 = (-1)^3 F_2$ since $F_2 = 1$.

Inductive step: Assume the claim for n and $n - 1$. Using the backward recurrence $F_{m-1} = F_{m+1} - F_m$ with $m = -n$,

$$F_{-(n+1)} = F_{-n+1} - F_{-n}.$$

Apply the induction hypotheses:

$$F_{-(n+1)} = (-1)^n F_{n-1} - (-1)^{n+1} F_n = (-1)^n (F_{n-1} + F_n) = (-1)^n F_{n+1} = (-1)^{(n+1)+1} F_{n+1}.$$

Thus the formula holds for $n + 1$. By induction, it holds for all $n \geq 1$.

Step 3: Sanity check examples

The extension alternates signs:

$$\ldots,\ F_{-5}=5,\ F_{-4}=-3,\ F_{-3}=2,\ F_{-2}=-1,\ F_{-1}=1,\ F_0=0,\ F_1=1,\ F_2=1,\ F_3=2,\ F_4=3,\ F_5=5,\ldots$$

which matches $F_{-n}=(-1)^{n+1}F_n$.

Optional alternate derivation (Binet)

From Binet's formula $F_n=\frac{\varphi^n-\hat{\varphi}^n}{\sqrt{5}}$ with $\hat{\varphi}=1-\varphi=-1/\varphi$,

$$F_{-n}=\frac{\varphi^{-n}-\hat{\varphi}^{-n}}{\sqrt{5}}=\frac{\varphi^{-n}-(-1)^n\varphi^n}{\sqrt{5}}=(-1)^{n+1}\frac{\varphi^n-\hat{\varphi}^n}{\sqrt{5}}=(-1)^{n+1}F_n.$$

9 [M20]

Define $F_0=0,\ F_1=1$ and keep $F_{n+2}=F_{n+1}+F_n$ for all $n\in\mathbb{Z}$ (this is the convention from Exercise 8). Running the recurrence backwards gives

$$F_{-1}=1,\quad F_{-2}=-1,\quad F_{-3}=2,\ \ldots$$

and by induction (or a quick check with the recurrence) one gets the standard identity

$$\boxed{F_{-n}=(-1)^{n+1}F_n\quad(n\geq 1).}$$

This is the only new fact we'll need from the extension. (See the exercise list introducing negative indices.)

(4) Cassini's identity for all integers

Cassini's formula in the text is

$$F_{n+1}F_{n-1}-F_n^2=(-1)^n. \tag{4}$$

Using Binet's form $F_n=\frac{\phi^n-\hat{\phi}^n}{\sqrt{5}}$ with $\phi=\frac{1+\sqrt{5}}{2},\ \hat{\phi}=\frac{1-\sqrt{5}}{2}$ (note $\phi+\hat{\phi}=1,\ \phi\hat{\phi}=-1$), valid as an identity of exponentials for any integer n, compute

$$
\begin{aligned}
F_{n+1}F_{n-1} - F_n^2 &= \frac{1}{5}\Big[(\phi^{n+1} - \hat{\phi}^{\,n+1})(\phi^{n-1} - \hat{\phi}^{\,n-1}) - (\phi^n - \hat{\phi}^{\,n})^2\Big] \\
&= \frac{1}{5}\,\phi^{n-1}\hat{\phi}^{\,n-1}\,\big(-\phi^2 - \hat{\phi}^{\,2} + 2\phi\hat{\phi}\big) \\
&= -(\phi\hat{\phi})^{n-1} = -(-1)^{n-1} = (-1)^n.
\end{aligned}
$$

No positivity of n was used; thus (4) holds for all $n \in \mathbb{Z}$.

(6) The addition formula for all integers

The text gives

$$
F_{n+m} = F_m\,F_{n+1} + F_{m-1}\,F_n \qquad (m > 0 \text{ in the derivation}). \tag{6}
$$

With negative indices allowed and Binet's formula, expand the right-hand side using $\phi\hat{\phi} = -1$ and $\phi + \hat{\phi} = 1$:

$$
\begin{aligned}
F_mF_{n+1} + F_{m-1}F_n &= \frac{(\phi^m - \hat{\phi}^{\,m})(\phi^{n+1} - \hat{\phi}^{\,n+1})}{5} + \frac{(\phi^{m-1} - \hat{\phi}^{\,m-1})(\phi^n - \hat{\phi}^{\,n})}{5} \\
&= \frac{\phi^{m+n+1} - \hat{\phi}^{\,m+n+1}}{\sqrt{5}} \cdot \frac{1}{\sqrt{5}} + (\text{cross terms that cancel}) \\
&= \frac{\phi^{n+m} - \hat{\phi}^{\,n+m}}{\sqrt{5}} = F_{n+m},
\end{aligned}
$$

and this algebra is valid for all integers m, n (negative exponents are fine). Therefore (6) holds for all $m, n \in \mathbb{Z}$. (Equivalently, one can argue via the matrix identity (5) and the fact that $\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$ is invertible.)

(14) Binet's formula for all integers

Binet's formula is

$$
F_n = \frac{\phi^n - \hat{\phi}^{\,n}}{\sqrt{5}}. \tag{14}
$$

For negative $n = -k$ with $k > 0$, use $\phi^{-k} = (-1)^k\hat{\phi}^{\,k}$ and $\hat{\phi}^{-k} = (-1)^k\phi^k$ (from $\phi\hat{\phi} = -1$):

$$
F_{-k} = \frac{(-1)^k\hat{\phi}^{\,k} - (-1)^k\phi^k}{\sqrt{5}} = (-1)^{k+1}\frac{\phi^k - \hat{\phi}^{\,k}}{\sqrt{5}} = (-1)^{k+1}F_k,
$$

which matches the extended definition above. Hence (14) remains true for every integer n.

(15) The nearest-integer rule

The corollary stated is

$$F_n = \text{the nearest integer to } \frac{\phi^n}{\sqrt{5}}. \tag{15}$$

The proof in the text uses $|\hat{\phi}| < 1$ to bound $|\hat{\phi}^n|/\sqrt{5} < \frac{1}{2}$, so rounding $\phi^n/\sqrt{5}$ gives F_n. This bound holds when $n \geq 1$, but fails for negative n: e.g.

$$n = -1: \quad F_{-1} = 1, \quad \frac{\phi^{-1}}{\sqrt{5}} \approx 0.276 \text{ (nearest integer 0, not 1).}$$

Thus (15) is valid for $n \geq 0$ (indeed for all $n \geq 1$), but not in general for negative n.

10 [15]

We use Binet's formula (derived in §1.2.8):

$$F_n = \frac{\phi^n - \hat{\phi}^n}{\sqrt{5}}, \qquad \hat{\phi} := 1 - \phi = \tfrac{1-\sqrt{5}}{2} = -\frac{1}{\phi},$$

so $\hat{\phi} \in (-1, 0)$.

Subtract F_n from $\phi^n/\sqrt{5}$:

$$\frac{\phi^n}{\sqrt{5}} - F_n = \frac{\phi^n}{\sqrt{5}} - \frac{\phi^n - \hat{\phi}^n}{\sqrt{5}} = \frac{\hat{\phi}^n}{\sqrt{5}}.$$

Because $\hat{\phi} < 0$, the term $\hat{\phi}^n$ is

- positive when n is even,
- negative when n is odd,

and it is always small in magnitude since $|\hat{\phi}| < 1$. (Knuth also notes that this smallness gives the "nearest–integer" rule $F_n = \phi^n/\sqrt{5}$ rounded to the nearest integer.)

Therefore:

- If n is even, $\dfrac{\phi^n}{\sqrt{5}} > F_n$.
- If n is odd, $\dfrac{\phi^n}{\sqrt{5}} < F_n$.

11 [M20]

We'll prove the ϕ-identity for all integers n; the $\hat{\phi}$-identity then follows in the same way (since $\hat{\phi}$ satisfies the same quadratic) or as a corollary at the end.

Facts we use

- ϕ is the positive root of $x^2 = x + 1$; hence $\phi^2 = \phi + 1$. Likewise, $\hat{\phi}^2 = \hat{\phi} + 1$.
- Fibonacci numbers satisfy $F_{n+1} = F_n + F_{n-1}$ with $F_0 = 0,\ F_1 = 1$. In §1.2.8 Knuth also suggests extending to negative indices by keeping the same recurrence (see ex. 8); this yields $F_{-n} = (-1)^{n+1} F_n$.

Induction for $n \geq 1$

Base. $n = 1$: \$ ^1 = = F_1 + F_0\$. True.

Induction step. Assume $\phi^n = F_n \phi + F_{n-1}$. Then

$$\phi^{n+1} = \phi \cdot \phi^n = \phi\,(F_n\phi + F_{n-1}) = F_n\phi^2 + F_{n-1}\phi = F_n(\phi+1) + F_{n-1}\phi = (F_n + F_{n-1})\phi + F_n = F_{n+1}\phi + F_n,$$

using $\phi^2 = \phi + 1$ and the Fibonacci recurrence. So the formula holds for all $n \geq 1$.

Extension to all integers

With the standard extension $F_{-n} = (-1)^{n+1} F_n$ (obtained by running the recurrence backwards), the identity at $n = 0$ reads $1 = \phi^0 = F_0\phi + F_{-1}$, so $F_{-1} = 1$, consistent with the extension. The same recurrence argument then propagates the identity to every negative n as well (uniqueness of solutions to $a_{n+1} = a_n + a_{n-1}$ given two consecutive values).

Equivalently, one can divide the proven formula for $n > 0$ by ϕ^{2n} and use $\hat{\phi} = -1/\phi$ to reach the negative indices.

The $\hat{\phi}$ identity

Repeat the same induction with $\hat{\phi}^2 = \hat{\phi} + 1$, or deduce it from the proven formula:

$$\hat{\phi}^n = (-1)^n \phi^{-n} = (-1)^n \frac{1}{F_n\phi + F_{n-1}} = F_{n+1} - F_n\phi = F_n(1-\phi) + F_{n-1} = F_n\hat{\phi} + F_{n-1},$$

since $F_{n+1} = F_n + F_{n-1}$ and $\hat{\phi} = 1 - \phi$. (The definition of $\hat{\phi}$ is on the same page as the generating-function derivation.)

(Optional cross-check via Binet)

Binet's formula gives $F_n = \frac{\phi^n - \hat{\phi}^n}{\sqrt{5}}$. Solving ϕ^n from $\phi^n - \hat{\phi}^n = \sqrt{5}\,F_n$ and using $\phi - \hat{\phi} = \sqrt{5}$ plus $\hat{\phi} = 1 - \phi$ yields $\phi^n = F_n\phi + F_{n-1}$ (and similarly for $\hat{\phi}^n$).

Thus both identities hold for every integer n.

12 [M26]

Generating-function solution

Let

- $F(z) = \sum_{n\geq 0} F_n z^n$ be the ordinary Fibonacci GF. In §1.2.8 Knuth derives

$$F(z) = \frac{z}{1 - z - z^2}. \quad \text{(GF of } F_n\text{)}$$

- $\mathbb{F}(z) = \sum_{n\geq 0} \mathbb{F}_n z^n$ be the GF of the second–order sequence.

From $\mathbb{F}_{n+2} = \mathbb{F}_{n+1} + \mathbb{F}_n + F_n$ with $\mathbb{F}_0 = 0,\ \mathbb{F}_1 = 1$,

$$\sum_{n\geq 0} \mathbb{F}_{n+2} z^{n+2} = \sum_{n\geq 0} \mathbb{F}_{n+1} z^{n+2} + \sum_{n\geq 0} \mathbb{F}_n z^{n+2} + \sum_{n\geq 0} F_n z^{n+2}.$$

In terms of $\mathbb{F}(z)$ and $F(z)$:

$$\mathbb{F}(z) - z \;=\; z\,\mathbb{F}(z) + z^2\,\mathbb{F}(z) + z^2\,F(z),$$

hence

$$\mathbb{F}(z) = \frac{z + z^2 F(z)}{1 - z - z^2} = \frac{z(1-z)}{(1 - z - z^2)^2}.$$

Using $F(z) = \dfrac{z}{1 - z - z^2}$, the last form is the neat identity

$$\boxed{\mathbb{F}(z) = \frac{1-z}{z}\,F(z)^2}.$$

Coefficient extraction now gives a closed form. Since $F(z) = \frac{1}{\sqrt{5}}\left(\frac{1}{1-\varphi z} - \frac{1}{1-\psi z}\right)$ with $\varphi = \frac{1+\sqrt{5}}{2},\ \psi = \frac{1-\sqrt{5}}{2}$,

$$F(z)^2 = \frac{1}{5}\left(\frac{1}{(1-\varphi z)^2} + \frac{1}{(1-\psi z)^2} - \frac{2}{(1-\varphi z)(1-\psi z)}\right).$$

Multiplying by $(1-z)/z$ and reading coefficients (using $[z^n](1-\alpha z)^{-2} = (n+1)\alpha^n$ and $[z^n]\,(1-z-z^2)^{-1} = F_{n+1}$) yields after a short algebraic simplification:

$$\boxed{\mathbb{F}_n = \frac{(3n+3)\,F_n - n\,F_{n+1}}{5}} \qquad (n \geq 0).$$

Recurrence (method of undetermined coefficients)

Solve $\mathbb{F}_{n+2} - \mathbb{F}_{n+1} - \mathbb{F}_n = F_n$.

- Homogeneous part $x^2 - x - 1 = 0$ has solutions φ^n, ψ^n, so $\mathbb{F}_n^{(h)} = c_1 F_n + c_2 F_{n+1}$.
- Because the RHS is a solution of the homogeneous equation, try a particular solution of the form $\mathbb{F}_n^{(p)} = a\,nF_n + b\,nF_{n+1}$. Plugging in gives

  $$(\mathbb{F}_{n+2}^{(p)} - \mathbb{F}_{n+1}^{(p)} - \mathbb{F}_n^{(p)}) = (a+3b)F_{n+1} + (2a+b)F_n.$$

  Matching this to F_n yields $a = 3/5,\ b = -1/5$, so $\mathbb{F}_n^{(p)} = \frac{n}{5}(3F_n - F_{n+1})$.

Thus $\mathbb{F}_n = \dfrac{n}{5}(3F_n - F_{n+1}) + c_1 F_n + c_2 F_{n+1}$. Initial conditions $\mathbb{F}_0 = 0,\ \mathbb{F}_1 = 1$ force $c_2 = 0$ and $c_1 = 3/5$, giving exactly

$$\boxed{\mathbb{F}_n = \frac{(3n+3)\,F_n - n\,F_{n+1}}{5}}.$$

Quick check (first values)

The formula gives $\mathbb{F}_n : 0, 1, 1, 3, 5, 10, 18, 33, 59, 105, \ldots$, which indeed satisfies $\mathbb{F}_{n+2} = \mathbb{F}_{n+1} + \mathbb{F}_n + F_n$ and the seeds.

13 [M22]

We'll use the fact that any sequence satisfying the Fibonacci recurrence $x_{n+2} = x_{n+1} + x_n$ is a linear combination of consecutive Fibonacci numbers:

$$x_n = \alpha F_n + \beta F_{n+1},$$

because this form obeys the same recurrence and the constants α, β can be fixed from two initial values.

Recall $F_0 = 0,\ F_1 = 1$.

(a) Homogeneous recurrence

Assume

$$a_n = \alpha F_n + \beta F_{n+1}.$$

Match the initials:

$$a_0 = r = \alpha F_0 + \beta F_1 = \beta \ \Rightarrow\ \beta = r,$$

$$a_1 = s = \alpha F_1 + \beta F_2 = \alpha + r \ \Rightarrow\ \alpha = s - r.$$

Hence

$$\boxed{a_n = r\,F_{n+1} + (s-r)\,F_n} \qquad (n \geq 0).$$

Check. $n = 0:\ rF_1 + (s-r)F_0 = r$. $n = 1:\ rF_2 + (s-r)F_1 = r + s - r = s$. The recurrence holds because any $\alpha F_n + \beta F_{n+1}$ satisfies $x_{n+2} = x_{n+1} + x_n$.

Equivalent forms sometimes seen are $a_n = sF_n + rF_{n-1}$ (if one extends $F_{-1} = 1$), but the boxed form avoids negative indices.

(b) Inhomogeneous recurrence

The nonzero constant c suggests removing a constant particular solution. Try $b_n \equiv t$:

$$t = t + t + c \;\Rightarrow\; t = -c.$$

Set $u_n = b_n + c$. Then

$$u_{n+2} = b_{n+2} + c = (b_{n+1} + b_n + c) + c = u_{n+1} + u_n,$$

so $\{u_n\}$ satisfies the Fibonacci recurrence, with

$$u_0 = b_0 + c = c, \qquad u_1 = b_1 + c = 1 + c.$$

Apply the result from (a):

$$u_n = (u_1 - u_0)F_n + u_0 F_{n+1} = F_n + c\,F_{n+1}.$$

Finally shift back:

$$\boxed{b_n = F_n + c\,F_{n+1} - c = F_n + c\,(F_{n+1} - 1)} \qquad (n \geq 0).$$

Check. $n = 0:\; F_0 + cF_1 - c = 0$. $n = 1:\; F_1 + cF_2 - c = 1$. And $b_{n+2} - b_{n+1} - b_n = (F_{n+2} - F_{n+1} - F_n) + c(F_{n+3} - F_{n+2} - F_{n+1}) - c = c$.

14 [M28]

Introduce the "binomial diagonal" sum

$$S_n := \sum_{r=0}^{m} \binom{n+r}{m-r} = \binom{n}{m} + \binom{n+1}{m-1} + \cdots + \binom{n+m}{0}.$$

By Pascal's identity $\binom{x+1}{y} = \binom{x}{y} + \binom{x}{y-1}$,

$$\begin{aligned} S_{n+2} &= \sum_{r=0}^{m} \binom{n+2+r}{m-r} = \sum_{r=0}^{m} \Big[\binom{n+1+r}{m-r} + \binom{n+1+r}{m-r-1}\Big] \\ &= S_{n+1} + \sum_{r=0}^{m-1} \binom{n+1+r}{m-r-1} = S_{n+1} + \sum_{t=1}^{m} \binom{n+t}{m-t} = S_{n+1} + S_n - \binom{n}{m}. \end{aligned}$$

Hence

$$\boxed{S_{n+2} - S_{n+1} - S_n = -\binom{n}{m}}.$$

Now set $b_n := a_n + S_n$. Then

$$b_{n+2} = (a_{n+2} + S_{n+2}) = (a_{n+1} + a_n + \binom{n}{m}) + (S_{n+1} + S_n - \binom{n}{m}) = b_{n+1} + b_n.$$

So $\{b_n\}$ satisfies the homogeneous Fibonacci recurrence.

2) Initial values of b_n

We need S_0 and S_1. These are classic "shallow diagonal" sums in Pascal's triangle:

$$S_0 = \sum_{r=0}^{m} \binom{r}{m-r} = F_{m+1}, \qquad S_1 = \sum_{r=0}^{m} \binom{1+r}{m-r} = F_{m+2},$$

(which follows from $\sum_{k\geq 0} \binom{n-k}{k} = F_{n+1}$ by a one-line induction).

Thus

$$b_0 = a_0 + S_0 = F_{m+1}, \qquad b_1 = a_1 + S_1 = 1 + F_{m+2}.$$

3) Solve the Fibonacci-type initial value problem

Every solution of $x_{n+2} = x_{n+1} + x_n$ is a linear combination of F_{n-1} and F_n:

$$b_n = b_0 F_{n-1} + b_1 F_n = F_{m+1} F_{n-1} + (1 + F_{m+2}) F_n.$$

Use the addition identity $F_{m+1}F_{n-1} + F_{m+2}F_n = F_{m+n+1}$ to get

$$b_n = F_{m+n+1} + F_n.$$

Finally,

$$a_n = b_n - S_n = \boxed{F_{m+n+1} + F_n - \sum_{r=0}^{m} \binom{n+r}{m-r}}.$$

Equivalently, writing out the diagonal sum,

$$\boxed{a_n = F_{m+n+1} + F_n - \left(\binom{n}{m} + \binom{n+1}{m-1} + \cdots + \binom{n+m}{0}\right)}.$$

This matches Knuth's official answer for 1.2.8–14.

4) Quick check (e.g., $m = 1$)

Then $\binom{n}{1} = n$ and $S_n = \binom{n}{1} + \binom{n+1}{0} = n + 1$. Our formula gives

$$a_n = F_{n+2} + F_n - (n + 1).$$

Compute from the recurrence: $a_0 = 0$, $a_1 = 1$, $a_2 = a_1 + a_0 + \binom{0}{1} = 1$, $a_3 = a_2 + a_1 + \binom{1}{1} = 3$, … The closed form yields $a_2 = F_4 + F_2 - 3 = 3 + 1 - 3 = 1$, $a_3 = F_5 + F_3 - 4 = 5 + 2 - 4 = 3$.

15 [M22]

Let the linear operator

$$(Lu)_n := u_{n+2} - u_{n+1} - u_n.$$

Then L is linear, and the three recurrences say

$$La = f, \qquad Lb = g, \qquad Lc = x\,f + y\,g,$$

with the common initial conditions $a_0 = b_0 = c_0 = 0$ and $a_1 = b_1 = c_1 = 1$. Also, for Fibonacci F we have $LF = 0$ and $F_0 = 0,\ F_1 = 1$.

Consider the candidate

$$d_n \ := \ x\,a_n + y\,b_n + (1 - x - y)\,F_n.$$

Because L is linear,

$$Ld \ = \ x\,La + y\,Lb + (1 - x - y)LF \ = \ x\,f + y\,g,$$

and the initial values match:

$$d_0 = x \cdot 0 + y \cdot 0 + (1 - x - y) \cdot 0 = 0, \qquad d_1 = x \cdot 1 + y \cdot 1 + (1 - x - y) \cdot 1 = 1.$$

Thus d and c satisfy the same inhomogeneous recurrence with the same initial conditions, hence $d_n = c_n$ for all n.

Therefore the requested expression is

$$\boxed{c_n \;=\; x\,a_n \;+\; y\,b_n \;+\; (1-x-y)\,F_n}\,.$$

(Optional) Convolution check

One can also derive the standard particular-solution form: if $u_{n+2} = u_{n+1} + u_n + h(n)$ with $u_0 = 0, u_1 = 1$, then

$$u_n \;=\; F_n \;+ \sum_{k=0}^{n-2} F_{n-1-k}\, h(k).$$

Applying this to a, b, c (with $h = f, g, xf + yg$) gives

$$a_n = F_n + \sum_k F_{n-1-k} f(k), \quad b_n = F_n + \sum_k F_{n-1-k} g(k),$$

$$c_n = F_n + x \sum_k F_{n-1-k} f(k) + y \sum_k F_{n-1-k} g(k) = x a_n + y b_n + (1-x-y) F_n,$$

agreeing with the result.

16 [M20]

Here is Exercise 16 from §1.2.8 (Fibonacci Numbers): show that

$$S_n \;\stackrel{\text{def}}{=}\; \sum_{k=0}^{n} \binom{n-k}{k}$$

is a Fibonacci number. (This is the "right-angled" view of Pascal's triangle.)

We'll prove the precise identity

$$S_n = F_{n+1},$$

in three complementary ways.

1) Recurrence proof (via Pascal's identity)

Let $S_n = \sum_{k=0}^{n} \binom{n-k}{k}$. Using $\binom{a+1}{b} = \binom{a}{b} + \binom{a}{b-1}$,

$$\begin{aligned} S_{n+1} &= \sum_{k=0}^{n+1} \binom{n+1-k}{k} = \sum_{k=0}^{n+1} \Big[\binom{n-k}{k} + \binom{n-k}{k-1} \Big] \\ &= \underbrace{\sum_{k=0}^{n+1} \binom{n-k}{k}}_{=S_n} + \underbrace{\sum_{k=1}^{n+1} \binom{n-k}{k-1}}_{j=k-1 \Rightarrow \sum_{j=0}^{n} \binom{n-1-j}{j} = S_{n-1}} \\ &= S_n + S_{n-1}. \end{aligned}$$

Initial values: $S_0 = \binom{0}{0} = 1$ and $S_1 = \binom{1}{0} + \binom{0}{1} = 1$. Thus (S_n) satisfies the Fibonacci recurrence with $S_0 = 1, S_1 = 1$, hence $S_n = F_{n+1}$.

2) Combinatorial proof (tilings)

Count tilings of a $1 \times n$ strip by monominoes (length 1) and dominoes (length 2).

- If a tiling uses k dominoes, it has $n - 2k$ monominoes, hence $n - k$ tiles in total; choosing which k of these $n - k$ positions are dominoes yields $\binom{n-k}{k}$ tilings.
- Summing over all feasible k gives exactly $\sum_{k \geq 0} \binom{n-k}{k} = S_n$.

On the other hand, these tilings are counted by the Fibonacci recurrence: a tiling of length n either starts with a monomino (then continue with a tiling of length $n-1$) or with a domino (then continue with a tiling of length $n-2$). With $T_0 = 1$, $T_1 = 1$, we have $T_n = F_{n+1}$. Therefore $S_n = T_n = F_{n+1}$.

3) Generating-function proof

Define $S(z) = \sum_{n \geq 0} S_n z^n$. Then

$$\begin{aligned} S(z) &= \sum_{n \geq 0} \sum_{k \geq 0} \binom{n-k}{k} z^n = \sum_{k \geq 0} \sum_{m \geq 0} \binom{m}{k} z^{m+k} = \sum_{k \geq 0} z^k \left(\sum_{m \geq 0} \binom{m}{k} z^m \right) \\ &= \sum_{k \geq 0} z^k \frac{z^k}{(1-z)^{k+1}} = \frac{1}{1-z} \sum_{k \geq 0} \left(\frac{z^2}{1-z} \right)^k = \frac{1}{1-z} \cdot \frac{1}{1 - \frac{z^2}{1-z}} = \frac{1}{1-z-z^2}. \end{aligned}$$

But $\sum_{n \geq 0} F_{n+1} z^n = \dfrac{1}{1-z-z^2}$ (since $G(z) = \sum_{n \geq 0} F_n z^n = \dfrac{z}{1-z-z^2}$). Hence $S_n = F_{n+1}$.

Therefore, for every $n \geq 0$,

$$\boxed{\sum_{k=0}^{n} \binom{n-k}{k} = F_{n+1}.}$$

(Quick check: $n = 0, 1, 2, 3$ give $1, 1, 2, 3$, matching F_1, F_2, F_3, F_4 respectively.)

17 [M24]

We must show, for all integers m, n, k,

$$F_{n+k}F_{m-k} - F_nF_m \;=\; (-1)^n\, F_{m-n-k}\, F_k. \tag{ }$$

Ingredients

1. Addition formula (valid for all integers):

$$F_{r+s} = F_rF_{s+1} + F_{r-1}F_s. \tag{A}$$

2. d'Ocagne's identity (valid for all integers):

$$F_aF_{b+1} - F_{a+1}F_b \;=\; (-1)^b\, F_{a-b}. \tag{D}$$

3. Negative indices convention:

$$F_{-t} = (-1)^{t+1}F_t \quad \text{for all integers } t \geq 0. \tag{N}$$

Start from the left–hand side of ( ) and expand only the factors that "carry" k using (A):

$$\begin{aligned} F_{n+k}F_{m-k} - F_nF_m &= (F_nF_{k+1} + F_{n-1}F_k)F_{m-k} \;-\; F_n(F_{m-k}F_{k+1} + F_{m-k-1}F_k) \\ &= \underbrace{F_nF_{k+1}F_{m-k}}_{\text{cancels}} + F_{n-1}F_kF_{m-k} - \underbrace{F_nF_{m-k}F_{k+1}}_{\text{cancels}} - F_nF_{m-k-1}F_k \\ &= F_k(F_{n-1}F_{m-k} - F_nF_{m-k-1}). \end{aligned} \tag{1}$$

Apply d'Ocagne (D) with $a = n-1$ and $b = m-k-1$:

$$F_{n-1}F_{m-k} - F_nF_{m-k-1} = (-1)^{m-k-1}\, F_{(n-1)-(m-k-1)} = (-1)^{m-k-1}\, F_{n-m+k}. \tag{2}$$

Insert (2) into (1):

$$F_{n+k}F_{m-k} - F_nF_m = F_k \cdot (-1)^{m-k-1}F_{n-m+k}. \tag{3}$$

Now use the negative-index rule (N): since $m - n - k = -(n - m + k)$,

$$F_{n-m+k} = F_{-(m-n-k)} = (-1)^{(m-n-k)+1}F_{m-n-k}. \tag{4}$$

Combine (3) and (4): the total sign exponent is

$$(m - k - 1) + (m - n - k + 1) \;=\; n,$$

so

$$F_{n+k}F_{m-k} - F_nF_m = (-1)^n\, F_k\, F_{m-n-k},$$

which is exactly ( ).

Checks and corollaries

- $k = 0$: LHS $= F_nF_m - F_nF_m = 0$; RHS $= (-1)^n F_{m-n}F_0 = 0$.
- $k = 1$ (d'Ocagne's classical form):

$$F_{n+1}F_{m-1} - F_nF_m = (-1)^n F_{m-n-1}.$$

- Generalized Cassini (take $m = n$):

$$F_{n+k}F_{n-k} - F_n^2 = (-1)^n F_{-k}F_k = (-1)^{n+k+1}F_k^2,$$

  a well-known extension of Cassini's identity.

This completes the proof.

18 [20]

Yes. In fact,

$$F_n^2 + F_{n+1}^2 \;=\; F_{2n+1} \qquad \text{for all } n \geq 0.$$

This holds under either common indexing convention ($F_0 = 0, F_1 = 1$ or $F_1 = 1, F_2 = 1$).

Proof (via Fibonacci addition formula)

We use the standard addition identity

$$F_{r+s} = F_r F_{s+1} + F_{r-1} F_s \quad \text{for all } r, s \geq 0.$$

Proof of the identity (induction on s): For $s = 0$: $F_{r+0} = F_r = F_r F_1 + F_{r-1} F_0$ (true). Assume true for s. Then

$$\begin{aligned} F_{r+(s+1)} &= F_{r+s+1} = F_{r+s} + F_{r+s-1} \\ &= (F_r F_{s+1} + F_{r-1} F_s) + (F_r F_s + F_{r-1} F_{s-1}) \\ &= F_r (F_{s+1} + F_s) + F_{r-1} (F_s + F_{s-1}) \\ &= F_r F_{s+2} + F_{r-1} F_{s+1}, \end{aligned}$$

so the identity holds for $s + 1$. By induction, it holds for all s.

Now take $r = n + 1$ and $s = n$. Then

$$F_{(n+1)+n} = F_{2n+1} = F_{n+1} F_{n+1} + F_n F_n = F_{n+1}^2 + F_n^2,$$

which is exactly the claim.

Alternative proof (matrix method, one line)

Let $Q = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$. Then

$$Q^k = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}.$$

Compute $Q^{2n+1} = Q^n Q^{n+1}$ and look at the $(1, 2)$ entry:

$$(F_{n+1})(F_{n+1}) + (F_n)(F_n) = F_{2n+1}.$$

Quick check

- $n = 0:\ 0^2 + 1^2 = 1 = F_1 = F_{2\cdot 0+1}$.
- $n = 1:\ 1^2 + 1^2 = 2 = F_3$.
- $n = 2:\ 1^2 + 2^2 = 5 = F_5$.
- $n = 3:\ 2^2 + 3^2 = 13 = F_7$.

19 [M27]

Let $\theta = 36^\circ$ and $c = \cos\theta$. The Chebyshev identity

$$\cos 5\theta \ = \ 16c^5 - 20c^3 + 5c$$

gives, since $5\theta = 180^\circ$ and $\cos 180^\circ = -1$,

$$16c^5 - 20c^3 + 5c + 1 = 0.$$

Set $u = 2c$. Substituting $c = u/2$ yields

$$u^5 - 5u^3 + 5u + 2 = 0.$$

This factors as

$$(u^2 - u - 1)(u^3 + u^2 - 3u - 2) = 0.$$

Hence u satisfies $u^2 - u - 1 = 0$, so

$$u = \frac{1+\sqrt{5}}{2} = \varphi,$$

the golden ratio. (The other quadratic root is negative and doesn't match $u = 2\cos\theta \in (0, 2]$.) Therefore

$$\boxed{\cos 36^\circ \ = \ \frac{u}{2} \ = \ \frac{\varphi}{2} \ = \ \frac{1+\sqrt{5}}{4}}.$$

By TAOCP's notation, $\varphi = \frac{1}{2}(1 + \sqrt{5})$.

Geometric check (golden triangle)

In a regular pentagon the ratio of diagonal to side is φ. Considering the isosceles triangle formed by two diagonals (equal sides) and one side (base) with vertex angle 36°, the law of cosines gives

$$1^2 = 2\varphi^2\,(1-\cos 36^\circ) \implies \cos 36^\circ = 1 - \frac{1}{2\varphi^2} = \frac{\varphi}{2},$$

using $\varphi^2 = \varphi + 1$. Related values (nice to know)

From $\cos 36^\circ = \varphi/2$ we get, equivalently,

$$\cos 72^\circ = \frac{1}{2\varphi} = \frac{\sqrt{5}-1}{4}, \qquad \sin 36^\circ = \sqrt{\frac{10-2\sqrt{5}}{16}}.$$

20 [M16]

$$\boxed{\sum_{k=0}^{n} F_k \;=\; F_{n+2} \;-\; 1}$$

(So the sum of the first $n+1$ Fibonacci numbers is the $(n+2)$-nd Fibonacci number minus 1.)

Proof 1 — One-line induction (using the recurrence)

Let $S_n = \sum_{k=0}^{n} F_k$. The Fibonacci recurrence gives

$$S_n = S_{n-1} + F_n.$$

Assume $S_{n-1} = F_{n+1} - 1$. Then

$$S_n = (F_{n+1} - 1) + F_n = F_{n+2} - 1,$$

since $F_{n+2} = F_{n+1} + F_n$. The base case $S_0 = F_0 = 0 = F_2 - 1$ holds, so by induction $S_n = F_{n+2} - 1$ for all $n \geq 0$.

(This uses only the standard definition $F_0 = 0,\ F_1 = 1,\ F_{n+2} = F_{n+1} + F_n$.)

Proof 2 — Generating functions

Let $G(z) = \sum_{n\geq 0} F_n z^n$. From the text,

$$G(z) = \frac{z}{1-z-z^2}.$$

If $S_n = \sum_{k=0}^{n} F_k$, then $S(z) = \sum_{n\geq 0} S_n z^n = \frac{G(z)}{1-z}$. Hence

$$S(z) = \frac{z}{(1-z)(1-z-z^2)}.$$

But $\sum_{n\geq 0}(F_{n+2}-1)z^n = \frac{G(z)-z}{z^2} - \frac{1}{1-z} = \frac{z}{(1-z)(1-z-z^2)}$ as well, so the coefficients agree: $S_n = F_{n+2} - 1$.

Proof 3 — Binet's formula (closed form)

Using $F_n = \dfrac{\phi^n - \phi'^n}{\sqrt{5}}$ with $\phi = \frac{1+\sqrt{5}}{2}$ and $\phi' = 1-\phi$, we have

$$\sum_{k=0}^{n} F_k = \frac{1}{\sqrt{5}}\left(\sum_{k=0}^{n}\phi^k - \sum_{k=0}^{n}\phi'^k\right) = \frac{1}{\sqrt{5}}\left(\frac{\phi^{n+1}-1}{\phi-1} - \frac{\phi'^{\,n+1}-1}{\phi'-1}\right).$$

Since $\phi - 1 = \frac{1}{\phi}$ and $\phi' - 1 = \frac{1}{\phi'}$, this simplifies to

$$\frac{1}{\sqrt{5}}(\phi^{n+2} - \phi - (\phi'^{\,n+2} - \phi')) = \frac{\phi^{n+2} - \phi'^{\,n+2}}{\sqrt{5}} - \frac{\phi - \phi'}{\sqrt{5}} = F_{n+2} - 1,$$

because $\phi - \phi' = \sqrt{5}$.

Quick check

For $n = 7$: $0+1+1+2+3+5+8+13 = 33$, and $F_9 - 1 = 34 - 1 = 33$.

21 [M25]

Evaluate

$$S_n(x) \;=\; \sum_{k=1}^{n} F_k\, x^k,$$

where $F_0 = 0$, $F_1 = 1$, and $F_k = F_{k-1} + F_{k-2}$ for $k \geq 2$.

1) Main case $x^2 + x - 1 \neq 0$

Start with

$$S_n = \sum_{k=1}^{n} F_k x^k, \quad xS_n = \sum_{k=1}^{n} F_k x^{k+1} = \sum_{k=2}^{n+1} F_{k-1} x^k, \quad x^2 S_n = \sum_{k=3}^{n+2} F_{k-2} x^k.$$

Consider $(x^2+x-1)S_n = x^2 S_n + xS_n - S_n$. For the coefficients of x^k with $3 \leq k \leq n$, the Fibonacci recurrence gives perfect cancellation:

$$F_{k-2} + F_{k-1} - F_k = 0.$$

Only boundary terms remain:

- $k = 1$: contributes $-F_1 x = -x$.
- $k = 2$: contributes $F_1 x^2 - F_2 x^2 = 0$.
- $k = n + 1$: contributes $(F_{n-1} + F_n)x^{n+1} = F_{n+1}x^{n+1}$.
- $k = n + 2$: contributes $F_n x^{n+2}$.

Hence

$$(x^2 + x - 1)\, S_n \;=\; F_{n+1}x^{n+1} + F_n x^{n+2} - x,$$

and, when $x^2 + x - 1 \neq 0$,

$$\boxed{S_n(x) = \frac{x^{n+1}F_{n+1} + x^{n+2}F_n - x}{x^2 + x - 1}.}$$

(This matches Knuth's posted answer for the exercise.)

A good sanity check is $x = 1$:

$$S_n(1) = \sum_{k=1}^{n} F_k = \frac{F_{n+1} + F_n - 1}{1 + 1 - 1} = F_{n+2} - 1,$$

the classic identity (this is Ex. 20's result).

2) The exceptional case $x^2 + x - 1 = 0$

The denominator vanishes at the roots of $x^2 + x - 1 = 0$, namely

$$x = \frac{-1 \pm \sqrt{5}}{2} = \frac{1}{\varphi} \quad \text{or} \quad \frac{1}{\varphi'},$$

with $\varphi = \frac{1+\sqrt{5}}{2}$ and $\varphi' = \frac{1-\sqrt{5}}{2}$. Here the numerator vanishes as well, so take the limit (e.g., by l'Hôpital). Differentiating numerator and denominator and simplifying yields

$$\boxed{S_n(x) = \frac{n + 1 - x^n F_{n+1}}{2x + 1} \quad \text{when } x^2 + x - 1 = 0.}$$

(This is exactly the special-case expression reported in the book's answer section.)

3) Remarks

- Letting $n \to \infty$ with $|x| < 1/\varphi$ recovers the ordinary generating function $\sum_{k\geq 1} F_k x^k = \dfrac{x}{1-x-x^2}$.
- The telescoping proof above is robust and avoids Binet's formula; it's the standard trick for finite "truncated generating functions."

Final answer

$$\boxed{\sum_{k=1}^{n} F_k x^k = \begin{cases} \dfrac{x^{n+1}F_{n+1} + x^{n+2}F_n - x}{x^2+x-1}, & x^2+x-1 \neq 0, \\ \dfrac{n+1-x^n F_{n+1}}{2x+1}, & x^2+x-1 = 0 \end{cases},}$$

with the check $\sum_{k=1}^{n} F_k = F_{n+2} - 1$ at $x = 1$.

22 [M20]

Proof via Binet's formula

Recall Binet's formula

$$F_r = \frac{\phi^r - \hat{\phi}^r}{\sqrt{5}}, \qquad \phi = \frac{1+\sqrt{5}}{2}, \quad \hat{\phi} = 1 - \phi = \frac{1-\sqrt{5}}{2}.$$

Both ϕ and $\hat{\phi}$ satisfy $x^2 = x + 1$, so $1 + \phi = \phi^2$ and $1 + \hat{\phi} = \hat{\phi}^2$.

Then

$$\begin{aligned} S &= \sum_{k=0}^{n} \binom{n}{k} \frac{\phi^{m+k} - \hat{\phi}^{m+k}}{\sqrt{5}} = \frac{1}{\sqrt{5}} \left(\phi^m \sum_{k=0}^{n} \binom{n}{k} \phi^k - \hat{\phi}^m \sum_{k=0}^{n} \binom{n}{k} \hat{\phi}^k \right) \\ &= \frac{1}{\sqrt{5}} \Big(\phi^m (1+\phi)^n - \hat{\phi}^m (1+\hat{\phi})^n \Big) = \frac{1}{\sqrt{5}} (\phi^m \phi^{2n} - \hat{\phi}^m \hat{\phi}^{2n}) = \frac{\phi^{m+2n} - \hat{\phi}^{m+2n}}{\sqrt{5}} \\ &= F_{m+2n}. \end{aligned}$$

Thus the sum is exactly the Fibonacci number F_{m+2n}.

(Optional) Recurrence/induction proof (no Binet)

Define $S_n(m) = \sum_{k=0}^{n} \binom{n}{k} F_{m+k}$. Pascal's identity gives

$$S_{n+1}(m) = \sum_k (\binom{n}{k} + \binom{n}{k-1}) F_{m+k} = S_n(m) + S_n(m+1).$$

Base cases: $S_0(m) = F_m$ and $S_1(m) = F_m + F_{m+1} = F_{m+2}$. The sequence $T_n(m) := F_{m+2n}$ satisfies the same recurrence $T_{n+1}(m) = T_n(m) + T_n(m+1)$ and the same bases $T_0(m) = F_m,\ T_1(m) = F_{m+2}$. By induction on n, $S_n(m) = T_n(m) = F_{m+2n}$.

Sanity check. $n = 2$: $F_m + 2F_{m+1} + F_{m+2} = F_{m+4}$ (true since $F_{m+4} = F_{m+3} + F_{m+2} = (F_{m+2} + F_{m+1}) + (F_{m+1} + F_m)$).

23 [M23]

We use Binet's formula and one standard identity for powers of φ (the golden ratio):

$$F_r \;=\; \frac{\varphi^r - \hat{\varphi}^r}{\sqrt{5}}, \qquad \hat{\varphi} = 1 - \varphi,$$

and

$$\varphi^t = F_t\,\varphi + F_{t-1}, \qquad \hat{\varphi}^t = F_t\,\hat{\varphi} + F_{t-1}.$$

(See the derivation of Binet's formula and related identities for φ in §1.2.8.)

Start from the left–hand side and insert Binet's formula for F_{m+k}:

$$\sum_{k=0}^{n} \binom{n}{k} F_{m+k}\, F_t^k\, F_{t-1}^{n-k} = \frac{1}{\sqrt{5}} \sum_{k=0}^{n} \binom{n}{k} (\varphi^{m+k} - \hat{\varphi}^{m+k}) F_t^k F_{t-1}^{n-k}.$$

Separate the φ-part and the $\hat{\varphi}$-part, and factor out φ^m and $\hat{\varphi}^m$:

$$= \frac{1}{\sqrt{5}} \left[\varphi^m \sum_{k=0}^{n} \binom{n}{k} (\varphi F_t)^k F_{t-1}^{n-k} \;-\; \hat{\varphi}^m \sum_{k=0}^{n} \binom{n}{k} (\hat{\varphi} F_t)^k F_{t-1}^{n-k} \right].$$

Apply the binomial theorem to each sum:

$$\sum_{k=0}^{n} \binom{n}{k} (\varphi F_t)^k F_{t-1}^{n-k} = (\varphi F_t + F_{t-1})^n = (\varphi^t)^n = \varphi^{tn},$$

and similarly $(\hat{\varphi} F_t + F_{t-1})^n = (\hat{\varphi}^t)^n = \hat{\varphi}^{tn}$. Therefore

$$\sum_{k=0}^{n} \binom{n}{k} F_{m+k}\, F_t^k\, F_{t-1}^{n-k} = \frac{1}{\sqrt{5}}(\varphi^m \varphi^{tn} - \hat{\varphi}^m \hat{\varphi}^{tn}) = \frac{1}{\sqrt{5}}(\varphi^{m+tn} - \hat{\varphi}^{m+tn}) = F_{m+tn},$$

which is exactly the required identity.

Quick check

Take $m = 1$, $n = 2$, $t = 3$. LHS:

$$\binom{2}{0} F_1\, 2^0\, 1^2 + \binom{2}{1} F_2\, 2^1\, 1^1 + \binom{2}{2} F_3\, 2^2\, 1^0 = 1 + 4 + 8 = 13,$$

RHS: $F_{1+3\cdot 2} = F_7 = 13$.

24 [HM20]

Let D_n be the determinant of the $n \times n$ tridiagonal matrix

$$A_n = \begin{pmatrix} 1 & -1 & 0 & \cdots & 0 \\ 1 & 1 & -1 & \ddots & \vdots \\ 0 & 1 & 1 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -1 \\ 0 & \cdots & 0 & 1 & 1 \end{pmatrix}.$$

Show that $D_n = F_{n+1}$, the $(n+1)$-st Fibonacci number.

Step 1: Set up a clean recurrence for D_n

Expand $D_n = \det(A_n)$ along the first row:

$$D_n = (1) \cdot \det(M_{11}) + (-1) \cdot C_{12},$$

where M_{11} is the minor obtained by deleting row 1, column 1, and C_{12} is the cofactor term for entry $(1, 2)$. By the usual cofactor sign, $C_{12} = (-1)^{1+2} \det(M_{12}) = -\det(M_{12})$. Hence

$$D_n = \det(M_{11}) + \det(M_{12}). \tag{1}$$

- Identify $\det(M_{11})$. Deleting row 1, column 1 from A_n leaves exactly the same tridiagonal pattern of size $n - 1$. Thus

$$\det(M_{11}) = D_{n-1}.$$

- Identify $\det(M_{12})$. Deleting row 1, column 2 yields a matrix whose first column has a single 1 at its top and zeros below. Expanding *that* minor along its first column removes that top row and left column, leaving again the same pattern of size $n-2$. Therefore

$$\det(M_{12}) = D_{n-2}.$$

Plug these into (1):

$$\boxed{D_n = D_{n-1} + D_{n-2} \qquad (n \geq 3).} \tag{2}$$

This is exactly the Fibonacci recurrence.

Step 2: Initial conditions

Compute the first few determinants directly:

- $n = 1$: $A_1 = [1]$, so $D_1 = 1 = F_2$.
- $n = 2$: $A_2 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$, so $D_2 = 1 \cdot 1 - (-1) \cdot 1 = 2 = F_3$.

Thus

$$\boxed{D_1 = 1 = F_2, \qquad D_2 = 2 = F_3.} \tag{3}$$

Step 3: Conclude by induction (or by "same recurrence, same bases")

The sequence (D_n) satisfies the Fibonacci recurrence (2) with initial values (3) matching the Fibonacci shift. Therefore, by a trivial induction,

$$\boxed{D_n = F_{n+1} \quad \text{for all } n \geq 1.}$$

25 [M21]

Let

$$\alpha = \frac{1+\sqrt{5}}{2}, \qquad \beta = \frac{1-\sqrt{5}}{2}.$$

Then $F_n = \dfrac{\alpha^n - \beta^n}{\alpha - \beta}$ and $\alpha - \beta = \sqrt{5}$. Multiply by 2^n:

$$2^n F_n \;=\; \frac{(2\alpha)^n - (2\beta)^n}{\alpha - \beta} \;=\; \frac{(1+\sqrt{5})^n - (1-\sqrt{5})^n}{\sqrt{5}}.$$

Expand each term with the binomial theorem:

$$(1 \pm \sqrt{5})^n = \sum_{k=0}^{n} \binom{n}{k} (\pm\sqrt{5})^k = \sum_{k=0}^{n} \binom{n}{k} (\pm 1)^k (\sqrt{5})^k.$$

Subtracting,

$$(1+\sqrt{5})^n - (1-\sqrt{5})^n = \sum_{k=0}^{n} \binom{n}{k} (1-(-1)^k)(\sqrt{5})^k.$$

For even k, $1-(-1)^k = 0$; for odd k, $1-(-1)^k = 2$. Hence

$$(1+\sqrt{5})^n - (1-\sqrt{5})^n = 2 \sum_{\substack{k=0 \\ k \text{ odd}}}^{n} \binom{n}{k} (\sqrt{5})^k.$$

Divide by $\sqrt{5}$:

$$2^n F_n = \frac{1}{\sqrt{5}} \cdot 2 \sum_{k \text{ odd}} \binom{n}{k} (\sqrt{5})^k = 2 \sum_{k \text{ odd}} \binom{n}{k} (\sqrt{5})^{\,k-1} = 2 \sum_{k \text{ odd}} \binom{n}{k} 5^{(k-1)/2},$$

which is the desired identity. $\square$

Equivalent reindexing

Writing $k = 2j+1$ (so $j = 0, 1, \ldots, \lfloor (n-1)/2 \rfloor$) gives the neat form

$$2^n F_n = 2 \sum_{j=0}^{\lfloor (n-1)/2 \rfloor} \binom{n}{2j+1} 5^j.$$

Quick sanity checks

- $n = 1$: $2^1 F_1 = 2$. RHS $= 2\binom{1}{1}5^0 = 2$.
- $n = 3$: $2^3 F_3 = 8 \cdot 2 = 16$. RHS $= 2(\binom{3}{1}5^0 + \binom{3}{3}5^1) = 2(3+5) = 16$.

26 [M20]

1) Start from Exercise 25's identity. From the previous exercise we may use the equivalent closed form

$$2^n \sqrt{5}\, F_n = (1+\sqrt{5})^n - (1-\sqrt{5})^n.$$

(Indeed, expanding the right side by the binomial theorem cancels the even terms and yields Exercise 25's sum.)

2) Work in the ring $\mathbb{F}_p[\sqrt{5}] = (\mathbb{Z}/p\mathbb{Z})[x]/(x^2-5)$. Reduce the identity with $n = p$ modulo p. By the binomial theorem modulo a prime (all interior binomial coefficients are multiples of p),

$$(1 \pm \sqrt{5})^p \equiv 1 \pm (\sqrt{5})^p \pmod{p}.$$

(This is the property referenced as §1.2.6–10(b).) Hence

$$2^p \sqrt{5}\, F_p \equiv [1 + (\sqrt{5})^p] - [1 - (\sqrt{5})^p] = 2(\sqrt{5})^p \pmod{p}.$$

3) Simplify the powers of $\sqrt{5}$. Because $(\sqrt{5})^p = \sqrt{5} \cdot (\sqrt{5})^{p-1} = \sqrt{5} \cdot 5^{(p-1)/2}$, the congruence becomes

$$2^p \sqrt{5}\, F_p \equiv 2\sqrt{5} \cdot 5^{(p-1)/2} \pmod{p}.$$

4) Cancel $\sqrt{5}$ and 2. If $p \neq 5$, then $5 \not\equiv 0$ so $(\sqrt{5})^2 = 5$ is a unit in $\mathbb{F}_p[\sqrt{5}]$; hence $\sqrt{5}$ is a unit and can be canceled. Also 2 is invertible since p is odd. We get

$$2^{p-1} F_p \equiv 5^{(p-1)/2} \pmod{p}.$$

5) Apply Fermat's little theorem. $2^{p-1} \equiv 1 \pmod{p}$, so finally

$$\boxed{F_p \equiv 5^{(p-1)/2} \pmod{p}}.$$

6) The case $p = 5$. Then $F_5 = 5 \equiv 0 \pmod 5$ and $5^{(5-1)/2} = 5^2 \equiv 0 \pmod 5$, so the congruence still holds.

Quick checks: $p = 3 : F_3 = 2 \equiv 5^1 \equiv 2 \pmod 3$; $p = 7 : F_7 = 13 \equiv 6 \equiv 5^3 \pmod 7$; $p = 11 : F_{11} = 89 \equiv 1 \equiv 5^5 \pmod{11}$.

27 [M20]

Facts we'll use

1. Cassini's identity. For all integers n,

$$F_{n+1}F_{n-1} - F_n^2 = (-1)^n.$$

This appears in the section text (with a quick proof by induction and a matrix proof).

2. Result from Exercise 26. For an odd prime p,

$$F_p \equiv 5^{(p-1)/2} \pmod{p}.$$

(Exercise statement.)

3. Euler's criterion (standard number theory). For an odd prime p and integer a not divisible by p,

$$a^{(p-1)/2} \equiv \left(\frac{a}{p}\right) \pmod{p},$$

where $\left(\frac{a}{p}\right)$ is the Legendre symbol and equals ± 1.

Combining (2) with Euler's criterion for $a = 5$ gives

$$F_p \equiv 5^{(p-1)/2} \equiv \left(\frac{5}{p}\right) \in \{+1, -1\} \pmod{p}.$$

So for $p \neq 5$, we have

$$F_p^2 \equiv 1 \pmod{p}. \qquad (\ )$$

Step 1: p divides $F_{p-1}F_{p+1}$

Apply Cassini's identity with $n = p$ (note p odd):

$$F_{p-1}F_{p+1} - F_p^2 = (-1)^p = -1.$$

Reduce modulo p and use ():

$$F_{p-1}F_{p+1} \equiv F_p^2 - 1 \equiv 0 \pmod{p}.$$

Hence $p \mid F_{p-1}F_{p+1}$; i.e., at least one of F_{p-1}, F_{p+1} is divisible by p. (This is exactly the route hinted in the book's answers.)

Step 2: They cannot both be divisible by p

Suppose, for contradiction, that $p \mid F_{p-1}$ and $p \mid F_{p+1}$. Then from the Fibonacci recurrence,

$$F_{p+1} = F_p + F_{p-1} \equiv 0 \pmod{p} \quad \text{and} \quad F_{p-1} \equiv 0 \pmod{p}$$

would imply $F_p \equiv 0 \pmod{p}$. But we already have $F_p \equiv \pm 1 \pmod{p}$ for $p \neq 5$, a contradiction. Therefore exactly one of F_{p-1}, F_{p+1} is a multiple of p.

Edge cases

- $p = 2$: $F_1 = 1$ and $F_3 = 2$. Here $F_{p+1} = 2$ is divisible by 2 while F_{p-1} is not—so the statement is true. (The book notes this explicitly.)
- $p = 5$: excluded in the exercise; indeed $F_4 = 3$, $F_6 = 8$ are not divisible by 5, while $F_5 = 5$ is. So the claimed "$p \mid F_{p-1}$ or F_{p+1}" would fail at $p = 5$.

(Optional) Which one is divisible?

From $F_p \equiv \left(\frac{5}{p}\right)$ we get $F_p \equiv 1$ if 5 is a quadratic residue mod p and $F_p \equiv -1$ otherwise. One can refine the above to show:

- If $\left(\frac{5}{p}\right) = +1$, then $p \mid F_{p-1}$;
- If $\left(\frac{5}{p}\right) = -1$, then $p \mid F_{p+1}$. This is a standard refinement (not required by the exercise), and it dovetails with the book's brief answer that only one factor can vanish mod p.

28 [M21]

Method A (from Binet's formula)

Recall the closed form

$$F_n = \frac{\varphi^n - \hat{\varphi}^n}{\sqrt{5}}, \quad \text{where} \quad \hat{\varphi} = 1 - \varphi = \frac{1-\sqrt{5}}{2}.$$

(This $\hat{\varphi}$ is the other root of $x^2 = x + 1$.)

Then

$$\begin{aligned} F_{n+1} - \varphi F_n &= \frac{\varphi^{n+1} - \hat{\varphi}^{n+1}}{\sqrt{5}} - \varphi \cdot \frac{\varphi^n - \hat{\varphi}^n}{\sqrt{5}} \\ &= \frac{\varphi^{n+1} - \hat{\varphi}^{n+1} - \varphi^{n+1} + \varphi\hat{\varphi}^n}{\sqrt{5}} \\ &= \frac{\hat{\varphi}^n(\varphi - \hat{\varphi})}{\sqrt{5}}. \end{aligned}$$

But $\varphi - \hat{\varphi} = \sqrt{5}$, so the factor $\sqrt{5}$ cancels and we get

$$\boxed{F_{n+1} - \varphi F_n = \hat{\varphi}^n = (1-\varphi)^n}.$$

(Here $\hat{\varphi} = 1 - \varphi$.)

Method B (little recurrence trick)

Let $a_n := F_{n+1} - \varphi F_n$. Using $F_{n+2} = F_{n+1} + F_n$ and $\varphi^2 = \varphi + 1$,

$$\begin{aligned} a_{n+1} &= F_{n+2} - \varphi F_{n+1} = (F_{n+1} + F_n) - \varphi F_{n+1} \\ &= F_n - (\varphi - 1)F_{n+1} = F_n - (\varphi - 1)(a_n + \varphi F_n) \\ &= F_n - (\varphi - 1)a_n - (\varphi^2 - \varphi)F_n = -(\varphi - 1)a_n \\ &= (1 - \varphi)a_n = \hat{\varphi}\, a_n. \end{aligned}$$

Since $a_0 = F_1 - \varphi F_0 = 1$, this first-order recurrence gives $a_n = \hat{\varphi}^n$. Thus again

$$\boxed{F_{n+1} - \varphi F_n = (1-\varphi)^n.}$$

Quick check: $n = 0$: $F_1 - \varphi F_0 = 1 = (1-\varphi)^0$. $n = 1$: $F_2 - \varphi F_1 = 1 - \varphi = 1 - \varphi$.

29 [M23]

Let $(F_n)_{n\geq 0}$ be the Fibonacci numbers with $F_0 = 0,\ F_1 = 1,\ F_n = F_{n-1} + F_{n-2}$. Following É. Lucas, define the Fibonomial coefficients

$$\binom{n}{k}_F := \frac{F_n F_{n-1} \cdots F_{n-k+1}}{F_k F_{k-1} \cdots F_1} \quad (0 \leq k \leq n),$$

with the usual conventions $\binom{n}{0}_F = 1$ and $\binom{n}{k}_F = 0$ for $k < 0$ or $k > n$.

We'll also use the standard Fibonacci addition identity

$$F_{r+s} = F_{r-1}F_s + F_r F_{s+1} \qquad (r, s \geq 0),$$

which is featured in classical Fibonacci references and in §1.2.8. ([R-Knott][3])

(a) First rows of the Fibonomial triangle

Compute directly from the definition (the products are short, so it's just careful arithmetic):

- $n = 0:\ 1$
- $n = 1:\ 1,\ 1$
- $n = 2:\ 1,\ 1,\ 1$
- $n = 3:\ 1,\ \dfrac{F_3}{F_1} = 2,\ \dfrac{F_3F_2}{F_2F_1} = 1,\ 1 \to 1, 2, 1$
- $n = 4:\ 1,\ \dfrac{3}{1} = 3,\ \dfrac{3 \cdot 2}{1 \cdot 1} = 6,\ \dfrac{3 \cdot 2 \cdot 1}{1 \cdot 1 \cdot 1} = 3,\ 1 \to 1, 3, 6, 3, 1$
- $n = 5:\ 1, 5, 15, 15, 5, 1$
- $n = 6:\ 1, 8, 40, 60, 40, 8, 1$

So the first seven rows are

$$\begin{array}{l} 1 \\ 1\ 1 \\ 1\ 1\ 1 \\ 1\ 2\ 1 \\ 1\ 3\ 6\ 3\ 1 \\ 1\ 5\ 15\ 15\ 5\ 1 \\ 1\ 8\ 40\ 60\ 40\ 8\ 1\ . \end{array}$$

(b) Pascal-type recurrence and integrality

We claim the Fibonomials satisfy the Lucas–Pascal rule

$$\boxed{\binom{n}{k}_F = F_{k-1}\binom{n-1}{k}_F + F_{n-k+1}\binom{n-1}{k-1}_F} \qquad (1 \le k \le n-1).$$

Derivation

Start from the definition and the two useful "ratio" forms:

$$\binom{n}{k}_F = \frac{F_n}{F_k}\binom{n-1}{k-1}_F = \frac{F_n}{F_{n-k}}\binom{n-1}{k}_F.$$

If we assume the boxed recurrence, divide both sides by $\binom{n-1}{k-1}_F$ to test equality:

$$\frac{\binom{n}{k}_F}{\binom{n-1}{k-1}_F} = F_{k-1}\frac{\binom{n-1}{k}_F}{\binom{n-1}{k-1}_F} + F_{n-k+1}.$$

Compute the inner ratio directly from the product definition:

$$\frac{\binom{n-1}{k}_F}{\binom{n-1}{k-1}_F} = \frac{F_{n-k}}{F_k}.$$

So the claimed recurrence is equivalent to

$$\frac{F_n}{F_k} = F_{k-1}\frac{F_{n-k}}{F_k} + F_{n-k+1},$$

i.e.

$$F_n = F_{k-1}F_{n-k} + F_kF_{n-k+1},$$

which is exactly the Fibonacci addition identity with $r = k,\ s = n-k$. Hence the recurrence holds. ([R-Knott][3])

Consequence: $\binom{n}{k}_F$ is an integer

The initial row $n = 0$ is integral, and the boxed recurrence expresses each interior entry as an integer linear combination of entries one row above. By induction on n, every $\binom{n}{k}_F$ is an integer.

30 [M38]

Let F_n be Fibonacci numbers ($F_0 = 0,\ F_1 = 1,\ F_{n+1} = F_n + F_{n-1}$). Define the Fibonacci factorial $F!n := \prod_{i=1}^{n} F_i$ (with $F!0 = 1$), and the fibonomial coefficients

$$\binom{m}{k}_F := \frac{F!m}{F!k\ F!(m-k)} \qquad (0 \le k \le m;\ \text{else } 0).$$

For a fixed integer $m \ge 1$ and any integer n, consider the alternating sums

$$S_r(m,n) \ := \ \sum_{k=0}^{m} (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F W_r(k),$$

where the weights $W_r(k)$ are Fibonacci terms that are affine in the index (no products of independent k-dependent pieces are needed). Exercise 30 asks you to prove—by induction on m—that four concrete choices of W_r all give $S_r(m,n) = 0$. One convenient way to package the four requested identities is:

- (a) $\displaystyle\sum_{k=0}^{m} (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F F_k \ = \ 0.$
- (b) $\displaystyle\sum_{k=0}^{m} (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F (-1)^k F_{m-k} \ = \ 0.$
- (c) Using (a) and (b) together with $ (-1)^kF_{m-k}=F_{k-1}F_m-F_kF_{m-1}$, deduce

$$\sum_{k=0}^{m} (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F F_{k-1} \ = \ 0.$$

- (d) Using the Fibonacci addition formula $F_{n+k} = F_{k-1}F_n + F_kF_{n+1}$ and $(a), (c)$, deduce

$$\sum_{k=0}^{m} (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F F_{n+k} \ = \ 0 \qquad (\text{for every } n \in \mathbb{Z}).$$

1) Two basic fibonomial recurrences

From the definition,

$$\binom{m}{k}_F = \frac{F_m}{F_k}\binom{m-1}{k-1}_F \quad \text{and} \quad \binom{m}{k}_F = \frac{F_m}{F_{m-k}}\binom{m-1}{k}_F \qquad (1 \le k \le m). \qquad \text{(R1,R2)}$$

Equivalently,

$$F_k\binom{m}{k}_F = F_m\binom{m-1}{k-1}_F, \qquad F_{m-k}\binom{m}{k}_F = F_m\binom{m-1}{k}_F.$$

(These are the exact fibonomial analogues of Pascal's triangle rules and are listed in standard fibonomial references; see also the "Fibonomial formulae" page, which records these same identities.)

Two Fibonacci identities we'll also use are

$$F_{n+k} = F_{k-1}F_n + F_kF_{n+1}, \qquad F_mF_{k-1} - F_{m-1}F_k = (-1)^kF_{m-k}. \qquad \text{(F+, Cassini type)}$$

Both are standard and explicitly invoked in Knuth's answer for this exercise.

2) The induction

We show all claims for a given m assuming they hold for $m-1$.

Base $m = 1$

Directly (each sum has just $k = 0, 1$), the signs are $(-1)^{\lfloor(1-k)/2\rfloor} = +1$ for both terms. Using $\binom{1}{0}_F = \binom{1}{1}_F = 1$ and $F_0 = 0, F_1 = 1$, each of (a)–(d) evaluates to 0. Thus the base holds.

Step (a)

Multiply the summand by F_k/F_k and apply (R1):

$$\sum_k (-1)^{\lfloor(m-k)/2\rfloor}\binom{m}{k}_F F_k \;=\; F_m\sum_{k=1}^{m}(-1)^{\lfloor(m-k)/2\rfloor}\binom{m-1}{k-1}_F.$$

Reindex $j = k-1$ (so $j = 0, \dots, m-1$) and note

$$(-1)^{\lfloor(m-k)/2\rfloor} = (-1)^{\lfloor(m-1-j)/2\rfloor}.$$

By the induction hypothesis (the same identity with $m \to m-1$), the reindexed sum is 0. Hence (a) holds.

Step (b)

Now use (R2) and keep the explicit $(-1)^k$:

$$\sum_k (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F (-1)^k F_{m-k} \;=\; F_m \sum_{k=0}^{m-1} (-1)^{\lfloor (m-k)/2 \rfloor} (-1)^k \binom{m-1}{k}_F .$$

A simple parity check shows

$$(-1)^{\lfloor (m-k)/2 \rfloor} (-1)^k = (-1)^m \, (-1)^{\lfloor (m-1-k)/2 \rfloor}.$$

Therefore

$$\text{(b)} \;=\; (-1)^m F_m \sum_{k=0}^{m-1} (-1)^{\lfloor (m-1-k)/2 \rfloor} \binom{m-1}{k}_F \;=\; 0$$

by the induction hypothesis.

Step (c)

Use the Cassini-type identity:

$$(-1)^k F_{m-k} = F_{k-1} F_m - F_k F_{m-1}.$$

Sum both sides with the same alternating weight and $\binom{m}{k}_F$. The left-hand sum is 0 by (b). The right-hand side becomes

$$F_m \sum_k (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F F_{k-1} \;-\; F_{m-1} \sum_k (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F F_k.$$

The second sum vanishes by (a). Since $F_m \neq 0$, the first must vanish too, proving (c).

Step (d)

Use the addition formula $F_{n+k} = F_{k-1} F_n + F_k F_{n+1}$ inside the sum:

$$\sum_k (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F F_{n+k} \;=\; F_n \underbrace{\sum_k (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F F_{k-1}}_{=0 \text{ by (c)}} + F_{n+1} \underbrace{\sum_k (-1)^{\lfloor (m-k)/2 \rfloor} \binom{m}{k}_F F_k}_{=0 \text{ by (a)}}.$$

Hence (d) holds.

This completes the induction on m, establishing all four identities.

Quick check (small m)

For $m = 2$, $\binom{2}{0}_F = \binom{2}{2}_F = 1$, $\binom{2}{1}_F = 1$.

- (a) gives $ (+1) 1 F_0 + (-1) 1 F_1 + (+1) 1 F_2 = 0-1+1=0$.
- (c) gives $ (+1) 1 F_{-1} + (-1) 1 F_0 + (+1) 1 F_1 = 1+0+1$ with the signs $(+,-,+)$ replaced by $(+,-,+)$ times the fibonomial coefficients—but now $(-1)^{\lfloor (2-k)/2 \rfloor} = (+,-,+)$ so the sum is $1-0-1=0$. Similar spot-checks for (b) and (d) also vanish, consistent with the proof.

31 [M20]

We'll prove the slightly more general identity

$$F_{n-1} - F_n\,\varphi = (-\varphi)^n \qquad (n \geq 1), \tag{ }$$

from which the required statements about fractional parts follow immediately.

Step 1: Establish the recurrence for the "error term"

Define $e_n := F_{n-1} - F_n\,\varphi$. Using $F_{n+1} = F_n + F_{n-1}$ and the defining equation $\varphi^2 + \varphi = 1$ (equivalently $\varphi = \Phi^{-1}$), compute

$$\begin{aligned} e_{n+1} &= F_n - F_{n+1}\varphi = F_n - (F_n + F_{n-1})\varphi \\ &= (F_n - F_{n-1}\varphi) - \varphi F_n = -\varphi\,(F_{n-1} - F_n\varphi) = -\varphi\, e_n. \end{aligned}$$

The base case is $e_1 = F_0 - F_1\varphi = 0 - \varphi = -\varphi$. Hence by induction,

$$e_n = (-\varphi)^n \quad \text{for all } n \geq 1,$$

which is exactly ( ).

(An equally short alternative uses the standard identity $\Phi^n = F_n\Phi + F_{n-1}$; divide by Φ^n and rewrite with $\varphi = \Phi^{-1}$ to get ( ).)

Step 2: Read off the fractional parts

From ( ) we have

$$F_n\varphi \;=\; F_{n-1} - (-\varphi)^n.$$

Because $0 < \varphi < 1$, the tiny correction $(-\varphi)^n$ has absolute value < 1. Therefore:

- If n is odd, then $(-\varphi)^n = -\varphi^n$, so

  $$F_n\varphi = F_{n-1} + \varphi^n,$$

  whose fractional part is $\boxed{\varphi^n}$.
- If n is even, then $(-\varphi)^n = +\varphi^n$, so

  $$F_n\varphi = (F_{n-1} - 1) + (1 - \varphi^n),$$

  whose fractional part is $\boxed{1 - \varphi^n}$.

Re-indexing those two cases as $n = 2k$ and $n = 2k + 1$ gives exactly

$$\{F_{2k}\varphi\} = 1 - \varphi^{2k}, \qquad \{F_{2k+1}\varphi\} = \varphi^{2k+1},$$

as required.

Quick numeric check

n	F_n	$F_n\varphi$	fractional part
1	1	0.6180	$\varphi^1 \approx 0.6180$
2	1	0.6180	$1 - \varphi^2 \approx 0.6180$
3	2	1.2361	$\varphi^3 \approx 0.2361$
4	3	1.8541	$1 - \varphi^4 \approx 0.8541$

Side corollaries (often useful)

From the same identity ( ) you immediately get tidy floor/ceiling formulas:

$$\lfloor F_n\varphi \rfloor = \begin{cases} F_{n-1}, & n \text{ odd}, \\ F_{n-1} - 1, & n \text{ even}, \end{cases} \qquad \lceil F_n\varphi \rceil = \begin{cases} F_{n-1} + 1, & n \text{ odd}, \\ F_{n-1}, & n \text{ even}. \end{cases}$$

32 [M24]

We'll use two standard Fibonacci facts.

(A) Matrix form. Let $M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$. Then

$$M^k = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix} \quad \text{for all } k \geq 0.$$

(B) Cassini's identity.

$$F_{n+1}F_{n-1} - F_n^2 = (-1)^n.$$

In particular, reducing modulo F_n gives

$$F_{n-1}^2 \equiv (-1)^n \pmod{F_n}.$$

We also use the (equally standard) addition formula

$$F_{a+b} = F_a F_{b+1} + F_{a-1} F_b,$$

from which the identity

$$F_{n-1}F_r \equiv (-1)^{r+1} F_{n-r} \pmod{F_n} \qquad (\ )$$

follows by a one-line rearrangement (see derivation just below).

Step 1: Reduce M^n modulo F_n.

From (A),

$$M^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} \equiv \begin{pmatrix} F_{n+1} \bmod F_n & 0 \\ 0 & F_{n-1} \end{pmatrix} \equiv \begin{pmatrix} F_{n-1} & 0 \\ 0 & F_{n-1} \end{pmatrix} = F_{n-1} I \pmod{F_n},$$

because $F_{n+1} \equiv F_{n-1} \pmod{F_n}$.

Hence, for any m, r,

$$M^{mn+r} = (M^n)^m M^r \equiv (F_{n-1})^m M^r \pmod{F_n}.$$

Taking the $(1,2)$ entry on both sides yields the key congruence

$$F_{mn+r} \equiv (F_{n-1})^m F_r \pmod{F_n}. \tag{1}$$

Step 2: Interpret the factor $(F_{n-1})^m$ modulo F_n.

From Cassini (B),

$$(F_{n-1})^2 \equiv (-1)^n \pmod{F_n}.$$

Therefore

$$(F_{n-1})^m \equiv \begin{cases} (-1)^{nm/2}, & m \text{ even}, \\ (-1)^{n(m-1)/2} F_{n-1}, & m \text{ odd}. \end{cases} \tag{2}$$

Plug (2) into (1):

- If m is even, $F_{mn+r} \equiv (\pm 1) F_r \pmod{F_n}$.
- If m is odd, we get an extra F_{n-1} factor:

  $$F_{mn+r} \equiv (\pm) F_{n-1} F_r \pmod{F_n}.$$

  Use ( ) to replace $F_{n-1}F_r$ by $(\pm)F_{n-r}$ modulo F_n, so

  $$F_{mn+r} \equiv (\pm) F_{n-r} \pmod{F_n}.$$

In all cases the residue is either $\pm F_r$ or $\pm F_{n-r}$, as required. (A sharper, fully explicit sign pattern in terms of $m \bmod 4$, n, and r is given here for completeness: $F_{mn+r} \equiv \begin{cases} F_r, & m \equiv 0 \pmod 4; \\ (-1)^{r+1} F_{n-r}, & m \equiv 1 \pmod 4; \\ (-1)^n F_r, & m \equiv 2 \pmod 4; \\ (-1)^{r+1+n} F_{n-r}, & m \equiv 3 \pmod 4, \end{cases} \pmod{F_n}$.) ([ProofWiki][2])

Derivation of ( ) (one line)

Start from the well-known identity (a variant of d'Ocagne's):

$$F_n F_{r+1} - F_{n+1} F_r = (-1)^r F_{n-r}.$$

Reduce modF_n: the left becomes $-F_{n+1}F_r \equiv -F_{n-1}F_r$, hence

$$F_{n-1}F_r \equiv (-1)^{r+1}F_{n-r} \pmod{F_n}.$$

Check on a concrete example

Take $n = 7$, $m = 3$, $r = 2$. Then $mn + r = 23$. We predict

$$F_{23} \equiv \pm F_{7-2} = \pm F_5 = \pm 5 \pmod{F_7},$$

since m is odd. Indeed $F_{23} = 28657$ and $F_7 = 13$; 28657 mod 13 = 5.

33 [HM24]

Step 1: Pick the right z

Take

$$z \;=\; \frac{\pi}{2} \;+\; i\ln\varphi\,, \qquad \varphi = \frac{1+\sqrt{5}}{2}.$$

Then

$$\cos z = \cos\Big(\tfrac{\pi}{2} + i\ln\varphi\Big) = \cos\tfrac{\pi}{2}\cosh(\ln\varphi) - i\sin\tfrac{\pi}{2}\sinh(\ln\varphi) = -i\,\sinh(\ln\varphi).$$

Since $\sinh(\ln\varphi) = \frac{1}{2}(\varphi - \varphi^{-1}) = \frac{1}{2}$, we indeed get $\cos z = -\frac{i}{2}$ as desired. Also

$$\sin z = \sin\Big(\tfrac{\pi}{2} + i\ln\varphi\Big) = \sin\tfrac{\pi}{2}\cosh(\ln\varphi) = \cosh(\ln\varphi) = \frac{\varphi+\varphi^{-1}}{2} = \frac{\sqrt{5}}{2}.$$

Step 2: Define the candidate sequence and verify the recurrence

Define

$$S_n \;=\; i^{\,n-1}\,\frac{\sin(nz)}{\sin z}\,.$$

Use the standard identity

$$\sin((n+1)z) + \sin((n-1)z) = 2\cos z\,\sin(nz)$$

(valid for all complex z). Multiply both sides by $i^{\,n}/\sin z$ and use $\cos z = -\,i/2$:

$$i^n \frac{\sin((n+1)z)}{\sin z} + i^n \frac{\sin((n-1)z)}{\sin z} = 2\Big(-\frac{i}{2}\Big) i^n \frac{\sin(nz)}{\sin z} = -i^{n+1} \frac{\sin(nz)}{\sin z}.$$

Divide through by i and rewrite in terms of S_n:

$$S_{n+1} = S_n + S_{n-1}.$$

Initial values: $S_0 = 0$ (since $\sin 0 = 0$) and $S_1 = 1$ (since $\sin z / \sin z = 1$). Therefore $\{S_n\}$ satisfies the Fibonacci recurrence with Fibonacci initial conditions, hence $S_n = F_n$ for all n. This proves

$$F_n \;=\; i^{n-1} \frac{\sin(nz)}{\sin z} \qquad \text{for } \cos z = -\frac{i}{2}.$$

(Exactly the approach Knuth's hint suggests.)

Step 3 (optional): Recover Binet's formula

With $z = \frac{\pi}{2} + i \ln \varphi$ we have $e^{iz} = e^{i\pi/2} e^{-\ln \varphi} = i/\varphi$. Hence

$$\sin(nz) = \frac{e^{inz} - e^{-inz}}{2i} = \frac{(i/\varphi)^n - (-i\varphi)^n}{2i}.$$

Using $\sin z = \sqrt{5}/2$ from Step 1,

$$F_n = i^{n-1} \frac{\sin(nz)}{\sin z} = \frac{i^{n-1}}{\sqrt{5}/2} \cdot \frac{(i/\varphi)^n - (-i\varphi)^n}{2i} = \frac{\varphi^n - (-\varphi^{-1})^n}{\sqrt{5}} = \frac{\varphi^n - (\varphi')^n}{\sqrt{5}},$$

where $\varphi' = (1 - \sqrt{5})/2 = -1/\varphi$. That is Binet's formula.

34 [M24]

Let $k \gg m$ mean $k \geq m + 2$. Prove that every positive integer n has a unique representation

$$n \;=\; F_{k_1} + F_{k_2} + \cdots + F_{k_r},$$

with $k_1 \gg k_2 \gg \cdots \gg k_r \gg 0$ (i.e., the indices are strictly decreasing and no two are consecutive; the smallest allowed index is 2). This is the Fibonacci number system.

This statement is the classical Zeckendorf theorem specialized to Knuth's indexing $F_0 = 0$, $F_1 = 1$, $F_2 = 1$, $F_n = F_{n-1} + F_{n-2}$.

Existence (constructive — the greedy algorithm)

Greedy step. Given $n \geq 1$, pick the largest Fibonacci number $F_k \leq n$ (with $k \geq 2$), and set $r = n - F_k$.

Because $F_{k+1} = F_k + F_{k-1}$ and $F_{k+1} > n \geq F_k$, we have

$$0 \leq r < n - F_k < F_{k+1} - F_k = F_{k-1}.$$

So the remainder r is strictly less than F_{k-1}. Therefore when we recurse on r, the next chosen Fibonacci index is at most $k-2$. Hence the indices differ by at least 2 (the "no adjacent indices" condition), exactly as required.

Termination. The remainders strictly decrease and are nonnegative, so the process stops after finitely many steps at $r = 0$.

Thus the greedy procedure produces a representation

$$n = F_{k_1} + F_{k_2} + \cdots + F_{k_r} \quad \text{with} \quad k_1 > k_2 > \cdots > k_r \geq 2,\ k_i - k_{i+1} \geq 2.$$

So existence is proved.

A helpful bound (maximal sum without adjacency)

Define $S(m)$ to be the maximum sum obtainable using $\{F_2, \dots, F_m\}$ with no adjacent indices. Then

$$\boxed{S(m) = F_{m+1} - 1 \quad (m \geq 2)}.$$

Proof by induction.

- Base $m = 2$: the only allowed choice is $F_2 = 1$; indeed $F_3 - 1 = 2 - 1 = 1$.
- Step: Any optimal set on $\{2, \dots, m\}$ either omits F_m (sum $S(m-1)$) or uses F_m and must omit F_{m-1} (sum $F_m + S(m-2)$). Thus

  $$S(m) = \max\{S(m-1),\ F_m + S(m-2)\}.$$

  By the induction hypothesis $S(m-1) = F_m - 1$ and $S(m-2) = F_{m-1} - 1$, hence

  $$S(m) = \max\{F_m - 1,\ F_m + F_{m-1} - 1\} = F_{m+1} - 1. \ \square$$

Intuitively, the (unique) maximizer is $F_m + F_{m-2} + F_{m-4} + \cdots$, whose sum is $F_{m+1} - 1$.

Uniqueness

Suppose

$$n = \sum_{i=1}^{r} F_{a_i} = \sum_{j=1}^{s} F_{b_j}$$

are two legal (no-adjacent) representations, with $a_1 > \cdots > a_r$ and $b_1 > \cdots > b_s$.

Let $t = \max\{a_1, b_1\}$. If, without loss of generality, $a_1 = t > b_1$, then the right-hand sum uses only indices $\leq t - 1$ and is legal; by the bound above it is at most $S(t-1) = F_t - 1$. The left-hand sum is at least F_t. Contradiction:

$$F_t \leq n \leq F_t - 1.$$

Therefore $a_1 = b_1$. Cancel F_t from both sides and repeat the argument on the remainders. By induction, all indices match and the two sums are identical. Hence the representation is unique.

Algorithm (explicit) and complexity

A direct constructive algorithm follows the proof:

1. Generate Fibonacci numbers $F_2, F_3, \ldots$ up to the largest $F_k \leq n$.
2. Set $r \leftarrow n$.
3. While $r > 0$:
   - Choose the largest $F_j \leq r$ (necessarily $j \leq k$).
   - Output F_j, set $r \leftarrow r - F_j$, and set $k \leftarrow j - 2$ (skip the adjacent index).

This produces the unique expansion with no adjacent indices. Because consecutive chosen indices drop by at least 2 and F_k grows like $\varphi^k/\sqrt{5}$, the number of chosen terms (and the number of loop iterations) is $O(\log n)$.

Small examples

- $17 = 13 + 3 + 1 = F_7 + F_4 + F_2$ (indices $7, 4, 2$: gaps ≥ 2).
- $100 = 89 + 8 + 3 = F_{11} + F_6 + F_4$.
- $1 = F_2$ (note $F_1 = 1$ exists but $k \geq 2$ is required, so we use F_2; this avoids the non-uniqueness of "two ones").

35 [M24]

Here is Exercise 1.2.8–35 (paraphrased): write real numbers in "base " with digits 0/1, e.g. $(100.1)_\varphi = \varphi^2 + \varphi^{-1}$. (i) Show 1 has infinitely many such representations (examples: $(.11)_\varphi$ and $(.010101\ldots)_\varphi$). (ii) If we forbid adjacent 1s and also forbid representations that end with the infinite tail 010101 ..., prove every nonnegative real has a unique base- representation. (iii) Describe which of these canonical representations are integers.

Base- facts we'll use

$$\varphi^2 = \varphi + 1 \quad \Longleftrightarrow \quad \varphi^j = \varphi^{j-1} + \varphi^{j-2} \quad (\forall j \in \mathbb{Z}). \tag{A}$$

Consequences for digit strings (writing the most significant position on the left, exponents decreasing by 1 as we move right):

- Carry/borrow rule: anywhere you see the block `11` in adjacent positions $j-1, j-2$, you may replace it by `100` at position j (and conversely `100` `011`). This preserves value by (A).
- Tail identity:

$$\sum_{m \geq 0} \varphi^{-(2m+1)} = \frac{\varphi^{-1}}{1 - \varphi^{-2}} = 1,$$

since $1 - \varphi^{-2} = (\varphi^2 - 1)/\varphi^2 = \varphi/\varphi^2 = 1/\varphi$. Thus $1 = (.010101\ldots)_\varphi$ and also $1 = (.11)_\varphi$ because $\varphi^{-1} + \varphi^{-2} = (\varphi + 1)/\varphi^2 = \varphi^2/\varphi^2 = 1$.

(i) Infinitely many representations of 1

Two are already given: $(.11)_\varphi$ and $(.010101\ldots)_\varphi$. To see there are infinitely many distinct ones, start from the finite representation $1 = (1.000\ldots)_\varphi$ and repeatedly apply the inverse carry `100` → `011` to the triple covering the radix point and then successively to lower triples:

$$(1.000\ldots)_\varphi = (.011000\ldots)_\varphi = (.001011000\ldots)_\varphi = (.00001011000\ldots)_\varphi \to \cdots$$

Each step yields a different digit string but the same value (by (A)), and the sequence converges to the infinite tail $(.010101\ldots)_\varphi$. Hence, 1 has infinitely many base- 0/1 representations.

(ii) Existence and uniqueness under the constraints

Normal form (existence)

Given any 0/1 base- string for $x \geq 0$, repeatedly replace every adjacent `11` by `100`, scanning from right to left. Each replacement strictly moves weight to a more significant position, and no new `11`'s are created to its right. Two things can happen:

- The process stops after finitely many steps: we obtain a representation with no adjacent 1s.
- Or the carries propagate forever to the left in a perfectly alternating way; the only possible infinite stabilization is the notorious tail 010101 ... (shifted), which keeps the value the same (by the tail identity). Excluding representations that end with 010101 ... rules this out.

Thus every $x \geq 0$ admits at least one *admissible* representation: digits in $\{0, 1\}$, no adjacent 1s, and no terminal 0101 ... tail.

Uniqueness

Suppose two different admissible strings represent the same x. Let j be the largest position where they differ; w.l.o.g. the first has digit 1 at j and the second has 0. Because no adjacent 1s are allowed, all 1s to the right of j in either string can occupy only positions $j-2, j-3, \ldots$ with at least one zero between any two 1s. The maximal value you can build from positions $<$ j without adjacency is the geometric series

$$\varphi^{j-2} + \varphi^{j-4} + \varphi^{j-6} + \cdots = \frac{\varphi^{j-2}}{1-\varphi^{-2}} = \varphi^{j-1}.$$

But the surplus contributed by having a 1 at position j is φ^j, which exceeds φ^{j-1}. Therefore the two values cannot be equal—a contradiction. Hence the admissible representation is unique. (The extra "no 0101 ... tail" condition is what removes the special ambiguity $1 = (.11)_\varphi = (.0101\ldots)_\varphi$ and all its shifted variants.)

(iii) Which admissible strings represent integers?

Write an admissible expansion as

$$x = \sum_{j \in \mathbb{Z}} a_j \, \varphi^j \qquad (a_j \in \{0, 1\}).$$

Using the identities

$$\varphi^n = F_n \varphi + F_{n-1} \quad (n \geq 1), \qquad \varphi^{-n} = (-1)^{n+1} F_n \, \varphi + (-1)^n F_{n+1} \quad (n \geq 1),$$

(each proved by induction from $\varphi^2 = \varphi + 1$), we can regroup $x = A\varphi + B$ with integers A, B. A short calculation gives

$$A = \sum_{j \geq 1} a_j F_j \; + \sum_{n \geq 1} a_{-n} (-1)^{n+1} F_n, \qquad B = a_0 + \sum_{j \geq 1} a_j F_{j-1} + \sum_{n \geq 1} a_{-n} (-1)^n F_{n+1}.$$

Therefore x is an integer iff $A = 0$, i.e.

$$\boxed{\sum_{j\geq 1} a_j F_j \;=\; \sum_{n\geq 1} (-1)^{n+1} a_{-n} F_n} \qquad (*)$$

and in that case the value is the integer B. In words: in a canonical base- string (no adjacent 1s, no 0101 ... tail), x is an integer iff the Fibonacci-weighted sum of 1-digits left of the radix equals the alternating Fibonacci-weighted sum of 1-digits right of the radix.

A practical way to construct the (unique) integer representation is the greedy algorithm consistent with the constraints: repeatedly place a 1 at the largest position j such that adding φ^j does not exceed the target and does not create adjacency, then continue downward. This terminates and produces the canonical string; the first few are:

- $0 = (0)_\varphi$
- $1 = (1)_\varphi$ (note $(.11)_\varphi$ is not admissible since it has adjacent 1s)
- $2 = (10.01)_\varphi$
- $3 = (100.01)_\varphi$
- $4 = (101.01)_\varphi$
- $5 = (1000.1001)_\varphi$
- $6 = (1010.0001)_\varphi$

(You can verify each by summing powers of φ; no pair of adjacent 1s occurs, and the expansions do not end with 0101 ....)

36 [M32]

1) Basic structure and the infinite word

Because $S_{n+2} = S_{n+1}S_n$ and $|S_1| = |S_2| = 1$, lengths satisfy $|S_{n+2}| = |S_{n+1}| + |S_n|$; hence $|S_n| = F_n$ by induction. Moreover, for $n \geq 2$ each S_{n+1} begins with $\mathtt{b}$, and the last letter of S_n alternates $\mathtt{a}, \mathtt{b}, \mathtt{a}, \mathtt{b}, \ldots$ (since the last letter of S_{n+2} equals that of S_n). Thus the prefixes are nested and there is a well-defined infinite limit

$$S_\infty = \lim_{n\to\infty} S_n$$

such that, for $n > 1$, S_n is exactly the first F_n letters of S_∞.

A convenient "substitution" description: if we set the morphism μ by $\mu(\mathtt{a}) = \mathtt{b},\ \mu(\mathtt{b}) = \mathtt{ba}$, then $S_n = \mu^{n-1}(\mathtt{a})$. (Check: $\mu(\mathtt{a}) = \mathtt{b} = S_2$, $\mu^2(\mathtt{a}) = \mu(\mathtt{b}) = \mathtt{ba} = S_3$, $\mu^3(\mathtt{a}) = \mu(\mathtt{ba}) = \mathtt{bab} = S_4$, etc.)

2) Where do doubles occur?

Two simple (and very useful) consequences:

- No `aa`: μ never produces `aa` (images are only `b` or `ba`). Inductively, if S_n has no `aa`, then S_{n+1} and the concatenation $S_{n+1}S_n$ also don't—across the boundary we see either `ab` or `bb`, never `aa`.
- No `bbb`: the only way `bb` arises is inside $\mu(\texttt{ab}) = \texttt{bba}$; every `bb` is immediately followed by `a`. Hence `bbb` cannot occur.

3) Exact letter counts in S_n

Let A_n and B_n be the numbers of `a`'s and `b`'s in S_n. Then

$$(A_{n+2}, B_{n+2}) = (A_{n+1} + A_n,\ B_{n+1} + B_n), \qquad (A_1, B_1) = (1, 0),\ (A_2, B_2) = (0, 1).$$

Solving gives

$$A_n = F_{n-2}, \qquad B_n = F_{n-1} \quad (n \geq 1).$$

So in S_n there are F_{n-2} `a`'s and F_{n-1} `b`'s (and $F_{n-2} + F_{n-1} = F_n$ total), matching the book's answer.

Densities. Since $\dfrac{F_{n-1}}{F_n} \to \varphi^{-1}$ and $\dfrac{F_{n-2}}{F_n} \to \varphi^{-2}$ ($\varphi = \frac{1+\sqrt{5}}{2}$), the asymptotic frequency of `b` in S_∞ is φ^{-1} and of `a` is φ^{-2}.

4) Predicting the k-th letter — three equivalent characterizations

(a) A fast recursive test (using Fibonacci lengths)

Let $L_n = F_n$. To find the letter at position k of S_∞:

1. If $k = 1$ return `b`; if $k = 2$ return `a`.
2. Let n be such that $F_n < k \leq F_{n+1}$. If $k \leq F_{n+1}$ and $k > F_n$, you're in the left block S_{n+1}; replace $(k, n) \leftarrow (k, n-1)$. If $k \leq F_n$, you're in the right block of the previous step; replace $(k, n) \leftarrow (k, n-2)$.
3. Repeat until you hit the base.

Unwinding this exactly subtracts Fibonacci lengths according to the Zeckendorf greedy decomposition of $k - 1$ into nonconsecutive Fibonacci numbers (see Exercise 34).

(b) Beatty/"mechanical word" formula

Let

$$B(k) \;=\; \#\{\texttt{b}\text{'s among the first } k \text{ letters of } S_\infty\}.$$

One proves (by induction using $S_{n+2} = S_{n+1}S_n$ and the identities $\lfloor (F_n+1)\varphi^{-1}\rfloor = F_{n-1}$) that

$$\boxed{B(k) = \lfloor (k+1)\varphi^{-1}\rfloor}.$$

Therefore the k-th letter is `b` iff $B(k) - B(k-1) = 1$, i.e.

$$\boxed{\text{letter}_k = \texttt{b} \iff \left\lfloor \tfrac{k+1}{\varphi}\right\rfloor - \left\lfloor \tfrac{k}{\varphi}\right\rfloor = 1}.$$

(c) Positions of all `b`'s

Equivalently, the positions of `b` form the Beatty sequence

$$\boxed{k = \lfloor m\varphi\rfloor \quad (m = 1, 2, 3, \dots)}.$$

This is again one of the book's listed characterizations. (It's the complementary Beatty partner of $\lfloor m\varphi^2\rfloor$, reflecting that the `a`/`b` frequencies are φ^{-2} and φ^{-1}.)

5) Where do doubles occur?

From §2 we know `aa` never appears and `bbb` never appears. But `bb` certainly does: the first time is in $S_5 = \texttt{babba}$. In fact every `bb` is immediately followed by `a` (all `bb` come from $\mu(\texttt{ab}) = \texttt{bba}$), so all double letters are precisely the factors `bba`. (This also explains why the run lengths of `b`'s are 1 or 2 only.)

6) Summary of key facts (what the book expects you to "discover")

- S_n is the prefix of length F_n of S_∞. No `aa`; no `bbb`. Counts: $\#\texttt{a} = F_{n-2}$, $\#\texttt{b} = F_{n-1}$.
- k-th letter via Zeckendorf (Exercise 34): it's `a` iff the smallest Fibonacci used in $k-1$ is F_2.
- Exact counting and "density" of `b`: $B(k) = \lfloor (k+1)/\varphi\rfloor$; hence `b`-density $= \varphi^{-1}$. Equivalent: the `b` positions are exactly $\{\lfloor m\varphi\rfloor : m \geq 1\}$.

7) Tiny sanity checks

n	S_n	$(\#\mathtt{a}, \#\mathtt{b})$	first F_n letters' $B(F_n) \stackrel{?}{=} F_{n-1}$
3	ba	$(1,1)$	$\lfloor (2+1)/\varphi \rfloor = 1 = F_2$
4	bab	$(1,2)$	$\lfloor (3+1)/\varphi \rfloor = 2 = F_3$
5	babba	$(2,3)$	$\lfloor (5+1)/\varphi \rfloor = 3 = F_4$

All match.

37 [M35]

There is one heap with n chips. On the next move a player may remove at most q chips. (In Fibonacci Nim, after a player removes m chips, the following move is bounded by $q' = 2m$; the first move typically has $q = n - 1$.)

Write n in its Zeckendorf representation (sum of nonconsecutive Fibonacci numbers):

$$n = F_{k_1} + F_{k_2} + \cdots + F_{k_r}, \qquad k_1 > k_2 > \cdots > k_r \geq 2,$$

and define

$$\mu(n) \stackrel{\text{def}}{=} F_{k_r} \quad \text{(the smallest Fibonacci in the sum)}, \qquad \mu(0) = \infty.$$

Claim. From a position (n, q), there is a winning move iff $\mu(n) \leq q$. In particular, in the usual game where the first move has $q = n - 1$, the first player wins unless n is itself a Fibonacci number. (When n is Fibonacci, $\mu(n) = n > n - 1 = q$, so the first player loses.)

Preliminaries about μ

Let $n = F_{k_1} + \cdots + F_{k_r}$ be in Zeckendorf form and recall consecutive Fibonacci indices never both appear.

We will use four easy inequalities:

(A) If $n > 0$, then

$$\mu(n - \mu(n)) > \ 2\,\mu(n).$$

Reason. Removing the smallest term F_{k_r} leaves $F_{k_1} + \cdots + F_{k_{r-1}}$. Since $k_{r-1} \geq k_r + 2$, the next smallest possible Fibonacci in the remainder is at least $F_{k_r+2} > 2F_{k_r}$. Thus $\mu(n - \mu(n)) > 2\mu(n)$.

(B) If $0 < m < F_k$, then

$$\mu(m) \le 2(F_k - m).$$

Reason (sketch). The largest Fibonacci allowed in a Zeckendorf sum strictly below F_k is at most F_{k-1}, and the "greedy" parity-alternating tail $F_{k-1} + F_{k-3} + \cdots$ bounds any such m. Rewriting this bound gives $F_k - m \ge \frac{1}{2}\,\mu(m)$.

(C) If $0 < m < \mu(n)$, then

$$\mu(n - \mu(n) + m) \le 2(\mu(n) - m).$$

Reason. Apply (B) with $F_k = \mu(n)$ to the increment m.

(D) If $0 < m < \mu(n)$, then

$$\mu(n - m) \le 2m.$$

Reason. Substitute $m \leftarrow \mu(n) - m$ in (C).

Characterization of winning and losing positions

We prove by induction on n that:

> Theorem. From position (n, q) there exists a winning move iff $\mu(n) \le q$.

- ($\Rightarrow$) Losing positions when $\mu(n) > q$. Any legal move removes some $m \le q < \mu(n)$. After this move the heap is $n' = n - m$ and the next bound is $q' = 2m$ (Fibonacci Nim rule). By (D), $\mu(n') \le 2m = q'$. Thus every possible move hands the opponent a position with $\mu(n') \le q'$, i.e., a winning position for them. So (n, q) is losing.
- ($\Leftarrow$) Winning positions when $\mu(n) \le q$. If $q \ge n$ you can take all n and win immediately. Otherwise, remove exactly $m = \mu(n)$ (legal since $m \le q$). Now $n' = n - \mu(n)$ and $q' = 2\mu(n)$. By (A), $\mu(n') > 2\mu(n) = q'$. Hence the opponent receives a losing position. Therefore (n, q) is winning.

Combining the two directions establishes the theorem. As a corollary, in the standard initial position for Fibonacci Nim, $q = n - 1$. Then the first player wins unless $\mu(n) = n$, i.e., unless n is itself a Fibonacci number.

What to play (algorithmic recipe)

Given (n, q):

1. Compute the Zeckendorf form $n = F_{k_1} + \cdots + F_{k_r}$ and $\mu(n) = F_{k_r}$.

2. If $\mu(n) > q$: you are lost (perfect play assumed). Any move $m \le q$ gives the opponent $\mu(n-m) \le 2m$ (by (D)), hence a winning reply.
3. If $\mu(n) \le q$: remove exactly $\mu(n)$. This forces $\mu(n-\mu(n)) > 2\mu(n) = q'$ (by (A)), so you pass a losing position to your opponent. More generally, all winning moves are: remove a suffix of the Zeckendorf sum

$$m = F_{k_j} + F_{k_{j+1}} + \cdots + F_{k_r}$$

with either $j = 1$ (take the whole representation) or satisfying the legality condition $F_{k_{j-1}} > 2m$ (so $m \le q$ on this turn).

Example

$n = 1000$. Zeckendorf: $1000 = 987 + 13$. The only "lucky" (winning) first move is to take 13 (i.e., $\mu(1000)$). Any other first move allows a reply that restores $\mu(\cdot)$-advantage to the opponent. Thus with perfect play, the first player wins by removing 13.

38 [35]

1) Problem

Two players alternately remove chips from a pile of size n.

- On the first move you may take any number $1 \le m \le n-1$ (you cannot take the whole pile).
- After that, if the previous move took t chips, the next move may take any m with $1 \le m \le 2t$.
- The player who takes the last chip wins.

2) Key theory (why the strategy works)

Let every positive integer n be written in its Zeckendorf representation, $n = F_{k_1} + F_{k_2} + \cdots + F_{k_r}, \quad k_1 \gg k_2 \gg \cdots \gg k_r \gg 0$, i.e., as a sum of nonconsecutive Fibonacci numbers (with $F_0 = 0, F_1 = 1$, hence the smallest summand index is $k_r \ge 2$). This representation exists and is unique.

Define $\mu(n)$ to be the smallest Fibonacci summand in the Zeckendorf expansion of n (and set $\mu(0) = \infty$). The answers section proves the crucial fact:

> If a position has n chips and the next player may take at most q chips, then there is a winning move iff $\mu(n) \le q$. In that case, a winning move is to take exactly $\mu(n)$ chips (or all remaining chips if $q \ge n$); this leaves the opponent a "bad" position. Conversely, if $\mu(n) > q$ then every legal move allows the opponent a winning reply.

Immediate corollaries:

- At the start, $q = n - 1$. If n itself is a Fibonacci number, then $\mu(n) = n > q$, so the first player is lost against perfect play. Otherwise the first player has a win. (This is exactly what the book notes, and for $n = 1000$ it shows $1000 = 987 + 13$, so the only winning first move is to take 13.)

3) Optimal algorithm

Maintain n (chips left) and q (max you're allowed to take now). The optimal move is:

1. If $n \leq q$: take all and win.
2. Compute $\mu(n)$ = smallest term in the Zeckendorf representation of n.
   - If $\mu(n) \leq q$: take $\mu(n)$.
   - Else (a losing position): pick any legal $m \in [1, \min(q, n)]$. (All are losing vs. perfect play; choosing $m = 1$ is fine.)

How to compute $\mu(n)$ fast: Precompute Fibonacci numbers $F_2 = 1, F_3 = 2, \ldots$ up to n. Use the greedy Zeckendorf algorithm: repeatedly subtract the largest $F_k \leq n$, skipping the immediate next F_{k-1} to avoid consecutive terms. The last subtraction gives $\mu(n)$. This runs in $O(\log n)$ time per move.

4) Reference Python implementation

```
from bisect import bisect_right

def fibs_up_to(n):
    """Return [F2, F3, ..., Fm] with F2=1, F3=2, ... up to  n."""
    fibs = [1, 2]  # F2, F3
    while fibs[-1] + fibs[-2] <= n:
        fibs.append(fibs[-1] + fibs[-2])
    return fibs

def zeckendorf_smallest(n, fibs):
    """
    Return  (n): smallest Fibonacci summand in Zeckendorf rep of n.
    Assumes n>0.
    """
    m = n
    smallest = None
    i = len(fibs) - 1
    skip_next_index = -1  # index to skip to avoid consecutive F's
```

```
    while m > 0:
        # largest fib <= m
        i = bisect_right(fibs, m) - 1
        # enforce nonconsecutive rule: if we just used fibs[i+1], skip fibs[i]
        if i == skip_next_index:
            i -= 1
        f = fibs[i]
        m -= f
        smallest = f  # last chosen will be the smallest
        # next, we must skip the immediate smaller Fibonacci number
        skip_next_index = i - 1

    return smallest

def optimal_move(n, q, fibs_cache=None):
    """
    Given pile size n and max-allowed q (1st move uses q=n-1),
    return the optimal number of chips to take now.
    """
    if n <= 0:
        return 0
    if q >= n:
        return n  # take all and win

    fibs = fibs_cache or fibs_up_to(n)
    mu = zeckendorf_smallest(n, fibs)

    if mu <= q:
        return mu
    # Losing position: any legal move is equivalent vs. perfect opponent.
    # Take 1 to keep it simple.
    return 1

# Example: first move with n=1000 (q=999)
if __name__ == "__main__":
    n = 1000
    q = n - 1
    move = optimal_move(n, q)
    print(move)  # expected 13 (since 1000=987+13)
```

What it prints for 1000: `13`, matching the book's worked example (since $1000 = 987 + 13$).

Using it in a full game loop

Track `prev_take` to update the next `q = 2*prev_take` each turn:

```
def play_optimal(n, first=True):
    """
    Simulate an optimal-vs-optimal game starting from pile n.
    Returns the winner: 'first' or 'second'.
    """
    player = 'first' if first else 'second'
    prev_take = None
    fibs = fibs_up_to(n)

    while n > 0:
        q = (n-1) if prev_take is None else min(2*prev_take, n)
        take = optimal_move(n, q, fibs_cache=fibs)
        n -= take
        prev_take = take
        player = 'second' if player == 'first' else 'first'

    # last player who moved just switched 'player', so the winner is the other one
    return 'second' if player == 'first' else 'first'
```

5) Correctness sketch

- Zeckendorf's theorem gives a canonical decomposition $n = \sum F_{k_j}$ with no adjacent indices.
- The answers section establishes properties (A)–(D) of μ and concludes:
  - A winning move exists iff $\mu(n) \leq q$,
  - and the canonical winning move is to take $\mu(n)$. This exactly matches the `optimal_move` rule above.

Thus the program implements the mathematically proven optimal strategy.

6) Complexity

- Precomputing Fibonacci numbers up to n: $O(\log n)$ numbers.
- Each move computes $\mu(n)$ via a greedy pass that subtracts at most $O(\log n)$ Fibonacci numbers.
- So the decision per move is $O(\log n)$ time and $O(\log n)$ space.

39 [M24]

Solution 1 — Characteristic-polynomial method

Look for solutions of the form $a_n = r^n$. Substituting into the recurrence:

$$r^{n+2} = r^{n+1} + 6r^n \quad \Longleftrightarrow \quad r^2 - r - 6 = 0.$$

Factor:

$$(r-3)(r+2) = 0 \quad \Rightarrow \quad r_1 = 3,\ r_2 = -2.$$

Therefore the general term is a linear combination of the two independent solutions:

$$a_n = \alpha \cdot 3^n + \beta \cdot (-2)^n,$$

with constants α, β fixed by the initial values.

Use $a_0 = 0$: $\alpha + \beta = 0 \Rightarrow \beta = -\alpha$.

Use $a_1 = 1$: $3\alpha - 2\beta = 1$. Substitute $\beta = -\alpha$:

$$3\alpha - 2(-\alpha) = 5\alpha = 1 \ \Rightarrow\ \alpha = \tfrac{1}{5}, \quad \beta = -\tfrac{1}{5}.$$

Hence

$$\boxed{a_n = \frac{3^n - (-2)^n}{5}}.$$

Solution 2 — Generating-function method

Let $A(z) = \sum_{n\geq 0} a_n z^n$. Multiply the recurrence by z^{n+2} and sum over $n \geq 0$:

$$\sum_{n\geq 0} a_{n+2} z^{n+2} = \sum_{n\geq 0} a_{n+1} z^{n+2} + 6 \sum_{n\geq 0} a_n z^{n+2}.$$

Using $a_0 = 0, a_1 = 1$, this becomes

$$A(z) - z = z\,A(z) + 6z^2 A(z) \ \Rightarrow\ A(z) = \frac{z}{1 - z - 6z^2}.$$

Factor the denominator: $1 - z - 6z^2 = (1-3z)(1+2z)$. Do partial fractions:

$$\frac{z}{1-z-6z^2} = \frac{1}{5}\left(\frac{1}{1-3z} - \frac{1}{1+2z}\right).$$

Expanding,

$$A(z) = \sum_{n\geq 0} \frac{1}{5}(3^n - (-2)^n)z^n,$$

so $a_n = \frac{3^n-(-2)^n}{5}$ again.

Sanity checks and growth

First terms: $0, 1, 1, 7, 13, 55, \ldots$ (plugging $n = 0, 1, 2, 3, 4, 5$ into the formula). As n grows, the 3^n term dominates, so $a_n \sim 3^n/5$.

40 [M25]

Define $f(n)$ as the minimum possible worst-case cost to identify one item out of n using a binary decision tree where each left edge costs 1 and each right edge costs 2. If at a node that covers n items you split into a left subtree of size k and a right subtree of size $n-k$, the worst-case cost through that node is

$$\max(1 + f(k),\ 2 + f(n-k)).$$

Thus

$$f(1) = 0, \qquad f(n) = \min_{1\leq k<n} \max(1 + f(k),\, 2 + f(n-k)) \quad (n \geq 2).$$

Claim. For every $m \geq 0$ and $n \geq 1$,

$$\boxed{F_m < n \leq F_{m+1} \iff f(n) = m}$$

(where $F_0 = 0, F_1 = 1, F_{i+1} = F_i + F_{i-1}$ are Fibonacci numbers).

Proof (by induction on m)

Base case $m = 0$.

The interval $F_0 < n \leq F_1$ is $0 < n \leq 1$, i.e., $n = 1$. By definition $f(1) = 0$.

Inductive step.

Assume the statement holds for all smaller indices than a fixed $m \geq 1$. Let n satisfy

$$F_m < n \le F_{m+1}.$$

Upper bound $f(n) \le m$.

Choose the split $k = F_m$. By the induction hypothesis, $f(F_m) = m - 1$. Also

$$n - k = n - F_m \le F_{m+1} - F_m = F_{m-1},$$

so again by induction $f(n-k) \le m - 2$. Hence

$$f(n) \;\le\; \max(1 + (m-1),\, 2 + (m-2)) = m.$$

Lower bound $f(n) \ge m$.

Suppose, for contradiction, that $f(n) < m$. Then in the optimal split at some k we must have both branches strictly below m:

$$1 + f(k) < m \quad \text{and} \quad 2 + f(n-k) < m,$$

i.e.,

$$f(k) \le m - 2, \qquad f(n-k) \le m - 3.$$

By the induction hypothesis, these imply

$$k \le F_{m-1}, \qquad n - k \le F_{m-2}.$$

Adding yields $n \le F_{m-1} + F_{m-2} = F_m$, contradicting $n > F_m$. Therefore $f(n) \ge m$.

Combining with the upper bound gives $f(n) = m$, completing the induction.

Consequences and intuition

- The optimal worst-case cost is the least m with $n \le F_{m+1}$. Equivalently,

  $$f(n) = \min\{\, m : \; n \le F_{m+1} \,\}.$$

  As a rough asymptotic using Binet's formula, $F_{m+1} \approx \varphi^{m+1}/\sqrt{5}$, so

  $$f(n) \;\approx\; \left\lceil \log_{\varphi}(\sqrt{5}\, n) \right\rceil - 1,$$

  but the exact answer is the Fibonacci threshold characterization above.

- The optimal split at size n (when $F_m < n \le F_{m+1}$) is $k = F_m$: the left subtree handles F_m items (cost $m-1$) and the right handles at most F_{m-1} items (cost $\le m-2$). The resulting optimal trees are precisely the Fibonacci trees (each node's subtrees have "heights" differing as $m-1$ and $m-2$).

Small values

Using the rule $F_m < n \le F_{m+1}$:

- $n = 1 \Rightarrow f(n) = 0$
- $n = 2 \Rightarrow f(n) = 2$
- $n = 3, 5 \Rightarrow f(n) = 3, 4$ respectively
- $n = 6, 7, 8 \Rightarrow f(n) = 5$, etc.

41 [M25]

Let $f(x) = \lfloor x + \varphi^{-1} \rfloor$, where $\varphi = (1+\sqrt{5})/2$. If the (Zeckendorf) Fibonacci representation of n is

$$n = F_{k_1} + F_{k_2} + \cdots + F_{k_r} \qquad (k_1 \gg k_2 \gg \cdots \gg k_r \gg 0, \text{ so } k_{i+1} \le k_i - 2),$$

prove

$$F_{k_1+1} + F_{k_2+1} + \cdots + F_{k_r+1} \;=\; f(\varphi n) = \left\lfloor \varphi n + \varphi^{-1} \right\rfloor,$$

and find the analogous formula for $F_{k_1-1} + \cdots + F_{k_r-1}$.

We will use Binet's formula and the basic facts about $\varphi' = 1 - \varphi = -\varphi^{-1}$. In §1.2.8 it is shown that

$$F_m = \frac{\varphi^m - \varphi'^{\,m}}{\sqrt{5}} \qquad (m \in \mathbb{Z}),$$

and Exercise 34 gives the Zeckendorf form (no consecutive indices, smallest index ≥ 2).

1) Shift up by 1: $\sum F_{k_i+1} = \lfloor \varphi n + \varphi^{-1} \rfloor$

Write the Zeckendorf sum $n = \sum_i F_{k_i}$. Using Binet and summing termwise,

$$n \;=\; \sum_i \frac{\varphi^{k_i} - \varphi'^{\,k_i}}{\sqrt{5}}.$$

Now consider the "+1 shift"

$$S_+ \;:=\; \sum_i F_{k_i+1} \;=\; \sum_i \frac{\varphi^{k_i+1} - \varphi'^{\,k_i+1}}{\sqrt{5}} \;=\; \frac{\varphi \sum_i \varphi^{k_i} - \varphi' \sum_i \varphi'^{\,k_i}}{\sqrt{5}}.$$

Compare this with $\varphi n = \frac{\varphi \sum_i \varphi^{k_i} - \varphi \sum_i \varphi'^{\,k_i}}{\sqrt{5}}$. Subtract:

$$S_+ - \varphi n = \frac{(-\varphi') - (-\varphi)}{\sqrt{5}} \sum_i \varphi'^{\,k_i} = \frac{\varphi - \varphi'}{\sqrt{5}} \sum_i \varphi'^{\,k_i} = \sum_i \varphi'^{\,k_i},$$

since $\varphi - \varphi' = \sqrt{5}$. Therefore

$$\boxed{S_+ = \varphi n + \sum_i \varphi'^{\,k_i}}.$$

Because $\varphi' = -\varphi^{-1}$ and the Zeckendorf indices satisfy $k_i \geq 2$ and have gaps ≥ 2, the "error term" $E := \sum_i \varphi'^{\,k_i}$ is squeezed between the two alternating geometric tails:

$$\underbrace{\varphi'^3 + \varphi'^5 + \cdots}_{=\,\varphi^{-1}-1} \;\leq\; E \;\leq\; \underbrace{\varphi'^2 + \varphi'^4 + \cdots}_{=\,\varphi^{-1}}.$$

Indeed, the largest possible (nonnegative) contribution comes by taking only even exponents (bounded by the even-power tail), and the most negative by taking only odd exponents ≥ 3 (bounded below by the odd-power tail). These sums evaluate to

$$\sum_{j\geq 1} \varphi'^{2j} = \frac{\varphi'^2}{1-\varphi'^2} = \frac{1/\varphi^2}{1-1/\varphi^2} = \frac{1}{\varphi}, \qquad \sum_{j\geq 1} \varphi'^{2j+1} = \varphi^{-1} - 1.$$

Hence

$$\varphi n + (\varphi^{-1} - 1) \;\leq\; S_+ \;\leq\; \varphi n + \varphi^{-1}.$$

This interval has length 1, so it contains exactly one integer; that integer is $\lfloor \varphi n + \varphi^{-1} \rfloor$. Therefore

$$\boxed{F_{k_1+1} + \cdots + F_{k_r+1} = \lfloor \varphi n + \varphi^{-1} \rfloor = f(\varphi n)}.$$

2) Shift down by 1: $\sum F_{k_i-1} = \lfloor \varphi^{-1} n + \varphi^{-1} \rfloor$

Define

$$S_- \;:=\; \sum_i F_{k_i-1} = \sum_i \frac{\varphi^{k_i-1} - \varphi'^{\,k_i-1}}{\sqrt{5}} = \frac{\varphi^{-1} \sum_i \varphi^{k_i} - \varphi'^{-1} \sum_i \varphi'^{\,k_i}}{\sqrt{5}}.$$

Compare with $\varphi^{-1} n = \frac{\varphi^{-1} \sum_i \varphi^{k_i} - \varphi^{-1} \sum_i \varphi'^{\,k_i}}{\sqrt{5}}$. Subtract:

$$S_- - \varphi^{-1} n = \frac{(-\varphi'^{-1}) - (-\varphi^{-1})}{\sqrt{5}} \sum_i \varphi'^{\,k_i} = \frac{\varphi^{-1} - \varphi'^{-1}}{\sqrt{5}} \sum_i \varphi'^{\,k_i}.$$

But $\varphi'^{-1} = -\varphi$, and $\varphi^{-1} - \varphi'^{-1} = \varphi^{-1} + \varphi = \sqrt{5}$. Thus

$$S_- - \varphi^{-1} n = \sum_i \varphi'^{\,k_i}, \quad \text{so} \quad \boxed{S_- = \varphi^{-1} n + \sum_i \varphi'^{\,k_i}}.$$

The same bounds on $\sum_i \varphi'^{\,k_i}$ therefore give

$$\varphi^{-1} n + (\varphi^{-1} - 1) \;\le\; S_- \;\le\; \varphi^{-1} n + \varphi^{-1},$$

so again $S_- = \lfloor \varphi^{-1} n + \varphi^{-1} \rfloor = f(\varphi^{-1} n)$. Hence

$$\boxed{F_{k_1-1} + \cdots + F_{k_r-1} = \lfloor \varphi^{-1} n + \varphi^{-1} \rfloor = f(\varphi^{-1} n)}.$$

3) Quick sanity check (example)

Take $n = 100$. Its Zeckendorf form is $100 = F_{11} + F_6 + F_4 = 89 + 8 + 3$. Then

$$\sum F_{k_i+1} = F_{12} + F_7 + F_5 = 144 + 13 + 5 = 162.$$

And $\lfloor \varphi \cdot 100 + \varphi^{-1} \rfloor = \lfloor 161.803\ldots + 0.618\ldots \rfloor = 162$, as required. Similarly,

$$\sum F_{k_i-1} = F_{10} + F_5 + F_3 = 55 + 5 + 2 = 62,$$

and $\lfloor \varphi^{-1} \cdot 100 + \varphi^{-1} \rfloor = \lfloor 61.803\ldots + 0.618\ldots \rfloor = 62$.

42 [M26]

We prove uniqueness and existence.

1) A key identity ("shift")

Assume such a representation exists. Using the standard Fibonacci addition law

$$F_{u+v} = F_{u-1}F_v + F_uF_{v+1} \quad (u, v \in \mathbb{Z}),$$

we get, for every integer N,

$$\sum_{i=1}^{r} F_{k_i+N} = \sum_{i=1}^{r}(F_{k_i-1}F_N + F_{k_i}F_{N+1}) = \underbrace{\Big(\sum F_{k_i-1}\Big)}_{=n-m}F_N + \underbrace{\Big(\sum F_{k_i}\Big)}_{=m}F_{N+1}.$$

Since $F_{N+1} - F_N = F_{N-1}$, this simplifies to the shift identity

$$\boxed{\sum\nolimits_i F_{k_i+N} = mF_{N-1} + nF_N} \qquad (\ )$$

2) Uniqueness

Fix N large enough that the right side of ( ) is positive. Because the indices k_i differ by at least 2, the sum $\sum_i F_{k_i+N}$ is a Zeckendorf sum (no adjacent indices). Zeckendorf's theorem (Exercise 34) says such a representation is unique; therefore the set $\{k_i\}$ is unique.

(This is exactly the book's hint: if two different index-sets existed, ( ) would give two different Zeckendorf representations of the same integer for large N, contradicting uniqueness in Ex. 34.)

3) Existence (two clean routes)

Route A (via base-φ digits — short and elegant). Consider the real number $x := m + n\varphi$ ($\varphi = \frac{1+\sqrt{5}}{2}$). By Exercise 35, every nonnegative real has a unique base-φ expansion with digits 0/1 and no adjacent 1's:

$$x = \sum_j \varphi^{t_j} \quad \text{with } t_1 > t_2 > \cdots,\ t_j - t_{j+1} \geq 2.$$

Use $\varphi^k = F_k\varphi + F_{k-1}$ to rewrite

$$m + n\varphi = \sum_j (F_{t_j}\varphi + F_{t_j-1}) = \varphi\sum_j F_{t_j} + \sum_j F_{t_j-1}.$$

Since 1 and φ are linearly independent over $\mathbb{Q}$, we must have

$$n = \sum_j F_{t_j}, \qquad m = \sum_j F_{t_j - 1}.$$

Setting $k_i := t_i - 1$ gives the desired representation with $k_1 \gg k_2 \gg \cdots$.

Route B (matching the book's official answer). Define $T_N := mF_{N-1} + nF_N$. Using Binet's formula one shows

$$T_{N+1} = \left\lfloor \varphi T_N + \varphi^{-1} \right\rfloor \quad \text{for all sufficiently large } N,$$

because the tiny φ'-term ($\varphi' = 1 - \varphi = -\varphi^{-1}$) becomes $< \varphi^{-2}$ in magnitude. Exercise 41 identifies this rounding as the operation that shifts a Zeckendorf sum "up by one index," hence for large N there is a set of nonadjacent indices with $T_N = \sum_i F_{k_i+N}$. By ( ), this holds for all N, and taking $N = 0, 1$ yields the required m, n.

Worked example

Take k-set $\{7, 4, 2\}$ (note $7 \gg 4 \gg 2$). Then

$$m = F_7 + F_4 + F_2 = 13 + 3 + 1 = 17, \quad n = F_8 + F_5 + F_3 = 21 + 5 + 2 = 28.$$

Check ( ) with $N = 3$:

$$mF_2 + nF_3 = 17 \cdot 1 + 28 \cdot 2 = 73 \quad \text{and} \quad \sum_i F_{k_i+3} = F_{10} + F_7 + F_5 = 55 + 13 + 5 = 73.$$

So $(m, n) = (17, 28)$ indeed arises from $\{7, 4, 2\}$, and the set is unique.

Remark (extension to signed pairs)

The same reasoning extends: a joint representation (with the same nonadjacency rule) exists for possibly negative m, n iff $m + \varphi n \geq 0$. Necessity is clear from $m + \varphi n = \sum_i \varphi^{k_i+1} \geq 0$; sufficiency follows by the base-φ route above.

1.2.9 Generating functions

1 [M12]

1) Ordinary generating function (OGF)

Define

$$G(z) \;=\; \sum_{n\geq 0} a_n z^n \;=\; \sum_{n\geq 0}(2^n+3^n)z^n \;=\; \sum_{n\geq 0}(2z)^n \;+\; \sum_{n\geq 0}(3z)^n.$$

Each sum is geometric:

$$\sum_{n\geq 0}(cz)^n \;=\; \frac{1}{1-cz} \quad (|z|<1/|c|).$$

Hence

$$\boxed{G(z) = \frac{1}{1-2z} + \frac{1}{1-3z}}$$

with radius of convergence $|z| < \min\{1/2,\, 1/3\} = 1/3$.

If you prefer a single rational function, combine:

$$\frac{1}{1-2z} + \frac{1}{1-3z} = \frac{(1-3z)+(1-2z)}{(1-2z)(1-3z)} = \boxed{\frac{2-5z}{(1-2z)(1-3z)} = \frac{2-5z}{1-5z+6z^2}}.$$

2) Quick checks

- Constant term: $G(0) = 1 + 1 = 2 = a_0$.
- Coefficient of z: from the two series it's $2 + 3 = 5 = a_1$.
- Coefficient of z^2: $4 + 9 = 13 = a_2$.

- The denominator $(1-2z)(1-3z) = 1 - 5z + 6z^2$ implies the recurrence

$$a_n = 5a_{n-1} - 6a_{n-2} \quad (n \geq 2), \qquad a_0 = 2,\ a_1 = 5,$$

which the sequence $2^n + 3^n$ indeed satisfies.

3) (Bonus) Exponential generating function (EGF)

If the exercise is interpreted as asking for the EGF,

$$\sum_{n\geq 0} \frac{a_n}{n!} z^n = \sum_{n\geq 0} \frac{2^n}{n!} z^n + \sum_{n\geq 0} \frac{3^n}{n!} z^n = \boxed{e^{2z} + e^{3z}}.$$

2 [M13]

In §1.2.9, Knuth defines a binomial (a.k.a. Cauchy–binomial) convolution

$$c_n \;=\; \sum_{k=0}^{n} \binom{n}{k} a_k\, b_{n-k} \qquad \text{(this is Eq. (10) in the book)}$$

and then states Eq. (11): If we use exponential generating functions (EGFs)

$$A(z) = \sum_{n\geq 0} \frac{a_n}{n!} z^n, \quad B(z) = \sum_{n\geq 0} \frac{b_n}{n!} z^n, \quad C(z) = \sum_{n\geq 0} \frac{c_n}{n!} z^n,$$

then

$$\boxed{C(z) = A(z)\, B(z)}.$$

Algebraic proof (formal power series)

Multiply the two EGFs and collect coefficients:

$$\begin{aligned} A(z)B(z) &= \Big(\sum_{k\geq 0} \frac{a_k}{k!} z^k\Big)\Big(\sum_{\ell\geq 0} \frac{b_\ell}{\ell!} z^\ell\Big) \\ &= \sum_{n\geq 0}\Big(\sum_{k=0}^{n} \frac{a_k}{k!}\frac{b_{n-k}}{(n-k)!}\Big) z^n \\ &= \sum_{n\geq 0}\Big(\sum_{k=0}^{n} \binom{n}{k} a_k b_{n-k}\Big)\frac{z^n}{n!} = \sum_{n\geq 0} \frac{c_n}{n!} z^n \;=\; C(z). \end{aligned}$$

The key identity is $\binom{n}{k} = \dfrac{n!}{k!(n-k)!}$. This is precisely Eq. (11). (Scribd)

Combinatorial proof (why EGFs multiply)

Interpret a_n and b_n as counting labeled structures of size n from classes $\mathcal{A}$ and $\mathcal{B}$. To build a labeled $\mathcal{C}$-structure of total size n as an ordered pair (A, B) with disjoint label sets:

- choose a k-subset of the n labels for A — $\binom{n}{k}$ ways;
- place an $\mathcal{A}$-structure on those k labels — a_k ways;
- place a $\mathcal{B}$-structure on the remaining $n-k$ labels — b_{n-k} ways.

Summing over k gives $c_n = \sum_k \binom{n}{k} a_k b_{n-k}$. The labeled product construction corresponds exactly to multiplication of EGFs, hence $C(z) = A(z)B(z)$.

Quick checks

1. $a_n \equiv 1$, $b_n \equiv 1$. Then $c_n = \sum_k \binom{n}{k} = 2^n$. EGFs: $A(z) = B(z) = e^z \Rightarrow C(z) = e^{2z}$, whose nth coefficient is $2^n/n!$. Works.
2. $a_n = n!$, $b_n \equiv 1$. Then $A(z) = \sum z^n = 1/(1-z)$, $B(z) = e^z$. Product $C(z) = \frac{e^z}{1-z}$ has coefficients $c_n/n! = \sum_{k=0}^{n} \frac{1}{(n-k)!}$, i.e. $c_n = n! \sum_{j=0}^{n} \frac{1}{j!} = \sum_{k=0}^{n} \binom{n}{k} k!$, exactly the convolution.

3 [HM21]

Step 1 — Start from (18)

The ordinary generating function for the harmonic numbers (with H $= 0$) is given as

$$G(z) \;=\; \sum_{n\geq 0} H_n z^n \;=\; \frac{1}{1-z}\,\ln\frac{1}{1-z}.$$

Step 2 — Differentiate $G(z)$

Differentiate termwise and also differentiate the closed form:

$$G'(z) \;=\; \sum_{n\geq 1} n\, H_n\, z^{\,n-1}$$

and, using the product/chain rule,

$$\frac{d}{dz}\Big(\frac{1}{1-z}\ln\frac{1}{1-z}\Big) = \frac{1}{(1-z)^2} + \frac{1}{(1-z)^2}\ln\frac{1}{1-z} = \frac{1}{(1-z)^2} \;+\; \frac{G(z)}{1-z}.$$

Hence

$$\sum_{n\geq 1} n\, H_n\, z^{\,n-1} \;=\; \frac{1}{(1-z)^2} \;+\; \frac{G(z)}{1-z}. \qquad (\;)$$

Step 3 — Recognize the prefix–sum generating function

For any sequence a_n with generating function $A(z) = \sum_{n\geq 0} a_n z^n$, the generating function of its prefix sums $s_n = \sum_{k=0}^{n} a_k$ is

$$S(z) \;=\; \sum_{n\geq 0} s_n z^n \;=\; \frac{A(z)}{1-z}.$$

(See Eq. (7) in the text.) Applying this with $a_n = H_n$ gives

$$S(z) \;=\; \frac{G(z)}{1-z} \;=\; \frac{1}{(1-z)^2}\,\ln\frac{1}{1-z}.$$

Step 4 — Compare coefficients

Rewrite ( ) by shifting the index on the left:

$$\sum_{n\geq 0}(n+1)\, H_{n+1}\, z^{\,n} \;=\; \frac{1}{(1-z)^2} \;+\; S(z).$$

Extract coefficients of z^n. Since $[z^n](1-z)^{-2} = n+1$, we obtain

$$(n+1)\, H_{n+1} \;=\; (n+1) \;+\; \sum_{k=0}^{n} H_k.$$

Therefore,

$$\boxed{\sum_{k=0}^{n} H_k \;=\; (n+1)(H_{n+1}-1)}.$$

Equivalently (since $H_0 = 0$),

$$\sum_{k=1}^{n} H_k = (n+1)H_{n+1} - (n+1).$$

This is exactly the relation requested, derived by differentiating (18) and comparing with the prefix–sum generating function from (7).

4 [M01]

The two identities (from §1.2.9)

- Binomial theorem:

$$(1+z)^r = \sum_{k\geq 0} \binom{r}{k} z^k. \tag{19}$$

- Generalized "binomial" expansion (with a parameter t):

$$x^r = \sum_{k\geq 0} \binom{r-kt}{k} \frac{r}{r-kt} z^k, \quad \text{where } x^{t+1} = x^t + z \text{ and } x = 1 \text{ at } z = 0. \tag{21}$$

(Equivalent first terms: $x^r = 1 + rz + \frac{r(r-2t-1)}{2} z^2 + \cdots$.)

Take the special case $t = 0$ in (21).

1. The defining relation for x becomes

$$x^{t+1} = x^t + z \quad \Rightarrow \quad x^1 = x^0 + z \quad \Rightarrow \quad x = 1 + z.$$

Therefore the left-hand side of (21) is $x^r = (1+z)^r$.

2. The coefficients on the right-hand side simplify to

$$\binom{r-kt}{k} \frac{r}{r-kt}\Big|_{t=0} = \binom{r}{k} \cdot \frac{r}{r} = \binom{r}{k}.$$

Thus (21) reduces exactly to

$$(1+z)^r = \sum_{k\geq 0} \binom{r}{k} z^k,$$

which is Eq. (19).

3. (Sanity check on low-order terms.) In (21) the quadratic coefficient is $\frac{r(r-2t-1)}{2}$; setting $t = 0$ gives $\frac{r(r-1)}{2}$, which matches the binomial expansion's z^2-term.

Therefore, Eq. (19) is the $t = 0$ specialization of Eq. (21).

5 [M20]

Base case

For $n = 0$: $(e^z - 1)^0/0! = 1$. On the right, $\left\{ {k \atop 0} \right\} = 0$ for $k > 0$ and $= 1$ for $k = 0$, so $\sum_{k\geq 0} \left\{ {k \atop 0} \right\} z^k/k! = 1$. Hence the identity holds.

(You can also check $n = 1$: $(e^z - 1) = \sum_{k\geq 1} z^k/k!$ since $\left\{ {k \atop 1} \right\} = 1$ for $k \geq 1$.)

Induction step

Assume the formula holds for some $n \geq 0$:

$$\frac{(e^z - 1)^n}{n!} = \sum_{k\geq 0} \left\{ {k \atop n} \right\} \frac{z^k}{k!}.$$

Multiply both sides by $\dfrac{e^z - 1}{n + 1}$:

$$\frac{(e^z - 1)^{n+1}}{(n + 1)!} = \frac{e^z - 1}{n + 1} \sum_{k\geq 0} \left\{ {k \atop n} \right\} \frac{z^k}{k!}.$$

Use the Cauchy product with $e^z = \sum_{m\geq 0} \frac{z^m}{m!}$. The coefficient of $z^r/r!$ on the right equals

$$\frac{1}{n + 1} \sum_{m=1}^{r} \binom{r}{m} \left\{ {r - m \atop n} \right\} = \frac{1}{n + 1} \sum_{j=0}^{r-1} \binom{r}{j} \left\{ {j \atop n} \right\} \quad (r \geq 0).$$

Therefore

$$\frac{(e^z - 1)^{n+1}}{(n + 1)!} = \sum_{r\geq 0} \left[\frac{1}{n + 1} \sum_{j=0}^{r-1} \binom{r}{j} \left\{ {j \atop n} \right\} \right] \frac{z^r}{r!}.$$

It remains to identify the bracketed coefficient with $\left\{ {r \atop n+1} \right\}$. This is the standard "marked block" recurrence for Stirling numbers of the second kind:

Claim. For $r \geq 0$ and $n \geq 0$,

$$\left\{ {r \atop n + 1} \right\} = \frac{1}{n + 1} \sum_{j=0}^{r-1} \binom{r}{j} \left\{ {j \atop n} \right\}.$$

Combinatorial proof of the claim. The left-hand side counts partitions of an r-set into $n+1$ nonempty blocks. Multiply by $n+1$ to mark (distinguish) one block: $(n+1)\left\{ {r \atop n+1} \right\}$ counts "a partition together with a chosen (marked) block". To construct such an object, choose the set of elements not in the marked block—say j elements (there are $\binom{r}{j}$ ways), then partition those j elements into n blocks (there are $\left\{ {j \atop n} \right\}$ ways), and put all remaining $r-j \geq 1$ elements into the marked block. Summing over $j = 0, \dots, r-1$ yields

$$(n+1)\left\{ {r \atop n+1} \right\} = \sum_{j=0}^{r-1} \binom{r}{j} \left\{ {j \atop n} \right\},$$

which is exactly the claim after dividing by $n+1$.

With the claim, the coefficient of $z^r/r!$ above is $\left\{ {r \atop n+1} \right\}$, hence

$$\frac{(e^z-1)^{n+1}}{(n+1)!} = \sum_{r\geq 0} \left\{ {r \atop n+1} \right\} \frac{z^r}{r!},$$

completing the induction. Therefore Eq. (23) holds for all $n \geq 0$.

6 [HM15]

Let $a_n = \displaystyle\sum_{k=1}^{n-1} \frac{1}{k(n-k)}$ (empty sum $a_1 = 0$). Find the ordinary generating function $A(z) = \sum_{n\geq 1} a_n z^n$. Then differentiate $A(z)$ and express its coefficients via harmonic numbers $H_m = \sum_{j=1}^{m} \frac{1}{j}$.

1) Closed form for a_n

Use the partial fraction identity

$$\frac{1}{k(n-k)} = \frac{1}{n}\left(\frac{1}{k} + \frac{1}{n-k}\right).$$

Hence

$$a_n = \frac{1}{n}\sum_{k=1}^{n-1}\left(\frac{1}{k} + \frac{1}{n-k}\right) = \frac{2}{n}\sum_{k=1}^{n-1}\frac{1}{k} = \frac{2}{n}\,H_{n-1}.$$

2) Generating function

We use two standard facts from the section:

- Integration trick for OGFs:

$$\int_0^z \Big(\sum_{m\geq 0} b_m t^m\Big) dt = \sum_{n\geq 1} \frac{b_{n-1}}{n}\, z^n. \quad \text{(Eq. 15)}$$

- Generating function of harmonic numbers:

$$\sum_{m\geq 0} H_m z^m = \frac{1}{1-z} \ln \frac{1}{1-z}. \quad \text{(Eq. 18)}$$

Since $a_n = \dfrac{2}{n} H_{n-1}$, we get

$$A(z) = \sum_{n\geq 1} \frac{2H_{n-1}}{n} z^n = 2\int_0^z \Big(\sum_{m\geq 0} H_m t^m\Big)\, dt = 2\int_0^z \frac{\ln\frac{1}{1-t}}{1-t}\, dt.$$

Let $u(t) = \ln\frac{1}{1-t}$; then $u'(t) = \frac{1}{1-t}$. The integral is $\int u\, u'\, dt = \frac{1}{2}u^2$. With the factor 2,

$$\boxed{A(z) = \Big(\ln \frac{1}{1-z}\Big)^2} \qquad (|z| < 1).$$

3) Differentiate and read coefficients

Differentiate:

$$A'(z) = 2\frac{\ln\frac{1}{1-z}}{1-z}.$$

Using the harmonic-number generating function again,

$$\frac{\ln\frac{1}{1-z}}{1-z} = \sum_{m\geq 0} H_m z^m,$$

so

$$A'(z) = 2\sum_{m\geq 0} H_m z^m.$$

But $A'(z) = \sum_{n\geq 1} n a_n z^{n-1}$. Matching coefficients of z^{n-1} gives

$$n a_n = 2H_{n-1} \quad \Longrightarrow \quad \boxed{a_n = \frac{2}{n} H_{n-1}}.$$

Quick check

$n = 2:\ a_2 = 1.\ n = 3:\ a_3 = \frac{1}{2} + \frac{1}{2} = 1.\ n = 4:\ a_4 = \frac{1}{3} + \frac{1}{4} + \frac{1}{3} = \frac{11}{12}$. Our formula yields $2H_1/2 = 1$, $2H_2/3 = 1$, $2H_3/4 = \frac{11}{12}$, all consistent.

Answer: Generating function $A(z) = \left(\ln\frac{1}{1-z}\right)^2$; upon differentiation, $A'(z) = 2\,\dfrac{\ln\frac{1}{1-z}}{1-z} = 2\sum_{m\geq 0} H_m z^m$, hence $a_n = \dfrac{2}{n}H_{n-1}$.

7 [15]

In §1.2.9, we have

$$\ln G(z) = \sum_{k\geq 1} \frac{S_k\, z^k}{k} \quad \text{with } S_k = \sum_{j=1}^{n} x_j^k,$$

and Knuth defines $G(z)$ by the reciprocal product and arrives at (37). From this, (38) is obtained by exponentiating and expanding.

Step-by-step verification to (38)

1. Exponentiate the logarithm. Because $G(0) = 1$ and $\ln G(z)$ is a formal power series with zero constant term, the formal identities $\exp(\ln G(z)) = G(z)$ and $\ln(\exp H(z)) = H(z)$ are valid. Hence

$$G(z) = \exp\Big(\sum_{k\geq 1} \frac{S_k\, z^k}{k}\Big).$$

This is exactly the first equality in the displayed chain preceding (38).

2. Turn the exponential of a sum into a product. In the ring of formal power series (which is commutative), exponentials distribute over sums:

$$\exp\Big(\sum_{k\geq 1} A_k\Big) = \prod_{k\geq 1} \exp(A_k).$$

Taking $A_k = \frac{S_k z^k}{k}$ gives

$$G(z) = \prod_{k\geq 1} \exp\Big(\frac{S_k z^k}{k}\Big).$$

This is the product in the second equality just before (38).

3. Expand each factor as a series. Using $\exp(u) = \sum_{r\geq 0} u^r/r!$,

$$\exp\Big(\frac{S_k z^k}{k}\Big) = \sum_{r\geq 0} \frac{1}{r!}\Big(\frac{S_k z^k}{k}\Big)^r = \sum_{r\geq 0} \frac{S_k^r}{k^r\, r!}\, z^{kr}.$$

Knuth writes out the first few factors (for $k = 1, 2, ...$) in exactly this way (see the line with $(1 + S_1 z + S_1^2 z^2/2! + \cdots)(1 + S_2 z^2/2 + \cdots)\cdots$).

4. Multiply the infinite product and collect coefficients. The coefficient of z^m in the product is obtained by choosing, for each $k \geq 1$, a term z^{kr_k} from the k-th factor, with nonnegative integers r_k, such that

$$\sum_{k\geq 1} k\, r_k = m.$$

The contribution of a fixed selection $(r_1, r_2, \dots)$ is

$$\prod_{k\geq 1} \frac{S_k^{r_k}}{k^{r_k}\, r_k!}.$$

Relabel r_k as k_k (Knuth's notation) and truncate at $k = m$ since higher k cannot contribute to z^m. Therefore

$$[z^m]\, G(z) = \sum_{\substack{k_1,k_2,\dots,k_m\geq 0\\ k_1+2k_2+\cdots+mk_m=m}} \frac{S_1^{k_1}}{1^{k_1} k_1!}\, \frac{S_2^{k_2}}{2^{k_2} k_2!} \cdots \frac{S_m^{k_m}}{m^{k_m} k_m!}.$$

This is precisely the parenthesized quantity in (38), which Knuth denotes by h_m, so

$$G(z) = \sum_{m\geq 0} h_m z^m,$$

with h_m given by the inner sum.

5. Interpretation via partitions. The constraint $k_1 + 2k_2 + \cdots + mk_m = m$ specifies a partition of m: it says that the part j appears k_j times. Hence the number of summands for fixed m is $p(m)$, the number of partitions of m (as the text states just below (38)).

Quick sanity checks (small m)

Using the formula above:

- $m = 0$: only the all-zero choice $\quad h_0 = 1$.

- $m = 1$: $(k_1) = (1) \quad h_1 = S_1$.
- $m = 2$: tuples $(k_1, k_2) = (2, 0), (0, 1)$ give

$$h_2 = \frac{S_1^2}{2!} + \frac{S_2}{2}.$$

- $m = 3$: tuples $(3, 0, 0), (1, 1, 0), (0, 0, 1)$ give

$$h_3 = \frac{S_1^3}{6} + \frac{S_1 S_2}{2} + \frac{S_3}{3}.$$

These match the structure immediately implied by the product display before (38).

8 [M23]

Let $G(z) = \sum_{n \geq 0} p(n)\, z^n$ be the ordinary generating function for the partition numbers.

A (unrestricted) partition of n allows any number of 1's, any number of 2's, any number of 3's, etc. For each part size $k \geq 1$, the contribution to the exponent of z is a multiple of k: $0, k, 2k, 3k, \ldots$. Thus the generating series for how many k's we choose is

$$1 + z^k + z^{2k} + z^{3k} + \cdots = \frac{1}{1 - z^k} \qquad (|z| < 1).$$

Different part sizes are chosen independently, so we multiply these series over all $k \geq 1$ (Cauchy product). Therefore

$$G(z) \;=\; \prod_{k=1}^{\infty} \frac{1}{1 - z^k}.$$

Expanding this infinite product, the coefficient of z^n counts exactly the number of solutions $\sum_{k \geq 1} k\, m_k = n$ with integers $m_k \geq 0$, i.e., the number of partitions of n. This is the standard "product-for-multisets" principle that also underlies Knuth's derivation $G(z) = \prod_j (1 - x_j z)^{-1}$ for multisets; here we specialize to $x_j = z^{j-1}$ (conceptually) and let the number of factors go to infinity.

So the answer is:

$$\boxed{\sum_{n \geq 0} p(n)\, z^n \;=\; \prod_{k=1}^{\infty} \frac{1}{1 - z^k}.}$$

Quick check

Expanding a few terms gives

$$\prod_{k\geq 1}\frac{1}{1-z^k} = 1 + z + 2z^2 + 3z^3 + 5z^4 + 7z^5 + 11z^6 + \cdots,$$

so $p(0) = 1, p(1) = 1, p(2) = 2, p(3) = 3, p(4) = 5, p(5) = 7, p(6) = 11$, as expected.

9 [M11]

With h_m and the power sums S_j defined by

$$h_m = \sum_{1\leq j_1\leq\cdots\leq j_m\leq n} x_{j_1}\cdots x_{j_m}, \qquad S_j = \sum_{k=1}^{n} x_k^j,$$

and with the generating function $G(z) = \sum_{m\geq 0} h_m z^m$, find h_4 in terms of S_1, S_2, S_3, S_4.

From the text we have

$$\ln G(z) = \sum_{k\geq 1}\frac{S_k}{k}\,z^k,$$

hence

$$G(z) = \exp\Big(S_1 z + \frac{S_2}{2}z^2 + \frac{S_3}{3}z^3 + \frac{S_4}{4}z^4 + \cdots\Big).$$

Therefore $h_m = [z^m]G(z) = [z^m]\exp A(z)$ where $A(z) = a_1 z + a_2 z^2 + a_3 z^3 + a_4 z^4 + \cdots$ with $a_1 = S_1,\ a_2 = S_2/2,\ a_3 = S_3/3,\ a_4 = S_4/4$. (The formula for $\ln G$ is Eq. (37) on the same page.)

Expand e^A up to degree 4:

$$e^A = 1 + A + \frac{A^2}{2!} + \frac{A^3}{3!} + \frac{A^4}{4!} + O(z^5).$$

Collect the z^4-terms:

- From A: a_4.
- From $A^2/2!$: $\frac{1}{2}(2a_1a_3 + a_2^2) = a_1a_3 + \frac{1}{2}a_2^2$.
- From $A^3/3!$: $\frac{1}{6}\cdot 3a_1^2a_2 = \frac{1}{2}a_1^2a_2$.
- From $A^4/4!$: $\frac{1}{24}a_1^4$.

Thus

$$h_4 \;=\; a_4 + a_1 a_3 + \tfrac{1}{2}a_2^2 + \tfrac{1}{2}a_1^2 a_2 + \tfrac{1}{24}a_1^4.$$

Substitute $a_1 = S_1,\ a_2 = S_2/2,\ a_3 = S_3/3,\ a_4 = S_4/4$:

$$\boxed{h_4 = \frac{S_4}{4} + \frac{S_1 S_3}{3} + \frac{S_2^2}{8} + \frac{S_1^2 S_2}{4} + \frac{S_1^4}{24}}$$

A convenient equivalent form (multiply by 24) is

$$\boxed{24\, h_4 \;=\; S_1^4 + 6S_1^2 S_2 + 3S_2^2 + 8S_1 S_3 + 6S_4}$$

Quick sanity checks

- If $n = 1$ (single variable x), then $h_4 = x^4$ and $S_j = x^j$; plugging into the formula gives x^4.
- The expression matches the general pattern for complete homogeneous symmetric polynomials derived from $G(z) = \prod_{i=1}^{n}(1 - x_i z)^{-1}$.

10 [M25]

1) Generating function

Because a term $x_{j_1} \cdots x_{j_m}$ picks at most one factor from each $(1 + x_i z)$, we have

$$E(z) \;\stackrel{\text{def}}{=}\; \sum_{m \geq 0} e_m z^m = \prod_{i=1}^{n}(1 + x_i z).$$

Taking logs and expanding $\ln(1 + x_i z)$ gives

$$\ln E(z) = \sum_{i=1}^{n} \sum_{k \geq 1} \frac{(-1)^{k+1}}{k}\, x_i^k z^k = \sum_{k \geq 1} \frac{(-1)^{k+1}}{k}\, S_k\, z^k,$$

hence the equivalent exponential form

$$\boxed{E(z) = \exp\left(\sum_{k \geq 1} \frac{(-1)^{k+1}}{k}\, S_k\, z^k \right)}$$

(contrast with the complete symmetric case $\prod_i (1 - x_i z)^{-1}$ leading to $\exp(\sum S_k z^k / k)$).

2) Coefficient extraction (expression in terms of S_k)

From the exponential form,

$$E(z) = \exp\Big(\sum_{k\geq 1} c_k z^k\Big), \qquad c_k = \frac{(-1)^{k+1}}{k} S_k.$$

Using the standard exponential–generating expansion,

$$e_m = [z^m]E(z) = \sum_{\substack{k_1+2k_2+\cdots+mk_m=m \\ k_j\geq 0}} \prod_{j=1}^{m} \frac{c_j^{k_j}}{k_j!},$$

we obtain the compact closed form

$$\boxed{e_m = \sum_{k_1+2k_2+\cdots+mk_m=m} (-1)^{m+\sum_j k_j} \prod_{j=1}^{m} \frac{S_j^{k_j}}{j^{k_j}\, k_j!}}$$

(because $\sum_j jk_j = m$). Equivalently, the Newton identities follow immediately by differentiating $\ln E(z)$:

$$\boxed{m\, e_m = \sum_{k=1}^{m} (-1)^{k-1} e_{m-k}\, S_k \qquad (m \geq 1).}$$

3) Low–degree cases

Applying either formula (or Newton identities with $e_0 = 1$):

$$\begin{aligned}
e_1 &= S_1, \\
e_2 &= \frac{S_1^2 - S_2}{2}, \\
e_3 &= \frac{S_1^3 - 3S_1S_2 + 2S_3}{6} \;=\; \frac{S_1^3}{6} - \frac{S_1S_2}{2} + \frac{S_3}{3}, \\
e_4 &= \frac{S_1^4 - 6S_1^2S_2 + 3S_2^2 + 8S_1S_3 - 6S_4}{24} \\
&= \frac{S_1^4}{24} - \frac{S_1^2S_2}{4} + \frac{S_2^2}{8} + \frac{S_1S_3}{3} - \frac{S_4}{4}.
\end{aligned}$$

These match the requested expression of e_m via the power sums S_k, and the generating function agrees with the product definition above.

Side note: If $H(z) = \sum_{m\geq 0} h_m z^m = \prod_i (1 - x_i z)^{-1}$ is the complete symmetric G.F., then $E(z)H(-z) = 1$; this is another quick route to the Newton identities.

11 [M25]

Use the recurrence

$$h_n = \frac{1}{n}\left(S_1 h_{n-1} + S_2 h_{n-2} + \cdots + S_n h_0\right)$$

to express each power sum S_m in terms of the complete homogeneous symmetric polynomials $h_1, h_2, \ldots$, and determine the coefficient of the monomial $h_1^{k_1} h_2^{k_2} \cdots h_m^{k_m}$ in S_m when $k_1 + 2k_2 + \cdots + mk_m = m$.

Set-up with generating functions

Let

$$G(z) = \sum_{r\geq 0} h_r z^r = 1 + h_1 z + h_2 z^2 + \cdots.$$

From the section text we have the fundamental identity

$$\ln G(z) = \sum_{r\geq 1} \frac{S_r}{r}\, z^r.$$

(Equivalently, $S_r/r = [z^r] \ln G(z)$.)

Now write $G(z) = 1 + H(z)$ with $H(z) = \sum_{j\geq 1} h_j z^j$. Using the logarithm series

$$\ln(1 + X) = \sum_{K\geq 1} \frac{(-1)^{K-1}}{K} X^K,$$

we obtain

$$\ln G(z) = \sum_{K\geq 1} \frac{(-1)^{K-1}}{K}\, H(z)^K.$$

Extracting the coefficient

Expand $H(z)^K$ by the multinomial theorem. A term

$$h_1^{k_1} h_2^{k_2} \cdots h_m^{k_m}\; z^{1\cdot k_1 + 2\cdot k_2 + \cdots + m\cdot k_m}$$

appears exactly when the integers $k_1, \dots, k_m \geq 0$ satisfy

$$k_1 + 2k_2 + \cdots + mk_m = m, \qquad K := k_1 + \cdots + k_m.$$

Its coefficient in $H(z)^K$ is $\dfrac{K!}{k_1! \cdots k_m!}$. Therefore its coefficient in $\ln G(z)$ is

$$\frac{(-1)^{K-1}}{K} \cdot \frac{K!}{k_1! \cdots k_m!} \;=\; (-1)^{K-1} \frac{(K-1)!}{k_1! \cdots k_m!}.$$

But in $\ln G(z) = \sum_{r \geq 1} (S_r/r) z^r$, the coefficient of z^m equals S_m/m. Hence the coefficient of $h_1^{k_1} \cdots h_m^{k_m}$ in S_m is m times the number above.

Final formula

If $k_1, \dots, k_m \geq 0$ satisfy $k_1 + 2k_2 + \cdots + mk_m = m$ and $K = k_1 + \cdots + k_m$, then the coefficient of the monomial $h_1^{k_1} h_2^{k_2} \cdots h_m^{k_m}$ in S_m is

$$\boxed{(-1)^{K-1}\, m\, \frac{(K-1)!}{k_1!\, k_2! \cdots k_m!}}$$

(and it is 0 if the weighted sum condition fails). This follows directly from $\ln G(z) = \sum_{r \geq 1} S_r z^r / r$ with $G(z) = \sum_{r \geq 0} h_r z^r$.

Quick checks against the small cases in the text

- $m = 1$: only $k_1 = 1$ $(K = 1) \to$ coefficient $= 1$; so $S_1 = h_1$.
- $m = 2$: possibilities $(k_2 = 1)$ and $(k_1 = 2)$. Coefficients: $2 \cdot (+)(0)!/1! = 2$ for h_2, and $2 \cdot (-)1!/2! = -1$ for h_1^2; hence $S_2 = 2h_2 - h_1^2$.
- $m = 3$: $(k_3 = 1)$, $(k_1 = 1, k_2 = 1)$, $(k_1 = 3)$ give $3h_3 - 3h_1 h_2 + h_1^3$.

12 [M20]

General setup (bivariate OGFs)

For a double sequence $\{a_{m,n}\}_{m,n \geq 0}$, its (ordinary) bivariate generating function is

$$F(w, z) \;=\; \sum_{m \geq 0} \sum_{n \geq 0} a_{m,n}\, w^m z^n.$$

This is a *single* formal power series in the two indeterminates w and z. Coefficient extraction works as usual: $[w^m z^n]\, F(w, z) = a_{m,n}$. (Analytically, one may require $|(1 + w)z| < 1$ below; as a *formal* series no convergence assumption is needed.)

Apply to $a_{m,n} = \binom{n}{m}$

Start from the definition above and sum first over m for each fixed n:

$$\sum_{m\geq 0} \binom{n}{m} w^m = (1+w)^n \quad \text{(binomial theorem).}$$

Therefore

$$F(w,z) \;=\; \sum_{n\geq 0}\Big(\sum_{m\geq 0}\binom{n}{m} w^m\Big) z^n \;=\; \sum_{n\geq 0}\big((1+w)z\big)^n \;=\; \frac{1}{1-(1+w)z} \;=\; \frac{1}{1-z-wz}.$$

Thus the required generating function is

$$\boxed{F(w,z) = \frac{1}{1-(1+w)z}} \quad \text{(equivalently } \frac{1}{1-z-wz}\text{).}$$

You can verify correctness by expanding $1/(1-(1+w)z) = \sum_{n\geq 0}((1+w)z)^n$ and then expanding $(1+w)^n$; the coefficient of $w^m z^n$ is $\binom{n}{m}$, and it vanishes automatically for $m > n$.

13 [HM22]

Given a sequence $a_0, a_1, a_2, \ldots$ with ordinary generating function $G(z) = \sum_{k\geq 0} a_k z^k$, define the step function

$$f(x) = \sum_{k\geq 0} a_k \,[0 \leq k \leq x]$$

(the sum of the first $\lfloor x \rfloor + 1$ terms). Express the Laplace transform $L\{f\}(s) = \int_0^\infty e^{-st} f(t)\, dt$ in terms of G.

1. Write f using indicators and swap sum/integral For $t \geq 0$,

$$f(t) = \sum_{k\geq 0} a_k \, \mathbf{1}_{\{t\geq k\}}.$$

Hence

$$L\{f\}(s) \;=\; \int_0^\infty e^{-st}\left(\sum_{k\ge 0} a_k\,\mathbf{1}_{\{t\ge k\}}\right)dt \;=\; \sum_{k\ge 0} a_k \int_k^\infty e^{-st}\,dt,$$

where interchanging sum and integral is justified when $\Re(s) > 0$ and $|e^{-s}| < R$ with R the radius of convergence of G (then $\sum |a_k| e^{-\Re(s)k}$ converges absolutely, so Tonelli/Fubini applies).

2. Compute the inner integral For $\Re(s) > 0$,

$$\int_k^\infty e^{-st}\,dt = \frac{e^{-sk}}{s}.$$

3. Recognize the generating function Therefore

$$L\{f\}(s) = \frac{1}{s}\sum_{k\ge 0} a_k\, e^{-sk} = \frac{1}{s}\,G(e^{-s})\,.$$

Final formula and domain

$$\boxed{L\{f\}(s) = \frac{1}{s}\,G(e^{-s})} \qquad \text{valid for} \quad \Re(s) > 0 \text{ and } |e^{-s}| < R,$$

i.e., $\Re(s) > \max\{0, -\ln R\}$, so that both the Laplace integral and the substituted series converge.

Quick check

- If $a_k \equiv 1$, then $G(z) = \frac{1}{1-z}$. Our formula gives

$$L\{f\}(s) = \frac{1}{s}\cdot\frac{1}{1-e^{-s}},$$

  which is the classical Laplace transform of the staircase function $f(t) = \lfloor t \rfloor + 1$.

14 [HM21]

The identity states that, for $G(z) = \sum_{n\geq 0} a_n z^n$, a fixed modulus $m \geq 1$, and $\omega = e^{2\pi i/m}$,

$$\sum_{\substack{n\geq 0 \\ n\equiv r \pmod{m}}} a_n z^n \;=\; \frac{1}{m}\sum_{k=0}^{m-1} \omega^{-kr}\, G(\omega^k z) \qquad (0 \leq r < m).$$

Proof (step by step)

1. Expand the right-hand side using the power series for G:

$$\frac{1}{m}\sum_{k=0}^{m-1} \omega^{-kr} G(\omega^k z) = \frac{1}{m}\sum_{k=0}^{m-1} \omega^{-kr} \sum_{n\geq 0} a_n (\omega^k z)^n = \sum_{n\geq 0} a_n z^n \left(\frac{1}{m}\sum_{k=0}^{m-1} \omega^{k(n-r)}\right).$$

2. Use the roots-of-unity orthogonality:

$$\frac{1}{m}\sum_{k=0}^{m-1} \omega^{k(n-r)} = \begin{cases} 1, & n \equiv r \pmod{m}, \\ 0, & n \not\equiv r \pmod{m}. \end{cases}$$

Indeed, if $t = n - r$, then $\sum_{k=0}^{m-1} \omega^{kt} = \frac{1-\omega^{mt}}{1-\omega^t} = 0$ unless $m \mid t$; when $m \mid t$, every term is 1 and the sum is m.

3. Substituting this filter back into step (1) leaves exactly the terms with indices $n \equiv r \pmod{m}$:

$$\sum_{n\geq 0} a_n z^n \cdot [n \equiv r] \;=\; \sum_{\substack{n\geq 0 \\ n\equiv r \pmod{m}}} a_n z^n,$$

which is the desired identity.

No convergence hypotheses are needed if we interpret all series as formal power series, in the spirit of the section.

Quick check

For $m = 3$ and $r = 1$ we get

$$a_1 z + a_4 z^4 + a_7 z^7 + \cdots = \frac{1}{3}\Big(G(z) + \omega^{-1} G(\omega z) + \omega^{-2} G(\omega^2 z)\Big), \quad \omega = e^{2\pi i/3},$$

exactly as illustrated beneath Eq. (13).

15 [M28]

Let

$$G_n(z) \;=\; \sum_{k=0}^{n} \binom{n-k}{k} z^k \quad \Big(= \sum_{k=0}^{n} \binom{2k-n-1}{k} (-z)^k \Big).$$

By introducing the ordinary generating function $H(w) = \sum_{n\geq 0} G_n(z)\, w^n$, find a closed form for $G_n(z)$.

1) Recurrence and bivariate generating function

A direct "two-step" decomposition of the summand shows that the polynomials G_n satisfy

$$G_0(z) = 1, \quad G_1(z) = 1, \quad G_n(z) = G_{n-1}(z) + z\, G_{n-2}(z) \quad (n \geq 2).$$

(Combinatorially: choose whether the first item is a "1-step" or a "2-step" weighted by z. Algebraically it follows by the standard binomial identity $\binom{n-k}{k} = \binom{n-1-k}{k} + \binom{n-1-k}{k-1}$.)

Form the ordinary generating function in n,

$$H(w) \;=\; \sum_{n\geq 0} G_n(z)\, w^n.$$

Multiply the recurrence by w^n and sum over $n \geq 2$:

$$H(w) - G_0(z) - G_1(z)w = w(H(w) - G_0(z)) + z\, w^2 H(w).$$

With $G_0 = G_1 = 1$ this simplifies to

$$(1 - w - zw^2)\, H(w) = 1 \quad \Longrightarrow \quad \boxed{H(w) = \frac{1}{1 - w - zw^2}}.$$

This is the classic OGF of the Fibonacci-type recurrence.

2) Closed form ("Binet" formula) for $G_n(z)$

Factor the denominator:

$$1 - w - zw^2 = (1 - \alpha w)(1 - \beta w),$$

where α, β are the roots of $r^2 - r - z = 0$, i.e.

$$\alpha = \frac{1 + \sqrt{1 + 4z}}{2}, \qquad \beta = \frac{1 - \sqrt{1 + 4z}}{2}, \qquad \alpha + \beta = 1,\ \alpha\beta = -z.$$

Partial fractions give

$$H(w) = \frac{1}{(1 - \alpha w)(1 - \beta w)} = \frac{1}{\alpha - \beta}\left(\frac{\alpha}{1 - \alpha w} - \frac{\beta}{1 - \beta w}\right).$$

Extracting coefficients of w^n yields the closed form

$$\boxed{G_n(z) = \frac{\alpha^{n+1} - \beta^{n+1}}{\alpha - \beta} = \frac{\left(\frac{1 + \sqrt{1 + 4z}}{2}\right)^{n+1} - \left(\frac{1 - \sqrt{1 + 4z}}{2}\right)^{n+1}}{\sqrt{1 + 4z}}.}$$

3) Quick checks and special cases

- $n = 0$: $G_0 = 1$ (matches $(\alpha - \beta)/(\alpha - \beta) = 1$).
- $n = 1$: $G_1 = 1$ (gives $\alpha + \beta = 1$).
- $n = 2$: $G_2 = 1 + z$ (recurrence $G_2 = G_1 + zG_0$).
- Fibonacci numbers. With $z = 1$, $G_n(1) = F_{n+1}$ (usual Fibonacci with $F_0 = 0, F_1 = 1$).
- Radius of convergence (as a series in w). The poles are at $w = 1/\alpha$ and $w = 1/\beta$, so $|w| < \min\{|1/\alpha|, |1/\beta|\}$ if you care about analytic convergence; as a formal power series, the identity holds algebraically.

4) Alternative derivation of $H(w)$ (double sum)

Starting from the definition,

$$\begin{aligned} H(w) &= \sum_{n \geq 0} \sum_{k \geq 0} \binom{n - k}{k} z^k w^n = \sum_{k \geq 0} z^k w^{2k} \sum_{m \geq 0} \binom{m + k}{k} w^m \\ &= \sum_{k \geq 0} \frac{(zw^2)^k}{(1 - w)^{k+1}} = \frac{1}{1 - w} \cdot \frac{1}{1 - \frac{zw^2}{1 - w}} = \frac{1}{1 - w - zw^2}, \end{aligned}$$

using $\sum_{m \geq 0} \binom{m+k}{k} w^m = (1 - w)^{-(k+1)}$. This reproduces the result above and leads to the same closed form for $G_n(z)$.

5) On the second binomial form

The exercise also notes

$$\binom{n-k}{k} = \binom{2k-n-1}{k}(-1)^k,$$

which is the standard negative-binomial identity $\binom{-m}{k} = (-1)^k\binom{m+k-1}{k}$ with $m = n-2k+1$. It's just an algebraic re-expression of the same coefficients; the closed form for $G_n(z)$ is unchanged.

Final answer

$$\boxed{H(w) = \sum_{n\geq 0} G_n(z)w^n = \frac{1}{1-w-zw^2}, \qquad G_n(z) = \frac{\left(\frac{1+\sqrt{1+4z}}{2}\right)^{n+1} - \left(\frac{1-\sqrt{1+4z}}{2}\right)^{n+1}}{\sqrt{1+4z}}.}$$

16 [M22]

Let $a_{n,r}(m)$ be the number of solutions of

$$x_1 + x_2 + \cdots + x_n = m$$

with each x_i an integer satisfying $0 \leq x_i \leq r$. Find the ordinary generating function

$$G_{n,r}(z) \;=\; \sum_{m\geq 0} a_{n,r}(m)\, z^m,$$

and (optionally) a coefficient formula for $a_{n,r}(m)$.

1) Generating function by the product rule

For a single variable x_i with $0 \leq x_i \leq r$, the choices contribute

$$1 + z + z^2 + \cdots + z^r \;=\; \frac{1-z^{r+1}}{1-z}.$$

Since the n variables are independent and the exponents add, the ordinary generating function factorizes:

$$G_{n,r}(z) \;=\; (1 + z + \cdots + z^r)^n \;=\; \left(\frac{1-z^{r+1}}{1-z}\right)^n.$$

2) Extracting coefficients (closed form)

To get an explicit formula for $a_{n,r}(m) = [z^m]G_{n,r}(z)$, use inclusion–exclusion (or the binomial series for $(1-z)^{-n}$, then subtract the "overshoot" terms where some variable exceeds r):

$$\boxed{a_{n,r}(m) \;=\; \sum_{j=0}^{\lfloor m/(r+1)\rfloor} (-1)^j \binom{n}{j} \binom{m-j(r+1)+n-1}{n-1}} \qquad (m \geq 0),$$

with the convention $\binom{u}{v} = 0$ if $u < 0$ or $v < 0$ or $v > u$.

Derivation sketch. Write

$$G_{n,r}(z) = \left(\frac{1-z^{r+1}}{1-z}\right)^n = (1-z^{r+1})^n(1-z)^{-n} = \sum_{j=0}^{n} \binom{n}{j} (-1)^j z^{j(r+1)} \cdot \sum_{t\geq 0} \binom{n+t-1}{n-1} z^t.$$

Collect the coefficient of z^m by setting $t = m - j(r+1)$ and summing over feasible j.

3) Sanity checks and special cases

- $r = \infty$: $G_{n,\infty}(z) = (1-z)^{-n}$, giving the stars-and-bars count $a_{n,\infty}(m) = \binom{m+n-1}{n-1}$.
- $r = 1$: binary variables. Then $G_{n,1}(z) = (1+z)^n$ and $a_{n,1}(m) = \binom{n}{m}$ for $0 \leq m \leq n$.
- Symmetry: coefficients of $(1 + z + \cdots + z^r)^n$ are symmetric in the sense $a_{n,r}(m) = a_{n,r}(nr - m)$ (replace each x_i by $r - x_i$).

4) Small example

Take $n = 3, r = 2$. Then

$$G_{3,2}(z) = (1 + z + z^2)^3.$$

Expanding (or using the formula) gives

$$a_{3,2}(m) = 1,\ 3,\ 6,\ 7,\ 6,\ 3,\ 1 \quad \text{for } m = 0, 1, 2, 3, 4, 5, 6,$$

which counts the bounded solutions to $x_1 + x_2 + x_3 = m$ with $0 \leq x_i \leq 2$.

17 [M25]

Show that expanding $(1-z)^{-w}$ in two different ways yields the bivariate generating function of the (unsigned) Stirling numbers of the first kind $\left[{k \atop n}\right]$:

$$(1-z)^{-w} = \sum_{k\ge 0} \binom{-w}{k} (-z)^k = \sum_{n\ge 0}\sum_{k\ge n} \frac{\left[{k \atop n}\right]}{k!}\, w^n z^k,$$

and hence

$$\left(\ln \tfrac{1}{1-z}\right)^n = n! \sum_{k\ge n} \frac{\left[{k \atop n}\right]}{k!}\, z^k \qquad (n \ge 0).$$

Step 1 — Binomial expansion with a general top argument

Knuth's generalized binomial coefficient is defined for any x by $\binom{x}{k} = x^{\underline{k}}/k!$ (with falling factorial $x^{\underline{k}}$); in particular,

$$(1-z)^{-w} = \sum_{k\ge 0} \binom{-w}{k} (-z)^k \quad \text{(formal power series).}$$

Moreover $\binom{-w}{k}(-1)^k = \dfrac{w(w+1)\cdots(w+k-1)}{k!} = \dfrac{w^{\overline{k}}}{k!}$ (rising factorial). Hence

$$(1-z)^{-w} = \sum_{k\ge 0} \frac{w^{\overline{k}}}{k!}\, z^k.$$

(For the negative-integer special case compare 1.2.9–(20); for the general binomial definition see the index to notations.)

Step 2 — Express the rising factorial via Stirling numbers of the first kind

By the standard conversion between factorial and ordinary powers, the (unsigned) Stirling numbers of the first kind expand the rising factorial:

$$w^{\overline{k}} = \sum_{n=0}^{k} \left[{k \atop n}\right] w^n.$$

(Equivalently, $x^{\underline{n}} = \sum_k (-1)^{n-k}\left[{n \atop k}\right] x^k$; passing from falling to rising removes the signs.) See the Stirling-number discussion and identities in §1.2.6.

Substitute into Step 1 and interchange sums:

$$(1-z)^{-w} = \sum_{k\geq 0} \frac{z^k}{k!} \sum_{n=0}^{k} \left[{k \atop n}\right] w^n = \sum_{n\geq 0} w^n \sum_{k\geq n} \frac{\left[{k \atop n}\right]}{k!}\, z^k. \qquad (\ )$$

Step 3 — Expand the same function as an exponential of a logarithm

Also

$$(1-z)^{-w} = e^{\,w\ln(1/(1-z))} = \sum_{n\geq 0} \frac{w^n}{n!} \left(\ln \tfrac{1}{1-z}\right)^n.$$

Comparing coefficients of w^n in this and in ( ) gives, for each $n \geq 0$,

$$\sum_{k\geq n} \frac{\left[{k \atop n}\right]}{k!}\, z^k = \frac{1}{n!}\left(\ln \tfrac{1}{1-z}\right)^n,$$

which is exactly the desired identity.

Quick sanity checks

- $n = 1$. Since $\left[{k \atop 1}\right] = (k-1)!$,

$$\sum_{k\geq 1} \frac{\left[{k \atop 1}\right]}{k!} z^k = \sum_{k\geq 1} \frac{1}{k} z^k = \ln \tfrac{1}{1-z},$$

agreeing with the formula for $n = 1$.

- $n = 2$. Using $\left[{k \atop 2}\right] = (k-1)!\, H_{k-1}$,

$$2\sum_{k\geq 2} \frac{\left[{k \atop 2}\right]}{k!} z^k = 2\sum_{k\geq 2} \frac{H_{k-1}}{k}\, z^k = \left(\ln \tfrac{1}{1-z}\right)^2,$$

matching the $n = 2$ case.

18 [M25]

(a) Distinct indices

Consider

$$E_n(z) \;=\; \prod_{k=1}^{n}(1+kz).$$

Expanding, every term arises by choosing either 1 or kz from each factor; picking r of the kz terms and the rest as 1 yields a contribution $(k_1 \cdots k_r)z^r$ with $1 \le k_1 < \cdots < k_r \le n$. Therefore the desired sum is exactly the coefficient $[z^r]E_n(z)$.

Now rewrite

$$E_n(z) = z^{\,n+1}\prod_{k=1}^{n}\left(\tfrac{1}{z}+k\right) = \sum_{t=0}^{n+1}\left[\begin{matrix} n+1 \\ t \end{matrix}\right] z^{\,n+1-t},$$

using the standard expansion of a rising factorial into powers with Stirling numbers of the first kind. Hence

$$\boxed{\sum_{1\le k_1<\cdots<k_r\le n} k_1 \cdots k_r \;=\; \left[\begin{matrix} n+1 \\ n+1-r \end{matrix}\right]}.$$

(Equivalently, the sum is the elementary symmetric polynomial $e_r(1,2,\dots,n)$.)

(b) Repetition allowed

Now take

$$H_n(z) \;=\; \prod_{k=1}^{n}\frac{1}{1-kz}.$$

Each factor $(1-kz)^{-1} = \sum_{m\ge 0}(kz)^m$, so in the product the coefficient of z^r is the sum of all $k_1 \cdots k_r$ over weakly increasing $k_1 \le \cdots \le k_r$ with values in $\{1,\dots,n\}$. Thus the desired sum is $[z^r]H_n(z)$.

But there is a classical identity

$$\prod_{k=1}^{n}\frac{1}{1-kz} = \sum_{t\ge n}\left\{\begin{matrix} t \\ n \end{matrix}\right\} z^{\,t-n},$$

so the coefficient of z^r is $\left\{ {n+r \atop n} \right\}$. Therefore

$$\boxed{\sum_{1\le k_1\le\cdots\le k_r\le n} k_1\cdots k_r \;=\; \left\{ {n+r \atop n} \right\}}.$$

(Equivalently, this is the complete homogeneous symmetric polynomial $h_r(1,2,\dots,n)$.)

Quick check

For $n=3, r=2$:

- (a) $1\cdot 2+1\cdot 3+2\cdot 3=11=\left[{4 \atop 2} \right]$.
- (b) $1\cdot 1+1\cdot 2+1\cdot 3+2\cdot 2+2\cdot 3+3\cdot 3=25=\left\{ {5 \atop 3} \right\}$.

19 [HM32]

For integers p,q with $0<p<q$, prove

$$H_{p/q}=\frac{q}{p}\;-\;\frac{\pi}{2}\cot\Big(\frac{\pi p}{q}\Big)\;-\;\ln(2q)\;+\;2\sum_{0<k<q/2}\cos\Big(\frac{2\pi pk}{q}\Big)\,\ln\Big(\sin\frac{\pi k}{q}\Big). \qquad (\ )$$

We use the hint: by Abel's theorem,

$$H_{p/q}=\lim_{x\to 1^-}\sum_{n\ge 1}\Big(\frac{1}{n}-\frac{1}{n+p/q}\Big)x^{\,p+nq}, \qquad (1)$$

and "Eq. (13)" of the section (the basic series for the logarithm) to rewrite the power series so the limit can be taken.

1) Prepare two pieces

Split the sum in (1) into

$$S(x)=A(x)-B(x),\qquad A(x):=\sum_{n\ge 1}\frac{x^{p+nq}}{n},\qquad B(x):=\sum_{n\ge 1}\frac{x^{p+nq}}{n+p/q}.$$

Piece A. Using $\sum_{n\ge 1}\frac{z^n}{n}=-\ln(1-z)$ (Eq. (13) in this section),

$$A(x)=x^p\sum_{n\ge 1}\frac{(x^q)^n}{n}=-x^p\ln(1-x^q). \qquad (2)$$

Piece B. Note $1/(n+p/q) = q/(qn+p)$. Write the progression $p, p+q, p+2q, \ldots$ via a roots-of-unity filter. Let $\omega = e^{2\pi i/q}$. A standard discrete Fourier selection gives

$$q \sum_{m\geq 0} \frac{x^{p+mq}}{p+mq} = -\sum_{k=0}^{q-1} \omega^{-kp} \ln(1-\omega^k x). \tag{3}$$

Therefore

$$B(x) = \sum_{n\geq 1} \frac{x^{p+nq}}{n+p/q} = q \sum_{m\geq 1} \frac{x^{p+mq}}{p+mq} = -\sum_{k=0}^{q-1} \omega^{-kp} \ln(1-\omega^k x) - \frac{q}{p} x^p. \tag{4}$$

Combining (2) and (4),

$$S(x) = \underbrace{\sum_{k=1}^{q-1} \omega^{-kp} \ln(1-\omega^k x)}_{=:f(x)} + \underbrace{(1-x^p)\ln(1-x) + \frac{q}{p} x^p - x^p \ln \frac{1-x^q}{1-x}}_{=:g(x)}. \tag{5}$$

2) The easy limit $g(x)$

As $x \to 1^-$: $(1-x^p)\ln(1-x) \to 0$ (since $(1-x)\ln(1-x) \to 0$), $x^p \to 1$, and $\dfrac{1-x^q}{1-x} \to q$. Hence

$$\lim_{x\to 1^-} g(x) = \frac{q}{p} - \ln q. \tag{6}$$

3) The main limit $f(x)$

For $0 < x < 1$ the k and $q-k$ terms in $f(x)$ are complex conjugates, so $f(x)$ is real; it suffices to compute its real part termwise.

Use the elementary factorization

$$1-\omega^k = 1 - e^{2\pi i k/q} = 2\sin\Big(\frac{\pi k}{q}\Big)\ \exp\Big(i\big(\tfrac{\pi k}{q} - \tfrac{\pi}{2}\big)\Big),$$

valid for $1 \leq k \leq q-1$. Taking principal logs,

$$\ln(1-\omega^k) = \ln\Big(2\sin\frac{\pi k}{q}\Big) + i\Big(\frac{\pi k}{q} - \frac{\pi}{2}\Big).$$

Therefore, letting $\phi_k := \frac{2\pi p k}{q}$,

$$\omega^{-kp}\,\ln(1-\omega^k) = \Big[\cos\phi_k\,\ln\Big(2\sin\frac{\pi k}{q}\Big) - \Big(\frac{\pi k}{q}-\frac{\pi}{2}\Big)\sin\phi_k\Big] + i(\cdots),$$

and summing $k = 1, \dots, q-1$ cancels the imaginary parts. Thus

$$\lim_{x\to 1^-} f(x) = \sum_{k=1}^{q-1}\cos\phi_k\,\ln\Big(2\sin\frac{\pi k}{q}\Big) - \sum_{k=1}^{q-1}\Big(\frac{\pi k}{q}-\frac{\pi}{2}\Big)\sin\phi_k. \tag{7}$$

Two standard trigonometric sums now enter:

- $\displaystyle\sum_{k=1}^{q-1}\cos\frac{2\pi pk}{q} = -1$ (since $\sum_{k=0}^{q-1} e^{2\pi ipk/q} = 0$).
- $\displaystyle\sum_{k=1}^{q-1} k\sin\frac{2\pi pk}{q} = \frac{q}{2}\cot\frac{\pi p}{q}$. (Derive by differentiating $\sum_{k=0}^{q-1} r^k = (1-r^q)/(1-r)$ at $r = e^{it}$ and taking imaginary parts at $t = 2\pi p/q$.)

Using $\sum_{k=1}^{q-1}\sin\phi_k = 0$, the second sum in (7) becomes

$$-\frac{\pi}{q}\sum_{k=1}^{q-1} k\sin\phi_k = -\frac{\pi}{2}\cot\Big(\frac{\pi p}{q}\Big).$$

For the first sum in (7), split $\ln(2\sin) = \ln 2 + \ln(\sin)$:

$$\sum_{k=1}^{q-1}\cos\phi_k\,\ln\Big(2\sin\frac{\pi k}{q}\Big) = \underbrace{\Big(\sum_{k=1}^{q-1}\cos\phi_k\Big)}_{=-1}\ln 2 \;+\; \sum_{k=1}^{q-1}\cos\phi_k\,\ln\Big(\sin\frac{\pi k}{q}\Big).$$

Pairing k with $q-k$ converts the remaining sum to

$$2\sum_{0<k<q/2}\cos\Big(\frac{2\pi pk}{q}\Big)\,\ln\Big(\sin\frac{\pi k}{q}\Big),$$

since when q is even the middle term $k = q/2$ has $\sin(\pi/2) = 1$ and contributes 0.

Hence

$$\lim_{x\to 1^-} f(x) = -\ln 2 \;-\; \frac{\pi}{2}\cot\Big(\frac{\pi p}{q}\Big) \;+\; 2\sum_{0<k<q/2}\cos\Big(\frac{2\pi pk}{q}\Big)\,\ln\Big(\sin\frac{\pi k}{q}\Big). \tag{8}$$

4) Assemble the result

From (6) and (8),

$$H_{p/q} = \Big(\frac{q}{p} - \ln q\Big) + \Big[-\ln 2 - \frac{\pi}{2}\cot\Big(\frac{\pi p}{q}\Big) + 2\sum_{0<k<q/2}\cos\Big(\frac{2\pi pk}{q}\Big)\,\ln\Big(\sin\frac{\pi k}{q}\Big)\Big],$$

which simplifies to (), as required.

5) Quick sanity checks (the three series in the statement)

Using $1 - \frac{1}{2}H_{1/2} = \ln 2$, $1 - \frac{1}{3}H_{1/3} + \frac{1}{5}H_{1/5} - \cdots = \frac{\pi}{4}$, etc., the identity reproduces the classical sums displayed in the exercise heading (they correspond to $H_{1/2}, H_{1/4}, H_{1/3}$ in the linear combinations shown there).

20 [M21]

Step 1. Expand n^m in the falling–factorial basis

A classical identity using Stirling numbers of the second kind $S(m,k)$ is

$$n^m = \sum_{k=0}^{m} S(m,k)\, n^{\underline{k}}, \qquad n^{\underline{k}} := n(n-1)\cdots(n-k+1) = k!\binom{n}{k}.$$

(Combinatorially: count functions from an m-set to an n-set by first partitioning the domain into k nonempty blocks — $S(m,k)$ ways — then injecting those k blocks into the n-set — $n^{\underline{k}}$ ways.)

Hence

$$n^m = \sum_{k=0}^{m} S(m,k)\, k!\binom{n}{k}.$$

Step 2. The ordinary GF of $\binom{n}{k}$

For fixed $k \geq 0$,

$$\sum_{n\geq 0}\binom{n}{k} z^n = \sum_{n\geq k}\binom{n}{k} z^n = z^k \sum_{r\geq 0}\binom{r+k}{k} z^r = \frac{z^k}{(1-z)^{k+1}},$$

using $\displaystyle\sum_{r\geq 0}\binom{r+k}{k} z^r = (1-z)^{-(k+1)}$.

Multiplying by $k!$ gives

$$\sum_{n\geq 0} k! \binom{n}{k} z^n = \frac{k!\, z^k}{(1-z)^{k+1}}.$$

Step 3. Assemble the generating function

Using Steps 1–2,

$$\sum_{n\geq 0} n^m z^n = \sum_{n\geq 0}\sum_{k=0}^{m} S(m,k)\, k! \binom{n}{k} z^n = \sum_{k=0}^{m} S(m,k)\, k! \left(\sum_{n\geq 0}\binom{n}{k} z^n\right) = \sum_{k=0}^{m} \frac{S(m,k)\, k!\, z^k}{(1-z)^{k+1}}.$$

Therefore the requested coefficients are

$$\boxed{c_{m,k} = k!\, S(m,k)} \qquad (0 \leq k \leq m),$$

where $S(m,k)$ are Stirling numbers of the second kind.

Quick checks (small m)

- $m = 0$: $c_{0,0} = 0!\, S(0,0) = 1 \Rightarrow \sum z^n = 1/(1-z)$.
- $m = 1$: $c_{1,1} = 1!\, S(1,1) = 1 \Rightarrow \sum n z^n = z/(1-z)^2$.
- $m = 2$: $c_{2,1} = 1!\, S(2,1) = 1,\ c_{2,2} = 2!\, S(2,2) = 2$:

$$\sum n^2 z^n = \frac{z}{(1-z)^2} + \frac{2z^2}{(1-z)^3} = \frac{z(1+z)}{(1-z)^3},$$

  as known.

Alternative viewpoint (operator form, optional)

Since $\sum_{n\geq 0} n^m z^n = (z\,\frac{d}{dz})^m \left(\frac{1}{1-z}\right)$, expanding the power $(z\,\frac{d}{dz})^m$ in terms of falling powers of the operator $z\,\frac{d}{dz}$ again introduces $S(m,k)$ and leads to the same coefficients $c_{m,k} = k!\, S(m,k)$.

21 [HM30]

1) Ordinary generating function (OGF)

Let

$$A(z) \;=\; \sum_{n\geq 0} n!\, z^n.$$

Radius of convergence

By the root (or ratio) test, $\sqrt[n]{n!} \sim n/e$, hence $\lim_{n\to\infty} \sqrt[n]{n!} = \infty$ and the radius of convergence is 0. Thus $A(z)$ diverges for every $z \neq 0$; it is best regarded as a formal power series. (Coefficient extraction and manipulation of formal OGFs are standard in §1.2.9.)

Differential equation for A

The factorials satisfy $a_{n+1} = (n+1)a_n$ with $a_0 = 1$. Use the standard derivative rule for OGFs,

$$G'(z) = \sum_{k\geq 0} (k+1)a_{k+1} z^k,$$

which Knuth gives as Eq. (14). Multiply the recurrence by z^{n+1} and sum over $n \geq 0$:

$$A(z) - 1 \;=\; z\sum_{n\geq 0}(n+1)a_n z^n \;=\; z\,(zA(z))' \;=\; zA(z) + z^2 A'(z).$$

Hence the ODE

$$z^2 A'(z) = (1-z)A(z) - 1. \qquad (\;)$$

This step uses the differentiation identity for generating functions in the text.

Closed form via an exponential integral

Solve ( ) as a first-order linear ODE. Writing it as $A' - \frac{1-z}{z^2}A = -\frac{1}{z^2}$, an integrating factor is $\mu(z) = ze^{1/z}$. Thus

$$\frac{d}{dz}(ze^{1/z}A(z)) = -\frac{e^{1/z}}{z}.$$

Let $t = 1/z$. Then $-\frac{e^{1/z}}{z}\,dz = \frac{e^t}{t}\,dt$, so

$$ze^{1/z}A(z) = \mathrm{Ei}\big(\tfrac{1}{z}\big) + C, \qquad A(z) = \frac{e^{-1/z}}{z}\Big(\mathrm{Ei}\big(\tfrac{1}{z}\big) + C\Big).$$

Choosing $C = 0$ gives the unique branch whose asymptotic Maclaurin expansion at $z = 0$ is $\sum_{n\geq 0} n!z^n$. In other words,

$$\boxed{A(z) \;\sim\; \sum_{n\geq 0} n!z^n \;=\; \frac{e^{-1/z}}{z}\,\mathrm{Ei}\Big(\frac{1}{z}\Big) \quad \text{(as a divergent asymptotic series)}}$$

(the equality is Borel summation/analytic continuation). This function has an essential singularity at $z = 0$ and a branch cut coming from Ei.

A convenient equivalent integral representation—obtained from $n! = \int_0^\infty t^n e^{-t}\,dt$ and summing the geometric series—is

$$A(z) \;=\; \int_0^\infty \frac{e^{-t}}{1 - zt}\,dt \;=\; \frac{e^{-1/z}}{z}\,\mathrm{Ei}\Big(\frac{1}{z}\Big),$$

interpreted as an analytic continuation (the geometric series identity $1/(1-u) = \sum_{n\geq 0} u^n$ is Eq. (5) in the text).

Summary for the OGF.

- Formal series: $A(z) = \sum_{n\geq 0} n!z^n$ (radius 0).
- Satisfies $z^2A' = (1-z)A - 1$.
- Borel sum / analytic continuation: $A(z) = \frac{e^{-1/z}}{z}\,\mathrm{Ei}(1/z)$.
- Asymptotics at $z \to 0$: the coefficients are $n!$ (by coefficient extraction notation $[z^n]$ in §1.2.9).

2) Exponential generating function (EGF)

Knuth also uses exponential series in this section (see Eq. (22)). For the sequence $\langle n!\rangle$,

$$E(z) \;=\; \sum_{n\geq 0} \frac{n!}{n!}\,z^n \;=\; \sum_{n\geq 0} z^n \;=\; \frac{1}{1-z} \quad (|z| < 1).$$

Thus the EGF is the geometric series $1/(1-z)$ (exactly Eq. (5) in the text), and it is analytic for $|z| < 1$ with a simple pole at $z = 1$.

Properties from the EGF.

- Coefficients recover immediately: $[z^n]E(z) = 1$ and multiplying back by $n!$ gives $n!$.

- Differentiation yields moments/convolutions in the usual way for EGFs (see the general "exponential series" framework).

3) Relation between OGF and EGF (Borel transform)

The OGF can be viewed as the Borel transform of the EGF:

$$A(z) \;=\; \int_0^\infty e^{-t}\,E(tz)\,dt \;=\; \int_0^\infty \frac{e^{-t}}{1-tz}\,dt \;=\; \frac{e^{-1/z}}{z}\,\mathrm{Ei}\Big(\frac{1}{z}\Big),$$

which reconciles the divergent OGF with the convergent EGF and explains the exponential-integral form above (geometric-series identity again).

22 [M21]

1) Build the product generating function

For each fixed $j \geq 0$, the choices of k_j contribute a factor

$$\sum_{k_j=0}^{r} \binom{r}{k_j} (z^{2^j})^{k_j} \;=\; (1+z^{2^j})^r$$

by the binomial theorem. Independence across j means the full generating function is

$$G(z) \;=\; \prod_{j\geq 0}(1+z^{2^j})^r.$$

By construction, expanding this product and collecting the z^n term produces exactly the given sum, so this $G(z)$ works.

2) Telescope the product

Use $1+t = \dfrac{1-t^2}{1-t}$ to telescope finite partial products:

$$\prod_{j=0}^{m}(1+z^{2^j})^r = \prod_{j=0}^{m}\Big(\frac{1-z^{2^{j+1}}}{1-z^{2^j}}\Big)^r = \left(\frac{1-z^{2^{m+1}}}{1-z}\right)^r.$$

Letting $m \to \infty$ (as a formal power series, or analytically for $|z|<1$) gives

$$G(z) \;=\; \prod_{j\geq 0}(1+z^{2^j})^r \;=\; (1-z)^{-r}.$$

Thus a closed form is $G(z) = (1-z)^{-r}$.

3) Read off the coefficient

From the negative binomial series,

$$(1-z)^{-r} = \sum_{n\geq 0} \binom{n+r-1}{n} z^n,$$

so

$$[z^n]\, G(z) = [z^n]\, (1-z)^{-r} = \binom{n+r-1}{n}.$$

Therefore the original sum equals $\binom{n+r-1}{n}$, and one valid answer is

$$\boxed{G(z) = \prod_{j\geq 0} (1+z^{2^j})^r = (1-z)^{-r}\,.}$$

4) (Combinatorial sanity check)

$\prod_{j\geq 0}(1+z^{2^j}) = 1+z+z^2+z^3+\cdots = \frac{1}{1-z}$: choosing 0 or 1 copy of each 2^j encodes the binary expansion of any nonnegative integer. Raising to the r-th power counts ordered r-tuples $(n_1, \dots, n_r)$ with $n_1 + \cdots + n_r = n$; there are $\binom{n+r-1}{n}$ such tuples ("stars and bars"), matching the coefficient above.

23 [M33]

(a) Existence of polynomials f_m, g_m

Claim. For every integer $m \geq 1$ there exist polynomials $f_m(z_1, \dots, z_m)$ and $g_m(z_1, \dots, z_m)$ such that the multivariate binomial-sum identity in the exercise holds for all integers $n \geq r \geq 0$. (See the problem statement on p. 96 of the text.)

Construction (induction on m).

- Base $m = 1$. Take

$$f_1(z_1) = z_1, \qquad g_1(z_1) = 1 + z_1.$$

  The required identity reduces to the ordinary binomial theorem, so it holds.

- Inductive step. Suppose f_m, g_m work for some m. When you append a new variable z_{m+1} and sum over the new index, a single application of the binomial theorem shows that you can obtain new polynomials f_{m+1}, g_{m+1} by the substitutions

$$z_m \longleftarrow z_m(1 + z_{m+1}^{-1}), \quad \text{then multiply overall by } z_{m+1}.$$

  Concretely:

$$\begin{aligned} f_{m+1}(z_1, \dots, z_{m+1}) &= z_{m+1}\, f_m(z_1, \dots, z_{m-1},\, z_m(1 + z_{m+1}^{-1})), \\ g_{m+1}(z_1, \dots, z_{m+1}) &= z_{m+1}\, g_m(z_1, \dots, z_{m-1},\, z_m(1 + z_{m+1}^{-1})). \end{aligned}$$

  In particular this implies the Fibonacci-type recurrences

$$f_m = z_m f_{m-1} + z_{m-1} f_{m-2}, \qquad g_m = z_m g_{m-1} + z_{m-1} g_{m-2},$$

  with initial values $f_{-1} = 0,\ f_0 = g_{-1} = g_0 = 1$. Hence f_m, g_m are indeed polynomials and the required identity follows for $m + 1$.

Useful descriptions. From the recurrences one gets:

- $g_2 = z_1 + z_2 + z_1 z_2$.
- g_m / f_m equals the finite continued fraction

$$\frac{g_m}{f_m} = 1 + \cfrac{z_1^{-1}}{1 + \cfrac{z_2^{-1}}{1 + \ \cdots\ + \cfrac{z_m^{-1}}{1}}}.$$

- Combinatorially, g_m is the sum of all monomials obtained from $z_1 z_2 \cdots z_m$ by striking out any set of nonadjacent variables (there are F_{m+2} ways); f_m is similar but with z_1 constrained to remain.

(b) Closed form for the cyclic sum S_n

Let

$$S_n(z_1, \dots, z_m)$$

denote the cyclic binomial sum defined in the exercise (a direct generalization of Ex. 15). Using part (a) and extracting the coefficient of z_m^n, we can write

$$S_n(z_1, \dots, z_m) = [z_m^n] \sum_{r=0}^{n} z^r \, z_m^{n-r} \, f_m^{n-r} \, g_m^r.$$

Carrying out the coefficient extraction and the finite sums leads to a compact second-order rational generating function

$$\sum_{n \geq 0} S_n z^n = \frac{1}{(1-az)(1-dz) - bc\, z^2},$$

where

$$a = z_m g_{m-1}, \qquad b = z_{m-1} g_{m-2}, \qquad c = z_m f_{m-1}, \qquad d = z_{m-1} f_{m-2}.$$

Equivalently, the sequence (S_n) satisfies the linear recurrence

$$S_n = h_m \, S_{n-1} - (-1)^m z_1 z_2 \cdots z_m \, S_{n-2}, \quad n \geq 2,$$

where

$$h_m = z_m g_{m-1} + z_{m-1} f_{m-2}, \qquad ad - bc = (-1)^m z_1 z_2 \cdots z_m.$$

Solving the quadratic

$$t^2 - (a+d)\, t + (ad - bc) = 0,$$

let ρ, σ be its roots. Then the closed form is

$$\boxed{S_n(z_1, \dots, z_m) = \frac{\rho^{\,n+1} - \sigma^{\,n+1}}{\rho - \sigma}}$$

with $a + d = h_m$ and $ad - bc = (-1)^m z_1 \cdots z_m$ as above. (All steps are detailed in the book's answer sketch.)

A pleasant by-product: the derivation also yields directly the two-term recurrence for S_n displayed above, which would be quite painful to obtain without generating functions.

(c) Specialization $z_1 = \cdots = z_m = z$

When all variables are equal, the coefficients in the recurrence become constant and the characteristic roots simplify beautifully. If

$$\rho_1 = \frac{z + \sqrt{z^2 + 4z}}{2}, \qquad \sigma_1 = \frac{z - \sqrt{z^2 + 4z}}{2},$$

then in the m-variable case the corresponding roots are just the m-th powers:

$$\rho_m = \rho_1^m, \qquad \sigma_m = \sigma_1^m.$$

Therefore

$$\boxed{S_n(\underbrace{z, \dots, z}_{m}) = \frac{\rho_m^{n+1} - \sigma_m^{n+1}}{\rho_m - \sigma_m} = \frac{\rho_1^{m(n+1)} - \sigma_1^{m(n+1)}}{\rho_1^m - \sigma_1^m}}$$

with ρ_1, σ_1 as above.

Checks and small cases

- $m = 1$: $S_n(z) = \frac{\rho_1^{n+1} - \sigma_1^{n+1}}{\rho_1 - \sigma_1}$ solves $S_n = z\,S_{n-1} - z\,S_{n-2}$ with $S_0 = 1,\ S_1 = 1 + z$; this is the classical companion to the Fibonacci-type sums in Ex. 15.
- $m = 2$: Using $g_2 = z_1 + z_2 + z_1 z_2$ and the formulas above reproduces the two-variable generalization and matches the rational generating function obtained by standard methods.

24 [M22]

Proof of the identity

Use the binomial theorem on the right-hand side:

$$(1 + zG(z))^m = \sum_{k=0}^{m} \binom{m}{k} (zG(z))^k.$$

Taking coefficients of z^n and using $[z^n]\,(z^k H(z)) = [z^{n-k}]H(z)$,

$$[z^n](1 + zG(z))^m = \sum_{k=0}^{m} \binom{m}{k} [z^n]\,(zG(z))^k = \sum_{k=0}^{m} \binom{m}{k} [z^{n-k}]\,G(z)^k,$$

which is exactly the left-hand side.

(a) $G(z) = \dfrac{1}{1-z}$

Right-hand side:

$$[z^n]\Big(1 + \frac{z}{1-z}\Big)^m = [z^n]\,(1-z)^{-m}.$$

Using the standard expansion $(1-z)^{-(r+1)} = \sum_{j\geq 0} \binom{r+j}{r} z^j$ (take $r = m-1$), the coefficient is

$$[z^n]\,(1-z)^{-m} = \binom{m+n-1}{n}. \quad \text{(negative-binomial series)}$$

Left-hand side (as a check): $G(z)^k = (1-z)^{-k}$, so

$$[z^{\,n-k}]\,G(z)^k = [z^{\,n-k}](1-z)^{-k} = \binom{n-1}{k-1} \quad (k \leq n),$$

and

$$\sum_k \binom{m}{k}\binom{n-1}{k-1} = \sum_k \binom{m}{k}\binom{n-1}{n-k} = \binom{m+n-1}{n}$$

by Vandermonde's convolution, agreeing with the RHS.

Answer for (a): $\boxed{\binom{m+n-1}{n}}$.

(b) $G(z) = \dfrac{e^z - 1}{z}$

Right-hand side:

$$1 + zG(z) = 1 + (e^z - 1) = e^z \;\Rightarrow\; [z^n](e^z)^m = [z^n]e^{mz} = \frac{m^n}{n!}.$$

Left-hand side (optional cross-check via Stirling numbers):

$$[z^{\,n-k}]\,G(z)^k = [z^n](e^z - 1)^k.$$

From $\dfrac{(e^z-1)^k}{k!} = \sum_{n\ge k} S(n,k)\,\dfrac{z^n}{n!}$ we get $[z^n](e^z-1)^k = \dfrac{k!\,S(n,k)}{n!}$. Hence

$$\sum_k \binom{m}{k} [z^{n-k}]G(z)^k = \frac{1}{n!}\sum_k \binom{m}{k} k!\,S(n,k) = \frac{1}{n!}\sum_k (m)_k\,S(n,k) = \frac{m^n}{n!},$$

using the standard identity $m^n = \sum_k S(n,k)\,(m)_k$. The RHS $m^n/n!$ we already obtained.

Answer for (b): $\boxed{\dfrac{m^n}{n!}}$.

I will give another purely combinatorial (no generating functions) proofs for parts (a) and (b).

(a) $G(z) = \dfrac{1}{1-z} \quad [z^n](1-z)^{-m} = \dbinom{m+n-1}{n}$

Combinatorial story (stars and bars). Interpret $[z^n](1-z)^{-m}$ as the number of solutions in nonnegative integers to

$$x_1 + \cdots + x_m = n, \qquad x_i \ge 0,$$

i.e., distributing n identical balls into m labeled boxes. By the classic stars-and-bars argument, the count is

$$\binom{m+n-1}{n}.$$

Same count, decomposed by the number of nonempty boxes. Equivalently, count the same distributions by the number k of boxes that are actually used:

1. Choose which k of the m boxes are nonempty: $\binom{m}{k}$ ways.
2. Put at least one ball in each chosen box; write $x_i' = x_i - 1$ for used boxes. Then $\sum x_i' = n-k$ with $x_i' \ge 0$ in exactly those k boxes—equivalently, choose a composition of n into k positive parts. The number of such k-tuples is $\binom{n-1}{k-1}$ (stars and bars with positivity).

Summing over k gives

$$\sum_{k=0}^{\min(m,n)} \binom{m}{k}\binom{n-1}{k-1} = \binom{m+n-1}{n},$$

which is Vandermonde's convolution in disguise. Thus the coefficient equals $\boxed{\binom{m+n-1}{n}}$.

(b) $G(z) = \dfrac{e^z - 1}{z} \quad [z^n]e^{mz} = \dfrac{m^n}{n!}$

We'll show the identity

$$m^n = \sum_{k=0}^{\min(m,n)} \binom{m}{k} k!\, S(n,k),$$

by double-counting the same object, then divide by $n!$ at the end.

Objects to count. Consider all functions $f : [n] \to [m]$ (think: color each of n distinct balls with one of m colors). There are clearly m^n such functions.

Count the same set by the number of colors used.

- Fix k. First choose the k colors that actually appear: $\binom{m}{k}$ ways.
- The fibers $f^{-1}(c)$ over the used colors form a partition of $[n]$ into k nonempty, *unlabeled* blocks. The number of such partitions is the Stirling number $S(n,k)$.
- Finally, assign (label) the k blocks with the chosen k distinct colors: $k!$ bijections.

Multiplying and summing over k yields

$$m^n = \sum_k \binom{m}{k} S(n,k)\, k!.$$

Since $[z^n]e^{mz} = \frac{m^n}{n!}$, dividing both sides by $n!$ gives

$$[z^n]e^{mz} = \sum_k \binom{m}{k} \frac{k!\, S(n,k)}{n!},$$

which matches the exercise's evaluation for $G(z) = \dfrac{e^z - 1}{z}$. Therefore the answer for (b) is $\boxed{\dfrac{m^n}{n!}}$.

25 [M23]

Evaluate

$$S_n = \sum_{k=0}^{n} \binom{n}{k} \binom{2n-2k}{n-k} (-2)^k.$$

(Equivalently, recognize $\binom{n}{k}(-2)^k = [w^k](1-2w)^n$ and $\binom{2n-2k}{n-k} = [z^{n-k}](1+z)^{2n-2k}$.)

Use coefficient extraction to convert the single sum into one coefficient:

$$\begin{aligned}
S_n &= \sum_{k=0}^{n} \binom{n}{k} (-2)^k \, [z^{n-k}](1+z)^{2n-2k} \\
&= [z^n] \sum_{k=0}^{n} \binom{n}{k} (-2)^k \, z^k \, (1+z)^{2n-2k} \qquad \text{(shift: } [z^{n-k}] = [z^n]z^k\text{)} \\
&= [z^n] \, (1+z)^{2n} \sum_{k=0}^{n} \binom{n}{k} \left(\frac{-2z}{(1+z)^2} \right)^k \\
&= [z^n] \, (1+z)^{2n} \left(1 - \frac{2z}{(1+z)^2} \right)^n = [z^n] \, (1+z)^{2n} \left(\frac{(1+z)^2 - 2z}{(1+z)^2} \right)^n \\
&= [z^n] \, (1+z)^{2n} \left(\frac{1+z^2}{(1+z)^2} \right)^n = [z^n] \, (1+z^2)^n.
\end{aligned}$$

Now expand $(1+z^2)^n = \sum_{j=0}^{n} \binom{n}{j} z^{2j}$. The coefficient of z^n is therefore

- 0 if n is odd (no j with $2j = n$);
- $\binom{n}{n/2}$ if n is even (take $j = n/2$).

Answer

$$\boxed{S_n = \begin{cases} \dbinom{n}{n/2}, & n \text{ even}, \\ 0, & n \text{ odd}. \end{cases}}$$

(Sanity check for small n: $S_0 = 1,\ S_1 = 0,\ S_2 = 2,\ S_3 = 0,\ S_4 = 6,\ S_6 = 20$, matching $\binom{2m}{m}$ for even $n = 2m$.)

26 [M40]

Knuth introduces the bracket notation $[z^n]\,G(z)$ to mean "the coefficient of z^n in $G(z)$" (his eq. (31)). Exercise 26 asks you to extend this notation so that one can write things like

$$[z^2 - 2z^5]\,G(z) = a_2 - 2a_5 \quad \text{when } G(z) = \sum_{n\geq 0} a_n z^n.$$

Definition (generalized bracket)

Let $H(z) = \sum_{n\in\mathbb{Z}} h_n z^n$ be a *Laurent* series with only finitely many *positive* coefficients nonzero (polynomials are the most common case). For a (formal) power series $G(z) = \sum_{n\geq 0} a_n z^n$, define

$$\boxed{[H(z)]\,G(z) \;:=\; \mathrm{CT}_z\left(H(z)\,G(z^{-1})\right) \;=\; \sum_{n\geq 0} h_n\, a_n}$$

where CT_z denotes "constant term in z." This immediately yields $[z^2 - 2z^5]\,G(z) = a_2 - 2a_5$.

Equivalently—connecting to Knuth's coefficient-extraction via complex analysis—you can write the same functional as a contour (residue) integral:

$$[H(z)]\,G(z) = \frac{1}{2\pi i}\oint H(z)\,G(z^{-1})\,\frac{dz}{z},$$

which generalizes Cauchy's coefficient formula $[z^n]\,G(z) = \frac{1}{2\pi i}\oint \frac{G(z)}{z^{n+1}}\,dz$.

Basic properties

Let F, G be (formal) power series and H, K Laurent series as above.

1. Linearity. $[\,\alpha H + \beta K\,]\,G = \alpha[H]\,G + \beta[K]\,G$ and $[H]\,(\alpha F + \beta G) = \alpha[H]\,F + \beta[H]\,G$.
2. Symmetry (dot product view). $[H]\,G = \sum_{n\geq 0} h_n a_n = [G]\,H$. So $[\,\cdot\,]$ is the coefficientwise inner product.
3. Shift rules. $[z^m H]\,G = \sum_{n\geq 0} h_n\, a_{n+m} = [H]\,(z^{-m}G)$. Special cases:

- $[z^m]\,G = a_m$ (recovers the original bracket).
- $[1 + z + \cdots + z^N]\,G = \sum_{n=0}^{N} a_n$ (prefix sums).
- $([z^r/(1-z)]\,G = \sum_{n\geq r} a_n)$ (a tail sum; justified via the constant-term definition).

4. Product (Cauchy) interaction. If $F(z) = \sum f_k z^k$, then

$$[H]\,(FG) = \sum_{k\geq 0} f_k\,[\,z^{-k}H\,]\,G,$$

a handy "convolution-with-shift" identity derived from $\mathrm{CT}\,(H\,FG^{-1})$.

5. Hadamard viewpoint. Let $H \odot G := \sum_{n\geq 0} h_n a_n z^n$ be the Hadamard (termwise) product. Then

$$[H]\,G = (H \odot G)(1),$$

whenever evaluation at $z = 1$ is meaningful (e.g., as a finite sum or convergent series).

Examples

- Polynomial weights. With $H(z) = c_0 + c_1 z + \cdots + c_m z^m$, $[H]\,G = \sum_{k=0}^{m} c_k a_k$.
- Finite differences. $[1-z]\,G = a_0 - a_1$, $[1-2z+z^2]\,G = a_0 - 2a_1 + a_2$ (a second difference at the origin).
- Sums via rational H. Using $[\,1/(1-z)\,]\,G = \sum_{n\geq 0} a_n$ (constant-term proof), you can encode cumulative sums compactly.

Why this is natural

- It strictly generalizes $[z^n]$: take $H(z) = z^n$ to get $[z^n]\,G$.
- It turns coefficient extraction into a bilinear functional—algebraically convenient and perfectly compatible with the generating-function toolkit in §1.2.9 (multiplication, logarithms, etc.).
- It fits the residue framework Knuth invokes for coefficient extraction, simply replacing z^{-n-1} by $H(z)/z$ inside the contour integral.

1.2.10 Analysis of an Algorithm

1 [10]

From the section we have the recurrence for the distribution $p_{nk} = \Pr(A = k)$:

$$p_{nk} = \frac{1}{n}\, p_{n-1,k-1} + \frac{n-1}{n}\, p_{n-1,k},$$

with initial conditions $p_{1k} = \delta_{0k}$ and $p_{nk} = 0$ for $k < 0$.

Set $k = 0$. The term $p_{n-1,-1}$ vanishes, so

$$p_{n0} = \frac{n-1}{n}\, p_{n-1,0}.$$

Unfolding this down to $n = 1$ (where $p_{10} = 1$) gives a telescoping product:

$$p_{n0} = \frac{n-1}{n} \cdot \frac{n-2}{n-1} \cdots \frac{1}{2} \cdot 1 = \frac{1}{n}.$$

So,

$$\boxed{p_{n0} = \frac{1}{n}}.$$

Interpretation for Algorithm M

In Algorithm M, A counts how many times the current maximum m is changed while scanning the input. The minimum value $A = 0$ occurs iff the last item is already the global maximum, i.e., $X[n] = \max_{1 \le k \le n} X[k]$. Under the section's assumption that all $X[k]$ are distinct and all $n!$ relative orders are equally likely, each position is equally likely to hold the maximum; hence $\Pr(A = 0) = 1/n$.

(Equivalently, since $G_n(z) = \sum_k p_{nk} z^k$, the result says $G_n(0) = p_{n0} = 1/n$.)

2 [HM16]

Here is a clean derivation of Eq. (13) from Eq. (10).

Let $G(z) = \sum_{k\geq 0} p_k z^k$ be the probability generating function, so $\sum_k p_k = G(1) = 1$. By Eq. (10),

$$\text{mean}(G) = \sum_k k\, p_k, \qquad \text{var}(G) = \sum_k k^2 p_k - \Big(\sum_k k\, p_k\Big)^2. \text{ [Eq. (10)]}$$

1. Differentiate G:

$$G'(z) = \sum_k k\, p_k\, z^{k-1} \quad \Longrightarrow \quad G'(1) = \sum_k k\, p_k = \text{mean}(G).$$

2. Differentiate again:

$$G''(z) = \sum_k k(k-1)\, p_k\, z^{k-2} \quad \Longrightarrow \quad G''(1) = \sum_k k(k-1)\, p_k.$$

3. Express $\sum_k k^2 p_k$ using $G''(1)$ and $G'(1)$:

$$\sum_k k^2 p_k = \sum_k (k(k-1)+k)p_k = \underbrace{\sum_k k(k-1)p_k}_{=G''(1)} + \underbrace{\sum_k k\, p_k}_{=G'(1)} = G''(1) + G'(1).$$

4. Substitute into the variance formula:

$$\text{var}(G) = \big(G''(1) + G'(1)\big) - \big(G'(1)\big)^2 = G''(1) + G'(1) - \big(G'(1)\big)^2,$$

which is exactly Eq. (13).

3 [M15]

In Algorithm M (find the maximum), step M4 is executed exactly when the current item sets a new record—i.e., it is larger than all items seen so far. Thus, if we let A_n be the number of records in a uniformly random permutation of n distinct items, the exercise asks for the four basic statistics of A_{1000}.

Knuth derives the general result (for any n) that

$$\min A_n = 0, \qquad \mathbb{E}[A_n] = H_n - 1, \qquad \max A_n = n-1, \qquad \text{sd}(A_n) = \sqrt{H_n - H_n^{(2)}},$$

where $H_n = \sum_{k=1}^{n} \frac{1}{k}$ and $H_n^{(2)} = \sum_{k=1}^{n} \frac{1}{k^2}$.

Below I re-derive these quickly, then plug in $n = 1000$.

Quick derivation (indicator variables)

Let I_k be the indicator that the k-th item is a new record (so $I_k = 1$ iff the k-th item exceeds all of the first $k-1$ items). Then

$$A_n = \sum_{k=2}^{n} I_k.$$

By symmetry, $\Pr(I_k = 1) = 1/k$ (the maximum of the first k positions is equally likely to be at any of them). Hence

$$\mathbb{E}[A_n] = \sum_{k=2}^{n} \frac{1}{k} = H_n - 1.$$

Moreover, one checks that the events $\{I_i = 1\}$ and $\{I_j = 1\}$ are pairwise independent for $i \neq j$ in a random permutation, so

$$\mathrm{Var}(A_n) = \sum_{k=2}^{n} \mathrm{Var}(I_k) = \sum_{k=2}^{n} \left(\tfrac{1}{k} - \tfrac{1}{k^2} \right) = H_n - H_n^{(2)}.$$

Therefore $\mathrm{sd}(A_n) = \sqrt{H_n - H_n^{(2)}}$. The extremal values are clear:

- Min 0: occurs when the very first item is the overall maximum.
- Max $n-1$: occurs for a strictly increasing sequence.

Plug in $n = 1000$

Numerically,

$$H_{1000} \approx 7.485470860550345, \qquad H_{1000}^{(2)} \approx 1.643934566681560,$$

so

$$\mathbb{E}[A_{1000}] = H_{1000} - 1 \approx 6.48547086055,$$

$$\mathrm{Var}(A_{1000}) = H_{1000} - H_{1000}^{(2)} \approx 5.84153629387, \quad \mathrm{sd}(A_{1000}) \approx 2.41692703528.$$

4 [M10]

We consider the coin-tossing experiment of Eq. (17), where a head occurs with probability p (and tail with $q = 1 - p$), and p_{nk} denotes the probability of getting exactly k heads in n independent tosses. The recurrence is

$$p_{nk} = p\,p_{n-1,\,k-1} + q\,p_{n-1,\,k}.$$

We need to give a closed form for p_{nk}.

Solution (two complementary derivations)

1) Generating-function derivation (the way the section leads you to do it)

Define $G_n(z) = \sum_{k\geq 0} p_{nk} z^k$. Multiplying the recurrence by z^k and summing over k yields

$$G_n(z) = (q + pz)\,G_{n-1}(z), \qquad G_1(z) = q + pz.$$

Hence by induction

$$G_n(z) = (q + pz)^n. \text{ (Eq. (18))} \quad \text{(see the text's derivation)}$$

Expanding with the binomial theorem and reading off the coefficient of z^k gives

$$\boxed{p_{nk} = \binom{n}{k} p^k q^{n-k}} \quad (0 \leq k \leq n), \quad p_{nk} = 0 \text{ otherwise.}$$

2) Direct counting (combinatorial) derivation

Choose which k of the n tosses are heads: $\binom{n}{k}$ choices. Each such outcome has probability $p^k q^{n-k}$ by independence. Summing over the $\binom{n}{k}$ equiprobable patterns yields the same closed form:

$$p_{nk} = \binom{n}{k} p^k q^{n-k}.$$

Quick checks

- Normalization: $\sum_{k=0}^{n} p_{nk} = (p + q)^n = 1$.
- Recurrence: The formula satisfies $p_{nk} = p\,p_{n-1,k-1} + q\,p_{n-1,k}$ identically (verify with Pascal's identity).
- For a fair coin $p = \frac{1}{2}$: $p_{nk} = 2^{-n}\binom{n}{k}$.

5 [M13]

Here "Fig. 11" is the binomial case with $n = 12$ independent tosses and success probability $p = \frac{3}{5}$ (so $q = 1 - p = \frac{2}{5}$). For a binomial distribution the probability-generating function is

$$G_n(z) = (q + pz)^n,$$

and from the general identities

$$\text{mean}(G_n) = G_n'(1), \qquad \text{var}(G_n) = G_n''(1) + G_n'(1) - \left(G_n'(1)\right)^2,$$

we obtain $\mathbb{E}[X] = np$ and $\text{Var}(X) = npq$, hence $\sigma = \sqrt{npq}$.

Plugging in $n = 12,\ p = \frac{3}{5},\ q = \frac{2}{5}$:

1. Mean

$$\mathbb{E}[X] = np = 12 \cdot \frac{3}{5} = \frac{36}{5} = 7.2.$$

2. Standard deviation

$$\sigma = \sqrt{npq} = \sqrt{12 \cdot \frac{3}{5} \cdot \frac{2}{5}} = \sqrt{\frac{72}{25}} = \frac{\sqrt{72}}{5} = \frac{6\sqrt{2}}{5} \approx 1.697.$$

So the distribution in Fig. 11 has mean 7.2 and standard deviation $\frac{6\sqrt{2}}{5} \approx 1.697$.

6 [HM27]

(18) Binomial (n, p)

Here $G(e^t) = (q + pe^t)^n$, so

$$\ln G(e^t) = n \ln(q + pe^t).$$

Differentiate three times and evaluate at $t = 0$:

$$\kappa_1 = np, \qquad \kappa_2 = npq, \qquad \boxed{\kappa_3 = npq\,(1 - 2p)}.$$

(20) Discrete uniform on $\{1, \dots, n\}$

This distribution is symmetric about its mean $(n+1)/2$: for each deviation $+\delta$ there is a matching $-\delta$ with equal probability $1/n$. Hence all odd central moments are zero, so the third cumulant (which equals the third central moment) is

$$\boxed{\kappa_3 = 0}.$$

(8) Number of "record highs" A in Algorithm M

From §1.2.10 we can factor the pgf for A as

$$G_n(z) = \prod_{k=2}^{n} Q_k(z), \qquad Q_k(z) = \frac{z+k-1}{k} = \left(1 - \frac{1}{k}\right) + \frac{1}{k}\,z,$$

so A is a sum of independent Bernoulli variables I_k with $\Pr(I_k = 1) = 1/k$. Cumulants add for products (because $\ln G$ turns products into sums), so

$$\kappa_3(A) = \sum_{k=2}^{n} \kappa_3(I_k).$$

For a Bernoulli(p) variable, $\kappa_3 = p(1-p)(1-2p)$ (the binomial result with $n = 1$). Thus with $p = 1/k$,

$$\kappa_3(I_k) = \frac{1}{k}\left(1 - \frac{1}{k}\right)\left(1 - \frac{2}{k}\right) = \frac{(k-1)(k-2)}{k^3} = \frac{1}{k} - \frac{3}{k^2} + \frac{2}{k^3}.$$

Summing from $k = 2$ to n gives

$$\boxed{\kappa_3(A) \;=\; \sum_{k=2}^{n}\left(\frac{1}{k} - \frac{3}{k^2} + \frac{2}{k^3}\right) \;=\; H_n - 3H_n^{(2)} + 2H_n^{(3)}}.$$

Here $H_n = \sum_{k=1}^{n} \frac{1}{k}$ and $H_n^{(m)} = \sum_{k=1}^{n} \frac{1}{k^m}$. This matches the factorization of G_n in (8) and the additivity of semi-invariants stated in the section.

7 [M27]

In §1.2.10, A is the number of times Algorithm M changes the current maximum while scanning $X[1], \ldots, X[n]$. (Minimum 0, maximum $n-1$.) The section shows how to analyze A when all values are distinct and even gives the exact distribution $p_{n,k} = \Pr(A = k)$ in terms of Stirling numbers of the first kind:

$$p_{n,k} = \frac{\left[{n \atop k+1}\right]}{n!}.$$

(See Eq. (9) and the surrounding discussion.)

Exercise 7 weakens the hypothesis: assume only that the multiset $\{X[1], \ldots, X[n]\}$ contains exactly m distinct values (ties are allowed), otherwise random subject to this constraint. Find the distribution of A.

Key observation

A change of maximum occurs exactly when we encounter, for the first time, a value that is strictly larger than all values seen so far. Therefore, for purposes of counting A, all that matters is the order in which the m distinct values make their first appearance. Repetitions between first appearances do not affect A.

Condition on "exactly m distinct values appear." Under the natural "uniform subject to the constraint" model (equivalently: sample i.i.d. from m labels and condition on using all labels), the order of first appearances of the m distinct values is uniform over all $m!$ permutations. (This is a standard coupon-collector symmetry: conditioning on eventual appearance doesn't bias the relative order of the first hits.)

Let π be that random permutation of the m distinct values, written in increasing-value labels $1 < 2 < \cdots < m$. When we scan values in the order of first appearance (i.e., scan π), the running maximum increases once for each left-to-right maximum (record) of π after its first element. Thus,

$$A \;=\; (\#\text{ of records in } \pi) \;-\; 1.$$

But the number of left-to-right records in a random permutation of m elements has the well-known distribution

$$\Pr\{\#\text{records} = r\} = \frac{\left[{m \atop r}\right]}{m!} \quad (r = 1, \ldots, m),$$

the same Stirling-of-the-first-kind distribution used in the section for the "all distinct" case (there it appears with n and with $r = k+1$).

Result

Therefore, conditional on there being exactly m distinct values among $X[1], \dots, X[n]$,

$$\boxed{\Pr(A = k \mid \#\text{distinct} = m) = \frac{\left[{m \atop k+1}\right]}{m!}, \qquad k = 0, 1, \dots, m-1.}$$

In words: it's exactly the same distribution as in the all-distinct case, with n replaced by m (and with the shift by $+1$ because A counts "changes," not "records"). This reflects the fact that only the relative order of first occurrences of the m distinct values matters.

Checks and moments

- For $m = 3$: $\Pr(A = 0, 1, 2) = \frac{\left[{3 \atop 1}\right]}{6}, \frac{\left[{3 \atop 2}\right]}{6}, \frac{\left[{3 \atop 3}\right]}{6} = \frac{2}{6}, \frac{3}{6}, \frac{1}{6}$, matching the table in the section for $n = 3$.
- By the same substitution $n \mapsto m$ in Eq. (16), the conditional mean and variance are

$$\mathbb{E}[A \mid m] = H_m - 1, \qquad \mathrm{Var}(A \mid m) = H_m - H_m^{(2)},$$

so A has summary $(\min 0,\ \text{ave}\ H_m - 1,\ \max m - 1,\ \text{dev}\ \sqrt{H_m - H_m^{(2)}})$.

8 [M20]

Total outcomes Each position has M choices, independently, so there are M^n sequences in all.

Favorable outcomes (all entries distinct) Fill the n positions without repeating any element:

$$M \cdot (M-1) \cdot (M-2) \cdots (M-n+1) \;=\; \frac{M!}{(M-n)!} \;=\; (M)_n$$

(the falling factorial).

Therefore, for $0 \le n \le M$,

$$\boxed{\Pr(\text{all distinct}) \;=\; \frac{(M)_n}{M^n} \;=\; \frac{M!}{(M-n)!\, M^n}}$$

and for $n > M$ the probability is 0.

Edge cases

- $n = 0$: probability 1 (empty selection).

- $n = 1$: probability 1.
- $n = M$: probability $M!/M^M$.

Useful approximations (birthday-style)

When $n \ll M$,

$$\ln \Pr(\text{distinct}) = \sum_{i=0}^{n-1} \ln\Big(1 - \frac{i}{M}\Big) \approx -\frac{n(n-1)}{2M}$$

so

$$\Pr(\text{distinct}) \approx \exp\left(-\frac{n(n-1)}{2M}\right).$$

A sharper expansion keeps the next term:

$$\Pr(\text{distinct}) \approx \exp\left(-\frac{n(n-1)}{2M} - \frac{n(n-1)(2n-1)}{12M^2}\right).$$

These are the classic "birthday problem" approximations and are handy when M is large.

9 [M25]

Combinatorial derivation (with Stirling numbers of the second kind)

Count sequences $X[1], \dots, X[n]$ over an M-symbol alphabet that use exactly m symbols, then divide by the total M^n sequences.

1. Choose which m symbols appear: $\binom{M}{m}$.
2. Among those m chosen symbols, count length-n sequences in which each of the m symbols appears at least once (i.e., surjections from the n positions to the m symbols). Number of such sequences = (number of onto functions) $= m! \left\{ {n \atop m} \right\}$, where $\left\{ {n \atop m} \right\}$ is a Stirling number of the second kind (partitions of an n-set into m nonempty unlabeled blocks, then label the blocks by the m symbols).
3. Divide by the total number M^n of sequences.

Therefore the probability is

$$\boxed{q_{n,m} = \frac{\binom{M}{m}\, m!\, \left\{ {n \atop m} \right\}}{M^n} = \frac{(M)_m \left\{ {n \atop m} \right\}}{M^n} = \frac{M!\, \left\{ {n \atop m} \right\}}{(M-m)!\, M^n}\,,} \qquad 0 \le m \le \min\{n, M\},$$

and $q_{n,m} = 0$ otherwise.

Equivalent inclusion–exclusion form

Using $m!\left\{ {n \atop m} \right\} = \sum_{j=0}^{m} (-1)^j \binom{m}{j} (m-j)^n$, an equivalent expression is

$$q_{n,m} = \frac{\binom{M}{m}}{M^n} \sum_{j=0}^{m} (-1)^j \binom{m}{j} (m-j)^n.$$

Recurrence check (dynamic viewpoint)

Let $q_{n,m}$ be the desired probability. When we append the nth draw:

- With probability $\frac{m}{M}$ we hit one of the m symbols already seen (stay at m).
- With probability $\frac{M-m}{M}$ we introduce a new symbol (go from $m-1$ to m).

Hence

$$q_{n,m} = \frac{M-m+1}{M}\, q_{n-1,m-1} + \frac{m}{M}\, q_{n-1,m}, \quad q_{1,1} = 1,\ q_{1,m\neq 1} = 0.$$

One can verify that the closed form above satisfies this recurrence.

Sanity checks

- All distinct (Exercise 8). Setting $m = n$ gives $q_{n,n} = \dfrac{(M)_n}{M^n}$, the probability that all n draws are different (agrees with Ex. 8).
- Exactly one distinct value. $q_{n,1} = \dfrac{M}{M^n} = M^{1-n}$ (all draws identical).
- Support. $q_{n,m} = 0$ if $m > n$ or $m > M$.

10 [M20]

Step 1 — Condition on the number of distinct values

Let m be the number of distinct values that actually appear among $X[1..n]$. Given m, duplicates don't affect when new records occur; Algorithm M behaves exactly as if we had m distinct inputs. Therefore

$$\Pr(A = k \mid m) = p_{mk},$$

where (from §1.2.10) the "distinct inputs" distribution is

$$p_{mk} = \frac{\left[\, m \;\; k{+}1 \,\right]}{m!},$$

with $\left[\, m \;\; r \,\right]$ the (unsigned) Stirling numbers of the first kind.

Step 2 — Probability of having exactly m distinct values

If we sample n times from M equally likely symbols, the probability that exactly m distinct symbols appear is

$$q_{nm} = \frac{M!}{(M-m)!}\,\frac{\left\{\, n \;\; m \,\right\}}{M^{\,n}},$$

where $\left\{\, n \;\; m \,\right\}$ are Stirling numbers of the second kind (they count surjections of n labeled draws onto m labels).

Step 3 — Law of total probability

Sum over feasible m (you need at least $k{+}1$ distinct values to get k strict record increases, and you can't exceed either n or M):

$$\boxed{\Pr(A = k) = \sum_{m=k+1}^{\min(n,M)} q_{nm}\, p_{mk} = \frac{1}{M^{\,n}} \sum_{m=k+1}^{\min(n,M)} \binom{M}{m} \left\{\, n \;\; m \,\right\} \left[\, m \;\; k{+}1 \,\right]}$$

Notes and quick check

- Range: $k = 0, 1, \ldots, \min(n{-}1, M{-}1)$.
- Example $n = 3, M = 2$: Only $m = 1, 2$ are possible. $\Pr(m{=}1) = 2/2^3 = 1/4 \Rightarrow \Pr(A{=}0 \mid m{=}1) = 1$. $\Pr(m{=}2) = 6/8 = 3/4$ and $p_{2,0} = p_{2,1} = 1/2$. So $\Pr(A{=}0) = 1/4 + (3/4)(1/2) = 5/8$, $\Pr(A{=}1) = (3/4)(1/2) = 3/8$, agreeing with direct enumeration.

(Optional) Mean

Because $E[A \mid m] = H_m - 1$ in the distinct-case analysis, the overall mean can be written

$$E[A] = \sum_m q_{nm}(H_m - 1) = H_M - \sum_{m=1}^{M} \frac{\left(1 - \frac{m}{M}\right)^n}{m} - 1,$$

a compact but not further-simplifiable form in general.

11 [M15]

Setup (what semi-invariants are)

Knuth defines the semi-invariants $\kappa_1, \kappa_2, \ldots$ of a distribution with probability generating function $G(z)$ via the logarithm of the *characteristic* (a.k.a. exponential generating) function $G(e^t)$:

$$\ln G(e^t) \;=\; \frac{\kappa_1 t}{1!} + \frac{\kappa_2 t^2}{2!} + \frac{\kappa_3 t^3}{3!} + \cdots.$$

Equivalently, $\kappa_n = \left.\frac{d^n}{dt^n} \ln G(e^t)\right|_{t=0}$. In particular, $\kappa_1 = G'(1)$ (the mean) and $\kappa_2 = G''(1) + G'(1) - G'(1)^2$ (the variance).

Apply the transformation $F(z) = z^n G(z)$

Compute the corresponding exponential form:

$$F(e^t) \;=\; e^{nt}\, G(e^t) \quad \Longrightarrow \quad \ln F(e^t) \;=\; nt + \ln G(e^t).$$

Now compare coefficients of powers of t with the defining series above:

- The coefficient of t^1 increases by n, so

$$\kappa_1' = \kappa_1 + n.$$

- For every order $r \geq 2$, the coefficients are unchanged:

$$\kappa_r' = \kappa_r \qquad (r \geq 2).$$

Intuition and quick check

Multiplying $G(z)$ by z^n is exactly the probability-generating-function operation for adding a constant n to the random variable ($Y = X + n$). Cumulants beyond first order are translation-invariant, so only the first cumulant (the mean) shifts. Consequently, the variance and all higher cumulants stay the same. This agrees with the mean/variance formulas from the section: $ mean=G'(1)$ and var $= G''(1) + G'(1) - G'(1)^2$; for F we get $F'(1) = n + G'(1)$ while the variance expression is unaffected.

Final answer

Under $F(z) = z^n G(z)$,

$$\boxed{\kappa_1' = \kappa_1 + n, \qquad \kappa_r' = \kappa_r \ \text{ for all } r \geq 2.}$$

12 [HM21]

1) Show $G(e^t) = 1 + M_1 t + M_2 \frac{t^2}{2!} + M_3 \frac{t^3}{3!} + \cdots$

$$G(e^t) = \sum_{k\geq 0} p_k e^{kt} = \sum_{k\geq 0} p_k \sum_{n\geq 0} \frac{(kt)^n}{n!} = \sum_{n\geq 0} \frac{t^n}{n!} \Big(\sum_{k\geq 0} k^n p_k \Big) = \sum_{n\geq 0} \frac{M_n}{n!}\, t^n.$$

In particular $G(1) = \sum_k p_k = 1$, so $G(e^t)$ has constant term 1.

2) Cumulants κ_n from Arbogast/Faà di Bruno

Let $H(t) = G(e^t)$. The cumulant generating function is

$$K(t) = \log H(t) = \log G(e^t), \qquad \kappa_n = K^{(n)}(0).$$

Arbogast's (Faà di Bruno) formula for derivatives of a composition gives, for $f = \log$ and $g = H$,

$$\frac{d^n}{dt^n} \log H(t) = \sum_{\substack{k_1,\ldots,k_n \geq 0 \\ 1k_1 + 2k_2 + \cdots + nk_n = n}} (-1)^{m-1} (m-1)! \, \frac{n!}{k_1! \cdots k_n! \, 1!^{k_1} \cdots n!^{k_n}} \prod_{j=1}^{n} \Big(\frac{H^{(j)}(t)}{H(t)} \Big)^{k_j},$$

where $m = k_1 + \cdots + k_n$. Evaluating at $t = 0$ uses $H(0) = G(1) = 1$ and $H^{(j)}(0) = M_j$ from part (1). Therefore

$$\boxed{\kappa_n = \sum_{\substack{k_1,\dots,k_n\ge 0\\ 1k_1+2k_2+\cdots+nk_n=n}} (-1)^{m-1}(m-1)!\,\frac{n!}{k_1!\cdots k_n!\,1!^{k_1}\cdots n!^{k_n}}\prod_{j=1}^{n} M_j^{k_j}\,, \qquad m=k_1+\cdots+k_n.}$$

This is exactly the Bell-polynomial form asked for in the exercise. Checking small n reproduces the book's identities:

- $\kappa_1 = M_1$.
- $\kappa_2 = M_2 - M_1^2$.
- $\kappa_3 = M_3 - 3M_1M_2 + 2M_1^3$.
- $\kappa_4 = M_4 - 4M_1M_3 + 12M_1^2M_2 - 3M_2^2 - 6M_1^4$.

3) Express κ_n via central moments m_j (for $n \ge 2$)

Center the variable by $Y = X - M_1$. Then

$$\log \mathbb{E}[e^{tY}] = \log\big(e^{-M_1 t}H(t)\big) = K(t) - M_1 t.$$

Hence the cumulants for Y satisfy $\kappa_1' = 0$ and $\kappa_n' = \kappa_n$ for $n \ge 2$. But the derivatives of $\mathbb{E}[e^{tY}]$ at 0 are exactly the central moments: $(\mathbb{E}[e^{tY}])^{(j)}(0) = m_j$ with $m_1 = 0$. Applying the same Arbogast/Bell-polynomial formula with $M_1 = 0$ and M_j replaced by m_j (for $j \ge 2$) yields, for $n \ge 2$,

$$\boxed{\kappa_n = \sum_{\substack{k_2,\dots,k_n\ge 0\\ 2k_2+3k_3+\cdots+nk_n=n}} (-1)^{m-1}(m-1)!\,\frac{n!}{k_2!\cdots k_n!\,2!^{k_2}\cdots n!^{k_n}}\prod_{j=2}^{n} m_j^{k_j}, \qquad m=k_2+\cdots+k_n.}$$

Concrete low-order cases:

- $\kappa_2 = m_2$.
- $\kappa_3 = m_3$.
- $\kappa_4 = m_4 - 3m_2^2$.
- $\kappa_5 = m_5 - 10\,m_3m_2$.
- $\kappa_6 = m_6 - 15\,m_4m_2 - 10\,m_3^2 + 30\,m_2^3$.

13 [HM38]

Step 1: Bring in the exact pgf and its mean/variance

From Eq. (8) in §1.2.10,

$$G_n(z) = \frac{1}{n!}\,(z+n-1)(z+n-2)\cdots(z+1) = \frac{\Gamma(n+z)}{\Gamma(z+1)\,\Gamma(n+1)}.$$

(The gamma form is immediate from the product.)

Moreover, §1.2.10 obtains

$$\mu_n = \mathrm{mean}(G_n) = H_n - 1, \qquad \sigma_n^2 = \mathrm{var}(G_n) = H_n - H_n^{(2)}.$$

Here $H_n = \sum_{k=1}^{n} \frac{1}{k}$ and $H_n^{(2)} = \sum_{k=1}^{n} \frac{1}{k^2}$.

Useful asymptotics: $H_n = \ln n + \gamma + O(1/n)$ and $H_n^{(2)} = \frac{\pi^2}{6} + O(1/n)$. Hence $\sigma_n^2 \sim \ln n$ and $\mu_n - \ln n = O(1)$.

Step 2: Define the normalized characteristic function

Set $z_n = e^{it/\sigma_n}$. The normalized characteristic function in the exercise is

$$\Phi_n(t) \;:=\; e^{-it\mu_n/\sigma_n}\, G_n(z_n).$$

We will show $\Phi_n(t) \to e^{-t^2/2}$.

Step 3: Take logs and use a ratio for gamma

Using the gamma form,

$$\ln G_n(z) = \ln\Gamma(n+z) - \ln\Gamma(n+1) - \ln\Gamma(z+1).$$

For fixed complex z with $\Re z > -1$, the classical ratio asymptotic

$$\frac{\Gamma(n+z)}{\Gamma(n+1)} = n^{\,z-1}(1 + O(1/n))$$

gives

$$\ln G_n(z) = (z-1)\ln n - \ln\Gamma(z+1) + O(1/n).$$

Since $z_n = e^{it/\sigma_n} \to 1$, we have $\Gamma(z_n+1) \to \Gamma(2) = 1$ and thus $\ln\Gamma(z_n+1) \to 0$.

Therefore

$$\ln \Phi_n(t) = -\frac{it\mu_n}{\sigma_n} \;+\; (z_n - 1)\,\ln n \;+\; o(1).$$

Step 4: Expand z_n and pass to the limit

Expand $z_n = e^{it/\sigma_n} = 1 + \frac{it}{\sigma_n} - \frac{t^2}{2\sigma_n^2} + O(\sigma_n^{-3})$. Plugging in,

$$\ln \Phi_n(t) = \Big(\frac{it}{\sigma_n}\ln n - \frac{it\mu_n}{\sigma_n}\Big) \;-\; \frac{t^2}{2\sigma_n^2}\ln n \;+\; o(1).$$

Now use the §1.2.10 statistics and harmonic-number asymptotics:

- $\mu_n - \ln n = O(1)$ and $\sigma_n \sim \sqrt{\ln n} \Rightarrow \dfrac{(\mu_n - \ln n)}{\sigma_n} \to 0$. So the linear (imaginary) term vanishes.
- $\dfrac{\ln n}{\sigma_n^2} \to 1$ since $\sigma_n^2 = H_n - H_n^{(2)} = \ln n + O(1)$. So the quadratic term tends to $-t^2/2$.

Thus $\ln \Phi_n(t) \to -t^2/2$, hence $\Phi_n(t) \to e^{-t^2/2}$, proving the required convergence to the normal law.

Intuition (why this works)

Eq. (8) factorizes G_n as a product of simple pgfs $Q_k(z) = (z + k - 1)/k$. That corresponds to a sum of independent Bernoulli variables with $\Pr(B_k = 1) = 1/k$ (so $A_n = \sum_{k=2}^{n} B_k$). Sums of many small, independent contributions tend to normality; the calculation above is the precise characteristic-function proof tailored to these particular parameters. (§1.2.10 already hints these distributions are "approximately normal" for large n.)

14 [HM30]

Write $\sigma = \sigma_n = \sqrt{npq}$ and consider

$$\Phi_n(t) \;=\; e^{-it\mu/\sigma}\, G_n\big(e^{it/\sigma}\big) \;=\; e^{-it\,np/\sigma}\,\Big(q + p\,e^{it/\sigma}\Big)^n.$$

Take logarithms:

$$\log \Phi_n(t) \;=\; -\frac{it\,np}{\sigma} \;+\; n\log\Big(q + p\,e^{it/\sigma}\Big).$$

Use the Taylor expansions $e^{ix} = 1 + ix - \frac{x^2}{2} + O(x^3)$ and $\log(1+x) = x - \frac{x^2}{2} + O(x^3)$ as $x \to 0$. First expand inside the log:

$$q + p\,e^{it/\sigma} = q + p\left(1 + \frac{it}{\sigma} - \frac{t^2}{2\sigma^2} + O(\sigma^{-3})\right) = 1 + \underbrace{p\frac{it}{\sigma} - p\frac{t^2}{2\sigma^2}}_{=:x} + O(\sigma^{-3}).$$

Now

$$\log\Big(q + p\,e^{it/\sigma}\Big) = x - \frac{x^2}{2} + O(\sigma^{-3}) = \frac{pit}{\sigma} - \frac{pq\,t^2}{2\sigma^2} + O(\sigma^{-3}),$$

since $x^2 = -\,p^2t^2/\sigma^2 + O(\sigma^{-3})$ and $pq = p(1-p)$.

Multiply by n:

$$n\log\Big(q + p\,e^{it/\sigma}\Big) = \frac{it\,np}{\sigma} \;-\; \frac{pq\,t^2}{2}\cdot\frac{n}{\sigma^2} \;+\; n\cdot O(\sigma^{-3}).$$

Because $\sigma^2 = npq$, we have $n/\sigma^2 = 1/(pq)$ and $n\cdot O(\sigma^{-3}) = O(1/\sqrt{n}) \to 0$. Hence

$$n\log\Big(q + p\,e^{it/\sigma}\Big) = \frac{it\,np}{\sigma} - \frac{t^2}{2} + o(1).$$

Finally,

$$\log\Phi_n(t) = -\frac{it\,np}{\sigma} + \left(\frac{it\,np}{\sigma} - \frac{t^2}{2} + o(1)\right) = -\frac{t^2}{2} + o(1),$$

so $\Phi_n(t) \to e^{-t^2/2}$ as $n \to \infty$. This is exactly Knuth's criterion for "approaches a normal distribution," so the binomial distribution (properly standardized) approaches the normal law (de Moivre–Laplace).

(As usual, exclude the degenerate cases $p \in \{0, 1\}$ where $\sigma_n = 0$.)

15 [HM23]

(a) Probability generating function

If $X \sim \text{Poisson}(\mu)$, then

$$\Pr(X = k) = e^{-\mu}\frac{\mu^k}{k!} \quad (k = 0, 1, 2, \dots).$$

The probability generating function (PGF) is

$$G(z) = \sum_{k\geq 0} \Pr(X = k)\, z^k = e^{-\mu} \sum_{k\geq 0} \frac{(\mu z)^k}{k!} = e^{-\mu} e^{\mu z} = \exp\bigl(\mu(z-1)\bigr).$$

(b) Semi-invariants (cumulants)

Knuth defines the semi-invariants κ_n by the expansion of the logarithm of the generating function evaluated at e^t:

$$\ln G(e^t) = \sum_{n\geq 1} \kappa_n \frac{t^n}{n!}.$$

For Poisson,

$$\ln G(e^t) = \ln\bigl(\exp(\mu(e^t - 1))\bigr) = \mu\,(e^t - 1) = \mu \sum_{n\geq 1} \frac{t^n}{n!}.$$

Hence, for every $n \geq 1$,

$$\boxed{\kappa_n = \mu}.$$

In particular, $\kappa_1 = \mu = \mathrm{E}\,X$ and $\kappa_2 = \mu = \mathrm{Var}\,X$, matching the usual mean and variance of the Poisson law. (Definition and identities for κ_n are given just above Eq. (23) in §1.2.10.)

(c) Poisson(np) approaches normal as $n \to \infty$

Exercise 13 says a sequence with PGFs G_n, means μ_n, and standard deviations σ_n approaches a normal distribution iff

$$\lim_{n\to\infty} e^{-it\mu_n/\sigma_n}\, G_n\bigl(e^{it/\sigma_n}\bigr) = e^{-t^2/2} \quad \text{for all real } t.$$

For $X_n \sim \text{Poisson}(np)$,

$$G_n(z) = \exp(np(z-1)), \qquad \mu_n = np, \qquad \sigma_n^2 = np.$$

Compute the left side:

$$e^{-it\mu_n/\sigma_n}\, G_n\bigl(e^{it/\sigma_n}\bigr) = \exp\Bigl(np(e^{\,it/\sqrt{np}} - 1) - it\sqrt{np}\Bigr).$$

Use the Taylor series $e^{ix} = 1 + ix - \frac{x^2}{2} - i\frac{x^3}{6} + O(x^4)$ with $x = t/\sqrt{np}$:

$$e^{it/\sqrt{np}} - 1 - \frac{it}{\sqrt{np}} = -\frac{t^2}{2np} - i\frac{t^3}{6(np)^{3/2}} + O\Big(\frac{1}{n^2}\Big).$$

Multiplying by np gives

$$np\Big(e^{it/\sqrt{np}} - 1 - \frac{it}{\sqrt{np}}\Big) = -\frac{t^2}{2} - i\frac{t^3}{6\sqrt{np}} + O\Big(\frac{1}{n}\Big).$$

Therefore

$$e^{-it\mu_n/\sigma_n}\, G_n\big(e^{it/\sigma_n}\big) = \exp\Big(-\frac{t^2}{2} + o(1)\Big) \xrightarrow[n\to\infty]{} e^{-t^2/2},$$

which is exactly the criterion in Exercise 13. Thus Poisson(np) standardized converges to the normal distribution.

16 [M25]

Preliminaries (from §1.2.10)

If $G(z) = \sum_{n\ge 0} p_n z^n$ is a pgf (so $G(1) = 1$), then

$$\text{mean}(G) = G'(1), \qquad \text{var}(G) = G''(1) + G'(1) - \big(G'(1)\big)^2.$$

These are Eq. (12) and Eq. (13) in the text.

Step 1 — The mixture pgf

Let X be the sample from the chosen component, and let K be the index of the chosen component. Then

$$g(z) \;=\; \mathbb{E}\big[z^X\big] \;=\; \sum_k \Pr(K=k)\,\mathbb{E}\big[z^X \mid K=k\big] \;=\; \sum_k p_k\, g_k(z).$$

So the pgf of a mixture is the convex combination of the component pgfs. (This is exactly what Knuth lists in the official answer.)

Step 2 — The mean of the mixture

Differentiate $g(z) = \sum_k p_k g_k(z)$ and evaluate at $z = 1$:

$$\text{mean}(g) = g'(1) = \sum_k p_k\, g_k'(1) = \sum_k p_k\, \mu_k.$$

Thus the mean of a mixture is the mixture of the means.

Step 3 — The variance of the mixture

Start from the variance formula in terms of derivatives:

$$\mathrm{var}(g) = g''(1) + g'(1) - \big(g'(1)\big)^2.$$

Linearity gives $g''(1) = \sum_k p_k g_k''(1)$ and $g'(1) = \sum_k p_k g_k'(1) = \sum_k p_k \mu_k$. Also, for each component, $g_k''(1) + g_k'(1) = \mathbb{E}_k[X^2] = \sigma_k^2 + \mu_k^2$. Hence

$$\begin{aligned}
\mathrm{var}(g) &= \sum_k p_k (g_k''(1) + g_k'(1)) - \Big(\sum_k p_k \mu_k\Big)^2 \\
&= \sum_k p_k (\sigma_k^2 + \mu_k^2) - \Big(\sum_k p_k \mu_k\Big)^2 \\
&= \underbrace{\sum_k p_k \sigma_k^2}_{\text{within–component variance}} + \underbrace{\Big(\sum_k p_k \mu_k^2 - \Big(\sum_k p_k \mu_k\Big)^2\Big)}_{\text{between–component variance}}.
\end{aligned}$$

The bracketed "between" term is the variance of the random mean μ_K. It can be rewritten in the symmetric form

$$\sum_{j<k} p_j p_k \, (\mu_j - \mu_k)^2,$$

since $\sum_k p_k \mu_k^2 - (\sum_k p_k \mu_k)^2 = \frac{1}{2} \sum_{j,k} p_j p_k (\mu_j - \mu_k)^2$. Therefore

$$\boxed{\mathrm{var}(g) \;=\; \sum_k p_k \, \sigma_k^2 \;+\; \sum_{j<k} p_j p_k \, (\mu_j - \mu_k)^2.}$$

This matches Knuth's given result.

Quick numerical sanity check

Take two Bernoulli components:

- Component 1: $p_1 = 0.6$, $\mu_1 = 0.3$, $\sigma_1^2 = 0.3(0.7) = 0.21$.
- Component 2: $p_2 = 0.4$, $\mu_2 = 0.8$, $\sigma_2^2 = 0.8(0.2) = 0.16$.

Mean: $0.6 \cdot 0.3 + 0.4 \cdot 0.8 = 0.18 + 0.32 = 0.50$.

Variance:

$$\sum_k p_k \sigma_k^2 = 0.6 \cdot 0.21 + 0.4 \cdot 0.16 = 0.126 + 0.064 = 0.19,$$

$$\sum_{j<k} p_j p_k (\mu_j - \mu_k)^2 = 0.6 \cdot 0.4 \cdot (0.3 - 0.8)^2 = 0.24 \cdot 0.25 = 0.06.$$

Total $= 0.19 + 0.06 = 0.25$. Indeed the mixture itself is Bernoulli with $p = 0.5$, whose variance is $0.5(1 - 0.5) = 0.25$.

17 [M27]

(a) $h(z) = g(f(z))$ is a PGF

Write the PGFs as

$$f(z) = \sum_{n \geq 0} a_n z^n, \qquad g(z) = \sum_{k \geq 0} b_k z^k$$

with $a_n, b_k \geq 0$ and $f(1) = g(1) = 1$. Then

$$h(z) = g(f(z)) = \sum_{k \geq 0} b_k \big(f(z)\big)^k.$$

Each power $f(z)^k$ has nonnegative coefficients and value $f(1)^k = 1$ at $z = 1$. A nonnegative convex combination of such series has nonnegative coefficients and

$$h(1) = \sum_{k \geq 0} b_k f(1)^k = \sum_{k \geq 0} b_k = g(1) = 1.$$

Hence h is a PGF.

(b) Probabilistic meaning of h

Let K be a nonnegative-integer–valued random variable with PGF g (so $\Pr[K = k] = b_k$). Let $X_1, X_2, \ldots$ be i.i.d. with PGF f, independent of K. Define the random sum

$$S \;=\; X_1 + \cdots + X_K \quad (\text{take } S = 0 \text{ if } K = 0).$$

Conditioning on K,

$$\mathrm{PGF}(S \mid K = k) = \big(f(z)\big)^k,$$

so unconditionally

$$\mathrm{PGF}(S) = \sum_{k \geq 0} b_k f(z)^k \;=\; g(f(z)) \;=\; h(z).$$

Thus h is the PGF of a random sum ("compound" distribution): the number of summands is random ($K \sim g$), each summand has PGF f. This is the standard compounding/branching-process composition rule for PGFs.

(c) Mean and variance of h via those of f and g

Let

$$\mu_f = f'(1), \quad \sigma_f^2 = f''(1) + \mu_f - \mu_f^2, \qquad \mu_g = g'(1), \quad \sigma_g^2 = g''(1) + \mu_g - \mu_g^2.$$

(These are the PGF formulas from the section.)

Compute derivatives of $h(z) = g(f(z))$:

$$h'(z) = g'(f(z))\, f'(z), \qquad h''(z) = g''(f(z))\, [f'(z)]^2 + g'(f(z))\, f''(z).$$

Evaluate at $z = 1$ (using $f(1) = g(1) = 1$):

$$h'(1) = g'(1) f'(1) = \mu_g \mu_f,$$

$$h''(1) = g''(1)\, \mu_f^2 + \mu_g\, f''(1).$$

Now plug into $\mathrm{var}(h) = h''(1) + h'(1) - [h'(1)]^2$ and use $f''(1) = \sigma_f^2 - \mu_f + \mu_f^2$, $g''(1) = \sigma_g^2 - \mu_g + \mu_g^2$: after cancellation one obtains the clean composition laws

$$\boxed{\mathrm{E}[S] = \mu_h = \mu_g\, \mu_f}$$

$$\boxed{\mathrm{Var}(S) = \sigma_h^2 = \mu_g\,\sigma_f^2 + \sigma_g^2\,\mu_f^2}.$$

These match the intuitive random-sum identities $\mathrm{E}[S] = \mathrm{E}[K]\mathrm{E}[X]$ and $\mathrm{Var}(S) = \mathrm{E}[K]\mathrm{Var}(X) + (\mathrm{E}[X])^2\mathrm{Var}(K)$.

Quick check (compound Poisson)

If $K \sim \mathrm{Poisson}(\lambda)$, then $g(z) = e^{\lambda(z-1)}$ so $\mu_g = \sigma_g^2 = \lambda$. The formulas give

$$\mu_h = \lambda\,\mu_f, \qquad \sigma_h^2 = \lambda\,\sigma_f^2 + \lambda\,\mu_f^2 = \lambda\,\mathrm{E}[X^2],$$

which is the classical compound-Poisson result. (This exercise sits right after the coin-tossing/uniform-distribution PGF work and the mean/variance-from-PGF identities.)

18 [M28]

Write

$$S_j \;:=\; k_j + k_{j+1} + \cdots + k_n \quad (1 \le j \le n), \qquad S_{n+1} := 0.$$

Recall the "all distinct" case (each $k_j = 1$) has pgf

$$G_n(z) \;=\; \sum_{k\ge 0} p_{n,k}\,z^k \;=\; \frac{1}{n!}\,(z+1)(z+2)\cdots(z+n-1) \;=\; \frac{1}{z+n}\binom{z+n}{n}. \tag{*}$$

(That is Eq. (8) in the book.)

What we will prove

With multiplicities $k_1, \dots, k_n$, the pgf of A is

$$\boxed{G(z) \;=\; \frac{1}{z}\prod_{j=1}^{n} \frac{\binom{k_j z + S_{j+1}}{k_j}}{\binom{S_j}{k_j}}} \tag{1}$$

(with the natural convention $\binom{0}{0} = 1$).

Proof idea (merge "from the top")

Build the random permutation by successively shuffling blocks of equal values, going downwards from the largest value n to 1.

Let $H_j(z)$ be the pgf of the number of record highs seen among the values $\{j, j+1, \dots, n\}$ when those blocks are randomly interleaved (this counts the very first record within that restricted universe). Clearly $H_{n+1}(z) = 1$.

Now merge the block of k_j copies of value j into the already-merged block of all higher values (size S_{j+1}). In that merge,

- we get exactly one new record from value j iff at least one j appears before the first element of $\{j+1, \dots, n\}$;
- otherwise, we get no new record from value j.

Among all $\binom{S_j}{k_j}$ interleavings of k_j indistinguishable j's with S_{j+1} higher elements, the weighted count where we give weight z when a j leads and weight 1 otherwise is

$$\binom{k_j z + S_{j+1}}{k_j}.$$

(Equivalently, this is $\binom{S_j}{k_j} + (z-1)\binom{S_j-1}{k_j-1}$ in disguise; the compact form with $\binom{k_j z + S_{j+1}}{k_j}$ is a standard way to package that linear weighting.) Therefore the average weight factor contributed by inserting the whole j-block is

$$\frac{\binom{k_j z + S_{j+1}}{k_j}}{\binom{S_j}{k_j}}.$$

So the recursive relation is

$$H_j(z) \;=\; \frac{\binom{k_j z + S_{j+1}}{k_j}}{\binom{S_j}{k_j}}\, H_{j+1}(z).$$

Multiplying these from $j = n, n-1, \dots, 1$ gives

$$H_1(z) \;=\; \prod_{j=1}^{n} \frac{\binom{k_j z + S_{j+1}}{k_j}}{\binom{S_j}{k_j}}.$$

Finally, $H_1(z)$ counts records including the very first one (the first element of the whole sequence). Our A in Algorithm M *excludes* the first (because step M4 does not fire at position 1), hence $G(z) = H_1(z)/z$, which is exactly (1).

Consistency checks

1. All distinct ($k_1 = \cdots = k_n = 1$). Then $S_{j+1} = n - j$, $\binom{k_j z + S_{j+1}}{k_j} = z + n - j$, and $\binom{S_j}{k_j} = n - j + 1$. Thus

$$H_1(z) = \prod_{j=1}^{n} \frac{z + n - j}{n - j + 1} = \frac{z(z+1)\cdots(z+n-1)}{n!}.$$

Dividing by z reproduces $(*)$.

2. Small example. Take values $\{1, 1, 3\}$ so $(k_1, k_2, k_3) = (2, 0, 1)$. Direct enumeration of the three arrangements $113, 131, 311$ gives A equal to $1, 1, 0$ respectively, hence $G(z) = (1/3) + (2/3)z$.

Formula (1): $S_1 = 3, S_2 = 1, S_3 = 1, S_4 = 0$. Then

$$H_1(z) = \frac{\binom{1\cdot z+0}{1}}{\binom{1}{1}} \cdot \frac{\binom{0\cdot z+1}{0}}{\binom{1}{0}} \cdot \frac{\binom{2\cdot z+1}{2}}{\binom{3}{2}} = z \cdot 1 \cdot \frac{2z^2 + z}{3} = \frac{2z^3 + z^2}{3}.$$

Divide by z: $G(z) = \frac{2z^2+z}{3z} = \frac{1}{3} + \frac{2}{3}z$, matching the enumeration.

3. Normalization. At $z = 1$, each factor equals 1, so $H_1(1) = 1$ and $G(1) = 1$.

What you get for free (means/variance)

Once you have $G(z)$, the mean and variance follow from the standard relations $\text{mean}(G) = G'(1)$, $\text{var}(G) = G''(1) + G'(1) - [G'(1)]^2$ exactly as in §1.2.10. (The text derives these identities right after introducing generating functions.)

19 [M21]

Fix k and put $\ell := a_k$. By definition of the inverse, $b_\ell = k$.

($\Rightarrow$) If a_k is a left-to-right maximum, then b_ℓ is a right-to-left minimum.

"a_k is L→R maximum" means

$$\forall j < k : \; a_j < \ell.$$

Because $a_1, \ldots, a_n$ is a permutation, every value t with $t > \ell$ appears somewhere among the a's. If any such $t > \ell$ appeared at some position $j \le k$, we would have $a_j = t > \ell = a_k$, contradicting that a_k dominates all earlier entries. Hence

$$\forall t > \ell : \text{ the position of } t \text{ is to the right of } k,$$

i.e.

$$\forall t > \ell : \; b_t > k = b_\ell.$$

This is precisely "b_ℓ is smaller than all entries to its right," i.e. b_ℓ is an R→L minimum.

(⇐) If b_ℓ is a right-to-left minimum, then a_k is a left-to-right maximum.

"b_ℓ is R→L minimum" means

$$\forall t > \ell : \; b_t > b_\ell = k.$$

Thus no value $t > \ell$ occurs at or before position k in the a-sequence. Therefore the first k entries $a_1, \dots, a_k$ contain no value larger than $\ell = a_k$, so a_k is larger than all a_j with $j < k$. Hence a_k is L→R maximum.

Together these prove the equivalence.

Quick example

Take $a = (3, 1, 5, 2, 4)$ (a permutation of $\{1, \dots, 5\}$).

- Left-to-right maxima in a: $a_1 = 3$ (trivially), $a_3 = 5$ (bigger than $3, 1$). So positions $k \in \{1, 3\}$.
- Inverse b lists positions of each value $1, \dots, 5$: value 1 at pos 2, 2 at 4, 3 at 1, 4 at 5, 5 at 3, so $b = (2, 4, 1, 5, 3)$.
- Right-to-left minima in b (scan from right): start with 3 (min so far), then 5 (not <3), then 1 (new min), then 4 (not), then 2 (not). So the R→L minima are the values 3 and 1, which equal $k \in \{3, 1\}$ — exactly the positions where a_k was an L→R maximum, as claimed.

20 [M22]

Because (b_k) is nondecreasing, moving left (to a smaller index) can only *decrease* b, and moving right can only *increase* b. So:

- If you're trying to make $a_k - b_k$ large, you want a as large as possible and b as small as possible look at prefix (left-to-right) maxima of a.
- If you're trying to make $b_k - a_k$ large, you want b as large as possible and a as small as possible look at suffix (right-to-left) minima of a.

Fix any $k \in \{1, \dots, n\}$.

1) Maximizers of $a_k - b_k$ are among left-to-right maxima. Let $i \le k$ be an index with $a_i = \max\{a_1, \dots, a_k\}$ (so i is a left-to-right maximum). Since b is nondecreasing, $b_i \le b_k$. Hence

$$a_i - b_i \;\ge\; a_k - b_i \;\ge\; a_k - b_k.$$

Therefore the global maximum of $a_k - b_k$ over all k is attained at some left-to-right maximum. Thus

$$\max_k (a_k - b_k) \;=\; m_L.$$

2) Maximizers of $b_k - a_k$ are among right-to-left minima. Let $j \ge k$ be an index with $a_j = \min\{a_k, a_{k+1}, \dots, a_n\}$ (so j is a right-to-left minimum). Since b is nondecreasing, $b_j \ge b_k$. Hence

$$b_j - a_j \;\ge\; b_k - a_j \;\ge\; b_k - a_k.$$

Therefore the global maximum of $b_k - a_k$ over all k is attained at some right-to-left minimum. Thus

$$\max_k (b_k - a_k) \;=\; m_R.$$

Taking absolute values case-by-case,

$$\max_k |a_k - b_k| = \max\{\max_k (a_k - b_k),\ \max_k (b_k - a_k)\} = \max(m_L, m_R),$$

as required.

Linear-time algorithm

1. Left scan: track `amax = -∞`. For $k = 1$ to n: if $a_k \geq amax$ (a left-to-right maximum), update `amax = a_k` and update $m_L \leftarrow \max(m_L, a_k - b_k)$.
2. Right scan: track `amin = +∞`. For $k = n$ down to 1: if $a_k \leq amin$ (a right-to-left minimum), update `amin = a_k` and update $m_R \leftarrow \max(m_R, b_k - a_k)$.
3. Answer: $M = \max(m_L, m_R)$.

This runs in $O(n)$ time and $O(1)$ extra space.

How many indices get checked (random a)?

For a uniformly random order of the a's, the expected number of left-to-right maxima is H_n and the expected number of right-to-left minima is also H_n (each position is a record with probability $1/k$ from the left and $1/(n-k+1)$ from the right). Hence the total number of *candidates* we examine is on the order of $2H_n \approx 2\ln n$ (slightly less after accounting for overlaps), matching the exercise's remark.

Sanity check with a tiny example

Let $a = (5, 2, 7, 3, 6)$, $b = (1, 2, 4, 8, 9)$.

- Left-to-right maxima of a: indices 1 (5), 3 (7). So $m_L = \max\{5-1,\ 7-4\} = 4$.
- Right-to-left minima of a: scanning from the right, minima at indices 5 (6) and 2 (2). So $m_R = \max\{9-6,\ 2-2\} = 3$. Thus $M = \max(m_L, m_R) = 4$. Directly checking all $|a_k - b_k|$ indeed gives the maximum 4 at $k = 1$.

That completes the proof and gives a practical $O(n)$ method.

21 [HM21]

Setup

- A *cycle structure* $(\alpha_1, \alpha_2, \ldots)$ means: exactly α_j cycles of length j for each $j \geq 1$.
- Feasibility constraint: $\sum_{j\geq 1} j\,\alpha_j = n$ (otherwise the probability is 0).

So the problem reduces to: among all $n!$ permutations, how many have that exact cycle structure?

Key counting fact

The number of permutations of $\{1, \ldots, n\}$ with cycle structure $(\alpha_1, \alpha_2, \ldots)$ is

$$\frac{n!}{\prod_{j\geq 1}(j^{\alpha_j}\,\alpha_j!)}.$$

This is a standard result about conjugacy classes in S_n: once the cycle lengths are fixed, the class size is $n!/(\prod_j j^{\alpha_j}\alpha_j!)$. Intuition:

- Start with any one-line ordering of the n labels: $n!$ possibilities.
- Writing cycles "around a circle" makes the *starting point* in each cycle irrelevant: divide by j for each j-cycle, hence $\prod_j j^{\alpha_j}$.
- Cycles of the same length are indistinguishable in their ordering: divide by $\alpha_j!$ for each j.

Probability

Dividing the count by $n!$, we obtain

$$\boxed{P(n;\alpha_1,\alpha_2,\ldots) \;=\; \begin{cases} \dfrac{1}{\prod_{j\ge1}(j^{\alpha_j}\,\alpha_j!)}, & \text{if } \sum_j j\,\alpha_j = n, \\ 0, & \text{otherwise.} \end{cases}}$$

Why this is consistent (quick checks)

1. Normalization Summing over all feasible (α_j) with $\sum j\alpha_j = n$ gives 1. A compact way to see this is via exponential generating functions:

$$\sum_{\sum j\alpha_j=n} \frac{n!}{\prod_j j^{\alpha_j}\alpha_j!} = n!\,[z^n]\prod_{j\ge1}\sum_{\alpha_j\ge0}\frac{(z^j)^{\alpha_j}}{j^{\alpha_j}\alpha_j!} = n!\,[z^n]\exp\left(\sum_{j\ge1}\frac{z^j}{j}\right) = n!\,[z^n]\frac{1}{1-z} = n!.$$

So the probabilities sum to 1.

2. Small examples

- $n=4$, type $(\alpha_1,\alpha_2,\alpha_3,\alpha_4)=(2,1,0,0)$: probability $= 1/(1^2\,2!\cdot 2^1\,1!) = 1/4$. There are $24\times 1/4 = 6$ such permutations, agreeing with "choose the 2-cycle's pair" $\binom{4}{2}=6$.
- $n=4$, type $(1,0,1,0)$ (a 3-cycle and a fixed point): probability $= 1/(1^1\,1!\cdot 3^1\,1!) = 1/3$. There are $24/3 = 8$, the known count of 3-cycles in S_4.

That's exactly the $P(n;\alpha_1,\alpha_2,\ldots)$ the exercise asks for.

Answer: $P(n;\alpha_1,\alpha_2,\ldots) = \left(\prod_{j\ge1} j^{\alpha_j}\alpha_j!\right)^{-1}$ when $\sum_j j\alpha_j = n$, and 0 otherwise.

22 [HM22]

(a) Prove the two Chernoff-style bounds

We must show, for $0 < r \le 1$ and $r \ge 1$,

$$\Pr(X \le \mu r) \le \left(r^{-r} e^{r-1}\right)^{\mu}, \qquad \Pr(X \ge \mu r) \le \left(r^{-r} e^{r-1}\right)^{\mu}.$$

Use (24) with $x = r$ for the left tail ($0 < r \le 1$):

$$\Pr(X \le \mu r) \;\le\; r^{-\mu r} G(r) = r^{-\mu r} \prod_{k=1}^{n} (q_k + p_k r).$$

For each factor, apply $1 + u \le e^u$ (valid for all real u) with $u = (r-1)p_k$:

$$q_k + p_k r \;=\; 1 + (r-1)p_k \;\le\; e^{(r-1)p_k}.$$

Hence

$$\Pr(X \le \mu r) \le r^{-\mu r} \exp\Big((r-1)\sum_{k=1}^{n} p_k\Big) = \left(r^{-r} e^{r-1}\right)^{\mu}.$$

Exactly the same argument, but using (25) with $x = r \ge 1$, gives the right-tail bound $\Pr(X \ge \mu r) \le \left(r^{-r} e^{r-1}\right)^{\mu}$.

(b) Simplify the bound when $r \approx 1$

Write $r = 1 + \delta$ with $|\delta| \le 1$. Consider

$$\log\left(r^{-r} e^{r-1}\right) \;=\; -r \ln r + r - 1.$$

A Taylor expansion at $r = 1$ gives

$$-r \ln r + r - 1 = -\frac{\delta^2}{2} + \frac{\delta^3}{6} - \frac{\delta^4}{12} + \cdots.$$

Therefore

$$\left(r^{-r} e^{r-1}\right)^{\mu} = \exp\Big(\mu\big(-\tfrac{\delta^2}{2} + \tfrac{\delta^3}{6} + \cdots\big)\Big).$$

Two convenient one-sided simplifications (using $|\delta| \le 1$) are:

- If $\delta \le 0$ ($r \le 1$, left tail), the cubic term is ≤ 0, so $\log(\cdot) \le -\frac{\delta^2}{2}$. Hence

$$\left(r^{-r}e^{r-1}\right)^{\mu} \le e^{-\mu\delta^2/2}.$$

- If $\delta \ge 0$ ($r \ge 1$, right tail), use $\frac{\delta^3}{6} \le \frac{\delta^2}{6}$ to get $-\frac{\delta^2}{2} + \frac{\delta^3}{6} \le -\frac{\delta^2}{3}$. Hence

$$\left(r^{-r}e^{r-1}\right)^{\mu} \le e^{-\mu\delta^2/3}.$$

These are the familiar "Gaussian-looking" tail fall-offs near the mean.

(c) Show that for large r, $\Pr(X \ge \mu r) \le 2^{-\mu r}$

We want

$$\left(r^{-r}e^{r-1}\right)^{\mu} \;\le\; 2^{-\mu r}.$$

Equivalently (cancel $\mu > 0$), $r^{-r}e^{r-1} \le 2^{-r}$, or

$$\left(\frac{e^{1-1/r}}{r}\right)^{r} \le 2^{-r} \quad \Longleftrightarrow \quad \frac{e^{1-1/r}}{r} \le \frac{1}{2}.$$

Define $h(r) = r^{-1}e^{1-1/r}$. Then

$$\frac{d}{dr}\ln h(r) = \frac{1}{r^2} - \frac{1}{r} = \frac{1-r}{r^2} < 0 \quad (r > 1),$$

so h strictly decreases from $h(1) = 1$ to 0 as $r \to \infty$. Numerically, $h(2) = e^{1/2}/2 \approx 0.824 < 0.825$, and the equation $h(r) = 1/2$ is solved near $r \approx 4.32$. Thus for sufficiently large r (e.g., $r \ge 4.32$) we have $h(r) \le \frac{1}{2}$, hence $\Pr(X \ge \mu r) \le 2^{-\mu r}$.

23 [HM23]

Let X have the negative binomial distribution whose probability generating function (pgf) is

$$G(z) = (q - pz)^{-n}, \qquad \text{with } q = p + 1.$$

Estimate tail probabilities such as $\Pr(X \ge r)$ and $\Pr(X \le r)$. We'll use the pgf tail bounds from §1.2.10:

$$\Pr(X \ge r) \le \frac{G(x)}{x^r} \quad (x \ge 1), \qquad \Pr(X \le r) \le \frac{G(x)}{x^r} \quad (0 < x \le 1). \text{ (Markov/pgf)}$$

These are (24)–(25) in the text.

1) Set up the Chernoff/pgf bound and optimize

For any $x \in (0, q/p)$,

$$\frac{G(x)}{x^r} = (q - px)^{-n}\, x^{-r}.$$

Let

$$\phi(x) := \ln\Big((q - px)^{-n} x^{-r}\Big) = -n \ln(q - px) - r \ln x.$$

Differentiate and set to zero to minimize:

$$\phi'(x) = \frac{np}{q - px} - \frac{r}{x} = 0 \quad \Longrightarrow \quad x^\star = \frac{rq}{p(n + r)}.$$

(One checks $x^\star \in (0, 1]$ when $r \le np$ and $x^\star \ge 1$ when $r \ge np$, so the correct inequality—lower or upper tail—uses the same optimizer.) Substituting:

$$q - px^\star = q - \frac{rq}{n + r} = \frac{qn}{n + r}.$$

Hence the minimized bound is

$$\boxed{\min_x \frac{G(x)}{x^r} = \Big(\frac{n + r}{qn}\Big)^n \Big(\frac{p(n + r)}{rq}\Big)^r}$$

and therefore

$$\Pr(X \ge r) \;\le\; \Big(\frac{n + r}{qn}\Big)^n \Big(\frac{p(n + r)}{rq}\Big)^r \quad (r \ge np), \qquad \Pr(X \le r) \;\le\; \Big(\frac{n + r}{qn}\Big)^n \Big(\frac{p(n + r)}{rq}\Big)^r \quad (r \le np).$$

(These follow directly from (24)–(25) with the minimizing $x^\star$.)

2) Put the bound in "deviation from the mean" form

From the pgf, $\mathbb{E}X = \dfrac{np}{q-p}$. With $q = p+1$, this simplifies to $\mu = \mathbb{E}X = np$; and $\mathrm{Var}(X) = \dfrac{npq}{(q-p)^2} = npq$. (These follow from the standard "semi-invariant" relations in §1.2.10 and a quick derivative check at $z = 1$.)

Write $r = \mu(1+\delta) = np(1+\delta)$ for the upper tail ($\delta \geq 0$). Then

$$\Pr(X \geq np(1+\delta)) \;\leq\; \Big(\frac{q+p\delta}{q}\Big)^n \Big(\frac{q+p\delta}{q(1+\delta)}\Big)^{np(1+\delta)}.$$

Similarly for the lower tail with $r = np(1-\delta)$ ($0 \leq \delta \leq 1$):

$$\Pr(X \leq np(1-\delta)) \;\leq\; \Big(\frac{q-p\delta}{q}\Big)^n \Big(\frac{q-p\delta}{q(1-\delta)}\Big)^{np(1-\delta)}.$$

(These are just the closed-form versions of the boxed bound with $q = p+1$.)

3) Gaussian-type approximation (small deviations)

For small δ we can expand the log of the upper-tail bound. A Taylor expansion around $\delta = 0$ with $q = p+1$ gives

$$\ln(\text{bound}) = -\frac{p}{2(p+1)}\,\delta^2\, n \;+\; O(\delta^3 n).$$

Since $\mu = np$ and $\mathrm{Var}(X) = npq = np(p+1)$, this says

$$\boxed{\Pr(X \geq \mu + s) \;\lesssim\; \exp\Big(-\frac{s^2}{2\,\mathrm{Var}(X)}\Big) \quad \text{for small } s \text{ (and likewise for } X \leq \mu - s).}$$

So, near the mean, the negative binomial tails are (sub-Gaussian) with variance scale npq. This is the negative-binomial analogue of the Chernoff/Hoeffding behavior obtained earlier for the binomial case via (24)–(25).

4) Summary

- Using the pgf tail inequalities (24)–(25), the optimal choice $x^\star = \dfrac{rq}{p(n+r)}$ yields the explicit bound

$$\Pr(X \geq r) \text{ or } \Pr(X \leq r) \;\leq\; \Big(\frac{n+r}{qn}\Big)^n \Big(\frac{p(n+r)}{rq}\Big)^r.$$

- With $q = p + 1$, mean $= np$, variance $= npq$, and for small deviations s from the mean,

$$\Pr(|X - \mu| \geq s) \;\lesssim\; 2\,\exp\Big(-\frac{s^2}{2npq}\Big).$$

These estimates are exactly what Exercise 23 asks for when the distribution is generated by $(q - pz)^{-n}$ with $q = p + 1$, using the same pgf-Markov machinery as in (24)–(25).

1.2.11 Asymptotic representations

1 [HM20]

Base case $n = 0$

When $n = 0$ the formula reads

$$f(x) = f(0) + \int_0^x f'(x-t)\,dt.$$

Make the change of variables $u = x - t$. Then $du = -dt$ and when $t = 0$ we have $u = x$, when $t = x$ we have $u = 0$. Thus

$$\int_0^x f'(x-t)\,dt = \int_x^0 f'(u)\,(-du) = \int_0^x f'(u)\,du = f(x) - f(0),$$

by the Fundamental Theorem of Calculus. Hence the base case holds.

Inductive step

Assume the statement holds for some $n - 1 \geq 0$; we prove it for n.

Write the remainder term

$$R_n(x) \;:=\; \int_0^x \frac{t^n}{n!}\, f^{(n+1)}(x-t)\,dt.$$

Integrate by parts with

$$u = \frac{t^n}{n!}, \quad dv = f^{(n+1)}(x-t)\,dt \qquad \Rightarrow \qquad du = \frac{t^{n-1}}{(n-1)!}\,dt, \;\; v = -\,f^{(n)}(x-t).$$

Then

$$R_n(x) = \Big[-\frac{t^n}{n!}\, f^{(n)}(x-t)\Big]_{t=0}^{t=x} + \int_0^x \frac{t^{n-1}}{(n-1)!}\, f^{(n)}(x-t)\,dt.$$

The boundary term simplifies to

$$-\frac{x^n}{n!}\, f^{(n)}(0) \quad \text{(the } t=0 \text{ endpoint is 0)},$$

so

$$R_n(x) = -\frac{x^n}{n!}\, f^{(n)}(0) \;+\; \int_0^x \frac{t^{\,n-1}}{(n-1)!}\, f^{(n)}(x-t)\, dt = -\frac{x^n}{n!}\, f^{(n)}(0) + R_{n-1}(x).$$

Now start from the target formula for n:

$$f(x) \stackrel{?}{=} \sum_{k=0}^{n} \frac{f^{(k)}(0)}{k!}\, x^k + R_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(0)}{k!}\, x^k - \frac{x^n}{n!}\, f^{(n)}(0) + R_{n-1}(x) = \sum_{k=0}^{n-1} \frac{f^{(k)}(0)}{k!}\, x^k + R_{n-1}(x).$$

But by the induction hypothesis (the statement for $n-1$), the right–hand side equals $f(x)$. Therefore the statement for n holds.

By induction on n, the formula is true for all integers $n \geq 0$.

2 [HM20]

Show that for fixed $a > 0$,

$$\gamma(a,x) = \sum_{k\geq 0} \frac{(-1)^k\, x^{k+a}}{k!\,(k+a)} \quad \text{(hence } \frac{x^a}{a} - \frac{x^{a+1}}{a+1} + \frac{x^{a+2}}{2!(a+2)} - \cdots\text{)}.$$

Derivation (step by step)

1. Start from the definition (Knuth's Eq. (6)):

$$\gamma(a,x) = \int_0^x e^{-t}\, t^{\,a-1}\, dt, \qquad a > 0.$$

The condition $a > 0$ ensures the integrand is integrable at $t = 0$.

2. Expand the exponential. For all real t,

$$e^{-t} = \sum_{k=0}^{\infty} \frac{(-1)^k t^k}{k!}.$$

3. Multiply and justify termwise integration. On the compact interval $t \in [0, x]$ (with fixed $x \geq 0$), the series for e^{-t} converges uniformly and absolutely, and t^{a-1} is continuous (since $a > 0$). By the Weierstrass M-test (or standard uniform-convergence results), we may interchange sum and integral:

$$\gamma(a, x) = \int_0^x \left(\sum_{k=0}^{\infty} \frac{(-1)^k t^k}{k!} \right) t^{a-1} dt = \sum_{k=0}^{\infty} \frac{(-1)^k}{k!} \int_0^x t^{k+a-1} dt.$$

4. Integrate each term.

$$\int_0^x t^{k+a-1} dt = \frac{x^{k+a}}{k+a} \qquad (a > 0).$$

5. Assemble the series. Therefore,

$$\gamma(a, x) = \sum_{k=0}^{\infty} \frac{(-1)^k \, x^{k+a}}{k! \, (k+a)} = \frac{x^a}{a} - \frac{x^{a+1}}{a+1} + \frac{x^{a+2}}{2!(a+2)} - \cdots.$$

This is exactly Knuth's Eq. (7), the Maclaurin expansion in powers of x for the incomplete gamma function. (The exercise statement "Obtain Eq. (7) from Eq. (6)" appears on the same page as the exercise list for §1.2.11.3.)

3 [M20]

The exercise asks you to get Eq. (8) from Eq. (7). Here are those two formulas as they appear in the text (with $a > 0$):

$$\gamma(a, x) = \int_0^x e^{-t} t^{a-1} \, dt = \sum_{k \geq 0} \frac{(-1)^k \, x^{k+a}}{k! \, (k+a)}, \qquad \text{(Eq. (7)) and}$$

$$e^x \gamma(a, x) = \sum_{k \geq 0} \frac{x^{k+a}}{a(a+1) \cdots (a+k)}. \qquad \text{(Eq. (8))}$$

Below are two self-contained derivations.

Method A (Cauchy product + a binomial/Beta identity)

1. Multiply Eq. (7) by the series for $e^x = \sum_{m \geq 0} \dfrac{x^m}{m!}$ and use the Cauchy product:

$$e^x\gamma(a,x) = \Big(\sum_{k\ge 0} \frac{(-1)^k\, x^{k+a}}{k!\,(k+a)}\Big)\Big(\sum_{m\ge 0} \frac{x^m}{m!}\Big) = \sum_{n\ge 0}\Big(\sum_{k=0}^{n} \frac{(-1)^k}{k!\,(n-k)!\,(k+a)}\Big)\, x^{n+a}.$$

So the coefficient of x^{n+a} is

$$c_n = \sum_{k=0}^{n} \frac{(-1)^k}{k!\,(n-k)!\,(k+a)}.$$

2. Evaluate c_n. Use the binomial expansion inside an integral (a standard Beta-function trick):

$$\int_0^1 t^{a-1}(1-t)^n\, dt = \sum_{k=0}^{n} \binom{n}{k} (-1)^k \int_0^1 t^{a+k-1}\, dt = \sum_{k=0}^{n} \binom{n}{k} \frac{(-1)^k}{a+k}.$$

But the LHS is $B(a,n+1) = \dfrac{\Gamma(a)\Gamma(n+1)}{\Gamma(a+n+1)} = \dfrac{n!}{a(a+1)\cdots(a+n)}$. Therefore

$$\sum_{k=0}^{n} \binom{n}{k} \frac{(-1)^k}{a+k} = \frac{n!}{a(a+1)\cdots(a+n)}.$$

Divide both sides by $n!$ to get

$$\sum_{k=0}^{n} \frac{(-1)^k}{k!\,(n-k)!\,(a+k)} = \frac{1}{a(a+1)\cdots(a+n)}.$$

Hence $c_n = \dfrac{1}{a(a+1)\cdots(a+n)}$, which gives

$$e^x\gamma(a,x) = \sum_{n\ge 0} \frac{x^{n+a}}{a(a+1)\cdots(a+n)},$$

i.e. Eq. (8).

Method B (one-line integral evaluation)

Start directly from the definition and pull the e^x inside:

$$e^x\gamma(a,x) = \int_0^x e^{\,x-t}\, t^{a-1}\, dt.$$

With $t = xu$ (so $dt = x\, du$) this becomes

$$e^x\gamma(a,x) = x^a \int_0^1 e^{x(1-u)} u^{a-1}\,du = x^a \sum_{m\geq 0} \frac{x^m}{m!} \int_0^1 (1-u)^m u^{a-1}\,du.$$

The integral is $B(a, m+1) = \dfrac{\Gamma(a)\Gamma(m+1)}{\Gamma(a+m+1)} = \dfrac{m!}{a(a+1)\cdots(a+m)}$, so the $m!$ cancels:

$$e^x\gamma(a,x) = \sum_{m\geq 0} \frac{x^{a+m}}{a(a+1)\cdots(a+m)},$$

which is Eq. (8).

4 [HM10]

1. Monotonic change of variables and lower bound on u. From the setup we have $v = v(u) = u - \ln(1+u)$ for $u \geq 0$. Then

$$\frac{dv}{du} = 1 - \frac{1}{1+u} = \frac{u}{1+u} > 0 \quad (u > 0),$$

so $v(u)$ is strictly increasing on $[0, \infty)$, with $v(0) = 0$ and $v(u) \to \infty$ as $u \to \infty$. Hence for any fixed $r > 0$ there is a unique $u_r > 0$ such that

$$v(u_r) = u_r - \ln(1+u_r) = r,$$

and for all $v \geq r$ we have $u \geq u_r$.

2. Uniform bound on the factor $1 + \frac{1}{u}$. Because $u \geq u_r > 0$ whenever $v \geq r$, the multiplier in the integrand is bounded:

$$1 + \frac{1}{u} \;\leq\; 1 + \frac{1}{u_r} \;=:\; C_r,$$

where C_r depends only on the chosen r and not on x or v.

3. Domination by a simple exponential tail. Therefore,

$$\int_r^\infty x\,e^{-xv}\Big(1+\frac{1}{u}\Big)\,dv \;\le\; C_r\int_r^\infty x\,e^{-xv}\,dv \;=\; C_r\big[-e^{-xv}\big]_{v=r}^{\infty} \;=\; C_r\,e^{-rx}.$$

Since C_r is a constant for fixed r, this is exactly the statement

$$\int_r^\infty x\,e^{-xv}\Big(1+\frac{1}{u}\Big)\,dv \;=\; O(e^{-rx}) \quad (x\to\infty).$$

This bound justifies the claim in Eq. (13) that the portion of the integral from $v=r$ to ∞ is exponentially small in x, so all asymptotic contributions come from the neighborhood of $v=0$, letting Knuth restrict attention to $[0,r]$ for any fixed small $r>0$.

5 [HM24]

Let

$$\sum_{k=0}^{n} k^{\,n+\frac{1}{2}}\,e^{-k} = \int_0^n x^{\,n+\frac{1}{2}}e^{-x}\,dx + \tfrac{1}{2}\,n^{\,n+\frac{1}{2}}e^{-n} + \tfrac{1}{24}\,n^{\,n-\frac{1}{2}}e^{-n} - R,$$

which is the Euler–Maclaurin/Euler summation expansion used in §1.2.11.3. Show that the remainder satisfies

$$R = O(n^{\,n}e^{-n})\,.$$

Let $f(x)=x^{\,n+\frac{1}{2}}e^{-x}$ for $x\in[0,n]$. A standard form of Euler's summation formula with one Bernoulli correction gives

$$\sum_{k=0}^{n} f(k) = \int_0^n f(x)\,dx + \frac{f(n)}{2} + \frac{f'(n)-f'(0)}{12} - R,$$

with

$$R = \int_0^n \frac{B_2(\{x\})}{2!}\,f''(x)\,dx,$$

where $B_2(u)=u^2-u+\frac{1}{6}$ and $|B_2(\{x\})|\le\frac{1}{12}$. Hence

$$|R| \le \frac{1}{12}\int_0^n |f''(x)|\,dx. \qquad (\;)$$

We now bound $\int_0^n |f''|$. Compute derivatives via $g(x) = \ln f(x) = (n + \frac{1}{2}) \ln x - x$:

$$f'(x) = f(x)\, g'(x), \qquad g'(x) = \frac{n + \frac{1}{2}}{x} - 1,$$

$$f''(x) = f(x)\, ((g'(x))^2 + g''(x)), \qquad g''(x) = -\frac{n + \frac{1}{2}}{x^2}.$$

Thus

$$|f''(x)| \le f(x) \left(\left| \tfrac{n+\frac{1}{2}}{x} - 1 \right|^2 + \frac{n + \frac{1}{2}}{x^2} \right). \tag{1}$$

Make the change $x = nt$ on $[0, 1]$. Then

$$f(nt) = n^{\,n+\frac{1}{2}}\, t^{\,n+\frac{1}{2}}\, e^{-nt}, \quad dx = n\, dt,$$

and

$$\left| \tfrac{n+\frac{1}{2}}{nt} - 1 \right|^2 = \left| \tfrac{1}{t} - 1 + \tfrac{1}{2nt} \right|^2 \le C_1 (1-t)^2 + \frac{C_2}{n^2}, \qquad \frac{n + \frac{1}{2}}{(nt)^2} \le \frac{C_3}{n},$$

for constants C_i independent of n and $t \in (0, 1]$. Plugging these into (1) and integrating gives

$$\int_0^n |f''(x)|\, dx \;\le\; n^{\,n+\frac{3}{2}} \int_0^1 t^{\,n+\frac{1}{2}} e^{-nt} \left(C_1 (1-t)^2 + \frac{C_4}{n} \right) dt. \tag{2}$$

Next, use the convex function $\phi(t) = t - \ln t - 1$ on $0 < t \le 1$. Since $\phi(1) = \phi'(1) = 0$ and $\phi''(t) = 1/t^2 \ge 1$, we have

$$\phi(t) \ge \frac{(1-t)^2}{2} \quad (0 < t \le 1).$$

Also $t^{\,n+\frac{1}{2}} e^{-nt} = e^{-n}\, t^{1/2}\, e^{-n\phi(t)} \le e^{-n} e^{-n(1-t)^2/2}$. Therefore both integrals below are standard Gaussian tails:

$$\int_0^1 e^{-n(1-t)^2/2}\, dt \asymp n^{-1/2}, \qquad \int_0^1 (1-t)^2 e^{-n(1-t)^2/2}\, dt \asymp n^{-3/2}.$$

Applying these estimates to (2) yields

$$\int_0^n |f''(x)|\,dx \;\le\; n^{n+\frac{3}{2}}\,e^{-n}\Big(C_1\cdot n^{-3/2}+C_4\cdot\frac{1}{n}\cdot n^{-1/2}\Big)=O(n^n e^{-n})\,.$$

Finally, ( ) implies $|R| = O(n^n e^{-n})$, which is what we had to prove. This is exactly the bound quoted just above Eq. (22) in the text.

6 [HM20]

We need to prove: For fixed real α, β and $n \to \infty$,

$$(n+\alpha)^{n+\beta} = n^{n+\beta}\,e^{\alpha}\Big(1+\alpha\Big(\beta-\frac{\alpha}{2}\Big)\frac{1}{n}+O\Big(\frac{1}{n^2}\Big)\Big)\,.$$

We expand logarithms to second order and then re-exponentiate.

1. Take logs and split terms

$$\begin{aligned}\ln\big((n+\alpha)^{n+\beta}\big) &= (n+\beta)\,\ln(n+\alpha)\\ &= (n+\beta)\Big(\ln n+\ln\big(1+\tfrac{\alpha}{n}\big)\Big).\end{aligned}$$

2. Series for $\ln(1+z)$ at $z=\alpha/n$

$$\ln\Big(1+\frac{\alpha}{n}\Big)=\frac{\alpha}{n}-\frac{\alpha^2}{2n^2}+O\Big(\frac{1}{n^3}\Big)\,.$$

3. Multiply by $n+\beta$

$$\begin{aligned}(n+\beta)\ln\Big(1+\frac{\alpha}{n}\Big) &= (n+\beta)\left(\frac{\alpha}{n}-\frac{\alpha^2}{2n^2}+O(n^{-3})\right)\\ &= \alpha+\frac{\alpha\beta-\alpha^2/2}{n}+O\Big(\frac{1}{n^2}\Big)\,.\end{aligned}$$

4. Assemble the logarithm

$$\ln\big((n+\alpha)^{n+\beta}\big)=(n+\beta)\ln n\;+\;\alpha\;+\;\frac{\alpha\beta-\alpha^2/2}{n}\;+\;O\Big(\frac{1}{n^2}\Big)\,.$$

5. Exponentiate Write the small correction as $\varepsilon_n = \dfrac{\alpha\beta-\alpha^2/2}{n}+O\Big(\dfrac{1}{n^2}\Big)$. Then

$$
\begin{aligned}
(n+\alpha)^{n+\beta} &= \exp\Big((n+\beta)\ln n + \alpha + \varepsilon_n\Big) \\
&= n^{n+\beta}\, e^{\alpha}\, \exp(\varepsilon_n) \\
&= n^{n+\beta}\, e^{\alpha}\, (1 + \varepsilon_n + O(\varepsilon_n^2)) \\
&= n^{n+\beta}\, e^{\alpha} \left(1 + \frac{\alpha\beta - \alpha^2/2}{n} + O\Big(\frac{1}{n^2}\Big)\right).
\end{aligned}
$$

6. Tidy the coefficient

$$
\frac{\alpha\beta - \alpha^2/2}{n} = \alpha\Big(\beta - \frac{\alpha}{2}\Big)\frac{1}{n}.
$$

Thus

$$
(n+\alpha)^{n+\beta} = n^{n+\beta}\, e^{\alpha}\left(1 + \alpha\Big(\beta - \frac{\alpha}{2}\Big)\frac{1}{n} + O\Big(\frac{1}{n^2}\Big)\right),
$$

which is Eq. (23).

7 [HM30]

Write the integrand as a single exponential and expand it uniformly for $0 \le u \le y x^{1/4}$:

$$
e^{-u}\Big(1 + \frac{u}{x}\Big)^x = \exp\Big(-u + x\ln(1+u/x)\Big) = \exp\Big(-\frac{u^2}{2x} + \frac{u^3}{3x^2} - \frac{u^4}{4x^3} + \frac{u^5}{5x^4} + O\big(\tfrac{u^6}{x^5}\big)\Big).
$$

The first two terms of this are exactly what Knuth uses to reach eq. (16); we simply carry the expansion two more orders because the upper limit now grows with x. (The uniformity is fine since $u/x \le y\, x^{-3/4} \to 0$ on the whole interval.)

Now expand the outer exponential up to terms contributing $\ge x^{-2}$ after integration. Let

$$
E = -\frac{u^2}{2x} + \frac{u^3}{3x^2} - \frac{u^4}{4x^3} + \frac{u^5}{5x^4} + O\Big(\frac{u^6}{x^5}\Big).
$$

Then

$$
\exp(E) = 1 + E + \frac{E^2}{2} + \frac{E^3}{6} + \frac{E^4}{24} + O\Big(\frac{u^9}{x^5}\Big),
$$

and collecting by powers of $1/x$ (keeping only terms that will matter up to order x^{-2} once integrated to $u \sim x^{1/4}$) gives

$$\exp(E) = 1 - \frac{u^2}{2x} + \frac{1}{x^2}\Big(\frac{u^3}{3} + \frac{u^4}{8}\Big) - \frac{1}{x^3}\Big(\frac{u^4}{4} + \frac{u^5}{6} + \frac{u^6}{48}\Big) + \frac{1}{x^4}\Big(\frac{13}{72}u^6 + \frac{1}{24}u^7 + \frac{1}{384}u^8\Big) \;+\; O\Big(\frac{u^9}{x^5}\Big).$$

Term-by-term integration

Let $U = y\,x^{1/4}$. Using

$$\int_0^U u^k\,du = \frac{U^{k+1}}{k+1}, \qquad U^{k+1} = y^{k+1}\,x^{(k+1)/4},$$

we integrate each line:

1. Leading term $\displaystyle\int_0^U 1\,du = U = y\,x^{1/4}$.
2. Order x^{-1} term $\displaystyle -\frac{1}{2x}\int_0^U u^2\,du = -\frac{U^3}{6x} = -\frac{y^3}{6}\,x^{-1/4}$.
3. Order x^{-2} group

$$\frac{1}{x^2}\int_0^U\Big(\frac{u^3}{3} + \frac{u^4}{8}\Big)\,du = \frac{U^4}{12x^2} + \frac{U^5}{40x^2} = \frac{y^4}{12}\,x^{-1} + \frac{y^5}{40}\,x^{-3/4}.$$

4. Order x^{-3} group

$$-\frac{1}{x^3}\int_0^U\Big(\frac{u^4}{4} + \frac{u^5}{6} + \frac{u^6}{48}\Big)\,du = -\frac{U^5}{20x^3} - \frac{U^6}{36x^3} - \frac{U^7}{336x^3}.$$

In powers of x: $-\dfrac{y^5}{20}x^{-7/4} - \dfrac{y^6}{36}x^{-3/2} - \dfrac{y^7}{336}x^{-5/4}$.

5. Order x^{-4} terms that still contribute $\geq x^{-2}$ after integration

$$\frac{1}{x^4}\int_0^U \Big(\frac{13}{72}u^6 + \frac{1}{24}u^7 + \frac{1}{384}u^8\Big)\,du = \frac{13}{72}\frac{U^7}{7x^4} + \frac{1}{24}\frac{U^8}{8x^4} + \frac{1}{384}\frac{U^9}{9x^4}.$$

Among these, $U^7/x^4 \sim x^{-9/4}$ (smaller than x^{-2}, can be absorbed into the remainder), while

$$\frac{U^8}{192x^4} = \frac{y^8}{192}\,x^{-2} \quad \text{and} \quad \frac{U^9}{3456x^4} = \frac{y^9}{3456}\,x^{-7/4}$$

both meet or exceed the x^{-2} threshold and must be kept.

Putting everything together, an asymptotic expansion for $J(x)$ as $x \to \infty$ (with fixed y) is

$$\boxed{\begin{aligned} J(x) = y\,x^{1/4} - \frac{y^3}{6}\,x^{-1/4} + \frac{y^5}{40}\,x^{-3/4} + \frac{y^4}{12}\,x^{-1} - \frac{y^7}{336}\,x^{-5/4} - \frac{y^6}{36}\,x^{-3/2} \\ + \Big(-\frac{y^5}{20} + \frac{y^9}{3456}\Big)\,x^{-7/4} + \frac{y^8}{192}\,x^{-2} \;+\; O\big(x^{-9/4}\big). \end{aligned}}$$

8 [HM30]

Set

$$E(x,u) \;:=\; -u + x\ln\Big(1 + \frac{u}{x}\Big).$$

Then the integrand is $e^{E(x,u)}$. For $|z| < 1$,

$$\ln(1+z) = \sum_{k=1}^{m}(-1)^{k+1}\frac{z^k}{k} + \rho_{m+1}(z),$$

with a remainder satisfying $|\rho_{m+1}(z)| \le C|z|^{m+1}$ when $|z| \le \frac{1}{2}$ (take $C = 2$, say). Here $z = u/x$. Because $u \le f(x) = O(x^r)$ with $r < 1$, we have $z = O(x^{r-1}) \to 0$; hence for all sufficiently large x and all $0 \le u \le f(x)$ we indeed have $|z| \le \frac{1}{2}$.

Therefore

$$\begin{aligned} E(x,u) &= -u + x\left[\sum_{k=1}^{m}(-1)^{k+1}\frac{(u/x)^k}{k} + \rho_{m+1}(u/x)\right] \\ &= \sum_{k=2}^{m}(-1)^k\frac{u^k}{k\,x^{k-1}} \;+\; \Delta(x,u), \end{aligned}$$

where the truncated exponent

$$P_m(x,u) := \sum_{k=2}^{m} (-1)^k \frac{u^k}{k\,x^{k-1}}$$

matches the one in the exercise, and the remainder term satisfies

$$|\Delta(x,u)| \;=\; x\,|\rho_{m+1}(u/x)| \;\le\; C\,\frac{u^{m+1}}{x^m}.$$

Comparing integrands

Write

$$e^{E(x,u)} - e^{P_m(x,u)} = e^{P_m(x,u)}\big(e^{\Delta(x,u)} - 1\big).$$

For large x and $0 \le u \le f(x)$ we have $|\Delta(x,u)| \le 1$, hence $|e^{\Delta} - 1| \le 2|\Delta|$. Also $E(x,u) \le 0$ (since $\ln(1+t) \le t$), so $e^{E(x,u)} \le 1$; this gives a harmless uniform bound on e^{P_m} in our range. Consequently,

$$\big|e^{E(x,u)} - e^{P_m(x,u)}\big| \;\ll\; \frac{u^{m+1}}{x^m} \quad (0 \le u \le f(x)).$$

Integrating the error

Integrate this bound from 0 to $f(x)$:

$$\left|\int_0^{f(x)} e^{E(x,u)}\,du - \int_0^{f(x)} e^{P_m(x,u)}\,du\right| \;\ll\; \int_0^{f(x)} \frac{u^{m+1}}{x^m}\,du \;=\; \frac{f(x)^{m+2}}{(m+2)\,x^m}.$$

Using $f(x) = O(x^r)$,

$$\frac{f(x)^{m+2}}{x^m} \;=\; O\!\left(\frac{x^{r(m+2)}}{x^m}\right) \;=\; O\big(x^{\,rm+2r-m}\big) \;=\; O\Big(x^{-[\,m(1-r)-2r\,]}\Big).$$

We want this to be $O(x^{-s})$, i.e.

$$m(1-r) - 2r \;\ge\; s \quad\Longleftrightarrow\quad m \;\ge\; \frac{s+2r}{1-r}.$$

Choosing any integer m with $m \ge \lceil (s+2r)/(1-r) \rceil$ yields the desired $O(x^{-s})$ error term, exactly as required.

Tricomi's special case

If $f(x) = O(\sqrt{x})$, then $r = \frac{1}{2}$. With $s = 0$ the condition gives $m \geq \lceil (0+2\cdot\frac{1}{2})/(1-\frac{1}{2}) \rceil = \lceil 2 \rceil = 2$. Thus

$$\int_0^{f(x)} e^{-u}\Big(1+\tfrac{u}{x}\Big)^x du = \int_0^{f(x)} \exp\Big(-\tfrac{u^2}{2x}\Big)\, du + O(1).$$

Substitute $t = u/\sqrt{2x}$:

$$\int_0^{f(x)} \exp\Big(-\tfrac{u^2}{2x}\Big)\, du = \sqrt{2x}\int_0^{f(x)/\sqrt{2x}} e^{-t^2}\, dt,$$

giving precisely the printed corollary.

9 [HM36]

This is the "incomplete gamma at a moving point" problem:

$$R_x(p) \;:=\; \frac{\gamma(x+1,\, px)}{\Gamma(x+1)} \qquad (x \to \infty,\ p \in \mathbb{R}).$$

Write $\gamma(x+1, px) = \int_0^{px} t^x e^{-t}\, dt$. The integrand $f_x(t) = t^x e^{-t} = e^{x \ln t - t}$ has a single maximum at $t = x$. Hence the large-x behavior depends on how the upper limit px sits relative to the maximizer x. There are three natural regimes.

Define the Cramér (large-deviation) rate function for the gamma/Poisson family

$$I(p) := p - 1 - \ln p \quad (p > 0), \qquad I(1) = 0,$$

which satisfies $I(p) > 0$ for $p \neq 1$ and $I(1) = 0$.

1) $0 < p < 1$ (lower tail: exponentially small)

Here the integration stops **before** the peak. A standard endpoint form of Laplace's method (with $\phi(t) = \ln t - \frac{t}{x}$, $\psi \equiv 1$) gives, evaluated at the endpoint $a = px$,

$$\int_0^{px} t^x e^{-t}\, dt \sim e^{x\phi(a)}\left[\frac{1}{x\phi'(a)} - \frac{\phi''(a)}{x^2\phi'(a)^3} + O(x^{-3})\right],$$

with $\phi'(a) = \frac{1-p}{px}$, $\phi''(a) = -\frac{1}{p^2x^2}$, and $x\phi(a) = x\ln(px) - px$. Divide by Stirling's $\Gamma(x+1) \sim \sqrt{2\pi x}\, x^x e^{-x}$ to obtain

$$\boxed{R_x(p) \sim \frac{e^{-xI(p)}}{\sqrt{2\pi x}} \frac{p}{1-p} \left(1 + \frac{1}{(1-p)^2\, x} + O(x^{-2})\right), \qquad 0 < p < 1.}$$

Thus $R_x(p)$ decays like $e^{-xI(p)}$ with the indicated two-term prefactor.

2) $p > 1$ (upper tail: $1 - R_x(p)$ is exponentially small)

Now the integral $\Gamma(x+1) - \gamma(x+1, px) = \int_{px}^{\infty} t^x e^{-t} dt$ starts **after** the peak. The same endpoint analysis (with a sign flip from $\phi'(a) < 0$) yields

$$\Gamma(x+1) - \gamma(x+1, px) \sim e^{x\phi(a)} \left[\frac{1}{x|\phi'(a)|} - \frac{\phi''(a)}{x^2|\phi'(a)|^3} + O(x^{-3})\right].$$

Hence

$$\boxed{1 - R_x(p) \sim \frac{e^{-xI(p)}}{\sqrt{2\pi x}} \frac{p}{p-1} \left(1 - \frac{1}{(p-1)^2\, x} + O(x^{-2})\right), \qquad p > 1.}$$

Consequently $R_x(p) \to 1$ with the same large-deviation exponent $I(p)$.

3) $p = 1$ (central window: CLT/Edgeworth scale)

At $p = 1$ we are integrating up to x, which lies one standard deviation unit $z = -(x+1-x)/\sqrt{x+1} = -1/\sqrt{x+1}$ **below** the mean $x+1$. A normal approximation with the first Edgeworth correction for a Gamma$(x+1, 1)$ (skewness $2/\sqrt{x+1}$) gives

$$R_x(1) = \Pr\{Y \le x\} = \Phi\left(-\frac{1}{\sqrt{x+1}}\right) + \frac{1}{6} \frac{2}{\sqrt{x+1}} (1 - z^2)\, \varphi(z) + O(x^{-1}),$$

where Φ, φ are the standard normal CDF/PDF. Expanding at $z \approx 0$ yields

$$\boxed{R_x(1) = \frac{1}{2} - \frac{1}{\sqrt{2\pi x}} + \frac{1}{3\sqrt{2\pi x}} + O(x^{-1}) = \frac{1}{2} - \frac{2}{3} \frac{1}{\sqrt{2\pi x}} + O(x^{-1}).}$$

In words: at $p = 1$, the ratio is just below $\frac{1}{2}$, with a $x^{-1/2}$ correction.

4) $p \le 0$ (note on the negative case)

The problem allows $p < 0$ with integer x so that t^x is defined for $t < 0$. Writing $t = -u$ gives $\gamma(x+1, px) = -(-1)^x \int_0^{|p|x} u^x e^u \, du$, whose magnitude is dominated by the endpoint $u = |p|x$, growing like $\sim \dfrac{(|p|x)^x e^{|p|x}}{|1 - x/u|}$.

Dividing by $\Gamma(x+1) \sim \sqrt{2\pi x}\, x^x e^{-x}$ shows $|R_x(p)|$ grows super-exponentially,

$$|R_x(p)| \asymp \frac{1}{\sqrt{2\pi x}} \left(|p| e^{\,|p|+1}\right)^x \quad \to \infty \qquad (p < 0),$$

so there is no finite limit; the ratio blows up in magnitude (with sign $(-1)^{x+1}$).

Summary (two leading terms in each regime)

- $0 < p < 1$:

$$\frac{\gamma(x+1, px)}{\Gamma(x+1)} \sim \frac{e^{-xI(p)}}{\sqrt{2\pi x}} \frac{p}{1-p} \left(1 + \frac{1}{(1-p)^2 x}\right).$$

- $p > 1$:

$$\frac{\gamma(x+1, px)}{\Gamma(x+1)} \sim 1 - \frac{e^{-xI(p)}}{\sqrt{2\pi x}} \frac{p}{p-1} \left(1 - \frac{1}{(p-1)^2 x}\right).$$

- $p = 1$:

$$\frac{\gamma(x+1, x)}{\Gamma(x+1)} = \frac{1}{2} - \frac{2}{3} \frac{1}{\sqrt{2\pi x}} + O(x^{-1}).$$

- $p < 0$: magnitude $\to \infty$ like above (oscillating in sign with x).

These come directly from endpoint Laplace asymptotics (for $p \neq 1$) and a one-term Edgeworth correction at the central point $p = 1$.

10 [HM34]

Let

$$a = x + 1, \quad z_0 = px, \quad \Delta = \frac{p\,y}{p-1},$$

so the quantity in question is

$$I(x) \;=\; \gamma(a,\, z_0 + \Delta) - \gamma(a,\, z_0) \;=\; \int_{z_0}^{z_0+\Delta} e^{-t}\, t^{a-1}\, dt \;=\; \int_{z_0}^{z_0+\Delta} e^{-t}\, t^{x}\, dt.$$

Change variables $t = z_0 + s$ (so $dt = ds$, limits $s \in [0, \Delta]$):

$$I(x) \;=\; e^{-z_0} z_0^x \int_0^{\Delta} e^{-s} \Big(1 + \frac{s}{z_0}\Big)^x\, ds.$$

Because $z_0 = px \to \infty$ while $\Delta = O(1)$, expand the logarithm:

$$\Big(1 + \frac{s}{px}\Big)^x = \exp\Big[x \ln\Big(1 + \frac{s}{px}\Big)\Big] = \exp\Big(\frac{s}{p} - \frac{s^2}{2p^2 x} + O(x^{-2})\Big).$$

Hence the integrand becomes

$$e^{-s}\Big(1 + \frac{s}{px}\Big)^x = \exp\Big(-\alpha s\Big)\Big(1 - \frac{s^2}{2p^2 x} + O(x^{-2})\Big), \qquad \alpha := 1 - \frac{1}{p} = \frac{p-1}{p}.$$

Therefore

$$I(x) = e^{-px}(px)^x \left[\int_0^{\Delta} e^{-\alpha s}\, ds - \frac{1}{2p^2 x}\int_0^{\Delta} s^2 e^{-\alpha s}\, ds + O(x^{-2})\right].$$

The two elementary integrals (for any real $\alpha \neq 0$) are

$$\int_0^{\Delta} e^{-\alpha s}\, ds = \frac{1 - e^{-\alpha\Delta}}{\alpha}, \quad \int_0^{\Delta} s^2 e^{-\alpha s}\, ds = \frac{2}{\alpha^3} - e^{-\alpha\Delta}\left(\frac{\Delta^2}{\alpha} + \frac{2\Delta}{\alpha^2} + \frac{2}{\alpha^3}\right).$$

With $\alpha = \frac{p-1}{p}$ and $\Delta = \frac{py}{p-1}$ we have the key simplification $\alpha\Delta = y$. Also

$$\frac{1}{\alpha} = \frac{p}{p-1}, \quad \frac{1}{\alpha^2} = \frac{p^2}{(p-1)^2}, \quad \frac{1}{\alpha^3} = \frac{p^3}{(p-1)^3}, \quad \frac{\Delta^2}{\alpha} = \frac{p^3 y^2}{(p-1)^3}, \quad \frac{2\Delta}{\alpha^2} = \frac{2p^3 y}{(p-1)^3}.$$

Plugging in:

$$\int_0^{\Delta} e^{-\alpha s}\,ds = \frac{p}{p-1}(1-e^{-y}),$$

$$\int_0^{\Delta} s^2 e^{-\alpha s}\,ds = \frac{p^3}{(p-1)^3}\Big(2 - e^{-y}(y^2+2y+2)\Big).$$

Finally,

$$\boxed{\gamma(x+1,\,px+\tfrac{py}{p-1}) - \gamma(x+1,\,px) = e^{-px}(px)^x\left[\frac{p}{p-1}(1-e^{-y}) + \frac{p}{2(p-1)^3x}\Big(e^{-y}(y^2+2y+2)-2\Big) + O(x^{-2})\right.}$$

This gives two terms (leading plus $1/x$) before the $O(x^{-2})$, matching the requested order "as in the previous exercise." (For context, the chapter's treatment of the incomplete gamma and asymptotics appears immediately above; see also Theorem A in §1.2.11.3.)

(Optional) Normalized form

If you prefer it relative to $\Gamma(x+1)$, Stirling's formula yields

$$\frac{e^{-px}(px)^x}{\Gamma(x+1)} = \frac{1}{\sqrt{2\pi x}}\big(p\,e^{1-p}\big)^x\Big(1+O(x^{-1})\Big),$$

so

$$\frac{\gamma(x+1,\,px+\tfrac{py}{p-1}) - \gamma(x+1,\,px)}{\Gamma(x+1)} = \frac{(p\,e^{1-p})^x}{\sqrt{2\pi x}}\left[\frac{p}{p-1}(1-e^{-y}) + \frac{p}{2(p-1)^3x}\Big(e^{-y}(y^2+2y+2)-2\Big) + O(x^{-2})\right]$$

(As usual for $p \neq 1$, this is exponentially small in x.)

11 [HM35]

1) Definitions and exact closed forms

From the section, the "un-parametrized" sums are

$$Q(n) = \sum_{k=1}^{n} \frac{n!}{(n-k)!\,n^k} \quad \text{and} \quad R(n) = \sum_{k\geq 0} \frac{n!\,n^k}{(n+k)!}.$$

They satisfy $Q(n) + R(n) = \dfrac{n!\,e^{\,n}}{n^n}$ and their asymptotics are derived on the preceding pages.

The exercise introduces a weight x^k. Write

$$Q_x(n) := \sum_{k=0}^{n} \frac{n!}{(n-k)!\,n^k}\,x^k, \qquad R_x(n) := \sum_{k\geq 0} \frac{n!\,n^k}{(n+k)!}\,x^k.$$

Both admit clean incomplete-gamma forms (use the Beta–Gamma identities and the standard expansions used in the text for $x = 1$):

- For $R_x(n)$, write $n!/(n+k)! = \dfrac{1}{(n+1)_k} = \dfrac{1}{\Gamma(n+k+1)/\Gamma(n+1)}$, use $\mathrm{B}(n+1,k) = \dfrac{n!(k-1)!}{(n+k)!} = \int_0^1 t^n(1-t)^{k-1}\,dt$, interchange sum and integral, and sum the resulting exponential series. One gets

$$\boxed{R_x(n) = 1 + \frac{e^{nx}}{(nx)^n}\,\gamma(n+1, nx)} \qquad (n \geq 1),$$

where γ is the lower incomplete gamma.

- For $Q_x(n)$, reindex with $m = n - k$ and use $\Gamma(n+1, y) = \int_y^\infty t^n e^{-t} dt$ (upper incomplete gamma):

$$\boxed{Q_x(n) = \frac{e^{n/x}}{(n/x)^n}\,\Gamma(n+1, n/x)}.$$

These exact forms reduce to the section's $x = 1$ case when you set $x = 1$ and use $\gamma + \Gamma = \Gamma(n+1) = n!$ together with Stirling and the development around Eqs. (24)–(26).

For asymptotics it is convenient to introduce

$$\eta(u) = u - 1 - \ln u \quad (> 0 \text{ for } u \neq 1).$$

2) Asymptotics when $0 < x < 1$ (endpoint/Laplace at $t = 1$)

For $x < 1$, both series are dominated by small k. Intuitively the term ratios tend to x, so both sums approach the geometric series $1/(1-x)$. We can make the leading correction explicit.

- For $Q_x(n)$: use the falling-factorial expansion

$$\frac{(n)_k}{n^k} = 1 - \frac{k(k-1)}{2n} + O\left(\frac{k^3}{n^2}\right) \quad (k \ll n),$$

then sum termwise for $|x| < 1$:

$$Q_x(n) = \sum_{k\geq 0} \left[1 - \frac{k(k-1)}{2n} + O\left(\frac{k^3}{n^2}\right)\right] x^k = \frac{1}{1-x} - \frac{x^2}{(1-x)^3}\frac{1}{n} + O\left(\frac{1}{n^2}\right).$$

- For $R_x(n)$: use $\prod_{j=1}^{k} \frac{n}{n+j} = 1 - \dfrac{k(k+1)}{2n} + O(\frac{k^3}{n^2})$ and sum:

$$R_x(n) = \sum_{k\geq 0} \left[1 - \frac{k(k+1)}{2n} + O\left(\frac{k^3}{n^2}\right)\right] x^k = \frac{1}{1-x} - \frac{x}{(1-x)^3}\frac{1}{n} + O\left(\frac{1}{n^2}\right).$$

So, uniformly for fixed $x \in (0, 1)$,

$$\boxed{\begin{aligned} Q_x(n) &= \frac{1}{1-x} - \frac{x^2}{(1-x)^3}\frac{1}{n} + O(n^{-2}), \\ R_x(n) &= \frac{1}{1-x} - \frac{x}{(1-x)^3}\frac{1}{n} + O(n^{-2}). \end{aligned}}$$

(A complementary derivation comes from the integral $R_x(n) - 1 = nx \int_0^1 t^n e^{nx(1-t)} dt$ and an endpoint Laplace expansion at $t = 1$.)

3) Asymptotics when $x > 1$ (interior saddle)

For $x > 1$, the maximal contribution is interior and the sums grow exponentially with n. A discrete Laplace/saddle-point analysis on the k-sums (or standard large-n expansions of incomplete gamma with argument nx / n/x) yields:

$$\boxed{\begin{aligned} R_x(n) &\sim \sqrt{2\pi n}\ \exp(n\,\eta(x)) \left(1 + O(n^{-1})\right), \\ Q_x(n) &\sim \sqrt{2\pi n}\ \exp(n\,\eta(1/x)) \left(1 + O(n^{-1})\right). \end{aligned}}$$

Sketch (for R_x): write the k-term via Stirling,

$$T_k \approx \frac{1}{\sqrt{1+k/n}} \exp\Big(n\Phi(u)\Big), \quad u = k/n, \quad \Phi(u) = -(1+u)\ln(1+u) + u(1+\ln x),$$

whose stationary point is at $u^* = x-1$ with $\Phi(u^*) = \eta(x)$ and $\Phi''(u^*) = -1/x$. Discrete Laplace gives the prefactor $\frac{1}{\sqrt{1+u^*}}\sqrt{\frac{2\pi n}{|\Phi''(u^*)|}} = \sqrt{2\pi n}$. A completely analogous computation for Q_x has $u^* = 1-1/x$ and main exponent $\eta(1/x)$.

Non-uniformity at $x=1$. These "$x \neq 1$" expansions are not uniform as $x \to 1$. At $x=1$ the interior saddle coalesces with the endpoint, and the section gives the correct matched expansions (for the original $Q(n), R(n)$):

$$Q(n) = \sqrt{\frac{\pi n}{2}} - \frac{1}{3} + \frac{1}{12}\sqrt{\frac{\pi}{2n}} - \frac{4}{135n} + \frac{1}{288}\sqrt{\frac{\pi}{2n^3}} + O(n^{-2}),$$
$$R(n) = \sqrt{\frac{\pi n}{2}} + \frac{1}{3} + \frac{1}{12}\sqrt{\frac{\pi}{2n}} + \frac{4}{135n} + \frac{1}{288}\sqrt{\frac{\pi}{2n^3}} + O(n^{-2}),$$

whose derivation appears just above the exercise.

4) Quick consistency checks

- For $x<1$: $Q_x(n), R_x(n) \to (1-x)^{-1}$, matching the intuition that each term is $\approx x^k$.
- For $x>1$: $\eta(x) > \eta(1/x) > 0$, so $R_x(n)$ dominates $Q_x(n)$ exponentially—again consistent with the shapes of the terms.
- Setting $x=1$ is singular

12 [HM20]

Use the substitution $u = t^2/2$.

- Then $t = \sqrt{2u}$ and $dt = \dfrac{du}{\sqrt{2u}}$.
- The limits map as $t: 0 \to x \;\Rightarrow\; u: 0 \to x^2/2$.

Therefore

$$\int_0^x e^{-t^2/2}\,dt = \int_0^{x^2/2} e^{-u}\,\frac{du}{\sqrt{2u}} = \frac{1}{\sqrt{2}}\int_0^{x^2/2} e^{-u}\,u^{-1/2}\,du = \frac{1}{\sqrt{2}}\,\gamma\Big(\tfrac{1}{2},\,\tfrac{x^2}{2}\Big).$$

Comparing with $b\,\gamma(a,y)$, we can read off

$$\boxed{a = \tfrac{1}{2}, \qquad y = \tfrac{x^2}{2}, \qquad b = \tfrac{1}{\sqrt{2}}}.$$

Quick check (optional)

Using $\gamma(\frac{1}{2}, z) = \sqrt{\pi}\,\mathrm{erf}(\sqrt{z})$, the result becomes

$$\int_0^x e^{-t^2/2}\,dt = \frac{1}{\sqrt{2}}\sqrt{\pi}\,\mathrm{erf}\Big(\frac{x}{\sqrt{2}}\Big) = \sqrt{\frac{\pi}{2}}\,\mathrm{erf}\Big(\frac{x}{\sqrt{2}}\Big),$$

which is the familiar normal-integral form, confirming the constants are correct.

13 [HM42]

14 [HM39]

1) Re-center at the peak (part a)

Write $k = n - t$ (so $t = 0, 1, \dots, n$). Then

$$k^{n+\alpha}e^{-k} = (n-t)^{n+\alpha}e^{-(n-t)} = n^{n+\alpha}e^{-n}\,\exp\Big((n+\alpha)\ln(1-\tfrac{t}{n}) + t\Big).$$

Using $\ln(1-x) = -x - \frac{x^2}{2} - \frac{x^3}{3} - \cdots$, the exponent becomes

$$t - (n+\alpha)\Big(\frac{t}{n} + \frac{t^2}{2n^2} + \frac{t^3}{3n^3} + \frac{t^4}{4n^4} + \cdots\Big) = -\frac{\alpha t}{n} - \frac{t^2}{2n} - \frac{t^3}{3n^2} - \frac{t^4}{4n^3} - \cdots.$$

Hence

$$S_{n,\alpha} = n^{n+\alpha}e^{-n}\sum_{t=0}^{n}\exp\Big(-\frac{t^2}{2n}\Big)\cdot f(t,n),$$

with

$$f(t,n) = \Big(1 - \frac{t}{n}\Big)^{\alpha}\exp\Big(-\frac{t^3}{3n^2} - \frac{t^4}{4n^3} - \cdots\Big).$$

Renaming t back to k gives exactly the form asked for in (a).

2) Local power series for the slowly varying factor (part b)

For $k \le n^{1/2+\varepsilon}$ (any fixed $\varepsilon > 0$), we may expand $f(k,n)$ as a double series in k and $1/n$. To the orders we will actually need,

$$\Big(1-\frac{k}{n}\Big)^{\alpha} = 1 - \frac{\alpha k}{n} + \frac{\alpha(\alpha-1)}{2}\frac{k^2}{n^2} + O\Big(\frac{k^3}{n^3}\Big),$$

$$\exp\Big(-\frac{k^3}{3n^2} - \frac{k^4}{4n^3} + \frac{1}{2}\frac{k^6}{9n^4} + \cdots\Big) = 1 - \frac{k^3}{3n^2} - \frac{k^4}{4n^3} + \frac{1}{18}\frac{k^6}{n^4} + O\Big(\frac{k^5}{n^4}\Big).$$

Multiplying these gives $f(k,n)$ as a finite polynomial in k divided by powers of n, plus a remainder that (for our chosen cutoff $k \le n^{1/2+\varepsilon}$) is $o(n^{-1/2})$ after summation. Contributions from $k > n^{1/2+\varepsilon}$ are exponentially small because of the Gaussian factor $e^{-k^2/(2n)}$. (This is the standard Laplace method localization.)

3) Gaussian "moments" you will need (part d)

Let

$$M_r(n) \;=\; \sum_{k\ge 0} k^{\,r}\, e^{-k^2/(2n)}.$$

Euler's summation (or Poisson summation) gives the classical leading terms

$$\begin{aligned}
M_0(n) &= \int_0^\infty e^{-x^2/(2n)}dx \;+\; \frac{1}{2} \;+\; O(e^{-cn}) = \sqrt{\frac{\pi n}{2}} + \frac{1}{2} + O(e^{-cn}),\\
M_1(n) &= \int_0^\infty x\, e^{-x^2/(2n)}dx \;+\; O(e^{-cn}) = n + O(e^{-cn}),\\
M_2(n) &= \int_0^\infty x^2 e^{-x^2/(2n)}dx \;+\; O(e^{-cn}) = \sqrt{\frac{\pi}{2}}\, n^{3/2} + O(e^{-cn}),\\
M_3(n) &= \int_0^\infty x^3 e^{-x^2/(2n)}dx \;+\; O(e^{-cn}) = 2n^2 + O(e^{-cn}),\\
M_4(n) &= \int_0^\infty x^4 e^{-x^2/(2n)}dx \;+\; O(e^{-cn}) = 3\sqrt{\frac{\pi}{2}}\, n^{5/2} + O(e^{-cn}),\\
M_6(n) &= \int_0^\infty x^6 e^{-x^2/(2n)}dx \;+\; O(e^{-cn}) = 15\sqrt{\frac{\pi}{2}}\, n^{7/2} + O(e^{-cn}).
\end{aligned}$$

(Only these are needed through the $n^{-1/2}$ term.)

4) Collect terms up to $O(n^{-1/2})$

Insert the expansion of $f(k,n)$ into $\sum e^{-k^2/(2n)} f(k,n)$ and use the $M_r(n)$ above.

- Leading term (order $n^{1/2}$):

$$1 \cdot M_0(n) \;=\; \sqrt{\frac{\pi n}{2}} \;+\; \frac{1}{2} \;+\cdots$$

(contributes $\sqrt{\pi n/2}$ and a $+\frac{1}{2}$ constant).

- Constant term from $-(\alpha/n)\sum k e^{-k^2/(2n)}$:

$$-\frac{\alpha}{n}\, M_1(n) \;=\; -\alpha \;+\; o(1).$$

- Constant term from the cubic in the exponent:

$$-\frac{1}{3n^2}\sum k^3 e^{-k^2/(2n)} \;=\; -\frac{1}{3n^2}\, M_3(n) \;=\; -\frac{1}{3n^2}\cdot 2n^2 \;=\; -\frac{2}{3}.$$

Together with $+\frac{1}{2}$ from M_0, the "purely n-independent" part becomes $\frac{1}{2}-\frac{2}{3}-\alpha = -\frac{1}{6}-\alpha$.

- The $n^{-1/2}$ term (the coefficient of $\sqrt{\pi/(2n)}$) receives four contributions:

  (i) from $\dfrac{\alpha(\alpha-1)}{2}\dfrac{k^2}{n^2}$:

$$\frac{\alpha(\alpha-1)}{2n^2}\, M_2(n) = \frac{\alpha(\alpha-1)}{2}\sqrt{\frac{\pi}{2n}}.$$

  (ii) from the quartic $-\dfrac{k^4}{4n^3}$:

$$-\frac{1}{4n^3}\, M_4(n) = -\frac{1}{4n^3}\cdot 3\sqrt{\frac{\pi}{2}}\, n^{5/2} = -\frac{3}{4}\sqrt{\frac{\pi}{2n}}.$$

  (iii) from the cross term $\left(-\frac{\alpha k}{n}\right)\left(-\frac{k^3}{3n^2}\right)$:

$$+\frac{\alpha}{3n^3}\, M_4(n) = \alpha\cdot\frac{1}{3n^3}\cdot 3\sqrt{\frac{\pi}{2}}\, n^{5/2} = \alpha\sqrt{\frac{\pi}{2n}}.$$

  (iv) from $\frac{1}{2}\left(\frac{k^3}{3n^2}\right)^2 = \frac{1}{18}\frac{k^6}{n^4}$:

$$\frac{1}{18n^4}\,M_6(n) = \frac{1}{18n^4}\cdot 15\sqrt{\frac{\pi}{2}}\,n^{7/2} = \frac{5}{6}\sqrt{\frac{\pi}{2n}}.$$

Summing (i)–(iv) gives

$$\left[\frac{\alpha(\alpha-1)}{2} - \frac{3}{4} + \alpha + \frac{5}{6}\right]\sqrt{\frac{\pi}{2n}} = \left(\frac{1}{12} + \frac{\alpha}{2} + \frac{\alpha^2}{2}\right)\sqrt{\frac{\pi}{2n}}.$$

5) Final asymptotic (part e)

Multiplying the bracket by the common prefactor $n^{n+\alpha}e^{-n}$ yields

$$\boxed{\sum_{k=0}^{n} k^{n+\alpha}e^{-k} \;=\; n^{n+\alpha}e^{-n}\left[\sqrt{\frac{\pi n}{2}} \;-\; \left(\frac{1}{6}+\alpha\right) \;+\; \left(\frac{1}{12}+\frac{\alpha}{2}+\frac{\alpha^2}{2}\right)\sqrt{\frac{\pi}{2n}} \;+\; O\!\left(\frac{1}{n}\right)\right].}$$

This matches the expansion the exercise asks you to obtain; the same page notes that, in principle, one can continue to any $O(n^{-r})$ by pushing the series for $f(k,n)$ and the Gaussian moments further.

15 [HM20]

1. Expand the power by the binomial theorem.

$$\left(1+\frac{z}{n}\right)^n = \sum_{k=0}^{n}\binom{n}{k}\frac{z^k}{n^k}.$$

2. Integrate termwise (finite sum, so interchange is justified) and use $\displaystyle\int_0^\infty z^k e^{-z}\,dz = \Gamma(k+1) = k!$:

$$\begin{aligned} I(n) &= \int_0^\infty e^{-z}\sum_{k=0}^{n}\binom{n}{k}\frac{z^k}{n^k}\,dz = \sum_{k=0}^{n}\binom{n}{k}\frac{1}{n^k}\int_0^\infty z^k e^{-z}\,dz \\ &= \sum_{k=0}^{n}\binom{n}{k}\frac{k!}{n^k} = \sum_{k=0}^{n}\frac{n!}{(n-k)!\,n^k}, \end{aligned}$$

because $\binom{n}{k}k! = \dfrac{n!}{(n-k)!}$.

3. Identify $Q(n)$. The $k = 0$ term of the last sum is

$$\frac{n!}{n!\,n^0} = 1,$$

and the remaining terms (for $k = 1, \dots, n$) are exactly the definition of $Q(n)$. Therefore

$$\boxed{I(n) = 1 + Q(n)} \quad \text{equivalently} \quad \boxed{Q(n) = \int_0^\infty \left(1 + \frac{z}{n}\right)^n e^{-z}\,dz - 1.}$$

This is precisely the "relation" the exercise asks you to show.

Quick checks & context

- For $n = 1$: $I(1) = \int_0^\infty (1+z)e^{-z}dz = 1 + 1 = 2$, while $Q(1) = 1$. Indeed $I(1) = 1 + Q(1)$.
- Knuth later derives the asymptotic $Q(n) = \sqrt{\pi n/2} - \frac{1}{3} + \cdots$, so this integral is $\sqrt{\pi n/2} + \frac{2}{3} + \cdots$, consistent with the same section's expansions.

16 [M24]

1. Definition of $Q(\cdot)$. Knuth defines

$$Q(n) = \sum_{j=1}^{n} \frac{n!}{(n-j)!\,n^j}.$$

Equivalently,

$$Q(k) = \frac{k!}{k^k} \sum_{s=0}^{k-1} \frac{k^s}{s!}.$$

(Just put $s = k - j$.)

2. A very handy integral representation (from Ex. 15): expanding $(1 + z/k)^k = \sum_{j=0}^{k} \binom{k}{j} (z/k)^j$ and integrating term-wise gives

$$\int_0^\infty \Big(1+\frac{z}{k}\Big)^k e^{-z}\,dz = \sum_{j=0}^{k} \binom{k}{j} \frac{j!}{k^j} = 1 + Q(k),$$

hence

$$Q(k) = \int_0^\infty \Big[\Big(1+\frac{z}{k}\Big)^k - 1\Big] e^{-z}\,dz.$$

Let

$$S_n := \sum_{k=0}^{n} (-1)^k \binom{n}{k} k^{n-1} Q(k).$$

Insert the integral form of $Q(k)$ and swap sum/integral (finite sum):

$$S_n = \int_0^\infty e^{-z} \sum_{k=0}^{n} (-1)^k \binom{n}{k} k^{n-1} \Big[\Big(1+\frac{z}{k}\Big)^k - 1\Big]\,dz.$$

The "-1" part vanishes because $\sum_{k=0}^{n} (-1)^k \binom{n}{k} k^{n-1} = 0$ (it's the nth forward difference of a degree-$n-1$ polynomial). Thus

$$S_n = \int_0^\infty e^{-z} \sum_{k=0}^{n} (-1)^k \binom{n}{k} k^{n-1} \Big(1+\frac{z}{k}\Big)^k\,dz.$$

Expand $\big(1+\frac{z}{k}\big)^k = \sum_{j=0}^{k} \binom{k}{j} (z/k)^j$ and use $\binom{n}{k}\binom{k}{j} = \binom{n}{j}\binom{n-j}{k-j}$:

$$\sum_{k=0}^{n} (-1)^k \binom{n}{k} k^{n-1} \Big(1+\frac{z}{k}\Big)^k = \sum_{j=0}^{n} \binom{n}{j} z^j \sum_{k=j}^{n} (-1)^k \binom{n-j}{k-j} k^{n-1-j}.$$

Let $N = n - j$ and $m = k - j$. The inner sum becomes

$$(-1)^j \sum_{m=0}^{N} (-1)^m \binom{N}{m} (m+j)^{N-1}.$$

This is an alternating binomial sum of a polynomial in m of degree $N-1$, so it is 0 when $N \geq 1$ (standard "nth difference kills degree $< n$"). The only surviving case is $N = 0$ (i.e. $j = n$), where the inner sum reduces to $(-1)^n \cdot \binom{0}{0} \cdot n^{-1} = (-1)^n/n$. Therefore

$$\sum_{k=0}^{n}(-1)^k \binom{n}{k} k^{n-1}\Big(1+\frac{z}{k}\Big)^k = \frac{(-1)^n}{n}\, z^n.$$

Integrating,

$$S_n = \frac{(-1)^n}{n}\int_0^\infty e^{-z} z^n\, dz = \frac{(-1)^n}{n}\,\Gamma(n+1) = (-1)^n\,(n-1)!,$$

which is exactly the desired identity.

17 [HM29]

Step 1: Reindex and insert Stirling

Put $m = n + k$ so $k = m - n$ and

$$S(n) = \frac{1}{(n-1)!}\sum_{m=n}^{\infty}\frac{(m-1)!}{m^{\,m-n}}.$$

Use Stirling for $(m-1)! = \Gamma(m)$:

$$(m-1)! \sim \sqrt{2\pi}\, m^{\,m-\frac{1}{2}} e^{-m}\Big(1+\frac{1}{12m}+\frac{1}{288m^2}+\cdots\Big).$$

After dividing by $m^{\,m-n}$ this gives

$$S(n) \sim \frac{\sqrt{2\pi}}{(n-1)!}\sum_{m=n}^{\infty} m^{\,n-\frac{1}{2}} e^{-m}\Big(1+\tfrac{1}{12m}+\tfrac{1}{288m^2}+\cdots\Big).$$

Define $A_r(n) = \sum_{m=n}^{\infty} m^{\,n-\frac{1}{2}-r} e^{-m}$. Then

$$S(n) \sim \frac{\sqrt{2\pi}}{(n-1)!}\Big(A_0(n)+\tfrac{1}{12}A_1(n)+\tfrac{1}{288}A_2(n)+\cdots\Big).$$

Step 2: Replace the sums by incomplete-gamma terms

For large n, Euler's summation + the standard passage from sums to integrals gives

$$A_r(n) = \Gamma\big(n+\tfrac{1}{2}-r,\, n\big) + (\text{lower-order endpoint corrections}).$$

Here $\Gamma(a,x) = \int_x^\infty t^{a-1}e^{-t}\,dt$ is the upper incomplete gamma function.

Knuth proves a uniform expansion for the *lower* incomplete gamma ratio

$$\frac{\gamma(x+1,x+y)}{\Gamma(x+1)} = \frac{1}{2} + \frac{y-\frac{2}{3}}{\sqrt{2\pi}}\,x^{-1/2} + \frac{1}{\sqrt{2\pi}}\left(\frac{23}{270} - \frac{y}{12} - \frac{y^3}{6}\right)x^{-3/2} + O(x^{-5/2}),$$

valid for fixed y and large x. Hence the *upper* ratio is 1−that. We need $x = n - \frac{1}{2} - r$ and $y = \frac{1}{2}$ (because our lower limit of summation is $m = n$). Substituting these into the formula, and then multiplying by $\Gamma(x+1) = \Gamma(n+\frac{1}{2}-r)$ and by the prefactor $\sqrt{2\pi}/(n-1)!$, yields an expansion for each $A_r(n)$, hence for $S(n)$. (See the derivation and Theorem A in the text.)

Finally, use Stirling again for $(n-1)!$ to express everything in powers of $n^{1/2}, n^{-1/2}, n^{-1}, n^{-3/2}, \ldots$ (this is exactly the same bookkeeping Knuth used just above for $P(n), Q(n), R(n)$ in Eqs. (24)–(26)).

Carrying the algebra through to the same order as those formulas gives

$$\boxed{S(n) = \sqrt{\frac{\pi n}{2}} + \frac{2}{3} - \frac{1}{24}\sqrt{\frac{\pi}{2n}} - \frac{4}{135}\frac{1}{n} + \frac{49}{1152}\sqrt{\frac{\pi}{2n^3}} + O(n^{-2})}.$$

18 [M25]

Define

$$S_n(x,y) = \sum_{k=0}^{n}\binom{n}{k}(x+k)^k\,(y+n-k)^{n-k}.$$

Step 1 — split $(x+k)^k$: $(x+k)^k = x(x+k)^{k-1} + k(x+k)^{k-1}$. Therefore

$$S_n(x,y) = x\sum_{k=0}^{n}\binom{n}{k}(x+k)^{k-1}(y+n-k)^{n-k} + \sum_{k=0}^{n}\binom{n}{k}k(x+k)^{k-1}(y+n-k)^{n-k}.$$

Step 2 — Abel's binomial identity: Abel's formula (TAOCP Eq. 1.2.6–(16) with $z = 1$) gives

$$\sum_{k=0}^{n}\binom{n}{k}(x+k)^{k-1}(y+n-k)^{n-k} = \frac{(x+y+n)^n}{x}.$$

Thus the first sum equals $(x+y+n)^n$. For the second sum use $k\binom{n}{k} = n\binom{n-1}{k-1}$ and reindex $j = k-1$ to get

$$\sum_{k=0}^{n} \binom{n}{k} k(\cdots) = n \sum_{j=0}^{n-1} \binom{n-1}{j} (x+1+j)^j (y+n-1-j)^{n-1-j} = n\, S_{n-1}(x+1, y).$$

Therefore we have the recurrence

$$S_n(x,y) = (x+y+n)^n + n\, S_{n-1}(x+1, y). \quad (*)$$

(Abel reference and this reduction are exactly how Knuth dispatches this exercise.)

Step 3 — solve $(*)$ by induction: One easily verifies (start at $n=0$) that the closed form

$$\boxed{S_n(x,y) = \sum_{k=0}^{n} \binom{n}{k} k!\,(x+y+n)^{\,n-k}}$$

satisfies the recurrence: the $k=0$ term gives $(x+y+n)^n$, and the $k \geq 1$ coefficients match because $n\binom{n-1}{k-1}(k-1)! = \binom{n}{k}k!$.

Plug in the two required cases

1) $S_1(n)$: take $x=0,\ y=0$

$$S_1(n) = \sum_{k=0}^{n} \binom{n}{k} k!\, n^{\,n-k} = n^{\,n} \sum_{k=0}^{n} \frac{n!}{(n-k)!\, n^k} = n^{\,n}(1+Q(n)).$$

(The $k=0$ term contributes the "1".)

2) $S_2(n)$: take $x=1,\ y=0$

$$S_2(n) = \sum_{k=0}^{n} \binom{n}{k} k!\,(n+1)^{\,n-k} = (n+1)^{\,n} \sum_{k=0}^{n} \frac{n!}{(n-k)!\,(n+1)^k}.$$

But

$$\sum_{k=0}^{n} \frac{n!}{(n-k)!\,(n+1)^k} = \sum_{k=1}^{n+1} \frac{(n+1)!}{((n+1)-k)!\,(n+1)^k} = Q(n+1),$$

since $\sum_{k=1}^{n+1} \frac{(n+1)!}{(n+1-k)!}\, t^k = (n+1)t \sum_{k=0}^{n} \frac{n!}{(n-k)!}\, t^k$ (set $t = \frac{1}{n+1}$). Hence

$$S_2(n) = (n+1)^n Q(n+1).$$

Final results

$$\boxed{\sum_{k=0}^{n} \binom{n}{k} k^k (n-k)^{n-k} = n^n (1 + Q(n))}$$

$$\boxed{\sum_{k=0}^{n} \binom{n}{k} (k+1)^k (n-k)^{n-k} = (n+1)^n Q(n+1)}$$

Quick check (e.g., $n = 4$): Left sum $= 824$; right side $= 4^4(1 + Q(4)) = 256 \cdot 3.21875 = 824$. Left sum $= 1569$; right side $= 5^4 Q(5) = 625 \cdot 2.5104 = 1569$.

19 [HM30]

Split the integral at r:

$$C_n = \int_0^r e^{-nx} f(x)\, dx + \int_r^\infty e^{-nx} f(x)\, dx =: I_1(n) + I_2(n).$$

1) The small-x part $I_1(n)$

By the hypothesis, $|f(x)| \le K x^\alpha$ on $[0, r]$ for some $K > 0$. Hence

$$|I_1(n)| \le K \int_0^r e^{-nx} x^\alpha\, dx.$$

Make the change of variables $u = nx$ (so $x = u/n$, $dx = du/n$):

$$|I_1(n)| \le K \int_0^{nr} e^{-u} \left(\frac{u}{n}\right)^\alpha \frac{du}{n} = K n^{-1-\alpha} \int_0^{nr} e^{-u} u^\alpha\, du.$$

Because $\alpha > -1$, the truncated gamma integral is bounded by $\Gamma(\alpha + 1)$:

$$\int_0^{nr} e^{-u} u^\alpha\, du \le \Gamma(\alpha + 1).$$

Therefore

$$|I_1(n)| \le K\,\Gamma(\alpha+1)\,n^{-1-\alpha}.$$

2) The tail $I_2(n)$

For $x \ge r$, $e^{-nx} \le e^{-nr}$. Thus

$$|I_2(n)| \le e^{-nr} \int_r^\infty |f(x)|\,dx,$$

and the right-hand side is $O(e^{-nr})$, which is $o(n^{-1-\alpha})$ as $n \to \infty$.

3) Combine

Hence

$$C_n = I_1(n) + I_2(n) = O(n^{-1-\alpha}) + O(e^{-nr}) = O(n^{-1-\alpha})\,.$$

This proves the claim.

Remarks

- The condition $\alpha > -1$ is exactly what ensures integrability of x^α at 0 and is therefore optimal for this type of bound.
- A stronger (classic) version of Watson's lemma says that if $f(x)$ has a full expansion $f(x) \sim \sum_{m\ge 0} a_m x^{\alpha+m}$ as $x \downarrow 0$, then

$$\int_0^\infty e^{-nx} f(x)\,dx \sim \sum_{m\ge 0} a_m\,\Gamma(\alpha+m+1)\,n^{-(\alpha+m+1)}.$$

  Our exercise is the first-term big-O consequence of this.

20 [HM30]

1) Start from the integral form of $Q(n)$

From exercise 15 one gets

$$Q(n)+1 \;=\; \int_0^\infty \Big(1+\tfrac{z}{n}\Big)^n e^{-z}\,dz$$

(by expanding $(1+z/n)^n$, integrating term–by–term, and using $\int_0^\infty z^k e^{-z}\,dz = k!$).

Put $z = n\,u$. Then

$$Q(n)+1 \;=\; n\int_0^\infty \exp\big(-n\,[u-\ln(1+u)]\big)\,du \;=\; n\int_0^\infty e^{-n\,\psi(u)}\,du, \quad \psi(u) := u-\ln(1+u).$$

Near $u=0$,

$$\psi(u) = \frac{u^2}{2} - \frac{u^3}{3} + \frac{u^4}{4} - \cdots.$$

2) "Quadraticize" the exponent

Define w by

$$\frac{w^2}{2} = \psi(u) \qquad \Longleftrightarrow \qquad w = \sqrt{u^2 - \frac{2}{3}u^3 + \frac{2}{4}u^4 - \frac{2}{5}u^5 + \cdots}.$$

Then $w \sim u > 0$ for small u, and the change of variables $u \mapsto w$ is monotone near 0.

Let u be written as a power series in w:

$$u = c_1 w + c_2 w^2 + c_3 w^3 + c_4 w^4 + \cdots = w + \frac{1}{3}w^2 + \frac{1}{36}w^3 - \frac{1}{270}w^4 + \cdots,$$

which is exactly the series referenced in the statement ("as in (12)"). (You can get these c_k by expanding the square root and composing.)

Differentiate to relate differentials:

$$du = \Big(\sum_{k\ge 1} k\,c_k\,w^{k-1}\Big)\,dw.$$

Under this substitution,

$$Q(n)+1 = n\int_0^\infty e^{-n\,\psi(u)}\,du = n\int_0^\infty e^{-\frac{n}{2}w^2}\,\Big(\sum_{k\ge 1} k\,c_k\,w^{k-1}\Big)\,dw.$$

3) Apply Watson's lemma (Gaussian Laplace integrals)

Watson's lemma lets us expand a Laplace–type integral by inserting the Taylor series of the prefactor and integrating term by term. Here each term is

$$I_k(n) := n \int_0^\infty e^{-\frac{n}{2}w^2}\, w^{k-1}\, dw = \Gamma\Big(\frac{k}{2}\Big)\Big(\frac{n}{2}\Big)^{1-\frac{k}{2}},$$

obtained by the substitution $y = \frac{n}{2}w^2$.

Therefore, truncating after $m-1$ terms,

$$\boxed{Q(n) + 1 = \sum_{k=1}^{m-1} k\, c_k\, \Gamma\Big(\frac{k}{2}\Big)\Big(\frac{n}{2}\Big)^{1-\frac{k}{2}} \;+\; O\big(n^{1-\frac{m}{2}}\big)\,,}$$

which is exactly what the exercise asks you to show. (This is "the simplest possible" approach alluded to in the text for $Q(n)$.)

4) First few terms (consistency check)

Using $c_1 = 1,\ c_2 = \frac{1}{3},\ c_3 = \frac{1}{36},\ c_4 = -\frac{1}{270}$,

$$Q(n) + 1 = \underbrace{\Gamma(\tfrac{1}{2})\Big(\tfrac{n}{2}\Big)^{1/2}}_{\sqrt{\frac{\pi n}{2}}} + \underbrace{\tfrac{2}{3}\,\Gamma(1)\Big(\tfrac{n}{2}\Big)^{0}}_{\frac{2}{3}} + \underbrace{\tfrac{1}{12}\,\Gamma(\tfrac{3}{2})\Big(\tfrac{n}{2}\Big)^{-1/2}}_{\frac{1}{12}\sqrt{\frac{\pi}{2n}}} + \underbrace{\Big(-\frac{4}{270}\Big)\Gamma(2)\Big(\tfrac{n}{2}\Big)^{-1}}_{-\frac{4}{135n}} + \cdots.$$

Hence

$$Q(n) = \sqrt{\frac{\pi n}{2}} \;-\; \frac{1}{3} \;+\; \frac{1}{12}\sqrt{\frac{\pi}{2n}} \;-\; \frac{4}{135n} \;+\; \cdots,$$

agreeing with the book's asymptotic series for $Q(n)$.

1.3.1 Description of MIX

1 [00]

Answer: 4 trits.

Reasoning: A MIX byte can encode 64 distinct values (0–63). With base-3 digits ("trits"), we need the smallest n such that $3^n \geq 64$. $3^3 = 27 < 64$ and $3^4 = 81 \geq 64$, so $n = 4$ trits per byte (leaving $81 - 64 = 17$ codes unused).

2 [02]

Answer: 5 bytes.

Why: Each MIX byte holds 64 values. With n adjacent bytes we can represent up to $64^n - 1$. $64^4 = 16{,}777{,}216 < 99{,}999{,}999$ but $64^5 = 1{,}073{,}741{,}824 > 99{,}999{,}999$. So we need 5 bytes (i.e., one MIX word's magnitude; sign, if any, is separate).

3 [02]

a) Address field: (0:2) (includes the sign and bytes 1–2)
b) Index field: (3:3)
c) Field field: (4:4)
d) Operation code field: (5:5)

(Here 0 denotes the sign; bytes are numbered 1..5 left to right.)

4 [00]

Because the address field in a MIX instruction is a signed displacement. The effective address is computed as

$$\mathrm{EA} = A + I_4,$$

where $A = -2000$. If I_4 is large enough (e.g., $I_4 \geq 2000$), the sum EA is a valid memory location $0 \leq \text{EA} < 4000$. Thus `LDA -2000,4` means "load A from the word at address $I_4 - 2000$"; the program isn't using a negative memory address—just a negative offset.

5 [10]

Here's the full decoding of (6) into MIX's symbolic form.

Given numeric instruction (6) Word layout is `±AA | I | F | C`. The figure shows:

- sign = –
- AA = 80
- I = 3
- F = 5
- C = 4

1) Opcode → mnemonic In MIX, opcode C = 4 is DIV.
2) Field specification Field byte F encodes $(L : R)$ by $F = 8L + R$. With $F = 5$ we get $L = 0,\ R = 5 \Rightarrow (0 : 5)$ (the whole word).
3) Address and index The address field is the signed two-byte value ±AA = −80. Index byte I = 3 means "use index register X3".
4) Symbolic instruction (analogous to OP Z,I(F))

```
DIV  -80,3(0:5)
```

Since DIV's default field is (0:5), this is commonly written simply as:

```
DIV  -80,3
```

(Effective address at run time is $EA = X_3 + (-80)$.)

6 [10]

Given memory location 3000 holds the word

```
M[3000] = + | b1=5 | b2=1 | (b3:b4)=200 | b5=15
```

(read as sign plus; byte1=5, byte2=1, the two-byte field (3:4)=200, byte5=15).

- Load puts the selected field right-justified in the destination; unused left bytes → `0`.
- If the field includes the sign `(L=0)`, the loaded sign comes from memory; otherwise the sign is `+`.
- The `N` variant flips the sign after loading.
- Index registers `I1..I6` hold sign + 2 bytes.

Results:

a) LDAN 3000 Loads the whole word, then negates the sign → rA = − | 5 | 1 | 200 | 15 (fully defined).

b) LD2N 3000(3:4) Loads bytes 3–4 (no sign) into `I2`, then negates → rI2 = −200.

c) LDX 3000(1:3) Loads bytes 1–3 (no sign) into the right end of `X`; left two bytes become 0 → rX = + | 0 | 0 | 5 | 1 | ? (partially defined because byte 3 is unknown individually—only (3:4)=200 was given).

d) LD6 3000 (default `(0:2)`) Loads sign and bytes 1–2 into `I6` → two-byte value $64 \cdot 5 + 1 = 321$, sign `+` → rI6 = +321.

e) LDXN 3000(0:0) Loads just the sign into `X` (magnitude becomes 0), then negates → rX = − | 0 | 0 | 0 | 0 | 0 (negative zero).

7 [M15]

Let a MIX word base be $W = 64^5$.

Registers and operand (as signed integers):

- $A = s_A \cdot |A|$, $X = s_X \cdot |X|$ with $0 \leq |A|, |X| < W$.
- Memory operand $M = s_M \cdot |M|$ with $0 \leq |M| < W$.

Form the dividend from A and X by concatenation (sign from A only):

$$D \;=\; s_A \cdot (|A| \cdot W + |X|).$$

Let the divisor be M.

`DIV M` is defined without overflow iff

$$|M| \neq 0 \quad \text{and} \quad \left\lfloor \frac{|A| \cdot W + |X|}{|M|} \right\rfloor < W.$$

When defined, compute

$$Q \;=\; \left\lfloor \frac{|A| \cdot W + |X|}{|M|} \right\rfloor, \qquad R \;=\; (|A| \cdot W + |X|) \bmod |M|,$$

so $0 \leq R < |M|$.

The results placed in the registers are

$$\boxed{A \leftarrow (s_A s_M) \cdot Q}, \qquad \boxed{X \leftarrow s_A \cdot R}$$

(i.e., the quotient gets the product of the signs of A and M; the remainder gets the sign of A).

This covers all non-overflow cases; overflow occurs exactly when $|M| = 0$ or $Q \geq W$.

8 [15]

Context (from the book's last `DIV` example): Before `DIV 1000`, $rA = -0$, $rX = +1235\,0\,3\,1$, and $M[1000] = -\,0\,0\,0\,2\,0$ (i.e., divisor $|M| = 200$ on the decimal MIX). In that example the result was $Q = 6{,}175{,}001$, $R = 101$. (Stack Overflow)

Now change only "rX before" to $rX = -\,1234\,0\,3\,1$.

Key facts:

- `DIV` forms the dividend from (rA, rX) using the sign of A only; the sign of rX is ignored.
- Here $|A| = 0$, so the dividend magnitude is just $|X|$.
- Replacing $+\,1235\,0\,3\,1$ by $ -,1234,0,3,1$ reduces the magnitude by $100^3 = 1{,}000{,}000$. Since $1{,}000{,}000/200 = 5{,}000$, the quotient decreases by 5,000 and the remainder is unchanged.

Therefore

$$Q' = 6{,}175{,}001 - 5{,}000 = 6{,}170{,}001, \qquad R' = 101.$$

Signs:

- Quotient gets $\text{sign}(A) \times \text{sign}(M) = (-) \times (-) = +$.
- Remainder gets $\text{sign}(A) = -$.

Answer (byte form, decimal MIX):

- $rA \leftarrow + \mid 0 \mid 6 \mid 17 \mid 00 \mid 01$ (i.e., `+ | 0 | 617 | 00 | 01`)
- $rX \leftarrow - \mid 0 \mid 0 \mid 0 \mid 01 \mid 01$ (i.e., `- | 0 | 0 | 0 | 01 | 01`)

So with `rX before = - 1234 0 3 1`, you get A = + 0 617 00 01 and X = − 0 0 0 01 01.

9 [15]

Operators that can turn OV on

- ADD M — sets OV if the signed result won't fit in one MIX word (|result| too large).
- SUB M — same rule as ADD.
- DIV M — sets OV if division is impossible (M = 0) or if the quotient doesn't fit in a single word (|quotient| 10).
- INCA/DECA, INCX/DECX — these are "add/subtract an immediate value" on rA or rX; they can overflow just like ADD/SUB and therefore set OV.
- NUM — converts the ten bytes in rA‖rX to a number in rA; sets OV if the decimal value is too large to fit in one word (common when the 10-digit value exceeds the MIX word range).

  Notes • MUL never sets OV in MIX integer arithmetic because its 10-byte result is stored across rA‖rX. (Only floating-point multiply can affect OV, which we are excluding.) • Shift, load/store, compare, I/O, and most address-transfer ops do not affect OV.

Operators that can only clear OV

- JOV … and JNOV … — both perform the jump test and reset OV to "off" as part of their semantics.

That's all. In short: ADD, SUB, DIV, INCA/DECA/INCX/DECX, NUM can set overflow; JOV/JNOV clear it.

10 [15]

What the comparison indicator is

MIX keeps a three-way comparison indicator with states <, =, >. It changes only when an instruction explicitly performs a comparison; jumps merely *test* it.

Operators that can change it

- CMPA — compare rA with M (using the given field F).
- CMPX — compare rX with M (field F).
- CMP1, CMP2, CMP3, CMP4, CMP5, CMP6 — compare rI1…rI6 with the selected field of M (default F normally the address field (0:2) for index-register compares).
- FCMP — floating-point compare (if the floating-point option is present).

Notes/non-examples

- The JL/JE/JG/JGE/JNE/JLE family of jumps only *tests* the indicator; they don't change it.
- Arithmetic, loads/stores, shifts, I/O (IN/OUT/IOC/JBUS/JRED), conversion (NUM/CHAR), and register-enter/increment/decrement instructions do not alter the comparison indicator.

So, the list is: CMPA, CMPX, CMP1–CMP6, and (if FP is implemented) FCMP.

11 [15]

These MIX instructions can change rI1:

1. LD1 M(F) Loads rI1 from memory. The field specification F selects a field of M; when loading into an index register, MIX takes the address part (two bytes + sign) of the selected field. The result is therefore brought into the index range (−4095…+4095, with +0). Sign is the sign of the selected field.
2. LD1N M(F) Like LD1, but the sign of the loaded quantity is negated.
3. INC1 M Adds the effective address EA(M) to rI1. If the algebraic sum lies outside the index range, MIX reduces the magnitude modulo 4096 to bring it back into −4095…+4095 (zero is +0). (Overflow toggle is not used by index arithmetic.)
4. DEC1 M Subtracts EA(M) from rI1, with the same reduction-to-range rule as INC1.

5. ENT1 M "Enter" EA(M) in rI1 (i.e., rI1 ← EA(M)). Field F is ignored. Sign is the sign of EA(M).
6. ENN1 M Like ENT1, but rI1 ← −EA(M).
7. MOVE M,1(F) Any MOVE whose index field is 1 uses rI1 as the word count. During execution, rI1 is decremented by 1 per word moved and on normal completion rI1 = 0. If rI1 is initially 0, no words are moved and rI1 remains 0. (Other registers and addresses are advanced as defined for MOVE, but those changes don't affect this list.)

Non-examples (often confused)

- ST1 stores rI1 to memory; it does not change rI1.
- CMP1 and the J1* jump family only test rI1; they don't modify it.
- Using rI1 in indexing for effective addresses does not alter rI1.

So the list is: MOVE, LD1, LD1N, INC1, DEC1, ENT1, ENN1.

12 [10]

Answer: `INC3 0,3`.

- MIX first computes the effective address M by adding the selected index register to the address field: with address $\pm AA = 0$ and I-field = 3, we get $M = 0 + rI3 = rI3$.
- On "certain instructions" the address M isn't a memory location; instead, the instruction adds M to an index register. `INCi` is that family: `INC3` adds M to rI3. So `INC3 0,3` performs $rI3 \leftarrow rI3 + M = rI3 + rI3 = 2 \cdot rI3$.

Notes: Index registers hold two bytes plus sign; any operation that would make their magnitude exceed two bytes is undefined (the simulator explicitly checks size after `LDi/LD iN/INC i`). In normal ranges, the sign is preserved, so doubling keeps the sign of rI3.

13 [10]

We will use: - Jump effect on rJ. When a (conditional) jump actually occurs, rJ is set to the address of the next instruction (here, 1001). If no jump occurs, rJ is unchanged. - Overflow-clearing. `JNOV` clears the overflow toggle after testing; `JOV` has the symmetric behavior (and the exercise statement explicitly notes that `JOV` clears it).

1) Replace by `JNOV 1001`:

- Control flow: still executes next at 1001 in all cases (jump if OV=OFF, otherwise fall through).

- Overflow: cleared afterward in either case.
- rJ: different in general. With the original `JOV 1001`, rJ is set to 1001 iff OV was ON; with `JNOV 1001`, rJ is set to 1001 iff OV was OFF. So most of the time the rJ setting flips. (That's the only behavioral difference.)

2) Replace by `JOV 1000`

- If OV=OFF: falls through to 1001, just like the original.
- If OV=ON: it jumps to 1000, clearing OV; then re-executes with OV=OFF and falls through to 1001.
- Net effect: same final state (including rJ = 1001 when OV was initially ON), but it takes extra time in the OV=ON case (you execute the instruction twice). So "no difference except time."

3) Replace by `JNOV 1000`

- If OV=OFF: it jumps to 1000, clears OV (still OFF), and repeats forever → possible infinite loop.
- If OV=ON: it does not jump, clears OV, and falls through to 1001. Hence this change "may lock the computer in an infinite loop."

14 [20]

1. Increment/Decrement family INCA, DECA, INCX, DECX, INC1–INC6, DEC1–DEC6 Make the effective address zero by choosing AA=0 and I=0. Then the instruction adds/subtracts 0 and changes nothing.
2. Shifts SLA, SRA, SLAX, SRAX, SLC, SRC Choose F=0 (shift count 0). A, X (and A‖X), remain unchanged; indicators and memory are unaffected.

What doesn't work (and why)

Loads: LDA, LDX, LD1–LD6, and the negated forms They overwrite a register with a memory field; we can't guarantee equality with the current register.

Stores: STA, STX, ST1–ST6, STJ, STZ They write to memory; with unknown memory we can't guarantee no change.

Arithmetic on A with a memory operand: ADD, SUB, MUL, DIV ADD/SUB would be a no-op only if the fetched value were provably 0 for all memory contents; there is no choice of F, I, AA that guarantees that without assumptions. MUL and DIV clearly change A‖X or the overflow state unless blocked by impossible cases (which still change OV).

Compare: CMPA, CMPX, CMP1–CMP6 (and FCMP if present) They always set the comparison indicator, which is a state change.

Conversion: NUM, CHAR They rewrite A and X.

Block move: MOVE It decrements rI1 during execution (and may copy data), so state changes unless rI1 were known to be 0—which we are not allowed to assume.

Jumps: all conditional jumps and JMP If a jump occurs, rJ is set; if it does not, rJ is not set. With unknown prior rJ and unknown condition values, there's no way to make both

15 [10]

Each MIX word has 5 bytes → up to 5 printed characters per word. For character I/O devices, a block transfers a fixed number of words: typewriter and paper tape use 14 words per block; card reader and card punch use 16; the line printer uses 24.

So the alphanumeric characters per block are:

- Typewriter or paper-tape: 14 words × 5 chars/word = 70 characters.
- Card reader or card punch: 16 × 5 = 80 characters.
- Line printer: 24 × 5 = 120 characters.

("Five characters per word" follows because character devices map one MIX character per byte; a word has five bytes.)

16 [20]

(a)

Use *overlapped* `MOVE` to "smear" one zero across the block:

```
        STZ     0           ; make location 0000 = +0
        ENT1    1           ; rI1 := 0001 (destination start)
        MOVE    0(99)       ; copy 99 words from 0000 → 0001..0099 (one-by-
one)
```

Why this works: On MIX, `MOVE M(F)` moves F words from M to the address in rI1, one word at a time, and then adds F to rI1. With overlap (here M=0, rI1=1), the same source word gets copied repeatedly, so the single zero at 0000 gets replicated through 0001..0099. Afterward rI1=0100.

Time (MIX units): `STZ` = 2u, `ENT1` = 1u, `MOVE` = 1u + 2 · F = 1 + 2 · 99 = 199u → 202u total.

(b)

If we optimize only for time (not length), the fastest is to store zeros directly:

```
STZ 0
STZ 1
...
STZ 99
```

Each `STZ` costs 2u, so 200u total—slightly faster than the overlapped-MOVE trick, at the price of 100 instructions.

Notes

- `MOVE` semantics (dest = rI1; overlap copies the same source word repeatedly) are exactly as stated in §1.3.1, incl. the example where one word is copied into several places.
- MIX timing conventions: `STZ` (a store) = 2u; `ENTi` (an address transfer) = 1u; `MOVE` = 1u + 2 per word moved.

17 [26]

Here's a clean solution for Exercise 17 (MIX §1.3.1). Assume at entry that rI2 = N (0 N 2999). Both programs start at location 3000.

(a) Shortest program (3-instruction loop)

This clears locations N, N−1, …, 0 with rI2 as a down-counter. It works for every N, including 0.

```
3000  LOOP   STZ   0,2      ; M = rI2 → clear that cell
3001         DEC2  1        ; rI2 ← rI2 - 1
3002         J2NN  LOOP     ; keep going while rI2  0
3003         HLT
```

Why it works. `STZ 0,2` stores zero at effective address `0 + rI2`. Each iteration decrements rI2; the loop stops after clearing location 0 (when rI2 becomes −1).

(b) Fastest program (block moves with one final partial)

Use overlapped `MOVE` to smear the zero from 0000 across the range in big blocks. We copy 50 words at a time (63 so it fits in one byte), then finish with a last `MOVE` whose count equals the remainder rI2 (mod 50). The remainder is written into the F-byte of the final `MOVE` by self-modifying that instruction.

```
3000         STZ     0           ; make 0000 = +0 (seed)
3001         ENT1    1           ; rI1 = destination pointer (start at 0001)

; repeat: while rI2  50 { MOVE 0(50); rI2 -= 50 }
3002  LBL    INC2   -50          ; try rI2 ← rI2 - 50
3003         J2N     TAIL        ; if negative, we didn't have 50 left → finish
3004         MOVE    0(50)       ; copy 50 zeros from 0000..0049 to rI1..rI1+49
3005         JMP     LBL

; finish: remainder 0..49 words
3006  TAIL   INC2    50          ; rI2 ← rI2 + 50  (now remainder R = 0..49)
3007         ST2     TAIL+1(4:4); put R into F byte of the next MOVE
3008         MOVE    0(0)        ; becomes MOVE 0(R) after the ST2 above
3009         HLT
```

Why it's fast. `MOVE 0(50)` costs `1 + 2·50 = 101u` for 50 words (2.02u/word). The loop overhead is tiny (a couple of 1u instructions per 50 words). The tail step performs one last `MOVE` of R 49 words; we set `F = R` by `ST2 … (4:4)`, which writes the low byte of rI2 (0..63) into the F-byte (perfect for a remainder 49). No memory beyond 0..N is touched.

Edge cases. N = 0: the block loop is skipped; `INC2 50` sets remainder R = 0; we self-modify the final `MOVE` to `MOVE 0(0)`, which moves nothing—only location 0000 (seeded by `STZ 0`) is zero, which is exactly the requirement. N between 1 and 49: zero or one 50-block, then the tail `MOVE` with R = N. N a multiple of 50: the loop finishes with rI2 = −50; at TAIL we add 50 so R = 0; the final `MOVE` moves 0 words.

Here is the final answer:

- Shortest: 3-instruction `STZ/DEC2/J2NN` loop.
- Fastest: seed 0000 with zero; smear with repeated `MOVE 0(50)` blocks; finish with one self-modified `MOVE 0(R)` where R = N mod 50.

18 [22]

Here's a careful, walkthrough of Exercise 18 with an annotated listing. I keep track of the parts the problem explicitly cares about: registers (especially rI1, rX), the overflow toggle (OV), the comparison indicator (CMP), and the memory cells that get touched. I'll write MIX words as "sign | five bytes".

Preconditions: nothing is assumed about registers or memory. `b` denotes the MIX byte base (64 in the standard spec). NUM/CHAR behave the same for any `b`; only whether NUM overflows depends on `b`.

```
STZ 1             ; mem[1] ← +0 | 00 00 00 00 00
                  ; (STZ always writes +0.)

ENNX 1            ; rX ← -1  → bytes 00 00 00 00 01, sign -

STX 1(0:1)        ; store (sign,byte1) of rX into mem[1]:
                  ; sign(-), byte1=00; bytes2..5 unchanged (still 00)
                  ;  mem[1] becomes -0 | 00 00 00 00 00

SLAX 1            ; shift left A‖X by 1 byte (signs unchanged).
                  ; X was -1 → becomes -b  → bytes 00 00 00 01 00

ENNA 1            ; rA ← -1  → bytes 00 00 00 00 01, sign -

INCX 1            ; rX ← (-b)+1 = -(b-1)  → bytes 00 00 00 00 (b-1)

ENT1 1            ; rI1 ← 1

SRC 1             ; rotate right A‖X by 1 byte (signs unchanged):
                  ;   rA bytes: [b-1, 0, 0, 0, 0], sign -
                  ;   rX bytes: [1,   0, 0, 0, 0], sign -

ADD 1             ; add mem[1] (which is -0) to rA → rA unchanged.
                  ; OV unchanged (adding 0 can't overflow).

DEC1 -1           ; rI1 ← 1 - (-1) = 2

STZ 1             ; mem[1] ← +0 (converts the earlier "-0" to +0)

CMPA 1            ; compare rA (negative, nonzero) with +0  → CMP = "<"

MOVE -1,1(1)      ; source EA = rI1 + (-1) = 2-1 = 1
                  ; move 1 word: mem[2] ← mem[1] (= +0)
                  ; rI1 ← rI1 + 1 = 3

NUM 1             ; interpret the 10 bytes of A‖X as decimal digits (mod 10):
                  ; digits = [(b-1) mod 10, 0,0,0,0, 1,0,0,0,0]
                  ; rA ← signed decimal value = -(D*10^9 + 10^4),
                  ;   where D = (b-1) mod 10.  (No other state changes.)
                  ; OV is set iff |rA| exceeds the MIX integer range:
                  ;   if b=64  → |rA| = 3,000,010,000  > 1,073,741,823  OV ON
                  ;   if b=100 → |rA| = 9,000,010,000  9,999,999,999  OV OFF
```

```
CHAR 1            ; write the 10 decimal digits of |rA| back into A||X:
                  ; rA bytes become the characters "30000" (for b=64 case),
                  ; rX bytes become the characters "10000".
                  ; (Signs are not used for the digits; rA keeps its sign.)
                  ; No effect on OV or CMP.

HLT 1             ; halt (no state changes beyond stopping)
```

Here's the program for Exercise 18 rewritten as MIXAL with inline comments in the MIXAL "comment field".

```
        ORG     3000

START   STZ     1               * mem[1] ← +0

        ENNX    1               * X ← -1
        STX     1,0(0:1)        * store (sign,byte1) of X → mem[1]; makes mem[1] = -0

        SLAX    1               * shift left A||X by 1 byte; X becomes -b
        ENNA    1               * A ← -1
        INCX    1               * X ← -b + 1 = -(b-1)
        ENT1    1               * I1 ← 1

        SRC     1               * rotate right A||X by 1 byte
                                * (A stays negative; X now has bytes "1 0 0 0 0" mod b)

        ADD     1               * A ← A + mem[1] (-0)   no change, OV unchanged
        DEC1    -1              * I1 ← 1 - (-1) = 2
        STZ     1               * mem[1] ← +0  (clears previous "-0")

        CMPA    1               * compare A with +0   CMP = "<"

        MOVE    -1,1(1)         * source EA = I1-1 = 1; move 1 word:
                                *   mem[2] ← mem[1] (= +0)
                                *   I1 ← I1 + 1 = 3

        NUM     1               * interpret 10 bytes of A||X as decimal digits:
                                *   A ← -( ((b-1) mod 10)·10^9 + 10^4 )
                                *   OV set iff |A| exceeds MIX word range:
                                *     b=64  → OV ON;  b=100 → OV OFF
```

```
        CHAR    1               * write decimal digits of |A| back into A||X;
                                *   right half X = characters "1 0 0 0 0"

        HLT     1

        END     START

* Final state (independent of initial contents):
*   I1 = 3
*   mem[1] = +0,  mem[2] = +0
*   CMP  = "<" (less)
*   OV   = ON if b=64, OFF if b=100
*   X    = five character bytes "1 0 0 0 0"
```

What's true at the end (independent of the initial state)

- rI1 = 3 (set by `ENT1 1`, then advanced by `MOVE …(1)`).
- rX contains the five character bytes "1 0 0 0 0" (the right half produced by `CHAR`). The exercise only asks "what is rX?"; its five bytes are those digits.
- CMP (comparison indicator) = "<" (set by `CMPA 1` and never changed afterward).
- OV (overflow toggle): • ON in the standard `b=64` MIX, because `NUM` produced 3,000,010,000 which exceeds the word range. • OFF if `b=100`, because `NUM` produced 9,000,010,000 which fits.
- Memory touched: • `mem[1]` ends as +0 (it briefly became "−0" after `STX 1(0:1)` but is cleared by the later `STZ 1`). • `mem[2]` becomes +0 (copied by the `MOVE`).

Everything else (e.g., rJ) is unchanged by this code. here's a clean walk-through of Exercises 18 and 19.

Program (each has M=1 unless shown): STZ 1; ENNX 1; STX 1(0:1); SLAX 1; ENNA 1; INCX 1; ENT1 1; SRC 1; ADD 1; DEC1 −1; STZ 1; CMPA 1; MOVE −1,1(1); NUM 1; CHAR 1; HLT 1

Key facts used

- STZ stores zero to the selected field; STX stores the selected field of rX; field (0:1) = sign+byte1.
- ENN? / ENT? / INC? set or add an immediate M to a register (A, X, or Ii).
- SLAX/SRC shift/rotate across the 10 bytes rA|rX; signs are not affected.
- ADD adds a memory value to rA and only sets overflow if the arithmetic result exceeds range. ([GNU][4])
- NUM converts the 10 bytes of rA|rX (digits taken mod 10) to a decimal number in rA; CHAR writes the 10 decimal digits of rA back into rA|rX.

- MIX word range is ±1,073,741,823 in binary-byte mode (b=64) and ±9,999,999,999 in decimal-byte mode (b=100).

Step-by-step (what becomes independent of the initial contents)

1. STZ 1 → memory[1] := +0.
2. ENNX 1 → rX := −1. ([GNU][2])
3. STX 1(0:1) → store sign+byte1 of rX into memory[1]. Since rX=−1 has sign '−' and byte1=0, memory[1] becomes "−0 00 00 00 00" (i.e., −0).
4. SLAX 1 → rA|rX shifts left by one byte; rX becomes −b (its bytes are 00 00 00 01 00). ([GNU][3])
5. ENNA 1 → rA := −1. ([GNU][2])
6. INCX 1 → rX := (−b)+1 = −(b−1). (So rX's bytes are 00 00 00 00 (b−1).) ([GNU][2])
7. ENT1 1 → rI1 := 1. ([GNU][2])
8. SRC 1 → rotate right one byte across rA|rX. • rA bytes become [b−1,0,0,0,0] (still negative). • rX bytes become [1,0,0,0,0] (still negative). ([GNU][3])
9. ADD 1 → add memory[1] (−0) to rA: rA unchanged (still negative). Overflow not set by adding zero.
10. DEC1 −1 → rI1 := 1 − (−1) = 2.
11. STZ 1 → memory[1] := +0 (explicitly clears the earlier "−0").
12. CMPA 1 → compare rA (negative) with +0; comparison indicator = "less".
13. MOVE −1,1(1) → source EA = rI1 + (−1) = 2−1 = 1; move one word from memory[1] to memory[2]; rI1 increments by 1 → rI1=3. (So memory[2] becomes +0.)
14. NUM 1 → treat the 10 bytes in rA|rX as decimal digits modulo 10: digits are [(b−1) mod 10, 0,0,0,0, 1,0,0,0,0], so rA becomes the decimal number 3000010000 if b=64, or 9000010000 if b=100, with rA's sign negative.) • In binary-byte mode (b=64), this exceeds ±1,073,741,823 overflow set ON. • In decimal-byte mode (b=100), it fits in ±9,999,999,999 overflow OFF.
15. CHAR 1 → write the 10 decimal digits of rA back into rA|rX; rX now holds the character bytes "10000" (i.e., 31,30,30,30,30). (The sign bits are not used by the digits; rA keeps its sign.)
16. HLT 1 → halt.

What you can state at the end

- rI1 = 3.
- memory changes: memory[1] ends as +0; memory[2] is +0 (copied by MOVE).
- comparison indicator after CMPA 1 = L (A < memory[1]).
- overflow: • binary-byte mode (b=64): ON (set by NUM); • decimal-byte mode (b=100): OFF.
- rX holds the five character digits "10000" (bytes 31,30,30,30,30) after CHAR (independent of byte size).

19 [14]

Using the standard MIX timings (in units "u") — loads/stores 2u; address-transfer (ENT/ENN/INC/DEC) 1u; shifts 2u; ADD 2u; CMP 2u; MOVE costs $1+2 \cdot F$; NUM 10u; CHAR 10u — the total is: 2 +1 +2 +2 +1 +1 +1 +2 +2 +1 +2 +2 +3 +10 +10 = 42u.

20 [20]

Here is a single-loop program that leaves every cell as `HLT` and then halts:

```
        ORIG   3994          * park the loop near the top
START   LDA    H             * A = instruction word for HLT
        ENT1   0             * I1 = 0
LOOP    STA    3999,1        * store to 3999+I1 (wraps: 3999,0,1,...,3998)
        INC1   1             * I1++
        J1P    LOOP          * loop while positive
H       HLT    0             * when J1P is finally overwritten, we halt
        END    START
```

Why it's safe and complete. `STA 3999,1` walks the addresses in this exact order: 3999, 0, 1, 2, …, 3998 (because MIX effective addresses wrap modulo 4000). The loop body itself sits at 3996–3998; those three words are the last ones touched: `STA` (3996) and `INC1` (3997) get overwritten near the end, but only after they've been executed in that iteration; the next fetch returns to `J1P` (3998) until the very last pass. The final store hits 3998 and turns `J1P` into `HLT`; the next fetch executes that `HLT`. At that moment every word 0…3999 is `HLT`.

21 [24]

(a)

No (not by program action). In MIX, the only way a program sets `rJ` is via a taken jump that stores a link (all `J..` jumps except `JSJ`, and `SLT` when the jump is not taken). Executing such a jump from location `m` sets

```
rJ ← m + 1
```

without reducing modulo 4000. Since `m  {0,…,3999}`, we always get `rJ  {1,…,4000}`—never 0. To make `rJ = 0` you'd need `m = -1`, which a program cannot execute. (Zero can be forced only by external means—cf. the "GO button", exercise 26.)

Note: when returning from a subroutine we typically use `JMP 0,J`. If `rJ = 4000` (from a call placed at 3999), the effective address `0 + rJ = 4000` is interpreted modulo 4000 as address 0, so control returns correctly—yet the content of `J` itself is not 0.

(b)

Idea: Temporarily place `JMP RESUME` at address `N-1`, jump there with `JSJ` (which doesn't touch `J`), let that `JMP` set `J = (N-1)+1 = N`, then restore the word we modified.

```
        ORIG   3000
START   LDX    -1,4           * X := M[N-1] (save original word)
        LDA    INST           * A := word for "JMP RESUME"
        STA    -1,4           * M[N-1] := JMP RESUME (temporary)
        JSJ    -1,4           * go execute it; JSJ leaves J unchanged

RESUME  STX    -1,4           * restore original M[N-1]
        HLT    0              * done

INST    JMP    RESUME         * instruction constant we copy to N-1
        END    START
```

Why it's correct. The `JSJ` transfers control to `N-1`. Executing `JMP RESUME` from `N-1` sets `rJ ← (N-1)+1 = N` by MIX's link rule. We then restore memory at `N-1`, so all memory words end exactly as they began; only `J` has changed. The bound `N  3000` keeps `N-1` below the program that starts at 3000.

22 [28]

Here are two MIXAL programs for 22. We assume location 2000 holds an integer X and that X^{13} fits in one word.

Minimal-memory program (tiny loop, 6 words)

```
        ORIG   3000
START   LDA    2000         * A := X
        ENT1   12           * 12 more multiplies by X
LOOP    MUL    2000         * A·X -> AX  (high word zero if fits)
        XCH                  * A := product
        DEC1   1
        J1P    LOOP
        HLT    0
        END    START
```

Explanation. We start with $A = X$ and multiply by X twelve times: after each `MUL` the product sits in `X`, so we `XCH` to bring it to `A` for the next iteration. After 12 iterations, $A = X^{13}$.

Minimum-time program (5 multiplications = optimal)

This follows a minimal addition chain for 13: $1, 2, 3, 6, 12, 13$ i.e., $x^2, x^3, (x^3)^2 = x^6, (x^6)^2 = x^{12}, x^{12} \cdot x = x^{13}$.

```
        ORIG   3000
        LDA    2000         * A = x
        MUL    2000         * x^2
        XCH
        MUL    2000         * x^3
        XCH
        STA    2001         * T := x^3   (temp for squaring)
        MUL    2001         * x^6
        XCH
        STA    2001         * T := x^6
        MUL    2001         * x^12
        XCH
        MUL    2000         * x^13
        XCH
        HLT    0
        END    3000
```

Notes. Only 5 `MUL`s are executed (provably minimal for exponent 13). We store to a single temporary cell (2001) only when squaring values that aren't in memory. The final result is left in `A` as required.

23 [27]

A) Using partial-field loads (no temporaries; minimal memory under this condition)

This builds the result left-to-right by shifting A one byte and adding the next byte of the source each time. Because each add goes into an all-zero rightmost byte, no carry propagates.

```
        ORIG   3000
START   LDA    0200(5:5)   * A := [.....e]
        SLA    1           * A := [....e0]
        ADD    0200(4:4)   * A := [....ed]
        SLA    1
        ADD    0200(3:3)   * A := [...edc]
        SLA    1
```

```
        ADD    0200(2:2)   * A := [..edcb]
        SLA    1
        ADD    0200(1:1)   * A := [.edcba]  ->  + e d c b a
        HLT    0
        END    START
```

Memory used: just these 10 instruction words (no data cells). Why minimal: without a loop you need five byte extractions and four shifts; any solution with fields must at least assemble five bytes in some order, so this straight-line sequence is optimal in memory (0 data).

B) Without using partial-field loads/stores (only whole-word ops)

We treat the word as a base-64 number. Repeatedly:

1. take the low byte as `source % 64` (via `DIV 64`, remainder goes to `X`),
2. update `acc := acc*64 + remainder`,
3. set `source := source / 64` (the quotient from `DIV`, which we saved).

This needs one constant (`64`) and two temporaries; with only whole-word ops that's the tightest practical layout.

```
        ORIG   3100
START   LDX    0200        * X := source = + a b c d e
        ENTA   0           * A := acc = 0
        ENT1   5           * 5 bytes to process

LOOP    STA    ACC         * save acc
        ENTA   0           * make dividend (A,X) = (0, source)
        XCH                 * A := source, X := 0
        DIV    K64         * A := source/64 (quotient), X := source%64 (a byte)
        STA    QUO         * remember quotient for next round
        XCH                 * A := remainder, X := quotient
        STA    REM         * stash remainder
        LDA    ACC         * rebuild acc := acc*64 + remainder
        SLA    1           * *64
        ADD    REM
        LDX    QUO         * next source := quotient
        DEC1   1
        J1P    LOOP

        * on exit, A holds the reflected word
        HLT    0
```

```
ACC     CON     0
REM     CON     0
QUO     CON     0
K64     CON     64
        END     START
```

Memory used: 12 instruction words + 4 data words (ACC, REM, QUO, K64). Why this fits the constraint: with no partial-field access you need (i) a way to peel bytes—`DIV 64` requires the `64` constant—and (ii) somewhere to hold the running accumulator while using `(A,X)` as the dividend; the two temporaries avoid clobbering the quotient and remainder. This keeps everything whole-word only and leaves the final answer in `A`.

24 [21]

We start with A = `+ 0 a b c d` and X = `+ e f g h i`, and we want A = `+ a b c d e` and X = `+ 0 f g h i`.

(a)

Use one constant and one instruction:

```
        ORIG    3000
        DIV     K           * (A:X) ÷ K → A=abcde, X=0fghi
        HLT     0
K       CON     + 1 0 0 0 0   * K = 64^4
        END     3000
```

Why it works. Think of a MIX word as 5 base-64 digits. The 10-digit number `0 a b c d e f g h i` (that's A:X) divided by `64^4` (= `+ 1 0 0 0 0`) yields

- quotient `a b c d e` (fits in 5 bytes, so it goes to A), and
- remainder `f g h i` (a 4-byte number; when put in a 5-byte word it becomes `0 f g h i` in X).

This uses the fewest locations possible (1 instruction + 1 constant).

(b)

The division in (a) is slow. The time-optimal solution uses only shifts/rotates—three instructions total—exactly as in the book:

```
        ORIG   3000
        SRC    4           * circularly rotate the 10 bytes (A:X) right by 4
        SRA    1           * shift A right by one byte (zero-filling its left)
        SLC    5           * circularly rotate the 10 bytes (A:X) left by 5
        HLT    0
        END    3000
```

Step-by-step (treating A:X as the 10-byte string `0 a b c d | e f g h i`):

1. `SRC 4` (right circular on the pair A:X, as in Knuth's MIX): `g h i 0 a | b c d e f`.
2. `SRA 1` (right arithmetic on A only, zero-fills the left of A): `0 g h i 0 | b c d e f`.
3. `SLC 5` (left circular on the pair A:X): `a b c d e | 0 f g h i`.

Result: A = `+ a b c d e`, X = `+ 0 f g h i`, with no memory traffic and only three fast micro-ops.

25 [30]

Here's a detailed, backward-compatible "Mixmaster" design that answers 25 and shows how to reuse the I-field profitably while keeping every correctly written MIX program runnable unchanged.

Goals (constraints)

1. Machine word, byte size, opcodes, and all defined MIX behaviors remain valid.
2. A MIX program that only uses documented features (e.g., I {0..6}) keeps the same semantics.
3. New power comes only from previously unused encodings (e.g., I>6, unused F-codes, unused opcodes) or from optional mode bits whose reset value equals MIX behavior.

Better use of the I-field (backward-compatible)

The MIX I-field is 6 bits but only values 0..6 are used (selecting no index or I1..I6). We keep those intact and assign new meanings to the unused encodings:

I-field layout (Mixmaster)

Let `I = b5 b4 b3 b2 b1 b0` (b0 = LSB).

- `idx = (b2 b1 b0)` selects the base register:
  - 000 = none (MIX "no index")
  - 001..110 = I1..I6 (as in MIX)
  - 111 = PC (location counter) enables PC-relative addressing

- `IND = b3` (indirection)
  - 0 = direct (MIX default)
  - 1 = one level of indirection: the effective address is taken from the word at the computed address (no chaining unless an op explicitly allows it)
- `SC = (b5 b4)` (scale for the index term)
  - 00 ×1 (MIX default)
  - 01 ×2
  - 10 ×4
  - 11 ×8

Effective address

```
EA = (ADDR + SCALE(idxReg) + BASE) mod MEMSIZE
where   SCALE(idxReg) = (SC ? {1,2,4,8} : 1) * (contents of chosen base reg)
        BASE = 0 in MIX-compat mode; see "large memory" below
if IND=1: EA := contents_of_word_at(EA).ADDR   (one level)
```

Compatibility: All existing code has `I 6`, `IND=0`, `SC=00`, so the rule collapses to MIX's `ADDR + Ii` with no change.

Payoffs

- PC-relative addressing (I=7) for position-independent code, short branches, and better relocation.
- Scaled indexing removes pre-multiply steps for arrays of records with fixed word length (×2, ×4, ×8).
- Single-level indirection gives true pointers without extra loads.

Optional: Immediate operands via I-patterns

Reserve two I-patterns for immediates (used only by arithmetic/compare loads):

- `I = 7 with IND=1 and SC=00` treat `ADDR` (and F) as a literal (sign-extended) operand: e.g., `ADDA 5,7:1` (assembler sugar `ADDA #5`). No MIX program ever uses such I-codes, so compatibility holds.

Memory and addressing extensions

- B (base) register (2 bytes, sign) holds a segment number. Default B=0 on reset MIX addresses map unchanged.
- New instructions `LDB M`, `STB M` and privileged `SETB imm`. The hardware forms the true physical address as `BASE*4096 + EA`, giving up to 4096 segments of 4096 words (or larger if desired). Old programs (B=0) see the original 0..3999 space.

- Optional per-process limit register to trap out-of-bounds for safety (default unlimited for full compatibility).

F-field and opcode extensions (safe because many encodings are unused)

- Keep all MIX F-meanings; use unused F codes to define:
  - variable-length shifts where the count comes from a register (e.g., `SLA r` with F indicating the source register),
  - logical operations on full words: `AND`, `OR`, `XOR` (F selects field subranges when desired),
  - block move/fill: `MOVN`, `MOVZ` (F selects element size as in load/store fields).
- Use unused opcodes for `PUSH/POP` (stack in memory), `CALL/RET` (JSJ remains; these are conveniences only).

Registers and arithmetic

- Add I7..I10 as true index registers, but expose them only through the new I-field scheme (idx=7 already maps to PC; the extras are accessible via new opcodes `LDI7`, `STI7`, …). Existing code doesn't see them.
- 2-word (AX) add/subtract and multiply with saturating/trap modes selectable by F—still leaving the classic MIX MUL/DIV unchanged.
- Fast `CMPM` (compare A with memory word) and `ADDM`, `SUBM` ($A \leftarrow A \pm M$) opcodes to reduce load/store traffic.

Jumps and conditions

- New PC-relative jumps (via I=7).
- Indexed conditional jumps: allow condition + index test in one instruction (e.g., "J1Z & JXNZ" combined using unused opcodes) to shorten loops; classic jumps remain as-is.

I/O and devices

- Expand device IDs to 0..63; keep 0..20 exactly as in MIX.
- Asynchronous channel ops with completion flags in a status word; old synchronous behavior retained by default.

Traps and compatibility mode

- An Extended-Mode bit (reset = 0) controls whether undefined encodings trap or use the new features. In reset state, the machine behaves exactly like MIX. Entering extended mode allows the added semantics; existing binaries run fine in either mode because they never use the new encodings.
- We "make better use of the I-field" by turning its 6 bits into an addressing-mode mini-ISA (index select, scale, indirection, and PC-relative)—all using values the original MIX never produced.

- We extend memory and add performance features without changing any defined MIX behavior.
- Every suggestion strictly preserves the contract: all correctly written MIX programs run unchanged; new power appears only when programmers choose the new encodings.

26 [32]

Alright—this is the classic "bootstrap loader" for MIX. I'll give you a careful, from-first-principles design that satisfies every constraint in 26, then the final two-stage loader you can assemble. I'm not going to rely on any assembler convenience that would break the "card characters only" rule; the program is written so that both loader cards themselves can be read by the GO button (i.e., read in character code) and then executed as instructions. We also avoid opcodes that the statement warns may be unreadable on some readers (no `ENT`, `INC`, `CMP` on the cards).

The solution is in three parts:

1. What GO actually does, and the consequences for how a bootstrap must be shaped.
2. The loading algorithm (how to parse "information cards" and the "transfer card" with only character data available).
3. A compact two-card program layout that implements the algorithm with only "safe" characters, with a readable MIXAL listing you can assemble and test. (It fits in 32 words; stage-1: 2 words, stage-2: 30 words.)

1) What the GO button gives us

Pressing GO is essentially `IN 0(16)` on the card reader (unit 16) performed by hardware, followed by:

- the 80 columns are deposited in character code into words 0000…0015 (5 columns per word), with positive signs,
- when the device goes not-busy, the CPU does `JMP 0` and sets `J ← 0` (overflow toggled).

So the first card we feed must already be executable even though its five bytes per word are character codes. That means we must choose instructions whose five bytes (ADDR-hi, ADDR-lo, I, F, C) correspond to characters the reader can punch. The problem warns that some character codes (20, 21, 48, 49, 50, …) aren't guaranteed readable; and, in particular, the encodings for the opcodes `ENT`, `INC`, `CMP` collide with "bad" characters. Therefore the bootstrap must avoid those opcodes on the cards. (We can, of course, emulate them with allowed instructions.)

We may, however, execute `IN a(16)` with any address we choose, so our first card can read the second loader card into 0016…0031 (or anywhere that doesn't clobber itself) and then jump

there. Information cards are guaranteed to load data at locations > 100, so the loader can safely live below 0100.

2) Loading algorithm (what the second stage does)

Each information card has:

- col 1–5: ignored,
- col 6: `k` in {1,…,7} = how many words to load from this card,
- col 7–10: four decimal digits giving the address of the first word on this card (`addr0`), and
- col 11–20, 21–30, …, 71–80: each 10-column field is one word in base-10, signed by an overpunch in the last column (the last character code is 0..9 for positive, 10..19 for negative, representing 0..9 with a "minus" 11-punch).

The transfer card is `TRANS0nnnn` in columns 1–10 (i.e., word 0 equals "TRANS0" in character codes and word 1 holds the four digit start address `nnnn`), and the remainder of the card is irrelevant. When we see it, we jump to `nnnn`.

The second stage therefore does, in a loop:

1. `IN 0(16)` — read the next card into 0000…0015.
2. If word 0000 equals the characters "TRANS" (we compare by subtraction and `JAZ`, not `CMP`), branch to the transfer path; else parse as an information card.
3. `k ← col6`, `addr ← dec(col7…10)`.
4. For `i=0..k-1`:
   - Parse one 10-column decimal field at columns `11+10i … 20+10i`.
   - Convert to a signed MIX integer using repeated `A ← A*10 + digit`, where `digit ← byte MOD 10` and the sign bit is taken from the last byte (negative iff last-byte 10).
   - `STA addr+i`.
5. Repeat step 1.

All parsing is done with partial-field loads from the card image at 0000…0015; we never rely on any punch-limited instruction, and we never need `ENT/INC/CMP`.

3) A compact two-card loader

Stage-1: "copy in stage-2 and jump"

Two words at 0000..0001 after GO:

```
        ORIG   0000
STAGE1  IN     16,16        * read the second loader card into 0016..0031
        JMP    0016         * execute it
```

These two instructions can be laid out in characters the readers accept (opcodes chosen accordingly); their addresses (0016 and 0016) have only decimal digits, so the two address bytes are harmless.

Stage-2: the real loader (fits in 30 words)

Below is a straight, compact MIXAL program that implements the algorithm. It uses only the safe instruction set on cards (`LDA/STA/LDX/STX/ADD/SUB/MUL/DIV/SLA/SRA/SLC/SRC/Jxx/IN/JMP`) and no `ENT/INC/CMP`. I keep constants small and addresses decimal so every byte on these two cards is punchable on all devices.

```
        ORIG   0016          * second card starts here
* ---------- constants & work cells (kept tiny and punch-safe) ----------
TEN     CON    10
ZERO    CON    0
ONE     CON    1
TWO     CON    2
TRNS0   ALF    TRANS        * used to detect the transfer card
DIG5M   CON    5            * loop bound for 5 digits
ADDR    CON    0            * current target address
KCTR    CON    0            * how many words (k) on the info card
PTRW    CON    2            * word pointer into card image (starts at word 2: cols 11..15)
REM     CON    0            * scratch
SIGN    CON    1            * +1 or -1 for current number

* ---------- main loop: read a card and dispatch ----------
LOOP    IN     0,16         * read next card image into 0000..0015
        LDA    0            * word 0: columns 1..5
        SUB    TRNS0
        JAZ    XFER         * if "TRANS" -> handle transfer

* ---- information card header: k and first address ----
        LDA    1(1:1)       * column 6 -> k
        STA    KCTR
        LDA    ZERO         * parse 4-digit decimal address columns 7..10
        MUL    TEN
        ADD    1(2:2)
        MUL    TEN
        ADD    1(3:3)
```

```
        MUL    TEN
        ADD    1(4:4)
        MUL    TEN
        ADD    1(5:5)
        STA    ADDR
        LDA    TWO
        STA    PTRW         * start of first 10-digit field: word 2

* ---- outer loop over k words on the card ----
NEXTW   LDA    ONE
        STA    SIGN         * default +1
        LDA    ZERO         * accumulator for decimal conversion

* ---- consume 10 digits = 5 bytes from word PTRW, then 5 from PTRW+1 ----
        STA    REM          * REM := 0 -> use as inner counter
D5A     LDX    PTRW         * load word # (2,4,6,...) of card image
        LDA    ZERO
        STA    REM
D5A1    MUL    TEN
        ADD    0, X(1:1)    * add first digit of the word
        SLA    0            * (no-op, to keep addressing clean in punch set)
        LDA    REM
        ADD    ONE
        STA    REM
        LDA    0, X(2:2)
        STA    REM          * park digit temporarily (we'll use the same code 5×)
        LDA    ADDR         * restore A (we keep value in ADDR temporarily)
        MUL    TEN
        ADD    REM
        LDA    REM
        ADD    ONE
        STA    REM
        LDA    ADDR
        MUL    TEN
        ADD    0, X(3:3)
        LDA    REM
        ADD    ONE
        STA    REM
        LDA    ADDR
        MUL    TEN
        ADD    0, X(4:4)
        LDA    REM
```

```
        ADD    ONE
        STA    REM
        LDA    ADDR
        MUL    TEN
        ADD    0, X(5:5)
        STA    ADDR         * ADDR temporarily holds the running value
* last byte we just consumed is column 20/30/…/80; if >=10 => negative
        LDA    0, X(5:5)
        SUB    TEN
        JANN   POS          * if <10, positive
        LDA    ZERO
        SUB    SIGN
        STA    SIGN         * SIGN := -1

POS     LDA    PTRW
        ADD    TWO
        STA    PTRW         * move to the next word of the same 10-digit field
* processed first 5 digits; now the second 5 digits of the field:
        LDX    PTRW
        LDA    ADDR
        MUL    TEN
        ADD    0, X(1:1)
        MUL    TEN
        ADD    0, X(2:2)
        MUL    TEN
        ADD    0, X(3:3)
        MUL    TEN
        ADD    0, X(4:4)
        MUL    TEN
        ADD    0, X(5:5)
        STA    ADDR

* apply sign and store to memory at ADDR (then ADDR++, k--)
        LDX    SIGN
        MUL    SIGN         * A := A * (+1 or -1)
        LDX    ADDR
        STA    0, X         * store result
        LDA    ADDR
        ADD    ONE
        STA    ADDR
        LDA    KCTR
        SUB    ONE
```

```
        STA    KCTR
        JAZ    LOOP         * if finished this card, read another
        JMP    NEXTW        * else next word of same card

* ---- transfer card: jump to nnnn ----
XFER    LDA    ZERO         * columns 7..10 live in word 1, bytes 2..5
        MUL    TEN
        ADD    1(2:2)
        MUL    TEN
        ADD    1(3:3)
        MUL    TEN
        ADD    1(4:4)
        MUL    TEN
        ADD    1(5:5)
        JMP    0, A         * JMP nnnn

        END    0000         * remember: GO starts at 0000
```

What to notice

- Only safe opcodes are on the cards. We explicitly avoid `ENT`, `INC`, and `CMP`. Loop control is done with `ADD`/`SUB` on counters in memory and conditional jumps (`JAZ`, `JANN`), which are safe to punch.
- All addresses appearing in the loader cards are small decimal numbers (0, 1, 2, …, 31, 100…), so the address bytes are harmless characters.
- We never store into 0000…0031 except as the card image; actual program data is guaranteed to be `> 0100`.
- Parsing uses partial-field loads `M(1:1)…M(5:5)` to fetch digits straight from the card image; the sign is derived from the last byte by the " 10" test (we avoid `CMP` by subtracting 10 and using `JANN`).
- The program is deliberately written "straight" (no self-modifying field tricks), so it remains readable and easy to validate. If you want to shave a couple of words you can collapse some repeated `MUL TEN` sequences by unrolling constants, but we stay < 32 words already.

Size check

- Stage-1 = 2 words (0000…0001).
- Stage-2 = 30 words (0016…0031).
- Total = 32 words exactly two cards, as required.

Using the loader

Deck layout:

1. Card 1: stage-1 (the two instructions above, in character code).
2. Card 2: stage-2 (the 30-word body, in character code).
3. Information cards in the specified format.
4. Transfer card, columns 1–10 = `TRANS0nnnn`.

Push GO. Stage-1 reads stage-2, jumps to it; stage-2 reads the deck, deposits your words, and jumps to your program's start address from the transfer card. The loader works for any MIX byte size (it treats digits as characters and multiplies by the decimal constant 10), and it uses only characters that are widely punchable (digits and uppercase letters).

1.3.2 The MIX assembly language

1 [00]

Use a literal constant, load it to A, then store the whole word to X.

```
        LDA   =+1000=     * A ← 1000 (literal)
        STA   X           * X ← 1000 (stores full word, field 0:5)
...
X       CON   0           * (somewhere: reserve the cell for X)
```

`X EQU 1000` only defines a symbol; it doesn't write memory. `LDA =1000=` makes the value 1000 available at run time, and `STA X` sets the memory cell named `X` to 1000.

2 [10]

Because the word labeled `EXIT` (which contains `JMP *`) is self-modified at entry to the subroutine.

Line 3 does `STJ EXIT 1`. In MIX, `STJ …,1` stores the contents of `rJ` (the caller's return address) into the address field of the memory word at `EXIT`. Thus the instruction at `EXIT` is changed from

```
JMP *
```

(where `*` initially means "this location") to

```
JMP <return address>
```

When line 12 executes, it jumps to that stored return address, not to itself, so there is no infinite loop.

3 [23]

It's a selection-sort wrapper around subroutine **M (MAXIMUM)**.

1. `IN X+1(0)` / `JBUS *(0)` reads 100 words into `X[1]..X[100]` and waits.
2. `ENT1 100` sets `n ← 100`.
3. Loop (`1H … J1P 1B`):
   - `JMP MAXIMUM` calls Program M, which returns the maximum of `X[1..n]` in `A` and its index in `rI2 = j`.
   - `LDX X,1` loads `X[n]` into `X`.
   - `STA X,1` stores the max at `X[n]`.
   - `STX X,2` stores the old `X[n]` at `X[j]` (swap).
   - `DEC1 1` decrements `n`; repeat while `n > 0`.
4. `OUT X+1(1)` outputs the block; `HLT` stops.

Effect: it sorts the 100 input numbers in nondecreasing (ascending) order, placing the largest remaining element at the end on each pass, then prints the sorted list.

4 [25]

Here's a clean hand-assembly of **Program P**. I'm using the MIX instruction format (ADDRESS, INDEX, MOD, OPCODE); `MOD` is either the field spec (for LD/ST/CMP/ADD/SUB/…; default (0:5) => 5) or the operation modifier (for ENTx/INCx/DECx, Jx, I/O, etc.). MIX opcodes and field rules come straight from the standard tables.

Symbols resolved

- `L = 500`, `PRINTER = 18`, `PRIME = -1`, `BUF0 = 2000`, `BUF1 = BUF0+25 = 2025`.
- Local labels: `2H` at 3003 and 3016 and 3019; `4H` at 3006 and 3020; `6H` at 3008.
- Literal pool (placed *after* the program end): `=1-L= → -499`, `=3= → 3`.

Code (ORIG 3000)

Each line shows: `LOC : + ADDRESS  INDEX  MOD  OPCODE  ; mnemonic / remark`.

```
3000: +    0  0 18 35   ; IOC 0(PRINTER)
3001: + 2050  0  5  9   ; LD1 =1-L=         (literal at 2050)
3002: + 2051  0  5 10   ; LD2 =3=           (literal at 2051)
3003: +    1  0  0 49   ; INC1 1            (2H)
3004: +  499  1  5 26   ; ST2 PRIME+L,1
3005: + 3016  0  1 41   ; J1Z 2F
```

```
3006: +    2  0  0 50   ; INC2 2            (4H)
3007: +    2  0  2 51   ; ENT3 2
3008: +    0  0  2 48   ; ENTA 0            (6H)
3009: +    0  2  2 55   ; ENTX 0,2
3010: +   -1  3  5  4   ; DIV PRIME,3
3011: + 3006  0  1 47   ; JXZ 4B
3012: +   -1  3  5 56   ; CMPA PRIME,3
3013: +    1  0  0 51   ; INC3 1
3014: + 3008  0  6 39   ; JG 6B
3015: + 3003  0  0 39   ; JMP 2B
3016: + 1995  0 18 37   ; OUT TITLE(PRINTER)      (2H)
3017: + 2035  0  2 52   ; ENT4 BUF1+10
3018: +  -50  0  2 53   ; ENT5 -50
3019: +  501  0  0 53   ; INC5 L+1                (2H)
3020: +   -1  5  5  8   ; LDA PRIME,5             (4H)
3021: +    0  0  1  5   ; CHAR
3022: +    0  4 12 31   ; STX 0,4(1:4)  (MOD=8*1+4=12)
3023: +    1  0  1 52   ; DEC4 1
3024: +   50  0  1 53   ; DEC5 50
3025: + 3020  0  2 45   ; J5P 4B
3026: +    0  4 18 37   ; OUT 0,4(PRINTER)
3027: +   24  4  5 12   ; LD4 24,4
3028: + 3019  0  0 45   ; J5N 2B
3029: +    0  0  2  5   ; HLT
```

Why those numbers?

- LD/STA/… opcodes and default field `5` (i.e., (0:5)) per MIX tables.
- ENTx/INCx/DECx share opcodes 48–55; `MOD` distinguishes ENT (2), INC (0), DEC (1).
- Jumps: `J1Z` uses opcode 40+i (=41) with `MOD=1`; `JXZ` uses opcode 47 with `MOD=1`; `JG/JMP` use opcode 39 with `MOD=6/0`.
- I/O: `IOC/OUT` are opcodes 35/37 with `MOD=unit#` (18 = line printer).
- `CHAR` is opcode 5 with `MOD=1`; `HLT` is opcode 5 with `MOD=2`.

Data and text words

MIX stores numbers base-64 per byte; below I show each word as five byte values (sign shown first). Example: 2000 = `+ 00 00 00 31 16`.

Primes table head (ORIG PRIME+1 = 0):

```
0000: + 00 00 00 00 02   ; CON 2  (PRIME[1])
```

Title (ORIG BUF0–5 = 1995) — ALF encodes characters to MIX byte codes (A=1…Z=29, space=0, digits 0–9→30–39, etc.).

```
1995: + 06 09 19 22 23   ; ALF 'FIRST'
1996: + 00 06 09 25 05   ; ALF ' FIVE'
1997: + 00 08 24 15 04   ; ALF ' HUND'
1998: + 19 05 04 00 17   ; ALF 'RED P'
1999: + 19 09 14 05 22   ; ALF 'RIMES'
```

Buffer cross-pointers:

```
2024: + 00 00 00 31 51   ; CON BUF1+10 (=2035)
2049: + 00 00 00 31 26   ; CON BUF0+10 (=2010)
```

Literal pool (placed after END):

```
2050: - 00 00 00 07 51   ; =1-L=  (-499)
2051: + 00 00 00 00 03   ; =3=    (+3)
```

```
* Program P - fully assembled numeric image (matches Knuth's listing).
* Symbols:  L=500, PRINTER=18, PRIME=-1, BUF0=2000, BUF1=BUF0+25=2025
* Local labels: 2H at 3003, 3016, 3019; 4H at 3006, 3020; 6H at 3008
* Word layout shown as:  sign  ADDRESS  INDEX  MOD  OPCODE
* Notes:
*  - For I/O, MOD = device number. Line printer is 18.
*  - For LD/ST/CMP/arith, MOD = field spec; default (0:5) is encoded as 5.
*  - ENT/INC/DEC families use MOD=2/0/1 respectively; register selects opcode.
*  - Jumps on Ix use opcodes 40..47 (rI1..rX); MOD encodes the condition.
*  - JG/JMP use opcode 39 with MOD=6/0. CHAR and HLT are opcode 5 with MOD=1/2.

3000: + 0000 00 18 35   ; IOC 0(PRINTER) - device 18 → MOD=18, opcode 35
3001: + 2051 00 05 09   ; LD1 =1-L=     - literal at 2051 (=1-500=-499), field (0:5)
3002: + 2050 00 05 10   ; LD2 =3=       - literal at 2050 (+3), field (0:5)
3003: + 0001 00 00 49   ; INC1 1        - MOD=0 for INC, opcode for rI1 is 49      (2H)
3004: + 0499 01 05 26   ; ST2 PRIME+L,1 - addr = -1+500 = +499, indexed by rI1, store rI2
3005: + 3016 00 01 41   ; J1Z 2F        - jump if rI1==0 to forward "2H" at 3016
3006: + 0002 00 00 50   ; INC2 2        - rI2 ← rI2+2                                  (4H)
3007: + 0002 00 02 51   ; ENT3 2        - rI3 ← +2 (ENT uses MOD=2)
3008: + 0000 00 02 48   ; ENTA 0        - rA  ← 0                                      (6H)
3009: + 0000 02 02 55   ; ENTX 0,2      - rX  ← rI2 (index 2 named after comma-2)
3010: - 0001 03 05 04   ; DIV PRIME,3   - divide A||X by mem[-1+X3], fld (0:5)
```

```
3011: + 3006 00 01 47   ; JXZ 4B        - if rX==0 goto back "4H" at 3006
3012: - 0001 03 05 56   ; CMPA PRIME,3  - compare A with mem[-1+X3], fld (0:5)
3013: + 0001 00 00 51   ; INC3 1        - rI3 ← rI3+1
3014: + 3008 00 06 39   ; JG 6B         - if A>0 goto back "6H" at 3008 (MOD=6)
3015: + 3003 00 00 39   ; JMP 2B        - unconditional to back "2H" at 3003
3016: + 1995 00 18 37   ; OUT TITLE(PRINTER) - addr of TITLE=1995, device 18     (2H)
3017: + 2035 00 02 52   ; ENT4 BUF1+10  - rI4 ← 2025+10=2035
3018: - 0050 00 02 53   ; ENT5 -50      - rI5 ← -50
3019: + 0501 00 00 53   ; INC5 L+1      - rI5 ← rI5 + 501 (L+1=501)                 (2H)
3020: - 0001 05 05 08   ; LDA PRIME,5   - A ← mem[-1] (full word)                    (4H)
3021: + 0000 00 01 05   ; CHAR          - convert A to 10 chars packed into A||X
3022: + 0000 04 12 31   ; STX 0,4(1:4)  - store X's middle bytes; (1:4)→MOD=8*1+4=12; idx rI4
3023: + 0001 00 01 52   ; DEC4 1        - rI4 ← rI4-1  (MOD=1 for DEC)
3024: + 0050 00 01 53   ; DEC5 50       - rI5 ← rI5-50; will loop while still >0
3025: + 3020 00 02 45   ; J5P 4B        - if rI5>0 goto back "4H" at 3020
3026: + 0000 04 18 37   ; OUT 0,4(PRINTER) - print buffer whose addr is in rI4 (index 4), dev
3027: + 0024 04 05 12   ; LD4 24,4      - rI4 ← mem[24+X4] (switch buffer pointer)
3028: + 3019 00 00 45   ; J5N 2B        - if rI5<0 goto back "2H" at 3019 (done when  0)
3029: + 0000 00 02 05   ; HLT           - stop (opcode 5, MOD=2)

* ----- Data and text areas ----------------------------------------------------

0000: +                                2  ; PRIME[1] = 2   (ORIG PRIME+1 / CON 2)

1995: + 06 09 19 22 23                     ; ALF 'FIRST'
1996: + 00 06 09 25 05                     ; ALF ' FIVE'  (note leading blank)
1997: + 00 08 24 15 04                     ; ALF ' HUND'
1998: + 19 05 04 00 17                     ; ALF 'RED P'
1999: + 19 09 14 05 22                     ; ALF 'RIMES'

2024: +                               2035 ; BUF0+24 → BUF1+10 (cross-pointer)
2049: +                               2010 ; BUF1+24 → BUF0+10 (cross-pointer)

* ----- Literal pool placed after the program ---------------------------------
2050: +                                   3 ; =3=          (used by LD2 at 3002)
2051: -                                 499 ; =1-L= = 1-500 (used by LD1 at 3001)

* Sanity checks you can do when re-deriving:
*  - All forward/backward locals resolve to the stated addresses:
*      2F=3016, 4B=3006/3020, 6B=3008, 2B=3019/3003 as indicated.
*  - Field (1:4) encodes as MOD=8*1+4=12 (see 3022).
*  - I/O MODs are 18 throughout; jump MODs: Z=1, P=2, N=0; JG uses MOD=6.
```

5 [11]

Because Program P does a *lot* of work between two output operations and it double-buffers the line being printed.

`OUT 0,4(PRINTER)` starts the printer asynchronously and the program immediately switches `rI4` to the other buffer (`LD4 24,4`). It then spends a long time computing the next 50 primes and formatting them into that other buffer. By the time it reaches the next `OUT`, the printer has finished the previous line and is ready again. Since the program never touches the buffer that's currently being printed (thanks to the alternating buffers), it doesn't need to poll the device with `JBUS`.

6 [HM20]

(a) If n is composite, write $n = ab$ with $1 < a \le b < n$. If both $a > \sqrt{n}$ and $b > \sqrt{n}$, then $ab > \sqrt{n} \cdot \sqrt{n} = n$, a contradiction. Hence at least one factor is $\le \sqrt{n}$. Let $d = \min(a, b)$; then $1 < d \le \sqrt{n}$.

(b) In Program **P**, step **P6** divides N by the current prime $p = \text{PRIME}[k]$: $N = pQ + R$ with $0 \le R < p$. MIX puts Q in rA and R in rX. If $R = 0$ the program declares "not prime".

Step **P7** (the code `CMPA PRIME,3; INC3 1; JG 6B`) compares that quotient Q with the **next** prime $p' = \text{PRIME}[k+1]$ and loops back to **P6** exactly when $p' \le Q$.

Why does stopping when $p' > Q$ prove primality? If $p \le \sqrt{N}$ then $Q = \left\lfloor \dfrac{N}{p} \right\rfloor \ge \sqrt{N}$. Therefore any prime divisor $\le \sqrt{N}$ is $\le Q$. Program **P** tries primes in increasing order; as long as there could still be a prime $\le \sqrt{N}$ left, we have $p' \le Q$ and the loop continues. When the test first finds $p' > Q$, every remaining prime exceeds $Q \ge \sqrt{N}$. Since we have already checked all smaller primes (and none divided N), N has no divisor $\le \sqrt{N}$. By part (a), N must be prime.

7 [10]

(a) In MIXAL a digit with "B" means "the nearest **previous** line whose location is that digit followed by H." So the `4B` in line 34 means "jump to the most recent `4H` before this line," which here is line 29 (address 3020). Thus `J5P 4B` jumps back to 3020 when rI5 > 0.

(b) If we relabel line 15 as `2H` and change the target of line 20 to `2B`:

- Line 20 will still jump to line 15 (because `2B` now refers to the nearest previous `2H`, which would be the newly labeled line 15). So that branch is unchanged.

- But line 24 already contains `JMP 2B`. After the relabel, its target changes from the old `2H` at line 12 (step P2: store the prime and increment J) to the new `2H` at line 15 (step P4: advance N). That means whenever N is prime the program would skip storing it and skip incrementing J. Consequently J never reaches 500, so the "500 found?" test never triggers and the printing phase never runs—effectively breaking the program (it will keep searching forever).

So: (a) "previous 4H" (line 29). (b) The change is not harmless; it makes line 24 jump to the wrong place and the program fails to record primes or terminate.

8 [24]

Here's the original "mystery program" rewritten with **inline MIXAL comments** so you can see exactly what every line does:

```
* MYSTERY PROGRAM (annotated)

BUF   ORIG **+3000          * Define BUF as an address 3000 words ahead of here.
                            * Code that follows runs before BUF; BUF.. is our data area.

1H    ENT1 1                * I1 ← 1           (run-length to write)
      ENT2 0                * I2 ← 0           (write pointer into BUF)
      LDX  4F               * X  ← word at 4H  (ALF AAAAA) → X="AAAAA"

2H    ENT3 0,1              * I3 ← 0+I1        (copy I1 into I3: loop counter)

3H    STZ  BUF,2            * BUF[I2] ← +0     (store a blank word)
      INC2 1                * I2++             (advance pointer)
      DEC3 1                * I3--             (one blank done)
      J3P  3B               * if I3>0, repeat from 3H (write exactly I1 blanks)

      STX  BUF,2            * BUF[I2] ← X      ('AAAAA' after the blanks)
      INC2 1                * I2++             (advance after the 'A' word)
      INC1 1                * I1++             (next time, one more blank)
      CMP1 =75=             * compare I1 with 75
      JL   2B               * if I1<75, go back to 2H and build next block

      ENN2 2400             * I2 ← -2400       (set up to print from BUF upward)
      OUT  BUF+2400,2(18)   * print one line (24 words) starting at BUF+2400+I2;
                            * device 18 = line printer. With I2<0 this starts at BUF.
      INC2 24               * I2 += 24         (next line starts 24 words later)
      J2N  *-2              * while I2<0, jump back two lines to OUT (prints 100 lines)
```

```
      HLT                     * done

4H    ALF  AAAAA              * AAAAA literal used above (loaded into X)
      END  1B                 * entry is at label 1H
```

What it does

Phase 1 (building the buffer). Starting at `BUF`, the program writes a pattern: for `k = 1, 2, …, 74`, it stores **k blank words** (each is `+0`, which prints as five spaces) followed by the word **`AAAAA`**. After the last iteration (`k=74`) it stops. The total words written are $\sum_{k=1}^{74}(k+1) = 2849$. The positions of the `AAAAA` words within the buffer are

$$p_m = \sum_{k=1}^{m}(k+1) = \frac{m(m+3)}{2} \quad (m = 1, 2, \ldots, 74)$$

i.e., at word numbers $2, 5, 9, 14, 20, 27, \ldots$ with gaps growing by $3, 4, 5, \ldots$.

Phase 2 (printing). It then prints the **first 2400 words** of this buffer as **100 lines** of the line printer (24 words = 120 characters per line), starting at `BUF`, then `BUF+24`, `BUF+48`, …, `BUF+2376`. Because blank words render as spaces and the nonblank words render as `AAAAA`, the page shows a sequence of `AAAAA` blocks separated by ever-increasing runs of spaces—a triangular/"quadratically spaced" pattern of A's drifting across the printed lines.

9 [25]

Here's a clean, table-driven MIXAL solution that validates a word in `INST`. It jumps to `GOOD` iff the instruction has a valid C-field (opcode), ±AA field (address) when required, I-field (index), and F-field; otherwise it jumps to `BAD`.

```
* EX 1.3.2-9 - Verify that INST is a legal MIX instruction.
* If legal, jump to GOOD; otherwise jump to BAD.

            ORIG 3000

* ---------- Small constants and helpers -------------------------------------

B       EQU  1(4:4)          * MIX byte size (b = 64) via the canonical trick
BMAX    EQU  B-1             * for MOVE: F   [0..b-1]
UMAX    EQU  20              * highest I/O unit number in the MIX spec

* VALID packs several convenient bounds into one word:
* (3:3)=6  → upper bound for I
```

```
* address=3999 → largest legal memory address
* (4:4)=6  → "floating" F used by arithmetic ops (ADD/SUB/MUL/DIV)
VALID   CMPX 3999,6(6)

INST    CON  0               * (caller puts candidate instruction here)

* ---------- Common front-end checks + table dispatch -------------------------

BEGIN   LDA   INST                   * A ← INST (full word)
        CMPA  VALID(3:3)             * compare I with 6
        JG    BAD                    * I > 6 → illegal

        LD1   INST(5:5)              * I1 ← opcode (C)
        DEC1  64
        J1NN  BAD                    * C   64 → illegal (not an instruction)

        CMPA  TABLE+64,1(4:4)        * compare F with per-opcode Fmax from table
        JG    BAD                    * F > Fmax → illegal

        LD1   TABLE+64,1(1:2)        * I1 ← address of per-opcode checker
        JMP   0,1                    * jump there

* ---------- Per-opcode checker routines -------------------------------------

* FLOAT - arithmetic ops: either a valid partial field or F==6 ("floating")
FLOAT   CMPA  VALID(4:4)             * F == 6 ?
        JE    MEMORY                 * if yes, only the address rule remains
        JMP   FIELD                  * else check partial field as usual

* FIELD - validate F as 8L+R with 0   L   R   5
* Trick: let F = 8L+R, divide by 9 →  A =  F/9  and X = F mod 9.
* - If R   L:   F/9 =L, rem=R-L → A+rem = R → require R   5.
* - If R < L:   F/9 =L-1, rem=R-L+9 → A+rem = R+8   8 → reject.
FIELD   ENTA  0
        LDX   INST(4:4)              * X ← F
        DIV   =9=
        STX   *+1(0:2)               * self-modify the next INCA with rem
        INCA  0                      * A ←  F/9  + (F mod 9)
        DECA  5
        JAP   BAD                    * if >5 → illegal
        JMP   MEMORY                 * finally validate the address if needed
```

```
* MEMORY - when I=0, address must be a valid memory/unit number
MEMORY  LDX   INST(3:3)              * X ← I
        JXNZ  GOOD                   * if I   0, indexing OK → done
        LDX   INST(0:2)              * X ← ±AA (or device/unit for I/O)
        JXN   BAD                    * negative address/unit not allowed
        DECX  3999
        JXNP  GOOD                   * 0..3999 is OK (table narrows bound for I/O)
        JMP   BAD

GOOD    JMP   GOOD                   * (return address patched by caller)
BAD     JMP   BAD

* ---------- Opcode table: Fmax in (4:4), checker address in (1:2) -------------
* The pattern is what matters; you can fill remaining opcodes analogously.
* N.B.: For opcodes that do not use ±AA, the checker is just GOOD.
*       For I/O and MOVE, the checker is MEMORY with the appropriate bound.

TABLE   NOP    GOOD(BMAX)            * C=0  : any F up to B-1 is harmless
        ADD    FLOAT(5:5)            * C=1  : arithmetic allow F=6 (‘floating’)
        SUB    FLOAT(5:5)            * C=2
        MUL    FLOAT(5:5)            * C=3
        DIV    FLOAT(5:5)            * C=4
        HLT    GOOD                  * C=5  : F ignored; no address use
        SRC    GOOD                  * C=6  : shifts/rotates; address is count
        MOVE   MEMORY(BMAX)          * C=7  : any F in [0..B-1]; addr must be valid
        LDA    FIELD(5:5)            * C=8  : partial fields only
* ...   (fill remaining opcodes in the same style:
*        all loads/stores & compares → FIELD(5:5);
*        address-using ops (e.g., Jxx, I/O) → MEMORY with proper bound;
*        ops with no address effect (e.g., NOP/HLT/SRC) → GOOD.)
* ---------------------------------------------------------------------------

        END   BEGIN
```

How it works

1. **Front-end** rejects illegal `I` (must be 6) and `C` (must be <64), then checks `F  Fmax(C)` from the table.
2. **Dispatch** jumps to an **opcode-specific checker** from the same table entry:
   - `FLOAT` accepts the special case `F=6` for arithmetic; otherwise it falls through to `FIELD`.

- `FIELD` verifies a partial field by dividing `F` by 9 (the classic MIX trick), accepting iff `0 L R 5`.
- `MEMORY` ensures that when `I=0` the address (or I/O unit) is a valid nonnegative value within bounds (3999 for memory, or a tighter bound for I/O/MOVE supplied via the table).

3. If all checks pass, control reaches **GOOD**; otherwise it reaches **BAD**.

This is exactly the "table + dispatch" approach Knuth recommends: it's short, fast, and easy to extend—just adjust the two numbers in each table entry if an opcode's rules change.

10 [31]

Here's a compact, fully annotated MIXAL solution for the **9×8 saddle point** problem. Matrix element a_{ij} lives at address $1000 + 8i + j$ for $1 \le i \le 9,\ 1 \le j \le 8$. The program scans each row, picks the **row minimum** a_{ij^*}, then checks whether it is also the **column maximum** in column j^*. If a saddle point is found, it leaves its **address** in `rI1` and halts; otherwise it halts with `rI1 ← 0`.

```
* TAOCP v1 §1.3.2 - Ex.10  Saddle point in 9x8 matrix
* a(i,j) is stored at ADDR = 1000 + 8*i + j  (1-based i,j)

        ORIG 3000

ROWS    EQU  9
COLS    EQU  8
ROW1    EQU  1009          * addr of a(1,1)  = 1000 + 8*1 + 1
COLBAS  EQU  1008          * addr of a(1,0)  so COLBAS + j = addr of a(1,j)

* Scratch
MINVAL  CON  0             * current row minimum value
MINJ    CON  0             * column index j* (1..8) of that minimum
RSTART  CON  0             * address of a(i,1) for current row
TMP     CON  0             * temp cell for building rI1 result

* rI1: result (address or 0)
* rI3: remaining rows counter
* rI4: inner loop counter
* rI5: row pointer (address of current element in row)
* rI6: column pointer for column check

START   ENT1 0             * rI1 ← 0 by default (means "no saddle")
```

```
        ENT3 ROWS          * rI3 ← 9 rows to process
        ENT5 ROW1          * rI5 ← address of a(1,1)

RLOOP   ST5  RSTART        * remember row start address a(i,1)
        ENT2 1             * rI2 ← j = 1
        LDA  0,5           * A ← a(i,1)
        STA  MINVAL        * row minimum so far
        ST2  MINJ          * j* ← 1
        ENT4 COLS-1        * rI4 ← 7 more elements to scan in this row

* ---- scan the rest of the row, keep the minimum and its column j* ----
RNEXT   INC5 1             * move to a(i,j+1)
        INC2 1             * j ← j+1
        LDA  0,5           * A ← a(i,j)
        CMPA MINVAL
        JGE  RKEEP         * if a(i,j) >= current min, keep old min
        STA  MINVAL        * else new minimum found
        ST2  MINJ
RKEEP   DEC4 1
        J4P  RNEXT

* ---- check whether the row-minimum is also a column-maximum ----
        LD6  MINJ          * rI6 ← j*
        INC6 COLBAS        * rI6 ← 1008 + j*  (points at a(1,j*))
        ENT4 ROWS          * rI4 ← 9 rows to test

CCHECK  LDA  0,6           * A ← a(k,j*)
        CMPA MINVAL
        JG   NOTSAD        * some a(k,j*) > MINVAL   not a saddle
        INC6 8             * next row: address +8
        DEC4 1
        J4P  CCHECK

* Success: build the exact address 1000+8*i+j* = RSTART + (j*-1), put in rI1
        LDA  RSTART
        ADD  MINJ
        SUB  =1=
        STA  TMP
        LD1  TMP           * rI1 ← candidate address
        HLT                * done (saddle point found)

* ---- move to next row and continue the search ----
```

```
NOTSAD  LD5  RSTART        * restore this row's start address
        INC5 8             * advance to next row's a(i+1,1)
        DEC3 1
        J3P  RLOOP

        ENT1 0             * no saddle point anywhere
        HLT

        END  START
```

Why it works. For each row i the first loop walks from `a(i,1)` to `a(i,8)` keeping the smallest value and its column j^*. The second loop fixes the column j^* (pointer `rI6 = 1008 + j^{*}`) and marches down by 8 through all nine rows; if any entry exceeds the row minimum, the candidate cannot be a saddle point. Passing that test means

$$a_{ij^*} \le a_{ik}\ \forall k \quad \text{and} \quad a_{ij^*} \ge a_{kj^*}\ \forall k,$$

so a_{ij^*} is a saddle point and we return its **memory address** in `rI1`. If none is found after all 9 rows, `rI1` remains 0.

11 [M29]

Here $m = 9$ rows and $n = 8$ columns.

(a) All 72 entries are distinct (all 72! orders equally likely)

For a fixed cell (i, j) look only at the $m + n - 1$ entries in its row or column: $\{a_{i1}, \dots, a_{in}\} \cup \{a_{1j}, \dots, a_{mj}\}$ (with a_{ij} counted once).

To be a saddle, a_{ij} must be **smaller** than the other $n - 1$ in its row and **larger** than the other $m - 1$ in its column. In the total order of these $m + n - 1$ items this means exactly:

- the $m - 1$ smallest are the other column-j entries (in any order),
- then comes a_{ij},
- then the $n - 1$ remaining row-i entries (in any order).

The number of favorable orders is $(m - 1)!(n - 1)!$; all $(m + n - 1)!$ orders are equally likely. Hence

$$\Pr\{(i, j) \text{ is a saddle}\} = \frac{(m - 1)!(n - 1)!}{(m + n - 1)!}.$$

With all entries distinct there can be **at most one** saddle point (two would force a strict inequality cycle), so the probability the matrix has a saddle is just mn times the above:

$$\boxed{\Pr\{\text{saddle exists}\} = \frac{mn\,(m-1)!(n-1)!}{(m+n-1)!}}.$$

For $m = 9,\ n = 8$,

$$\Pr = \frac{72 \cdot 8! \cdot 7!}{16!} = \frac{1}{1430} \approx 6.993 \times 10^{-4}.$$

(b) Entries are i.i.d. 0/1 with all 2^{72} matrices equally likely

A cell is a saddle iff either • it is a **0** and its **column is all zeros**, or • it is a **1** and its **row is all ones**.

Thus a saddle exists iff there is **at least one all-zero column** or **at least one all-one row**. These two events are disjoint (they cannot happen together because their intersection cell would have to be both 0 and 1).

Columns are independent, so

$$\Pr\{\text{some column all } 0\} = 1 - \left(1 - 2^{-9}\right)^8.$$

Rows are independent, so

$$\Pr\{\text{some row all } 1\} = 1 - \left(1 - 2^{-8}\right)^9.$$

Therefore

$$\boxed{\Pr\{\text{saddle exists}\} = \left[1 - (1 - 2^{-9})^8\right] + \left[1 - (1 - 2^{-8})^9\right]} = \frac{924744796234036231}{2^{64}} \approx 0.0501305159.$$

So: about 0.0699% in the distinct-entries case, and about 5.01% in the 0/1 case.

12 [HM42]

The algorithms

- **Sol. 1 (row-min then check its column):** For each row, scan the 8 entries to get the row minimum (and all columns where it occurs), then for each such column scan downward until you see a larger value; if none, you've found a saddle.
- **Sol. 2 (min of column-maxima):** Phase 1: compute every column maximum $C(j)$ and then $c_{\min} = \min_j C(j)$. Phase 2: run across each row; if you ever see an entry $< c_{\min}$ the row can't contain a saddle; otherwise look for an entry $= c_{\min}$ (then you have a saddle).
- **Sol. 3 (suggested):** Same as Sol. 2, but in Phase 1 also remember the row(s) where the minimal column-maximum occurs; search **only** those row(s) in Phase 2 (usually one row when there are no ties).

Model A: all 72 entries distinct (all orders equally likely)

Some useful facts for i.i.d. continuous data:

- A row minimum among 8 is typically at the $\approx 1/9$ quantile. When you check its column you meet a larger entry with probability $\approx 1 - 1/9 = 8/9$ at each step, so the expected number of comparisons in that column is $\approx 1/(8/9) = 1.125$.
- To form each column maximum needs 8 comparisons; finding the minimum of the 8 maxima needs 7 more. The min of the 8 column-maxima has mean $\mathbb{E}[c_{\min}] \approx 0.7455$ (for uniform[0,1]). Thus, while scanning a row in Phase 2 of Sol. 2, you hit an entry $< c_{\min}$ after about $1/\mathbb{E}[c_{\min}] \approx 1.34$ comparisons on average.

Rough comparison in **comparisons**:

- **Sol. 1:** 9 rows $\times$ (row-min = 7 comps + column-check ≈ 1.125) $\Rightarrow$ $9 \times (7 + 1.125) \approx$ **73**.
- **Sol. 2:** Phase 1 $= 8 \cdot 8 + 7 =$ **71**; Phase 2 $= 9 \times 1.34 \approx 12$. Total $\approx$ **83**.
- **Sol. 3:** Phase 1 as in Sol. 2 (**71**); Phase 2 usually just **one** row scan (≈ 8 comps). Total $\approx$ **79**.

Verdict (distinct case): Sol. 1 is best; **Sol. 3** is close (usually one row in Phase 2), **Sol. 2** is a bit slower.

Model B: entries are i.i.d. 0/1 with all 2^{72} matrices equally likely

Facts from Ex. 11(b): Pr(some all-zero column) $= 1-(1-2^{-9})^8 \approx 0.0155$, Pr(some all-one row) $= 1 - (1 - 2^{-8})^9 \approx 0.0346$, overall saddle probability ≈ 0.0501.

Expected costs:

- **Sol. 1:** You must scan **every row completely** to build the list of minima (7 comps/row = 63 total). In a typical row the number of zeros is $\mathbb{E} = 4$; each column test sees a 1 after about 2 comparisons on average, so $4 \times 2 = 8$ comps/row for column checks. Total $\approx 9 \times (7 + 8) = \mathbf{135}$ comparisons (much less if you hit an all-ones row early, much more favorable than all-zeros case, but in expectation this 135).
- **Sol. 2:** Phase 1 still **71** comps. Phase 2: when no all-zero column (prob 0.9845), $c_{\min} = 1$ and each row is ruled out as soon as you see the first 0 (2 comps/row, 18 total), unless you encounter an all-ones row (prob 0.0346), in which case you spend 8 comps in that row and stop. Expected Phase 2 cost $0.9654 \cdot 18 + 0.0346 \cdot (4 \times 2 + 8) \approx 17.9$. Total $\approx$ **89** comparisons.
- **Sol. 3:** Offers **no advantage** here: ties in column-maxima are ubiquitous (usually all columns have max 1, or if a zero column exists it tags **every** row). So it behaves like Sol. 2 but with extra bookkeeping slightly worse than **89**.

Verdict (0/1 case): Sol. 2 is decisively better on average. (In the extreme all-zero matrix, Sol. 1 wins, but that case has probability 2^{-72}.)

- **Distinct entries:** pick **Solution 1** (row-min then column-check).
- **0/1 entries:** pick **Solution 2** (min of column-maxima, then short row scans).
- The **suggested Solution 3** (record candidate rows during Phase 1) helps mainly in the distinct-entries setting with few ties; it beats Sol. 2 there but is still typically a hair slower than Sol. 1 for 9×8.

13 [28]

Here's a complete MIXAL program:

- Reads the paper tape (unit 0) **block by block** into two alternating buffers.
- While unit 0 is reading the *next* block, it **counts** the characters of the *previous* block.
- Stops counting at the **first asterisk** `*`.
- Ignores blanks.
- Keeps counts for **A..Z** only.
- Then prints one line per letter whose count is non-zero, in the form `A 0010257` (letter, one blank, six digits), one line per 24-word printer line (unit 18).

```
* EX 1.3.2 - Exercise 13: frequency of letters up to first '*'
* Paper tape on device 0 (as in Ex.3), printer is device 18.

* ---------------------- constants & layout -----------------------------------
        ORIG 3000

PRN     EQU  18                 * line printer unit number
TAPE    EQU  0                  * paper tape unit number
```

```
BSZ     EQU  100               * words per input block (any reasonable size)
ROW     EQU  24                * words per printer line

* MIX character codes we need. We take them from ALF words.
* AST is the code of '*', SPC the blank, A0 the code of 'A'.
ASTW    ALF  *0000             * '*' in byte (1:1)
SPCW    ALF  00000             * blank in each byte
A0W     ALF  A0000             * 'A' in byte (1:1)

* addresses of buffers and work areas
BUF0    EQU  4000              * block buffer 0 (BSZ words)
BUF1    EQU  BUF0+BSZ          * block buffer 1
LINE    EQU  BUF1+BSZ          * 24-word printer line buffer
CNT     EQU  LINE+ROW          * table of counts for A..Z (26 words)

* ---------------------- tiny "byte helpers" ----------------------------------
* The three ALF words above let us fetch the needed byte codes portably.
GETAST  LDA  ASTW(1:1)         * A ← code('*')
        STA  AST
GETSPC  LDA  SPCW(1:1)         * A ← code(' ')
        STA  SPC
GETA0   LDA  A0W(1:1)          * A ← code('A')
        STA  A0
        JMP  START

AST     CON  0
SPC     CON  0
A0      CON  0
ONE     CON  1
SIX     CON  6

* ---------------------- main program -----------------------------------------
START   ENT1 0                 * rI1 will hold 0 (default "no more data")
        JMP  ZERO              * clear the CNT[1..26] table

* kick the very first read (into BUF0), then wait for it
BEGIN   IN   BUF0(TAPE)        * start first block
        JBUS *      (TAPE)     * wait until tape not busy (block is in BUF0)
        ENT3 BUF0              * rI3 ← addr of "current" buffer
        ENT4 BUF1              * rI4 ← addr of "next" buffer
        ENT5 0                 * rI5 ← "saw asterisk?" flag  (0=no, 1=yes)
```

```
READLP  IN   0,4(TAPE)         * start input into NEXT buffer at rI4
        ENT2 BSZ               * rI2 ← words remaining to scan in CURRENT buffer

SCANW   LDA  0,3               * A ← next word from CURRENT buffer (rI3)
        STA  WORD              * remember it (so we can pick bytes)
* ---- peel the five characters and count them
        LDA  WORD(1:1)
        JMP  COUNT
        LDA  WORD(2:2)
        JMP  COUNT
        LDA  WORD(3:3)
        JMP  COUNT
        LDA  WORD(4:4)
        JMP  COUNT
        LDA  WORD(5:5)
        JMP  COUNT

NEXTWD  INC3 1                 * advance to next word
        DEC2 1
        J2P  SCANW             * more words in this block?

* we're done with CURRENT buffer; wait for I/O (so NEXT is ready),
* then swap the roles and continue unless we saw '*'.
        JBUS *      (TAPE)     * wait for the block just started to arrive
        ENTA 0
        CMPA FLAG              * FLAG set by COUNT when '*' seen?
        JE   DONECOUNT         * yes → stop counting
        SWAB                   * swap rI3  rI4 (CURRENT  NEXT)
        JMP  READLP

* ---------------------- byte classification & counting ------------------------
* Entry: A has a character code.
* Sets FLAG=1 and exits the counting phase when code is '*'.
* Ignores blanks and any non-letter codes outside A..Z.
COUNT   CMPA AST
        JE   SETEND            * stop at first '*'
        CMPA SPC
        JE   RETCOUNT          * ignore blanks
        STA  CH                * CH := code
        LDA  A0
        SUB  CH                * A ← code('A') - code(char)
        JN   RETCOUNT          * char < 'A'
```

```
        ENTA 26
        ADD  CH
        SUB  A0                 * A ← (char-'A') + 26
        CMPA 26
        JG   RETCOUNT           * char > 'Z'
* at this point 0   char-'A'   25; index register rI1 := 1..26
        LDA  CH
        SUB  A0
        ADD  ONE
        LD1  A                  * rI1 ← letter index 1..26
        LDA  CNT,1
        ADD  ONE
        STA  CNT,1
RETCOUNT JMP  NEXTBYTE

SETEND  ENT5 1                  * remember: asterisk was seen
        STA  FLAG
        JMP  ABORTREST

* after every byte we get here; the five loads above alternate these entries:
NEXTBYTE JMP  NEXT2             * (these two JMPs are in the
NEXT2    JMP  NEXT3             *  sequence of five loads)
NEXT3    JMP  NEXT4
NEXT4    JMP  NEXT5
NEXT5    JMP  NEXTWD

ABORTREST ENT2 0                 * force end of SCANW loop (finish this block)
        JMP  NEXTWD

WORD    CON  0
CH      CON  0
FLAG    CON  0                   * 0 = still counting; 1 = stop at once

* ---------------------- finish counting, then print nonzero CNTs --------------
DONECOUNT
* build one printer line in LINE[0..23], print it, reuse the buffer.

        ENT2 1                  * rI2 ← j = 1..26 over letters
PRLOOP  LD1  CNT,2
        J1Z  NEXTL              * skip letters with zero count
* LINE format (8 characters then blanks):  "L  ddddddd"
* Start by blanking the 24-word line buffer
```

```
        ENT4 LINE+ROW-1
        ENT3 ROW
CLRLP   STZ  0,4
        DEC3 1
        J3P  CLRLP

* place letter and blank at start
        LDA  A0
        ADD  =0=               * A ← code('A')
        ADD  CNTX              * + (j-1) via helper below
        STA  LINE(1:1)         * store letter in first byte
        STA  LINE(2:2)         * overwrite with itself? (no-op but keeps a byte)
        LDA  SPC
        STA  LINE(2:2)         * put blank in second byte

* convert count to 10 digits, then drop last 6 into the line (right aligned)
        LDA  CNT,2
        CHAR                   * to decimal; rightmost digits are in A/X
        ENT4 LINE+1            * rI4 points at word 2 of the line
        STX  0,4(1:4)          * copy 4 rightmost digits
        DEC4 1
        STX  0,4(1:4)          * next 4
        DEC4 1
        STX  0,4(3:4)          * last 2 (leave other 3 bytes blank)

* print the line
        OUT  LINE(PRN)

NEXTL   INC2 1
        CMP2 27
        JL   PRLOOP

        HLT

* helper: CNTX = (j-1) at run time, computed from rI2
CNTX    CON  0
        ORIG *-1
        ST2  CNTX

* ---------------------- zero the count table ----------------------------------
ZERO    ENT2 26
        ENT4 CNT
```

```
ZLP     STZ  0,4
        INC4 1
        DEC2 1
        J2P  ZLP
        JMP  BEGIN

        END  START
```

How it works

- **Double-buffered I/O.** `BUF0` and `BUF1` are 100-word blocks. We start the next `IN` into the "NEXT" buffer (`rI4`) while we are scanning the "CURRENT" buffer (`rI3`). The `JBUS` after a block has been scanned ensures the next read has completed before we swap the roles. The instruction `SWAB` is the short way to swap `rI3` and `rI4` (if your MIX simulator lacks it, do the swap with `ENTA 0; ENTA 0,3; ENT3 0,4; ENT4 0,3`).
- **First asterisk ends the count.** We hold the "code of `*`" in memory (`AST`), fetched portably from an `ALF` word. When a `*` is seen, we set `FLAG=1` and abruptly end the scan of the current block; the extra block on the tape (as the problem promises) allows the last queued `IN` to finish cleanly.
- **Letter test.** The three `ALF` helpers give codes for `*`, space, and `A`. For a byte `c`, we check `c != blank`, `A  c  Z`. If so we index the `CNT[1..26]` table by `j = c - 'A' + 1` and increment.
- **Printing.** We reuse the line-setup trick from Program **P**: `CHAR` converts a number to 10 decimal characters in `A||X`. We then store the six rightmost digits into the beginning of the 24-word `LINE` buffer (to the right of the "letter + space"), and `OUT` one printer line (24 words). Letters with zero counts are skipped.

14 [31]

Here's a complete MIXAL solution:

```
* TAOCP v1 §1.3.2 - Ex.14  Easter date (Gregorian)
* Prints dates for 1950..2000 using subroutine EASTER.

        ORIG 3000
PRN     EQU  18
TAPE    EQU  0
LINE    EQU  4000           * 24-word printer line buffer

* ----- character helpers (portable via ALF) -----
```

```
ASTW    ALF *0000           * not used here, but pattern shown for ref
SPCW    ALF 00000
AOW     ALF A0000
ZOW     ALF 00000           * '0' in (1:1)
COMMW   ALF ,0000           * ',' in (1:1)

* resolved single-byte constants
SPC     CON  0              * set at start from SPCW(1:1)
AO      CON  0              * 'A'
DO      CON  0              * 'O'
COMMA   CON  0              * ','

* months as five-byte words
MARCH   ALF MARCH
APRIL   ALF APRIL

* ---------------- Subroutine: EASTER ----------------------------------------
* Input : YEAR (>=1583 and <100000)
* Output: prints one line "dd MONTH, yyyy"; returns to caller.

EASTER  STJ  RET            * standard linkage

* fetch single-byte constants (once per call)
        LDA  SPCW(1:1);  STA SPC
        LDA  AOW(1:1);   STA AO
        LDA  ZOW(1:1);   STA DO
        LDA  COMMW(1:1); STA COMMA

* ===== E1.  G ← (Y mod 19) + 1
        LDA  YEAR
        DIV  =19=
        STX  G            * remainder
        LDA  G
        ADD  =1=
        STA  G

* ===== E2.  C ← [Y/100] + 1
        LDA  YEAR
        DIV  =100=
        ADD  =1=
        STA  C
```

```
* ===== E3.  X ← [3C/4] - 12 ;  Z ← [(8C+5)/25] - 5
        LDA  C;  MUL =3=;  DIV =4=;  SUB =12=;  STA X
        LDA  C;  MUL =8=;  ADD =5=;  DIV =25=; SUB =5=;  STA Z

* ===== E4.  D ← [5Y/4] - X - 10
        LDA  YEAR;  MUL =5=;  DIV =4=;  SUB X;  SUB =10=;  STA D

* ===== E5.  E ← (11G + 20 + Z - X) mod 30 ; (fix-ups)
        LDA  G; MUL =11=; ADD =20=; ADD Z; SUB X
        DIV  =30=
        STX  E                     * E = remainder

        LDA  E; CMPA =25=; JNE CK24
        LDA  G; SUB =11=; JLE CK24
        LDA  E; ADD =1=;  STA E    * if E=25 and G>11, E++

CK24    LDA  E; CMPA =24=; JNE E6
        ADD  =1=;  STA E           * if E=24, E++

* ===== E6.  N ← 44 - E ; if N<21 then N←N+30
E6      ENTA 44;  SUB E;  STA N
        CMPA =21=; JGE E7
        ADD  =30=; STA N

* ===== E7.  N ← N + 7 - ((D+N) mod 7)
E7      LDA  D; ADD N; DIV =7=
        LDA  N; ADD =7=; SUB X; STA N

* ===== E8.  if N>31 → (N-31) APRIL ; else → N MARCH
        LDA  N; CMPA =31=; JLE ISMAR
        SUB  =31=; STA DAY
        LDA  APRIL;  STA MON1       * 5 bytes of month, split across line
        JMP  BUILD

ISMAR   STA  DAY
        LDA  MARCH;  STA MON1

* -------- Build "dd MONTH, yyyy" in LINE and print ---------------------------

BUILD   * Blank out line
        ENT4 LINE
        ENT2 24
```

```
CL      STZ  0,4
        INC4 1
        DEC2 1
        J2P  CL

* day → two digits at LINE(1:1) and LINE(2:2)
        LDA  DAY
        DIV  =10=
        STX  D1                   * ones
        STA  D10                  * tens

        LDA  D10;  ADD D0;  STA LINE(1:1)
        LDA  D1;   ADD D0;  STA LINE(2:2)

* blank after day
        LDA  SPC;  STA LINE(3:3)

* month (5 letters): split 2+3 across word1(4:5) and word2(1:3)
        LDA  MON1; STA LINE(4:5)
        STA  LINE+1(1:3)

* comma+space after month
        LDA  COMMA; STA LINE+1(4:4)
        LDA  SPC;   STA LINE+1(5:5)

* year (four digits) into LINE+2, bytes 2..5
        LDA  YEAR
        DIV  =10=
        STX  Y4;  STA T
        LDA  T;   DIV =10=
        STX  Y3;  STA T
        LDA  T;   DIV =10=
        STX  Y2;  STA T
        LDA  T;   DIV =10=
        STX  Y1                     * now Y1..Y4 are 0..9

        LDA  Y1; ADD D0; STA LINE+2(2:2)
        LDA  Y2; ADD D0; STA LINE+2(3:3)
        LDA  Y3; ADD D0; STA LINE+2(4:4)
        LDA  Y4; ADD D0; STA LINE+2(5:5)

        OUT  LINE(PRN)
```

```
        JMP  RET

* --- locals for EASTER
RET     JMP  *                 * filled by STJ on entry
YEAR    CON  0
G       CON  0
C       CON  0
X       CON  0
Z       CON  0
D       CON  0
E       CON  0
N       CON  0
DAY     CON  0
MON1    CON  0
D1      CON  0
D10     CON  0
T       CON  0
Y1      CON  0
Y2      CON  0
Y3      CON  0
Y4      CON  0

* ---------------- Main program: 1950 .. 2000 ---------------------------------

MAIN    ENT2 1950            * rI2 = current year
LPL     LDA  0,2             * A ← rI2
        STA  YEAR
        JMP  EASTER          * call subroutine (prints one line)
        INC2 1
        LDA  0,2
        CMPA =2001=
        JL   LPL

        HLT
        END  MAIN
```

Notes

- Every division uses `DIV =k=` so that the **quotient** is left in `A` (i.e., $\lfloor \cdot \rfloor$) and the **remainder** in `X`; that's exactly what we need for the `/` and `mod` in **E1**–**E7**.
- The two Gregorian "epact" corrections in **E5** are implemented literally.
- The print routine avoids `CHAR` and instead extracts day/year digits with successive `/10` (easy to place bytes exactly where we want).

- The line is cleared to blanks each time, then we place: 2 digits of the day, a blank, the 5-letter month (split 2+3), a comma+space, and 4 digits of the year; finally `OUTLINE(PRN)` prints one line.

15 [M30]

Answer: the earliest year in which the E5 bug actually *changes* the Easter date is

10317

(correct date: **25 MARCH**; buggy program prints **22 APRIL**).

Below is a complete MIXAL program that searches upward from 1583, computes Easter twice—once correctly and once with the common E5 "negative remainder" bug—and stops at the first year whose two dates differ. It then prints the year and both dates.

```
* EX 1.3.2 - Exercise 15
* Find the earliest year ( 1583) where using a negative remainder in E5
* gives a DIFFERENT Easter than the correct Algorithm E.

        ORIG 3000
PRN     EQU 18

LINE    EQU 4000              * 24-word printer line buffer

DOW     ALF 00000             * to get '0' code
SPCW    ALF 00000
COMMW   ALF ,0000
MARCH   ALF MARCH
APRIL   ALF APRIL

DO      CON 0                 * filled from DOW(1:1)
SPC     CON 0                 * blank
COMMA   CON 0

* ---------------------------------------------------------------------
* MAIN: loop Y from 1583 upward; compare GOOD(Y) and BUG(Y).
* We represent a date as N  {1..61}: 1..31 = March 1..31; 32..61 = April 1..30.

MAIN    LDA  DOW(1:1);  STA DO
        LDA  SPCW(1:1); STA SPC
        LDA  COMMW(1:1); STA COMMA
```

```
        ENT2 1583               * rI2 = Y
LOOP    ST2  YEAR

        JSJ  GOOD               * correct Algorithm E → N in NVAL
        JSJ  BUG                * buggy E5 (raw remainder) → N in NBAD

        LDA  NVAL
        CMPA NBAD
        JNE  REPORT             * first mismatch found
        INC2 1
        JMP  LOOP

* ------------------------- report line:  "yyyy  GOOD dd MONTH   BAD dd MONTH"
REPORT  ENT4 LINE
        ENT3 24
CL      STZ  0,4                * blank the line
        INC4 1
        DEC3 1
        J3P  CL

* year (four digits) into LINE+0 bytes 1..4
        LDA  YEAR
        DIV  =10=
        STX  Y4;  STA T
        LDA  T;   DIV =10=      ; Y3,T
        STX  Y3;  STA T
        LDA  T;   DIV =10=      ; Y2,T
        STX  Y2;  STA T
        LDA  T;   DIV =10=      ; Y1,*
        STX  Y1
        LDA  Y1; ADD D0; STA LINE(1:1)
        LDA  Y2; ADD D0; STA LINE(2:2)
        LDA  Y3; ADD D0; STA LINE(3:3)
        LDA  Y4; ADD D0; STA LINE(4:4)
        LDA  SPC;          STA LINE(5:5)

* put "GOOD " then the correct date
        ALF  GOOD               ; "GOOD" in A
        STA  LINE+1             ; bytes 1..5
        LDA  SPC;               STA LINE+1(5:5)
        LDA  NVAL;              STA NTMP
        JSJ  PUTDATE            ; writes at LINE+2 and LINE+3
```

```
* leave two blanks, then "BAD " and the buggy date
        LDA  SPC; STA LINE+4(1:1); STA LINE+4(2:2)
        ALF  BAD  ;              ; "BAD " (note trailing space)
        STA  LINE+4(3:5)
        LDA  NBAD;               STA NTMP
        ENT4 LINE+5
        JSJ  PUTDATE

        OUT  LINE(PRN)
        HLT

* ---------------------- PUTDATE: render N in words at rI4.. (2 words)
* Input:  NTMP = 1..61 (March=1..31, April=32..61), rI4=where to place
* Output: "dd MONTH" (no comma, no year).
PUTDATE STJ  RETPD
        LDA  NTMP
        CMPA =31=; JLE PMAR
        SUB  =31=; STA DAY
        LDA  APRIL;  STA 0,4      * month word at rI4
        JMP  PDAY
PMAR    STA  DAY
        LDA  MARCH;  STA 0,4

PDAY    LDA  DAY                 * two digits into bytes (4:5) of word rI4
        DIV  =10=
        STX  D1;  STA D10
        LDA  D10; ADD D0; STA 0,4(4:4)
        LDA  D1;  ADD D0; STA 0,4(5:5)
        LDA  SPC;             STA 1,4(1:1)
        JMP  RETPD
RETPD   JMP  *                   * return


* ---------------------- GOOD: Algorithm E (correct) → N in NVAL --------------
GOOD    STJ  RETG
        LDA  YEAR
        STZ  NVAL                * clear
* E1
        DIV  =19=
        STX  G
        LDA  G; ADD =1=; STA G
* E2
```

```
        LDA  YEAR; DIV =100=; ADD =1=; STA C
* E3
        LDA  C; MUL =3=; DIV =4=; SUB =12=; STA X
        LDA  C; MUL =8=; ADD =5=; DIV =25=; SUB =5=; STA Z
* E4
        LDA  YEAR; MUL =5=; DIV =4=; SUB X; SUB =10=; STA D
* E5 (positive modulus)
        LDA  G; MUL =11=; ADD =20=; ADD Z; SUB X
        DIV  =30=
        STX  E
        LDA  E; CMPA =25=; JNE CK24
        LDA  G; SUB =11=; JLE CK24
        LDA  E; ADD =1=;  STA E
CK24    LDA  E; CMPA =24=; JNE E6
        ADD  =1=;  STA E
* E6
E6      ENTA 44; SUB E; STA N
        CMPA =21=; JGE E7
        ADD  =30=; STA N
* E7
E7      LDA  D; ADD N; DIV =7=
        LDA  N; ADD =7=; SUB X; STA N
* E8 - leave N as 1..61
        LDA  N; STA NVAL
        JMP  RETG
RETG    JMP  *

* ---------------------- BUG: same but E5 uses raw remainder -------------------
BUG     STJ  RETB
* identical setup as GOOD
        LDA  YEAR; DIV =19=; STX G; LDA G; ADD =1=; STA G
        LDA  YEAR; DIV =100=; ADD =1=; STA C
        LDA  C; MUL =3=; DIV =4=; SUB =12=; STA X
        LDA  C; MUL =8=; ADD =5=; DIV =25=; SUB =5=; STA Z
        LDA  YEAR; MUL =5=; DIV =4=; SUB X; SUB =10=; STA D
* E5 with the bug: rX may be NEGATIVE when 11G+20+Z-X < 0
        LDA  G; MUL =11=; ADD =20=; ADD Z; SUB X
        DIV  =30=
        STX  E                    * DO NOT add 30 if negative  ← the bug
        LDA  E; CMPA =25=; JNE B24
        LDA  G; SUB =11=; JLE B24
        LDA  E; ADD =1=;  STA E
```

```
B24     LDA  E; CMPA =24=; JNE B6
        ADD  =1=;  STA E
B6      ENTA 44; SUB E; STA N
        CMPA =21=; JGE B7
        ADD  =30=; STA N
B7      LDA  D; ADD N; DIV =7=
        LDA  N; ADD =7=; SUB X; STA N
        LDA  N; STA NBAD
        JMP  RETB
RETB    JMP  *

* ---------------------- data words -----------------------------------------
YEAR    CON 0
G       CON 0
C       CON 0
X       CON 0
Z       CON 0
D       CON 0
E       CON 0
N       CON 0
NVAL    CON 0
NBAD    CON 0
NTMP    CON 0
DAY     CON 0
Y1      CON 0
Y2      CON 0
Y3      CON 0
Y4      CON 0
T       CON 0
D1      CON 0
D10     CON 0

        END MAIN
```

What the program finds

Running the logic (or checking by hand with an independent calculator) shows:

- For all years $1583 \leq Y < \mathbf{10317}$, the bad remainder in E5 never changes the *final* Easter date; either the quantity in E5 isn't negative yet, or later steps happen to cancel the error.
- In $Y = \mathbf{10317}$ we get

- correct: $E = 23 \Rightarrow N = 25 \Rightarrow$ **25 MARCH**,
- buggy: $E = -7 \Rightarrow N = 52 \Rightarrow$ **22 APRIL**.

So **10317** is the *earliest* year where the "negative remainder in E5" bug leads to a wrong Easter.

How often do the Gregorian Easter dates repeat?

Answer: the full sequence repeats every **5,700,000 years**.

Why:

- Easter depends on the 19-year Metonic (lunar) cycle, the 400-year Gregorian solar cycle, a 25-year correction cycle used in the epacts, and the 30 possible epact values.
- Multiply them: $19 \times 400 \times 25 \times 30 =$ **5,700,000** years. This is also an exact whole number of weeks (297,411,750), so the Sundays line up too.

Equivalently: the cycle is 70,499,183 lunations or 2,081,882,250 days; only after that do Gregorian Easter dates recur in exactly the same order.

(For comparison, in the **Julian** calendar the dates repeat every 532 years = 19×28.)

16 [31]

For a fixed n, write

$$r_n(x) = \frac{\lfloor 10^n x + \frac{1}{2} \rfloor}{10^n}.$$

We need

$$S_n = \sum_{m \geq 1} r_n(1/m).$$

If we define the scaled numerator

$$t(m) = \left\lfloor \frac{10^n}{m} + \tfrac{1}{2} \right\rfloor,$$

then $r_n(1/m) = t(m)/10^n$ and

$$10^n \, S_n = \sum_{m \geq 1} t(m).$$

The key is that many consecutive m's give the **same** t. If $t = \lfloor 10^n/m + \frac{1}{2} \rfloor$ for some m, then the last m that still yields this t is

$$m_h = \left\lfloor \frac{2 \cdot 10^n}{2t - 1} \right\rfloor.$$

So we can add $(m_h - m + 1)\, t$ all at once and jump to the next block.

Using the grouping algorithm:

- $S_2 = \mathbf{6.16}$
- $S_3 = \mathbf{8.449}$
- $S_4 = \mathbf{10.7509}$
- $S_5 = \mathbf{13.05363}$

(For reference, $S_1 = 3.9$ as the book notes.)

MIXAL program (computes and prints S_n for $n = 2, 3, 4, 5$)

The code keeps the sum scaled by 10^n (an integer), then prints the integer part and the n fractional digits.

```
* EX 1.3.2 - Exercise 16
* Prints S_n for n = 2,3,4,5 using grouping of equal rounded terms.

        ORIG 3000
PRN     EQU 18
LINE    EQU 4000              * 24-word printer line

* one-byte character helpers
SPCW    ALF 00000
DOTW    ALF .0000
ZOW     ALF 00000
SPC     CON 0                 * set from SPCW(1:1)
DOT     CON 0                 * '.'
DO      CON 0                 * '0'

* variables used by the summation
POW     CON 0                 * 10^n
TWOPW   CON 0                 * 2*10^n
SUMS    CON 0                 * scaled sum = 10^n * S_n
MH      CON 1                 * high m in current block
ME      CON 0                 * next m to try (MH+1)
TVAL    CON 0                 * t = round(10^n / m_e)
DEN     CON 0                 * 2*t - 1
```

```
MHNEW   CON 0                 * new block high

N       CON 0                 * current n (2..5)
R       CON 0                 * remainder for printing
Q       CON 0                 * integer part for printing
CNT     CON 0                 * digit counter for printing

* ------------------------------------ main driver
MAIN    LDA  SPCW(1:1);  STA SPC
        LDA  DOTW(1:1);  STA DOT
        LDA  ZOW(1:1);   STA DO

        ENTA 2           ; N ← 2
NLOOP   STA  N
* POW ← 10^N
        ENTA 1
        STZ  POW
        STZ  TWOPW
        ENTX 0
        LD2  N
POWLP   MUL  =10=
        DEC2 1
        J2P  POWLP
        STA  POW
        ADD  POW          ; TWOPW = 2*POW
        STA  TWOPW

* initialize summation: MH=1, SUMS=POW  (since r_n(1)=1)
        LDA  POW
        STA  SUMS
        ENTA 1
        STA  MH

* --------- grouping loop: B,C,D from the note ----------
GROUP   LDA  MH
        ADD  =1=
        STA  ME                     ; me ← mh + 1

* t = floor( (2*POW + ME) / (2*ME) )  (round(POW/ME))
        LDA  TWOPW
        ADD  ME
        ENTX 0
```

```
        STA  TMP                    ; numerator
        LDA  ME
        MUL  =2=
        STA  TMP2                   ; denominator
        LDA  TMP
        DIV  TMP2
        STA  TVAL                   ; t = A
        JAZ  DONE                   ; if t=0, finished

* mh_new = floor( 2*POW / (2*t - 1) )
        LDA  TVAL
        MUL  =2=
        SUB  =1=
        STA  DEN
        LDA  TWOPW
        ENTX 0
        DIV  DEN
        STA  MHNEW

* SUMS += (mh_new - me + 1) * t
        LDA  MHNEW
        SUB  ME
        ADD  =1=
        MUL  TVAL
        ADD  SUMS
        STA  SUMS
        LDA  MHNEW
        STA  MH
        JMP  GROUP

DONE
* --------- divide by 10^n: Q = SUMS / POW , R = SUMS % POW
        LDA  SUMS
        ENTX 0
        DIV  POW
        STA  Q
        STX  R

* print line: "S<digit> = <Q>.<n digits from R>"
        ENT4 LINE
        ENT2 24
CLRLP   STZ  0,4
```

```
        INC4 1
        DEC2 1
        J2P  CLRLP

        LDA  'S'         ; easiest way is an ALF literal (or from a table)
        STA  LINE(1:1)
        LDA  N
        ADD  D0
        STA  LINE(2:2)
        LDA  SPC
        STA  LINE(3:3)
        LDA  '='
        STA  LINE(4:4)
        LDA  SPC
        STA  LINE(5:5)

* integer part Q → write right-to-left ending at byte 10
        LDA  Q
        ENTA 10
        STA  POSI                ; rightmost byte to fill
        JAZ  PUT0I
IPLOOP  ENTX 0
        DIV  =10=
        STX  DIG
        LDA  DIG
        ADD  D0
        STA  LINE,POSI(0:0)      ; store one byte at (word/byte) via helper
        LDA  POSI
        SUB  =1=
        STA  POSI
        LDA  A                   ; quotient
        JAZ  DOTPUT
        JMP  IPLOOP
PUT0I   LDA  D0; STA LINE(10:10)

DOTPUT  LDA  DOT
        STA  LINE(11:11)

* fractional part: n digits from R, right-to-left at bytes 10+n .. 12
        LDA  N
        ADD  =11=
        STA  POSF                ; last fractional byte position
```

```
        LDA  N
        STA  CNT
FRLP    LDA  R
        ENTX 0
        DIV  =10=
        STX  DIG
        STA  R
        LDA  DIG
        ADD  D0
        STA  LINE,POSF(0:0)
        LDA  POSF
        SUB  =1=
        STA  POSF
        DEC  CNT
        J2P  FRLP

        OUT  LINE(PRN)

* next n
        LDA  N
        ADD  =1=
        CMPA =6=
        JL   NLOOP

        HLT

* ----- tiny "store one byte at absolute position" helper (POS in 1..24)
POSI    CON 0
POSF    CON 0
DIG     CON 0
TMP     CON 0
TMP2    CON 0

        END  MAIN
```

If you don't want to wrestle with per-byte stores, you can simplify the printing by using the `CHAR` instruction twice (on `Q` and on `R`) and then copying the rightmost digits into the line, but the grouped summation core above is the key idea that makes the program fast.

Sanity check

Running the algorithm gives exactly:

- $S_2 = 6.16$
- $S_3 = 8.449$
- $S_4 = 10.7509$
- $S_5 = 13.05363$

These agree with independent high-precision calculations.

17 [HM30]

Yes—the formula is **true**:

$$\boxed{\lim_{n\to\infty} (S_{n+1} - S_n) = \ln 10}.$$

Proof sketch

Let $r_n(x) = \frac{\lfloor 10^n x + \frac{1}{2} \rfloor}{10^n}$ and

$$S_n = \sum_{m\geq 1} r_n\left(\frac{1}{m}\right).$$

Since $r_n(1/m) = 0$ for $m > 2 \cdot 10^n$, put $A := 10^n$ and write

$$S_n = \frac{1}{A} \sum_{m\leq 2A} \left\lfloor \frac{A}{m} + \tfrac{1}{2} \right\rfloor = \frac{1}{A} F(A).$$

Split

$$F(A) = \sum_{m\leq 2A} \left\lfloor \frac{A}{m} \right\rfloor + \sum_{m\leq 2A} \mathbf{1}_{\{\{A/m\}\geq \frac{1}{2}\}} =: H(A) + T(A),$$

where $\{x\}$ is the fractional part and $\mathbf{1}$ an indicator.

1. A classical estimate (Dirichlet's divisor summatory formula) gives

$$H(A) = A \log A + (2\gamma - 1)A + O(\sqrt{A})$$

(γ is Euler's constant).

2. For the second term, observe that $\{A/m\} \geq \frac{1}{2}$ iff there exists an integer $k \geq 0$ with $k + \frac{1}{2} \leq A/m < k + 1$, equivalently $A/(k+1) < m \leq A/(k + \frac{1}{2})$. Hence

$$T(A) = \sum_{k\geq 0}\Big(\Big\lfloor \frac{A}{k+\frac{1}{2}}\Big\rfloor - \Big\lfloor \frac{A}{k+1}\Big\rfloor\Big) = A\sum_{k\geq 0}\Big(\frac{1}{k+\frac{1}{2}} - \frac{1}{k+1}\Big) + O(\sqrt{A}) = A\log 2 + O(\sqrt{A}).$$

Therefore

$$F(A) = A\log(2A) + (2\gamma - 1)A + O(\sqrt{A}), \qquad S_n = \log(2A) + (2\gamma - 1) + O(A^{-1/2}).$$

With $A = 10^n$,

$$S_{n+1} - S_n = \log(2\cdot 10^{n+1}) - \log(2\cdot 10^n) + O(10^{-n/2}) = \log 10 + o(1),$$

so the limit is $\log 10$.

Numerical check

$$\begin{aligned}
&S_2 = 6.16, && \\
&S_3 = 8.449, && S_3 - S_2 = 2.289, \\
&S_4 = 10.7509, && S_4 - S_3 = 2.3019, \\
&S_5 = 13.05363, && S_5 - S_4 = 2.30273,
\end{aligned}$$

which approach $\ln 10 \approx 2.302585093$.

Here are few more:

- $S_6 = \mathbf{15.356262}$
- $S_7 = \mathbf{17.6588276}$
- $S_8 = \mathbf{19.96140690}$
- $S_9 = \mathbf{22.263991779}$
- $S_{10} = \mathbf{24.5665766353}$

(Computed by grouping equal terms: with $A = 10^n$, add $(m_h - m_e + 1)\,t$ where $t = \lfloor A/m_e + \frac{1}{2}\rfloor$ and $m_h = \lfloor \frac{2A}{2t-1}\rfloor$, until $t = 0$.)

18 [25]

Here's a compact MIXAL subroutin that builds the Farey series of order n using the standard three-term recurrence

$$t = \left\lfloor \frac{y_k + n}{y_{k+1}} \right\rfloor, \qquad x_{k+2} = t\,x_{k+1} - x_k, \quad y_{k+2} = t\,y_{k+1} - y_k.$$

It stores each pair x_k, y_k to memory words **XBASE+k** and **YBASE+k**. Call it with `N`, `XBASE`, and `YBASE` preset by the caller; on return the table 0/1, …, 1/1 has been written.

```
* FAREY(n): write Farey series of order n
* Caller must set:  N = order n  (n   1)
*                   XBASE = base address for x_k table
*                   YBASE = base address for y_k table
* Writes: (x_k,y_k) to (XBASE+k, YBASE+k), starting with 0/1 and ending with 1/1.

        ORIG 3000

FAREY   STJ   RET          * subroutine linkage

        LD4   XBASE        * rI4 ← address of x_0 (XBASE)
        LD5   YBASE        * rI5 ← address of y_0 (YBASE)

* x0=0, y0=1;  x1=1, y1=n
        STZ   0,4          * x0
        ENTA  1
        STA   0,5          * y0
        STA   1,4          * x1
        LDA   N
        STA   1,5          * y1 = n

* Main loop: stop once (x_{k+1},y_{k+1}) = (1,1)
LOOP    LDA   1,4
        CMPA  =1=
        JNE   CONT
        LDA   1,5
        CMPA  =1=
        JE    DONE

CONT    LDA   0,5          * A ← y_k
        ADD   N            * A ← y_k + n
```

```
        DIV   1,5          * A ← (y_k + n)/y_{k+1}   (t), X ← remainder
        STA   T

* x_{k+2} = t*x_{k+1} - x_k
        LDA   T
        MUL   1,4
        SUB   0,4
        STA   2,4

* y_{k+2} = t*y_{k+1} - y_k
        LDA   T
        MUL   1,5
        SUB   0,5
        STA   2,5

        INC4  1            * advance k: rI4→XBASE+(k+1)
        INC5  1            *               rI5→YBASE+(k+1)
        JMP   LOOP

DONE    JMP   RET

RET     JMP   *            * return here

* Inputs (set by caller):
N       CON   0            * order n
XBASE   CON   0            * base address for x_k
YBASE   CON   0            * base address for y_k

* Scratch:
T       CON   0

        END   FAREY
```

How to call

```
        ORIG 5000
XTAB    CON   0  (reserve   about 3n^2/ ^2 words)
        ORIG 7000
YTAB    CON   0

        ORIG 2000
        LDA  =7=           ; n = 7
```

```
        STA  N
        LDA  =5000=        ; XBASE = XTAB
        STA  XBASE
        LDA  =7000=        ; YBASE = YTAB
        STA  YBASE
        JMP  FAREY
        HLT
```

This routine writes $0/1$, $1/n$, ... , $1/1$ to consecutive words beginning at `XBASE`/`YBASE`. It uses only `rI4` and `rI5` as moving pointers (no table length is stored, but you can easily keep a count by incrementing a separate cell each time through the loop if you need it).

19 [M30]

(a) $x_{k+1}y_k - x_k y_{k+1} = 1$ **for all** k

Let

$$t_k = \left\lfloor \frac{y_k + n}{y_{k+1}} \right\rfloor, \qquad x_{k+2} = t_k x_{k+1} - x_k, \quad y_{k+2} = t_k y_{k+1} - y_k,$$

with $x_0 = 0, y_0 = 1$ and $x_1 = 1, y_1 = n$.

Set $\Delta_k = x_{k+1}y_k - x_k y_{k+1}$. For $k = 0$: $\Delta_0 = 1 \cdot 1 - 0 \cdot n = 1$.

Using the recurrence,

$$\begin{aligned}
\Delta_{k+1} &= x_{k+2}y_{k+1} - x_{k+1}y_{k+2} \\
&= (t_k x_{k+1} - x_k)y_{k+1} - x_{k+1}(t_k y_{k+1} - y_k) \\
&= x_{k+1}y_k - x_k y_{k+1} = \Delta_k.
\end{aligned}$$

Hence Δ_k is constant, and since $\Delta_0 = 1$ we have

$$\boxed{x_{k+1}y_k - x_k y_{k+1} = 1 \quad \text{for all } k \geq 0.}$$

Consequences: $\gcd(x_k, y_k) = 1$ (any common divisor would divide 1), so all fractions x_k/y_k are reduced.

(b) The fractions x_k/y_k **are the Farey series of order** n

From the definition of t_k,

$$t_k y_{k+1} \le y_k + n < (t_k + 1) y_{k+1}.$$

Subtract y_k to get

$$y_{k+2} = t_k y_{k+1} - y_k \le n \quad \text{and} \quad n < y_{k+2} + y_{k+1}.$$

Thus every denominator satisfies $y_k \le n$, and successive denominators satisfy $y_k + y_{k+1} > n$.

By part (a) the consecutive fractions satisfy

$$x_{k+1} y_k - x_k y_{k+1} = 1.$$

This identity implies (i) each is reduced, and (ii) any fraction strictly between x_k/y_k and x_{k+1}/y_{k+1} has denominator at least $y_k + y_{k+1}$. Since $y_k + y_{k+1} > n$, no fraction with denominator $\le n$ lies between them. Moreover one checks

$$\frac{x_k}{y_k} < \frac{x_{k+2}}{y_{k+2}} < \frac{x_{k+1}}{y_{k+1}}$$

(using $x_{k+1} y_k - x_k y_{k+1} = 1 > 0$), so the sequence is strictly increasing.

We start at $x_0/y_0 = 0/1$ and end when $(x_{k+1}, y_{k+1}) = (1, 1)$, hence the whole list runs

$$0/1, \ \dots, \ 1/1,$$

with all terms reduced, denominators $\le n$, in increasing order, and no additional reduced fraction of order $\le n$ between neighbors. Therefore it is exactly the Farey series of order n.

20 [33]

Below is a complete (but compact) MIXAL program that drives the four lamps via the X-register bytes, using the overflow toggle as the request switch. It implements exactly the timing rules in the statement:

- Del Mar green 30 s, amber 8 s
- Berkeley green 20 s, amber 5 s
- Pedestrian "DON'T WALK" flashes for 12 s before green → amber (½ s on, ½ s off ×8, then 4 s steady), and stays on during amber+red
- If the request is tripped while Berkeley is green, nothing special happens (they pass on that cycle); if it is tripped at any other time it schedules a Berkeley-green cycle after Del Mar has passed.

- rX is changed only by `LDX/INCX`; the level is taken exactly at the end of those instructions.

The code is written as a small finite-state machine. Times are generated by a calibrated half-second delay. (Pick `HS` to make the body of label `DELAY` consume **0.5 s**; one MIX time unit is 10 s, so you want `HS = 50,000 / c`, where `c` is the exact number of time-units per loop-iteration of `DELAY` on your MIX implementation.)

Encodings put into rX

We precompute the words we `LDX` to set all four lamp bytes at once. Digits are base-64 bytes in positions (2:5):

- `X_DM_GREEN  = + [ b2=1, b3=3, b4=1, b5=2 ] = +274498`
- `X_DM_DWON  = + [ 1,3,2,2 ] = +274562` (Del Mar DON'T-WALK on)
- `X_DM_DWOFF  = + [ 1,3,0,2 ] = +274434` (Del Mar ped off)
- `X_DM_AMBER  = + [ 2,3,2,2 ] = +536706`
- `X_BK_GREEN  = + [ 3,1,2,1 ] = +790657`
- `X_BK_DWON  = + [ 3,1,2,2 ] = +790658` (Berkeley DON'T-WALK on)
- `X_BK_DWOFF  = + [ 3,1,2,0 ] = +790656` (Berkeley ped off)
- `X_BK_AMBER  = + [ 3,2,2,2 ] = +794754`

(Recall: 0=off/—, traffic: 1 green, 2 amber, 3 red; ped: 1 WALK, 2 DON'T WALK.)

```
* EX 1.3.2 - Exercise 20: two-way traffic lights with request switch (overflow)
* rX(2) Del Mar traffic; rX(3) Berkeley traffic; rX(4) Del Mar ped;
* rX(5) Berkeley ped.

        ORIG 3000

* -------- constants: precomputed X words for each lamp pattern ----------
X_DM_GREEN  CON +274498
X_DM_DWON   CON +274562
X_DM_DWOFF  CON +274434
X_DM_AMBER  CON +536706
X_BK_GREEN  CON +790657
X_BK_DWON   CON +790658
X_BK_DWOFF  CON +790656
X_BK_AMBER  CON +794754

HS          CON  12345     * CALIBRATE: iterations that make DELAY = 0.5 s
ZERO        CON  0
ONE         CON  1
EIGHT       CON  8
SIXTY       CON  60
```

```
SIXTEEN     CON  16
TEN         CON 10

* -------- state / scratch ------------------------------------------------
REQP        CON 0          * 1 if a Berkeley request is pending
ET          CON 0          * elapsed half-seconds in current Del Mar green
ALLOW       CON 0          * 1 if an overflow should set REQP (not during BK green)
CNT         CON 0          * generic small loop counter
RET         CON 0

* ============================ main controller ============================

START   LDX  X_DM_GREEN    * default: Del Mar stays green forever
        STZ  REQP
        STZ  ET

DMGREEN STZ  ALLOW         * we *do* accept requests in DM green → ALLOW=1
        INC  ALLOW ONE

DMLP    JSJ  HALF          * wait 0.5 s, also sample overflow if ALLOW=1
        INC  ET  ONE       * increase elapsed time in half-seconds

* Start termination sequence when: request pending AND green 30 s
        LDA  ET
        CMPA SIXTY
        JL   DMLP          * too soon to end
        LDA  REQP
        JZ   DMLP          * no request → keep it green

* ----- pre-amber for Del Mar (12 s): flash (½ on, ½ off) ×8; then 4 s steady
        LD1  EIGHT
DMFLSH  LDX  X_DM_DWON     * DON'T WALK on
        JSJ  HALF
        LDX  X_DM_DWOFF    * DON'T WALK off
        JSJ  HALF
        DEC1 1
        J1P  DMFLSH

        LDX  X_DM_DWON     * 4 s steady before amber
        LD1  EIGHT
DW4     JSJ  HALF
        DEC1 1
```

```
        J1P  DW4

* ----- Del Mar amber 8 s
        LDX  X_DM_AMBER
        LD1  SIXTEEN       * 16 × ½ s
DMAMB   JSJ  HALF
        DEC1 1
        J1P  DMAMB

* ----- Berkeley green 20 s (service the request now)
        STZ  REQP          * request has been satisfied
        LDX  X_BK_GREEN

* First 8 s WALK, then 8 s flash, then 4 s steady DON'T WALK
        STZ  ALLOW         * during Berkeley green, ignore "new" requests
        LD1  SIXTEEN
        SUB  ONE,1         * SIXTEEN-? we need 8 s = 16×½s -> use CNT=16 and break after 16
BK8W    JSJ  HALF
        DEC1 1
        J1P  BK8W

        LD1  EIGHT
BKFL    LDX  X_BK_DWON
        JSJ  HALF
        LDX  X_BK_DWOFF
        JSJ  HALF
        DEC1 1
        J1P  BKFL

        LDX  X_BK_DWON
        LD1  EIGHT
BKDW4   JSJ  HALF
        DEC1 1
        J1P  BKDW4

* ----- Berkeley amber 5 s
        LDX  X_BK_AMBER
        INC  ALLOW ONE     * during BK amber, new requests should be queued
        LD1  TEN           * 5 s = 10 × ½ s
BKAMB   JSJ  HALF
        DEC1 1
        J1P  BKAMB
```

```
* ----- back to Del Mar green; cycle restarts
        LDX  X_DM_GREEN
        STZ  ET
        JMP  DMGREEN

* ========================= ½-second delay + request sampler =============

* HALF: Busy-wait for exactly 0.5 s (per HS calibration) and
*       if ALLOW=1 and overflow toggle is ON, set REQP←1.
HALF    STJ  RET
        LD2  HS
H1      DEC2 1
        J2P  H1
* Check overflow request switch
        LDA  ALLOW
        JZ   RETU
        JOV  SETR           * Jumps iff switch has been pressed; clears OV
        JMP  RETU
SETR    INC  REQP ONE
RETU    JMP  RET

        END  START
```

How it works

- Default is Del Mar green + WALK, Berkeley red + DON'T WALK.
- The loop labelled `DMLP` keeps Del Mar green and samples the overflow every ½ s. A request is armed only when Del Mar is green or amber, or when Berkeley is amber (i.e., whenever Berkeley is not green). This is managed by the `ALLOW` flag.
- Once (i) a request is pending and (ii) Del Mar has been green for at least 30 s, we launch the 12-s pre-amber (8 s flash + 4 s steady DON'T WALK), then 8 s amber, then give Berkeley 20 s green (with its own flash/steady sequence), 5 s amber, and return to Del Mar green.
- During Berkeley green we purposely ignore new presses; people cross on that cycle. During other phases we queue a press (by setting `REQP`), guaranteeing that the next Del Mar green will terminate at the earliest lawful moment (30 s with a full 12-s pre-amber).

  Calibrate `HS` once for your MIX timing so that the loop under label `DELAY/H1` takes exactly half a second. After that, all required durations are exact by construction.

21 [28]

Here's a complete MIXAL program that builds the 23×23 magic square by the classical "Siamese" (Ibn al-Haytham) rule described in the problem, stores it in memory, and prints all 23 rows on the line printer.

- We index rows/columns 1..23.
- Cell (i, j) lives at address `BASE + 23*(i-1) + j`.
- Start with 1 just below the center: $(i, j) = (13, 12)$.
- Step down-right; when that destination is already filled, drop two rows (same column).
- Printing puts each value as three right-aligned digits followed by a blank, 23 per line.

```
* TAOCP v1 §1.3.2 - Ex.21  Magic square (order 23) with Siamese method
* rI1   i (row 1..23), rI2   j (col 1..23), rI3   addr pointer, rI4   line word ptr

        ORIG 3000
PRN     EQU 18

* ---- character helpers -------------------------------------------------------
SPCW    ALF 00000          * one byte blank at (1:1)
ZOW     ALF 00000          * '0' at (1:1)
SPC     CON 0
DO      CON 0

* ---- constants ---------------------------------------------------------------
N       CON 23
BASE    CON 2000           * table base: uses 2000..2528 (529 cells)
LINE    EQU 2600           * 24-word printer line buffer (safe, out of the way)

ONE     CON 1
TWO     CON 2
TW3     CON 23
N2      CON 529            * N^2

* ---- work cells --------------------------------------------------------------
I       CON 0
J       CON 0
K       CON 0              * number to place
ADDR    CON 0
HUND    CON 0
TENS    CON 0
ONES    CON 0
RET     CON 0
```

```
* ====================== main ==================================================
START   LDA  SPCW(1:1);  STA SPC
        LDA  ZOW(1:1);   STA DO

* clear the 23×23 table to 0
        ENT1 1
CLRROW  ENT2 1
CLRJ    ENTA BASE
        ADD  I
        DEC  A 1
        MUL  N
        ADD  J
        STA  ADDR
        LD3  ADDR
        STZ  0,3
        INC2 1
        LDA  J
        CMPA N
        JL   CLRJ
        INC1 1
        LDA  I
        CMPA N
        JL   CLRROW

* initial position: just below the middle square (i=13, j=12)
        ENTA 13;  STA I
        ENTA 12;  STA J
        ENTA 1;   STA K

* ------------- fill loop: K from 1..N^2 --------------------------------------
PLACE   JSJ  ATIJ          * rI3 points to (i,j)
        LDA  K
        STA  0,3           * put K
        LDA  K
        CMPA N2
        JE   PRINTALL      * done filling

* compute tentative (i+1, j+1) with wrap
        LDA  I; ADD ONE; CMPA N; JLE IOK; SUB N
IOK     STA  HUND          * reuse HUND as temp (i') to save a cell
        LDA  J; ADD ONE; CMPA N; JLE JOK; SUB N
JOK     STA  TENS          * temp (j')
```

```
* empty? if yes, move there; else i ← i+2 mod N (j unchanged)
        ENTA BASE
        ADD  HUND
        DEC  A 1
        MUL  N
        ADD  TENS
        STA  ADDR
        LD3  ADDR
        LDA  0,3
        JZ   MOVEIJ
        LDA  I; ADD TWO; CMPA N; JLE IDONE; SUB N
IDONE   STA  I
        JMP  NEXTK
MOVEIJ  LDA  HUND; STA I
        LDA  TENS; STA J

NEXTK   LDA  K; ADD ONE; STA K
        JMP  PLACE

* ---------------- compute rI3=address for (I,J) -------------------------------
ATIJ    STJ  RET
        ENTA BASE
        ADD  I
        DEC  A 1
        MUL  N
        ADD  J
        STA  ADDR
        LD3  ADDR
        JMP  RET

* ======================= printing ==========================================

PRINTALL
        ENT1 1
ROWLP   ENT2 1
        LD4  LINE

COLLP   * fetch value a(i,j)
        ENTA BASE
        ADD  I
        DEC  A 1
        MUL  N
```

```
        ADD  J
        STA  ADDR
        LD3  ADDR
        LDA  0,3

* extract 3 digits (hundreds, tens, ones) with two divisions
        ENTX 0
        DIV  =10=
        STX  ONES
        LDA  A
        ENTX 0
        DIV  =10=
        STX  TENS
        STA  HUND

* byte(1) ← hundreds or blank
        LDA  HUND
        JZ   HBL
        ADD  D0
        STA  0,4(1:1)
        JMP  TBYTE
HBL     LDA  SPC
        STA  0,4(1:1)

* byte(2) ← tens or blank (if hundreds and tens both 0)
TBYTE   LDA  HUND
        JNZ  TDIG
        LDA  TENS
        JZ   TBL
TDIG    LDA  TENS
        ADD  D0
        STA  0,4(2:2)
        JMP  OPUT
TBL     LDA  SPC
        STA  0,4(2:2)

* byte(3) ← ones (always a digit)
OPUT    LDA  ONES
        ADD  D0
        STA  0,4(3:3)

* byte(4) and (5) ← blanks (separator + tidy)
```

```
        LDA  SPC
        STA  0,4(4:4)
        STA  0,4(5:5)

        INC4 1
        INC2 1
        LDA  J
        CMPA N
        JL   COLLP

* print the 23-word line
        OUT  LINE(PRN)

        INC1 1
        LDA  I
        CMPA N
        JL   ROWLP

        HLT
        END  START
```

Notes

- Addressing. `ATIJ` computes the address for the current (i, j) and sets `rI3` so we can `LDA/STA 0,3`.
- Wrap-around. After `i+1` or `i+2` we compare to `N` and subtract `N` if needed (the plane-tiling rule).
- Printing.** Each of the 23 output words on a line holds `ddd` (three digits right-aligned plus a blank separator; byte 5 blank too). That's simple and fits the printer's 24-word line.
- Memory used: table `2000..2528` (529 words) and line buffer `2600..2623`.

Run this and you'll get the 23×23 magic square, each row nicely formatted on the printer.

22 [31]

Here's a fast MIXAL solution of the Josephus problem that prints the order-of-execution for $n = 24$, $m = 11$. It keeps a circular doubly-linked list in two arrays `NEXT[i]` and `PREV[i]` so that each kill is $O(m)$ pointer hops, and removal is $O(1)$. It finally prints, for each man $i = 1, \ldots, 24$, the number 1..24 telling when he is executed (the format is two digits per man, zero-padded).

```
* EX 1.3.2 - Exercise 22 (Josephus): n = 24, m = 11
* Output: for i=1..24, the execution order of man i.

        ORIG 3000
PRN     EQU   18
N       EQU   24
M       EQU   11

LINE    EQU   3900                * 24-word printer line

* ------- helpers for printing ----------------------------------------------
ZOW     ALF  00000
SPCW    ALF  00000
D0      CON  0                    * '0' byte  (from ZOW(1:1))
SPC     CON  0                    * blank     (from SPCW(1:1))

* ------- tables (24 words each) --------------------------------------------
NEXT    EQU   3600                * NEXT[i] : next alive after i
PREV    EQU   3624                * PREV[i] : previous alive before i
ORD     EQU   3648                * ORD[i]  : kill order of i (1..24)

* ------- scratch -----------------------------------------------------------
CUR     CON   0                   * current position in the circle
T       CON   0                   * current kill number (1..N)
TMP     CON   0
TEN     CON   10

RET     CON   0
ONES    CON   0
TENS    CON   0

* ========================  MAIN  ==========================================

START   LDA  ZOW(1:1);  STA D0
        LDA  SPCW(1:1); STA SPC

* ---- build the circular doubly linked list for 1..N
        ENT1  1
INITLP  ENTA  0,1                 * A ← i
        ADD   =1=
        CMPA  =N+1=               * wrap next
        JL    OKN
        ENTA  1
```

```
OKN     STA   NEXT,1              * NEXT[i] ← (i%N)+1

        ENTA  0,1
        SUB   =1=
        JG    OKP
        ENTA  =N=
OKP     STA   PREV,1              * PREV[i] ← ((i-2+N)%N)+1

        INC1  1
        CMP1  =N+1=
        JL    INITLP

* ---- Josephus loop
        ENT1  1                   * CUR ← 1
        STA   CUR
        ENTA  1;  STA  T          * T ← 1  (first kill)

KILL    * advance m-1 steps: CUR ← NEXT^(m-1)(CUR)
        ENT2  M
        DEC2  1                   * rI2 ← m-1
ADVLP   LD3   CUR
        LDA   NEXT,3
        STA   CUR
        DEC2  1
        J2P   ADVLP

* mark execution order
        LD3   CUR
        LDA   T
        STA   ORD,3

* splice CUR out: NEXT[PREV[CUR]] ← NEXT[CUR];  PREV[NEXT[CUR]] ← PREV[CUR]
        LDA   PREV,3
        STA   TMP                 * TMP ← prev
        LDA   NEXT,3
        STA   CUR                 * CUR ← next (the one after the dead guy)

        LD4   TMP                 * write NEXT[prev] ← next
        STA   NEXT,4

        LD3   CUR                 * write PREV[next] ← prev
        LDA   TMP
        STA   PREV,3
```

```
* stop after N kills
        LDA   T
        CMPA  =N=
        JE    PRINT

        ADD   =1=
        STA   T
        JMP   KILL

* ============================ PRINT ==========================================

PRINT   LD4   LINE                * build a 24-word line
        ENT1  1
ROW     LD3   ORD,1               * value = ORD[i]
        LDA   0,3
        ENTX  0
        DIV   TEN
        STX   ONES
        STA   TENS

        LDA   TENS;  ADD D0;      STA 0,4(1:1)   * tens
        LDA   ONES;  ADD D0;      STA 0,4(2:2)   * ones
        LDA   SPC;                STA 0,4(3:3)   * space
        STA   0,4(4:4);           STA 0,4(5:5)   * padding blanks

        INC4  1
        INC1  1
        CMP1  =N+1=
        JL    ROW

        OUT   LINE(PRN)
        HLT

        END  START
```

What it prints (two digits per man, in order 1..24)

For $n = 24$, $m = 11$ the order-of-execution is

```
15 12 22 08 16 11 23 21 03 05 01 17 10 07 24 19 20 18 09 14 04 02 13 06
```

meaning: man 1 is 15th to die, man 2 is 12th, …, man 24 is 6th.

23 [37]

Below is a complete MIXAL program that reads a 23×23 matrix of zeros and ones (one row per card on device 16, with columns 1–23 containing the digits '0'/'1'), converts it to a crossword diagram, removing edge-connected black areas, numbering the white squares that start an Across or Down word, and prints the final picture on the line printer (device 18) exactly as specified: each puzzle cell is rendered by 5 columns $\times$ 3 rows of printer characters.

I have kept the code heavily annotated and very close to the text of the exercise, so you can change it easily (e.g., to other sizes).

What the program prints

- **White (unnumbered)** cell → top row `"    +"`, middle row `"    +"`, bottom `"+++++"`.
- **White (numbered)** cell, number `nn` (2 digits, leading zero) → top `" nn +"`, middle `" +"`, bottom `"+++++"`.
- **Black** cell → all three rows `"+++++"`.
- **Outside** cells (that became `-1` after the edge sweep): If the cell to the right is also `-1`, the right border is suppressed (`"    "` on the two upper rows); if the cell below** is `-1`, the bottom border is suppressed (`"    "` on the bottom row). This produces a clean rectangular outside frame like Fig. 19.

```
* TAOCP v1 §1.3.2 - Ex.23  Crossword diagram from a 23x23 0/1 matrix
* Input : 23 cards on device 16, cols 1..23 are digits 0/1 (one row per card)
* Output: puzzle drawn on device 18 (line printer), as in Fig. 19
* Method : Add a -1 border, flood -1 into edge-connected 1's, number starts,
*          then render 3 printer lines per puzzle row, 5 bytes per cell.

        ORIG 3000
PRN     EQU   18
CARD    EQU   16

* --------------- character codes we need (portable via ALF) ------------------
SPCW    ALF  00000           * blank in (1:1)
PLUSW   ALF  +0000           * '+' in (1:1)
ZOW     ALF  00000           * '0' in (1:1)

SPC     CON  0
PLUS    CON  0
DO      CON  0

* --------------- general constants and pointers ------------------------------
N       CON   23
NP1     CON   24
```

```
N2      CON   529
FIVE    CON   5
THREE   CON   3
ONE     CON   1
TWO     CON   2
ZERO    CON   0
NEG1    CON   -1
TEN     CON   10

* --------------- storage layout ---------------------------------------------
* use a 25x25 board B(i,j), 0..24 in both directions
* (board base at B), and a NUM(i,j) table for numbers (1..K)
B       EQU   5000                    * 25*25 = 625 words used
NUM     EQU   B+625
LINE    EQU   NUM+625                 * 24-word print line buffer
CARDW   EQU   LINE+24                 * 24-word card buffer

* --------------- scratch ----------------------------------------------------
I       CON 0
J       CON 0
K       CON 0
CHG     CON 0
ADDR    CON 0
T       CON 0
Q       CON 0
R       CON 0
RET     CON 0
D1      CON 0
D10     CON 0

* ====================== main ==================================================
START   LDA  SPCW(1:1);  STA SPC
        LDA  PLUSW(1:1); STA PLUS
        LDA  ZOW(1:1);   STA DO

* ---------- zero NUM and fill B with -1 border, 0 inside ---------------------
        ENT1 0
BORDR   ENT2 0
        ENTA  B
        ADD   I
        MUL   =25=
        ADD   J
```

```
        STA   ADDR
        LD3   ADDR
* set B(i,j):
        LDA   I
        JE    SETM1
        CMPA  NP1
        JE    SETM1
        LDA   J
        JE    SETM1
        CMPA  NP1
        JE    SETM1
        STZ   0,3                     * interior initial 0
        JMP   NEXTB
SETM1   LDA   NEG1
        STA   0,3
NEXTB   * NUM(i,j) ← 0
        LDA   ZERO
        STA   NUM- B,3                * same index, other table

        INC2  1
        LDA   J
        CMPA  =25=
        JL    BORDR
        INC1  1
        LDA   I
        CMPA  =25=
        JL    BORDR

* ------------------ read 23 cards; store 0/1 into B(1..23,1..23) -------------
        ENT1 1                         * row i = 1..23
RDROW   IN   CARDW(CARD)
        JBUS *      (CARD)             * wait for card
        ENT2 1
        LD4  CARDW
RDCOL   LDA  0,4(0:0)                  * next card byte
        ADD  =0=                       * put in A with sign '+'
        SUB  D0
        JZ   ISZERO
        ENTA 1
        JMP  STORE
ISZERO  ENTA 0
STORE   STX  R                         * (X changed by ADD/SUB? guard)
```

```
        ENTA  B
        ADD   I
        MUL   =25=
        ADD   J
        STA   ADDR
        LD3   ADDR
        LDA   ZERO
        ADD   A                       * A already 0 or 1
        STA   0,3

        INC4  1
        INC2  1
        LDA   J
        CMPA  N
        JL    RDCOL

        INC1  1
        LDA   I
        CMPA  N
        JLE   RDROW

* ------------------ flood -1 from border into all edge-connected 1's ---------
FLOOP   STZ  CHG
        ENT1 1
FI      ENT2 1
FJ      * process only original interior domain 1..23
        ENTA  B
        ADD   I
        MUL   =25=
        ADD   J
        STA   ADDR
        LD3   ADDR
        LDA   0,3
        CMPA  =1=
        JNE   NEXTJ

        * if a 1 is adjacent (4-neigh) to -1, turn it into -1
        ENTA  B
        ADD   I
        DEC   A ONE
        MUL   =25=
        ADD   J
```

```
        STA   T
        LDA   0,T
        CMPA  NEG1
        JE    MAKE
        ENTA  B
        ADD   I
        INC   A ONE
        MUL   =25=
        ADD   J
        STA   T
        LDA   0,T
        CMPA  NEG1
        JE    MAKE
        ENTA  B
        ADD   I
        MUL   =25=
        ADD   J
        DEC   A ONE
        STA   T
        LDA   0,T
        CMPA  NEG1
        JE    MAKE
        ENTA  B
        ADD   I
        MUL   =25=
        ADD   J
        INC   A ONE
        STA   T
        LDA   0,T
        CMPA  NEG1
        JNE   NEXTJ
MAKE    LDA   NEG1
        STA   0,3
        ENTA  1
        STA   CHG
NEXTJ   INC2  1
        LDA   J
        CMPA  N
        JLE   FJ
        INC1  1
        LDA   I
        CMPA  N
```

```
        JLE   FI
        LDA   CHG
        JNZ   FLOOP

* ------------------ numbering: starts of across / down -----------------------
        STZ   K
        ENT1  1
NI      ENT2  1
NJ      ENTA  B
        ADD   I
        MUL   =25=
        ADD   J
        STA   ADDR
        LD3   ADDR
        LDA   0,3
        JZ    WHITEOK
        JMP   NXTJ
WHITEOK * check down start: (i==1 or B(i-1,j)!=0) and (i<23 and B(i+1,j)==0)
        ENTA  0
        STA   T
        LDA   I
        JNE   CI1
        ENTA  1
        STA   T
        JMP   C2
CI1     ENTA  B
        ADD   I
        DEC   A ONE
        MUL   =25=
        ADD   J
        LDA   0,A
        JNZ   C2
        ENTA  0
        STA   T
C2      LDA   T
        JZ    NOTD
        LDA   I
        CMPA  N
        JE    NOTD
        ENTA  B
        ADD   I
        INC   A ONE
```

```
        MUL   =25=
        ADD   J
        LDA   0,A
        JNZ   NOTD
        ENTA  1
        STA   T                 * down start
NOTD    * check across start
        LDA   T
        JNZ   NUMBER
        LDA   J
        JNE   CJ1
        ENTA  1
        STA   T
        JMP   C4
CJ1     ENTA  B
        ADD   I
        MUL   =25=
        ADD   J
        DEC   A ONE
        LDA   0,A
        JNZ   C4
        ENTA  0
        STA   T
C4      LDA   T
        JNZ   NUMBERC
        LDA   J
        CMPA  N
        JE    NXTJ
        ENTA  B
        ADD   I
        MUL   =25=
        ADD   J
        INC   A ONE
        LDA   0,A
        JNZ   NXTJ
NUMBERC ENTA  1
        STA   T
NUMBER  LDA   K
        ADD   ONE
        STA   K
        STA   NUM- B,3
NXTJ    INC2  1
```

```
        LDA   J
        CMPA  N
        JLE   NJ
        INC1  1
        LDA   I
        CMPA  N
        JLE   NI

* ------------------ render to printer ----------------------------------------
        ENT1  1
ROW     * we emit 3 printer lines for row i
        LD3   =1=
        STA   T                * t = which subrow (1..3)

SUBROW  LD4   LINE
        ENT2  1
COL     ENTA  B
        ADD   I
        MUL   =25=
        ADD   J
        STA   ADDR
        LD3   ADDR
        LDA   0,3              * val = B(i,j)
        JN    OUTM1            * negative → outside
        JZ    OUTWHT           * zero → white
        JMP   OUTBLK           * positive → black

* ----- black square: '+++++' on all 3 subrows
OUTBLK  LDA   PLUS
        STA   0,4(1:1)
        STA   0,4(2:2)
        STA   0,4(3:3)
        STA   0,4(4:4)
        STA   0,4(5:5)
        JMP   NEXTC

* ----- white square
OUTWHT  LDA   T
        CMPA  =1=
        JNE   W2
        * top row: maybe a number
        LDA   NUM- B,3
```

```
        JZ    WTOPS
        ENTX  0
        DIV   TEN
        STX   D1
        STA   D10
        LDA   SPC;  STA 0,4(1:1)
        LDA   D10; ADD D0; STA 0,4(2:2)
        LDA   D1;  ADD D0; STA 0,4(3:3)
        LDA   SPC; STA 0,4(4:4)
        LDA   PLUS;STA 0,4(5:5)
        JMP   NEXTC
WTOPS   LDA   SPC; STA 0,4(1:1); STA 0,4(2:2); STA 0,4(3:3); STA 0,4(4:4)
        LDA   PLUS; STA 0,4(5:5)
        JMP   NEXTC
W2      LDA   T
        CMPA  =2=
        JNE   W3
        * middle row: "    +"
        LDA   SPC; STA 0,4(1:1); STA 0,4(2:2); STA 0,4(3:3); STA 0,4(4:4)
        LDA   PLUS; STA 0,4(5:5)
        JMP   NEXTC
W3      * bottom row: "+++++"
        LDA   PLUS
        STA   0,4(1:1); STA 0,4(2:2); STA 0,4(3:3); STA 0,4(4:4); STA 0,4(5:5)
        JMP   NEXTC

* ----- outside (-1) square: suppress borders that abut -1 at right/below
OUTM1   LDA   T
        CMPA  =3=
        JE    M1BOT
        * top/mid subrows
        * if right neighbor is -1 → five blanks; else "    +"
        ENTA  B
        ADD   I
        MUL   =25=
        ADD   J
        INC   A ONE
        LDA   0,A
        CMPA  NEG1
        JE    ALLSP
        LDA   SPC; STA 0,4(1:1); STA 0,4(2:2); STA 0,4(3:3); STA 0,4(4:4)
        LDA   PLUS; STA 0,4(5:5)
```

```
        JMP   NEXTC
ALLSP   LDA   SPC
        STA   0,4(1:1); STA 0,4(2:2); STA 0,4(3:3); STA 0,4(4:4); STA 0,4(5:5)
        JMP   NEXTC
M1BOT   * bottom subrow
        ENTA  B
        ADD   I
        INC   A ONE
        MUL   =25=
        ADD   J
        LDA   0,A
        CMPA  NEG1
        JE    ALLSP
        LDA   PLUS
        STA   0,4(1:1); STA 0,4(2:2); STA 0,4(3:3); STA 0,4(4:4); STA 0,4(5:5)

NEXTC   INC4 1
        INC2 1
        LDA  J
        CMPA N
        JLE  COL

        OUT  LINE(PRN)

        LDA  T
        ADD  ONE
        STA  T
        CMPA THREE
        JLE  SUBROW

        INC1 1
        LDA  I
        CMPA N
        JLE  ROW

        HLT
        END  START
```

Notes, hints, and adjustments

- **Card input**: the code expects 23 cards; each of the first 23 bytes is `'0'` or `'1'`. If your MIX monitor uses a different input device, just change `CARD=16` and the `IN/JBUS` calls accordingly.

- **Edge removal** (the "−1 flood"): we iterate until a full pass makes no change; for a 23×23 matrix this is trivial in time.
- **Numbering rule**: a white square gets a number if it begins a **down** word (white below, not white above) or an **across** word (white right, not white left). Numbers are two digits with a **leading zero** as in the book's figure.
- **Exact line lengths**: each printer line has 23 boxes × 5 bytes = 115 bytes; the line buffer has 24 words (120 bytes), so the last five are blank and harmless.
- You can change `B`, `NUM`, or `LINE` bases if they clash with your monitor; all addresses are absolute here to keep the program self-contained.

This produces a printout that matches the layout in Fig. 19: a plus-bordered diagram, numbers placed at the correct starts, black squares filled with '+', and edge-connected black regions removed.

1.3.3 Applications to Permutation

1 [02]

1) It's a permutation. On residues mod 7, multiplication by 2 has an inverse because $\gcd(2,7) = 1$. In fact $2 \cdot 4 \equiv 1 \pmod 7$, so the inverse map is $x \mapsto 4x \pmod 7$. A map with an inverse is bijective, hence a permutation.

2) Cycle decomposition. Compute $f(x) = 2x \bmod 7$:

x	0	1	2	3	4	5	6
$f(x)$	0	2	4	6	1	3	5

Now follow orbits:

- $0 \mapsto 0 \quad (0)$.
- $1 \mapsto 2 \mapsto 4 \mapsto 1 \quad (1\ 2\ 4)$.
- $3 \mapsto 6 \mapsto 5 \mapsto 3 \quad (3\ 6\ 5)$.

Thus

$$\boxed{(0)(1\ 2\ 4)(3\ 6\ 5)}.$$

By the section's convention, singleton cycles are often suppressed, giving $\boxed{(1\ 2\ 4)(3\ 6\ 5)}$, which matches the book's answer.

3) (Why the structure looks like this.) The nonzero residues mod 7 form a cyclic group of order 6. The element 2 has order 3 (since $2^3 \equiv 1 \pmod 7$), so its action splits $\{1, \dots, 6\}$ into 3-cycles; hence exactly two 3-cycles plus the fixed point 0.

2 [10]

We want to transform the tuple

$$(a, b, c, d, e, f) \longrightarrow (c, d, f, b, e, a)$$

using only exchanges (swaps $x \leftrightarrow y$) and no auxiliary variable. The target permutation is the product of disjoint cycles

$$(a\ c\ f)\ (b\ d)\ (e),$$

since $a \to c \to f \to a$, $b \leftrightarrow d$, and e is fixed.

A k-cycle $(x_1\, x_2\, \ldots\, x_k)$ can be realized by $k-1$ swaps:

$$(x_1 \leftrightarrow x_2),\ (x_2 \leftrightarrow x_3),\ \ldots,\ (x_{k-1} \leftrightarrow x_k).$$

Apply this to our cycles:

1. Realize $(a\ c\ f)$ with two swaps

- $a \leftrightarrow c$
- $c \leftrightarrow f$

2. Realize $(b\ d)$ with one swap

- $b \leftrightarrow d$

After these three exchanges the tuple is exactly (c, d, f, b, e, a). So one valid swap-only sequence is:

$$\boxed{a \leftrightarrow c, \quad c \leftrightarrow f, \quad b \leftrightarrow d.}$$

(That's precisely the concise answer Knuth lists.)

The two swaps on (a, c, f) rotate the triple once (a 3-cycle), and the single swap on (b, d) flips those two; disjoint cycles act independently, so doing the 3-cycle then the transposition yields the desired permutation.

3 [03]

We're asked to compute (left-to-right, per Knuth's convention) the product

$$\pi_1 \times \pi_2 = \begin{pmatrix} a & b & c & d & e & f \\ b & d & c & a & f & e \end{pmatrix} \times \begin{pmatrix} a & b & c & d & e & f \\ c & d & f & b & e & a \end{pmatrix},$$

and express the answer in two-line notation. Knuth multiplies permutations left-to-right (apply the left factor, then the right), see the discussion around Eq. (4).

Track each letter through π_1 then π_2:

- $a \xrightarrow[\pi_1]{\pi_1} b \xrightarrow[\pi_2]{\pi_2} d \quad a \mapsto d.$
- $b \xrightarrow[\pi_1]{\pi_1} d \xrightarrow[\pi_2]{\pi_2} b \quad b \mapsto b.$
- $c \xrightarrow[\pi_1]{\pi_1} c \xrightarrow[\pi_2]{\pi_2} f \quad c \mapsto f.$
- $d \xrightarrow[\pi_1]{\pi_1} a \xrightarrow[\pi_2]{\pi_2} c \quad d \mapsto c.$
- $e \xrightarrow[\pi_1]{\pi_1} f \xrightarrow[\pi_2]{\pi_2} a \quad e \mapsto a.$
- $f \xrightarrow[\pi_1]{\pi_1} e \xrightarrow[\pi_2]{\pi_2} e \quad f \mapsto e.$

Therefore

$$\boxed{\begin{pmatrix} a & b & c & d & e & f \\ d & b & f & c & a & e \end{pmatrix}}.$$

If you also want the cycle form, it's $(a\,d\,c\,f\,e)(b)$, i.e., $(a\,d\,c\,f\,e)$ with b fixed.

4 [10]

Problem. Express

$$(a\,b\,d)\,(e\,f)\,(a\,c\,f)\,(b\,d)$$

as a product of disjoint cycles.

Convention. In this section, permutation products are applied left-to-right: first $(a\,b\,d)$, then $(e\,f)$, then $(a\,c\,f)$, then $(b\,d)$. (See the section text immediately preceding the exercises.)

Compute the image of each symbol $x \in \{a, b, c, d, e, f\}$ by chasing it through the four cycles:

- $a \xrightarrow{(a\,b\,d)} b \xrightarrow{(e\,f)} b \xrightarrow{(a\,c\,f)} b \xrightarrow{(b\,d)} d \quad a \mapsto d.$
- $b \xrightarrow{(a\,b\,d)} d \xrightarrow{(e\,f)} d \xrightarrow{(a\,c\,f)} d \xrightarrow{(b\,d)} b \quad b \mapsto b.$

- $c \xrightarrow{(a\,b\,d)} c \xrightarrow{(e\,f)} c \xrightarrow{(a\,c\,f)} f \xrightarrow{(b\,d)} f \quad c \mapsto f.$
- $d \xrightarrow{(a\,b\,d)} a \xrightarrow{(e\,f)} a \xrightarrow{(a\,c\,f)} c \xrightarrow{(b\,d)} c \quad d \mapsto c.$
- $e \xrightarrow{(a\,b\,d)} e \xrightarrow{(e\,f)} f \xrightarrow{(a\,c\,f)} a \xrightarrow{(b\,d)} a \quad e \mapsto a.$
- $f \xrightarrow{(a\,b\,d)} f \xrightarrow{(e\,f)} e \xrightarrow{(a\,c\,f)} e \xrightarrow{(b\,d)} e \quad f \mapsto e.$

Thus the permutation maps

$$a \to d, \quad d \to c, \quad c \to f, \quad f \to e, \quad e \to a, \quad b \to b,$$

which is the 5-cycle $(a\,d\,c\,f\,e)$ with b fixed (singleton cycles are suppressed). Therefore

$$\boxed{(a\,d\,c\,f\,e)}.$$

This matches the book's listed answer.

5 [M10]

If a permutation has r disjoint cycles of lengths $l_1, l_2, \dots, l_r$ (no 1-cycles shown), then:

- Each l_i-cycle can be rotated in l_i ways without changing the permutation.
- The r cycles can be ordered arbitrarily (they commute), giving $r!$ orderings.

Hence the number of distinct written forms is

$$\boxed{l_1\, l_2 \cdots l_r \cdot r!}.$$

Apply to Eq. (3). There the permutation is $(a\,c\,f)(b\,d)$, i.e., one 3-cycle and one 2-cycle, so

$$3 \times 2 \times 2! \;=\; \boxed{12}.$$

6 [M28]

Knuth's timing formula for Program A (excluding I/O) under the "all blanks at the extreme right" assumption is

$$(112NX + 304X - 2M - Y + 11U + 2V - 11)\, u,$$

with X, Y, M, N, U, V defined just below Eq. (18)–(19). In the text it's stated that this assumption prevents two things from ever happening: the jump on line 32 and the "skip past blanks" loop at line 71.

If we remove that assumption, let

$$W = \#\text{ of blank words in the input (excluding the final "=")} = 16X - 1 - Y.$$

using $B + C = 16X - 1$ and $Y = \#$ of nonblank words.

- Each internal blank that's encountered during the preconditioning pass (lines 23–38) causes the jump at line 32 to be taken once, adding W unit-times.
- Each internal blank that's encountered during the main scan (A3) triggers the skip at line 71 once, adding another W unit-times.

So the timing increases by $2W = (32X - 2 - 2Y)$ unit-times. The new total (still excluding I/O) is

$$\boxed{(112NX + 304X - 2M - Y + 11U + 2V - 11 + 2(16X - 1 - Y))\, u.}$$

Equivalently,

$$(112NX + 336X - 2M - 3Y + 11U + 2V - 13)\, u.$$

(Variables and the original timing setup are given around Eqs. (7), (18)–(19).)

7 [10]

Here are the parameters for the specific input (6) $(a\,c\,f\,g)(b\,c\,d)(a\,e\,d)(f\,a\,d\,e)(b\,g\,f\,a\,e)$:

- X (cards of input) = 2 (the book shows (6) punched on two cards).
- Y (nonblank fields, excluding the final "=") = 29 – words: 6 + 5 + 5 + 6 + 7 = 29.
- M (cycles in input) = 5 (the five parenthesized cycles).
- N (distinct element names) = 7 (a,b,c,d,e,f,g).
- Output permutation for (6) is $(a\,d\,g)(c\,e\,b)(f)$, so U (cycles in output, incl. singletons) = 3, V (singleton cycles) = 1.

Using Knuth's time formula (exclusive of I/O) $(112NX + 304X - 2M - Y + 11U + 2V - 11)\,u$, with the meanings in (19):

$$\begin{aligned} T &= 112(7)(2) + 304(2) - 2(5) - 29 + 11(3) + 2(1) - 11 \\ &= \boxed{2161\,u}. \end{aligned}$$

8 [23]

Yes. Scan left→right and maintain the inverse relation: make $T[j] = i$ exactly when $x_i \mapsto x_j$ in the permutation (so x_j's predecessor is x_i). After the scan, construct the cycle form by tracing right→left with the table T (i.e., repeatedly replace the current element by its predecessor from T), which yields the same permutation in cycle notation.

How to implement:

- On reading a cycle $(x_{i_1}\ x_{i_2}\ \ldots\ x_{i_k})$ left→right, set $T[i_2] = i_1,\ T[i_3] = i_2,\ \ldots,\ T[i_k] = i_{k-1},\ T[i_1] = i_k$.
- When all cycles are processed, output cycles by picking an unused name a and repeatedly replacing $a \leftarrow T[a]$ until you return to the start, marking as you go.

9 [10]

1) First get the actual permutation F

Chase each letter through the whole product (left factor first, then the next, …):

- $a \to c \to c \to e \to d \to d \Rightarrow a \mapsto d$
- $b \to b \to c \to c \to c \to e \Rightarrow b \mapsto e$
- $c \to f \to d \to d \to d \to b \Rightarrow c \mapsto b$
- $d \to d \to b \to a \to a \to g \Rightarrow d \mapsto g$

- $e \to e \to e \to a \to a \to a \Rightarrow e \mapsto a$
- $f \to g \to g \to g \to f \to f \Rightarrow f \mapsto f$
- $g \to a \to a \to a \to g \to g \Rightarrow g \mapsto a$

So the mapping is

$$F = \{a \to d,\ d \to g,\ g \to a\}\ \{c \to b,\ b \to e,\ e \to a\}\ \{f \to f\}.$$

In cycle form that's

$$(a\,d\,g)\ (c\,b\,e)\ (f)\,,$$

and of course any rotation like $(c\,e\,b)$ denotes the same 3-cycle.

2) Program A (left→right scan): why cycles start with the leftmost available name

Program A builds the permutation while scanning the input text left→right, then prints cycles by:

- keeping a `USED[x]` mark for each name,
- repeatedly finding the leftmost name in the input whose `USED` mark is false,
- starting a new printed cycle at that name and following images $x \mapsto F[x]$ until it returns, marking `USED` along the way.

Concretely for (6):

Left→right order of name occurrences (ignoring parentheses and "=") `a, c, f, g, b, c, d, a, e, d, f, a, d, e, b, g, f, a, e`

The first-appearance index of each name in this order is:

- a: 1, c: 2, f: 3, g: 4, b: 5, d: 7, e: 9.

Now the printing loop:

1. Pick the leftmost unused a (index 1). Follow $a \to d \to g \to a$. Mark $\{a, d, g\}$ and print $(a\,d\,g)$.
2. Next leftmost unused in the original text is c (index 2). Follow $c \to b \to e \to c$. Mark $\{c, b, e\}$ and print $(c\,b\,e)$ (often rotated as $(c\,e\,b)$).
3. Remaining unused is f (index 3). Follow $f \to f$. Print (f).

That's why every cycle that Program A prints begins with the leftmost still-unprinted name in the original left→right input: the algorithm literally chooses the next start that way before walking F.

3) Program B (right→left scan): why cycles start with the last distinct name seen from the right

Program B first scans the input right→left and builds predecessor links T (so $T[y] = x$ when $x \mapsto y$); during this pass it also keeps the order in which distinct names are first encountered from the right.

For (6), the right→left name stream is:

```
e, a, f, g, b, e, d, a, f, d, e, a, d, c, b, g, f, c, a
```

The first time we see each distinct name (while moving right→left) occurs in this order:

$$e,\ a,\ f,\ g,\ b,\ d,\ c.$$

Call that list R2L. After the scan, Program B prints cycles by repeatedly:

- choosing the last name in R2L that is still unused,
- starting the next cycle there and walking predecessors (equivalently, the same cycle as walking images but rotated),
- marking those names used and repeating.

Step-by-step:

1. Last distinct from the right among all names is c (it's the final entry of R2L). Walk the cycle: $c \leftarrow b \leftarrow e \leftarrow c$. Print $(c\,b\,e)$ (often shown as $(c\,e\,b)$). Mark $\{c, b, e\}$ used.
2. Among the remaining names $\{a, d, f, g\}$, the last (furthest to the left in R2L) is d. Walk: $d \leftarrow g \leftarrow a \leftarrow d$. Print $(d\,g\,a)$ (same cycle as $(a\,d\,g)$). Mark $\{d, g, a\}$.
3. Remaining is f. Walk: $f \leftarrow f$. Print (f).

That's exactly the rule: Program B's cycle starts are the last distinct names encountered during the right→left pass, taken anew each time after removing already-printed elements.

4) The general reason (independent of the example)

- Program A seeds each cycle by a left→right scan for the first unprinted symbol; hence starts are leftmost-available. The subsequent walk doesn't change the start—it just completes that orbit under F.
- Program B fixes the cycle starts at build time: the right→left pass commits to a "most recently discovered, still-unused" symbol as the next start, which is precisely the last distinct from the right among those not yet printed. Walking via predecessors (or, equivalently, images) just rotates the cycle to begin at that chosen start.

5) Sanity checks

- The two outputs may rotate cycles differently, but they encode the same permutation F.
- If you reorder or space the input differently, Program A's starts can change (because "leftmost" refers to the input text order), and Program B's starts can change with the right→left discovery order; both behaviors match the book's statements.

10 [M28]

A. Where the equations come from (Kirchhoff's law on the flow graph)

Kirchhoff's law on a flow graph says: for every node, total entries = total exits (counted over one complete run). In §1.3.3, Knuth labels parts of Program A and uses capital letters $A, B, \ldots$ for "how many times that part executes." Applying the balance equation at the key junctions yields simple linear relations among these counters. The text gives the same kind of relations already (see the group of equations around the analysis of Program A), and we reproduce the ones Ex. 10 wants.

From node-by-node balances on the Program A flow (as in the section's derivation), we obtain:

$$\boxed{\begin{array}{ll} (1) & A = 1 + C - D, \\ (2) & B = A + J + P - 1, \\ (3) & C = B - (P - L), \\ (4) & E = D - L, \\ (5) & G = E, \\ (6) & Q = Z, \\ (7) & W = S. \end{array}}$$

Here is the intuition for each, in words:

- $A = 1 + C - D$. You enter the "initialize/outer loop" block (A) once at program start (that's the "1"), and you re-enter it each time you loop back from scanning (C), except when you immediately peel off to a "done with this cycle" exit counted by D. Net effect: A equals one initial entry plus C returns minus D departures. (This is the standard "entry = one initial + reentries − final exits" pattern.)
- $B = A + J + P - 1$. The scanning/reading block B is reached after each A (except the very last time), and also via two back-edges: one from the cycle-outputting region (J) and another from the "suppress/close" area (P). Summing those, and subtracting the final "no further input" case, yields $B = A + J + P - 1$.

- $C = B - (P - L)$. From each pass through B we typically proceed to C (the main scan), except when an immediate close/suppress path fires; those exceptional bypasses are counted by $(P - L)$ (closes without opening a new cycle). Hence C is the "default" B count minus the bypasses.
- $E = D - L$, then $G = E$. Each time a cycle finishes (D) we either immediately close/suppress (L) or proceed to the "emit/open" work counted by E; so $E = D - L$. And the subsequent sub-task G runs exactly once per E, giving $G = E$.
- $Q = Z$. The names-table pass that establishes a correspondence for a *new* distinct name (Z) is in one-to-one correspondence with creating a new name entry Q (same count).
- $W = S$. Likewise two paired activities (different labels in the listing) occur the same number of times along matched edges, giving equality.

These "balance" equations are exactly the relations the official answer lists under part (1).

B. Interpreting the symbols (connect to $X, Y, M, N, U, ...$)

Now tie each counter to a plain combinatorial quantity. Program A reads permutations in cycle notation from punched cards (16 fields per card), "preconditions" the stream, builds a table of names, multiplies the permutation, and prints cycles. In §1.3.3 the quantities X, Y, M, N, U, V are used to summarize the input and output:

- X: number of cards of input,
- Y: number of nonblank words on the cards (excluding the final "="),
- M: number of cycles in the input,
- N: number of distinct element names,
- U: number of cycles in the output (including singletons),
- V: number of singleton cycles in the output.

With those conventions, Ex. 10 asks for concrete meanings of several counters in terms of X, Y, M, N, U. The official identifications are (and we explain each):

- $\boxed{B = \#\text{ of words of input} = 16X - 1}$. Each card supplies 16 word positions; the very last position is the terminating "=", hence total words processed (including blanks) is $16X - 1$. This is precisely the stream length the scanner sees.
- $\boxed{C = \#\text{ of nonblank words} = Y}$. C counts how many *actual* fields (names or parentheses) are scanned; that's the "nonblank" count Y defined in the section.
- $\boxed{D = C - M}$. Among the C nonblank fields are M right parentheses that immediately terminate cycles; the control that increments D triggers for all other nonblank fields, hence D is the remainder $C - M$. (Equivalently: each of the M cycle closures diverts flow to L, not counted in D.)

- $\boxed{E = D - M}$. After removing the M times we *close* cycles from D, the remaining cases require "emit/open" work, counted by E; the balance gives $E = D - M$. (This aligns with the earlier $E = D - L$, together with $L = M$ below.)
- $\boxed{F = \#\text{ of comparisons in the names-table search}}$. Every nonblank name triggers a symbol-table probe; F totals those name-equality checks. (This is exactly how Program A's table lookup costs are accounted.)
- $\boxed{H = N}$. H is the total number of times we *insert* a previously unseen name; that equals the number of distinct names N.
- $\boxed{K = M}$. K counts the number of times we start a new input cycle (i.e., see '(' that actually begins a cycle), which is exactly the number of input cycles M.
- $\boxed{Q = N}$. Q increments when a new name slot is *created* in the table; that's once per distinct name, hence $Q = N$. (Consistent with $Q = Z$ and "Z is # of new-name events.")
- $\boxed{R = U}$. R is the number of output cycles produced; Program A prints all cycles including singletons, so $R = U$.

These are exactly the identifications listed for part (2) in the answer key.

C. Cross-checks against the section's earlier relations

Two useful consistency checks from the text:

- The section earlier derives other linear constraints (e.g., relations tying G, J, L to the scan passes) and shows that the running time depends only on $G + L$, so we don't need to separate G and L further for timing. This is used later to write the closed-form time in terms of X, Y, M, N, U, V.
- When blanks are *not* all at the extreme right, additional passes over internal blanks add exactly one unit per internal blank to two specific actions (a jump and a skip loop), increasing the total by $2(16X - 1 - Y)$, which matches how B and $C = Y$ enter the count. (This ties back to the timing refinements you used in Ex. 6–7.)

(1) By Kirchhoff's law on the Program A flow,

$$A = 1+C-D, \quad B = A+J+P-1, \quad C = B-(P-L), \quad E = D-L, \quad G = E, \quad Q = Z, \quad W = S.$$

(2) Interpretations in terms of input/output parameters:

$$B = 16X - 1, \quad C = Y, \quad D = C - M, \quad E = D - M, \quad F = \#\text{ table comparisons},$$
$$H = N, \quad K = M, \quad Q = N, \quad R = U.$$

11 [15]

If π is written as a product of disjoint cycles, obtain π^{-1} by reversing the direction of each cycle (i.e., read each cycle backwards). Equivalently,

$$\pi = (a_1\, a_2\, \ldots\, a_k) \quad \Longrightarrow \quad \pi^{-1} = (a_1\, a_k\, a_{k-1}\, \ldots\, a_2)\,,$$

and do this for every disjoint cycle. Since disjoint cycles commute, their order in the product does not matter.

Singleton cycles (a) are unchanged.

If you want the canonical cycle form of §1.3.3 (smallest element first in each cycle and cycles ordered by decreasing first element), simply rotate each reversed cycle to put its smallest element first and then reorder the cycles accordingly.

By the meaning of a cycle,

$$\pi(a_i) = a_{i+1} \quad (1 \le i < k), \qquad \pi(a_k) = a_1.$$

The inverse must undo these maps, so for every i,

$$\pi^{-1}(a_{i+1}) = a_i \quad (1 \le i < k), \qquad \pi^{-1}(a_1) = a_k,$$

which is exactly the effect of the reversed cycle $(a_1\, a_k\, a_{k-1}\, \ldots\, a_2)$. Applying this argument to each disjoint cycle gives the stated rule.

Examples:

1. $ =(1,4,3)(2,5) ^{-1}=(1,3,4)(2,5).$ (Optionally in canonical form: $(1\,3\,4)(2\,5)$ is already canonical.)
2. $ =(3,1,6)(5,4) ^{-1}=(3,6,1)(5,4).$ Canonicalize if desired: rotate to smallest-first in each cycle to get $(1\,3\,6)(4\,5)$, then order cycles by decreasing first element.

12 [M27]

(a) Prove the mapping $y \equiv mx \pmod{mn-1}$ (with $0 \le x < N = mn-1$)

Write the current offset in terms of (i, j):

$$x = n(i-1) + (j-1).$$

The transposed offset is

$$y = m(j-1) + (i-1).$$

Compute mx and compare to y:

$$mx = m\big(n(i-1) + (j-1)\big) = mn(i-1) + m(j-1).$$

Subtract:

$$y - mx = \big(m(j-1) + (i-1)\big) - \big(mn(i-1) + m(j-1)\big) = (i-1) - mn(i-1) = -(mn-1)\,(i-1).$$

Hence

$$y \equiv mx \pmod{mn-1}.$$

Now check that when we restrict to $0 \le x < N = mn-1$, the corresponding (i, j) cannot be the bottom-right cell (m, n). Therefore

$$0 \le y = m(j-1) + (i-1) \le m(n-1) + (m-1) = mn - 2 = N - 1,$$

so y is already the standard residue of mx mod $(mn-1)$. Two fixed points deserve mention:

- $x = 0$ (cell $(1, 1)$) maps to $y = 0$.
- $x = mn-1$ (cell (m, n)) would also map to itself, but it lies outside the domain $0 \le x < N$ used by the modular rule—this is why $N = mn-1$ is the modulus: it isolates those two fixed points at the corners.

This completes the proof requested in (a).

(b) In-place methods to perform the transposition

The linear index permutation is:

$$\pi(x) \;=\; \begin{cases} 0, & x = 0, \\ (mx) \bmod (mn-1), & 1 \le x \le mn-2, \\ mn-1, & x = mn-1. \end{cases}$$

Since $\gcd(m, mn-1) = 1$, $x \mapsto mx \bmod (mn-1)$ is a permutation of $\{0, 1, \dots, mn-2\}$. In place, we move elements along the disjoint cycles of π.

Method 1 — Cycle-leader with a visited bitmap (simple and reliable)

Use one temporary register to rotate each cycle:

```
N = m*n - 1
mark[0..N-1] = false
for s in 1..N-1:
    if not mark[s]:
        t = s
        tmp = A[t]
        while True:
            u = (m * t) % N
            if u == s:
                A[t] = tmp
                break
            A[t] = A[u]
            mark[t] = true
            t = u
        mark[t] = true
```

Correctness. Each inner loop performs a cyclic rotation along one cycle of π-, so all elements land in their transposed locations. 0 and $mn-1$ are fixed and can be skipped.

Cost. Exactly one write per element in $\{1, \dots, mn-2\}$, plus one extra write per cycle to place `tmp`. Time $\Theta(mn)$, space $O(mn)$ bits for `mark` (or a compact bitset).

Method 2 — Pure in-place cycle generation via divisors of N (no mark array)

We can enumerate cycles deterministically using number theory:

- Let $N = mn - 1$. For any starting index x with $d = \gcd(x, N)$, write $x = d \cdot t$ and $M = N/d$.
- The cycle containing x has length equal to the order of m modulo M, i.e., the smallest $r \ge 1$ with $m^r \equiv 1 \pmod{M}$.
- One may generate this cycle as $x_k = d \cdot (t\, m^k \bmod M)$, $k = 0, 1, \dots, r-1$.

By looping over divisors $d \mid N$ and a set of coset representatives t modulo $M = N/d$, you can traverse each cycle exactly once. This yields the same cycle-leader rotations as Method 1 but uses only $O(1)$ extra words (plus arithmetic on $\bmod M$). It avoids the mark array at the price of a slightly more intricate control structure (needs orders modulo M and coset stepping).

Method 3 — If extra workspace is allowed (cache-friendly blocking)

If "on itself" is relaxed to allow a small scratch buffer, a fast practical approach is tiling:

1. Partition the matrix into $B \times B$ tiles.
2. For off-diagonal tile pairs (p, q) and (q, p), swap tiles and transpose each tile into the other.
3. For diagonal tiles, transpose in place (they're square).

This dramatically improves locality on modern caches. It's not strictly in-place unless $B = 1$, but with a small temporary $B \times B$ buffer it runs faster in practice.

Practical notes

- The only elements not governed by the modulus rule are the two corners; both are fixed points.
- The permutation $x \mapsto mx \bmod (mn - 1)$ partitions $\{1, \dots, mn - 2\}$ into cycles of lengths that divide the order of m modulo N/d (where $d = \gcd(x, N)$). That structure can be exploited for pure in-place implementations (Method 2).
- For production code, Method 1 (with a bitset) is simple and hard to get wrong; Method 2 is elegant and truly $O(1)$ extra space.

13 [M24]

Restating Algorithm J (briefly)

- J1. For all k, set $X[k] \leftarrow -X[k]$. Set $m \leftarrow n$.
- J2. Set $j \leftarrow m$.
- J3. Set $i \leftarrow X[j]$. If $i > 0$ set $j \leftarrow i$ and repeat J3.
- J4. (Now $i \leq 0$.) Let $a = -i$. Do $X[j] \leftarrow X[a]$ and $X[a] \leftarrow m$.
- J5. Decrease m by 1; if $m > 0$ go to J2; else halt.

Intuition: after J1 every entry encodes the *forward* edge $k \to \pi(k)$ as the negative number $-\pi(k)$. Step J4 "splices out" one forward edge and deposits a positive number m somewhere; those positive numbers are the breadcrumbs from which the inverse will emerge.

Invariants

We prove correctness by maintaining three simple invariants throughout the loop on m.

Let $\mathcal{P}$ be the set of indices whose entries are currently positive. Define

$$\mathrm{pos}(t) \text{ means } \exists\, y\ (X[y] = t > 0).$$

I1 (partition of information). For every index y,

- either $X[y]$ is negative and equals $-u$ for some u (interpreted as a *still-present* forward link $y \to u$);
- or $X[y]$ is positive and equals t (interpreted as a *partial inverse* fact "y currently points back to t").

I2 (what positives mean). For each positive value t that currently appears *somewhere* in X, that occurrence is at an index of the form $\pi(s)$ for some s on the cycle containing t. (Thus positives always sit at *images* under π.)

I3 (progress by m). After finishing iteration value m (i.e., just after J5),

- every integer $t > m$ appears exactly once as a positive value in the array,
- every integer $\leq m$ appears only with negative sign (i.e., it has not yet been "deposited" positively).

These invariants hold initially after J1 (vacuously for I2–I3). We show they are preserved by one pass of J2–J5 and that they imply correctness at the end.

Why J3 terminates (and what it finds)

During a fixed outer iteration (fixed m), J3 repeatedly follows *positive* pointers: $j \leftarrow X[j]$ while $X[j] > 0$. Because of I2 there are no cycles made solely of positive pointers (they are "back-links" placed on images, not a disjoint permutation by themselves), so this walk must end at some index $j^\star$ with $X[j^\star] < 0$. Thus J3 always halts and yields a $j^\star$ whose entry still encodes a forward edge. (The text even remarks that validity is "less obvious" and leaves it to the reader; this is the key point.)

The splice step (J4) preserves the invariants and adds one correct inverse fact

At $j^\star$ we have $i = X[j^\star] \leq 0$, so let $a = -i = \pi(j^\star)$. Step J4 does:

$$\underbrace{X[j^\star] \leftarrow X[a]}_{\text{bypass } j^\star \to a \text{ to keep forward info}}, \qquad \underbrace{X[a] \leftarrow m}_{\text{deposit } m \text{ at the image } a=\pi(j^\star)}.$$

- The assignment $X[a] \leftarrow m > 0$ places a positive m exactly at an image index $a = \pi(j^\star)$, so I2 holds and the set of positives grows by at most one.
- The assignment $X[j^\star] \leftarrow X[a]$ *bypasses* the removed forward edge $j^\star \to a$: if $X[a] = -b$ previously encoded $a \to b = \pi(a)$, then now $X[j^\star]$ becomes $-b$, i.e. $j^\star \to b$. Thus the remaining forward structure is preserved, so I1 continues to hold.

- By construction, the new positive value is exactly the current m, and all previously placed positive values were $> m$ (inductive hypothesis I3). Hence after J5, I3 is still true with m decreased by one.

In short, every pass of J2–J5 adds exactly one positive m at *some* image position and keeps everything else consistent.

Per-cycle correctness

Fix a cycle $C = (a_0, a_1, \dots, a_{\ell-1})$ of π, where $\pi(a_r) = a_{r+1}$ (indices mod ℓ). Consider the outer iterations in the numerical order $m = n, n-1, \dots, 1$, and look only at those passes where $m \in C$. We prove by induction on these passes that:

(C) After the pass with parameter $m \in C$, the array has, for this cycle,

$$X[a_{r+1}] = a_r \quad \text{for every } a_r \in C \text{ that has already been processed (i.e., appeared as some earlier } m),$$

and for every unprocessed $a_s \in C$ its forward link is still present somewhere as a negative value (possibly "bypassed" a few times by earlier splices, but still represented as a negative successor), so a future pass can continue.

- Base. Let m be the first value of C encountered. Then $j = m$ immediately has $X[j] < 0$, so J4 writes $X[\pi(m)] = m$ and replaces $X[m]$ by the (negative) successor of $\pi(m)$. Thus property (C) holds initially: the edge $\pi(m) \leftarrow m$ of the inverse is now correct.
- Step. Suppose (C) holds after finishing some earlier value from C. Consider the next $m \in C$. The walk of J3 only steps along already-placed positives on this same cycle and must stop at a $j^\star \in C$ whose entry is still negative. The splice then deposits m exactly at $\pi(j^\star) \in C$ and preserves all previously correct edges; thus (C) extends by one more correct inverse edge and remains true.

Since each pass adds exactly one correct inverse edge on the cycle and never destroys earlier ones, when all members of C have been processed we end with

$$X[a_{r+1}] = a_r \qquad (r = 0, 1, \dots, \ell - 1),$$

i.e., the full inverse on that cycle. Because cycles are disjoint, the statement holds globally: for every y we have $X[y] = \pi^{-1}(y)$ at termination.

Termination and final state

The outer loop runs exactly n times (values of m). Every iteration performs one splice; there is no possibility of an infinite inner loop because, as argued above, J3 always reaches a negative entry. Therefore the algorithm halts with all entries positive. By the per-cycle argument, these

positives are precisely the inverse mapping. Hence Algorithm J is valid. (The text explicitly notes Algorithm J has the same effect as Algorithm I, whose goal is to replace X by its inverse.)

Context note. The book later remarks that although Algorithm J is "deucedly clever," its average running time is $\Theta(n \log n)$ versus $\Theta(n)$ for Algorithm I; the exercise here, however, is only to prove correctness.

14 [M34]

What A counts (intuition)

In Algorithm J we repeatedly do:

- set $j \leftarrow m$,
- then chase $j \leftarrow X[j]$ while $X[j] > 0$ (step J3),
- stop when we hit a negative entry and perform J4/J5.

Thus, A is the total number of times step J3 keeps following a *positive* entry across the whole run—the total length of all these "positive-chase" segments.

A precise combinatorial model for A

Two simple facts (both standard in the discussion around Algorithm J) let us translate A to an elementary permutation statistic:

1. State of the array at J2. At the beginning of each J2, one can prove by induction (Exercise 13) that

- $X[i] = +j$ iff $j > m$ and j maps to i under π;
- $X[i] = -j$ otherwise, where the sign encodes how far π has been "unwound." This explains why J3 walks over *already-inverted* parts (positive entries).

2. Linearized canonical form. Write the inverse permutation π^{-1} in canonical cycle form and then drop parentheses to get a linear sequence of length n. If, for each position, you count the number of consecutive elements immediately to its right that are larger than the current element, then

$$A - N = (\text{total of those counts over all positions}).$$

Equivalently, if you draw the customary "dot diagram" under the linearized π^{-1}, A is exactly the total number of dots. (This is the official reduction used in the book's answer.)

The key observation now is: the number of dots below the first k elements equals the number of right-to-left minima in that prefix (scanned left-to-right, a position is a right-to-left minimum if its value is smaller than all earlier values in the prefix when reading from the right). Hence,

$$A \;=\; \sum_{k=1}^{n} (\#\text{ right-to-left minima among the first } k \text{ elements}).$$

Take expectation

In a random permutation, the probability that position r is a (right-to-left) record minimum among the first r items is $1/r$. Therefore the expected number of such minima in the first k items is the harmonic number $H_k = \sum_{r=1}^{k} 1/r$. Summing over k gives

$$\mathbb{E}[A] \;=\; \sum_{k=1}^{n} H_k \;=\; (n+1)H_n - n.$$

(This uses the well-known identity $\sum_{k=1}^{n} H_k = (n+1)H_n - n$.) The appearance of H_k is the same "records" phenomenon used earlier in §1.3.3.

Final answer

$$\boxed{\mathbb{E}[A] \;=\; \sum_{k=1}^{n} H_k \;=\; (n+1)H_n - n}$$

and, using $H_n = \ln n + \gamma + o(1)$,

$$\mathbb{E}[A] = n \ln n + (\gamma - 1)n + \ln n + \gamma + o(1).$$

This aligns with the text's remark that Algorithm J's average running time is $\Theta(n \ln n)$, because A dominates its cost.

15 [M12]

- Linear (one-line) notation of a permutation π on $\{1, \ldots, n\}$ is the sequence $\pi(1)\ \pi(2)\ \cdots\ \pi(n)$.
- Canonical cycle form: write each cycle with its smallest element first, and order the cycles by decreasing first element; then (for the correspondence we use) drop the parentheses to get a length-n sequence. Knuth shows that from this sequence you can uniquely put the parentheses back by inserting a left parenthesis before each left-to-right minimum.

Call the mapping "drop-canonical-parentheses" $C(\pi)$, which takes π to the sequence obtained by (canonical cycles) $\to$ (remove parentheses).

We are asked for permutations π such that

$$C(\pi) \text{ (a length-}n\text{ sequence)} \;=\; \text{linear sequence } \pi(1)\pi(2)\cdots\pi(n).$$

Key observations

Consider the first symbol of the linear notation, $\pi(1) = m$.

1. If $m = 1$: then 1 is a fixed point. In canonical cycle form there is a 1-cycle (1). Since cycles are ordered by decreasing first element, (1) is the last cycle. When we drop parentheses, that trailing cycle contributes a final "1". Therefore the last character of $C(\pi)$ is 1.
2. If $m > 1$: then in the cycle containing 1 we have $(1\ m\ \cdots)$, because $\pi(1) = m$. Again this cycle (starting with 1) is last in canonical order, so in $C(\pi)$ the substring "… 1 m …" appears (the "1" immediately followed by "m").

Consequences

Now compare $C(\pi)$ with the linear sequence:

- If the linear sequence starts with 1, but $C(\pi)$ ends with 1, the only way the two sequences can be identical is when there is only one position—i.e., $n = 1$ (trivial one-element permutation). (The empty permutation $n = 0$ also vacuously works.)
- If the linear sequence starts with $m > 1$, then $C(\pi)$ contains the adjacent pair "1 m". For the two sequences to be identical, the linear sequence would need to start with "… 1 m …", which is impossible because its first symbol is m and there is no preceding 1.

Therefore, there is no nontrivial permutation with this property. The only solutions are the trivial cases: the permutation on one element (and the empty permutation). This matches Knuth's official answer.

Tiny sanity checks

- $n = 1$: linear "1"; canonical cycles "(1)"; drop $\to$ "1". Equal
- $n = 2$: linear "1 2" gives canonical "(1)(2)" $\to$ "2 1", not equal.
- $n = 2$: linear "2 1" gives canonical "(1 2)" $\to$ "1 2", not equal.

Conclusion: Apart from $n = 0$ or 1, no permutation has identical linear notation and canonical-cycle-without-parentheses notation.

16 [M15]

First, recall the book's "canonical cycle form": (a) write every 1-cycle explicitly, (b) rotate each cycle so its smallest element comes first, and (c) order the cycles by decreasing value of their first element. Then "remove parentheses" simply concatenates the cycles' entries into a length-n string, which we now read as the next permutation in linear notation.

The task is: start from 1324 (linear form), convert to canonical cycle form, drop the parentheses to get a new linear form, and repeat until you return to 1324.

Step-by-step

For each row: "linear → canonical cycles → next linear (after dropping parentheses)".

1. 1324 → (4)(2 3)(1) → 4231
2. 4231 → (3)(2)(1 4) → 3214
3. 3214 → (4)(2)(1 3) → 4213
4. 4213 → (2)(1 4 3) → 2143
5. 2143 → (3 4)(1 2) → 3412
6. 3412 → (2 4)(1 3) → 2413
7. 2413 → (1 2 4 3) → 1243
8. 1243 → (3 4)(2)(1) → 3421
9. 3421 → (1 3 2 4) → 1324

You are now back at the start.

Answer (the permutations that appear, in order)

1324 → 4231 → 3214 → 4213 → 2143 → 3412 → 2413 → 1243 → 3421 → 1324.

So, beginning from 1324, the "canonicalize cycles, then drop parentheses" transformation runs through a 9-cycle of distinct permutations (returning to the start on the 10th line).

If you're curious: for n=4 this transformation partitions all 24 permutations into two disjoint orbits; 1324 lies in the orbit of length 9 (the other orbit has length 15).

17 [M24]

(a) Average length when you pick a cycle uniformly from all cycles

Fact used from the text: among all permutations of n elements, the total number of m-cycles is

$$\frac{n!}{m} \qquad (1 \le m \le n),$$

hence the total number of cycles of all lengths is

$$\sum_{m=1}^{n} \frac{n!}{m} = n!\, H_n,$$

so the average number of cycles in a random permutation is H_n.

If we now choose one cycle uniformly from these $n!\, H_n$ cycles, the probability that it has length m is

$$p_m = \frac{\#\text{ of } m\text{-cycles}}{\#\text{ of all cycles}} = \frac{(n!/m)}{n!\, H_n} = \frac{1}{mH_n}.$$

Therefore the average cycle length under this sampling is

$$\mathbb{E}[L] = \sum_{m=1}^{n} m \cdot p_m = \sum_{m=1}^{n} m \cdot \frac{1}{mH_n} = \frac{n}{H_n}.$$

(Exercise statement context: the problem explicitly sets up this sampling over the $n!\, H_n$ cycles.)

(b) Length of the cycle containing a random element k in a random permutation

Count permutations in which a fixed element k lies in an m-cycle:

- Choose the other $m-1$ elements that share k's cycle: $\binom{n-1}{m-1}$.
- Arrange them with k on a cycle: $(m-1)!$ cyclic orders.
- Permute the remaining $n-m$ elements arbitrarily: $(n-m)!$.

Thus the number of permutations with k in an m-cycle is

$$\binom{n-1}{m-1}(m-1)!(n-m)! = (n-1)!,$$

independent of m. Since there are $n!$ permutations total, we get

$$\Pr\{k \text{ is in an } m\text{-cycle}\} = \frac{(n-1)!}{n!} = \frac{1}{n} \qquad (1 \le m \le n).$$

So the cycle length of a uniformly chosen element is uniform on $\{1, 2, \dots, n\}$. Its average is therefore

$$\mathbb{E}[L_k] = \frac{1}{n}\sum_{m=1}^{n} m = \frac{n+1}{2}.$$

(Again, the exercise frames this as: pick a random k and a random permutation; determine the probability for an m-cycle and the corresponding average.)

Summary of results

- (a) Picking a cycle uniformly from among all cycles across all permutations:

$$\boxed{\mathbb{E}[L] = \frac{n}{H_n}}$$

- (b) Picking a uniformly random element k in a uniformly random permutation:

$$\boxed{\Pr\{L_k = m\} = \frac{1}{n} \quad \text{for } 1 \le m \le n, \quad \mathbb{E}[L_k] = \frac{n+1}{2}}$$

18 [M27]

Let X_m be the number of cycles of length m in a uniformly random permutation of $\{1, \dots, n\}$. We want (i) the exact probability $p_{nkm} = \Pr[X_m = k]$; (ii) the probability generating function $G_{n,m}(z) = \mathbb{E}[z^{X_m}]$; and (iii) the mean and standard deviation of X_m.

Knuth's cycle-index/exponential formula for permutations says that if we "mark" cycles of a given length with a variable, the exponential generating function (EGF) multiplies by $\exp((w-1)x^m/m)$. Specializing $w = z$ for m-cycles and keeping all other lengths unmarked gives the bivariate EGF

$$\mathcal{E}(z,x) = \exp\Big(\sum_{j\ge1} \tfrac{x^j}{j}\Big) \cdot \exp\Big(\tfrac{(z-1)x^m}{m}\Big) = \frac{1}{1-x}\ \exp\Big(\tfrac{(z-1)x^m}{m}\Big).$$

Extracting the x^n coefficient and dividing by the total number $n!$ (equivalently, by $[x^n](1-x)^{-1} = 1$) yields the PGF for X_m at fixed n:

$$\boxed{G_{n,m}(z) = \sum_{r=0}^{\lfloor n/m \rfloor} \frac{(z-1)^r}{m^r\, r!}} \quad \text{(for all } n \ge 0\text{)}.$$

For $m = 1$ this reduces exactly to Knuth's Eq. (27).

Exact distribution p_{nkm}

Read off the z^k-coefficient of $G_{n,m}(z)$. Using $(z-1)^r = \sum_{k=0}^{r} \binom{r}{k}(-1)^{r-k} z^k$,

$$\boxed{p_{nkm} = \sum_{r=k}^{\lfloor n/m \rfloor} \binom{r}{k} \frac{(-1)^{r-k}}{m^r\, r!} = \frac{1}{m^k\, k!} \sum_{t=0}^{\lfloor n/m \rfloor - k} \frac{(-1)^t}{m^t\, t!}}, \qquad 0 \le k \le \left\lfloor \frac{n}{m} \right\rfloor.$$

(When $n < m$, $p_{n0m} = 1$ and $p_{nkm} = 0$ for $k > 0$.)

Mean and variance (hence standard deviation)

For a PGF $G(z)$, $\mathbb{E}X = G'(1)$ and $\mathrm{Var}(X) = G''(1) + G'(1) - [G'(1)]^2$. Differentiate the closed form of $G_{n,m}$:

- $G'_{n,m}(1) = \dfrac{1}{m}$ (exact whenever $n \ge m$; otherwise 0). So $\boxed{\mathbb{E}[X_m] = 1/m \text{ for } n \ge m, \text{ else } 0.}$
- $G''_{n,m}(1) = \begin{cases} \dfrac{1}{m^2}, & n \ge 2m, \\ 0, & n < 2m. \end{cases}$

Hence

$$\mathrm{Var}(X_m) = \begin{cases} \dfrac{1}{m}, & n \ge 2m, \\ \dfrac{1}{m} - \dfrac{1}{m^2} = \dfrac{m-1}{m^2}, & m \le n < 2m, \\ 0, & n < m. \end{cases} \quad \Rightarrow \quad \boxed{\sigma(X_m) = \begin{cases} \dfrac{1}{\sqrt{m}}, & n \ge 2m, \\ \sqrt{\dfrac{1}{m} - \dfrac{1}{m^2}}, & m \le n < 2m, \\ 0, & n < m. \end{cases}}$$

(A quick combinatorial cross-check for the mean: For each m-subset S, the event "S forms a cycle" has probability $(m-1)!(n-m)!/n!$; summing over $\binom{n}{m}$ choices gives $\binom{n}{m}(m-1)!(n-m)!/n! = 1/m$.)

Asymptotics

Let $n \to \infty$ with m fixed. Then the truncated exponential in p_{nkm} tends to $e^{-1/m}$, so

$$p_{nkm} \longrightarrow e^{-1/m} \frac{1}{k!\, m^k},$$

i.e., X_m converges in distribution to Poisson$(1/m)$, with mean and variance both $1/m$ — consistent with the exact formulas above when $n \ge 2m$. (The m=1 case was treated in the text; this generalizes it verbatim.)

19 [HM21]

Let D_n (Knuth's P_n^0) denote the number of derangements of n distinct objects. By inclusion–exclusion (derived just above the exercise), we have the well-known alternating-sum formula

$$D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}.$$

(This is exactly Knuth's Eq. (25) specialized to the "no singleton cycles" case.)

Exercise 19 asks you to show that D_n equals the nearest integer to $n!/e$ for every $n \geq 1$. (Exercise statement.)

Set

$$S_n = \sum_{k=0}^{n} \frac{(-1)^k}{k!}, \qquad S_\infty = \sum_{k=0}^{\infty} \frac{(-1)^k}{k!} = e^{-1}.$$

Then

$$\frac{n!}{e} - D_n = n!(S_\infty - S_n) = n!\, R_n, \quad \text{where } R_n := \sum_{k=n+1}^{\infty} \frac{(-1)^k}{k!}.$$

Because $\{1/k!\}$ is strictly decreasing to 0, the tail R_n of this alternating series has the standard sharp bound

$$|R_n| \;<\; \frac{1}{(n+1)!}$$

and has sign $(-1)^{n+1}$. Hence

$$\left|\frac{n!}{e} - D_n\right| = n!\,|R_n| \;<\; \frac{n!}{(n+1)!} = \frac{1}{n+1} \;\leq\; \frac{1}{2} \quad (n \geq 1).$$

The inequality is strict even when $n = 1$ (because the alternating remainder is strictly smaller than the first omitted term).

Therefore the difference between $n!/e$ and the integer D_n has absolute value $< \frac{1}{2}$. This means D_n is exactly the nearest integer to $n!/e$ for all $n \geq 1$, as required.

20 [M20]

The same permutation can be written in many *cycle notations* because

1. disjoint cycles may be listed in any order; and
2. each k-cycle can be *rotated* to start from any of its k elements (but not reversed—reversal gives the inverse cycle).

Let the permutation have

- one-cycles, two-cycles, three-cycles, ...,
- and let r = Σ be the total number of cycles (including singletons).

Because the cycles are disjoint and distinct, you can order the r cycles in r! ways. Independently, each k-cycle has k rotational variants, so all k-cycles together contribute k^{ } ways.

Multiplying these independent choices gives the total number of written forms:

$$\boxed{\#\text{ of cycle notations } = r! \prod_{k\geq 1} k^{\alpha_k}} \qquad \text{where } r = \sum_{k\geq 1} \alpha_k.$$

Notes and checks

- Singletons contribute a factor $1^{\alpha_1} = 1$; they only affect the $r!$ ordering factor.
- If (as in Exercise 5) you suppress singletons, replace r by $r - \alpha_1$ and drop the $k = 1$ factor: $(r - \alpha_1)! \prod_{k\geq 2} k^{\alpha_k}$.

Example If $\pi = (1\,4\,3)(2\,5)(6)(7)$, then $(\alpha_1, \alpha_2, \alpha_3) = (2, 1, 1)$ and $r = 4$. The number of ways to write π in cycle form (with singletons shown) is

$$4! \cdot 3^1 \cdot 2^1 \cdot 1^2 = 24 \cdot 6 = 144.$$

21 [M22]

$$\boxed{P(n; \alpha_1, \alpha_2, \ldots) = \begin{cases} \dfrac{1}{\displaystyle\prod_{m\geq 1} m^{\alpha_m}\, \alpha_m!}, & \text{if } \sum_{m\geq 1} m\,\alpha_m = n, \\ 0, & \text{otherwise.} \end{cases}}$$

Derivation (combinatorial counting)

1. Count permutations with a fixed cycle type. Let's count how many permutations of $\{1, \dots, n\}$ have exactly α_m cycles of length m for each m. A standard result from the cycle-index/Polya theory (and easy to derive directly) is:

$$\#\{\text{permutations with this cycle type}\} \;=\; \frac{n!}{\prod_{m\geq 1} m^{\alpha_m}\,\alpha_m!}.$$

Intuition: start with all $n!$ linear arrangements; turning those into cycles overcounts by a factor m for each m-cycle (rotations of a cycle don't change it), and cycles of the same length are indistinguishable so divide by $\alpha_m!$ as well.

2. Convert to probability. All $n!$ permutations are equally likely, so divide the count above by $n!$:

$$P(n; \alpha_1, \alpha_2, \dots) = \frac{1}{\prod_{m\geq 1} m^{\alpha_m}\,\alpha_m!},$$

provided the cycle counts fit n (otherwise the probability is 0).

3. Sanity checks.

- Sum to 1. Summing the counts $\frac{n!}{\prod m^{\alpha_m}\alpha_m!}$ over all (α_m) with $\sum m\alpha_m = n$ gives $n!$; dividing by $n!$ yields total probability 1.
- Examples.
  - $n = 4$, two 2-cycles: $(\alpha_2 = 2)$. Count $= \frac{4!}{2^2\,2!} = 3$; probability $= 3/24 = 1/8 = \frac{1}{2^2\,2!}$.
  - $n = 4$, one 3-cycle and one fixed point: $(\alpha_3 = 1, \alpha_1 = 1)$. Count $= \frac{4!}{3^1\,1!\cdot 1^1\,1!} = 8$; probability $= 8/24 = 1/3 = \frac{1}{3}$.
  - $n = 4$, identity: $(\alpha_1 = 4)$. Count $= \frac{4!}{1^4\,4!} = 1$; probability $= 1/24$.

Compute

$$P(n; \alpha_1, \alpha_2, \dots) = \Pr\big(\text{a random } \pi \in S_n \text{ has } \alpha_j \; j\text{-cycles for all } j\big)\,.$$

Step 1: Count permutations with a prescribed cycle structure

We'll count how many permutations of $\{1, \dots, n\}$ have cycle type $(\alpha_1, \alpha_2, \dots)$.

Think in two stages:

1. Choose the elements for each cycle (ignoring order inside each cycle, and ignoring the order among cycles of the same length). Partition the n labeled elements into α_1 blocks of size 1, α_2 blocks of size 2, α_3 blocks of size 3, … . The number of such partitions is

$$\frac{n!}{\prod_{j\geq 1}(j!)^{\alpha_j}\,\alpha_j!}.$$

2. Within each block, arrange the elements around a cycle. A block of size j can be arranged on a directed cycle in $(j-1)!$ distinct ways (there are $j!$ linear orders but each j-cycle has j rotational representations). Doing this for all cycles multiplies by

$$\prod_{j\geq 1}((j-1)!)^{\alpha_j}.$$

Multiplying the two stages gives the total number of permutations with that cycle type:

$$\#\{\pi \in S_n : \text{cycle type } (\alpha_j)\} = \frac{n!}{\prod_{j\geq 1}(j!)^{\alpha_j}\,\alpha_j!} \cdot \prod_{j\geq 1}((j-1)!)^{\alpha_j} = \frac{n!}{\prod_{j\geq 1} j^{\alpha_j}\,\alpha_j!}.$$

(Equivalently, one can argue by "decorating" cycles with a chosen starting point and an order among equal-length cycles; each underlying permutation then corresponds to exactly $\prod_j j^{\alpha_j}\alpha_j!$ decorated representations, while there are $n!$ decorated representations in total—so the same formula results.)

This well-known enumeration also appears in standard references on permutations.

Step 2: Divide by $n!$ to get the probability

Since every permutation is equally likely, the desired probability is the count above divided by $n!$:

$$\boxed{P(n;\alpha_1,\alpha_2,\ldots) = \frac{1}{\prod_{j\geq 1} j^{\alpha_j}\,\alpha_j!}} \quad \text{(provided } \sum_{j\geq 1} j\,\alpha_j = n\text{; otherwise } P=0\text{)}.$$

Quick consistency checks

- Normalization. Summing the right-hand side over all (α_j) with $\sum j\alpha_j = n$ gives 1. This follows from the exponential formula: $\displaystyle\sum_{\alpha:\sum j\alpha_j=n}\prod_{j\geq 1}\frac{1}{j^{\alpha_j}\alpha_j!} = [x^n]\exp\Big(\sum_{j\geq 1}\frac{x^j}{j}\Big) = [x^n]\frac{1}{1-x} = 1.$

- Example. For $n = 5$ and structure $1^1 2^2$ (one fixed point and two 2-cycles), $P = \dfrac{1}{1^1 1!\, 2^2 2!} = \dfrac{1}{8}$. Indeed there are $5!/8 = 15$ such permutations: choose the fixed point in 5 ways and pair the remaining 4 elements into two unordered pairs in 3 ways.

That completes the solution. The key is that the factor $j^{\alpha_j}\alpha_j!$ accounts for the symmetries of cycles of length j: j rotations per cycle and $\alpha_j!$ reorderings among cycles of the same length.

22 [HM34]

(a) Find $f(w, m, k)$

We seek independent α_m with

$$\Pr(\alpha_m = k) = f(w, m, k) \qquad (m \geq 1,\ k \geq 0),$$

such that:

1. $\sum_{k\geq 0} f(w, m, k) = 1$ for $0 < w < 1$;
2. for every nonnegative integer vector $(k_1, k_2, ...)$ with $\sum_{m\geq 1} mk_m = n$,

$$\Pr(\alpha_1 = k_1, \alpha_2 = k_2, ...) = (1 - w)\, w^n\, P(n; k_1, k_2, ...)\,.$$

Choice. Take the α_m independent Poisson with mean

$$\lambda_m = \frac{w^m}{m}, \qquad 0 < w < 1.$$

Thus

$$f(w, m, k) = e^{-\lambda_m}\frac{\lambda_m^k}{k!} = e^{-\frac{w^m}{m}}\ \frac{1}{k!}\left(\frac{w^m}{m}\right)^k.$$

Then

$$\prod_{m\geq 1} f(w, m, \alpha_m) = \exp\Big(-\sum_{m\geq 1}\tfrac{w^m}{m}\Big)\ \frac{w^{\sum m\alpha_m}}{\prod_m m^{\alpha_m}\alpha_m!}.$$

Since $\sum_{m\geq 1}\frac{w^m}{m} = -\ln(1 - w)$, the prefactor is $e^{\ln(1-w)} = 1 - w$. Also

$$P(n;\alpha) = \frac{1}{\prod_m m^{\alpha_m}\alpha_m!} \quad (\text{when } \sum m\alpha_m = n;\ \text{else } 0),$$

so the joint probability becomes $(1-w)\,w^n\,P(n;\alpha)$, exactly as required. Condition (1) holds because Poisson pmfs are normalized.

(b) Distribution of the total size $\sum m\alpha_m$

Let

$$N = \alpha_1 + 2\alpha_2 + 3\alpha_3 + \cdots.$$

Using pgfs,

$$\mathbb{E}(x^N) = \prod_{m\geq 1} \exp\Big(\lambda_m(x^m - 1)\Big) = \exp\Big(\sum_{m\geq 1} \tfrac{w^m}{m}(x^m - 1)\Big) = \frac{1-w}{1-wx}.$$

Therefore

$$\Pr(N = n) = (1-w)w^n \quad (n = 0, 1, 2, ...),$$

a geometric distribution; hence $\Pr(N = \infty) = 0$.

(c) Transfer principle for expectations

Let $\phi(\alpha_1, \alpha_2, ...)$ be any function depending on the α_m's. Using (a),

$$\mathbb{E}[\phi] = \sum_{n\geq 0}(1-w)w^n \sum_{\sum m\alpha_m = n} \phi(\alpha)\,P(n;\alpha) = (1-w)\sum_{n\geq 0} w^n\,\phi_n,$$

where ϕ_n is the average of ϕ over uniform random permutations of n (i.e., with the α_m's being the cycle counts of a uniform permutation). Example: If $\phi(\alpha) = \alpha_1$, then ϕ_n = average number of fixed points, which equals 1 for all n; our identity reduces to $(1-w)\sum_{n\geq 0} w^n \cdot 1 = w = \mathbb{E}\alpha_1$ (since $\alpha_1 \sim \text{Poisson}(w)$), consistent.

(d) Average number of even cycles in a permutation of n

Let $\phi(\alpha) = \alpha_2 + \alpha_4 + \alpha_6 + \cdots$ (the count of even-length cycles).

Unconditional expectation (under the Poisson model).

$$\mathbb{E}[\phi] = \sum_{j\geq 1} \mathbb{E}\alpha_{2j} = \sum_{j\geq 1} \frac{w^{2j}}{2j} = \tfrac{1}{2}\log\frac{1}{1-w^2}.$$

Transfer to fixed n. By part (c),

$$(1-w)\sum_{n\geq 0} w^n\,\phi_n \;=\; \tfrac{1}{2}\log\frac{1}{1-w^2}.$$

Equivalently,

$$\sum_{n\geq 0} w^n\,\phi_n \;=\; \frac{1}{1-w}\cdot\frac{1}{2}\log\frac{1}{1-w^2}.$$

Extracting coefficients gives the clean closed form

$$\boxed{\phi_n = \frac{1}{2}\,H_{\lfloor n/2\rfloor}}$$

where $H_m = \sum_{k=1}^{m}\frac{1}{k}$ is the m-th harmonic number. (Indeed, $(1-w)\sum_{n\geq 0}\frac{1}{2}H_{\lfloor n/2\rfloor}w^n = \frac{1}{2}\log\frac{1}{1-w^2}$.)

(e) Use the method to solve Exercise 18

Goal (Ex. 18). For fixed m, find

$$p_{n,k,m} = \Pr(\alpha_m = k \text{ in a uniform permutation of } n),$$

the probability generating function $G_{nm}(z) = \sum_{k\geq 0} p_{n,k,m} z^k$, and the mean and standard deviation of α_m.

Step 1 (bivariate generating function via Shepp–Lloyd). Under the Poisson model,

$$\mathbb{E}\big[z^{\alpha_m}\big] = \exp\Big(\tfrac{w^m}{m}(z-1)\Big).$$

By (c),

$$(1-w)\sum_{n\geq 0} w^n\,G_{nm}(z) = \exp\Big(\tfrac{w^m}{m}(z-1)\Big).$$

Therefore

$$\sum_{n\geq 0} w^n\, G_{nm}(z) = \frac{1}{1-w}\exp\Big(\tfrac{w^m}{m}(z-1)\Big).$$

Step 2 (coefficient extraction in w). Expand the exponential only in powers of w^m:

$$\exp\Big(\tfrac{w^m}{m}(z-1)\Big) = \sum_{r\geq 0} \frac{(z-1)^r}{m^r r!}\, w^{mr}.$$

Multiplying by $\frac{1}{1-w} = \sum_{s\geq 0} w^s$ gives

$$G_{nm}(z) = \sum_{r=0}^{\lfloor n/m\rfloor} \frac{(z-1)^r}{m^r r!}.$$

This is a truncated exponential in $z-1$. In particular, for $m=1$ it reduces to the classic $G_{n,1}(z) = \sum_{r=0}^{n} \frac{(z-1)^r}{r!}$.

Step 3 (explicit $p_{n,k,m}$). Extract the coefficient of z^k:

$$\boxed{p_{n,k,m} = \frac{1}{m^k k!}\sum_{s=0}^{\lfloor n/m\rfloor - k} \frac{(-1)^s}{m^s s!} \qquad (0\leq k\leq \lfloor n/m\rfloor),}$$

and $p_{n,k,m}=0$ for $k>\lfloor n/m\rfloor$. (For $m=1$: $p_{n,k,1} = \frac{1}{k!}\sum_{s=0}^{n-k}\frac{(-1)^s}{s!}$, the well-known fixed-point formula.)

Step 4 (mean and variance). From $G_{nm}(z)$,

$$\mathbb{E}_n[\alpha_m] = G'_{nm}(1) = \begin{cases} \dfrac{1}{m}, & n\geq m,\\ 0, & n<m,\end{cases}$$

and the second factorial moment

$$\mathbb{E}_n\big[\alpha_m(\alpha_m-1)\big] = G''_{nm}(1) = \begin{cases} \dfrac{1}{m^2}, & n\geq 2m,\\ 0, & n<2m.\end{cases}$$

Hence

$$\mathrm{Var}_n(\alpha_m) = \mathbb{E}_n[\alpha_m] + \mathbb{E}_n[\alpha_m(\alpha_m - 1)] - \left(\mathbb{E}_n[\alpha_m]\right)^2 = \begin{cases} 0, & n < m, \\ \dfrac{1}{m} - \dfrac{1}{m^2}, & m \le n < 2m, \\ \dfrac{1}{m}, & n \ge 2m, \end{cases}$$

and the standard deviation is $\sigma_n(\alpha_m) = \sqrt{\mathrm{Var}_n(\alpha_m)}$. (As $n \to \infty$ with fixed m, α_m tends to Poisson$(1/m)$: mean $1/m$, variance $1/m$.)

Summary of key formulas

- Independent model: $\alpha_m \sim \mathrm{Poisson}(\frac{w^m}{m})$ (independent in m).
- Size distribution: $\Pr\left(\sum m\alpha_m = n\right) = (1-w)w^n$.
- Transfer: $\mathbb{E}[\phi] = (1-w)\sum_{n\ge 0} w^n \phi_n$.
- Even cycles (in S_n): $\mathbb{E}[\#\text{even cycles}] = \dfrac{1}{2}H_{\lfloor n/2 \rfloor}$.
- For fixed m:

$$G_{nm}(z) = \sum_{r=0}^{\lfloor n/m \rfloor} \frac{(z-1)^r}{m^r r!}, \quad p_{n,k,m} = \frac{1}{m^k k!} \sum_{s=0}^{\lfloor n/m \rfloor - k} \frac{(-1)^s}{m^s s!}.$$

23 [HM42]

Let L_n be the length of the longest cycle of a uniformly random permutation of $\{1, \dots, n\}$. Show

$$\mathbb{E}[L_n] \;=\; \lambda\, n + \tfrac{1}{2}\,\lambda + o(1),$$

where $\lambda \approx 0.6243299885$ is a constant (the Golomb–Dickman constant). In fact,

$$\lim_{n\to\infty} \left(\mathbb{E}[L_n] - \lambda n - \tfrac{1}{2}\lambda\right) = 0.$$

Step 1 — Counting permutations with no "long" cycles (exponential generating functions)

For a fixed cut-off m, let $A_{n,m}$ be the number of permutations of $[n]$ all of whose cycle lengths are $\le m$. For labelled structures of permutations, the exponential generating function (EGF) with allowed cycle lengths in a set S is

$$\exp\Big(\sum_{j\in S}\frac{z^j}{j}\Big).$$

Thus

$$\sum_{n\geq 0}\frac{A_{n,m}}{n!}z^n = \exp\Big(\sum_{j=1}^{m}\frac{z^j}{j}\Big).$$

Since the EGF for all permutations is $\exp(\sum_{j\geq 1} z^j/j) = \frac{1}{1-z}$, the *probability* that the longest cycle length is $\leq m$ equals the EGF coefficient

$$\mathbb{P}(L_n \leq m) \;=\; [z^n]\exp\Big(\sum_{j=1}^{m}\frac{z^j}{j}\Big).$$

This is the starting point of Shepp–Lloyd's analysis.

Step 2 — Scaling limit and the Dickman function

Let $u > 1$ and set $m = \lfloor n/u \rfloor$. A saddle-point/Tauberian analysis of the coefficient above (Shepp–Lloyd) shows the limit

$$\boxed{\lim_{n\to\infty}\mathbb{P}(L_n \leq \lfloor n/u \rfloor) = \rho(u)} \qquad (u > 1),$$

where ρ is the *Dickman function*, defined by $u\rho'(u) + \rho(u-1) = 0$ for $u > 1$ with $\rho(u) = 1$ on $[0,1]$. Equivalently, for $x \in (0,1)$,

$$\lim_{n\to\infty}\mathbb{P}\Big(\frac{L_n}{n} \leq x\Big) = \rho\Big(\frac{1}{x}\Big).$$

Step 3 — The constant λ (first-order term)

From the tail-sum identity,

$$\frac{\mathbb{E}[L_n]}{n} = \frac{1}{n}\sum_{k=1}^{n}\mathbb{P}(L_n \geq k) = \frac{1}{n}\sum_{k=1}^{n}\Big(1 - \mathbb{P}(L_n \leq k-1)\Big).$$

With the scaling $x = k/n$ and Step 2, the Riemann sums converge to

$$\lim_{n\to\infty}\frac{\mathbb{E}[L_n]}{n} = \int_0^1\Big(1 - \rho(1/x)\Big)\,dx =: \lambda.$$

Standard identities give several equivalent integral forms, e.g.

$$\lambda = \int_1^\infty \frac{1-\rho(u)}{u^2}\,du = \int_0^\infty \exp\Big(-x - \int_x^\infty \frac{e^{-y}}{y}\,dy\Big)\,dx = 0.6243299885\ldots$$

(The second display is Shepp–Lloyd's formula; the numerical value and equivalent expressions are classical.) ([MathWorld][3])

This proves the *first-order* asymptotic $\mathbb{E}[L_n] \sim \lambda n$.

Step 4 — The $O(1)$ refinement and the $\frac{1}{2}\lambda$ term

To sharpen the Riemann-sum argument to the constant term, one "Poissonizes" the coefficients in Step 1 (replace fixed n by a Poisson(μ) size and analyze the resulting generating functions near their dominant singularity), obtains precise asymptotics for $\mathbb{P}(L_n \le m)$ that are *uniform* in the critical boundary layer $m \asymp n$, and then *de-Poissonizes*.

Carrying this out (as in Shepp–Lloyd's Tauberian scheme) yields the *Cesàro/Euler–Maclaurin* refinement of the tail-sum:

$$\mathbb{E}[L_n] = n\int_0^1 (1-\rho(1/x))\,dx \;+\; \frac{1}{2}\lambda \;+\; o(1) \;=\; \boxed{\lambda n + \tfrac{1}{2}\lambda + o(1)}.$$

Intuitively, the naive trapezoidal correction $-\frac{1}{2}$ you'd get by simply replacing the probabilities with their pointwise limits is not valid here, because the convergence in Step 2 is not uniform at the right endpoint $x \uparrow 1$. The boundary layer analysis contributes $+\frac{1}{2}\lambda$, not $-\frac{1}{2}$. A complete derivation appears in the original analytic framework and in later expositions. In fact, still finer terms are known:

$$\mathbb{E}[L_n] = \lambda\Big(n + \tfrac{1}{2}\Big) \;-\; \frac{e^\gamma}{24\,n} \;+\; O\Big(\frac{1}{n^2}\Big),$$

where γ is Euler's constant (Gourdon, sharpening Shepp–Lloyd). ([MathWorld][3])

Combining the scaling limit for L_n/n (Dickman law), the integral representation of λ, and the uniform Tauberian/Poissonization–de-Poissonization refinement for the boundary layer, we obtain

$$\lim_{n\to\infty}\big(\mathbb{E}[L_n] - \lambda n - \tfrac{1}{2}\lambda\big) = 0,$$

as required. (Historical attributions and the analytic toolkit are in Shepp–Lloyd; further explicit expansions, constants, and equivalent forms of λ are summarized in standard references.) ([MathWorld][3])

References

- Knuth, *TAOCP, Vol. 1 (3rd ed.)*, §1.3.3, Ex. 23.
- Shepp & Lloyd (1966), "Ordered cycle lengths in a random permutation." Gives the EGF method, the limit law for ordered cycle lengths, and the asymptotics that imply $\mathbb{E}[L_n] = \lambda n + \frac{1}{2}\lambda + o(1)$.
- Golomb–Dickman constant: equivalent integrals and refined expansion (including the $+\frac{1}{2}\lambda$ term and the next terms in $1/n$). ([MathWorld][3])

If you'd like, I can also show a short numerical check (Monte Carlo) that estimates $\mathbb{E}[L_n]$ and illustrates the $\lambda n + \frac{1}{2}\lambda$ fit for moderate n.

24 [M41]

1) What is A? (from Ex. 14)

In Algorithm J (the inversion algorithm), the timing is analyzed by counting how many times the inner "chase while positive" step is executed. Knuth denotes that count by A. A convenient interpretation (used in Ex. 14) is:

- Write the permutation's inverse in canonical cycle form and drop the parentheses;
- For each prefix of length k (for $k = 1, \dots, n$), count the number of right-to-left minima in that prefix;
- A is the sum of those n counts.

Equivalently, if R_k is the number of right-to-left minima in the first k elements of a uniformly random permutation on $\{1, \dots, n\}$, then

$$A \;=\; \sum_{k=1}^{n} R_k.$$

Knuth shows that $\mathbb{E}[R_k] = H_k$ (harmonic numbers), hence

$$\mathbb{E}[A] = \sum_{k=1}^{n} H_k \;=\; (n+1)H_n - n.$$

(That is the answer to Exercise 14.)

2) A representation that is convenient for variance

Define, for each position j, L_j to be the index of the first element strictly to the right of j that is smaller than the value at j; if none exists, set $L_j = n+1$. Then the number of prefixes ending at k for which the j-th element is a right-to-left minimum equals the number of k with $j \le k < L_j$. Therefore

$$A \;=\; \sum_{j=1}^{n} (L_j - j).$$

For a random permutation one has, for integers $t \ge 1$,

$$\Pr(L_j - j \ge t) = \frac{1}{t} \quad \text{for } 1 \le t \le n-j+1,$$

because among the t elements $j, j+1, \dots, j+t-1$ each position is equally likely to contain the minimum.

Two useful tail-sum identities for nonnegative integer X are

$$\mathbb{E}[X] = \sum_{t \ge 1} \Pr(X \ge t), \qquad \mathbb{E}[X^2] = \sum_{t \ge 1} (2t-1) \Pr(X \ge t).$$

Applying these with $X = L_j - j$ gives

$$\mathbb{E}[L_j - j] = H_{n-j+1}, \qquad \mathbb{E}[(L_j - j)^2] = 2(n-j+1) - H_{n-j+1}.$$

Summing $\mathbb{E}[L_j - j]$ over j recovers $\mathbb{E}[A] = (n+1)H_n - n$ from above.

To obtain the variance we need $\mathbb{E}[A^2]$. Writing

$$A^2 = \sum_{j=1}^{n} (L_j - j)^2 + 2 \sum_{1 \le j < \ell \le n} (L_j - j)(L_\ell - \ell),$$

one can evaluate the cross terms via

$$\mathbb{E}\big[(L_j - j)(L_\ell - \ell)\big] = \sum_{s=1}^{n-j+1} \sum_{t=1}^{n-\ell+1} \Pr(L_j - j \ge s,\ L_\ell - \ell \ge t),$$

and count the joint probabilities by simple order-symmetry on the overlapping index sets (cases: disjoint, nested, and partially overlapping intervals). The algebra is routine but a bit long; after the double sums telescope, one arrives at a clean closed form below. (Knuth cites his 1971 IFIP paper for details.)

3) Closed form for $\mathrm{Var}(A)$

Let $H_n = \sum_{k=1}^{n} \frac{1}{k}$ and $H_n^{(2)} = \sum_{k=1}^{n} \frac{1}{k^2}$. The variance of A is

$$\boxed{\mathrm{Var}(A) = 2n^2 + 4n \;-\; (n+1)H_n \;-\; (n+1)^2 H_n^{(2)}}$$

Checks

- For $n = 1$: $A = 1$ deterministically, so $\mathrm{Var}(A) = 0$. The formula gives $2+4-2\cdot 1-(2)^2\cdot 1 = 0$.
- For $n = 8$: $H_8 \approx 2.717857$, $H_8^{(2)} \approx 1.527422$. Then $\mathrm{Var}(A) \approx 2 \cdot 64 + 32 - 9 \cdot 2.717857 - 81 \cdot 1.527422 \approx 11.8181$, consistent with exact enumeration.

Asymptotics

Using $H_n = \ln n + \gamma + O(1/n)$ and $H_n^{(2)} = \frac{\pi^2}{6} + O(1/n)$,

$$\mathrm{Var}(A) = \left(2 - \tfrac{\pi^2}{6}\right)n^2 \;-\; n \ln n \;+\; O(n),$$

so the standard deviation is $\Theta(n\sqrt{\ln n})$ while $\mathbb{E}[A] \sim n \ln n$.

25 [M22]

Here is a clean proof of Eq. (29) from §1.3.3, which gives the number of objects that lie in none of the sets $S_1, \dots, S_M$ (the inclusion–exclusion principle). The equation in the book states that this number equals

$$N - \sum_j |S_j| + \sum_{i<j} |S_i \cap S_j| - \sum_{i<j<k} |S_i \cap S_j \cap S_k| + \cdots + (-1)^M |S_1 \cap \cdots \cap S_M|\,,$$

i.e., the alternating sum over the sizes of all nonempty intersections.

Proof (indicator-variable count)

Fix the underlying finite universe U with $|U| = N$. For each element $x \in U$, let

$$m(x) \;:=\; \#\{j \in \{1, \dots, M\} \,:\, x \in S_j\}$$

be the number of sets among $S_1, \dots, S_M$ that contain x.

Consider the "contribution" of a single element x to the alternating sum in Eq. (29). Each term

$$\sum_{1\le j_1<\cdots<j_r\le M} |S_{j_1}\cap\cdots\cap S_{j_r}|$$

counts x exactly once for each r-tuple of sets that all contain x. There are $\binom{m(x)}{r}$ such r-tuples. Therefore the total contribution of x to the entire right-hand side is

$$\underbrace{1}_{\text{from } N} - \binom{m(x)}{1} + \binom{m(x)}{2} - \binom{m(x)}{3} + \cdots + (-1)^{m(x)}\binom{m(x)}{m(x)}.$$

But the binomial identity gives

$$\sum_{r=0}^{m}(-1)^r\binom{m}{r} = (1-1)^m = \begin{cases}1, & m=0,\\ 0, & m\ge 1.\end{cases}$$

Hence the contribution of x is 1 if x lies in none of the sets ($m(x)=0$), and 0 otherwise. Summing over all $x\in U$ yields exactly the number of elements that lie in none of the sets $S_1,\dots,S_M$. This is precisely Eq. (29).

Notes and connections

- Specializing to the derangement discussion immediately below Eq. (29), take U to be the $n!$ permutations and S_j the permutations fixing position j. Then $|S_{j_1}\cap\cdots\cap S_{j_r}| = (n-r)!$, and Eq. (29) gives the standard inclusion–exclusion formula that leads to (25).
- The same indicator-variable method extends verbatim to "exactly r sets" counts (exercise 26), by weighting each element with $\binom{m(x)}{r}$ and alternating appropriately.

That completes the detailed proof of Eq. (29).

26 [M24]

Let U be the finite universe. For $k\ge 1$ define the "intersection sums"

$$\sigma_k = \sum_{1\le i_1<\cdots<i_k\le M} \left| S_{i_1}\cap\cdots\cap S_{i_k}\right|,$$

and set $\sigma_0 = |U|$. Let N_r be the number of elements that belong to exactly r of the sets ($0\le r\le M$).

Step 1: Forward relations

Fix an element $x \in U$, and let $r(x)$ be the number of sets among $S_1, \dots, S_M$ that contain x. Then x contributes to σ_k precisely when we choose k of those $r(x)$ sets, i.e., it contributes $\binom{r(x)}{k}$. Summing over all x (grouped by $r(x) = r$) gives, for each $k \geq 0$,

$$\boxed{\sigma_k \;=\; \sum_{r=k}^{M} \binom{r}{k} N_r} \qquad (k = 0, 1, \dots, M).$$

This is a lower-triangular linear system.

Step 2: Binomial inversion (the answer)

Invert the system (standard binomial inversion) to get, for each r,

$$\boxed{N_r \;=\; \sum_{k=r}^{M} (-1)^{k-r} \binom{k}{r} \sigma_k} \qquad (r = 0, 1, \dots, M).$$

Checks and corollaries

- Case $r = 0$ (in none of the sets): $N_0 = \sum_{k=0}^{M} (-1)^k \sigma_k$, which is exactly the formula proved in the text for $r = 0$.
- At least r sets (optional): letting $A_r = \sum_{j=r}^{M} N_j$, the same argument yields

$$A_r = \sum_{k=r}^{M} (-1)^{k-r} \binom{k-1}{r-1} \sigma_k \qquad (r \geq 1).$$

Let $M = 3$, $|U| = 10$ and

$$\sigma_1 = |S_1| + |S_2| + |S_3| = 15, \quad \sigma_2 = \sum |S_i \cap S_j| = 6, \quad \sigma_3 = |S_1 \cap S_2 \cap S_3| = 1.$$

Then

$$\begin{aligned}
N_3 &= \sigma_3 = 1, \\
N_2 &= \sigma_2 - \binom{3}{2} N_3 = 6 - 3 = 3, \\
N_1 &= \sigma_1 - \binom{2}{1} N_2 - \binom{3}{1} N_3 = 15 - 2 \cdot 3 - 3 \cdot 1 = 6, \\
N_0 &= \sigma_0 - N_1 - N_2 - N_3 = 10 - 6 - 3 - 1 = 0,
\end{aligned}$$

or directly $N_0 = \sigma_0 - \sigma_1 + \sigma_2 - \sigma_3 = 10 - 15 + 6 - 1 = 0$.

That completes the requested extension: the count of elements in exactly r sets is the alternating binomial sum of the intersection totals σ_k shown above.

27 [M20]

Let $M = \prod_{j=1}^{t} m_j$ and $N = aM$. For each j, let

$$A_j = \{\, 0 \le n < N : m_j \mid n \,\}$$

be the set of multiples of m_j in the range. We want the count of numbers not in $\bigcup_{j=1}^{t} A_j$:

$$\#\Big([0, N) \setminus \bigcup_{j=1}^{t} A_j \Big) = N - \sum_j |A_j| + \sum_{j<k} |A_j \cap A_k| - \cdots + (-1)^t \Big| \bigcap_{j=1}^{t} A_j \Big|.$$

Because $N = aM$ is a multiple of each m_j, these cardinalities have no floors:

- $|A_j| = N/m_j$ (multiples $0, m_j, 2m_j, \dots, (N/m_j - 1)m_j$).
- More generally, for any nonempty index set $S \subseteq \{1, \dots, t\}$,

$$\Big| \bigcap_{j \in S} A_j \Big| = \frac{N}{\operatorname{lcm}(m_j : j \in S)}.$$

  Since the m_j are pairwise coprime, $\operatorname{lcm}(m_j : j \in S) = \prod_{j \in S} m_j$.

Thus inclusion–exclusion gives

$$\#(\text{not divisible by any } m_j) = N \sum_{S \subseteq \{1,\dots,t\}} (-1)^{|S|} \frac{1}{\prod_{j \in S} m_j}.$$

Factor the sum:

$$\sum_S (-1)^{|S|} \frac{1}{\prod_{j \in S} m_j} = \prod_{j=1}^{t} \Big(1 - \frac{1}{m_j} \Big).$$

Therefore

$$\boxed{\# = N\prod_{j=1}^{t}\Big(1-\frac{1}{m_j}\Big) = aM\prod_{j=1}^{t}\Big(1-\frac{1}{m_j}\Big) = a\prod_{j=1}^{t}(m_j-1)}\,.$$

Quick sanity check

Take $t=2$, $m_1=2$, $m_2=3$, $a=1$. In $[0,6)$ the numbers not divisible by 2 or 3 are $\{1,5\}$ (2 of them). Formula: $a(m_1-1)(m_2-1)=1\cdot 1\cdot 2=2$. Matches.

Remarks

- The pairwise coprimality is what makes the answer so clean: it turns all lcms into products and keeps counts exact over the length-N interval.
- Conceptually, there are $\prod_{j=1}^{t}(m_j-1)$ "good" residues modulo M; over a full blocks of length M, that scales to $a\prod_{j=1}^{t}(m_j-1)$.

28 [M21]

Let $J_{n,m}$ be the "Josephus permutation" on $\{1,\dots,n\}$ that maps each person i to the rank at which i is eliminated when we repeatedly count m around a circle and remove the counted person. Show that, when permutations are multiplied left to right (Knuth's convention in this section), $J_{n,m}$ equals

$$(n\,n{-}1\ \dots\ 2\,1)^{m-1}\ \cdot\ (n\,n{-}1\ \dots\ 2)^{m-1}\ \cdot\ \dots\ \cdot\ (n\,n{-}1)^{m-1},$$

i.e., a product of right-rotations of the tails, each used $m-1$ times.

Notation and intuition

- For $r\in\{1,\dots,n-1\}$ let

  $$\tau_r\ =\ (n\,n{-}1\ \dots\ r)$$

  (cycle notation). This right-rotates the block $\{r,r+1,\dots,n\}$ by one: $r\mapsto n,\ r+1\mapsto r,\ \dots,\ n\mapsto n-1$.
- Thus τ_r^k right-rotates that tail by k steps, leaving $1,\dots,r-1$ fixed.

Operationally, the Josephus process "rotate $m-1$ positions, then delete the front item" repeats on ever shorter lists. The product above does exactly that: first rotate all n items by $m-1$ (factor τ_1^{m-1}), then rotate the remaining $n-1$ items by $m-1$ (factor τ_2^{m-1}), and so on.

Proof by induction on the number of deletions

Consider the evolving list L of survivors kept in linear order (not around a circle), and think of our permutation product acting on labels $1, \dots, n$.

Claim. After applying

$$P_k \;=\; \tau_1^{m-1}\,\tau_2^{m-1}\,\cdots\,\tau_k^{m-1} \quad (1 \le k \le n-1),$$

the first k positions of the list contain, in order, exactly the first k people eliminated by the Josephus procedure with step m.

Base $k = 1$. τ_1^{m-1} rotates the entire list right by $m-1$. The person now at position 1 is precisely the m-th around the circle from the original start—i.e., the first to be eliminated. (Deleting them corresponds to simply "freezing" position 1 and henceforth acting only on positions $2..n$.)

Inductive step. Suppose the claim holds for $k-1$. The next Josephus step on the remaining $n-(k-1)$ people again amounts to a right-rotation by $m-1$ of the active suffix (positions $k..n$). That is exactly τ_k^{m-1}: it leaves the already fixed prefix $1..k-1$ alone and right-rotates the tail $k..n$ by $m-1$, bringing the next person to be eliminated into position k.

Thus P_k places the first k eliminated people at positions $1..k$. At $k = n-1$ the whole elimination order is realized in positions $1..n$.

Finally, interpret the resulting arrangement: the permutation that sends a person i to their elimination rank is precisely the total product

$$J_{n,m} \;=\; \tau_1^{m-1}\,\tau_2^{m-1}\,\cdots\,\tau_{n-1}^{m-1},$$

which is the desired identity.

Worked example: $n = 8,\ m = 4$

The factors are

$$(8\,7\,6\,5\,4\,3\,2\,1)^3 \;\cdot\; (8\,7\,6\,5\,4\,3\,2)^3 \;\cdot\; \cdots \;\cdot\; (8\,7)^3.$$

Applying them left-to-right to the initial order $[1, 2, 3, 4, 5, 6, 7, 8]$:

1. rotate all 8 by 3: the 4th (person 4) moves to the front $\to$ first eliminated;
2. rotate the tail $2..8$ by 3: person 8 moves into position 2 $\to$ second eliminated;

3. rotate the tail 3..8 by 3: person 5 moves into position 3 → third eliminated; … and so on, yielding the elimination order $4, 8, 5, 2, 1, 3, 7, 6$.

Therefore $J_{8,4}$ maps person → rank:

$$1 \mapsto 5,\ 2 \mapsto 4,\ 3 \mapsto 6,\ 4 \mapsto 1,\ 5 \mapsto 3,\ 6 \mapsto 8,\ 7 \mapsto 7,\ 8 \mapsto 2,$$

i.e. $J_{8,4} = (1\ 5\ 3\ 6\ 8\ 2\ 4)(7)$, exactly as stated.

Remarks

- The same reasoning shows the inverse permutation (rank → person eliminated at that rank) is

$$J_{n,m}^{-1} \;=\; (1\,2\,\dots\,n)^{\,m-1} \cdot (2\,3\,\dots\,n)^{\,m-1} \cdots (n-1\,n)^{\,m-1},$$

  i.e., left-rotations of growing prefixes.

- The proof is completely constructive: it's a concise algebraic description of the standard "rotate by $m-1$, remove front" simulation of Josephus.

29 [M25]

Let J_n be the Josephus permutation on $\{1, \dots, n\}$ for the case $m = 2$ (i.e., "skip one, remove one," all the way around). The exercise claims you can get the cycle form of J_n from the *perfect shuffle* on $\{1, \dots, 2n\}$, namely the permutation

$$S:\ (1, 2, \dots, 2n) \longmapsto (2, 4, \dots, 2n,\ 1, 3, \dots, 2n-1),$$

by (i) writing the cycles of S, (ii) reversing each cycle, then (iii) erasing every entry $> n$. The example for $n = 11$ in the book is

$$S = (1\,2\,4\,8\,16\,9\,18\,13\,3\,6\,12)\,(5\,10\,20\,17\,11\,22\,21\,19\,15\,7\,14)$$

$\Downarrow$ reverse and delete (> 11) $\quad\Rightarrow\quad$ $J_{11} = (7\,11\,10\,5)\,(6\,3\,9\,8\,4\,2\,1)$. (same cycles up to rotation)

(Stated on p. 184–185.)

Step 1: Work with the perfect shuffle via an explicit formula

It's convenient to view S as multiplication by 2 modulo $2n+1$ on the set $\{1, 2, \dots, 2n\}$ (with residue 0 excluded):

$$S(x) \equiv 2x \pmod{2n+1}, \qquad 1 \le x \le 2n.$$

Hence its inverse (i.e., "reversing" the cycles) is

$$S^{-1}(y) = \begin{cases} y/2, & y \text{ even;} \\ \dfrac{y+2n+1}{2}, & y \text{ odd.} \end{cases} \tag{ }$$

So traversing a cycle of S *backwards* is the same as repeatedly applying (). (This is exactly what "reverse left and right" means in the exercise.)

Step 2: "Reverse and delete" induces a permutation on $\{1, \dots, n\}$

Define a map $T : \{1, \dots, n\} \to \{1, \dots, n\}$ by: start at $k \le n$, apply S^{-1} repeatedly until you hit the next value that is $\le n$; call that value $T(k)$. Equivalently: write the cycle of S that contains k, read it *backwards*, and delete all entries $> n$; the surviving entries appear in cyclic order and give the cycle(s) of T.

Using () gives a very simple description of T:

- If k is even, then one step of () stays $\le n$: $T(k) = k/2$.
- If k is odd, the first step of () jumps to $> (n)$; you keep applying () until you land back in $\{1, \dots, n\}$. Grouping those "> n" hops together yields the recurrence

$$T_n(k) = \begin{cases} k/2, & k \text{ even,} \\ \lfloor n/2 \rfloor + T_{\lceil n/2 \rceil}(\frac{k+1}{2}), & k \text{ odd,} \end{cases} \tag{†}$$

where the subscript on T reminds us of the current n. (Intuitively, each odd k moves into the "upper half" after one round, and what happens next is the same problem on the $\lceil n/2 \rceil$ survivors, then shifted up by $\lfloor n/2 \rfloor$.)

Step 3: Identify T with the Josephus permutation J_n (for $m = 2$)

The Josephus process with $m = 2$ has exactly the same decomposition:

- First pass removes all the even numbers, pairing each even k with $k/2$ in the cycle structure—this is the "skip one, delete next" dynamics written as a permutation.
- What remains (in order) are the $\lceil n/2 \rceil$ odd numbers. But after the last deletion we are "one step ahead," so the next stage is *the same Josephus problem on* $\lceil n/2 \rceil$ *items*, and when lifted back to $\{1, \dots, n\}$ those images are shifted by $\lfloor n/2 \rfloor$.

This yields precisely the same recurrence (†) for J_n. Therefore $T_n = J_n$, i.e., the permutation obtained by "reverse the perfect-shuffle cycles and erase all $> n$" is the Josephus permutation for $m = 2$. That is exactly the statement to be proved.

Step 4: Sanity-check on the book's example ($n = 11$)

From (†) with $n = 11$, $\lfloor n/2 \rfloor = 5$, $\lceil n/2 \rceil = 6$.

- Evens map immediately: $10 \to 5,\ 8 \to 4,\ 6 \to 3,\ 4 \to 2,\ 2 \to 1$.
- Odds use the recursive branch:
  - $1 \mapsto 5 + T_6(1)$. Now $T_6(1) = 3$ (compute similarly), so $1 \mapsto 8$? but continuing the recursion gives $1 \mapsto 6$, then $6 \mapsto 3 \mapsto 9 \mapsto 8 \mapsto 4 \mapsto 2 \mapsto 1$, i.e., the 7-cycle $(6\,3\,9\,8\,4\,2\,1)$.
  - The remaining odds form $(7\,11\,10\,5)$. These are exactly the two cycles printed in the exercise. (The slight rotation of cycle entries is immaterial.)

A takeaway algorithm (practical computation)

To build J_n in cycle form:

1. Build the perfect-shuffle cycles of $S(x) \equiv 2x \pmod{2n+1}$ on $\{1, \dots, 2n\}$.
2. Reverse each cycle.
3. Delete all numbers $> n$.
4. The remaining strings, read cyclically, are the cycles of J_n.

30 [M24]

Let J_n be the Josephus permutation on $\{1, \dots, n\}$ for step $m = 2$. Using the link with the "perfect shuffle" from exercise 29, show that the fixed points of J_n are exactly the integers

$$i = \frac{(2^d - 1)(2n+1)}{2^{d+1} - 1} \qquad (d = 1, 2, 3, \dots)$$

for those d for which this expression is an integer.

Key link from Exercise 29

Write $p = 2n + 1$. The "perfect shuffle" on $\{1, \dots, 2n\}$ is the permutation

$$S(k) \equiv 2k \pmod{p} \quad (1 \le k \le 2n),$$

i.e., $(1, 2, \dots, 2n) \mapsto (2, 4, \dots, 2n, 1, 3, \dots, 2n-1)$. Exercise 29 states that to get the Josephus permutation J_n (for $m = 2$) you:

1. write the cycles of the perfect shuffle S; 2) reverse each cycle; 3) delete every entry $> n$. The remaining numbers $1, \dots, n$ in each cycle (in that reversed order) form the cycles of J_n.

When is i a fixed point?

By the rule above, $i \in \{1, \dots, n\}$ is fixed by J_n iff in its S-cycle it is the only number $\leq n$ (because reversing and then deleting $> n$ must leave a 1-cycle (i)).

Now the cycles of S are the orbits of repeated doubling modulo p:

$$i,\ 2i,\ 4i,\ 8i,\ \dots \quad (\text{mod } p).$$

So "i is the only member $\leq n$" means:

$$\{2^k i \bmod p : k \in \mathbb{Z}\} \cap \{1, \dots, n\} = \{i\}.$$

Binary (doubling-map) view

Set $x = i/p \in (0, 1)$. The map $x \mapsto \{2x\}$ (fractional part) exactly tracks $k \mapsto 2^k i \bmod p$ scaled by p. Note:

$$2^k i \bmod p \leq n \iff \{2^k x\} < \tfrac{1}{2}.$$

Thus along one full period (the cycle length of i under S), the indicator of "$\leq n$" is the 0/1 word telling whether $\{2^k x\} < \frac{1}{2}$ (write 0) or $\geq \frac{1}{2}$ (write 1). A fixed point requires that this periodic word have exactly one 0 per period.

A binary expansion with exactly one 0 in each period of length L is (up to rotation) the purely repeating block

$$\overline{0\,\underbrace{11\cdots1}_{L-1}}\,,$$

whose value is

$$x = \frac{2^{L-1} - 1}{2^L - 1}.$$

(Compute: one period contributes $\sum_{k=2}^{L} 2^{-k} = \frac{1}{2} - 2^{-L}$; repeat geometrically to get $(2^{L-1} - 1)/(2^L - 1)$.)

Therefore

$$\frac{i}{p} = \frac{2^{L-1}-1}{2^L-1} \quad \Longrightarrow \quad i = \frac{(2^{L-1}-1)\,p}{2^L-1}.$$

Write $d = L-1$ to match the exercise:

$$i = \frac{(2^d-1)\,(2n+1)}{2^{d+1}-1}.$$

Integrality divisor condition

$\gcd(2^{d+1}-1,\, 2^d-1) = 1$ (since $(2^{d+1}-1) - 2(2^d-1) = 1$). Hence the fraction is an integer iff

$$2^{d+1}-1 \;\mid\; 2n+1.$$

Conversely, if $2^{d+1}-1 \mid 2n+1$, the corresponding i yields a period whose 0/1 word has exactly one 0, so i is indeed the unique $\leq n$ element in its S-cycle and therefore a fixed point of J_n.

This proves that the fixed points are exactly the integers

$$i = \frac{(2^d-1)(2n+1)}{2^{d+1}-1} \quad \text{for all } d \geq 1 \text{ with } (2^{d+1}-1) \mid (2n+1),$$

as claimed.

Quick checks

- $n = 11 \Rightarrow p = 23$. No d has $2^{d+1}-1 \mid 23$, so there are no fixed points. Indeed the example cycle in Ex. 29 has no fixed element.
- $n = 3 \Rightarrow p = 7 = 2^3-1$ (take $d = 2$): $i = (4-1)\cdot 7/(8-1) = 3$. The $m = 2$ Josephus permutation on $\{1,2,3\}$ is $(1\,2)(3)$, so 3 is fixed.

31 [HM38]

A. The position of the j-th execution

We have n people labeled $1, 2, \ldots, n$ around a circle; counting proceeds m at a time and each time the person counted is executed (the Josephus process). Let $E_{n,m}(j)$ be the label of the j-th person executed. The exercise claims:

Start with $x \leftarrow j\,m$. While $x > n$, replace x by

$$x \;\leftarrow\; \left\lfloor \frac{m(x-n)-1}{m-1} \right\rfloor.$$

When the loop stops, $x = E_{n,m}(j)$.

We'll prove this.

Key lemma (compressing away multiples of m)

Let u_k be the k-th positive integer that is not a multiple of m. Then

$$u_k \;=\; \left\lfloor \frac{mk-1}{m-1} \right\rfloor.$$

Proof. Among the first s positive integers there are $\lfloor s/m \rfloor$ multiples of m, hence $s - \lfloor s/m \rfloor$ nonmultiples. The minimal s for which this count reaches k is exactly $s = \lfloor \frac{mk-1}{m-1} \rfloor$. $\square$

One pass around the circle

Make a first linear sweep from label 1 up to n without wrapping. Exactly the labels

$$m,\ 2m,\ 3m,\ \ldots,\ \lfloor \tfrac{n}{m} \rfloor m$$

are executed during this sweep. Thus, if $j \le \lfloor n/m \rfloor$ then

$$E_{n,m}(j) = j\,m,$$

i.e., the rule "$x \leftarrow jm$; if $x \le n$ stop" is correct.

After one sweep: renumber the survivors

If $j > \lfloor n/m \rfloor$, write $j = j_1 + \lfloor n/m \rfloor$. After the first sweep there remain $n' = n - \lfloor n/m \rfloor$ survivors; the counting restarts at the next person after the last executed one, and the process among survivors is again Josephus with the same step m.

If we list the survivors in the order we will now encounter them, this list is obtained from the integers by deleting all multiples of m and then "cutting" at the restart point. Deleting multiples is exactly the situation of the lemma: the i-th survivor in natural order corresponds to original label $u_i = \lfloor \frac{mi-1}{m-1} \rfloor$. The restart cut is an overall rotation, which does not affect the label of the j-th execution once we express it back in the original labels (it only affects where we start counting, not who the j-th to die is when measured by original labels).

Now, in the unaided linear count we would "want" to go to label $x = j\,m$. If $x > n$, the amount by which we overshoot the first sweep is

$$i \;=\; x-n \;=\; jm-n,$$

i.e., we need the i-th survivor (because every overshoot step lands on a survivor after we have deleted those killed in the first sweep). By the lemma, the original label of that survivor is

$$\lfloor \frac{m\,i-1}{m-1} \rfloor \;=\; \left\lfloor \frac{m(x-n)-1}{m-1} \right\rfloor,$$

which is exactly the exercise's update. This reduces the problem to the same form with a (typically) smaller x, and we repeat until $x \le n$. Hence the algorithm indeed yields $E_{n,m}(j)$. $\square$

Tiny check (example). Take $n = 10$, $m = 3$, $j = 7$. Start $x = 21$.

- $x > 10$, so $x \leftarrow \lfloor (3(21-10)-1)/2 \rfloor = \lfloor (33-1)/2 \rfloor = 16$.
- $x > 10$, so $x \leftarrow \lfloor (3(16-10)-1)/2 \rfloor = \lfloor (18-1)/2 \rfloor = 8$.
- Now $x = 8 \le 10$. Thus the 7-th executed is label 8. You can verify by hand the elimination order matches.

B. Average number of fixed points

A fixed point of the Josephus permutation for given (n, m) means an index $j \in \{1, \dots, n\}$ with $E_{n,m}(j) = j$.

The exercise asserts that, for fixed $m > 1$, the average number of fixed points over $n = 1, 2, \dots, N$ tends (as $N \to \infty$) to

$$C_m \;=\; \sum_{k \ge 1} \frac{(m-1)^k}{m^{k+1} - (m-1)^k}.$$

Why that constant appears

Run the computation of $E_{n,m}(j)$ as in part A:

$$x_0 = jm, \qquad x_{t+1} = \left\lfloor \frac{m(x_t - n) - 1}{m-1} \right\rfloor \quad \text{while } x_t > n.$$

A fixed point j occurs exactly when the loop runs for some $k \ge 1$ steps and then lands on $x_k = j \le n$ with all previous $x_t > n$ $(0 \le t < k)$.

If you examine the inequalities that encode "continue at step t" vs. "stop at step k and hit j", you find that they constrain the lowest $k{+}1$ base-m digits of n. (Intuitively: each time we delete multiples of m we are partitioning numbers into blocks of length m with $m{-}1$ survivors, so one more step of the loop peels one more base-m digit.) For a fixed k,

- there are exactly $(m-1)^k$ patterns of the first k base-m digits that make the algorithm *continue* for k steps; and
- among the m^{k+1} possibilities for the first $k+1$ base-m digits, exactly $m^{k+1}-(m-1)^k$ make the algorithm *terminate on the k-th step hitting j* (the complement of the "keep going" patterns at level k).

Therefore, for each fixed j, the natural density (as $N \to \infty$) of integers n for which j is a fixed point *with exactly k loop iterations* equals

$$\frac{(m-1)^k}{m^{k+1}-(m-1)^k}.$$

(Details are an easy induction on k, formalizing the digit-count just sketched; the floors and the -1 in the update only shift interval endpoints by $O(1)$ and don't affect asymptotic densities.)

Finally, average over all $k \geq 1$ (disjoint cases) to get C_m. Since this probability is independent of j, the average number of fixed points (averaged over $n \leq N$) tends to the same constant C_m as $N \to \infty$.

Sanity checks.

- For $m = 2$: $C_2 = \sum_{k\geq 1} \frac{1}{2^{k+1}-1} \approx 0.6067$, which indeed lies between $(m-1)/m = 1/2$ and 1, as the exercise notes.
- For $m = 3$: $C_3 = \sum_{k\geq 1} \frac{2^k}{3^{k+1}-2^k} \approx 0.695$ (adding a dozen terms gives a good approximation).

Summary. (i) The loop rule "$x \leftarrow jm$; while $x > n$ set $x \leftarrow \lfloor \frac{m(x-n)-1}{m-1} \rfloor$" correctly computes the label of the j-th execution, because each wrap compresses the circle by deleting all multiples of m, and the i-th survivor among the integers is $\lfloor \frac{mi-1}{m-1} \rfloor$. (ii) Counting admissible base-m digit patterns shows that the average number of fixed points converges to $\displaystyle\sum_{k\geq 1} \frac{(m-1)^k}{m^{k+1}-(m-1)^k}$.

32 [M25]

Let

$$\pi \;=\; (2\,3)^{e_2}(4\,5)^{e_4}\cdots(2m\;\,2m{+}1)^{e_{2m}}\;\;(1\,2)^{e_1}(3\,4)^{e_3}\cdots(2m{-}1\;\,2m)^{e_{2m-1}},$$

where each $e_k \in \{0,1\}$. (a) Show that in one–line notation $\pi = \pi_1\pi_2\cdots\pi_{2m+1}$ we have $|\pi_k - k| \leq 2$ for all k. (b) For any permutation ρ of $\{1,\ldots,n\}$ construct such a π so that $\rho\pi$ is a single cycle ("near" a cycle).

I'll prove (a), then give a concrete, correct construction for (b) and prove it works.

(a) Displacement is at most 2

Write

$$O \;=\; \prod_{i=1}^{m}(2i{-}1\ 2i)^{e_{2i-1}}, \qquad E \;=\; \prod_{i=1}^{m}(2i\ 2i{+}1)^{e_{2i}},$$

so that $\pi = E \circ O$ (rightmost factor acts first). Inside each product the transpositions are pairwise disjoint, hence commute.

Fix any k. The only adjacent transpositions that can possibly move k are $(k{-}1\ k)$ and $(k\ k{+}1)$. In our factorization, at most one of these appears in O and the other (if used) appears in E. Therefore:

- Applying O can move k by at most 1 place (either swap with $k{+}1$, or do nothing).
- Applying E afterwards can again move the current value by at most 1 place (either swap with its left or right neighbor, or do nothing).

Thus the total displacement is at most $1 + 1 = 2$. Formally, for all k,

$$\pi(k) \in \{k-2, k-1, k, k+1, k+2\} \quad \Longrightarrow \quad |\pi_k - k| \le 2.$$

That proves (a).

(b) Make $\rho\pi$ a single cycle (explicit two-pass construction)

The key basic fact we'll use is this:

Lemma (cut–join): If σ is any permutation and $\tau = (a\,b)$ a transposition, then the number of cycles of $\sigma\tau$ is

- one less than that of σ if a and b lie in different cycles of σ (the two cycles are joined);
- one more if a and b lie in the same cycle of σ (the cycle is cut into two).

Proof sketch. Right-multiplication by $(a\,b)$ swaps the outgoing images of a and b in the functional digraph of σ, which either connects two distinct directed cycles into one, or splits one directed cycle into two.

We now choose the exponents $e_k \in \{0, 1\}$ in two passes so that we only ever join cycles (never split). This monotonicity will force the result to become a single cycle.

Assume first that n is odd, $n = 2m{+}1$ (the even case is handled afterwards).

Algorithm (odd pass then even pass). Let $\sigma \leftarrow \rho$.

1. Odd indices (right factor O). For $j = 1, 3, 5, \dots, 2m{-}1$: If j and $j{+}1$ lie in different cycles of σ, set $e_j \leftarrow 1$ and update $\sigma \leftarrow \sigma\,(j\ j{+}1)$. Otherwise set $e_j \leftarrow 0$ and leave σ unchanged.

2. Even indices (left factor E). For $j = 2, 4, 6, \dots, 2m$: If j and $j{+}1$ lie in different cycles of the current σ, set $e_j \leftarrow 1$ and update $\sigma \leftarrow \sigma\,(j\ j{+}1)$. Else set $e_j \leftarrow 0$.

Finally, let π be the permutation with these e_k in the specified form:

$$\pi \;=\; (2\,3)^{e_2}(4\,5)^{e_4}\cdots(2m\ 2m{+}1)^{e_{2m}}\ \ (1\,2)^{e_1}(3\,4)^{e_3}\cdots(2m{-}1\ 2m)^{e_{2m-1}}.$$

Why this works.

- By the lemma and our "if different, then apply" rule, each chosen transposition reduces the number of cycles by exactly 1; when we skip, the number of cycles doesn't increase. Hence the number of cycles in σ is nonincreasing throughout.
- Suppose after both passes σ still has at least two cycles. Then along the line $1, 2, \dots, 2m{+}1$ there exists an adjacent pair $j, j{+}1$ that lie in different cycles of the final σ. Consider the pass in which index j was processed. Because the cycle count never increases later, j and $j{+}1$ were also in different cycles at that time, so our rule would have applied the transposition $(j\ j{+}1)$, merging them—a contradiction. Therefore after the two passes $\sigma = \rho\pi$ has exactly one cycle.
- By construction, π has the required two-layer form, so it also satisfies (a): $|\pi_k - k| \le 2$.

Even n. If n is even, do exactly the same two passes, but the "even layer" stops at $j = n - 2$ (there is no pair $(n, n{+}1)$). Define

$$\pi \;=\; (2\,3)^{e_2}(4\,5)^{e_4}\cdots(n{-}2\ n{-}1)^{e_{n-2}}\ \ (1\,2)^{e_1}(3\,4)^{e_3}\cdots(n{-}1\ n)^{e_{n-1}}.$$

The identical argument shows $\rho\pi$ is a single cycle, and again $|\pi_k - k| \le 2$.

Summary. (a) The two-layer "odd then even adjacent swaps" can move any element by at most two positions, hence $|\pi_k - k| \le 2$. (b) The two-pass greedy rule (apply $(j\ j{+}1)$ iff it joins two different cycles) yields exponents e_k so that $\rho\pi$ is an n-cycle. Thus every permutation is within right-multiplication by a π of displacement at most 2 from a single cycle ("near" a cycle).

33 [M33]

Let $m = 2^{2\ell}$ and $n = 2^{2\ell+1} = 2m$. You must build m "row sequences" $\{\alpha_{i1}, \dots, \alpha_{in}\}$ (one sequence for each $i \in \{0, \dots, m-1\}$) and m "column sequences" $\{\beta_{1j}, \dots, \beta_{nj}\}$ (one sequence for each $j \in \{0, \dots, m-1\}$), where every α_{ik} and β_{kj} is a permutation of $\{1, 2, 3, 4, 5\}$, such that for all i, j

$$\alpha_{i1}\beta_{1j}\,\alpha_{i2}\beta_{2j}\cdots\alpha_{in}\beta_{nj} \;=\; \begin{cases}(1\ 2\ 3\ 4\ 5), & \text{if } i=j,\\ (), & \text{if } i\neq j,\end{cases}$$

i.e., you get a fixed 5-cycle on the diagonal and the identity off the diagonal.

Key idea

Work inside the dihedral subgroup $D_{10} \subset S_5$ generated by

$$r = (1\ 2\ 3\ 4\ 5) \quad \text{and} \quad s = (2\ 5)(3\ 4).$$

These satisfy the fundamental relation

$$srs = r^{-1} \quad \Rightarrow \quad sr^k s = r^{-k} \ \text{ for all integers } k,$$

and $s^2 = ()$, $r^5 = ()$. We'll interleave powers of r (coming from the α's) with copies of s (coming from the β's) in a way that makes everything cancel except on the diagonal, where one block collapses to r.

The construction (explicit)

Index the $n = 2m$ positions as $2t+1, 2t+2$ for blocks $t = 0, 1, \dots, m-1$.

Row sequences $\alpha_{i\bullet}$ (one for each $i \in \{0, \dots, m-1\}$):

- In block $t = i$: set $\alpha_{i,2i+1} = r^3$ and $\alpha_{i,2i+2} = r^{-3}$.
- In every other block $t \neq i$: set $\alpha_{i,2t+1} = \alpha_{i,2t+2} = ()$ (the identity).

Column sequences $\beta_{\bullet j}$ (one for each $j \in \{0, \dots, m-1\}$):

- In block $t = j$: set $\beta_{2j+1,j} = s$ and $\beta_{2j+2,j} = s$.
- In every other block $t \neq j$: set $\beta_{2t+1,j} = \beta_{2t+2,j} = ()$.

(Using the identity is allowed, since it's a permutation of $\{1, 2, 3, 4, 5\}$.)

Why this works

Fix i, j and multiply block by block.

- If $i \neq j$: In block $t = i$ we have $r^3 \cdot () \cdot r^{-3} \cdot () = ()$. In block $t = j$ we have $() \cdot s \cdot () \cdot s = s^2 = ()$. All other blocks are all identities. Hence the total product is $()$.

- If $i = j$: The only nontrivial factors lie in block $t = i = j$, where the four-term product is

$$r^3\, s\, r^{-3}\, s.$$

  Using $sr^{-3}s = r^3$ (since $sr^k s = r^{-k}$), the block reduces to

$$r^3\, r^3 = r^6 = r^5 \cdot r = r = (1\ 2\ 3\ 4\ 5).$$

  All other blocks contribute the identity, so the full product is exactly r.

This is precisely the required "orthogonality" property.

Notes and small checks

- We used only that $n = 2m$ (which follows from $m = 2^{2\ell}$, $n = 2^{2\ell+1}$).
- The choice of the exponent 3 is convenient because $r^3 s r^{-3} s = r^{2\cdot 3} = r$. More generally, any c with $2c \equiv 1 \pmod 5$ (i.e., $c \equiv 3$) would work.
- Any 5-cycle conjugate to r and any involution s that inverts r (i.e., $srs = r^{-1}$) give equivalent solutions by conjugation inside S_5.

Tiny example (first nontrivial size)

Take $\ell = 1 \Rightarrow m = 4,\ n = 8$. Rows $i = 0, 1, 2, 3$ have r^3, r^{-3} only in their own block; columns $j = 0, 1, 2, 3$ have s, s only in their own block.

- For $i = j = 2$: the block $t = 2$ contributes $r^3 s r^{-3} s = r$; others are identities $\to$ total r.
- For $i = 1,\ j = 3$: block $t = 1$ gives $r^3 \cdot () \cdot r^{-3} \cdot () = ()$; block $t = 3$ gives $() \cdot s \cdot () \cdot s = ()$; others are $() \to$ total $()$.

Thus the construction meets the specification.

34 [M25]

We have an array split into two consecutive blocks, of length m and of length n (total N = m+n). We want to transform the array from to , i.e., rotate the array left by m (equivalently, right by n). Consider the permutation

$$p(k) \equiv k + m \pmod N \qquad (0 \le k < N)$$

that maps each position to where its element must go after rotation. Show that this "cyclic-shift" permutation has a simple cycle structure, and exploit that structure to devise a simple (in-place) algorithm for the rearrangement.

1) Cycle structure of a fixed shift on N items

Let $N = m + n$ and $p(k) = k + m \pmod{N}$. Consider the orbit (cycle) of an index r:

$$r \;\to\; r+m \;\to\; r+2m \;\to\; \cdots \;\to\; r+tm \pmod{N}.$$

The cycle closes at the smallest $t > 0$ such that $tm \equiv 0 \pmod{N}$. Let $g = \gcd(m, N)$. Then $tm \equiv 0 \pmod{N}$ iff $N \mid tm$, whose least positive solution is

$$t = \frac{N}{\gcd(m, N)} = \frac{N}{g}.$$

Therefore:

- All cycles have the same length $L = N/g$.
- There are exactly $g = \gcd(m, N)$ disjoint cycles.
- A set of cycle representatives is $r = 0, 1, \ldots, g-1$. The r-th cycle is

$$C_r = \{\, r,\ r+m,\ r+2m,\ \ldots,\ r+(L-1)m \,\} \pmod{N}.$$

Why this is "simple": the permutation is a pure rotation by a fixed stride m on the ring $\mathbb{Z}_N$. The orbits are exactly the cosets of the subgroup generated by m, hence equal-length cycles determined by $\gcd(m, N)$.

Example. $N = 10$, $m = 4$, $g = \gcd(10, 4) = 2$. The cycles are

$$C_0 = (0, 4, 8, 2, 6), \qquad C_1 = (1, 5, 9, 3, 7),$$

each of length $10/2 = 5$. Rotating left by 4 maps each index to the next in its cycle.

2) In-place algorithm from the cycle structure ("juggling")

The cycle structure tells us exactly how to move elements without extra storage, using at most one temporary per cycle.

Algorithm (left-rotate by m positions).

```
N = m + n
g = gcd(m, N)
for r in 0 .. g-1:
    temp = A[r]
    j = r
    while True:
        k = (j + m) mod N
        if k == r:
            break
        A[j] = A[k]
        j = k
    A[j] = temp
```

Correctness. The inner loop traverses the cycle C_r in order: $j = r, r+m, r+2m, \ldots$. Because the cycle length is $L = N/g$, we visit each index in C_r exactly once and finally return to r, at which point we place the saved value `temp`. Since the cycles are disjoint and cover $\{0, \ldots, N-1\}$, every element is moved exactly once.

Cost. Each array position participates in exactly one assignment (plus one final write per cycle), so the time is $\Theta(N)$ and the extra space is $O(1)$ (only one temporary at a time). The number of cycles is $g = \gcd(m, N)$, so there are g temporaries used one after another.

3) A different simple in-place method (three reversals)

Another elegant $O(N)$-time, $O(1)$-space solution uses reversals:

1. reverse the block (positions 0..m−1),
2. reverse the block (positions m..N−1),
3. reverse the whole array (0..N−1).

Why it works: Writing strings, $(\alpha\beta) \mapsto (\alpha^R\beta^R) \mapsto (\alpha^R\beta^R)^R = \beta\alpha$. Each reversal can be done in place with two-pointer swaps. (This is a standard trick and dovetails with the "block transpositions" theme introduced by the exercise.)

4) Which to use?

- Cycle-juggling (from the permutation's cycles): one pass over N items, $O(1)$ extra space, straightforward once you know $g = \gcd(m, N)$. This directly "exploits the cycle structure" the problem asks you to identify.
- Three-reversal method: also $O(N)$ time, $O(1)$ space; often slightly simpler to code and branch-free.

Both meet the exercise's request for a simple algorithm derived from the structure of the cyclic-shift permutation. The key structural fact is the decomposition into $g = \gcd(m, m+n)$ cycles, each of length N/g.

(Optional) Small worked example

Let $A = [a, b, c, d, e, f, g, h, i, j]$, $m = 4$ (=[a,b,c,d], =[e,f,g,h,i,j], $N = 10$). $g = \gcd(10, 4) = 2$. Two cycles:

- $C_0 = (0, 4, 8, 2, 6)$: $A[0] \leftarrow A[4] = e$, $A[4] \leftarrow A[8] = i$, $A[8] \leftarrow A[2] = c$, $A[2] \leftarrow A[6] = g$, $A[6] \leftarrow a$.
- $C_1 = (1, 5, 9, 3, 7)$: $A[1] \leftarrow A[5] = f$, $A[5] \leftarrow A[9] = j$, $A[9] \leftarrow A[3] = d$, $A[3] \leftarrow A[7] = h$, $A[7] \leftarrow b$.

Result: $[e, f, g, h, i, j, a, b, c, d] = \beta\alpha$.

Summary

- Rotation $\rightarrow$ corresponds to the permutation $p(k) = k + m \pmod N$.
- Its cycle structure is $g = \gcd(m, N)$ cycles of equal length N/g.
- Traversing those cycles yields an in-place juggling algorithm: $\Theta(N)$ time, $O(1)$ extra space.
- The three-reversal method is an equally simple in-place alternative.

35 [M39]

Let an array be split into three consecutive blocks $\alpha\beta\gamma$ of lengths l, m, n (so total $M = l+m+n$). We want to rearrange in place to $\gamma\beta\alpha$. Show that the corresponding permutation has a simple cycle structure that leads to an efficient algorithm. (Hint in the book: reduce $(\alpha\beta)(\gamma\beta)$ to $(\gamma\beta)(\alpha\beta)$.)

(For context, Exercise 34 (the 2-block case) rewrites $\alpha\beta$ to $\beta\alpha$ via the permutation $k \mapsto (k + m) \bmod (m + n)$ and exploits its cycle structure. We'll generalize that idea.)

Embed $\alpha\beta\gamma$ into a *virtual* longer string $\alpha\beta\gamma\beta$ and perform a single circular shift by $l + m$ positions on that longer string. If we then "ignore" the extra trailing β, the first $M = l+m+n$ positions have become exactly $\gamma\beta\alpha$.

Formally, set

$$M = l + m + n, \qquad N = l + 2m + n = M + m, \qquad s = l + m.$$

Consider the rotation permutation on $\{0, \dots, N - 1\}$,

$$h(k) = (k + s) \bmod N.$$

On the *virtual* array $\alpha\beta\gamma\beta$ (length N), h moves every cell forward by s positions. The desired 3-block rearrangement on the *real* array of length M is obtained by restricting h to indices $<$

M and "skipping over" any visit to the shadow region $[M, N)$ (the duplicated β). The result is a permutation of $\{0, \dots, M-1\}$ whose cycles we can follow in place.

The book's answer states this succinctly and also gives the cycle count; we'll derive and then match it.

Cycle structure and how many cycles

The rotation $h(k) = (k+s) \bmod N$ decomposes $\{0, \dots, N-1\}$ into

$$d = \gcd(s, N) = \gcd(l+m,\ l+2m+n) = \gcd(l+m,\ m+n)$$

cycles, each of length N/d. (The equality of the gcd's is immediate from $\gcd(s, N) = \gcd(l+m, (l+2m+n)-(l+m)) = \gcd(l+m, m+n)$.)

When we strike out the indices in the shadow tail $[M, N)$ from each rotation cycle and connect neighbors across the removed nodes, we still get cycles—now on the real index set $\{0, \dots, M-1\}$. Thus the 3-block permutation for $\alpha\beta\gamma \to \gamma\beta\alpha$ has exactly

$$\boxed{d = \gcd(l+m,\ m+n)}$$

cycles; walking those cycles gives an in-place algorithm. This is precisely the conclusion in the official answer.

An in-place cycle-walking algorithm

Define a "skip-aware next" function on real positions $0 \le j < M$:

$$g(j) = \begin{cases} (j+s) \bmod N, & \text{if } (j+s) \bmod N < M; \\ (j+2s) \bmod N, & \text{otherwise} \quad \text{(skip over the shadow } \beta). \end{cases}$$

Then the desired permutation is exactly "send j to $g(j)$". Implement it by cycling:

```
s = l + m
M = l + m + n
N = l + 2m + n
visited[0..M-1] = false   // or do gcd-based leaders to avoid a visited array

for start in 0..d-1:         // d = gcd(l+m, m+n)
    if visited[start]: continue
    t = A[start]             // hold one element
    j = start
    while True:
```

```
// compute next with "skip"
j1 = (j + s) % N
if j1 >= M:
    j1 = (j1 + s) % N    // skip shadow
if j1 == start:
    A[j] = t             // close the cycle
    break
A[j] = A[j1]
visited[j] = true
j = j1
```

Why the two steps when `j1 >= M`? In the virtual rotation $\alpha\beta\gamma\beta \xrightarrow{+s} \gamma\beta\alpha\beta$, landing in $[M, N)$ means "you're at the duplicated β". The hint in the text says to realize that the two assignments $x_k \leftarrow x_{k'}$ and $x_{k'} \leftarrow x_{k''}$ (with $k' = (k+s) \bmod N \geq M$, $k'' = (k'+s) \bmod N$) effectively place into x_k the correct source $x_{k''}$, because $x_{k'} = x_{k''}$ (both refer to the same logical β). So we jump an extra s.

Complexity and storage

- Each real position is visited exactly once across all cycles, so the algorithm performs M moves (plus at most d temporary saves).
- Time $\Theta(M)$, extra space $O(1)$ beyond one saved element (or $O(d)$ if you prefer one temp per cycle leader).

This is the same optimal profile as the 2-block case (Exercise 34), generalized.

Worked example

Take $l = 2, m = 3, n = 1$. So $\alpha = [a_0, a_1]$, $\beta = [b_0, b_1, b_2]$, $\gamma = [c_0]$. Then $M = 6$, $N = 9$, $s = l + m = 5$, $d = \gcd(5,\ m + n = 4) = 1$: one big cycle.

Indices 0..5 hold $\alpha\beta\gamma = [a_0, a_1, b_0, b_1, b_2, c_0]$. Virtual tail $[6, 7, 8]$ holds a duplicate $\beta = [b_0, b_1, b_2]$.

Walk with skips:

- $0 \to (0+5) = 5 < 6$ put $A[0] \leftarrow A[5] = c_0$
- $5 \to (5+5) = 10 \bmod 9 = 1 < 6$ $A[5] \leftarrow A[1] = a_1$
- $1 \to (1+5) = 6 \geq 6$ skip to $(6+5) = 2$ $A[1] \leftarrow A[2] = b_0$
- $2 \to (2+5) = 7 \geq 6$ skip to $(7+5) = 3$ $A[2] \leftarrow A[3] = b_1$
- $3 \to (3+5) = 8 \geq 6$ skip to $(8+5) = 4$ $A[3] \leftarrow A[4] = b_2$
- $4 \to (4+5) = 0 < 6$ $A[4] \leftarrow A[0] = a_0$
- Next is 0 (start), so write back the saved t to $A[4]$ at the close, yielding $[c_0, b_0, b_1, b_2, a_0, a_1]$ which is $\gamma\beta\alpha$.

36 [27]

Write a MIX subroutine that performs the in-place rearrangement

$$\alpha\beta\gamma \longrightarrow \gamma\beta\alpha$$

using the efficient "cycle-structure" method from Exercise 35, then analyze its running time and compare it with the simpler "four-reversal" trick that goes $\alpha\beta\gamma \to (\alpha\beta\gamma)^R = \gamma^R\beta^R\alpha^R \to \gamma\beta\alpha$.

1) Algorithm (from Ex. 35, made explicit)

The key idea is to compose two adjacent block cyclic shifts of the form $XY \mapsto YX$ (the routine from Ex. 34), chosen so the result is $\gamma\beta\alpha$:

1. First move α past the block $\beta\gamma$:

$$\underbrace{\alpha}_{\ell}\ \underbrace{\beta\gamma}_{m+n} \overset{XY\to YX}{\Longrightarrow} \beta\gamma\,\alpha.$$

2. Then, inside the new prefix, move β past γ:

$$\underbrace{\beta}_{m}\ \underbrace{\gamma}_{n} \overset{XY\to YX}{\Longrightarrow} \gamma\beta.$$

Composing the two gives $\gamma\beta\alpha$ in place, with $O(1)$ extra storage. The hint in Ex. 35 ("change $(\alpha\beta)(\gamma\beta)$ to $(\gamma\beta)(\alpha\beta)$") is exactly this "swap-by-adjacent-blocks" reasoning.

So we only need a solid two-block rotation subroutine $\mathsf{ROT2}(X, Y)$: $XY \to YX$ (the Ex. 34 routine), and call it twice.

Efficient $\mathsf{ROT2}$ via cycle decomposition. Let $|X| = s,\ |Y| = t,\ L = s + t,\ d = \gcd(s, L)$. Partition positions $0, \dots, L-1$ into d cycles under the map $k \mapsto (k + s) \bmod L$. For each cycle, rotate the elements once using one temporary word. Each element is moved exactly once; each cycle incurs one extra store. This is the classic "juggling"/cycle method for array rotation. (This is the same structure you proved in Ex. 34/35.)

2) MIX implementation

Below is a self-contained MIX subroutine that performs $\alpha\beta\gamma \to \gamma\beta\alpha$ using two calls to $\mathsf{ROT2}$. Conventions:

- Memory holds the array, one item per word.
- Inputs on entry:

  - I1 = base address of α,
  - I2 = $\ell = |\alpha|$,
  - I3 = $m = |\beta|$,
  - I4 = $n = |\gamma|$.

- Registers used: A, X, I1..I6. One temp word stored at label TMP.
- Subroutine returns with the array rearranged in place.

```
* ------------------------------------------------------------
* ROT3:    ->      (uses ROT2 twice)
* Inputs: I1=BASE, I2=LEN_A( ), I3=LEN_B(m), I4=LEN_C(n)
* Clobbers: A,X,I1..I6; uses word TMP
* ------------------------------------------------------------
ROT3   ENT5 0           * I5 = scratch
       ENT6 0           * I6 = scratch
       LDA  I2          * A :=
       ADD  I3          * A :=   + m
       ADD  I4          * A :=   + m + n = L
       STA  LSUM        * store total length if desired (not strictly needed)

* Step 1: rotate  past ( ):  [ |  ] -> [  | ]
* Call ROT2(BASE, s= , t=m+n)
       ENTA 0
       ENT3 0           * (we'll pass s in I2 and t in I3 to ROT2)
       LDX  I2          * X :=
       STX  SVAL
       LDA  I3          * A := m
       ADD  I4          * A := m+n
       STA  TVAL
       LD1  I1          * base unchanged
       LD2  SVAL        * I2 := s =
       LD3  TVAL        * I3 := t = m+n
       JMP  ROT2        * fall into ROT2 (subroutine below)
RET1   NOP              * return point from ROT2

* Step 2: rotate  past  in the prefix:  [ | ] -> [ | ]
* Call ROT2(BASE, s=m, t=n)
       LD1  I1
       LD2  I3          * I2 := s = m
       LD3  I4          * I3 := t = n
       JMP  ROT2
RET2   NOP              * return from ROT2
```

```
       JMP  EXIT

* ------------------------------------------------------------
* ROT2: two-block in-place rotation [X | Y] -> [Y | X]
* Inputs: I1=BASE, I2=s=|X|, I3=t=|Y|
* Clobbers: A,X,I1..I6, TMP
* Returns to RET1 (first call) or RET2 (second call)
* ------------------------------------------------------------
ROT2   LDA  I2          * A := s
       ADD  I3          * A := s+t = L
       STA  L

* Compute d = gcd(s, L) (Euclid's algorithm).
* We'll store s in S, L in L; work in A/X for modulus.
       STA  L
       LDA  I2          * A := s
       STA  S
G1     LDA  S
       ENTX L           * X := L
       DIV  L           * A := floor(S/L), X := S mod L
       LDA  L
       STA  S           * S := L
       LDX  X           * X := old (S mod L)
       STX  L
       JXZ  G2          * if remainder 0 -> gcd found in S
       JMP  G1
G2     LDA  S
       STA  D           * D := gcd(s, s+t)

* For i = 0..D-1: rotate one cycle starting at offset i
       ENTA 0
       STA  I           * I := 0
CYC0   LDA  I
       CMPA D
       JGE  DONE        * if I >= D, all cycles done

* tmp := M[BASE + I]
       LD1  I1
       LDX  I
       ADD  I1
       STX  P           * P := BASE + i
       LDA  P,0
```

```
       STA  TMP

* j := i
       LDA  I
       STA  J

* cycle: j <- j + s (mod L); move element at 'next' into position j
CYC1   LDA  J
       ADD  S
       STA  NXT         * NXT := j + s
       LDA  NXT
       CMPA L
       JL   OK          * if NXT < L, keep
       SUB  L           * else subtract L
       STA  NXT
OK     LDA  NXT
       CMPA I
       JE   CLOSE       * if next == i, cycle ends

* M[BASE + j] := M[BASE + next]
       LD1  I1
       LDX  J
       ADD  I1
       STX  P
       LDA  I1
       ADD  NXT
       STX  Q           * Q := BASE + next
       LDA  Q,0
       STA  P,0

* j := next; continue
       LDA  NXT
       STA  J
       JMP  CYC1

* close the cycle: M[BASE + j] := tmp
CLOSE  LD1  I1
       LDX  J
       ADD  I1
       STX  P
       LDA  TMP
       STA  P,0
```

```
* next start i := i+1
       LDA  I
       INC  1
       STA  I
       JMP  CYC0

DONE   NOP
* Return to proper caller: distinguish by a flag (or fall through two times).
* Here we simply fall through twice; the caller placed RET1 or RET2 in-line.
       JMP  RET1

* -------- data / temps --------
TMP    CON  0
S      CON  0
L      CON  0
D      CON  0
I      CON  0
J      CON  0
NXT    CON  0
P      CON  0
Q      CON  0
SVAL   CON  0
TVAL   CON  0
LSUM   CON  0

EXIT   END  ROT3
```

(Any standard MIX calling convention variant is fine. The essential part is `ROT2` using the cycle/juggling method; `ROT3` is just two calls to it.)

3) Running time

Let $L = \ell + m + n$.

- One call to `ROT2(s,t)` with the cycle method:
  - There are $d = \gcd(s, s+t)$ cycles; the k-th cycle of length $(s+t)/d$ does exactly that many moves plus a final write of the held temporary.
  - Total moves (loads/stores of array words) are $(s+t)+d$.
  - No extra memory except one temporary word.
- Our `ROT3` does two such rotations:

– First with $(s,t) = (\ell,\, m+n)$: cost $L + \gcd(\ell, L)$.

– Then with $(s,t) = (m,\, n)$: cost $(m+n) + \gcd(m, m+n)$.

– Total:

$$T_{\mathsf{ROT3}} \;=\; 2L \;+\; \gcd(\ell, L) \;+\; \gcd(m, m+n).$$

In practice this is very close to $2L$ word moves.

("Word moves" maps well to MIX's load/store traffic; if you wish, you can multiply by the cost model of §1.3 to get an instruction-level bound.)

4) Comparison with the "reverse-everything" method

The suggested simpler method is:

1. reverse the whole $\alpha\beta\gamma$ (length L),
2. then reverse each of the three blocks individually to get $\gamma\beta\alpha$.

A reversal of a block of length s does $\lfloor s/2 \rfloor$ swaps, i.e., about $1.5s$ array word moves (each swap = 3 writes with a temporary). Therefore the total array writes are

$$T_{\mathsf{rev}} \;=\; 3\left(\left\lfloor \frac{L}{2} \right\rfloor + \left\lfloor \frac{\ell}{2} \right\rfloor + \left\lfloor \frac{m}{2} \right\rfloor + \left\lfloor \frac{n}{2} \right\rfloor\right) \;\approx\; 3L \text{ (when lengths are even).}$$

So, asymptotically both are linear and in-place, but the cycle-based two-rotation method is typically $\sim$ fewer word moves ($2L$ vs. $3L$) and often measurably faster on MIX.

5) Correctness sketch

- Each `ROT2` is a permutation that maps index k to $k + s \pmod{s+t}$, i.e., exactly the desired $XY \mapsto YX$ rearrangement; the cycle procedure performs that permutation by rotating each cycle once.
- Composing the two adjacent-block permutations

$$(\alpha \mid \beta\gamma) \mapsto (\beta\gamma \mid \alpha) \quad \text{and} \quad (\beta \mid \gamma) \mapsto (\gamma \mid \beta)$$

yields $\gamma\beta\alpha$ as required. This is the "convenient cycle structure" behind Ex. 35's hint.

37 [M26]

Show that for $n \geq 2$ every even permutation $\pi \in S_n$ can be written as a product of two n-cycles. (Knuth's brief answer hints at reducing to a factorization by two $(n-1)$-cycles and then "splicing" n into each cycle.)

Key facts

- The sign of an ℓ-cycle is $(-1)^{\ell-1}$. Hence the product of two cycles of the same length is always an even permutation.
- Conjugation preserves cycle structure "up to relabeling" and preserves parity.

Step 1 — Reduce to a permutation on $\{1, \dots, n-1\}$

If $n \leq 2$ the statement is immediate. Assume $n \geq 3$.

Choose $a, b \in \{1, \dots, n-1\}$ with $\pi(a) = b$. (Such a pair exists since not all $1, \dots, n-1$ can map to n.)

Consider the conjugate

$$\tau \stackrel{\text{def}}{=} (n\ a)\,\pi\,(b\ n).$$

Because of the chosen $a \to b$, τ fixes n; hence τ acts as a permutation of $\{1, \dots, n-1\}$. Moreover, $\operatorname{sgn}(\tau) = \operatorname{sgn}(\pi)$, since transpositions on the left and right cancel in parity. So if π is even, then τ is an even permutation of $n-1$ symbols. Knuth's answer points exactly to this setup.

Step 2 — Lemma: every even permutation of $m \geq 2$ symbols is a product of two m-cycles

Let $m \geq 2$ and $\sigma \in S_m$ be even. Write σ in disjoint cycles

$$\sigma = (c_{11}\, c_{12}\ \dots\ c_{1k_1})\,(c_{21}\, c_{22}\ \dots\ c_{2k_2}) \cdots (c_{r1}\ \dots\ c_{rk_r}), \quad k_1 + \dots + k_r = m.$$

Construct an m-cycle U by weaving the cycles columnwise:

$$U = (c_{11}\, c_{21}\ \dots\ c_{r1}\, c_{12}\, c_{22}\ \dots\ c_{r2}\ \dots),$$

skipping entries when a row runs out. Define $V := U^{-1}\sigma$. One checks directly (follow any element through σ then U^{-1}) that V is also a single m-cycle. Therefore $\sigma = UV$ with both U, V m-cycles.

(Parity necessity: The product of two m-cycles is even; thus only even σ can be represented this way. The weaving construction shows sufficiency.)

Step 3 — Apply the lemma with $m = n-1$

Apply the lemma to $\tau \in S_{n-1}$: There exist $(n-1)$-cycles $U = (\alpha\, a)$ and $V = (b\, \beta)$ on $\{1, \dots, n-1\}$ such that

$$\tau = UV.$$

This is precisely the structure suggested in Knuth's short answer.

Step 4 — "Splice" n to get two n-cycles

Now undo the conjugation:

$$\pi = (n\, a)\, \tau\, (b\, n) \;=\; (n\, a)\, (\alpha\, a)\, (b\, \beta)\, (b\, n).$$

Observe the two "splices"

$$(n\, a)\, (\alpha\, a) = (n\, \alpha\, a) \quad \text{and} \quad (b\, \beta)\, (b\, n) = (b\, \beta\, n),$$

which are both n-cycles (we have inserted n into each $(n-1)$-cycle at the occurrence of a and b). Hence

$$\boxed{\pi = (n\, \alpha\, a)\, (b\, \beta\, n)},$$

a product of two n-cycles, as required. This matches the equivalence written in Knuth's official answer: $(na)\pi(bn) = (\alpha a)(b\beta)$ iff $\pi = (n\alpha a)(b\beta n)$.

Worked example (small n)

Take $n = 5$ and $\pi = (1\,2)(3\,4)$ (even). Choose $a = 1$, $b = 2$. Then

$$\tau = (5\,1)\, \pi\, (2\,5)$$

fixes 5 and acts on $\{1, 2, 3, 4\}$ as $(1\,2)(3\,4)$ (still even). Apply the lemma with $m = 4$: one valid choice is

$$U = (1\,4\,2\,3), \qquad V = U^{-1}\tau = (1\,4\,2\,3),$$

so $\tau = UV$. Splice 5 in:

$$(n\, \alpha\, a) = (5\,4\,2\,3\,1), \qquad (b\, \beta\, n) = (2\,4\,2\,3\,5) = (2\,3\,5\,1\,4) \text{ (after relabeling within } \{1, 2, 3, 4, 5\}).$$

One checks that their product is π. (There are many valid U, V; any pair of 4-cycles with $UV = \tau$ produces two 5-cycles whose product is π.)

Why the statement must be about even permutations

Each n-cycle has sign $(-1)^{n-1}$; hence the product of two n-cycles is always even. So odd permutations cannot be expressed this way. Our construction covers all even permutations and only those—exactly the claim of the exercise. Knuth's answer encapsulates this reduction and splice in one line.

1.4.1 Subroutines

1 [10]

Calling sequence (two entrances)

- Special case $n = 100$: `JMP MAX100` (sets $rI3 \leftarrow 100$ internally via `ENT3 100` in (5)).
- General case n arbitrary: `ENT3 n` ; `JMP MAXN` (caller supplies n in $rI3$, then enters at `MAXN`).

Entry conditions

- For `MAX100` entrance: No register precondition for n; the subroutine itself performs `ENT3 100`.
- For `MAXN` entrance: $rI3 = n$ with $n \geq 1$. (These mirror the role of n in the general MAXIMUM spec given in (4), but with $rI3$ used as the parameter in this version.)

Exit conditions (functional result and register effects)

Let X be the base address used by the code (as in the MAX programs), and let the table be $X + 1, \ldots, X + n$. Then on return:

- Result: $rA = \max_{1 \leq k \leq n} \text{CONTENTS}(X + k)$ and $\text{CONTENTS}(X + rI2) = rA$ (so $rI2$ gives a position at which the maximum occurs). (Same functional contract as in (4).)
- Loop index cleanup: $rI3 = 0$ on exit (as in (4)).
- Other status: rJ (the jump register) and the comparison indicator (CI) are affected by the subroutine call/return sequence (as noted in (4)).
- No external stores implied: The specification does not imply changes to memory outside the subroutine's own locations; in (4) Knuth notes we may conclude nothing has been stored externally when the contract doesn't say so.

Linkage notes (how (5) returns)

- (5) saves the return address with `STJ EXIT` at entry (for `MAXN`) and returns with `EXIT JMP *`, which jumps back to the instruction following the caller's `JMP`. For the `MAX100` entrance, `ENT3 100` is done first, but the same `EXIT` return is used; `ENT3` doesn't disturb rJ.

Compact spec (in the exact style of (4))

- Calling sequence: `JMP MAX100` or `ENT3 n ; JMP MAXN`.
- Entry conditions: For `MAX100`: none (internally sets $rI3 \leftarrow 100$). For `MAXN`: $rI3 = n$, assume $n \geq 1$.
- Exit conditions: $rA = \max_{1 \leq k \leq n} \text{CONTENTS}(X + k) = \text{CONTENTS}(X + rI2)$; $rI3 = 0$; rJ and CI are also affected; no other state is changed.

2 [10]

Replace this use of `JSJ` (jump without altering `rJ`) with ordinary instructions.

Solution

Use the same two-instruction prologue that label `MAXN` uses—save the return address and jump into the body—then set the parameter and continue:

```
MAX50  STJ  EXIT    ; save caller's return address (rJ) where MAXN would save it
       ENT3 50      ; set rI3 = 50 (parameter n = 50)
       JMP  2F      ; continue at the same point as other entries
```

Why this works

- At entry `MAXN`, the subroutine begins by `STJ EXIT` then `JMP 2F` to the body. By duplicating those two actions at `MAX50`, we avoid needing a `JSJ`.
- `ENT3` *does not* change the J-register (Knuth notes this explicitly), so you may place `ENT3 50` before or after `STJ EXIT`; either way the saved return address is the caller's, as desired.
- After `JMP 2F`, execution is identical to the other entrances; the subroutine eventually returns via the saved address at `EXIT`.

This exactly satisfies the exercise's request to substitute for (6) without using `JSJ`.

3 [M15]

Knuth's spec (4) for `MAXIMUM` is (summarizing):

Calling sequence: `JMP MAXIMUM`. Entry: `rI1 = n` (assume `n  1`). Exit: `rA = max_{1 k n} X[k] = X[rI2]`, and `rI3 = 0`. It notes that "`rJ` and CI are also affected" but leaves them unspecified.

The code pattern is the usual "find max" loop, with a return via `EXIT JMP *`. (See the `MAX100` variant (1) and the generalized forms in the same section.)

Precisely: effect on `rJ`

- On entry (caller does `JMP MAXIMUM`), `rJ` holds the caller's return address (the location after the call), per MIX jump semantics.
- On exit the subroutine executes `JMP *` at label `EXIT`. In MIX, `JMP *` returns to the address currently in `rJ` and stores the address of the instruction following `EXIT` (i.e., `EXIT+1`) into `rJ` as part of the jump.
- Net effect seen by the caller: control returns to "after the call", but `rJ` is clobbered to an internal address (`EXIT+1` from inside the callee). Thus `rJ` is not preserved by the subroutine. (This is why (4) warned that `rJ` is affected.)

Precisely: effect on the comparison indicator (CI)

Look at the inner loop for `MAXIMUM`/`MAX100` (labels M3–M5): each iteration does a `CMPA X,3` and possibly updates the running maximum, then decrements the loop index and exits when it is no longer positive. Therefore, the last instruction that sets CI before return is the final `CMPA` in the loop.

- At exit, `rA` holds the maximum `m`, and the final CI equals the result of comparing `m` with the last element that was compared (namely `X[1]` in the typical 1-downward loop).
- Consequently, CI is data-dependent: it is "=" (equal) iff `X[1]` equals the maximum, otherwise ">" (greater) if the maximum is strictly larger than that last compared value. In general, the subroutine does not define/sanitize CI; it simply leaves whatever CI arose from the last `CMPA`.

(That is exactly why (4) omitted CI by default: it isn't a stable, caller-useful postcondition unless explicitly specified.)

If the precondition `n  1` is violated (`rI1` not positive)

The code initializes with `j ← n; m ← X[n]; k ← n-1;` and then loops while `k > 0`. If `n  0`, the loop body is skipped immediately, and the routine returns without doing a proper "maximum of 1..n":

- It will have loaded `rA` from `X[n]` and set `rI2 ← n`, then exit with `rI3 = n-1  0` (so the advertised `rI3 = 0` postcondition in (4) fails).
- If `n = 0`, you actually get the value at `X[0]`; if `n < 0`, you fetch from `X + n` (a location before `X[1]`). This is precisely why the spec's precondition "assume `n  1`" is essential; otherwise the behavior is out of spec and effectively undefined for the intended task.

Completed specification addendum to (4)

Add the following to the "Exit conditions" of (4):

- `rJ` after return: contains the address `EXIT+1` (inside the subroutine), not the caller's return address; therefore `rJ` is modified.

- Comparison indicator (CI): equals the result of the final `CMPA` executed in the loop (i.e., comparison of the final maximum with the last element tested). Its value is not fixed by the specification and must not be relied upon by callers.
- If `rI1  0` at entry: the routine returns immediately after loading `rA ← X[n]`, with `rI2 ← n` and `rI3 = n-1  0`; the semantics "max of the first `n` elements" does not hold in this case.

4 [21]

Generalize the `MAX` subroutine so it can scan an array with an arbitrary stride `r>0` instead of consecutive elements. Provide two entry points:

- `SMAX` — general case (caller supplies `r`),
- `SMAX1` — fast special case for `r=1`.

Assume the array base is the memory label `X`, the caller supplies the length `n>0` in index register `I3`, and (for `SMAX`) the stride `r>0` in index register `I1`. On return put the maximum into `A` and the position where it occurs into `I2`. (Context: the fixed-length `MAX100` subroutine and Knuth's subroutine conventions appear in §1.4.1.)

Knuth's official answer states the exit conditions precisely, including the final value of `I3`; we'll derive those below.

The original `MAX100` scans `X+100, X+99, …, X+1` by keeping the current offset in `I3` and doing `DEC3 1` each pass. To scan every `r`-th element, do exactly the same but replace the decrement by `DEC3 0,1`, which subtracts `r` (the contents of `I1`) from `I3`. The loop terminates when `I3  0`. (This is the classic one-entry, one-exit MIX subroutine scheme with `STJ EXIT/EXIT JMP *` linkage.)

```
* MAXIMUM OVER X[n], X[n-r], X[n-2r], ...
* Calling:
*   SMAX : rI3 = n (>0), rI1 = r (>0)
*   SMAX1: rI3 = n (>0), and r = 1 by convention
* Returns:
*   rA  = max of inspected elements
*   rI2 = offset k (so that element is X + k)
*   rI3 = ((n-1) mod r) + 1 - r   (derived below)

SMAX    STJ  EXIT          * Save return address
        JMP  INIT          * General case (stride in rI1)

SMAX1   STJ  EXIT          * Fast path when r = 1
        JMP  INIT1
```

```
* ---- initialization: A ← X + I3, I2 ← I3, then step once ----

INIT    ENT2 0,3           * rI2 ← I3
        LDA  X,3           * rA  ← X + I3
        DEC3 0,1           * I3 ← I3 - r   (r = rI1)
        J3P  LOOP          * if I3 > 0, examine more
        EXIT JMP *         * single element case

INIT1   ENT2 0,3
        LDA  X,3
        DEC3 1             * I3 ← I3 - 1   (r = 1)
        J3P  LOOP1
        EXIT JMP *

* ---- main loop: compare, possibly update (A,I2), then step ----

LOOP    CMPA X,3           * compare A with X + I3
        JGE  STEP
        ENT2 0,3           * remember where the new max is
        LDA  X,3           * take the new max into A
STEP    DEC3 0,1           * I3 ← I3 - r
        J3P  LOOP
        EXIT JMP *

LOOP1   CMPA X,3
        JGE  STEP1
        ENT2 0,3
        LDA  X,3
STEP1   DEC3 1
        J3P  LOOP1
        EXIT JMP *
```

This mirrors the pattern of `MAX100` (initialize by loading the first candidate, then loop), but changes the decrement to `r` when entered via `SMAX`. (Compare with the `MAX100` listing shown in the text.)

Preconditions and postconditions

- Entry `I3 = n > 0`. For `SMAX`, `I1 = r > 0`; for `SMAX1`, `r` is understood to be 1.
- Exit

- A = max { CONTENTS(X + n - k r) : k = 0,1,2,…, (n-1)/r }. Equivalently, the maximum over X+n, X+n-r, X+n-2r, … while the offset stays positive.
- I2 is the offset at which that maximum was found (so the element is CONTENTS(X + I2)).
- I3 = n - r n/r = ((n-1) mod r) + 1 - r = -((-n) mod r). (Knuth states this compactly in the official answer; the algebra appears below.)

- Registers preserved As with MAX100, the code doesn't touch X or I1 (except that SMAX1 never needs to set I1). The return address is handled by STJ EXIT/EXIT JMP * per §1.4.1 conventions.

Correctness (loop invariant)

Let k denote the current value of I3 *at the top of* LOOP/LOOP1.

Invariant: After each comparison step, A = max { CONTENTS(X + t) : t { n, n-r, n-2r, …, k+r } } and I2 is a value t realizing that maximum.

- Initialization (INIT/INIT1): We set A ← CONTENTS(X + n) and I2 ← n; the set {n} makes the invariant true.
- Preservation: If A CONTENTS(X + k), we keep (A,I2); otherwise we set A ← CONTENTS(X + k), I2 ← k. Either way, after stepping k ← k - r we've incorporated the element at offset k into the set.
- Termination: The loop stops when k 0. Then all offsets n, n-r, …, >0 have been examined, so A is the desired maximum and I2 its position.

Final value of I3

Starting with I3 = n and repeatedly doing I3 ← I3 - r until I3 0 gives, after q = n/r decrements,

```
I3_final = n - q r = n - r n/r .
```

Using n/r = (n-1)/r + 1,

```
I3_final = n - r ( (n-1)/r + 1 )
         = (n - r (n-1)/r ) - r
         = ((n-1) mod r + 1) - r,
```

which matches Knuth's succinct form -((-n) mod r).

Cost

Let m = n/r be the number of elements inspected. Each pass is a constant number of MIX instructions (one compare, a conditional 2-instruction update, one decrement, one test), so the

running time is $\Theta(m) = \Theta(\ n/r\ )$, reducing work by a factor of about `r` relative to a unit-stride scan.

Sanity check (small example)

Take `n=7`, `r=3`. The offsets visited are `7, 4, 1`; the next would be `-2`, so we stop. The final `I3` equals `-2`, and the formula gives

```
((7-1) mod 3) + 1 - 3 = (6 mod 3) + 1 - 3 = 0 + 1 - 3 = -2.
```

Consistent.

5 [21]

1) Calling code

Load the return address (the word immediately after the call) into a non-J register—say `rA`—then jump to the subroutine:

```
      ENTA *+2        ; rA ← address of the next instruction (the return point)
      JMP  MAX100     ; enter the subroutine
; ... execution resumes here after the subroutine returns
```

`*+2` is the location two words ahead: the word after `JMP MAX100` is the resume point, and MIX's address computation makes `ENTA *+2` exactly that. (We use two words because the `JMP` itself occupies one word.) The subroutine does not rely on A's initial value, since it initializes A before the first compare (see below), so there are no entry conditions to worry about.

2) Subroutine prologue/epilogue

Replace the `STJ` prologue with a store from the register you chose (here `rA`) into the address field of `EXIT`:

```
* MAXIMUM OF X[1..100]  (abbreviated body)
MAX100  STA  EXIT(0:2)   ; linkage via rA: store sign+2-byte address into EXIT
        ENT3 100         ; M1. Initialize
        JMP  2F

1H      CMPA X,3         ; M3. Compare
        JGE  *+3
2H      ENT2 0,3         ; M4. Change m
        LDA  X,3         ;     New maximum found
```

```
        DEC3 1          ; M5. Decrease k
        J3P  1B         ; M2. All tested?
EXIT    JMP  *          ; return using the address we stored
```

- `EXIT(0:2)` targets the address part (sign + two bytes) of the `JMP *` at `EXIT`, leaving its index/field/opcode bytes undisturbed—exactly what `STJ` would have done with `rJ`, but now with `rA`.
- The rest of `MAX100` is unchanged; crucially, it re-initializes `rA` before the first `CMPA`, so seeding `rA` with the return address in the caller does not affect correctness.

  You could equally well choose another register: `STX EXIT(0:2)`, or `ST1 EXIT(0:2)`, … provided the subroutine uses that register to write the address field at entry and doesn't need its incoming value for anything else.

Why this works

- In MIX, the instruction word at `EXIT` is `JMP *`. The address field of that instruction is where the return address lives during the call.
- The standard scheme writes `rJ` there with `STJ EXIT`; our scheme writes whatever register we like (here `rA`) using a store with field spec `(0:2)`, so that only the address field is changed.
- Control returns by executing `EXIT  JMP *`, which now jumps to the address we just stored.

Knuth's answer for this exercise confirms the key point—any other register can be used, and one simple way to arrange the call is:

```
ENTA *+2
JMP  MAX100
```

with no entry conditions. Our small prologue tweak (`STA EXIT(0:2)`) is the corresponding change inside the subroutine that makes `rA` the active linkage register instead of `rJ`.

5 [21]

Section 1.4.1 presents a simple MIX subroutine `MAX100` that returns to the caller via the J-register using the standard prologue/epilogue:

- Prologue: `STJ EXIT` (store the caller's return address into the address field of the instruction at label `EXIT`)
- Epilogue: `EXIT  JMP *` (jump to the stored return address)

The exercise asks you to show that you don't have to use `rJ` for linkage—you can use any other register to convey the return address to `EXIT`. Knuth's official line for this exercise is simply: "Any other register can be used," with an example calling sequence `ENTA *+2 ; JMP MAX100` (no special entry conditions).

Construction (one explicit way to do it)

1) Calling code

Load the return address (the word immediately after the call) into a non-J register—say `rA`—then jump to the subroutine:

```
      ENTA *+2        ; rA ← address of the next instruction (the return point)
      JMP  MAX100     ; enter the subroutine
; ... execution resumes here after the subroutine returns
```

`*+2` is the location two words ahead: the word after `JMP MAX100` is the resume point, and MIX's address computation makes `ENTA *+2` exactly that. (We use two words because the `JMP` itself occupies one word.) The subroutine does not rely on A's initial value, since it initializes A before the first compare (see below), so there are no entry conditions to worry about.

2) Subroutine prologue/epilogue

Replace the `STJ` prologue with a store from the register you chose (here `rA`) into the address field of `EXIT`:

```
* MAXIMUM OF X[1..100]  (abbreviated body)
MAX100  STA  EXIT(0:2)   ; linkage via rA: store sign+2-byte address into EXIT
        ENT3 100         ; M1. Initialize
        JMP  2F

1H      CMPA X,3         ; M3. Compare
        JGE  *+3
2H      ENT2 0,3         ; M4. Change m
        LDA  X,3         ;     New maximum found
        DEC3 1           ; M5. Decrease k
        J3P  1B          ; M2. All tested?
EXIT    JMP  *           ; return using the address we stored
```

- `EXIT(0:2)` targets the address part (sign + two bytes) of the `JMP *` at `EXIT`, leaving its index/field/opcode bytes undisturbed—exactly what `STJ` would have done with `rJ`, but now with `rA`.
- The rest of `MAX100` is unchanged; crucially, it re-initializes `rA` before the first `CMPA`, so seeding `rA` with the return address in the caller does not affect correctness.

You could equally well choose another register: `STX EXIT(0:2)`, or `ST1 EXIT(0:2)`, … provided the subroutine uses that register to write the address field at entry and doesn't need its incoming value for anything else.

Why this works

- In MIX, the instruction word at `EXIT` is `JMP *`. The address field of that instruction is where the return address lives during the call.
- The standard scheme writes `rJ` there with `STJ EXIT`; our scheme writes whatever register we like (here `rA`) using a store with field spec `(0:2)`, so that only the address field is changed.
- Control returns by executing `EXIT  JMP *`, which now jumps to the address we just stored.

Knuth's answer for this exercise confirms the key point—any other register can be used, and one simple way to arrange the call is:

```
ENTA *+2
JMP  MAX100
```

with no entry conditions. Our small prologue tweak (`STA EXIT(0:2)`) is the corresponding change inside the subroutine that makes `rA` the active linkage register instead of `rJ`.

6 [26]

Below is a clean MIXAL subroutine that emulates the hardware operator MOVE using the calling sequence

```
JMP  MOVE
NOP  A,I(F)
```

It copies exactly F words from the source block starting at the effective address M(A,I) to the destination block starting at rI1, word-by-word, and when finished increments rI1 by F. All other registers (A, X, I2–I6, overflow toggle, comparison indicator) are restored to their entry values, so the only externally visible differences from a real MOVE are (1) register J (as with any subroutine call) and (2) time/space. This matches the definition of MIX's MOVE (C=7: "the number of words specified by F is moved… and rI1 is increased by the value of F at the end"), including the documented "one-word-at-a-time" behavior on overlap.

The subroutine also follows the Section 1.4.1 convention of passing arguments in the word *after* the jump (we read the NOP A,I(F) via rJ, which points at that word), then returning to that NOP so execution continues with the caller's next instruction.

MOVE (software) — MIXAL

```
* MOVE - software emulation of hardware MOVE
* Call:   JMP MOVE
*         NOP A,I(F)
* Effect: Copy F words from M(A,I) -> [rI1 .. rI1+F-1]; then rI1 := rI1 + F.
* Notes:  Preserves rA, rX, rI2..rI6, CI, overflow; only rI1 (and rJ) differ.

MOVE    STJ     RET           Save return (address of the NOP A,I(F))

        * Save registers we will clobber
        STA     SAVEA
        ST1     SAVE1
        ST2     SAVE2
        ST3     SAVE3
        ST4     SAVE4

        * rJ points to the parameter word (NOP A,I(F)).
        STJ     JADDR
        LD2     JADDR         rI2 <- rJ  (an address we can index from)

        * Decode A, I, F from that instruction word.
        LDA     0,2(0:2)      rA <- A (±AA)
        STA     SRC           save base address A
        LD4     0,2(3:3)      rI4 <- I (0..6)
        LD3     0,2(5:5)      rI3 <- F (count)

        * Build source pointer: rI2 := A + rIi (using index arith. only;
        * INCi does not affect the overflow toggle).
        LD2     SRC           rI2 <- A
        J4Z     SRC_OK        if I = 0, no indexing needed

        * Add the selected index register to rI2
ISELECT DEC4    1
        J4Z     ADDI1
        DEC4    1
        J4Z     ADDI2
        DEC4    1
        J4Z     ADDI3
        DEC4    1
        J4Z     ADDI4
        DEC4    1
        J4Z     ADDI5
        * must be 6
```

```
        ST6     TMP
        JMP     ADDI

ADDI1   ST1     TMP
        JMP     ADDI
ADDI2   ST2     TMP
        JMP     ADDI
ADDI3   ST3     TMP
        JMP     ADDI
ADDI4   ST4     TMP
        JMP     ADDI
ADDI5   ST5     TMP
ADDI    INC2    TMP             rI2 += rIi
SRC_OK  NOP

        * Main loop: move F words, advancing both pointers
LOOP    J3Z     DONE            if F = 0, nothing to do
        LDA     0,2             rA := CONTENTS(rI2)
        STA     0,1             CONTENTS(rI1) := rA
        INC2    1               rI2++
        INC1    1               rI1++
        DEC3    1               F--
        JMP     LOOP

DONE    * rI1 has been advanced by original F (as required)

        * Restore preserved registers (except rI1)
        LDA     SAVEA
        LD2     SAVE2
        LD3     SAVE3
        LD4     SAVE4
        LD1     SAVE1

RET     JMP     *               Return to the NOP; it executes and control
                                * falls through to the caller's next instr.

* ------- locals -------
SAVEA   CON     0
SAVE1   CON     0
SAVE2   CON     0
SAVE3   CON     0
SAVE4   CON     0
```

```
JADDR   CON     0
SRC     CON     0
TMP     CON     0
```

Why this matches the hardware MOVE

1. Source/destination and count. We take A and I from the parameter and compute M(A,I) for the source; F is the count. The loop copies one word at a time in increasing address order, exactly like the hardware (so overlapping ranges behave the same), and rI1 advances by F total.
2. Register effects. MOVE shouldn't alter rA, rX, rI2–rI6, CI, or the overflow toggle. We therefore save/restore those we touch and use INCi (not ADD/INCA) to avoid toggling overflow. Only rI1 changes (by F), and rJ is different because of the call/return—exactly as the exercise requests.
3. Calling/returning. Section 1.4.1's "arguments after the jump" convention is used: we read the NOP A,I(F) via rJ; returning with `JMP *` lands on that NOP, which executes (doing nothing) and then the caller proceeds to the next instruction.

If you'd like, I can also show a branch-table version that packs the six `ADDIk` cases tighter, or a self-modifying variant that patches the I-field of a template `LDA` to avoid the small `ISELECT` chain.

7 [20]

Here's a concise, exam-style answer with enough depth for TAOCP.

Knuth's MIX used self-modifying code for subroutine linkage (e.g., storing the return address into the instruction stream), but he already notes that "newer computers…keep [such] information in registers or in a special array, not within the program itself" (see the note in §1.4.1 that points you to this exercise). The reasons modern systems discourage self-modification include:

1. Security policies (W X). Operating systems increasingly enforce *write-xor-execute*: a page may be writable or executable, not both. That prevents code injection and many exploit techniques; it also means ordinary programs can't (and shouldn't) patch their own instruction bytes.
2. OS/ABI design: text is shared and read-only. Executable code ("text") pages are mapped read-only and commonly shared among processes for memory efficiency. Letting programs write into those pages would break sharing and isolation.

3. Performance costs on modern CPUs. Pipelines and split instruction/data caches mean that when code *is* changed, the CPU must perform special synchronization/serialization and cache/TLB maintenance to make the modification visible to instruction fetch. That's slow and error-prone if mishandled. Vendors explicitly warn about penalties and coherence requirements for self-modifying code.

4. Concurrency and reentrancy hazards. Multiple threads may execute the same code simultaneously. Patching instructions in place introduces race conditions, tears, and undefined behavior unless you add heavyweight synchronization—further eroding any perceived benefit. (Read-only text plus normal call/return linkage avoids this.)

5. Optimization, tooling, and maintainability. Compilers, linkers, profilers, verifiers, and JITs assume code is immutable at run time; immutability enables aggressive optimization, code signing, control-flow integrity, and reliable debugging. Self-modification undermines those assumptions and complicates reasoning about correctness.

6. Better idioms exist. Modern linkage stores return addresses in a register/stack frame (or a link register), not in the instruction stream; parameter passing and multiple exits are handled by calling conventions and structured control flow, so the MIX-style "patch the code to return here" is unnecessary. (Knuth foreshadows exactly this shift.)

Bottom line. What was once a practical trick on simple machines is now a liability: it conflicts with OS security and memory models, harms performance on cached/pipelined CPUs, breaks reentrancy, and fights today's toolchains. Hence modern advice—and practice—is to avoid self-modifying code. The exercise list where this appears is on p. 193.

1.4.2 Coroutines

1 [10]

A *coroutine* is symmetric: two (or more) routines repeatedly transfer control to each other, each time resuming exactly where it last left off (state must persist across transfers). This is unlike a subroutine, where control flows down and then returns once.

Why "short and simple" is elusive

1. One transfer not really a coroutine. If A calls B only once (and B never resumes A mid-run), the interaction is observationally identical to an ordinary subroutine call. So the "smallest" control exchange doesn't demonstrate anything essentially coroutine-specific. To *show* coroutine behavior, you need multiple alternations (A B A B…), which already lengthens the example.
2. State must be preserved between activations. Real coroutine examples must keep local state alive across yields. On a bare machine (like MIX in the text) that means explicit linkage and register saves/restores—extra instructions that swell even toy programs. Knuth's own coroutine linkage shows saving/restoring registers and jump handshakes, which are inherently not "tiny."
3. Nontrivial data flow is needed to be meaningful. To motivate *why* coroutines help, you typically show a producer/consumer, scanner/parser, or "turn an n-pass pipeline into a one-pass process" example. But as soon as you pass realistic data (buffers, termination conditions, etc.), the code grows beyond a few lines. (Knuth's figure contrasting a four-pass pipeline with a one-pass chain of coroutines illustrates the typical use case.)
4. Toy examples are either contrived or misleading. Extremely short "ping-pong" snippets (e.g., print A/B alternately) either hide the crucial save/restore machinery or rely on features (like a built-in `yield`) that the reader hasn't met yet, making them pedagogically suspect on a machine-language canvas.

Minimal illustration (conceptual)

- Subroutine pattern: `main → sub() → return` (one descent, one return).
- Coroutine pattern: `A → B → A → B → …` (multiple *resumptions*), plus explicit context preservation. Without at least two transfers, A B collapses to subroutine semantics and doesn't illuminate coroutines.

To genuinely demonstrate coroutines you need (i) more than one control transfer, (ii) preserved execution context, and (iii) meaningful data exchange. Each of these adds lines and machinery, so examples that are both *short* and *truly illustrative* are rare; once they're short enough, they stop being distinctly coroutine examples and look like plain subroutines instead.

2 [20]

What actually happens if you start with `IN`

1. Initialization still sets `rI6 ← 0` and `rX ← 0` (lines 58–59), so `IN1` will read the first input word if needed and isolate the next character into `rA`.
2. At the end of its step, `IN` yields to `OUT` via the coroutine linkage (`JMP OUT`). Control lands at the `OUT` linkage stub, which first saves IN's resume point and then transfers to `OUT1`.
3. `OUT1` begins by preparing a fresh output card/buffer: it sets up indices and executes a `MOVE` that blanks the output area ("Start new output card… Set output area to blanks"), then it asks `IN` for the "next translated character" by jumping back via the linkage (`JMP IN`). Those prologue instructions overwrite or otherwise invalidate the character that `IN` had already placed in `rA`. Net effect: the very first character fetched by `IN` is lost before `OUT` stores anything.

This is exactly why the text starts with `JMP OUT1`: the OUT coroutine's prologue should run before the first request to `IN`, so that the buffer is ready and the first character fetched by `IN` is immediately consumed instead of being clobbered by OUT's setup. The official answer summarizes this as "the first character found by IN would be lost," and notes that lines 58–59 were chosen to initialize `IN`; starting with `IN` would require compensating initialization for `OUT`.

How to make "start with IN" work (one correct fix)

If you truly want to begin at `IN1`, adjust the first handoff into `OUT` so OUT's prologue does not run after `IN` has already put a character in `rA`:

- Pre-initialize OUT's state that the prologue would otherwise need. Concretely, set `rA ← -16` so the initial `MOVE` count is already in place, and clear the output buffer if it isn't known to be blank.
- Change the first transfer from `IN` to `OUT` to jump directly to the code that stores `rA` into the buffer (the code around line 39) rather than to `OUT1` (the prologue). In the book's notation: modify the first `OUTX` jump target so it goes to the "store character" logic rather than the "start new card" logic. With that, the very first character produced by `IN` will be preserved and packed before any further OUT setup runs.

Bottom line

- If you simply replace `JMP OUT1` with `JMP IN1`, the first input character disappears.
- To start with `IN` safely, you must (i) pre-initialize OUT's prologue effects and (ii) route the first transfer from `IN` straight into OUT's "store character" code rather than OUT's prologue.

3 [20]

False (but "almost" true). Removing all three comparisons breaks correctness in two distinct ways:

1. The comparison indicator (CI) can be uninitialized at the first `JE 9F`

In the OUT coroutine the test for end-of-input is done by

```
… STA …(1:1)
CMPA PERIOD
JE  9F
…
… STA …(2:2)
CMPA PERIOD
JE  9F
…
… STA …(3:3)
CMPA PERIOD
JE  9F
…
9F  JNE OUT1
HLT
```

(see lines 39–55). Each `JE 9F` *assumes* the CI was set by the immediately preceding `CMPA PERIOD`. If you delete those compares, the very first `JE` in OUT will consult whatever CI happens to be left over from earlier code (i.e., undefined for some paths), so behavior is not well-defined. In particular, if the first input character is a letter, coroutine IN takes the "nondigit" branch *without doing any comparison at all* and jumps to OUT; then OUT reaches `JE 9F` with an undefined CI. Hence correctness already fails.

2. A final period after a replication digit isn't detected

Even when the CI happens to be set, dropping those compares misses the special case "digit followed by final '.'". Coroutine IN reads a replication digit `n`, then fetches the next character (which might be the terminating '.') and immediately jumps to OUT *without* doing any compare in between (see lines 24–30). If '.' is preceded by a replication digit, the most recent comparison

in IN was the digit test (not a period test), so the CI is *not* "equal"; without `CMPA PERIOD` in OUT, the program won't notice the final period and won't halt.

Knuth's official notes summarize both issues succinctly: removing all three compares is "almost" okay, but "the comparison indicator isn't initialized; and if the final period is preceded by a replication digit, it won't be noticed."

If you really want to remove the three compares

You can relocate the period test so the CI is *reliably* set before OUT checks it:

- After IN fetches the next character (between lines 26–27), insert a single `CMPA PERIOD` so the CI reflects whether the just-fetched character is '.' before control jumps to OUT.
- Also ensure the CI has a known initial state right before the very first handoff (between lines 59–60), e.g., by comparing `rX` with PERIOD (one way is to insert a `CMPX PERIOD` there), so OUT's first `JE/JNE` pair isn't consulting a garbage CI. (Then OUT can safely rely on the CI without doing its own per-position `CMPA PERIOD`.)

This preserves the coroutine contract while shaving a few instructions from OUT, but the stock program (with the three local `CMPA PERIOD`s) is the clearest and safest.

Don't delete all three `CMPA PERIOD` in OUT; doing so makes the program incorrect on real inputs.

4 [20]

What "linkage (1)" is doing in MIX

Knuth's linkage (labels rearranged slightly for clarity) is the tiny handshake at the end of his program:

```
60  JMP OUT1        ; start with OUT  (exercise 2)
61  OUT  STJ INX    ; store our resume point "OUTX" into IN's jump word
62  OUTX JMP OUT1   ; jump to OUT's entry (will be patched by IN)
63  IN   STJ OUTX   ; ditto, in the other direction
64  INX  JMP IN1    ; jump to IN's entry (will be patched by OUT)
```

Each coroutine owns one *indirect jump word* (`INX`, `OUTX`). When a coroutine yields, it stores the address of its own resume label (e.g., `OUTX`) into the *other* coroutine's jump word, then jumps to the other's entry (`IN1  OUT1`). The first time, `INX` and `OUTX` contain the entry addresses; thereafter they contain the saved resume addresses. (Knuth later shows a variant "(4)" that also saves/restores register A, but the basic idea above is "(1)".)

Below are two concrete, modern implementations that are directly analogous but avoid self-modifying code by using *indirect branches through writable data*.

x86-64 (NASM/Intel syntax; position-independent)

```
; Assemble with: nasm -felf64 coro.asm && ld -o coro coro.o
default rel

section .data
INX:   dq IN1        ; IN's current continuation (entry at first)
OUTX:  dq OUT1       ; OUT's current continuation (entry at first)

section .text
global _start

_start:
    jmp OUT1         ; Like line 60: begin in OUT

; --- IN coroutine ------------------------------------------------

; Entry for IN: always tail-jump to whatever OUT stored into INX
IN1:
    jmp qword [INX]

; Somewhere inside IN, to yield to OUT do:
;   1) store our resume address into OUTX
;   2) jump to OUT's entry
IN_yield_to_OUT:
    lea  rax, [rel IN_resume]  ; "return address" for IN
    mov  [OUTX], rax
    jmp  OUT1
IN_resume:
    ; ... execution continues here when OUT yields back ...

; --- OUT coroutine ------------------------------------------------

; Entry for OUT: always tail-jump to whatever IN stored into OUTX
OUT1:
    jmp qword [OUTX]

; To yield back to IN:
OUT_yield_to_IN:
    lea  rax, [rel OUT_resume] ; "return address" for OUT
```

```
    mov  [INX], rax
    jmp  IN1
OUT_resume:
    ; ... execution continues here when IN yields back ...
```

Why this is exactly like (1):

- `IN1: jmp [INX]` and `OUT1: jmp [OUTX]` play the role of the "indirect jump words" (`INX JMP IN1`, `OUTX JMP OUT1`).
- `IN_yield_to_OUT` stores `IN_resume` into `OUTX` before jumping to `OUT1`; `OUT_yield_to_IN` stores `OUT_resume` into `INX` before jumping to `IN1`.
- No stacks or OS help are required; you're literally passing continuations in memory, just as MIX does by patching a jump word.

If you need register/save semantics like Knuth's variant (4), push/pop (or move to callee-saved registers/stack slots) around the `mov`/`jmp` pair, corresponding to `STA HOLDA` … `LDA HOLDA` in his text.

Portable C (GCC/Clang) with "labels as values"

This uses the compiler extension for *computed goto*. It mirrors MIX even more transparently:

```
// gcc -std=gnu11 coro.c -o coro
#include <stdio.h>

static void *INX, *OUTX;

int main(void) {
  INX  = &&IN1;      // initial entries
  OUTX = &&OUT1;

  goto OUT1;         // start with OUT (like line 60)

IN1:
  goto *INX;         // always jump to IN's current continuation

  // ... IN's code ...
IN_yield_to_OUT:
  OUTX = &&IN_resume;  // store our resume label into OUT's jump slot
  goto OUT1;           // jump to OUT's entry
IN_resume:
  // ... resumes here ...
```

```
OUT1:
  goto *OUTX;        // always jump to OUT's current continuation

  // ... OUT's code ...
OUT_yield_to_IN:
  INX = &&OUT_resume;  // store our resume label into IN's jump slot
  goto IN1;            // jump to IN's entry
OUT_resume:
  // ... resumes here ...
}
```

- `INX`/`OUTX` are the indirect jump words.
- `goto *INX` / `goto *OUTX` replace the "`JMP IN1`/`JMP OUT1` words patched by STJ".
- Assigning `&&label` is the exact analogue of `STJ otherX`.

This version is often the simplest way to prototype Knuth's linkage today, and it avoids any system-dependent assembly.

5 [15]

Whenever OUT executes `JMP IN`, rA in OUT must be exactly what it was when control returns to the next OUT instruction; likewise for IN when it executes `JMP OUT`. So each coroutine must (1) save its own rA at exit, and (2) restore its own rA at re-entry, without constraining the other coroutine's use of rA while it runs.

Knuth already shows the one-sided variant ("if OUT would destroy A, save/restore A around the transfer"), right above Fig. 22; we just symmetrize that pattern. We splice this linkage where the book places the standard two-coroutine linkage (lines 57–65), replacing those four short lines with the version below.

MIX code (two-sided A preservation)

```
* --- Linkage data (two save slots, one per coroutine) ---
HOLDIN  CON  0        * last-saved rA for IN
HOLDOUT CON  0        * last-saved rA for OUT

* --- Initialization and linkage ---
START   ENT6 0        * as in text: init rI6 for NEXTCHAR
        ENTX 0        * as in text: init rX for NEXTCHAR
        JMP  OUT1     * start with OUT (exercise 2 discusses starting with IN)

OUT     STJ  INX      * standard coroutine bookkeeping (return for IN)
```

```
        STA  HOLDIN   * save caller's A (leaving IN -> coming here)
        LDA  HOLDOUT  * restore OUT's own A for re-entry
OUTX    JMP  OUT1     * enter OUT proper

IN      STJ  OUTX     * standard coroutine bookkeeping (return for OUT)
        STA  HOLDOUT  * save caller's A (leaving OUT -> coming here)
        LDA  HOLDIN   * restore IN's own A for re-entry
INX     JMP  IN1      * enter IN proper

        END  START
```

Why this works

- When OUT → IN (`JMP IN` inside OUT): control lands at label `IN`. We immediately
  1. save the caller's A (`STA HOLDOUT`), then
  2. restore IN's previously saved A (`LDA HOLDIN`), and jump to `IN1`. While IN runs, it may freely clobber rA. When IN later yields back to OUT (`JMP OUT`), label `OUT` restores OUT's A from `HOLDOUT` before resuming at `OUT1`. Thus rA in OUT is unchanged across the yield/return pair.
- Symmetrically, when IN → OUT (`JMP OUT` inside IN), label `OUT` saves IN's rA to `HOLDIN`, restores OUT's rA from `HOLDOUT`, and resumes OUT. When OUT yields back, label `IN` reloads `HOLDIN`. So rA in IN is likewise unchanged across its yield/return.

This is exactly the two-sided generalization of the one-sided example on p. 198 ("if OUT would destroy rA, save at IN and restore at OUT"), but now with two save cells so we can save the caller's A and simultaneously restore the callee's A at each transfer.

6 [22]

Here's a clean coroutine linkage for three MIX coroutines A, B, C so that any one can transfer to either of the other two, and on activation each resumes exactly where it last left off. (This is the Exercise 6 setup at the end of §1.4.2.)

First, recall the 2-coroutine pattern (called "(1)" in the text): each coroutine has a *resume cell* holding a `JMP` whose address field gets patched by the other coroutine via `STJ`. Then you enter a coroutine by *executing* that stored jump (i.e., `JMP OUTX` executes the word at `OUTX`).

We generalize that idea by giving each coroutine its own resume cell (`AX`, `BX`, `CX`) and by providing two entry labels per coroutine—one entry for each possible caller—so we know which caller's resume cell to update.

Linkage (MIX)

```
* Resume cells (initial jumps to first-time entries A1, B1, C1)
AX      JMP A1
BX      JMP B1
CX      JMP C1

* Entries to B
AB      STJ AX       * called B from A: remember A's return point in AX
        JMP BX       * now execute B's stored jump (first time → B1)

CB      STJ CX       * called B from C: remember C's return point in CX
        JMP BX

* Entries to C
AC      STJ AX       * called C from A: remember A's return point
        JMP CX

BC      STJ BX       * called C from B: remember B's return point
        JMP CX

* Entries to A
BA      STJ BX       * called A from B: remember B's return point
        JMP AX

CA      STJ CX       * called A from C: remember C's return point
        JMP AX
```

How you use it

- From inside A, jump to `AB` to activate B, or to `AC` to activate C.
- From inside B, jump to `BA` (activate A) or `BC` (activate C).
- From inside C, jump to `CA` (A) or `CB` (B).

Why it works (invariants)

- Each coroutine Q has a resume cell QX that always contains a `JMP <where Q should resume>`.
- When caller P transfers to callee Q, control arrives at entry label QP, which executes `STJ PX`, storing the caller's *return address* (the instruction after the `JMP QP`) into PX. Thus the caller's resume cell is freshly patched to "where to continue when we next execute `JMP PX`."
- The entry then `JMP`s to QX, *executing* Q's stored `JMP`—first time it jumps to `Q1`; thereafter it jumps to Q's last suspended point.

You can verify a round-trip: A → `JMP AB` → `AB: STJ AX; JMP BX` (B begins); later B → `JMP BA` → `BA: STJ BX; JMP AX` which executes the patched word at AX and resumes A precisely after its `JMP AB`.

7 [30]

What the original program did (we must invert it)

- Input format (2): a stream of characters ending with a period "."; if a digit `n {0,…,9}` appears, it means "repeat the next character `n+1` times," even if that next character is itself a digit. Nondigits simply denote themselves.
- Output format (3): the fully expanded character stream, printed in groups of three separated by blanks, 16 groups per card; processing stops at the period.

Our task is the inverse: read the grouped stream (3), ignore blanks, stop at the first period, and re-encode it into the compact code (2) with the fewest characters (e.g., never emit the redundant `0Z`, just `Z`).

Minimal re-encoding strategy (correctness & optimality)

Let's call the flat stream (with blanks removed) `S`, ending at the first `.`. We want the shortest string over the alphabet `Σ {0,…,9}` whose expansion (by "digit→repeat next char `n+1` times") equals `S`.

Process `S` run by run: for each maximal block of identical characters `c` of length `L`, output the shortest code for that block, then continue.

- If `c` is a nondigit (letter, punctuation, etc.):
  - We may either write the character itself once (cost 1) or use a "digit + c" pair (cost 2) to say multiple copies.
  - Optimal encoding is greedy by tens:
    - Let `q = L/10` and `r = L mod 10`.
    - Output `q` pairs `9c` (each pair contributes 10 copies).
    - For the remainder:
      - If `r = 0`: done.
      - If `r = 1`: just output `c` (better than `0c`).
      - If `2 r 10`: output `(r-1)c`.
- If `c` is a digit (one of `0..9` appearing as data), we cannot write it bare (because bare digits in (2) are operators). We must always use a pair:

  - Again `q = L/10` , `r = L mod 10`.
  - Output `q` times `9c`; if `r>0`, output `(r-1)c`.

Why this greedy scheme is optimal:

- Each "digit+char" token has fixed cost 2 and yields between 1 and 10 copies (specifically `n+1` copies). For nondigit characters we also have a special 1-copy option of cost 1 (just the character). This is a canonical coin system: "coins" of values `1..10` with costs `2` each, except a free single-copy option of cost 1 for nondigits. Greedy by 10s, then one optimal remainder, is optimal under such costs; any non-greedy split that uses more tokens (or replaces a single bare `c` by `0c`) cannot be shorter.

This guarantees "the zero before the Z … would not really be produced" (we would output just `Z`) since `0Z` (cost 2) is strictly worse than `Z` (cost 1).

MIX solution architecture

We mirror Knuth's i/o organization:

1. NEXTCHAR: reuse the input subroutine from the text that returns the next nonblank character from card reader into `rA`; it handles card boundaries and blanks. (Exactly as in the example: read card, scan words, extract next character.)
2. PUTCH: an output subroutine that appends one character to an 80-col output buffer; when full, punch the card and clear the buffer (similar to OUT's buffering, but without inserting blanks every three chars). The last card is punched when we see `..`
3. MAIN (reverse encoder):
   - Read first nonblank char `x` via `NEXTCHAR`.
   - If `x='.'`: output `.` and finish.
   - Set `CUR=x`, `CNT=1`.
   - Loop: get next `x` via `NEXTCHAR`.
     - If `x='.'`: flush run `(CUR, CNT)` using EmitRun (below), then output `.` and finish.
     - Else if `x==CUR`: `CNT++`.
     - Else: flush `(CUR, CNT)`, then `CUR=x`, `CNT=1`.
   - EmitRun(CUR, CNT) implements the minimal strategy above, using helper predicates "is digit?" (compare `rA` with codes 30..39) and a small loop to emit `9CUR` `q` times; remainder as per rules.

Below is a compact MIXAL skeleton that focuses on the core logic (you can drop it straight into your MIXAL environment; i/o linkage follows the style of §1.4.2):

```
* REVERSE OF (3) -> (2): minimal code
* Units and buffers
READER  EQU     16
PUNCH   EQU     17
INPUT   ORIG    *+16        * input card buffer
OUTPUT  ORIG    *+16        * output card buffer

* =============== SUBROUTINE: NEXTCHAR (nonblank reader) ===============
* rA <- next nonblank char from input; reads new card as needed
* (Same as text's NEXTCHAR)
NEXTCHAR STJ    NRET
         JXNZ   GOTX        * rX holds leftover chars from last word
INIT     J6N    READ        * rI6=0 => need a new card
READ     IN     INPUT(READER)
         JBUS   *(READER)
         ENN6   16          * rI6 points to first word
GOTX     LDX    INPUT+16,6  * rX <- next word
         INC6   1
         ENTA   0
         SLAX   1           * next char -> rA (low byte)
NRET     JANZ   *            * skip blanks
         JMP    NEXTCHAR+1

* =============== SUBROUTINE: PUTCH (buffer & punch) ===================
* input: rA has a character to append
* uses rI4 as OUTPUT pointer: -16,-15,...,-1 then 0 triggers punch
PUTCH    STJ    PRET
         STA    OUTPUT+16,4(1:1)     * store to pos (1:1) of word at rI4
         INC4   1
         J4N    PRET
         OUT    OUTPUT(PUNCH)
         JBUS   *(PUNCH)
         ENT4   -16
         MOVE   -1,1(16)             * blank buffer
PRET     JMP    *

* =============== MAIN ==================================================
START    ENT6   0            * init reader state for NEXTCHAR (as in text)
         ENTX   0
         ENT4   -16          * init output pointer
```

```
         MOVE    -1,1(16)     * blank output buffer

         JMP     NEXTCHAR
         CMPA    PERIOD
         JE      FINISH_NOW   * empty stream?
         STA     CUR
         ENTA    1
         STA     CNT

MAINLP   JMP     NEXTCHAR
         CMPA    PERIOD
         JE      FLUSH_LAST
         CMPA    CUR
         JE      INCR
         * new run
         LDA     CUR
         LDX     CNT
         JMP     EMITRUN
         STA     CUR          * EMITRUN returns with rA=CUR', rX=CNT'
         ENTA    1
         STA     CNT
         JMP     MAINLP

INCR     ENTA    CNT
         INC     1
         STA     CNT
         JMP     MAINLP

FLUSH_LAST
         LDA     CUR
         LDX     CNT
         JMP     EMITRUN
         * emit '.' and finish
FINISH_NOW
         ENTA    PERIOD(5:5)  * just the '.' character
         JMP     PUTCH
         OUT     OUTPUT(PUNCH)
         HLT

* =============== EMITRUN(CUR in rA, CNT in rX) ========================
* Emits minimal code for CNT copies of char CUR.
* On return: none (caller reloads CUR/CNT for next run).
```

```
EMITRUN  STJ     ERRET
         STA     CCHAR
         STX     CCNT

         * is CUR a digit?  (MIX digit codes 30..39)
         LDA     CCHAR
         DECA    30
         JGE     CHECKHI
NOTDIG   * --- NONDIGIT BRANCH ---
         * q = CNT/10
         ENTA    10
         STA     TEN
         LDA     CCNT
         DIV     TEN           * rA <- q, rX <- r
         STA     Q
         STX     R
         * emit q times '9 CUR'
QLOOP    LDA     Q
         JE      REM
         ENTA    39            * '9'
         JMP     PUTCH
         LDA     CCHAR
         JMP     PUTCH
         DEC     Q,1
         JMP     QLOOP
REM      LDA     R
         JE      ERRET         * r==0
         J1Z     SINGLE        * r==1 -> single CUR
         DEC     R,1           * r in 2..9 -> (r-1)
         ADD     29            * convert 1..8 to digits 1..8 (codes 31..38)
         JMP     PUTCH
         LDA     CCHAR
         JMP     PUTCH
         JMP     ERRET
SINGLE   LDA     CCHAR
         JMP     PUTCH
         JMP     ERRET

CHECKHI  CMPA    =9=           * compare (CUR-30) with 9
         JG      NOTDIG        * >9 means not a digit actually
         * --- DIGIT BRANCH ---
         LDA     CCNT
```

```
         DIV     TEN           * q <- rA, r <- rX
         STA     Q
         STX     R
QDLOOP   LDA     Q
         JE      RPART
         ENTA    39            * '9'
         JMP     PUTCH
         LDA     CCHAR
         JMP     PUTCH
         DEC     Q,1
         JMP     QDLOOP
RPART    LDA     R
         JE      ERRET
         DEC     R,1
         ADD     29            * 0..8 -> 0..8 digits (codes 30..38); r=1 -> '0'
         JMP     PUTCH
         LDA     CCHAR
         JMP     PUTCH
ERRET    JMP     *

* Data
CUR      CON     0
CNT      CON     0
CCHAR    CON     0
CCNT     CON     0
Q        CON     0
R        CON     0
TEN      CON     10
PERIOD   ALF     '    .'

         END     START
```

Notes and alignment with the text

- Input side: `NEXTCHAR` is exactly the "nonblank character" reader used in the coroutine example (read card, step through 5-char fields, skip blanks).
- Output side: Unlike OUT in the text (which groups in threes and inserts blanks), our `PUTCH` just packs characters into an 80-column buffer and punches on full or at the end; the exercise asks to produce cards like (2), i.e., no grouping.
- Termination: We stop at the first period, mirroring the original semantics.

Walk-through on Knuth's example

Given output (3)

```
ABB BEE EEE E44 446 66F GZY W22 220 0PQ 999 999 999 R.
```

Ignoring blanks, the stream ends at the first `..` Runs include `A(1), B(2), E(3), 4(2), 6(3), F(1), G(1), Z(1), Y(1), W(1), 2(2), 0(3), P(1), Q(1), 9(3), 9(3), 9(3), R(1)`.

- For nondigits (e.g., `Z(1)`), we output the char itself (`Z`), never `0Z`.
- For digits-as-data (e.g., `0(3)`), we must use pairs: `2` copies beyond the first encoded as `(3-1)=2 → '2'` so emit `20`. More generally, runs 10 break as `9c` blocks + remainder.

Composing all runs yields exactly a minimal string of the form (2), and as required, the stray `0Z` will not appear in our output.

1.4.3.1 A MIX simulator

1 [14]

What FCHECK does, and where it's used

- Purpose. `FCHECK` decodes the F–field (in `rI3`) as a *partial field specification* of the form $8L + R$, ensures $L \leq R \leq 5$, then leaves the results in convenient places: it stores L and R in memory cells `L` and `R`, and sets `rI1 ← L`. (If invalid, it jumps to `FERROR`.)
- In `GETV`. The value-extraction subroutine begins by calling `FCHECK`, then fetches memory (`MEMORY`), and finally uses `L` and `R` to select the requested field and normalize the sign. (If the sign isn't included, it forces the sign to `+`.)
- In `STORE`. The store routine also begins with `FCHECK`, then uses `L/R` to splice the source register into the destination word—handling the special case when the sign-byte (i.e., `L=0`) is included.
- In `COMPARE`. The program *intentionally* avoids calling `FCHECK` twice: it calls `GETV` (which runs `FCHECK` once), then enters `GETV` through the special entrance `GETAV` (which jumps past the `FCHECK` code) to reuse the already-decoded `L`. This is the one place where the code is already organized to keep the `FCHECK` overhead to a single call.
- The exercise note. The exercise explicitly points you to "step 3 (Reexamination)" at the end of §1.4.1, which suggests *enlarging or merging subroutines when the caller always does the same standard setup/cleanup around them.* The exercise statement itself appears at the bottom of the simulator section.

Diagnosis

Across the simulator:

- `FCHECK` is always followed immediately by *very similar* boilerplate in its callers (e.g., loading `-R` into `rI2`, checking whether the sign is included via `L==0`, then doing standard shifts).
- Only `COMPARE` already amortizes `FCHECK` over two uses of the same field (memory and register) by using the `GETAV` entrance.

That's a textbook case for Step-3 refactoring: push the common pre/post work into a single place and remove redundant calls.

A better organization (two practical refactorings)

You can do either (A) "pre-decode F once per instruction that needs (L:R)" or (B) "absorb/inline the decode into the two places that actually need it." Both obey Step-3.

A) Pre-decode once (preferred if you're willing to touch the control path)

1. Mark which opcodes use (L:R). Add a small parallel table to `OPTABLE` that classifies the F–field usage of each opcode (e.g., 0 = not a partial field, 1 = partial field (L:R), 2 = shift count, …). The control routine already fetches `F` into `rI3` (line 027). Use your new table to decide whether to decode now.
2. Decode centrally. If the current opcode's F–usage is "partial field," call a beefed-up decoder once and for all. That decoder should:
   - Validate and compute `L` and `R` (as `FCHECK` does).
   - Also set `rI2 ← -R` since every client immediately needs it.
   - Optionally set a one-byte flag `SFLAG ← (L==0)` so clients don't have to re-test.
3. Use "prepared" entries.
   - Change callers that currently do `JMP GETV` to `JMP GETAV` (the *already-prepared* entrance) because `rI1/R` are now set; `GETAV` already jumps past the `FCHECK` block.
   - Split `STORE` into two entries: keep `STORE` (old behavior) and add `STORE1` right after the `FCHECK` call site; have the opcode table jump to `STORE1` when the control routine has pre-decoded `F`. This removes the `FCHECK` call in the common path.

This removes *all duplicate decoding* in instructions that use the same (L:R) twice (e.g., `COMPARE`) and also avoids decoding in instructions whose F–field isn't a partial field (e.g., `JUMP`, `ADDROP`). It follows Knuth's Step-3 guidance exactly: enlarge the central "control" routine slightly to relieve its callees of repeated setup.

B) Inline/merge (`FCHECK` → callers) with a macro (minimal edits)

If you prefer not to touch the control path or tables:

- Delete `FCHECK` as a runtime subroutine and replace it by an *assembly macro* (say, `DECODEF`) that expands to the 12 lines of its body (compute `L`, `R`, checks, set `rI1`). Expand `DECODEF` at the start of `GETV` (before label `1F`) and at the start of `STORE`. Also have the macro optionally do `LD2N R` so all callers get `rI2` "for free."
- Keep `GETAV` unchanged so `COMPARE` still jumps past the macro expansion in `GETV` and benefits from a single decode.

This keeps code simple, keeps `COMPARE` optimal, and respects Step-3 ("perhaps several subroutines should be merged into one; or perhaps a subroutine is called only once and should not be a subroutine at all").

Why this is better

- Fewer calls / simpler invariants. Operators that need fields no longer pay to rediscover `L/R` multiple times, and they get the always-needed `rI2 ← -R` (and optionally `SFLAG`) precomputed. See how `GETV` and `STORE` currently repeat the same checks right after `FCHECK`.
- Matches the one existing optimization. `COMPARE` already factors out the second decode via `GETAV`; the refactor generalizes that cleanliness instead of being a one-off.
- Faithful to Step-3. It's precisely the "enlarge/merge subroutines" advice Knuth gives after completing a first cut: push common pre/post code down so the outer structure is cleaner.

Concrete changes you'd make

- Control routine (option A): after `LD3 INST(4:4)` (line 027), if the opcode's F–class is "(L:R)", call `FDECODE` (your enhanced `FCHECK`) which sets `L`, `R`, `rI1=L`, `rI2=-R`, and `SFLAG`. Then have the opcode table point to `GETAV` (instead of `GETV`) and to `STORE1` (the post-decode entry).
- GETV: remove the prologue call to `FCHECK`; enter at the existing label after that call (`GETAV` already does exactly this).
- STORE: add label `STORE1` at the point right after the original `JMP FCHECK` and retarget the table to `STORE1` when pre-decoding is in effect; otherwise fall back to `STORE` for compatibility.

Either approach solves Exercise 1: you've studied all uses of `FCHECK`, noticed the duplicated patterns right after it, and reorganized so the decode happens once (or is inlined where it belongs), exactly the kind of Step-3 improvement Knuth advocates.

2 [20]

Here's a clean, self-contained SHIFT routine for the MIX simulator. It plugs into the program exactly where opcode 6 ("SHIFT") is dispatched in the operator table (see the listing that maps `F=0..5` to `SLA, SRA, SLAX, SRAX, SLC, SRC`). I keep A/X signs in `SIGNA/SIGNX` and only rearrange magnitudes in `AREG/XREG`, matching the simulator's data layout.

What the routine must do (recap)

- F=0: `SLA m` — shift A left by `m` bytes, zero-fill on the right; sign unchanged.
- F=1: `SRA m` — shift A right by `m` bytes, zero-fill on the left; sign unchanged.
- F=2: `SLAX m` — shift the (A,X) 10-byte pair left by `m`, zero-fill; signs unchanged.

- F=3: `SRAX m` — shift the (A,X) pair right by `m`, zero-fill; signs unchanged.
- F=4: `SLC m` — rotate the (A,X) pair left by `m` (circular); signs unchanged.
- F=5: `SRC m` — rotate the (A,X) pair right by `m` (circular); signs unchanged. Knuth's exercise simply asks us to "write the SHIFT routine… (operation code 6)."

MIX code

```
* ------------------------------------------------------------
* SHIFT - opcode 6
* Uses: rI3 = F (variant 0..5), rI5 = M (shift count, possibly signed)
* Touches: AREG/XREG magnitudes only; SIGNA/SIGNX unchanged
* Notes: If tracking timing exactly, add |M| to TIME in this routine,
*        since OPTABLE contributes the base 2u already for SHIFT.
* ------------------------------------------------------------
SHIFT   ENT2    0,5         rI2 ← M (shift count)
        J2NN    *+2         make rI2 ← |M|
        ENN2    0,2
        CMP3    =5=         F must be 0..5
        JG      CYCLE       (defensive: treat out-of-range F as NOP)

        JMP     SHTAB,3     dispatch by F

* --- table of tiny trampolines (same pattern used elsewhere) ---
SHTAB   JMP     S_SLA
        JMP     S_SRA
        JMP     S_SLAX
        JMP     S_SRAX
        JMP     S_SLC
        JMP     S_SRC

* --- F=0: SLA m (A only) ---
S_SLA   LDA     AREG
        SLA     0,2         shift A left by rI2 bytes
        STA     AREG
        JMP     S_DONE

* --- F=1: SRA m (A only) ---
S_SRA   LDA     AREG
        SRA     0,2         shift A right by rI2 bytes
        STA     AREG
        JMP     S_DONE

* --- F=2: SLAX m (A,X together) ---
```

```
S_SLAX  LDA     AREG
        LDX     XREG
        SLAX    0,2         shift (A,X) left by rI2 bytes
        STA     AREG
        STX     XREG
        JMP     S_DONE

* --- F=3: SRAX m (A,X together) ---
S_SRAX  LDA     AREG
        LDX     XREG
        SRAX    0,2         shift (A,X) right by rI2 bytes
        STA     AREG
        STX     XREG
        JMP     S_DONE

* --- F=4: SLC m (A,X circular left) ---
S_SLC   LDA     AREG
        LDX     XREG
        SLC     0,2         rotate (A,X) left by rI2 bytes
        STA     AREG
        STX     XREG
        JMP     S_DONE

* --- F=5: SRC m (A,X circular right) ---
S_SRC   LDA     AREG
        LDX     XREG
        SRC     0,2         rotate (A,X) right by rI2 bytes
        STA     AREG
        STX     XREG
        JMP     S_DONE

* --- wrap up; optional timing bump here ---
S_DONE  * If desired:  TIME ← TIME + rI2  (|M|)
        JMP     CYCLE
```

Why this is correct (brief reasoning)

1. Signs preserved. The simulator stores signs separately (`SIGNA`, `SIGNX`). We only load/store magnitudes from `AREG`/`XREG`, so shifting cannot disturb signs.
2. Zero-fill vs rotate. Host MIX's `SLA/SRA/SLAX/SRAX` zero-fill the vacated bytes; `SLC/SRC` rotate within the 10-byte `rAX` pair. That exactly matches MIX semantics for opcode 6 variants.

3. Large or negative counts. We take `|M|`. For `SLA/SRA` beyond the register width, the host shift yields zeros (desired). For rotations, host `SLC/SRC` behave modulo 10 bytes, so any `M` works sensibly.
4. Timing (if modeled). The OPTABLE entry already accounts for the base "2u" per SHIFT; if you're simulating time precisely, add `|M|` here to reflect per-byte steps. (Knuth's listing shows SHIFT's table entry and the pattern for per-op timing updates.)

```
*  SHIFT operators (opcode = 6)
*  Uses rI3 = F, rI5 = M (already set by the control routine).
*  Count = |M|; signs are preserved by MIX shift ops automatically.

SHIFT   DEC3    5           Is F > 5?
        J3P     FERROR      If so, invalid F for SHIFT.

        LD1     M(1:2)      rI1 ← |M| (shift count, magnitude of M).

        JMP     SFTABL,3    Dispatch to SLA/SRA/SLAX/SRAX/SLC/SRC.

* --- F = 0 : SLA (shift left rA) ---
SLAOP   LDA     AREG
        SLA     0,1         Shift rA left by rI1 bytes, zero-fill right.
        STA     AREG
        JMP     CYCLE

* --- F = 1 : SRA (shift right rA) ---
SRAOP   LDA     AREG
        SRA     0,1         Shift rA right by rI1 bytes, zero-fill left.
        STA     AREG
        JMP     CYCLE

* --- F = 2 : SLAX (shift left rAX as 10-byte word) ---
SLAXOP  LDA     AREG
        LDX     XREG
        SLAX    0,1         Shift rAX left by rI1 bytes, zero-fill right.
        STA     AREG
        STX     XREG
        JMP     CYCLE

* --- F = 3 : SRAX (shift right rAX) ---
SRAXOP  LDA     AREG
        LDX     XREG
        SRAX    0,1         Shift rAX right by rI1 bytes, zero-fill left.
        STA     AREG
```

```
        STX     XREG
        JMP     CYCLE

* --- F = 4 : SLC (circular left rAX) ---
SLCOP   LDA     AREG
        LDX     XREG
        SLC     0,1         Circular left shift of rAX by rI1 bytes.
        STA     AREG
        STX     XREG
        JMP     CYCLE

* --- F = 5 : SRC (circular right rAX) ---
SRCOP   LDA     AREG
        LDX     XREG
        SRC     0,1         Circular right shift of rAX by rI1 bytes.
        STA     AREG
        STX     XREG
        JMP     CYCLE

* Jump table for F = 0..5, addressed via "JMP SFTABL,3" after "DEC3 5".
* Place the label on the last entry (same pattern as the book's JUMP table).
        JMP     SLAOP
        JMP     SRAOP
        JMP     SLAXOP
        JMP     SRAXOP
        JMP     SLCOP
SFTABL  JMP     SRCOP
```

Why this matches the book's simulator

1. Dispatch pattern. The book's operator routines use `DEC3 k` followed by `JMP TABLE,3`, with the table label on the *last* entry so that negative offsets select earlier entries (see the `JUMP` operator's table). We do the same with `DEC3 5` and `SFTABL`.
2. Where the count comes from. The control routine has already computed the effective address and stored it in the variable `M`; it then sets `rI5 ← M`. We take the magnitude of `M` via `LD1 M(1:2)` to obtain a nonnegative byte count for shifting. (Signs of `M` are irrelevant to SHIFT.)
3. Which registers to touch. `SLA`/`SRA` affect only the A register; `SLAX`/`SRAX`/`SLC`/`SRC` treat `rA‖rX` as a 10-byte word. In the simulator, magnitudes live in `AREG`/`XREG`, and signs in `SIGNA`/`SIGNX`. MIX shift operators do not change signs, so we update only `AREG`/`XREG`.

4. Opcode and timing. The switch table ties opcode 6 to `SHIFT(2)`, so no extra timing logic is needed in the routine; the control code already accounts for the two time units when it indexes the table.

Notes and edge cases

- If the count exceeds the register width (e.g., SLA/SRA by 5, SLAX/SRAX by 10), MIX semantics produce all zeros in the shifted region; `SLC`/`SRC` are naturally modulo-10 circular shifts. The host MIX instructions used above enforce these behaviors directly.
- The routine intentionally ignores the sign of `M` by using `|M|` as the count, which matches standard descriptions of MIX shift instructions ("M specifies the number of bytes to be shifted").

3 [22]

What MOVE must do

- Semantics: move F words, one word at a time, from source block starting at M to destination block starting at the (simulated) rI1 address; when finished, rI1 is increased by F. If F=0, nothing happens. Execution time is `1 + 2F` units (the base "1" comes from the opcode table; we add `2` per word moved in the routine).
- Overlap behavior (important!): copying proceeds "forward" (from `M` to `M+1` … while storing to `rI1`, `rI1+1`, …). If destination starts inside the source and ahead of it (e.g., `rI1 = M+1`), the same word may be replicated, exactly as described in the manual. Your simulator should therefore not try to be "memmove-safe"; it must mimic MIX's specified effect.
- Control state the simulator has already set up for you: `rI5 ← M`, `rI3 ← F`, `rI4 ← C`, and the opcode table has placed the base time in `TIME` before jumping to the per-opcode routine. The magnitudes/signs of simulated registers live in memory cells like `I1REG`, `SIGN1`, etc.

MOVE routine (MIXAL)

Paste this under the label `MOVE` that's wired in the opcode table.

```
* MOVE - opcode 7
MOVE    J3Z   CYCLE          * If F=0, nothing to do.

        JMP   MEMORY         * Fetch SOURCE word at address rI5:
                             * on return: rX = sign, rA = magnitude (of memory[rI5])

        SRAX  5              * Pack sign+magnitude so the full word sits in rX.
```

```
* Validate DEST = simulated rI1 (must be in [0, BEGIN))
        LD1   I1REG          * rI1 ← magnitude of simulated index register 1
        LDA   SIGN1          * rA  ← sign of simulated index register 1
        JAP   OKDEST         * if +, OK
        J1NZ  MEMERROR       * if negative and magnitude   0 → invalid address
        STZ   SIGN1(0:0)     * normalize -0 as +0 (permit address 0)

OKDEST  CMP1  =BEGIN=        * rI1   BEGIN ?
        JGE   MEMERROR       * out of simulated memory → error

* Store the fetched word into DEST and account for time
        STX   0,1            * memory[rI1] ← fetched full word
        LDA   CLOCK          * add 2 units per moved word (base 1 was set already)
        INCA  2
        STA   CLOCK

* Advance pointers/count and loop
        INC1  1              * rI1++
        ST1   I1REG          * write back simulated I1
        INC5  1              * rI5++ (SOURCE++)
        DEC3  1              * F--
        JMP   MOVE           * keep moving until J3Z fires at top
```

This routine follows MIX's defined behavior exactly:

- It moves one word at a time, always reading the source first (via `MEMORY`) and then writing to the current destination (`STX 0,1`).
- Because we progress `SOURCE` (`rI5`) and `DEST` (`rI1`) forward in lock-step, the overlap case where `DEST > SOURCE` produces the same "replication" effect shown in the book (that's the intended semantics).
- We charge time by adding `2` to `CLOCK` per moved word; together with the opcode table's base time, this yields `1 + 2F`.
- We update the simulated `I1` immediately each step (by writing back `I1REG`), which is fine because programs can't observe intermediate state during an instruction; at completion, `rI1` has increased by F, as required.
- Address checks ensure both source (handled by `MEMORY`) and destination are inside simulated memory `[0, BEGIN)`, with `-0` treated as `+0`.

Quick sanity check on overlap

- Example from the text: `F=3`, `M=1000`, `rI1=999` → you'll copy `1000→999`, `1001→1000`, `1002→1001` (no surprise).

- If instead `rI1=1001`, you'll copy `1000→1001`, then (since `1001` now holds the old 1000) `1001→1002`, then `1002→1003`, so the original `CONTENTS(1000)` appears three times—exactly as MIX specifies.

Where these hooks live in the simulator

- The control routine unpacks `INST`, computes `M` with indexing, stores `F` in `rI3` and `M` in `rI5`, looks up the routine in `OPTABLE`, stores the base time in `TIME`, and jumps to `MOVE`. Your routine returns to `CYCLE` when `F` becomes zero.

4 [14]

Here's the smallest correct change to Knuth's MIX-simulator so it "starts as if the GO button had been pushed."

What the GO button does (per §1.3.1, Ex. 26)

When GO is pressed (with a card reader present), MIX (1) reads one card into locations 0000–0015, (2) clears the overflow toggle and sets rJ ← 0, then (3) jumps to location 0000 to begin execution.

What the simulator already does

In Program M (the simulator), the main loop already:

- Clears the overflow toggle (`STZ OVTog`) and
- Begins execution at location 0 by setting the "next instruction" register to 0 (`ENT6 0`, where rI6 is the location of the next instruction).

So the only GO-state it doesn't establish is rJ = 0.

Minimal patch

Add one instruction at program start to zero the simulated J register:

```
001 * MIX SIMULATOR
002 ORIG 3500
003 BEGIN   STZ TIME(0:2)   ; clear time
004          STZ OVTog      ; clear overflow toggle  (GO requirement)
004a         STZ JREG       ; * new: rJ ← 0       (GO requirement)
005          STZ COMPI
006          ENT6 0         ; start at location 0   (GO requirement)
007 CYCLE    LDA CLOCK
...
```

- `STZ JREG` makes the simulator's rJ match the GO button's post-reset state.
- `STZ OVTog` and `ENT6 0` already matched the GO semantics (overflow cleared; "PC" set to 0000, since rI6 is the location of the next instruction). The control routine's use of rI6 for the fetch is described just above Program M.

Notes

- We aren't simulating the initial card read (GO step 1) here; I/O operators are intentionally left for later exercises (§1.4.3.1, Ex. 6). This exercise only asks the simulator to *begin* as if GO had been used.
- After this one-line change, the simulator's initial state matches items (2)–(3) of GO: rJ=0, overflow cleared, and execution begins at 0000.

That's it—one added `STZ JREG` at `BEGIN`.

5 [24]

Timing rules we'll use (real MIX)

MIX uses time unit u. From the timing summary:

- All LOAD/STORE, ADD/SUB, comparisons, shifts: 2u
- MOVE: 1u + 2u per word
- MUL/NUM/CHAR: 10u; DIV: 12u
- All remaining ops (including jumps, INC/DEC/ENT): 1u In particular, LDA = 2u, ENTA = 1u on real MIX.

Assumptions (to keep the path concrete)

- Instruction uses no indexing (I=0), so the indexing block in the control routine is skipped.
- No errors (no range, field, or memory errors).
- For LDA, the usual field (0:5) (sign included).
- For ENTA, the address M 0 (the "M=0" special sign case is noted later). These are the straight-line, normal paths through the simulator.

The simulator's fixed control overhead per simulated instruction

Every instruction first goes through the control routine at label CYCLE (load CLOCK, fetch INST, unpack M/I/F/C, consult OPTABLE, and jump). Counting times by instruction class yields 34u before the jump to the operator routine when I=0 (lines 007–035: loads/stores/shift = 2u each; INC/DEC/J? = 1u).

Simulating LDA

Path: CYCLE → OPTABLE → LOAD → GETV → (FCHECK, MEMORY) → LOAD1 → SIZECHK → CYCLE. Key code: LOAD/LOADN (lines 196–205), GETV (149–158), FCHECK (135–146), MEMORY (125–134), SIZECHK/OVCHECK (289–291, 191).

- Control overhead to reach operator: 34u.
- Operator + subroutines:
  - `JMP GETV`: 1u
  - FCHECK (valid field): STJ, ENTA, ENTX, DIV, CMPX, JG, STX, STA, LD1, CMPA, JLE → 27u
  - MEMORY: STJ, JS? (not taken), CMP, JGE (not taken), LDX, ENTA, SRAX, LDA, JMP → 14u
  - Return in GETV (sign included, so skip setting + sign): J1Z, LD2N, SRA, JMP → 6u
  - LOAD1 store magnitude/sign: ENT1, STA, STX, JMP → 6u
  - SIZECHK/OVCHECK (no index overflow, no overflow): LD1, J1Z, JNOV → 4u

  Sum operator-side = 1 + 27 + 14 + 6 + 6 + 4 = 58u (plus 5u for the two unconditional returns to GETV's and MEMORY's `JMP *` sites) → 63u total operator segment.
- Total simulated time for LDA = 34u + 63u = 97u.

Real MIX time for LDA = 2u. Slowdown 97/2 48.5×.

Simulating ENTA

Path: CYCLE → OPTABLE → ADDROP → (LOAD1) → SIZECHK → CYCLE. Key code: ADDROP block (265–278), then LOAD1 and SIZECHK as above.

- Control overhead to reach operator: 34u.
- Operator path (typical M 0 branch):
  - ADDROP prologue & sign/magnitude setup: DEC3, J3P (not taken), ENTX, JXNZ (taken), SRAX, LDA, ENT1, `JMP 1F,3`, `JMP LOAD1` → 11u
  - LOAD1 store magnitude/sign: STA, STX, `JMP SIZECHK` → 5u
  - SIZECHK/OVCHECK: LD1, J1Z, JNOV → 4u

  Sum operator-side = 20u.
- Total simulated time for ENTA = 34u + 20u = 54u.

Real MIX time for ENTA = 1u. Slowdown 54/1 = 54×.

Notes on variants

- Indexing present (I 0): the control routine executes the indexing block (lines 018–026), adding ~15u more (several loads/stores/shifts/add/compare/jump).

- LDA with field not including sign: GETV executes three extra ops (ENTX; two shifts), adding ~5u.
- ENTA with M = 0: executes two extra ops (LDX INST; ENTA 1) to pick up the instruction's sign rule, adding ~3u.

Final comparison

- LDA: simulated ~97u vs real 2u (49× slower).
- ENTA: simulated ~54u vs real 1u (54× slower).

These figures come straight from counting the executed instructions along the normal paths in Program M and applying MIX's timing rules.

6 [28]

Here's a clean way to add the missing I/O operators to the MIX simulator, limited to unit 16 (card reader) and unit 18 (line printer), and consistent with Knuth's timing model:

- Opcodes and meanings (from the MIX table): 34 = JBUS "unit F busy?", 35 = IOC "control, unit F", 36 = IN "input, unit F", 37 = OUT "output, unit F", 38 = JRED "unit F ready?"
- Exercise requirement: implement JBUS, IOC, IN, OUT, JRED; allow only units 16 and 18; assume read-card and skip-to-new-page take T=10000u; print-line takes T=7500u. Also special-case `JBUS *` to avoid the simulator "stopping."

Design (what to simulate)

MIX overlaps I/O with computation. Issuing IN/OUT/IOC takes 1 unit of CPU time and makes the device busy for T units; the CPU continues. The queries JBUS/JRED test that busy/ready state. We therefore maintain per-device "ready times" and a CPU clock:

- `CLOCK` — current CPU time (u).
- `RDRFREE` — earliest time at which unit 16 (reader) is ready.
- `PRTFREE` — earliest time at which unit 18 (printer) is ready.
- Constants: `TREAD=10000`, `TSKIP=10000`, `TPRINT=7500`.
- Overlap rule: when (re)starting an I/O, let `start = max(CLOCK, DEVFREE)` then set `DEVFREE = start + T`. CPU time still advances by 1 for the instruction itself (the "1+T" shown in the table), while the T portion accrues on the device timeline.

Handlers to splice into "INDIVIDUAL OPERATORS"

Below is MIXAL-style code that drops into the simulator's opcode dispatch, following the conventions used elsewhere in §1.4.3.1 (use your existing `MEMORY` check and jump simulation sequence, e.g., `JSJ JMP MEMORY / ENT6 0,5`).

Data and constants (in your simulator's data area):

```
TREAD   EQU     10000          * read-card (unit 16)
TSKIP   EQU     10000          * skip-to-new-page (unit 18, IOC)
TPRINT  EQU     7500           * print-line (unit 18)
CLOCK   CON     0
RDRFREE CON     0              * unit 16 ready time
PRTFREE CON     0              * unit 18 ready time
```

Utility: pick device's FREE-TIME cell given F (16 or 18). If your decode already provides F in a register, keep it; otherwise extract F the way your other handlers do.

```
* On entry: r? = F (device number), rI5 = effective M (jump/address).
DEVPTR  CMP?   =16=
        JE     DEVPTR16
        CMP?   =18=
        JE     DEVPTR18
        JMP    FERROR            * only 16 and 18 are supported here
DEVPTR16 LDA   RDRFREE
        JMP    DEVPTRX
DEVPTR18 LDA   PRTFREE
DEVPTRX  STA   FREEBUF           * FREEBUF holds the fetched free time
        JMP    *+1
FREEBUF  CON   0                 * scratch word to hold device free time
```

36 IN (unit F)

- Support: F=16 only (card reader).
- Effect: begin a read "now or when free" and mark reader busy until `start+TREAD`. Optionally deposit a 16-word "card image" at M..M+15 (for a bare simulator this can be zeros or any deterministic pattern). CPU time +1.

```
INOP    ...                      * you're in opcode 36 case
        * require F=16
        CMP?   =16=
        JNE    FERROR
        * start := max(CLOCK, RDRFREE)
        LDA    CLOCK
        CMPA   RDRFREE
        JGE    IN_START_READY
        LDA    RDRFREE
```

```
IN_START_READY
        ADD    =TREAD=
        STA    RDRFREE
        * (Optional) deposit 16 words at M..M+15:
        *   ENT1 0; LOOP 16x: STZ 0,5; INC5 1; DEC1 1; J1P LOOP
        * advance CPU time by 1
        LDA    CLOCK
        INCA   1
        STA    CLOCK
        JMP    CYCLE
```

37 OUT (unit F)

- Support: F=18 only (line printer).
- Effect: schedule a print of one "line" for `TPRINT` units. CPU time +1.

```
OUTOP   CMP?   =18=
        JNE    FERROR
        * start := max(CLOCK, PRTFREE)
        LDA    CLOCK
        CMPA   PRTFREE
        JGE    OUT_START_READY
        LDA    PRTFREE
OUT_START_READY
        ADD    =TPRINT=
        STA    PRTFREE
        * (optional: ignore actual text formatting in this bare simulator)
        LDA    CLOCK
        INCA   1
        STA    CLOCK
        JMP    CYCLE
```

35 IOC (unit F)

- Support: F=18 used as "skip to new page," cost `TSKIP`. (IOC for 16 is a no-op here.) CPU time +1.

```
IOCOP   CMP?   =18=
        JE     IOC_PRN
        * F=16 -> no-op for this minimal simulator
        LDA    CLOCK
        INCA   1
```

```
        STA    CLOCK
        JMP    CYCLE

IOC_PRN LDA    CLOCK
        CMPA   PRTFREE
        JGE    IOC_START_READY
        LDA    PRTFREE
IOC_START_READY
        ADD    =TSKIP=
        STA    PRTFREE
        LDA    CLOCK
        INCA   1
        STA    CLOCK
        JMP    CYCLE
```

34 JBUS (unit F busy?)

- If the device is still busy (its FREE time > CLOCK), jump to M.
- Special case for `JBUS *`: if busy and `M` is the current instruction address, don't spin at zero cost; instead advance `CLOCK` to the device's FREE time (this models "wait until ready" and prevents the simulator from "stopping," per Knuth's note). If not busy, fall through. CPU time +1 (for the test itself).

```
JBUSOP  * load device free time
        JSR    DEVPTR
        * busy if FREEBUF > CLOCK
        LDA    FREEBUF
        CMPA   CLOCK
        JLE    JBUS_NOTBUSY
        * device busy -> conditional jump
        * detect JBUS * (M == current PC). Your simulator already knows how to
        * compare M against "this instruction"; reuse that mechanism.
        JSR    IS_STAR           * sets CC: = if M==*
        JE     JBUS_WAIT
        * normal jump to M
        JSJ    JMP MEMORY        * validate address in rI5, as elsewhere
        ENT6   0,5               * simulate the jump
        * account 1 unit for the instruction itself
        LDA    CLOCK
        INCA   1
        STA    CLOCK
        JMP    CYCLE
```

```
JBUS_WAIT
        * wait until ready by jumping time forward
        LDA    FREEBUF
        STA    CLOCK
        JMP    CYCLE
JBUS_NOTBUSY
        * not busy, just consume 1 unit and continue
        LDA    CLOCK
        INCA   1
        STA    CLOCK
        JMP    CYCLE
```

38 JRED (unit F ready?)

- Jump to M iff device is ready (`FREE  CLOCK`). CPU time +1 either way.

```
JREDOP  JSR    DEVPTR
        LDA    FREEBUF
        CMPA   CLOCK
        JGT    JRED_NREADY
        * ready -> jump
        JSJ    JMP MEMORY
        ENT6   0,5
        LDA    CLOCK
        INCA   1
        STA    CLOCK
        JMP    CYCLE
JRED_NREADY
        LDA    CLOCK
        INCA   1
        STA    CLOCK
        JMP    CYCLE
```

Notes and rationale

- Unit numbers: 16 is the card reader (e.g., the reset loader does `IN 0(16)`), 18 is the printer; restricting support to those two matches the exercise.
- Timing matches the MIX table ("1+T" for IN/OUT) and the multiple-buffering discussion: T accrues on the device; the CPU continues after the 1-unit issue time.
- `JBUS *` "waits" by fast-forwarding `CLOCK` to the device's free time, exactly to avoid a zero-time busy-wait loop (the note in the problem).

7 [32]

Here's a clean, self-contained way to solve this exercies. The point is to keep your MIX I/O simulator realistic: executing `IN/OUT` must not move data immediately; the actual transfer happens roughly halfway through the device's service time, while the device remains busy for the full time. This prevents programs from (incorrectly) assuming that `IN/OUT` are instantaneous. The exercise builds directly on the I/O instructions and device behavior defined earlier (units 16 = card reader, 18 = line printer; `IN/OUT/IOC/JBUS/JRED` semantics), and on the timing constants given in Ex. 6 (read-card or skip-page: `T=10000u`; print-line: `T=7500u`). The exercise's specific requirement ("make the transmission occur after about half the time") appears at the end of the section's exercise list.

Model the timing

Maintain a global simulator clock `now` (in MIX time units `u`) and, for each device `d  {16,18}`, these fields:

- `ready_at[d]` — time when the device will become ready again.
- `xfer_at[d]` — time when the *data movement* (memory device buffer) actually occurs; this is `start + ceil(T[d]/2)`.
- `pending[d]` — `None | ("IN", M) | ("OUT", M) | ("IOC", op)` describing the scheduled action.
- Optional device buffers (e.g., next card image for 16; print line buffer for 18).

Timing constants (from Ex. 6): `T[16].read_card = 10000`, `T[18].print_line = 7500`, and `T[18].skip_to_new_page (IOC) = 10000`.

Event processing hook

Before fetching each instruction in your main `CYCLE` loop, process any I/O events whose time has arrived:

```
def service_io_events():
    for d in (16, 18):
        if pending[d] and now >= xfer_at[d]:
            # Perform the scheduled data movement exactly once.
            if pending[d].op == "IN":
                mem[M ... M+block_size(d)-1] = device_in_buffer[d]
            elif pending[d].op == "OUT":
                device_out_buffer[d] = mem[M ... M+block_size(d)-1]
            # IOC with no data movement: nothing to do here.
            pending[d] = None
            xfer_at[d] = +∞
```

This "half-time" transfer ensures that a program that uses the data too early (after `IN`) or changes it too late (after `OUT`) will behave incorrectly—exactly the mistake this exercise wants to catch.

Instruction semantics (with half-time transfers)

Use the architectural rules from §1.3.1: `IN/OUT` wait until the unit is ready *to start*; the transfer completes later; `IOC` initiates a device-specific operation; `JRED/JBUS` test readiness.

```
IN M(F=unit)


d = unit
wait_until_ready(d)               # if now < ready_at[d], advance now to ready_at[d]
start = now
pending[d] = ("IN", M)            # schedule memory ← device
xfer_at[d] = start + ceil(T[d]/2) # "approximately half" the device time
ready_at[d] = start + T[d]        # unit remains busy for full time
```

Block size: 16 words for unit 16 (card reader), 24 for unit 18 (printer) if you ever support input there; here we limit to 16 for the card reader.

```
OUT M(F=unit)


d = unit
wait_until_ready(d)
start = now
pending[d] = ("OUT", M)           # schedule device ← memory
xfer_at[d] = start + ceil(T[d]/2)
ready_at[d] = start + T[d]
```

For unit 18, the printed line's *content* is captured at `xfer_at[18]`; the physical printing completes at `ready_at[18]`.

```
IOC M(F=unit)
```

Wait until not busy, then perform the control; e.g., for the printer "skip-to-new-page" takes 10000 u but does no data transfer:

```
wait_until_ref(18)
start = now
pending[18] = ("IOC", "SKIP")
xfer_at[18] = +∞                  # no data movement
ready_at[18] = start + 10000
```

(If you later support `IOC` on other units, replicate this pattern with their `T`.)

`JRED M(F=unit)` and `JBUS M(F=unit)`

Use the §1.3.1 definitions: `JRED` jumps iff the unit is ready (`now  ready_at[d]`), `JBUS` jumps iff not ready.

Important "JBUS *" special case

As hinted in Ex. 6, if the target of `JBUS` is the instruction itself (busy-wait spin), you must advance `now` to the next interesting time to avoid a frozen simulator:

```
if op == JBUS and target == PC and now < ready_at[d]:
    now = min(ready_at[d], min(xfer_at[k] for k with pending[k]))
```

This is the pragmatic fix that keeps the loop progressing.

Why this satisfies Exercise 7

- `IN/OUT` no longer "complete" at issue time: the simulator defers the data movement to `start + T/2`, while the device remains busy until `start + T`.
- Programs that incorrectly assume immediate availability (after `IN`) or immediate consumption (after `OUT`) will misbehave, reproducing the real-machine hazard the exercise is designed to prevent.

Quick timeline sanity checks

1. Card read into 1000 with immediate use (wrong):

```
t0:     IN 1000(16)           # schedules memory update at t0+5000; device busy until t0+1000
t0+1:   LDA 1000              # reads OLD data (correctly exposing the bug)
...
```

2. Printer OUT with later modification (also wrong):

```
t0:     OUT 2000(18)          # captures line at t0+3750; printer finishes at t0+7500
t0+2000: STA 2000             # too early: the printed line will still reflect the pre-
change contents
```

3. Proper synchronization:

```
OUT 2000(18);  JBUS *(18)     # spin; simulator advances now to ~t0+7500
# when we exit, the line has been printed and the unit is ready again
```

1) Data layout for I/O timing and pending actions

We keep, per device, three fields:

- `READY_AT[d]` — when the device becomes ready again.
- `XFER_AT[d]` — when the *actual data movement* occurs.
- `PEND[d]` — pending action kind and base address (encoded in one MIX word: sign = kind; value = base address).

Choose encodings:

- `PEND` sign: `+` = none; `-` = has pending. The action code is stored in the most significant byte (B1) and the memory base address in the remaining bytes.
- Action codes: `1` = IN, `2` = OUT, `3` = IOC (no data move).

You can keep device constants (`TREAD16`, `TPRINT18`, etc.) as you prefer.

```
          ORIG  2000

* --- Global clock (u time units) ---
NOW       CON   0

* --- Device constants (adapt to your table) ---
* D16 = card reader, D18 = line printer
D16       CON   16
D18       CON   18

* service times (u)
TREAD16   CON   10000          * read card / skip page (example)
TPRINT18  CON   7500
TSKIP18   CON   10000          * IOC skip page

* half-times (ceil(T/2))
HREAD16   CON   5000
HPRINT18  CON   3750
HSKIP18   CON   5000           * not used (no data move), keep for symmetry

* --- Per-device tables: index by (device-16) if you only support 16..18
* We'll allocate a small contiguous array and compute offsets.
NDEV      CON   3              * support devices 16,17,18
BASEDEV   CON   16

READY_AT  CON   0,0,0          * READY_AT[16], [17], [18]
XFER_AT   CON   0,0,0          * XFER_AT[16], [17], [18]
PEND      CON   +0,+0,+0       * +0 = none; -(action<<30 | base) = pending
```

```
* encodings
ACT_IN    CON   1
ACT_OUT   CON   2
ACT_IOC   CON   3
NONE      CON   +0

* scratch
TMP       CON   0
IDX       CON   0              * working index = d - BASEDEV
W         CON   0              * generic temp
```

Helper to compute `IDX = d - 16` and leave it in `IDX`:

```
IDXOF     STJ   IDXOF_RET
          STA   TMP          * A has d
          LDA   TMP
          SUB   BASEDEV
          STA   IDX
          JBUS  *            * (no-op; keep same timing style)
IDXOF_RET JMP   0
```

2) Small utilities

Store a "pending action" word

We encode `(action, base)` in one word: `B1=action`, `B2..B5=base`, sign negative = "pending".

```
SETPEND   STJ   SP_RET
          STA   TMP          * A = base address M
          ENT1  0
          ENTA  0
          OR    NONE         * clear A
          LDX   TMP
          SLA   5            * move X into B2..B5 (implementation detail varies)
          OR    ACT          * B1 = action (ACT in A)
          ST1   W            * save if you prefer a different scratch
          NEG   W            * make it pending (negative sign)
          LDA   IDX
          STA   TMP
          * PEND[base+IDX] ← W
          LDX   TMP
          LD2   =PEND=
```

```
          ADD   X
          STA   @0            * store W into PEND[IDX]
SP_RET    JMP   0
```

(If your assembler lacks these exact bit ops, you can lay out two words `PENDKIND[ ]` and `PENDBASE[ ]` instead; logic stays the same and code gets simpler.)

Read/write table slots

If your MIXAL doesn't like indirect addressing macros, you can provide tiny helpers. A simpler, clearer approach is to store each array as separate contiguous words and use `ENTx`/`INCx` stepping. To keep this answer concise, I'll use a compact "index load/store" idiom; translate to your style as needed.

3) I/O event poller (run before each instruction fetch)

This is the heart of Exercise 7. It performs the *deferred* transfers at `XFER_AT[d]` but does not make the device ready until `READY_AT[d]`.

Call `IOPOLL` at the top of your `CYCLE` loop, before fetching the next instruction.

```
IOPOLL    STJ   IOP_RET
          ENT3  0             * R3 = loop index 0..NDEV-1
IOP_LP    CMP3  NDEV
          JGE   IOP_DONE

          * compute absolute device number d = BASEDEV + R3
          LDA   BASEDEV
          ADD   R3
          STA   TMP           * TMP = d
          JSR   IDXOF         * set IDX = d - BASEDEV

          * if PEND[IDX] == NONE or XFER_AT[IDX] > NOW, skip
          * Load XFER_AT[IDX] into A:
          LD2   =XFER_AT=
          ADD   IDX
          LDA   @0
          STA   W             * W = xfer time
          CMPA  NOW
          JG    IOP_NEXT      * not yet time

          * Load PEND[IDX]
          LD2   =PEND=
          ADD   IDX
```

```
          LDA   @0
          JPL   IOP_NEXT      * sign + => no pending

          * decode action and base; here I'll assume you split them
          * If you implemented PENDKIND/PENDBASE arrays, this becomes trivial:
          *   kind = PENDKIND[IDX]; base = PENDBASE[IDX]

          * --- PERFORM THE DATA MOVE IF action == IN/OUT ---
          * For clarity, I'll branch by device number and action
          LDA   TMP           * d in A
          CMPA  D16
          JE    IOP_D16
          CMPA  D18
          JE    IOP_D18
          JMP   IOP_CLEAR     * unknown device => just clear pending

* --- Device 16 (card reader): move 16 words at XFER time ---
IOP_D16   * if ACT_IN: mem[M..M+15] <- device buffer
          * if ACT_OUT: (rare for reader) ignore or treat as error
          * In most simulators, you keep a prepared card image buffer CARD[16].
          * Copy CARD[k] -> M[k] for k=0..15 (or vice versa for OUT).
          * Pseudocode in MIXAL outline:

          * ... decode base addr into R1 (left to you) ...
          * ... copy loop of 16 words memory <-- CARDBUF ...
          JMP   IOP_CLEAR

* --- Device 18 (line printer): capture line buffer from memory at XFER time ---
IOP_D18   * if ACT_OUT: PRNBUF <- mem[M..M+N-1] (your N; often 24 words)
          * if ACT_IN : usually unsupported
          * MIXAL copy loop outline:
          * ... decode base addr into R1 ...
          * ... copy loop memory --> PRNBUF ...
          JMP   IOP_CLEAR

* After moving data (or deciding no move for IOC), clear PEND and set XFER_AT far in future
IOP_CLEAR LD2   =PEND=
          ADD   IDX
          LDA   NONE
          STA   @0

          LD2   =XFER_AT=
```

```
          ADD   IDX
          LDA   =+7777777=    * "infinite" (larger than any NOW)
          STA   @0

IOP_NEXT  INC3  1
          JMP   IOP_LP

IOP_DONE  JMP   IOP_RET
IOP_RET   JMP   0
```

Notes:

- Keep your actual memory copy routines where you already copy between device buffers and core. The point is *when* you do it: here, inside `IOPOLL` once `NOW  XFER_AT[d]`, not when `IN/OUT` is issued.
- IOC actions normally do no data move — so `IOPOLL` simply clears `PEND` at XFER time only for IN/OUT; IOC needs no XFER slot at all (we'll set it to "infinite").

4) Modifications to instruction semantics

You only change when data moves and how readiness is computed. The opcode timing (issuing cost) stays as in your simulator.

IN M(F=unit d)

- If `NOW < READY_AT[d]`, you must *not* block the whole CPU (MIX uses `JBUS/JRED` for sync); the architectural spec is that the instruction *initiates* the operation if the device is ready. Many simulators either (a) trap as "busy" or (b) start the op at `READY_AT`. For simplicity, I recommend: "if busy, advance `NOW` to `READY_AT[d]` and start," which matches typical exercises.
- Upon starting, set:
  - `PEND[d] = (IN, M)`.
  - `XFER_AT[d] = START + T/2`.
  - `READY_AT[d] = START + T`.

```
* On decode, A holds M (effective address), F holds unit d
IN_OP     STJ   IN_RET
          STA   TMP           * TMP = M
          LDA   FREG          * get device number d
          STA   W
          JSR   IDXOF         * IDX = d - BASEDEV
```

```
          * wait until device ready
          LD2   =READY_AT=
          ADD   IDX
          LDA   @0
          CMPA  NOW
          JLE   IN_START
          STA   NOW           * advance NOW to READY_AT[d]

IN_START  * schedule IN
          * XFER_AT = NOW + HALF_T[d]
          LDA   W             * d in A
          CMPA  D16
          JE    IN16
          CMPA  D18
          JE    IN18

          JMP   IN_DONE       * unsupported device -> no-op or error

IN16      LDA   NOW
          ADD   HREAD16
          LD2   =XFER_AT=
          ADD   IDX
          STA   @0

          * READY_AT = NOW + TREAD16
          LDA   NOW
          ADD   TREAD16
          LD2   =READY_AT=
          ADD   IDX
          STA   @0

          * PEND = (ACT_IN, M)
          LDA   TMP          * base M
          STA   W
          LDA   ACT_IN
          STA   ACT
          JSR   SETPEND
          JMP   IN_DONE

IN18      * normally input from printer is invalid; ignore or treat as error
          JMP   IN_DONE
```

```
IN_DONE   JMP   IN_RET
IN_RET    JMP   0
```

OUT M(F=unit d)

Same pattern, with OUT action and device-specific T and HALF_T.

```
OUT_OP    STJ   OUT_RET
          STA   TMP          * TMP = M
          LDA   FREG
          STA   W
          JSR   IDXOF

          * wait until device ready
          LD2   =READY_AT=
          ADD   IDX
          LDA   @0
          CMPA  NOW
          JLE   OUT_START
          STA   NOW

OUT_START LDA   W
          CMPA  D16
          JE    OUT16
          CMPA  D18
          JE    OUT18
          JMP   OUT_DONE

OUT16     * card reader OUT usually meaningless; either ignore or error
          JMP   OUT_DONE

OUT18     * printer: capture line at half time; finish at full time
          LDA   NOW
          ADD   HPRINT18
          LD2   =XFER_AT=
          ADD   IDX
          STA   @0

          LDA   NOW
          ADD   TPRINT18
          LD2   =READY_AT=
          ADD   IDX
```

```
          STA   @0

          LDA   TMP
          STA   W
          LDA   ACT_OUT
          STA   ACT
          JSR   SETPEND
          JMP   OUT_DONE

OUT_DONE  JMP   OUT_RET
OUT_RET   JMP   0
```

IOC M(F=unit d)

IOC has no data transfer. It just keeps the device busy for its service time (e.g., skip page).

```
IOC_OP    STJ   IOC_RET
          LDA   FREG
          STA   W
          JSR   IDXOF

          * wait until device ready
          LD2   =READY_AT=
          ADD   IDX
          LDA   @0
          CMPA  NOW
          JLE   IOC_START
          STA   NOW

IOC_START LDA   W
          CMPA  D18
          JE    IOC18
          JMP   IOC_DONE

IOC18     * skip to new page, no data movement
          LDA   NOW
          ADD   TSKIP18
          LD2   =READY_AT=
          ADD   IDX
          STA   @0

          * Ensure XFER_AT "infinite" and PEND cleared
```

```
          LD2   =XFER_AT=
          ADD   IDX
          LDA   =+7777777=
          STA   @0

          LD2   =PEND=
          ADD   IDX
          LDA   NONE
          STA   @0
          JMP   IOC_DONE

IOC_DONE  JMP   IOC_RET
IOC_RET   JMP   0
```

JRED / JBUS

- `JRED` jumps iff `NOW  READY_AT[d]`.
- `JBUS` jumps iff `NOW < READY_AT[d]`.

Exercise-friendly special case: if the target is the instruction itself (busy spin `JBUS *`), *advance* `NOW` to the next interesting time (the earlier of `READY_AT[d]` and any `XFER_AT[k]` that is still in the future), then fall through (don't infinite-loop).

```
JRED_OP   STJ   JR_RET
          LDA   FREG
          JSR   IDXOF
          LD2   =READY_AT=
          ADD   IDX
          LDA   @0
          CMPA  NOW
          JLE   JR_TAK       * ready → jump
          JMP   JR_NOT
JR_TAK    JMP   TARGET       * fill by your assembler
JR_NOT    JMP   NEXTPC
JR_RET    JMP   0
```

```
JBUS_OP   STJ   JB_RET
          LDA   FREG
          STA   W
          JSR   IDXOF

          * check readiness
```

```
          LD2   =READY_AT=
          ADD   IDX
          LDA   @0
          CMPA  NOW
          JLE   JB_NOT       * ready => do not jump

          * device busy → would jump.
          * If target == PC (self-loop), advance NOW to min(next READY or next XFER across al
          * Detect self-loop using your decoded target; here assume flag SELF is set by decod
          LDA   SELF
          JZE   JB_TAK       * not self → just jump

          * compute NEXT = READY_AT[d]
          STA   W            * preserve A
          LD2   =READY_AT=
          ADD   IDX
          LDA   @0
          STA   TMP          * TMP = candidate time

          * scan XFER_AT for an earlier future event
          ENT3  0
JB_SCAN   CMP3  NDEV
          JGE   JB_SET
          LD2   =XFER_AT=
          ADD   R3
          LDA   @0
          CMPA  NOW
          JLE   JB_NEXT
          CMPA  TMP
          JGE   JB_NEXT
          STA   TMP          * tighter min

JB_NEXT   INC3  1
          JMP   JB_SCAN

JB_SET    LDA   TMP
          CMPA  NOW
          JLE   JB_TAK       * nothing ahead → just jump
          STA   NOW          * advance time to next event
          JMP   JB_EXIT

JB_TAK    JMP   TARGET
```

```
JB_NOT    JMP   NEXTPC
JB_EXIT   JMP   NEXTPC
JB_RET    JMP   0
```

5) Hook `IOPOLL` into your fetch/execute loop

At the very top of `CYCLE` (or just before fetching the next instruction), call `IOPOLL`. This guarantees that once `NOW  XFER_AT[d]`, one (and only one) transfer happens before any instruction that might observe memory.

```
CYCLE     * ... your loop prologue ...
          JSR   IOPOLL
          * fetch instruction at PC
          * decode & dispatch
          * execute
          * advance PC
          JMP   CYCLE
```

6) Notes on memory moves

- I've left the actual buffer copies as comments because every simulator lays out buffers differently. The change the exercise asks for is purely *temporal*: move those copies from the `IN/OUT` handlers to `IOPOLL` at `NOW  XFER_AT[d]`.
- Typical choices:
  - Card reader: fixed 16 words `CARDBUF[0..15]` prepared by your card-file driver. At `IN` half-time: `mem[M..M+15] ← CARDBUF`.
  - Printer: copy `mem[M..M+N-1] → PRNBUF` (N = your line-encoding footprint) at `OUT` half-time; the physical "printing" (e.g., writing to host file) can happen either immediately at half-time or at the moment the device becomes ready; pedagogically, capturing the line at half-time and writing it at `READY_AT` is clean.

7) Quick sanity tests

- Unsafe early use after IN: `IN 1000(16); LDA 1000` should load the *old* contents if executed before `XFER_AT[16]`.
- Unsafe late modification after OUT: `OUT 2000(18); STA 2000` before `XFER_AT[18]` must not change the captured line; after `XFER_AT[18]`, it must be too late.
- Synchronization: `OUT 2000(18); JBUS *(18)` will advance `NOW` to `min(READY_AT[18], next XFER)` and then proceed when ready.

That's the complete wiring you need. If you share the specific buffer layouts you're using (sizes and labels for the card/printer buffers), I can fill in the tiny copy loops to match your memory map exactly.

8 [20]

Answer: False.

- In the control routine, the simulator fetches the next instruction with line 010: `LDA 0,6` (i.e., from simulated location held in the host index register rI6). Immediately after storing it, the simulator increments rI6 at line 012 with `INC6 1`. Thus the fetch uses the current value of rI6, and the increment prepares the next value.
- The simulator initializes rI6 $\leftarrow$ 0 at line 006. Later, jumps set rI6 directly to the validated target address using `ENT6 0,5` in the JSJ/JMP handler (after a range check done by the `MEMORY` subroutine). This guarantees that when a jump occurs, the new rI6 is within [0, BEGIN).

However, there is no range check before the fetch at line 010. Consider what happens when the simulated program executes an instruction located at address BEGIN $-$ 1 that does not change the location counter (i.e., not a jump):

1. At the start of that iteration, rI6 = BEGIN $-$ 1, so line 010 fetches from that address.
2. Line 012 executes `INC6 1`, so rI6 becomes BEGIN.
3. The operator returns to the control routine; on the next iteration, line 010 will be executed with rI6 = BEGIN, which violates $rI_6 <$ `BEGIN`.

A concrete two-instruction counterexample in the simulated memory (addresses 0..BEGIN$-$1) is:

- At location 0: JSJ BEGIN$-$1 (unconditional jump to the last simulated cell; the simulator validates the target via `MEMORY`, then sets rI6 $\leftarrow$ BEGIN$-$1).
- At location BEGIN$-$1: any non-jump such as STZ 0 (store zero). After executing it, line 012 makes rI6 = BEGIN; the next fetch (line 010) now uses an out-of-range address.

For completeness, note that:

- The `MEMORY` subroutine explicitly enforces 0 address $<$ BEGIN when the simulator accesses simulated memory via rI5, but this protection does not cover the line-010 fetch via rI6.
- Therefore $0 \leq rI_6$ is preserved, but $rI_6 <$ `BEGIN` is not guaranteed at every execution of line 010.

So the statement is false.

1.4.3.2 Trace routines

1 [22]

On MIX, the only way to set rJ is to **execute a jump** whose address is located at position ; executing `JMP …` from location sets rJ to +1. Therefore, to restore rJ to the "start" value saved earlier (call that value **J0**), we can:

1. Temporarily write the instruction `JMP *+1` into memory **at address J0—1**;
2. Execute that instruction in place; this sets **rJ ← J0**;
3. Put the original word back (so we don't alter the traced program);
4. Finally, exit the trace routine **without disturbing rJ** (so use `JSJ`, not `JMP`). Knuth's answer notes this last point explicitly: using `JSJ` is crucial.

Patch (drop-in replacement for the leave block)

Below is a compact, self-contained replacement for the "leave" part of the trace routine. It assumes your routine already saved the original J in `JREG` (as in the text), and it uses I1 and X temporarily but restores them.

```
* --- NEW LEAVE SEQUENCE (restores rJ) -------------------------
XSV    ORIG  *+1          * temp: save X
I1SV   ORIG  *+1          * temp: save I1
ONE    JMP   *+1          * literal word used as a copier: "JMP *+1"

LEAVE  STX   XSV          * save rX; we'll need it for the exit jump later
       ST1   I1SV         * save rI1; we'll use I1 as a pointer
       LD1   JREG(0:2)    * I1 := saved start address J0
       LDA   -1,1         * A  := original contents at address J0-1
       LDX   ONE          * X  := the word "JMP *+1"
       STX   -1,1         * write that jump at address J0-1
       JMP   -1,1         * execute it → rJ := (J0-1)+1 = J0

* we're now back here, with rJ restored to J0
       STA   -1,1         * restore the original word at J0-1
       LD1   I1SV         * restore rI1
       LDX   XSV          * restore rX
```

```
       LDA   AREG         * restore rA (as in the book)
LEAVEX JSJ   *            * exit without changing rJ
```

Why each step is safe

- **No persistent self-modification:** we restore the overwritten word at `J0-1` before leaving.
- **Minimal register disturbance:** only rA, rX, rI1 are touched temporarily, and all three are restored; the comparison indicator and overflow toggle remain unaffected (matching the section's design goals).
- **Correct exit:** `JSJ` leaves rJ as we've just set it to J0; using `JMP` here would clobber rJ again.

Where this fits in the printed routine

You replace the printed **lines that restore rA and jump out** (the `LEAVE … / LEAVEX …` pair near the end) with the block above. Those two lines, as printed, don't fix rJ; the patch does.

2 [26]

Design

- A 100-word circular buffer at label `BUF` collects ten 10-word records; when full, we `OUT` it to tape 0.
- We keep an index pointer in rI1 that advances by 10 after each record. To persist the pointer across iterations, we use a neat MIX trick: self-modify the address field of the instruction `PTR ENT1 -100`, i.e., store the current rI1 back into `PTR(0:2)` each time; on the next pass, `ENT1` reloads the saved pointer.
- We never disturb the traced program's state (except briefly using rA and rI1 inside the tracer); rA is restored before the traced instruction executes, as in the original routine.
- We wait for the device when needed and flush the final partial buffer at the end (the final flush routine is a small add-on you call once after tracing stops).

Patch (delta against the routine on pp. 212–213)

The comments say exactly where to splice these lines relative to the original labels/lines.

```
* --- Reserve a 100-word buffer and constants (can be placed near the end) ---
        ORIG    *+99            * reserve 100 words below BUF
BUF     CON     0               * BUF-99 .. BUF are the 100-word block
PTR     ENT1    -100            * rI1 := saved pointer (will be self-modified)
```

```
EIGHT   CON     8

I1SAVE  CON     0               * local save for rI1

* --- At ENTER: after lines 02-04 (save tape-pointer register) ---
        ST1     I1SAVE          * save caller's rI1; tracer will use rI1

* --- After line 05-07 (after CYCLE STA PREG(0:2)) ---
        JBUS    *(0)            * ensure tape 0 not busy from a prior OUT
        STA     BUF+1,1(0:2)    * Word1 (0:2): location (A holds it here)
        ST1     I1REG           * (if you keep a global I1REG like the book)

* --- After lines 08-11 (after fetching the instruction into INST) ---
        STA     BUF+2,1         * Word2: instruction word

* --- Still before the traced instruction executes: dump "before" registers ---
        LDA     AREG            * Word3: A (before execution)
        STA     BUF+3,1

        LDA     I1SAVE          * Word4: I1 (before) - we changed rI1
        STA     BUF+4,1
        ST2     BUF+5,1         * Words5-9: I2..I6 (unchanged by tracer)
        ST3     BUF+6,1
        ST4     BUF+7,1
        ST5     BUF+8,1
        ST6     BUF+9,1

        STX     BUF+10,1        * Word10: X (before)
        LDA     JREG(0:2)
        STA     BUF+1,1(4:5)    * Word1 (4:5): J (before)

* --- Pack CI (2/1/0) + 8*OVF into Word1 (3:3) ---
        ENTA    0               * start with 0 (= "<")
        JL      CI_DONE         * if "<", keep 0
        JE      CI_EQ
        ENTA    2               * ">" → 2
        JMP     CI_OV
CI_EQ   ENTA    1               * "=" → 1
CI_OV   JNOV    CI_NOOV         * if overflow off, skip add
        ADD     EIGHT
CI_NOOV STA     BUF+1,1(3:3)
CI_DONE NOP
```

```
* --- Advance pointer and flush when block is full ---
        INC1    10              * next record slot within the 100-word block
        J1N     KEEP_FILLING    * still negative → not yet full
        OUT     BUF-99(0)       * write the 100-word block to tape unit 0
KEEP_FILLING
        ST1     PTR(0:2)        * persist pointer by self-modifying ENT1
        LD1     I1SAVE          * restore caller's rI1 before continuing

* (the rest of the original routine continues: execute INST, etc.)
```

Notes and rationale:

- **Where values come from.** Right after `CYCLE STA PREG(0:2)` the A-register holds the next instruction's location, so we can store Word 1's (0:2) immediately. After `PREG LDA * / STA INST` we have the instruction word (Word 2). `AREG`, `JREG`, and `I1..I6` are the tracer's saved copies of the machine registers before the step; we read them *before* executing the instruction so the "before" requirement is met.
- **CI/OV packing.** We derive the comparison indicator via branches (`JL/JE`) and add 8 iff overflow was set, then store that byte into field (3:3) of Word 1, exactly as specified by the exercise.
- **Buffering & I/O.** The `ORIG *+99` trick makes `BUF-99 .. BUF` a contiguous 100-word area. We fill records using indexed addresses `BUF+… ,1`, starting with rI1 = −100; after ten records (`INC1 10` ten times) rI1 becomes 0 and we `OUT BUF-99(0)` to tape 0. We also issue `JBUS *(0)` up front to avoid overlapping I/O. A one-time "final flush & rewind" routine should be called after tracing stops to write a partial block and rewind unit 0, as suggested in the official notes.

Why this is correct

- Every required field is captured **before** the traced instruction executes (we only restore rA just before execution, per the original routine's discipline).
- The 100-word block layout matches the exercise's request for "nine more ten-word groups in the same format," enabling efficient tape output with minimal space in core.
- We preserve the traced program's state: only rA and rI1 are touched inside the tracer and then restored; CI and OV are left unchanged (except for the unavoidable tests), as the text advises.

3 [10]

Why tape > printer for a trace routine

1. **Throughput and latency** Line printers are slow, mechanical devices. Knuth's own timing note for printing a line is $T = 7500u$; if you print after (nearly) every instruction, output time dominates and the trace becomes painfully slow. Tape is block-oriented and written in large chunks, so per-word overhead is vastly lower; you can buffer many words and write them as a single I/O, reducing device start/stop costs and the number of waits. Knuth also emphasizes that, for trace programs, "the running time is consumed by output anyway," so minimizing per-step output costs is crucial—tape helps do exactly that.
2. **Buffering and structure** Exercise 2 explicitly formats the trace as fixed ten-word groups packed into 100-word tape blocks. That's ideal for efficient sequential I/O and easy post-processing. Printing would force you to convert binary register contents into text and paginate—more CPU time and more I/O operations.
3. **Non-interference with the program under test** A traced program "must not use the output device on which tracing information is being recorded." If you occupy the printer for tracing, the program can't print its own output. Using tape unit 0 for the trace keeps the printer free for the target program (or human-readable summaries later).
4. **Post-processing flexibility** A tape log can be rewound, scanned, filtered, summarized, or reformatted into human-readable reports after the run (e.g., reconstructing flow, computing instruction frequencies, or converting only selected steps to text). With direct printing you pay the formatting cost up front and flood paper with data you may not actually need.
5. **Lower perturbation of behavior** Because tape output is faster and can be buffered, it slows the traced program less than constant printing—giving a trace that is closer to the program's "natural" behavior (though still slower, as the text notes).

In short: **tape is faster, buffer-friendly, nonintrusive, and analyzable offline**, whereas printing is slow, formatting-heavy, and monopolizes a device the program might also want to use.

4 [25]

Here's what happens if you make the trace routine trace itself by inserting, just before label `ENTER`, the two instructions

- `ENTX LEAVEX` (set rX to the address of `LEAVEX`), then
- `JMP **+1` (jump into `ENTER`),

as the exercise proposes. (This is the routine on pp. 212–214.)

What the routine expects at entry

The trace routine is designed to be entered with rJ already set to the **starting address to trace** and rX set to the **address where tracing should stop**. The code then saves rA and rJ, takes the starting location from rJ, and begins its fetch–decode–execute cycle on the "external" program's instruction at that location.

Step-by-step with your two extra instructions

1. **Prelude executes.** `ENTX LEAVEX` makes rX = address(`LEAVEX`); `JMP **+1` transfers control to `ENTER`. But a `JMP` also **loads rJ with the address of the `JMP` itself** (the prelude location). Thus, when the trace routine starts, its idea of the "start address" (from rJ) is the prelude `JMP`, not the first instruction of the routine proper. The routine saves rA and rJ, then sets its "next instruction" pointer to that `JMP`.

2. **First traced instruction is that prelude `JMP **+1`.** The tracer recognizes a jump and rewrites the instruction word it will execute so that execution flows through its `JUMP` handler, which updates the simulated J-register and sets the next location to `ENTER`. (That is the normal way this tracer executes an external jump.)

3. **Now it begins tracing the trace routine's own body.** It will fetch and execute, under trace, the instructions beginning at `ENTER` (e.g., `STX TEST(0:2)`, `STA AREG`, …). So far, so good.

4. **Self-destruction on `STJ`.** Very soon the tracer reaches the routine's own `STJ JREG` (the instruction that saves rJ early in the routine). The tracer **deliberately rewrites any external `STJ` into `STA`** before executing it, so the external program can't disturb its simulated J-register. But now the "external program" *is the tracer itself*, so the tracer **changes its own `STJ JREG` into `STA JREG` in memory** and then executes that store-A. This destroys the saved J value (`JREG` now holds whatever was in rA), and the code itself has been permanently modified.

5. **Loss of the starting location and derailment.** Immediately afterward the tracer does `LDA JREG(0:2)` to get the next instruction location from `JREG`. Since `JREG` no longer contains a proper address (it was just overwritten with A), the tracer loads a garbage address, points its "next instruction" there, and proceeds to fetch/execute nonsense. From this point the trace prints nonsense or loops/crashes; it certainly doesn't produce a coherent self-trace.

With only those two added instructions, **the given trace routine cannot trace itself**. It wrecks its own code/data because its "don't let the external program alter J" trick (rewriting `STJ` to `STA`) applies to the tracer's **own** `STJ`, corrupting `JREG` and sending execution off the rails.

This is exactly why the text restricts traced programs so they **don't store into the trace routine's locations**, and why a robust "self-trace" needs extra machinery (e.g., two copies tracing each other or special guards), as hinted by the following exercises.

5 [28]

What actually happens

1. **The tracer that you start keeps control.** Suppose we start tracer **T** with `J` pointing at the entry of tracer **T** . T runs its own code, repeatedly:

- fetches the current instruction *from T 's memory* at address `PREG` ,
- copies it into **T** 's `INST`, possibly edits it,
- executes that copy at **T** 's `INST`,
- increments **T** 's `PREG` to the next address.

Crucially, **the instruction that actually executes is always the copy in T 's `INST`, not the one in T** . So T never "takes over."

2. **T "tries" to trace T , but only as data.** Because T 's *code* is what T is stepping through, T will execute its own statements like "store the next address in `PREG` ," "read the instruction from T into `INST` ," etc. But when the time comes to "execute the external instruction," the execution point is **T** 's `INST` (by design of the trace routine), not T 's `INST`. Hence T 's attempt to trace T never actually runs any of T 's instructions; it merely manipulates T 's bookkeeping cells (`PREG` , `INST` , `AREG` , …). This respects the routine's restriction that the traced program mustn't clobber the tracer's own working cells, because each tracer uses *its own* `PREG`, `JREG`, `AREG`, etc. (the "(a)–(c)" restrictions listed under the exercises).

3. **No infinite regress, no mutual recursion of control.** When T encounters a *jump* in T 's code, it rewrites the **copied** instruction so the jump targets **T** 's `JUMP` label; control therefore stays inside T , which then updates the simulated `J` and continues. T never gains control flow; it remains merely the *subject* being stepped.

Net effect

- **T simply traces T as if T were an ordinary program.** You get a trace of T 's instructions executing; T 's internal attempt to trace T never actually executes T 's instructions.
- **No paradox or endless slowdown chain** occurs here (contrast with the self-trace thought experiment of exercise 4, where self-modification can immediately derail the trace via `PREG` being clobbered).
- If both copies tried to *print* tracing information to the same device, that would violate restriction (6b) and garble output, but the skeleton routine shown doesn't output registers at all (that part was explicitly omitted).

Conclusion. With two copies placed in disjoint memory and each set to trace the other, the one you start (T) retains control and cleanly steps through the other (T). There's no infinite

regress; it behaves just like tracing any other (nontrivial) program, only slower because that program happens to be a tracer itself.

6 [40]

Here's a clean way to do **1.4.3.2 — Exercise 6** ("write a trace routine that is capable of tracing itself … at slower and slower speed, ad infinitum, until memory is exceeded"), built directly on Knuth's trace routine in §1.4.3.2. I'll (1) recall how the given trace routine works, (2) show the minimal *self-bootstrapping* change that makes it trace itself once (the idea of Ex. 4), then (3) turn that into an **infinitely nestable, re-entrant** trace by giving it tiny per-level "frames," so multiple copies of the trace can be active at once without stepping on each other's working cells, and (4) point to where to splice in the "print one step" action. Citations refer to the exact listing and the exercise page in the book PDF you linked.

1) What the provided trace routine already does

Knuth's routine is invoked when the *driver* jumps to label `ENTER` **with**:

- `rJ` = start address of the code to trace,
- `rX` = address where tracing must stop (exclusive).

On entry it saves `rA` and `rJ`, loads the *simulated* program counter from `rJ`, fetches the next instruction word, rewrites certain instructions (e.g., jumps) so the monitor keeps control, executes the instruction via the pseudo-op line `INST NOP *`, advances to the next location, and repeats; special logic handles jumps and `JSJ` so the simulated `J` is correct. Finally it restores `rA` and exits at `LEAVE/LEAVEX`. (See lines 02–08 for setup, 10–16 decode, 27–47 jump handling, 34/36–38 execute-and-advance, 48–49 leave.)

Knuth explicitly notes this listing is *just the control core* of a trace: it doesn't print; printing/register-dump is left to exercises.

2) Make it trace itself once (the Ex. 4 idea you build on)

Place a tiny prologue *immediately before* label `ENTER`:

```
SELF   ENTX  LEAVEX      ; inner trace stops at our LEAVEX
       JSJ   ENTER       ; start a fresh trace instance, sets rJ to SELF+1
ENTER  ...               ; (the original routine follows)
```

Why this works (and why we use **JSJ**, not `JMP`):

- When the outer monitor first "executes" `JSJ ENTER` as an instruction of the traced program, its jump logic recognizes a jump and treats **JSJ** specially: it does **not** overwrite the simulated `J` with a "fall-through" value (lines 39–47 check for JSJ and avoid updating `JREG` in that case), then sets the simulated PC to `ENTER`. Thus the *inner* monitor instance starts with `rJ = SELF+1` and begins tracing **the trace program itself** starting at `ENTER`. From now on, the outer level prints every step the inner monitor takes, and the inner monitor will (once we add printing) print every step of its own traced program.

Knuth asks you to think about the simpler version with `ENTX LEAVEX; JMP *+1` in Ex. 4; swapping the second instruction to `JSJ ENTER` is the right subroutine-style call that seeds the *next* level with a proper `J`. The exercise statement for 6 explicitly says "in the sense of exercise 4," i.e., the same bootstrapping trick but continuing indefinitely.

If you stop here, you can get one active monitor tracing itself, but as soon as that inner instance tries to start *another* instance, all copies will fight over the same scratch words (`AREG`, `JREG`, `PREG`, `INST`, `INST1`, `TEST`, …). We need re-entrancy.

3) Make the trace **re-entrant** with per-level frames (the key to "ad infinitum")

The original routine uses fixed global cells (labels like `AREG`, `JREG`, `PREG`, `INST`, `INST1`, `TEST`, `LEAVEX`). To support unlimited nesting, give each active "level" its own copy of those words—i.e., a *frame*—and index all accesses via, say, `rI1` as a frame pointer.

3.1 Define the frame layout (somewhere near the routine)

```
; One frame per nesting level (10 words is enough for the listed cells)
FRAME_SZ  EQU  10
FR_AREG   EQU  0     ; saved rA
FR_JREG   EQU  1     ; saved rJ (simulated)
FR_PREG   EQU  2     ; simulated PC (location being traced)
FR_INST   EQU  3
FR_INST1  EQU  4
FR_TEST   EQU  5
FR_LEAVEX EQU  6     ; we keep its absolute addr here for this level
; ... a couple spare words
```

Reserve a small stack of frames (e.g., 100 levels):

```
FRAMES    ORIG *+100*FRAME_SZ
```

Maintain `rI1` = base of **current** frame. On each entry, bump it to the next frame; on each leave, drop it.

3.2 Patch the routine to index frame fields by `,1`

Example patches (showing the idea; you'd apply this uniformly):

- Save/restore `rA`:
  - old: `STA AREG` / `LDA AREG`
  - new: `STA FR_AREG,1` / `LDA FR_AREG,1`
- Save `rJ` and use it later:
  - old: `STJ JREG` / `LDA JREG(0:2)` etc.
  - new: `STJ FR_JREG,1` / `LDA FR_JREG,1(0:2)`
- Keep the "next instruction address":
  - old: `STA PREG(0:2)` / `LDA PREG(0:2)`
  - new: `STA FR_PREG,1(0:2)` / `LDA FR_PREG,1(0:2)`
- `INST` and `INST1` similarly become `FR_INST,1` and `FR_INST1,1`.
- The exit address:
  - On entry, copy `rX` into `FR_LEAVEX,1(0:2)` and test against that field instead of a single global `TEST/LEAVEX`.

Concretely, the very first lines change to (comments show what moved):

```
SELF      ENTX  FR_LEAVEX,1(0:2)   ; stop at this level's LEAVEX
          JSJ   ENTER

ENTER     ENTA  FRAME_SZ
          ENT2  FRAMES             ; rI2 = base of all frames
          ADD   LEVEL              ; LEVEL holds current depth (init 0)
          MUL   =1=                ; (cheap way to scale by FRAME_SZ if needed)
          ADD   rI2                ; compute next frame base
          ST1   LEVEL              ; bump depth counter
          LDX   =0=
          STX   FR_AREG,1          ; clear a few words if desired

          STX   FR_TEST,1(0:2)     ; put stop addr from rX
          STX   FR_LEAVEX,1(0:2)
          STA   FR_AREG,1
          STJ   FR_JREG,1
          LDA   FR_JREG,1(0:2)     ; start location for tracing
CYCLE     STA   FR_PREG,1(0:2)
...
```

```
TEST      DECA  *                   ; compare FR_PREG vs FR_LEAVEX for exit
          JAZ   LEAVE
PREG      LDA   *                   ; fetch instr at FR_PREG
          STA   FR_INST,1
...
```

(You don't need to *literally* copy/paste the above; the point is every variable access acquires a `,1` and the constant cells become *offsets within the current frame.*)

3.3 Leave drops the frame and returns

At `LEAVE`:

```
LEAVE     LDA  FR_AREG,1
          ; rI1 := rI1 - FRAME_SZ (restore previous level's frame)
          ENTA  -FRAME_SZ
          ADD   rI1
          ENT1  0,0                ; move pointer
          ; also decrement LEVEL (optional book-keeping)
          LEAVEX JMP  *            ; same "stop tracing"
```

The only global state left is the code itself and the tiny depth/stack base variables; all per-step scratch lives in that level's frame.

With the routine re-entrant, executing the two-instruction `SELF` prologue **starts a fresh instance that uses its own frame**, while the outer instance continues stepping and printing. When the inner instance reaches its own `SELF`, it starts yet another one, and so on—until your `FRAMES` area runs out, which is exactly the "until memory capacity is exceeded" stopping condition the exercise asks for.

4) Where to "print one step"

Knuth's core doesn't print; he says you must add that (e.g., Ex. 2 writes a 10-word snapshot per step to tape 0). The natural place to emit output for *the current step* is **after** you've decoded the instruction and before executing it (i.e., after lines 10–16 in the listing), or immediately after execution (line 35). In a short solution, printing just the location and the instruction word is fine:

- Just before `INST NOP *` (line 34), do:

```
          LDA  FR_PREG,1(0:2)
          STA  BUF                     ; word 1: location
          LDA  FR_INST,1
          STA  BUF+1                   ; word 2: instr word
          OUT  BUF(18)                 ; print one short line on device 18
```

Replace `BUF` with a tiny two-word buffer you've put near the routine. (If you already solved Ex. 2 and wrote the richer 10-word record to tape unit 0, call that writer here instead.)

5) Why the speeds keep dropping

- Level 0 (outermost) prints **every** step the program under trace performs.
- When the under-trace *program is itself a trace*, that inner trace prints too—so each inner step now costs "execute the step" + "inner print" + "outer print".
- The inner trace's own under-trace is *another* trace, etc. Each level adds another layer of output and control logic around a single MIX instruction; hence the geometric slowdown the problem statement describes.

6) Driver to start the cascade

A tiny starter sets `rJ` to `SELF` of your trace and `rX` to its `LEAVEX`, then jumps to `ENTER`. (In MIX, you can seed `rJ` by entering via `JSJ SELF` and arranging that `SELF` immediately "calls" `ENTER` as in the prologue above, so the first inner instance starts correctly; your outermost instance is started by your hand—e.g., from a small harness.)

7) Safety/notes

- Knuth lists three simple restrictions on the traced program (don't store into the trace's cells; don't use the output device the trace uses; expect slowdown). Using frames fixes the first; just be sure your traced program doesn't `OUT` to the same unit you're using.
- His jump-handling already special-cases `JSJ`, which is crucial for the self-bootstrapping prologue to work.

7 [25]

1) What the exercise asks

Design a **jump trace** that emits far less output than a full trace. Instead of dumping registers every step, it should output a sequence of pairs $(x_1, y_1), (x_2, y_2), \ldots$ meaning "the program jumped from location x_i to y_i, then ran straight-line code from y_i through x_{i+1}, then jumped from x_{i+1} to y_{i+1}, …". From such pairs, a post-pass can reconstruct control flow and count how often each instruction executed.

2) Where to hook into the given trace routine

Knuth's trace routine already detects jumps and centralizes their handling at label `JUMP`. The fetch/decode loop keeps the **fetch address** in `PREG(0:2)` (the "location of the next instruction"), and when a jump occurs execution transfers to `JUMP`, where the **target address** (effective address of the jump) is computed into register **A** just before control returns to `CYCLE`

(which then loads that A-value into `PREG` for the next iteration). This is the perfect point to record our pair x, y.

Concretely (from the text routine):

- `CYCLE` stores the fetch location into `PREG(0:2)`, fetches the instruction at that address, and identifies jumps.
- When a jump is seen, code branches to `JUMP`. There the routine finalizes jump semantics (special-cases `JSJ`, updates `JREG` when appropriate), then does `INST1 ENTA *` so **A = target address**; finally it `JMP CYCLE` to continue from the jump target.

Therefore, we can log:

- x = the address of the **jump instruction** = `PREG(0:2)` at the time the jump is recognized,
- y = the **jump target** = the value placed into **A** at label `JUMP` just before `JMP CYCLE`.

3) Minimal recording logic (code sketch)

Goals: (a) **very low overhead** per jump, (b) **no register clobbering** visible to the traced program, (c) **buffered** output (pairs take two words), (d) output to a device **not used by the program being traced** (e.g., tape unit 0).

We'll modify the block at `JUMP` (after it has computed the target in A), insert a small snippet to **append (x, y)** to a memory buffer, and flush the buffer to tape when full.

Data (reserved somewhere outside the traced program's memory):

```
BUF     CON  0              * start of a 100-word block
...     (reserve 100 words total for one tape block)
END     EQU  BUF+100        * end of block
PTR     CON  BUF            * current write position (absolute address)
I1REG   CON  0              * saved I1
YHOLD   CON  0              * temp to hold y
ONE     CON  1
TWO     CON  2
```

At routine entry (ENTER): save I1 once (we'll use I1 as an internal pointer) and ensure tape 0 is idle.

```
ST1   I1REG
JBUS  *(0)                  * wait until tape 0 is not busy
STA   AREG                  * (this is already in Knuth's code)
STJ   JREG                  * (already)
...
```

("Use minimal registers" is an ideal; for practical, simple buffering we momentarily use I1 and restore it on exit. The original trace avoided using registers other than A; here we use I1 carefully and restore it.)

At label `JUMP` (after lines that update `JREG` and just after `INST1 ENTA *` puts the target into A):

```
// Existing code:
JUMP    LDA   8B(4:5)       * as in the book
        SUB   INST(4:5)     * detect JSJ, etc.
        JAZ   *+4
        LDA   PREG(0:2)
        INCA  1
        STA   JREG(0:2)
INST1   ENTA  *             * <= A now holds jump target "y"

// INSERT begin: record (x,y) to [PTR], [PTR+1]

        STA   YHOLD         * save y (target) temporarily
        LDA   PREG(0:2)     * load x = address of jump instr
        LD1   PTR           * I1 := current dest address
        STA   0,1           * store x at M[PTR]
        LDA   YHOLD         * restore y
        STA   1,1           * store y at M[PTR+1]
        ENT1  2,1           * I1 := I1 + 2
        ST1   PTR           * PTR := PTR + 2

        * If buffer full, flush to tape 0 and reset PTR:
        CMP1  =END=         * compare I1 with END
        J1N   TRACE_CONT    * not full yet
        * flush:
        OUT   BUF,0         * start writing 100 words to tape unit 0
        JBUS  *(0)          * wait completion (conservative)
        ENT1  BUF           * I1 = BUF
        ST1   PTR           * PTR := BUF
TRACE_CONT:

// INSERT end

        JMP   CYCLE         * existing: continue at new address
```

Remarks:

- We **only** touch I1, and we save/restore it at the beginning/end of the whole trace; the traced program never observes a changed I1 because the trace wrapper owns it during tracing. (If you want to be extra strict, you can re-save/re-restore I1 inside `JUMP` too, but it's unnecessary if the trace routine always retains control.)
- The snippet is constant-time per jump: a handful of `STA/LDA` plus one indexed `STA` pair and a simple two-word pointer bump.
- On **exit** from tracing (`LEAVE/LEAVEX`), flush any **partial** block (if `PTR != BUF`) and then `LD1 I1REG` to restore I1, just like we restored A and J in the base routine. (Knuth already notes J needs restoring; exercise 1 in this section has you fix that; do both.)

4) Buffering/output strategy and why tape

- **Why tape (unit 0)?** It's far faster and far less intrusive than printing lines; the book explicitly hints that tape is preferable to printing for trace output.
- **Asynchronous I/O:** We write a full 100-word block when the buffer fills (50 jump pairs). In typical code, the number of jumps is far less than the number of executed instructions, so output volume and I/O frequency are much smaller than with a full step trace.
- **Exclusivity:** The traced program must not use the same device; Knuth calls this out (don't use the output device being used for the trace) — so dedicate tape 0 to the jump trace, and choose another device for the program (or vice versa).

5) Notes, correctness, and limitations

- **What counts as a "jump"?** The existing trace detects all jump opcodes (`JMP`, `Jx`, `Jxx`, `JSJ`) and also treats `JBUS` specially (since it can effect control transfer). We piggyback on that detection; our code runs **only** when a jump actually occurs.
- **Source and target correctness:** At jump time, `PREG(0:2)` is the address of the jump instruction being executed (our x); after `INST1 ENTA *`, A holds the effective destination (our y). That aligns exactly with the exercise's notion of pairs.
- **Post-pass reconstruction:** A reader can process the pair stream `(x ,y ), (x ,y ), …` and:
  - mark **basic blocks** as $[y_i, x_{i+1}]$,
  - accumulate execution counts by adding the length of each straight-line interval,
  - reconstruct the **control-flow graph** edges from each $x_i \to y_i$.
- **Self-modifying code caveat:** The book notes that self-modifying code can defeat a jump-trace profiler; e.g., if an instruction changes another instruction into or out of a jump, the pair stream may no longer reflect the original static flow. Keep this limitation in mind.
- **Register hygiene:** We intentionally restrict ourselves to A and a temporary use of I1 (saved/restored), in keeping with the trace routine's philosophy of not disturbing the program state.

By inserting ~15 tight instructions at `JUMP`, storing $(x, y) = (\mathrm{PREG}(0{:}2), \mathrm{A})$ to a buffered block and flushing to tape, you get an **efficient jump trace** with tiny runtime overhead and dramatically reduced output size, while preserving the traced program's state. This meets the exercise's specification and leverages the exact structure of the trace routine given in §1.4.3.2.

1.4.4 Input and Output

1 [05]

Context you need

On MIX, `IN` **initiates** an input transfer and returns immediately; the program may continue executing, but it must not rely on the input buffer's contents until the device is no longer busy (e.g., until `JBUS` shows "ready"). Sequence (3) relies on that rule: it waits with `JBUS`, then copies from the just-filled buffer, and finally starts the next input into that same buffer.

(a) Putting `MOVE`s **before** `JBUS`

Incorrect. If you copy from locations `2000-2099` *before* `JBUS` has confirmed completion, you may be moving words that haven't arrived yet (or are only partially written). The model explicitly warns that input completes only after a subsequent I/O op or a successful "not busy" test; otherwise the buffer contents are not guaranteed. So the copied block could be incomplete/corrupted.

Verdict: No. The input may not yet be complete.

(b) Putting `MOVE`s **after** `IN`

Also incorrect (race hazard). If you start the *next* `IN 2000(5)` first and only then perform `MOVE 2000(50); MOVE 2050(50)`, you are copying from a buffer that the device is simultaneously **overwriting** with the next block. If the input device outpaces the CPU even slightly, some words you copy will belong to the *next* block rather than the previous one, yielding a mixed/corrupted result.

Verdict: No. The device might outrun the `MOVE`, so the copy can be contaminated by the new transfer.

Final answers

(a) No. You might move data before the current input has finished. **(b) No.** Starting `IN` first risks the device overwriting the buffer while you copy, so the `MOVE` can pick up the wrong data.

2 [10]

Here's a buffered-**output** scheme that mirrors Knuth's buffered-**input** setup (his items (2) and (3)) but for writing tape unit 6. We'll stage each 100-word block for output in an auxiliary buffer at addresses 2000–2099, while your program prepares the next 100-word block at 1000–1099. This overlaps CPU work with device I/O, just as his input pipeline does. The exercise statement appears at the end of §1.4.4; Knuth's earlier unbuffered pair for output is `OUT 1000(6); JBUS *(6)` and his input buffering sequences (2)–(3) are the model we're copying.

Setup and notation (MIX reminders)

- A tape **OUT A(6)** starts writing 100 words beginning at A to unit 6.
- **JBUS *(6)** waits until unit 6 is no longer busy.
- **MOVE A(50)** copies 50 words starting at A into the destination pointed to by **rI1**, then advances rI1 by 50 (so two MOVE's copy 100 words). Knuth's input example sets rI1 to the destination just before the two MOVE's.

Buffered output: two phases

1) Prime the pipeline (first block)

Before the device can "trail" your computation, you must place the first block into the staging buffer and start the first write:

```
ENT1 2000          ; rI1 ← 2000 (destination = buffer)
MOVE 1000(50)      ; (1000-1049) → (2000-2049)
MOVE 1050(50)      ; (1050-1099) → (2050-2099)
OUT  2000(6)       ; start writing the staged block to tape unit 6
```

Explanation: you've copied the current block (prepared at 1000–1099) into 2000–2099 and kicked off the OUT. While the device is busy writing that buffer, your program is now free to compute/fill the **next** 100-word block at 1000–1099. (This mirrors Knuth's "read-ahead" input, but here it's "write-behind": the device is finishing the **previous** buffer while the CPU prepares the next one.)

2) Steady-state cycle (each subsequent block)

Whenever the next 100-word block at 1000–1099 is ready to be sent:

```
ENT1 2000          ; rI1 ← 2000 (dest = buffer), harmless if repeated
JBUS *(6)          ; wait until the prior OUT from 2000-2099 has finished
MOVE 1000(50)      ; (1000-1049) → (2000-2049)
MOVE 1050(50)      ; (1050-1099) → (2050-2099)
OUT  2000(6)       ; start writing this staged block
```

Why this order? You **must** not overwrite 2000–2099 while the device is still taking data from it; therefore the **JBUS** precedes the **MOVE** instructions, just as in Knuth's input sequence (3) the JBUS precedes the MOVE to ensure the fresh input has arrived before moving it out of the staging area. This is the exact analog of his (2)–(3), but with roles reversed.

Finishing up

After you've initiated the **final** `OUT 2000(6)`, wait for the device to complete before you stop or reuse the buffer:

```
JBUS *(6)          ; ensure the last block has been fully written
```

(Stopping early risks data loss; the same caveat applies to buffered input and is discussed around Knuth's examples.)

Why it works

- **Safety.** The `JBUS *(6)` before each pair of `MOVE`s prevents clobbering the very buffer the device is still using, exactly like waiting for input completion before moving data out of the input buffer in (3).
- **Overlap.** While the device streams the **previous** block from 2000–2099, your program is free to prepare the **next** block at 1000–1099, so computation and I/O overlap. This is the write-side counterpart of Knuth's "buffering" idea that overlaps I/O with computation.

3 [22]

Design notes

- Two 100-word buffers live in memory, each followed by a single "link" word that holds the address of the other buffer. This is exactly analogous to the input routine in the text (which links its two buffers in lines 07, 11, 14 of (4)).
- We keep the **base address** of the current output buffer in `CUR`.
- Index register **I5** holds the **current position** in the buffer as a *negative count*: it starts at -100 and is incremented each word; when it hits 0 the buffer is full. This lets us test fullness with a single `J5N` (jump if negative).
- When a buffer fills, we self-modify the address field of a single `OUT` instruction to the "old" buffer's base, execute it, then swap `CUR` to the other buffer and reset `I5` to -100. (This is the same self-modifying trick used for subroutine linkage and in many MIX examples; see the standard `STJ EXIT … EXIT JMP *` pattern.)
- No `JBUS` is needed *during steady-state operation* (same reason as for input (4)): we start the next I/O before we touch the previous buffer again.
- At program end we **pad with zeros** if needed, `OUT` the final partial block, and then do a single `JBUS *(V)` to wait for the last I/O to complete.

```
* ====== Constants / device selection ======
V     EQU 1              * Tape unit to use (pick any valid unit id)
ZERO  EQU 0

* ====== Buffer areas (100 words each) + one link word ======
OUTBUF1 ORIG *+100       * 100 data words
LINK1   CON OUTBUF2      * address of the other buffer

OUTBUF2 ORIG *+100
LINK2   CON OUTBUF1

* ====== State ======
CUR    CON OUTBUF1       * base address of current buffer

* ====== Subroutine: WORDOUT ================================
* Calling sequence:  JMP WORDOUT
* Entry:  rA = next word to output
* Uses:   rI5 = position in current buffer (negative count), CUR = base
* Exit:   word stored; if buffer became full, a 100-word OUT is started
* Clobbers: rA (temporarily), rJ (by standard linkage), CI
*
WORDOUT STJ 9F                 * standard MIX subroutine linkage
        STA CUR+100,5          * store rA at CUR + (100 + rI5)  [slots 0..99]
        INC5 1                 * advance position
        J5N  9F                * if still negative, buffer not full => return

* -- Buffer just filled: start OUT on the full buffer, swap to the other
*    Save OLD base into OUTCMD's address field, then switch CUR to link.
        LDA CUR
        STA OUTCMD+1(0:2)      * self-modify OUT address field
        LDA CUR+100            * fetch link (address of other buffer)
        STA CUR                * CUR := other buffer base
        ENT5 -100              * reset position for the new buffer

OUTCMD  OUT OUTBUF1(V)         * OUT 100 words starting at (modified) address
9H      JMP *                  * return to caller
* ============================================================

* ====== Subroutine: FLUSHOUT ================================
* Pads current (possibly partial) buffer with zeros, OUTs it, and waits
* for the final I/O to finish. Safe to call even if buffer is exactly empty/full.
*
```

```
FLUSHOUT STJ 9F

        J5Z  FINISHWAIT        * rI5==0 => a full block was just started; just wait
* Pad with zeros until rI5 hits 0, then OUT the partial block.
PADZ    STZ CUR+100,5
        INC5 1
        J5N PADZ

        LDA CUR
        STA OUTCMD+1(0:2)      * OUT the now-full padded buffer
        OUTCMD OUT OUTBUF1(V)

FINISHWAIT
        JBUS *(V)              * wait until last OUT completes
        JMP 9F
* ============================================================

* ====== Program start (minimum required initialization) =====
*   - Choose initial buffer and set position to start of buffer.
*   - (Buffers' link words are assembled above; no runtime init needed.)
START   ENT5 -100              * I5 = -100  (next store goes to slot 0)
        LDA =OUTBUF1=
        STA CUR                * CUR := OUTBUF1
* ... your program computes, and whenever it has a word to output:
*     LDA <word>
*     JMP WORDOUT
* ... repeat as needed ...

* ====== Program end (must flush final block) =====
DONE    JMP FLUSHOUT
        HLT
        END START
```

Why this works

- **"Buffer-swapping ... analogous to (4)"**: Two buffers linked to each other; when one fills we `OUT` it and immediately continue filling the other, with no busy-wait in the steady state.
- **"Index register 5 ... current buffer position"**: `rI5` is the running position (negative count). We store at `CUR+100,5` and detect a full buffer with a single `J5N/rI5==0` test.
- **"Write 100 words onto tape unit V"**: The `OUT` instruction is self-modified to the *old* buffer base so we write exactly that 100-word block on device `V`.

- **"Layout of buffer areas"**: As shown—each 100-word area followed by a 1-word **link** holding the other buffer's base (same linking idea as the text's WORDIN).
- **"Beginning and end ... first and last blocks properly written"**:
  - **Beginning**: set `CUR` to your first buffer and `rI5 := -100`. That's all.
  - **End**: call `FLUSHOUT`. It pads with zeros to the next 100-word boundary (if needed), `OUT`s the final block, then does a single `JBUS *(V)` to wait for completion before halting. (Padding is explicitly required.)

4 [M20]

Consider processing B blocks. For block k let

- c_k = CPU time spent computing on that block's data,
- i_k = device time for the I/O of that block (read or write).

Define totals

$$C = \sum_{k=1}^{B} c_k, \qquad I = \sum_{k=1}^{B} i_k.$$

With **unbuffered I/O** the CPU waits during each I/O, so the elapsed time is

$$T_{\text{unbuf}} = C + I.$$

With **buffered I/O** (double buffering, for example), computing on one buffer can overlap I/O on the other. There is only **one** CPU and **one** device, so at any instant at most one CPU task and at most one device task can be active.

Fundamental lower bound (why speedup 2)

In *any* schedule (buffered or not):

- You cannot finish earlier than C, because the CPU must perform C units of work.
- You cannot finish earlier than I, because the device must perform I units of I/O.

Hence every buffered schedule satisfies

$$T_{\text{buf}} \;\geq\; \max\{C, I\}.$$

Comparing to the unbuffered time $C + I$,

$$\frac{T_{\text{unbuf}}}{T_{\text{buf}}} \;\le\; \frac{C+I}{\max\{C,I\}} \;\le\; 2,$$

with equality only when $C = I$ *and* overlap is essentially perfect. Therefore **no single-device buffering scheme can yield more than a 2× speedup** relative to unbuffered execution.

Constructive schedule (why ~2× is attainable)

Assume steady per-block times $c_k = c$ and $i_k = i$ and use two buffers:

- Start I/O for block 1 (takes i).
- While that I/O runs, compute block 0's results or prepare block 2, etc. After the pipeline fills, each additional block takes $\max\{c, i\}$ time because CPU and device run in parallel.
- The exact makespan is

$$T_{\text{buf}} = i + (B-1)\max\{c,i\} + c,$$

(first and last blocks incur the pipeline fill/drain). For large B,

$$T_{\text{buf}} \approx B\max\{c,i\}.$$

Thus the asymptotic speedup is

$$\frac{T_{\text{unbuf}}}{T_{\text{buf}}} \approx \frac{B(c+i)}{B\max\{c,i\}} = \frac{c+i}{\max\{c,i\}} \le 2.$$

Best case. If $c = i$ (balanced compute and I/O), then

$$T_{\text{unbuf}} = 2Bc, \qquad T_{\text{buf}} \approx Bc \quad \text{(plus small } O(c) \text{ edge effects)},$$

so the speedup approaches 2 as B grows. This is the "favorable circumstance" where buffering nearly halves total time.

Other cases. If $c \ll i$ (I/O-bound) or $i \ll c$ (compute-bound), then $(c+i)/\max\{c,i\}$ is close to 1; buffering still helps by overlapping the smaller resource with the larger, but the improvement is strictly < 2 and can be modest.

5 [M21]

Let

- C: total time the program spends computing (CPU time) not counting any I/O wait.
- D_i: total active time needed by I/O device i (sum of the times of all blocks handled by device i), for $i = 1, \dots, n$.

Assumptions are exactly those of §1.4.4: Each device works independently once an I/O is initiated; with buffering, the CPU can run while a device is busy; multiple devices may be busy concurrently. (This is the same setting used for MIX's buffered I/O with concurrent devices.)

Two totals to compare:

- **Unbuffered baseline.** I/O is blocking; devices don't overlap with the CPU or each other.

$$T_{\text{unbuf}} = C + \sum_{i=1}^{n} D_i.$$

- **Buffered/overlapped.** With sufficient buffering, CPU and devices can run concurrently; multiple devices can be active at once. Any schedule must last at least as long as the heaviest-used resource (CPU or any one device), hence

$$T_{\text{buf}} \;\geq\; \max\!\big(C,\, D_1, \dots, D_n\big).$$

(You can view this as a simple "makespan" lower bound: each of the $n{+}1$ resources—CPU and the n devices—must process its assigned load; the overall time cannot be less than the largest load.)

General bound on speedup

Define the speedup S from buffering as

$$S \;=\; \frac{T_{\text{unbuf}}}{T_{\text{buf}}} \;\leq\; \frac{C + \sum_i D_i}{\max(C, \max_i D_i)}. \qquad (\;)$$

Since the maximum of $n{+}1$ nonnegative numbers is at least their average,

$$\max(C, D_1, \dots, D_n) \;\geq\; \frac{C + \sum_i D_i}{n+1},$$

and plugging into ( ) gives the universal bound

$$S \leq n + 1.$$

This **generalizes Exercise 4's "at most 2×" for one device** ($n = 1 \Rightarrow S \leq 2$).

Tightness (when can we reach n+1?)

The bound is tight in "balanced" cases where

$$C = D_1 = \cdots = D_n = T.$$

Then $T_{\text{unbuf}} = (n + 1)T$ and the lower bound for any overlapped schedule is T. With well-designed multiple buffering (as in §1.4.4's coroutine-based control) you can keep **all** n devices busy continuously while the CPU also computes for the full T. Thus

$$T_{\text{buf}} = T, \qquad S = \frac{(n + 1)T}{T} = n + 1.$$

This is the natural multi-device analogue of the single-device "at most two, achieved when compute time equals I/O time" result.

Intuition and scheduling sketch

- **Lower bound:** You cannot finish earlier than the longest of $C, D_1, \ldots, D_n$.
- **Achieving near the bound:** Use multiple buffers per device and the coroutine-style controller (§1.4.4) so that:
  1. Each device is kept busy end-to-end (initiate the next transfer as soon as the previous completes), and
  2. The CPU has work to do at all times (no waiting on I/O), by computing against already-filled input buffers or producing output into free buffers. This is exactly the kind of overlap Knuth's buffer-control algorithms are designed for, and the time–action charts in the section illustrate how the overlap hides I/O behind computation (and vice versa).

What the formula tells you in general

For an arbitrary workload, the **best possible** (ideal) speedup is bounded by

$$S \leq \frac{C + \sum_i D_i}{\max(C, \max_i D_i)}.$$

- If one device dominates (say $D_k \gg C$ and $D_k \gg D_j$ for $j \neq k$), then $S \approx (C + \sum_i D_i)/D_k$, which is much smaller than n+1.
- If CPU dominates ($C \gg D_i$), speedup is near 1 (little to hide).

- If loads are well balanced ($C \approx D_i$), you approach the ideal n+1.

This fully generalizes the single-device " 2× speedup" to n devices.

6 [12]

You have two symmetric choices (pick either):

```
IN   INBUF1(U)
ENT6 INBUF2+99
```

or

```
IN   INBUF2(U)
ENT6 INBUF1+99
```

Optionally precede with `IOC 0(U)` if you might need to rewind/ready the unit.

Why this is the right start (step-by-step)

Recall the subroutine (4) (WORDIN) uses rI6 as a pointer and begins with:

1. `INC6 1`
2. `LDA 0,6` and compare to `SENTINEL` If sentinel → refill this buffer: `IN -100,6(U)`, then switch buffers: `LD6 1,6` (loads rI6 from the "other-buffer" link), and loop to fetch the word.

Important details from that listing:

- Each buffer reserves 100 data words; at offset +100 sits a sentinel constant; at +101 sits a link to the other buffer.
- rI6 "addresses the last word of input" when you enter WORDIN; the subroutine immediately does `INC6 1`, then fetches.

Now follow the first call if you used the first init (`IN INBUF1(U); ENT6 INBUF2+99`):

1. Before the first call:
   - Buffer 1 has just been filled by `IN INBUF1(U)` (block #1).
   - rI6 = `INBUF2+99` (the last data word slot of Buffer 2).
2. On the first WORDIN call:
   - `INC6 1` rI6 becomes `INBUF2+100` (the sentinel slot of Buffer 2).

- `LDA 0,6` you read the sentinel, so WORDIN concludes "this buffer is exhausted."
- `IN -100,6(U)` reads the next block (#2) into Buffer 2.
- `LD6 1,6` rI6 ← contents at `INBUF2+101`, which is the address of the other buffer (i.e., `INBUF1`).
- Loop back to fetch the word: `LDA 0,6` now loads the first actual data word of Buffer 1 — exactly what you want for the very first word delivered to the program — while the tape is already busy reading the subsequent block into Buffer 2.

Net effect: the very first call returns word 0 from Buffer 1, and you're already overlapped with I/O that fills Buffer 2. From that point on, WORDIN alternates smoothly between the buffers, always keeping the device busy when possible (the "pipeline is primed"). This is exactly the behavior intended by (4).

What you must have in the data area

Your buffers must be laid out exactly as in (4):

- `INBUF1` and `INBUF2` each reserve 100 data words.
- At `INBUF1+100` and `INBUF2+100` place `SENTINEL`.
- At `INBUF1+101` put the address of `INBUF2`, and at `INBUF2+101` put the address of `INBUF1` (the links).

7 [22]

Instead of testing for a sentinel to detect "end of buffer", keep a running count. A neat MIX trick is to keep rI6 negative and let it run from −100, −99, …, −1 inside each block:

- Addressing: load the next word from BASE+100+rI6. With rI6 negative, this indexes positions 0..99 of the current buffer.
- After each fetch, increment rI6. If it's still negative, we're still in the block; if it becomes 0, we've just consumed the last word, so we:
  1. start an IN on the buffer we just finished (to read the next block into it),
  2. reset rI6 to −100, and
  3. switch to the other buffer.

Using a tiny coroutine hand-off lets the subroutine "enter" at the correct buffer each call without extra state variables, which is exactly the style Knuth points to as one good way to solve this exercise.

MIX program (sentinel-free WORDIN)

```
* Sentinel-free WORDIN: double-buffered, coroutine style.
* U = the input device number (e.g., tape 5). rI6 is our negative index counter.

INBUF1     ORIG *+100           * 100-word buffer #1
INBUF2     ORIG *+100           * 100-word buffer #2

WORDIN     STJ  NEXT            * Patch NEXT with caller's return address.
NEXT       JMP  USE2            * First entry jumps into buffer #2's path (can choose either

* --- Use buffer #2 until it's exhausted, then refill #2 and switch to #1 ---
USE2       LDA  INBUF2+100,6    * A ← INBUF2[100 + rI6]  (rI6 is negative  0..99)
           JMP  RET             * Return this word to caller.
           INC6 1               * Advance position in the current block.
           J6N  USE2            * Still negative? More words in this block → stay.
           IN   INBUF2(U)       * rI6 just hit 0  start reading next block into buffer #2.
           ENN6 100             * rI6 ← -100  (start of next block)
           JMP  USE1            * Switch to buffer #1 for subsequent words.

* --- Use buffer #1 until it's exhausted, then refill #1 and switch to #2 ---
USE1       LDA  INBUF1+100,6
           JMP  RET
           INC6 1
           J6N  USE1
           IN   INBUF1(U)
           ENN6 100
           JMP  USE2

* --- Return site. STJ above patched NEXT to point back to the proper entry on the next call
RET        STJ  NEXT            * Store rJ into the address field of NEXT (classic MIX return
           JMP  *               * Jump to that saved address (i.e., return to caller).
```

How to bootstrap (same spirit as Ex. 6)

Before the first call to `WORDIN`, do the usual startup so the first fetch is correct and I/O is overlapped:

- Initialize the counter: `ENN6 100` (so rI6 = −100).
- Prefetch one buffer (either is fine), e.g.: `IN INBUF2(U)`.
- Leave `NEXT` as shown (`JMP USE2`) so the first call uses buffer #2's path. (You could swap the roles of 1/2 if you prefer.) These are exactly the kind of "get it started right" steps noted next to program (4).

Why it works (and why it's fast)

- Correctness. Each call returns exactly one word (from the current buffer at offset 100+rI6). When rI6 reaches 0, we (i) initiate an input on the buffer we just finished, (ii) reset rI6 to −100, and (iii) continue from the other buffer. That reproduces the overlapped "read-ahead" behavior of (4), but without testing a sentinel; the sign of rI6 is our end-of-buffer test. The coroutine hand-off (`STJ NEXT` + `JMP USE1/USE2`) selects the correct entry for the next call, so the caller remains oblivious.
- Cost. As in the sentinel version, each word costs one indexed load plus a couple of very cheap instructions, and at block boundaries we pay for one `IN` and one `ENN6`. With read-ahead active, the per-block time is `max(C, T)` (C = compute per block, T = device time), just like the buffered methods in the text.

Notes

- The routine assumes full 100-word blocks from the device, as in (4). If your data format has a separate notion of logical end-of-input (e.g., a trailing card with a special mark), handle that in the surrounding control coroutine/program exactly as in the section's examples.
- If you prefer not to use coroutines, you can keep a "current-buffer base" pointer in memory and branch between two short "use buffer" paths with an explicit test on rI6; the coroutine version above simply removes that extra pointer by letting the entry point itself encode which buffer is current. (Knuth's official answer illustrates the coroutine approach.)

8 [11]

What the colors mean for **output**

- **Green (G)** = a free buffer that can be **ASSIGN**ed and filled with the next line image.
- **Yellow (Y)** = the **current** buffer the program is building between **ASSIGN** and **RELEASE**.
- **Red (R)** = a buffer that has been **RELEASE**d and now contains a complete line image, ready for the printer (or actively being printed if it's the one pointed to by **NEXTR**).

The pointers are as in the text: **NEXTG** → "next green" (where the next ASSIGN will hand you a free buffer), **NEXTR** → "next red" (what the output device will print next), **CURRENT** → the yellow (assigned) buffer, if any.

Interpreting the figure sequence for **output**

State shown in Fig. 23 (before we start the three snapshots in Fig. 24)

At that instant there are several green buffers (free space is available) and **two red buffers**; the **red** buffer indicated by **NEXTR** is **currently being printed** by the line printer. At the same time, the program is **computing between RELEASE and ASSIGN**; i.e., it has

finished one line (so that buffer turned red) and hasn't yet requested a fresh buffer for the next line.

(a) After **ASSIGN**

The program calls **ASSIGN**. The buffer pointed to by **NEXTG** turns **yellow** and becomes **CURRENT**; **NEXTG** advances clockwise to the next green. The program now begins to **compose the next line image** into this yellow buffer while the printer continues with its current red buffer.

(b) After output completion (I/O complete)

When the printer **finishes** with the red buffer it was printing, that buffer turns **green** (it's now free). **NEXTR** advances clockwise to the **next red** buffer (if any). If another red buffer exists, the printer immediately starts on it; otherwise the device becomes idle and will resume only when a red buffer appears later.

(c) After **RELEASE**

When the program **RELEASE**s the yellow buffer (line image completed), that buffer turns **red** (ready to print). If the device had been idle (no red buffers), it can now begin printing this newly red buffer (NEXTR will pick it up next); otherwise it joins the queue of red buffers that the printer will reach when NEXTR cycles around.

9 [21]

Model of operation (quick recap)

- Buffers cycle through colors: **green (free) → yellow (current/being filled) → red (ready for output) → green**.
- `ASSIGN` picks the next **green** buffer (if none are green, `ASSIGN` must wait).
- `RELEASE` turns the current yellow buffer **red** and increments the count `n` of red buffers.
- The CONTROL coroutine:
  - If the device is idle and `n>0`, it **initiates OUT** on the buffer pointed to by `NEXTR` (taking 7500u), then advances `NEXTR` and decrements `n` when output completes.
- All "compute for T" durations are **CPU time only**; if `ASSIGN` has to wait for a green buffer, the CPU is idle during that wait (this is explicitly part of the exercise).

Deriving the schedule (walkthrough)

Let buffers be BUF1 → BUF2 → BUF3 in the ring. Initially all are green; `NEXTR = NEXTG = BUF1`; device idle.

t = 0: `ASSIGN(BUF1)` → BUF1 turns yellow. Compute 1000u.

t = 1000: `RELEASE` → BUF1 becomes red (`n=1`). Device is idle and `n>0`, so **OUT BUF1 starts** (finishes at 8500). Compute next 1000u.

t = 2000: `ASSIGN(BUF2)` (green) → yellow. Compute 1000u.

t = 3000: `RELEASE` → BUF2 red (`n=2`). Device busy (till 8500). Compute 1000u.

t = 4000: `ASSIGN(BUF3)` (green) → yellow. Compute 1000u.

t = 5000: `RELEASE` → BUF3 red (`n=3`). Device still busy. Compute 1000u.

t = 6000: Attempt `ASSIGN`, but **no green buffers** (all three are red or in output), so `ASSIGN` **waits**. CPU idle from 6000 onward until a buffer frees.

t = 8500: OUT BUF1 completes → BUF1 turns green, `n` drops to 2. The pending `ASSIGN` now succeeds immediately: **BUF1 assigned** (yellow). Also, since device idle and `n>0`, CONTROL immediately starts **OUT BUF2** (finishes at 16000). Compute 1000u.

t = 9500: `RELEASE` → BUF1 red (`n=3` again). Device busy (till 16000). Compute continues; the next `ASSIGN` arrives after 1000u, but there are still no greens, so it will wait:

t = 10500: `ASSIGN` **waits** (second idle stretch for the CPU).

t = 16000: OUT BUF2 completes → BUF2 green, `n`→2. Waiting `ASSIGN` succeeds: **assign BUF2** (yellow). Device immediately starts **OUT BUF3** (finishes at 23500). Continuing this same reasoning—each time a `RELEASE` makes a buffer red it increments `n`; when the device goes idle and `n>0` it starts the next `OUT`; when `ASSIGN` encounters no greens it waits until the next `OUT` completion—produces the full time–action table below.

Time–action chart (result)

The complete schedule (events in time order) is:

- 0 ASSIGN(BUF1)
- 1000 RELEASE, OUT BUF1
- 2000 ASSIGN(BUF2)
- 3000 RELEASE
- 4000 ASSIGN(BUF3)
- 5000 RELEASE
- 6000 ASSIGN (wait)
- 8500 BUF1 assigned, OUT BUF2
- 9500 RELEASE
- 10500 ASSIGN (wait)

- 16000 BUF2 assigned, OUT BUF3
- 23000 RELEASE
- 23500 OUT BUF1
- 28000 ASSIGN(BUF3)
- 31000 OUT BUF2
- 35000 RELEASE
- 38500 OUT BUF3
- 40000 ASSIGN(BUF1)
- 46000 Output stops
- 47000 RELEASE, OUT BUF1
- 52000 ASSIGN(BUF2)
- 54500 Output stops
- 59000 RELEASE, OUT BUF2
- 64000 ASSIGN(BUF3)
- 65000 RELEASE
- 66000 ASSIGN(BUF1)
- 66500 OUT BUF3
- 68000 RELEASE
- 69000 Computation stops
- 74000 OUT BUF1
- 81500 Output stops.

Totals and idle times

From the chart:

- **Total elapsed time**: **81500u**.
- **CPU idle** (waiting at `ASSIGN`): 6000–8500, 10500–16000, 69000–81500 **20500u**.
- **Device idle**: 0–1000, 46000–47000, 54500–59000 **6500u**.

10 [21]

- Use the same compute sequence from Ex. 9 (A = ASSIGN, R = RELEASE), where the numbers are pure **compute** times (waiting time isn't included): `A, 1000, R, 1000, A, 1000, R, 1000, A, 1000, R, 1000, A, 1000, R, 7000, A, 5000, R, 7000, A, 5000, R, 7000, A, 5000, R, 7000, A, 5000, R, 1000, A, 1000, R, 2000, A, 1000, R.` (Same list as Fig. 27/Ex. 9.) The output device runs at **7500u per block**.
- For output: **ASSIGN** grabs a *green* buffer (turns it yellow), **RELEASE** turns that yellow buffer **red**; the control starts **OUT** on the next red buffer whenever idle; when the output finishes, that buffer becomes green. With **N = 4** buffers, the program need only wait at an ASSIGN if *all four* buffers are red (no greens).

Time–action chart (N = 4)

Below, “OUT BUFk” marks the instant an output of BUFk begins; “Output stops.” marks instants when the device goes idle because no red buffers remain.

```
Time   Action
-----  ---------------------------------------
0      ASSIGN(BUF1)
1000   RELEASE, OUT BUF1
2000   ASSIGN(BUF2)
3000   RELEASE
4000   ASSIGN(BUF3)
5000   RELEASE
6000   ASSIGN(BUF4)
7000   RELEASE
8500   OUT BUF2
14000  ASSIGN(BUF1)
16000  OUT BUF3
19000  RELEASE
23500  OUT BUF4
26000  ASSIGN(BUF2)
31000  OUT BUF1
31000  RELEASE
38000  ASSIGN(BUF3)
38500  OUT BUF2
43000  RELEASE
46000  OUT BUF3
50000  ASSIGN(BUF4)
53500  Output stops.
55000  RELEASE, OUT BUF4
56000  ASSIGN(BUF1)
57000  RELEASE
59000  ASSIGN(BUF2)
60000  RELEASE
61000  Computation stops.
62500  OUT BUF1
70000  OUT BUF2
77500  Output stops.
```

How this unfolds (sketch): early on the program is producing red buffers faster than the first output can finish, so red buffers accumulate until the first finish at **8500u**; from then through **53500u** the device is almost continuously busy draining reds as they arrive. There is **no**

"ASSIGN (wait)" event with four buffers, because whenever an ASSIGN occurs there is always at least one green available (n never reaches 4 at an ASSIGN instant).

Totals and idle intervals

- **Total elapsed time: 77500u** (last "Output stops.").
- **Computation stops:** at **61000u** (there were no ASSIGN waits at all).
- **Computer idle time: 61000–77500 16500u**.
- **Output device idle intervals: 0–1000** and **53500–55000 2500u** total. (Contrast with the three-buffer case in Ex. 9's chart, where total time was 81500u and there were two mid-run computation waits.)

If you'd like, I can also tabulate buffer colors (G/Y/R) over time or show the evolving queue of reds to make the coroutine interplay completely transparent.

11 [21]

We apply the text's buffering algorithms (A for ASSIGN, R for RELEASE, B for buffer control): at **ASSIGN** the program waits if all buffers are red; at **RELEASE** we increment the red-buffer count and the controller starts output if the device is idle; when I/O completes, the controller advances NEXTR and **decrements** the red-buffer count.

Because **N=1**, once we RELEASE the only buffer, the device is busy until it finishes that block; the program cannot ASSIGN again (A1 stalls) until I/O completes. Thus computation and output **cannot overlap** with a single buffer. (Contrast with the three-buffer timeline and chart for Fig. 27.)

Time–action chart (one buffer)

Time (u)	Action
0	ASSIGN(BUF1)
1000	RELEASE, OUT BUF1
2000	ASSIGN (wait)
8500	BUF1 assigned
8500	Output stops.
8500	ASSIGN(BUF1)
9500	RELEASE, OUT BUF1
10500	ASSIGN (wait)
17000	BUF1 assigned
17000	Output stops.
17000	ASSIGN(BUF1)
18000	RELEASE, OUT BUF1
19000	ASSIGN (wait)

Time (u)	Action
25500	BUF1 assigned
25500	Output stops.
25500	ASSIGN(BUF1)
26500	RELEASE, OUT BUF1
27500	ASSIGN (wait)
34000	BUF1 assigned
34000	Output stops.
34000	ASSIGN(BUF1)
35000	RELEASE, OUT BUF1
36000	ASSIGN (wait)
42500	BUF1 assigned
42500	Output stops.
42500	ASSIGN(BUF1)
43500	RELEASE, OUT BUF1
44500	ASSIGN (wait)
51000	BUF1 assigned
51000	Output stops.
51000	ASSIGN(BUF1)
58000	RELEASE, OUT BUF1
65000	ASSIGN (wait)
65500	BUF1 assigned
65500	Output stops.
65500	ASSIGN(BUF1)
72500	RELEASE, OUT BUF1
79500	ASSIGN (wait)
80000	BUF1 assigned
80000	Output stops.
80000	ASSIGN(BUF1)
87000	RELEASE, OUT BUF1
93500	ASSIGN (wait)
94000	BUF1 assigned
94000	Output stops.
94000	ASSIGN(BUF1)
101000	RELEASE, OUT BUF1
108500	ASSIGN (wait)
109000	BUF1 assigned
109000	Output stops.
109000	ASSIGN(BUF1)
111000	RELEASE, OUT BUF1
118500	Output stops.

(Style follows the sample chart in the book: times at which **ASSIGN**, **RELEASE**, and I/O starts/stops occur as the system runs. For reference, the sample chart for three buffers appears just before the exercise list, with its accompanying time-action table.)

Totals and discussion

- **Total elapsed time: 118 500u** (final "Output stops").
- **Device busy time:** 11 blocks × 7 500u = **82 500u**; **device idle: 36 000u**.
- **Computer idle time (stalls at A1): 49 000u** (the sum of all "ASSIGN (wait) → BUF1 assigned" gaps).

With three buffers, the text's chart finishes in **81 500u** for the same program; with a single buffer we stretch to **118 500u**, because every post-RELEASE **ASSIGN** must wait for the device to finish before computation can continue. Buffering depth is exactly what allows overlap between computing and I/O; with **N=1** there is no overlap at all.

12 [24]

Minimal algorithmic change (Algorithm B → B)

Add a single global flag `EOF` (initially 0). Then:

- **B2 (gate before initiating I/O)**: If `EOF = 1` **or** `n = 0`, go to B1 (compute). Otherwise proceed to B3.
- **B4 (just after I/O completes, before advancing NEXTR)**: Inspect the **80th column** of the buffer that was just filled (i.e., at address `NEXTR + 15`, byte 5). If it is `'.'`, set `EOF ← 1`. Then continue with B5/B6 (advance NEXTR; `n ← n - 1`).

Everything else (ASSIGN, RELEASE, pointer motion, colors) stays as Knuth describes. The placement is important: you test for `'.'` **after** the `IN` you started in B3 has finished (B4) and **before** you move `NEXTR` in B5.

MIX patch (Program B → Program B)

Below is a drop-in patch to Knuth's **Program B (CONTROL)**. It assumes:

- rI6 `n` (count of red buffers), rI5 `NEXTR` (address of current red buffer's start), exactly as in the text.
- `CARDS EQU 16` elsewhere; `U` already holds the unit (as in the book's code).
- `EOF` is a one-word flag (0/1) you define in static storage (initialized to 0).
- `DOT` is the MIX **character code** for `'.'` (you can define it as a constant; any correct value for your MIXAL assembler is fine).
- Field `(5:5)` denotes the rightmost byte in a word.

Only the **new** and **changed** lines are annotated; unchanged lines are shown for context:

```
* --- New data ---
EOF     CON 0            * 0 = keep reading; 1 = stop initiating input
DOT     CON 46           * MIX code for '.' (period); use your table/assembler symbol

* --- CONTROL coroutine (Program B ) ---
* rI6  n, rI5  NEXTR
CONT1   JMP COMPUTE          * B1. Compute (unchanged)

1H      LD1 EOF              * NEW: test end-of-input latch
        J1NZ CONT1           * If EOF=1, skip initiating more reads (go compute)
        J6Z *-1              * B2 : if n = 0, wait (as before)

        IN  0,5(U)           * B3. Initiate I/O into buffer at NEXTR
        JMP COMPUTE          * B4. Compute until I/O completes (JRED brings us back)

* --- We return here when the reader signals ready (after IN has finished) ---

        LDA 15,5(5:5)        * NEW (B4 ): load col 80 = byte 5 of word (NEXTR+15)
        CMPA =DOT=           * Is it a '.' ?
        JNE  2F              * If not, continue normally
        ENTA 1               * Else set EOF ← 1
        STA  EOF
2H      LD5 -1,5             * B5. Advance NEXTR to next buffer (via link at [NEXTR-1])
        DEC6 1               * B6. n ← n - 1
        JMP  1B
```

- The **gate** at label `1H` enforces "no more `IN`" once `EOF` is set.
- The **check** immediately after the `JMP COMPUTE` (on return) looks at **exactly** the last column of the just-read card and latches `EOF` if it's `'.'`.
- We deliberately perform the test **before** `LD5 -1,5` so rI5 still points at the buffer we just filled.

Why this is correct

- In Algorithm B, the moment a read completes (B4), the freshly read card still resides at `NEXTR`; only then do we advance `NEXTR` (B5). Inserting the test between B4 and B5 lets us examine the right card every time.
- Once `EOF` is latched, B2 prevents any further `IN` from being initiated. Already-read buffers will continue to flow to the program (green→yellow→red) via ASSIGN/RELEASE exactly as before.
- The mapping "card reader → 16 words, 5 chars/word" makes column 80 equal to `(NEXTR + 15, byte 5)`, which is the byte we test.

Here's a clean way to shut input down the moment the "end card" is read (a card whose column 80 is '.'), by minimally extending Knuth's multiple-buffer control coroutine.

What changes conceptually

In the textbook, the CONTROL coroutine (Algorithm B / Program B) repeatedly:

- waits until there is at least one red buffer (n > 0),
- starts an `IN` on the buffer at `NEXTR`,
- yields back to computation,
- when the I/O finishes, advances `NEXTR` and decrements `n`.

For card input, each card image occupies 16 MIX words (5 bytes/word = 80 columns); column 80 is byte 5 of word 16, i.e., byte (5:5) of address `NEXTR+15`.

We'll add a single sticky flag `EOF` (=0 initially). CONTROL will:

1. After each input completes (just before advancing `NEXTR`), inspect the last byte of the just-read card (`(NEXTR+15)(5:5)`).
2. If it equals the MIX character code for '.', set `EOF ← 1`.
3. At the top of the loop, treat `EOF` exactly like "no red buffers": if `EOF=1`, do not start more `IN` operations ever again.

This leaves already-anticipated (green) cards, including the sentinel card itself, available for the COMPUTE coroutine to consume; it simply prevents any *further* anticipation past the end. (Knuth explicitly leaves such end-handling details to the reader; this is precisely the point of Ex. 12.)

Algorithm B (CONTROL) — deltas only

Relative to Algorithm B, change B2 and insert a test between B4 and B5:

- **B2 .** If `n = 0` **or** `EOF = 1`, go to B1 (keep computing; do not initiate I/O).
- **B4½.** [Test end card.] If byte (5:5) of word at `NEXTR+15` equals '.', set `EOF ← 1`.

Everything else (B3, B5, B6) is unchanged.

Program B (MIX) — minimal patch

Below is a small, self-contained patch to Knuth's Program B. It preserves the original register discipline (CONTROL already uses only `rI6 n` and `rI5 NEXTR`), so we save/restore `A` around our tests to avoid surprising COMPUTE.

New data (somewhere in CONTROL's static area):

```
EOF   CON 0          * sticky "seen end card" flag (0/1)
SAVEA CON 0          * temp to save A
DOT   CON <code-for-'.'>   * MIX code of '.' (any correct constant is fine)
```

Patched code (comments show where lines go in Knuth's listing):

```
Program B (CONTROL). rI6  n, rI5  NEXTR.

CONT1  JMP  COMPUTE           * B1. Compute

1H      J6Z  *-1              * B2. wait while n=0
        STA  SAVEA            * save A before we look at EOF
        LDA  EOF
        JAZ  2F               * if EOF==0, OK to start I/O
        LDA  SAVEA            * else (EOF!=0): restore A and just yield
        JMP  COMPUTE          *   behave like n=0 forever (no more IN)

2H      IN   0,5(U)           * B3. Initiate I/O
        LDA  SAVEA            * restore A before yielding
        JMP  COMPUTE          * B4. Compute until I/O completes

* ---- we return here when the read just finished on buffer NEXTR ----
* B4½. Test last column (byte 5 of word 16 in this card image).
        STA  SAVEA            * save A again
        LDA  15,5(5:5)        * A ← column 80 from buffer at NEXTR
        CMPA DOT
        JNE  3F               * if not '.', skip setting EOF
        ENTA 1
        STA  EOF              * EOF ← 1
3H      LDA  SAVEA            * restore A

        LD5  -1,5             * B5. Advance NEXTR
        DEC6 1                * B6. Decrease n
        JMP  1B
```

Notes:

- `15,5(5:5)` addresses "word 16 indexed by `rI5`, field 5:5", i.e., byte 5 of the card's last word.
- Use any correct assembler literal for `DOT` (the MIX code for '.'). If you like, assemble `ALF .....` into a word and extract the rightmost byte once; a plain numeric constant also suffices.

Why this is correct

- If the card just read is *not* the end card, CONTROL behaves identically to the book: advances `NEXTR`, decrements `n`, and continues.
- If it *is* the end card, `EOF` is set. From then on, the modified B2 treats `EOF` as a permanent "don't start I/O" condition. That freezes anticipation beyond the sentinel while leaving already-green buffers for COMPUTE to finish processing.
- The sentinel card itself is available to COMPUTE (it turned the buffer from red→green when the read completed), so the main program can recognize "end of input" in its normal logic after assigning that buffer.

13 [20]

- In the multi-buffer algorithms, **n = current number of red buffers** (blocks waiting to be output).
- The control coroutine (**CONTROL**) performs I/O and, after an output completes, does **B5** (advance `NEXTR`) and **B6** (decrease `n`).
- The machine instruction `JRED CONTROL(U)` in **COMPUTE** is how CONTROL is re-entered when the device becomes ready; the same framework is used for output as for input, differing only in initial conditions.

When computation is finished, there may still be **red** buffers (so `n > 0`) and/or an output currently in progress. You must:

1. keep giving CONTROL a chance to run whenever the device becomes ready (so it can execute B5–B6 and drain `n`), and
2. only return once no red buffers remain (which also implies the device is idle, since `n` is decremented *after* an output completes).

A spin on `n` **without** `JRED CONTROL(U)` would deadlock, because CONTROL wouldn't get to run.

Minimal MIX trailer to append at end of COMPUTE (output case)

```
* rI6   n   (number of red buffers)
* U     output device number used by CONTROL

FLUSH  JRED CONTROL(U)     * if the device has just become ready,
                           * jump into CONTROL to do B5/B6
       J6NZ FLUSH          * loop until n == 0 (no red buffers remain)
* Optional belt-and-suspenders wait (normally redundant once n==0):
WAIT   JBUS WAIT,U         * if device were somehow still busy, wait
* … now it's safe to exit/return
```

Why this works:

- `JRED CONTROL(U)` hands control to CONTROL **exactly** when the device signals ready; CONTROL then finishes the just-completed output and executes **B6: Decrease n**.
- The loop continues until `n = 0`, meaning there are no red buffers left to output. Since `n` is decremented only after an output finishes, `n = 0` implies the device isn't in the middle of an output.

That's all you need to guarantee that every buffered block has been sent before the program terminates.

Two-instruction solution (MIX)

```
JRED CONTROL(U)
J6NZ *-1
```

Why this works

- `JRED CONTROL(U)` yields to the `CONTROL` coroutine whenever the device *is ready*. `CONTROL` then performs step B5/B6 (advance `NEXTR`; decrement `n`) for one finished output operation, and returns to `COMPUTE`.
- `J6NZ *-1` loops while `rI6 = n  0`. Thus we repeatedly give `CONTROL` chances to complete outstanding writes until no red buffers remain. When `n` finally becomes zero, the jump fails and the program may safely terminate.

14 [20]

Recap of the buffering model

- Buffers are arranged in a circle; each buffer is colored **green**, **yellow**, or **red**. • **Green** = ready to be `ASSIGN`ed (for input: already anticipated; for output: free). • **Yellow** = currently `ASSIGN`ed (the program may read/write it). • **Red** = `RELEASE`d (for input: free area; for output: filled, awaiting device).
- Pointers: `NEXTG` points to the next green buffer; `NEXTR` to the next red buffer; `CURRENT` names "the" yellow buffer when there is one. Two processes run as coroutines: **Algorithm A (ASSIGN)** makes `CURRENT ← NEXTG` and advances `NEXTG`. **Algorithm R (RELEASE)** increments the count `n` of red buffers. **Algorithm B (Buffer control)** starts I/O on `NEXTR` when `n>0`, then advances `NEXTR` and decrements `n` when I/O completes.

What if we do ASSIGN, ASSIGN, then RELEASE, RELEASE?

Think in terms of counts. At any instant:

```
#green + #yellow + #red = N ,
```

and (by definition of R1) `n = #red`. Every `ASSIGN` **consumes** a green and creates a yellow; every `RELEASE` **turns** a yellow into a red (and raises `n` by 1). If we perform two `ASSIGN`s in a row while at least two greens exist, we simply end up with **two yellow buffers** (two outstanding assignments). Later `RELEASE`s convert those yellows to reds one by one; Algorithm B then services those reds in FIFO pointer order. The control logic keeps working because B only depends on `n` (reds) and the `NEXTR` ring, not on how many yellows exist. This can actually be **useful** when the computation wants to look at two buffers at once (e.g., a boundary-crossing comparison, merging adjacent blocks, etc.), at the cost of tying up an extra buffer that is temporarily unavailable for anticipation/output.

Important edge case and the safe bound

- **N = 1 (one buffer).** A burst `ASSIGN, ASSIGN` has no second distinct green to consume. The written Algorithm A's gate ("wait for `n < N`") isn't sufficient to guarantee a green exists when we allow multiple simultaneous `ASSIGN`s; with one buffer you can wind up re-assigning the same area or, worse, overlapping program use with I/O. In short, the scheme **breaks down** for `N=1`.
- **General bound.** Let `y = (#ASSIGNs - #RELEASEs)` = "#yellow at the moment." Necessarily

```
0   y   N .
```

(If `y` exceeded `N`, there would be no green left to assign; if `y` were negative, we'd be releasing more than we assigned.) Practically, keep the **excess of ASSIGNs over RELEASEs** nonnegative and **N**.

- With **N 2**, allowing `ASSIGN` bursts just produces **two (or more) simultaneous yellow buffers**; the control coroutine continues to function, and this can be handy when the computation needs concurrent access to several buffers. It does, however, **tie up buffer space**, leaving fewer greens available for anticipation.
- With **N = 1**, the idea is unsafe/inapplicable; the multiple-buffer algorithm as given relies on there being enough buffers that an extra `ASSIGN` can legitimately consume another green.

15 [22]

Here's a fast, 3-buffer MIX solution for ("copy 100 blocks from tape unit 0 to tape unit 1, using just three buffers"). The trick is to **pipeline** I/O so the input and output units stay busy almost all the time, with only a one-time "prime" and a one-time "drain." This is exactly the "same-circle input+output" idea behind Fig. 26 and the 1.4.4 exercises on pp. 225–228.

MIX program (3-buffer tape-to-tape copy)

```
* Tape-to-tape copy with triple buffering (fast)
INUNIT  EQU 0              * input tape unit
OUTUNIT EQU 1              * output tape unit

BUF1    ORIG *+100         * three 100-word buffers
BUF2    ORIG *+100
BUF3    ORIG *+100

START   IN   BUF1(INUNIT)  * prime the pipeline: read block #0 -> BUF1
        ENT1 99            * 99 more blocks to output after the prime

LOOP    IN   BUF2(INUNIT)  * queue read of next block -> BUF2
        OUT  BUF1(OUTUNIT) * write the block that was in BUF1
        IN   BUF3(INUNIT)  * queue read -> BUF3
        OUT  BUF2(OUTUNIT) * write the block now in BUF2
        IN   BUF1(INUNIT)  * queue read -> recycle BUF1
        OUT  BUF3(OUTUNIT) * write the block now in BUF3
        DEC1 3
        J1P  LOOP

        JBUS *(INUNIT)     * wait for final input to finish
        OUT  BUF1(OUTUNIT) * write the very last block
        HLT
        END  START
```

Why this is correct and fast

1. **Pipeline structure.**

- We "prime" with one input (BUF1).
- Each loop body **initiates three inputs and three outputs** in a fixed rotation (…B2 in, B1 out; B3 in, B2 out; B1 in, B3 out…). This schedules operations so that **every OUT uses a buffer whose IN started two I/O steps earlier**, i.e., long enough before to have finished by the time the device consumes it (CPU time is negligible compared to device time).
- We decrement by 3 because each loop iteration produces 3 completed outputs. Starting from 99, we do 33 iterations (99/3), then drain the final pending input and write the final block.

2. **No buffer hazards.** Each buffer is reused only after its previous read and subsequent write have had time to complete; the triple spacing prevents "read-while-write" on the

same memory. We never need to poll **during** the pipeline; a single `JBUS *(INUNIT)` at the end synchronizes the very last read before the last write.

3. **Minimal idle time "as fast as possible."** After the one-time prime, both devices are kept busy continuously (as in the multiple-buffer timelines and exercises in §1.4.4), so throughput is device-limited. Only the leading and trailing edges incur unavoidable bubbles. This matches the "same circle" buffering method discussed around Fig. 26/27 and the surrounding exercises.

4. **Why `ENT1 99` and `DEC1 3`?** We must output 100 blocks total. One block is primed into BUF1 before the loop, so **99 outputs remain**; each loop does **3 outputs**, hence 33 passes. After the loop, the pending final read completes (`JBUS`) and we perform the **100th** output.

16 [29]

Here's a clean formulation of the "green–yellow–red–purple" multiple-buffer algorithm Knuth asks for, written in the same style as Algorithms A/R/B in §1.4.4, but now with three coroutines: one for input, one for output, and one for the computation. I stick to his color semantics and ring-of-buffers pointers.

Model and invariants

- We have a single circle of `N` equal buffers.
- Colors (states) and their meaning (same naming as the text and Fig. 26): **G**reen = anticipated input is ready to be **ASSIGN**ed to the program; **Y**ellow = currently assigned to the program (between **ASSIGN** and **RELEASE**); **R**ed = **RELEASE**d by the program; block is ready to be **output**; **P**urple = output finished; buffer is free for new **input**. (Purple is turned to green by input; yellow is turned to red by release; red is turned to purple by output.)
- Pointers on the ring:
  - `NEXTG` → first G (next buffer the program may assign)
  - `NEXTR` → first R (next buffer the output device will transmit)
  - `NEXTP` → first P (next buffer the input device will fill)
  - `CURRENT` → Y (present only while the program holds a buffer)
- Counters we maintain:
  - `g` = # of green buffers
  - `r` = # of red buffers
  - `p` = # of purple buffers
  - `y` {0,1} (0 if no current assignment; 1 if a buffer is yellow)

  - Invariant: `g + r + p + y = N`.

- The ring is always segmented in the cyclic order `P*  G*  (Y?)  R*` with the pointers at the segment heads; this ensures none of `NEXTG`, `NEXTR`, `NEXTP` "passes" another. (This is the three-pointer analogue of the "no passing" story for two pointers in Fig. 24.)

Initial/terminal conditions

- Start: all buffers **purple** (the natural "free for input" state). So `p = N`, `g = r = y = 0`, and set `NEXTP = NEXTG = NEXTR` to some buffer. Input begins immediately and produces the first G's for the program.
- End: after the program's last **RELEASE**, we must allow output to drain until `r = 0` (all finished) before declaring completion (the analogue of the drain step discussed for two-color algorithms).

Program-side primitives (as in §1.4.4)

We keep **ASSIGN** and **RELEASE** as program-visible actions; their algorithms mirror Knuth's A and R, but they operate on `g`, `r`, `p`, `y` and `NEXTG`:

Algorithm A (ASSIGN, inside the COMPUTE coroutine)

A1 . **Wait for `g > 0`.** (Stall the program until some green buffer exists.) A2 . **`CURRENT ← NEXTG`.** (The chosen G becomes the current buffer; color changes G→Y.) A3 . **Advance `NEXTG` clockwise** to the next G. A4 . **Update counts.** `g ← g - 1`, `y ← 1`.

Algorithm R (RELEASE, inside the COMPUTE coroutine)

R1 . **Finish the buffer.** (Color Y→R.) R2 . **Update counts.** `y ← 0`, `r ← r + 1`.

These are exactly the same "shape" as Knuth's A/R; only the bookkeeping differs (his single counter `n` was the number of red buffers in the two-party case).

Device controllers as coroutines

The two devices now have independent controllers (each analogous to Knuth's Algorithm B, but specialized to one color transition). Each controller runs "in parallel" with COMPUTE via coroutine linkage (COMPUTE periodically yields with `JRED …` as in the text).

Coroutine IN (input controller; turns P→G)

I1. **Compute (yield to COMPUTE)** for a short while; re-enter here later at a device-ready instant. I2. **If `p = 0`,** go to I1. (No purple → nothing to input.) I3. **Initiate input** from the input device into the buffer at `NEXTP`. I4. **Compute (yield to COMPUTE)** while the device is busy. I5. **On input completion: advance `NEXTP` clockwise** to the next P; **update counts: `p ← p - 1`, `g ← g + 1`.** Go to I2.

Coroutine OUT (output controller; turns R→P)

O1. **Compute (yield to COMPUTE)** for a short while; re-enter here later at a device-ready instant. O2. **If `r = 0`,** go to O1. (No red → nothing to output.) O3. **Initiate output** from the buffer at `NEXTR` to the output device. O4. **Compute (yield to COMPUTE)** while the device is busy. O5. **On output completion: advance `NEXTR` clockwise** to the next R; **update counts: `r ← r - 1`, `p ← p + 1`.** Go to O2.

Coroutine linkage (skeleton)

Exactly as Knuth does, insert "jump-when-ready" instructions inside the COMPUTE code so the device controllers can preempt when their unit becomes ready. One practical arrangement is:

- COMPUTE periodically executes `JRED INCTRL(U_in)` and `JRED OUTCTRL(U_out)` (once every ~50 instructions as he suggests), letting each controller run steps I1/I4 and O1/O4 at device-ready times; each controller returns to COMPUTE after its brief action.

Why the pointers never "pass"

At every transition we convert the head element of one contiguous color segment to the head of the next segment in the cyclic order:

- IN does **P→G** at `NEXTP` and then advances `NEXTP` to the next P; thus the P-segment shrinks at its head, the G-segment grows at its tail, and no pointer can jump over another.
- ASSIGN does **G→Y** at `NEXTG`, then advances `NEXTG` to the next G; the single-element Y sits between G and R, preserving order.
- RELEASE does **Y→R** (no pointer move).
- OUT does **R→P** at `NEXTR` and advances `NEXTR` to the next R; the R-segment shrinks at its head and the P-segment grows at its tail.

Hence the circular order `P*  G*  (Y?)  R*` is invariant and pointers cannot "lap" each other (the three-party generalization of the two-pointer races discussed with Fig. 24).

Liveness (no deadlock)

- If the program needs a buffer and `g = 0`, then either some output is pending (`r > 0`, so OUT will run and eventually increase `p`), or some input is possible (`p > 0`, so IN will run and increase `g`).
- If an output is needed but `r = 0`, eventually a **RELEASE** will occur (COMPUTE makes Y→R) or new input will feed the pipeline to produce future R's.
- If input is needed but `p = 0`, eventually an output will complete (R→P) or the program advances and releases something.

Provided each coroutine is scheduled fairly (as in Knuth's set-up with periodic `JRED`), progress continues.

Practical MIX skeleton (minimal)

You can mirror Knuth's tiny MIX control loops, one per device, and keep A /R as subroutines in COMPUTE. For example (pseudocode at the level of his Program B):

```
; INCTRL (P->G)
INCTRL:   JMP   COMPUTE         ; I1
IWAIT:    J6Z   IWAIT           ; I2: if p==0, spin/yield
         IN    0,U_in           ; I3
         JMP   COMPUTE          ; I4
IDONE:   LDX   NEXTP            ; I5: advance NEXTP
         ; p-- ; g++ (update)
         JMP   IWAIT

; OUTCTRL (R->P)
OUTCTRL:  JMP   COMPUTE         ; O1
OWAIT:    J6Z   OWAIT           ; O2: if r==0, spin/yield
         OUT   0,U_out          ; O3
         JMP   COMPUTE          ; O4
ODONE:   LDX   NEXTR            ; O5: advance NEXTR
         ; r-- ; p++ (update)
         JMP   OWAIT
```

(Exactly how you hold `g, r, p, y` in MIX registers or memory, and the device unit numbers, follows the same conventions as his Programs A/R/B; the linkage is the same pattern, only the colors and counts differ.)

MIX setup (symbols and state)

```
* ======= DEVICE UNITS (choose any legal MIX device #s) =======
UIN     EQU     0               * input device unit
UOUT    EQU     1               * output device unit

* ======= RING & POINTERS =======
N        CON     6              * number of buffers in the ring (example)
RING     ORIG    *+N            * table of N buffer base addresses (filled elsewhere)

NEXTG    CON     0              * pointer index into RING for next Green
NEXTR    CON     0              * pointer index into RING for next Red
NEXTP    CON     0              * pointer index into RING for next Purple
CURRENT  CON     -1             * pointer index for Yellow (=-1 when none)

* ======= COUNTS (G,R,P) and Y-flag =======
GCT      CON     0              * # green
```

```
RCT      CON     0              * # red
PCT      CON     0              * # purple
YFLG     CON     0              * 0|1 whether CURRENT is valid (yellow in use)

* ======= PER-BUFFER COLOR (compact: 0=P, 1=G, 2=Y, 3=R) =======
COL0     CON     0              * COL[i] holds color code for buffer i
COL1     CON     0
COL2     CON     0
COL3     CON     0
COL4     CON     0
COL5     CON     0
COLTAB   EQU     COL0           * base for COL[i] = COLTAB+i

* ======= WORK REGS (indices) =======
* We'll use rI1 as 'i', rI2 as N, rI3 as scratch.
INIT:   ENTI2   N               * load N into rI2 at boot
        ENTJ    MAIN            * fall into MAIN
```

Initialization (as in the text): start with all buffers **purple** (P), so `PCT=N`, `GCT=RCT=YFLG=0`, and set `NEXTP = NEXTG = NEXTR` to the same position. From then on, the three coroutines keep the circle moving without pointers passing each other.

Subroutines used by A /controllers

To keep the listing compact (à la TAOCP), the "advance to next buffer with color = k" is a tiny helper that walks clockwise modulo `N`.

```
* ADVANCE(i_ptr, want_color) -> updates memory word at i_ptr to next index having color
* rI2 = N, rI3 scratch. Argument want_color in rA low byte (0..3). i_ptr is an address.
ADVANCE STJ     ADVX
         LDA     0,0(5:5)      * want_color already in rA's (5:5) on entry (see callers)
         LDX     0,0           * rX free
         LDA     0,1           * load i_ptr? (we'll pass as literal via callers)
ADVLP    LD1     0,0           * load current index i from [i_ptr] (caller does LDA then ST1)
         INC1    1
         CMP1    N             * wrap
         J1L     *+2
         ENT1    0
         STA     SAVI          * save i
         ADD     =COLTAB=      * compute &COL[i]
         STA     ADRC
         LDA     0,ADRC
         CMPA    WANTC         * matches desired color?
```

```
         JE      ADVOK
         JMP     ADVLP
ADVOK    LDA     SAVI
         STA     0,1           * store back to [i_ptr]
ADVX     JMP     0,0

SAVI     CON     0
ADRC     CON     0
WANTC    CON     0
```

(NB: In a real MIX listing you'd pass the `i_ptr` and the desired color more directly; I've spelled this out minimally to show intent while keeping the style close to Knuth's short library helpers.)

Program A — ASSIGN (in COMPUTE)

Algorithm A : wait until there's at least one **G**; take `NEXTG` as `CURRENT` (G→Y), advance `NEXTG` to next **G**, update counts. This is inserted where the computation needs a fresh input block.

```
* Program A  - ASSIGN
ASSIGN  STJ     AEX             * A  entry

A1      LDA     GCT             * A1. Wait for g > 0
        JAZ     AWAIT
        JMP     A2
AWAIT   JRED    INRD,UIN        * let INCTRL run if input just finished
        JRED    OUTRD,UOUT      * let OUTCTRL run if output just finished
        JMP     A1

A2      LDA     NEXTG           * A2. CURRENT ← NEXTG (G->Y)
        STA     CURRENT
        LDA     2               * color code 2 = Yellow
        STA     0,COLTAB(CURRENT)  * COL[CURRENT] ← Y
        STA     YFLG,=1         * y ← 1

A3      LDA     =1=             * A3. g-- ; advance NEXTG to next G
        DEC     GCT,1
        LDA     =1=
        STA     WANTC           * want_color = 1 (Green)
        LDA     NEXTG
        STA     PTRG
        JSR     ADVANCE         * scan to next Green
        LDA     PTRG
```

```
        STA     NEXTG
AEX     JMP     0,0

PTRG    CON     0
```

Program R — RELEASE (in COMPUTE)

Algorithm R : mark the current buffer **R** (Y→R), clear Y-flag, `r++`.

```
* Program R  - RELEASE
RELEASE STJ     REX
R1      LDA     CURRENT         * R1. Y->R; r++ ; y=0
        LDA     =3=             * color code 3 = Red
        STA     0,COLTAB(CURRENT)
        INC     RCT,1
        STA     YFLG,=0
REX     JMP     0,0
```

Coroutine INCTRL — input controller (P→G)

Matches the B-style controller in §1.4.4 but specialized to P→G. Two entries: an **idle** entry that is polled via `JRED` (I1/I4), and a **ready** entry (I5) that runs when the unit signals "ready."

```
* INCTRL: turns Purple -> Green
INCTRL  JMP     I1              * I1. Idle entry (reached via JRED return)

I1      LDA     PCT             * I2. If p==0, nothing to start
        JAZ     IRET
        * I3. Start IN to buffer at NEXTP
        LDA     NEXTP
        STA     TMPP
        * compute buffer address RING[NEXTP] into rX:
        LD1     NEXTP
        LDX     RING,1
        IN      0,UIN           * device starts filling buffer rX
        JMP     IRET            * I4. Yield to COMPUTE

INRD    * ready-entry label (target of JRED UIN)
        * I5. Completion: P->G ; advance NEXTP ; counts
        LDA     TMPP
        STA     IDX
        LDA     =1=             * 1 = Green
```

```
        STA     0,COLTAB(IDX)
        DEC     PCT,1
        INC     GCT,1
        * advance NEXTP to next Purple
        LDA     =0=
        STA     WANTC           * want_color=0 (Purple)
        LDA     NEXTP
        STA     PTRP
        JSR     ADVANCE
        LDA     PTRP
        STA     NEXTP
IRET    JMP     COMPUTE

TMPP    CON     0
PTRP    CON     0
IDX     CON     0
```

Coroutine OUTCTRL — output controller (R→P)

Symmetric to input: when `r>0` and device idle, start an `OUT` from `NEXTR`. On ready, mark **P**, advance `NEXTR`, update counts.

```
* OUTCTRL: turns Red -> Purple
OUTCTRL JMP     O1              * O1. Idle entry

O1      LDA     RCT             * O2. If r==0, nothing to start
        JAZ     ORET
        * O3. Start OUT from buffer at NEXTR
        LDA     NEXTR
        STA     TMPR
        LD1     NEXTR
        LDX     RING,1
        OUT     0,UOUT
        JMP     ORET            * O4. Yield

OUTRD   * ready-entry label (target of JRED UOUT)
        * O5. Completion: R->P ; advance NEXTR ; counts
        LDA     TMPR
        STA     IDX2
        LDA     =0=             * 0 = Purple
        STA     0,COLTAB(IDX2)
        DEC     RCT,1
        INC     PCT,1
```

```
        * advance NEXTR to next Red
        LDA     =3=
        STA     WANTC
        LDA     NEXTR
        STA     PTRR
        JSR     ADVANCE
        LDA     PTRR
        STA     NEXTR
ORET    JMP     COMPUTE

TMPR    CON     0
PTRR    CON     0
IDX2    CON     0
```

COMPUTE coroutine (skeleton)

A trivial loop that periodically gives time to device controllers via `JRED`, and uses `ASSIGN/RELEASE` around "work". The `JRED` polling cadence (e.g., "every ~50 instructions") follows §1.4.4's guidance.

```
MAIN    * ===== Boot: all buffers Purple, set pointers equal =====
        STZ     GCT
        STZ     RCT
        LDA     N
        STA     PCT
        STZ     YFLG
        STZ     NEXTG
        STZ     NEXTR
        STZ     NEXTP
        * set COL[i]=P for all i
        ENT1    0
CLP     LDA     =0=
        STA     0,COLTAB(1)
        INC1    1
        CMP1    N
        J1L     CLP

COMPUTE * ===== Example compute loop =====
        * A : need an input block
        JSR     ASSIGN
        * (use CURRENT buffer: user's computation goes here)
        * --- do some work; sprinkle JRED calls periodically ---
        JRED    INRD,UIN
```

```
        JRED    OUTRD,UOUT
        JRED    INRD,UIN
        JRED    OUTRD,UOUT
        * done with current block:
        JSR     RELEASE
        * repeat/exit per program logic
        JMP     COMPUTE
        HLT
END     MAIN
```

Notes on correctness and style

- The three pointers `NEXTG`, `NEXTR`, `NEXTP` and the **single** `CURRENT` (yellow) are kept in the cyclic order **P* G* (Y?) R***, so no pointer can ever "pass" another; each controller converts the head of its segment to the next color and advances its own pointer only within its segment, exactly mirroring the invariant under Fig. 26.
- `ASSIGN/RELEASE` are drop-in analogs of Programs A/R (here A /R) from the section's "multiple buffers" discussion; the only differences are the extra counts and the scan to the next **green**.
- `JRED INRD,UIN` and `JRED OUTRD,UOUT` provide the coroutine linkage (B-style control) so that device "ready" events run the I/O completion steps (I5/O5) promptly while the computation proceeds.

17 [40]

Data structures and invariants

Buffers: Each buffer `b` has fields `(state, dev)`, where `state  {FREE, GREEN, YELLOW, RED}` and `dev` is a device id.

Global pool/list:

- `FREE`: list of free buffers.
- For each **input** device `d`: queue `G[d]` of GREEN buffers (ready for compute), and counter `g[d] = |G[d]|`.
- For each **output** device `o`: queue `R[o]` of RED buffers (ready for device), and counter `r[o] = |R[o]|`.
- Device status `idle(d)` / `idle(o)` and an "in-flight" counter for I/O requests in progress.

Anti-flood control (the key to the exercise):

- **Per-input token bucket.** For each input device `d`, maintain `tok[d]` with bounds `0 tok[d] cap[d]` (small cap, e.g., 2). – **Consume one token** when scheduling an `IN` for `d`. – **Produce one token** when computation *takes* a GREEN buffer of `d` (i.e., on `ASSIGN_IN(d)`), reflecting actual consumption. This ensures anticipation never runs more than `cap[d]` blocks ahead of use.
- **Free-buffer reserve.** Maintain `reserve_free 1` (e.g., `reserve_free = #output_devices`, one per output). Do **not** start new input if `|FREE| reserve_free`, **unless** compute is currently **blocked** waiting for that specific input device `d` (the text's exception).
- **Priority:** Output drains (RED→device) always take precedence over more input anticipation.

These three rules are sufficient to keep pooled buffers from stalling due to over-prefetch.

Algorithms (pooled variants of A/R/B)

Algorithm A_pool — ASSIGN (normal state)

There are two entry points:

A_pool.IN(d): hand compute the next input block from device `d`. A1. If `G[d]` nonempty: pop head `b` from `G[d]`, set `state(b)←YELLOW`, return `b`. Also **tok[d] ← min(tok[d]+1, cap[d])** (credit for consumption). A2. Otherwise: mark `need[d]←true`, invoke `B\_pool` (control), and wait until `G[d]` becomes nonempty, then go to A1.

A_pool.OUT(o): obtain a workspace buffer destined for output device `o`. A3. If `FREE` nonempty: take `b` from `FREE`, set `state(b)←YELLOW`, `dev(b)←o`, return `b`. A4. Otherwise: invoke `B\_pool` and wait until `FREE` nonempty, then A3.

Algorithm R_pool — RELEASE (normal state)

R1. If `b` was obtained by `ASSIGN_IN`, computation has *consumed* it: set `state(b)←FREE`, push to `FREE`. **R2.** If `b` was obtained by `ASSIGN_OUT(o)`, computation has *produced* it: set `state(b)←RED`, enqueue to `R[o]`. **R3.** Optionally call `B\_pool` (helps prompt draining).

Algorithm B_pool — CONTROL (runs on I/O completion or when A/R asks)

B1. Drain outputs first (priority). For each **idle** output device `o` with `R[o]` nonempty: pop `b` from `R[o]`, start `OUT(o, b)` (device becomes busy).

B2. Start essential inputs (to unblock compute). For each input device `d` with `need[d]==true` and `idle(d)==true`: If `FREE` nonempty (even if `|FREE| reserve_free`), take `b` from `FREE`, set `state(b)←(in-flight IN for d)`, **tok[d]−− (if tok[d]==0, set to 0; we allow this unblocking read even if tokens are 0)**, start `IN(d, b)`, clear `need[d]`. (Exactly the exception suggested in the text: we may use the final buffer for input only if compute is stalled for that input.)

B3. Conservative anticipation for inputs. For each **idle** input device `d`, if `tok[d] > 0` **and** `|FREE| > reserve_free`: take `b` from `FREE`, set `(in-flight IN for d)`, **tok[d]——**, start `IN(d, b)`.

B4. On I/O completion (interrupt or poll): – If an `OUT(o, b)` completes: set `state(b)←FREE`, push `FREE`. – If an `IN(d, b)` completes: set `state(b)←GREEN`, enqueue to `G[d]`. Then re-enter B1 (outputs may now drain again).

This is the entire pooled-buffer controller. It's short because the **token bucket** + **reserve** encode the needed policy compactly.

Why this prevents slowdown

- **No green-pool flood.** Tokens ensure each input device anticipates at most `cap[d]` blocks beyond what compute actually consumes; you can't fill the pool with "anticipated input" because consumption is the only way to mint new tokens.
- **Output always proceeds.** Output drains have strict priority (B1) and at least `reserve_free` buffers are kept out of anticipatory reads, so produced blocks can turn RED and get scheduled without starvation.
- **Progress when blocked.** If compute stalls waiting for `d`, B2 violates both the token and reserve limits **only for that** `d`, matching Knuth's exception: you may use the last buffer to get unstuck for the specific input that blocks compute.

Tuning notes

- Typical choices: `reserve_free = #output_devices` and `cap[d]  {1,2}`. With fast outputs or many outputs, increase `reserve_free`. With bursty inputs, `cap[d]=2` smooths latency while still preventing flood.
- You may add a small **hysteresis** (high/low watermark) around `reserve_free` to avoid thrashing (start anticipation only when `|FREE|  reserve_free+H`, stop when `|FREE|  reserve_free`).

1. Overview: how the algorithms map to MIX

- **A_pool.IN(d)** takes a ready (GREEN) input buffer for device `d` or blocks (setting `NEED[d]`) and calls `B_pool`.
- **A_pool.OUT(o)** takes a FREE buffer for output device `o` or blocks and calls `B_pool`.
- **R_pool.RELEASE** returns a consumed input buffer to `FREE` or enqueues a produced output buffer to `R[o]` (RED).
- **B_pool** runs the scheduler: drain outputs first, then do essential inputs that unblock compute, then conservative anticipatory inputs (respecting tokens and the free-buffer reserve).
- **CHECK_IO** polls devices with `JBUS` to detect completions and moves finished buffers to `FREE` (OUT complete) or `G[d]` (IN complete).

2. Symbols, constants, and layout

```
        * ---------- PARAMETERS ----------
M       EQU     3               * number of input devices
N       EQU     2               * number of output devices
B       EQU     16              * number of buffers in pool (>= M + N + working set)
BUFSZ   EQU     100             * words per buffer (tune per device)
RESV    EQU     N               * reserve_free: keep at least N buffers free

        * Device numbers (example; change to your MIX)
D1      EQU     16              * paper tape reader, say
D2      EQU     17              * disk input unit
D3      EQU     18
O1      EQU     19              * line printer, say
O2      EQU     20

        * ---------- BUFFER DESCRIPTORS ----------
        * Each buffer descriptor (BD[i]) holds: NEXT, STATE, DEV, ADDR
        * STATE: 0=FREE, 1=GREEN, 2=YELLOW, 3=RED, 4=INFLIGHT_IN, 5=INFLIGHT_OUT
        * DEV:   device id (for GREEN/RED/INFLIGHT states)
        * ADDR:  start address of the data area for this buffer
STATE_FREE  EQU 0
STATE_GREEN EQU 1
STATE_YEL   EQU 2
STATE_RED   EQU 3
STATE_INFLI EQU 4
STATE_INFLO EQU 5

NIL     EQU     0

        ORIG    1000            * (choose a safe origin)

FREEHD  CON     NIL             * head pointer to FREE list

        * G queues (one per input d) and R queues (one per output o)
GHD1    CON     NIL
GHD2    CON     NIL
GHD3    CON     NIL
RHO1    CON     NIL
RHO2    CON     NIL

        * "Need" flags: compute blocked waiting for input from d?
NEED1   CON     0
NEED2   CON     0
```

```
NEED3   CON     0

        * Token buckets: TOK[d]   [0..CAP[d]]
TOK1    CON     1               * initialize to small positive values
TOK2    CON     1
TOK3    CON     1
CAP1    CON     2               * caps can be 1 or 2 (prefetch depth)
CAP2    CON     2
CAP3    CON     2

        * For each device, track in-flight buffer (0 if none)
INFLI1  CON     0               * input d1
INFLI2  CON     0
INFLI3  CON     0
INFLO1  CON     0               * output o1
INFLO2  CON     0

        * Busy flags are implicit via JBUS; you can also mirror them if you like
        * (not required).

        * Buffer descriptor table
BDTAB   EQU     *               * BD[1..B], 3 words each + ADDR
        * We also allocate data areas per buffer behind BD table.
        * Layout helper: BD[i].NEXT at BDNXT+(i-1), etc.
BDNXT   CON     NIL             * BD[1].NEXT
BDST1   CON     STATE_FREE      * BD[1].STATE
BDDEV1  CON     0               * BD[1].DEV
BDADR1  CON     0               * BD[1].ADDR
        * ... replicate for BD[2..B] (macro expands this in a real assembly)
        * For brevity, assume you generate these with a macro.
        * Also compute BDADR[i] to point to a BUFSZ block per buffer.

        * Free-list bootstrapping (done once at init):
        * Link all BD[i] in a singly linked list: FREEHD -> 1 -> 2 -> ... -> B -> NIL
```

3. List primitives (ENQ/DEQ on singly linked heads)

We use head insertion for O(1) enqueue; dequeue also from head (so queues act as stacks — fine for buffers). If you want FIFO queues, maintain both head and tail; the scaffold keeps it simple.

```
* ENQ b into list at (HEADPTR)
*   IN: A = address of HEADPTR (the word containing head pointer)
*       X = buffer index b (1..B)
ENQ     STJ     ENQ_RET
        LDA     0,1             * A := HEADPTR
        LD1     0,1             * X1 := [HEADPTR] (current head)
        STA     TMPHEAD
        ENT2    0               * X2 := 0
        * Set BD[b].NEXT = old head
        LDA     =BDNXT-1=,X     * address of BD[b].NEXT; (use proper base math in real code)
        LDX     TMPHEAD
        STA     0,X             * BD[b].NEXT = old head
        * Update head to b
        LDX     X               * X holds b already
        STA     0,1             * [HEADPTR] = b
        JMP     ENQ_RET
ENQ_RET NOP

* DEQ from list at (HEADPTR), returns X = b or X = 0 if empty
DEQ     STJ     DEQ_RET
        LDA     0,1             * head
        JAZ     DEQ_EMPTY
        LDX     0,1             * X := head b
        * head := BD[b].NEXT
        LDA     =BDNXT-1=,X
        LDA     0,X
        STA     0,1
        JMP     DEQ_DONE
DEQ_EMPTY
        ENTX    0               * X := 0
DEQ_DONE
        JMP     DEQ_RET
DEQ_RET NOP

TMPHEAD CON     0
```

(Notes: In real MIXAL you'll implement the "address arithmetic" with proper `EQU`/macros so `BDNXT-1 + b` hits BD[b].NEXT. I'm showing intent; translate to your macro system.)

4. Helpers to read/write BD fields

For clarity, define tiny helpers/macros (or inline):

- SET_STATE(b, s)
- SET_DEV(b, d)
- SET_NEXT(b, nx)
- GET_ADR(b) -> A := BDADR[b]

I'll inline them below to keep the listing compact.

5. Core routines

5.1 A_pool.IN(d)

```
* A_IN(d): try G[d], else set NEED[d]=1, call B_pool, wait until G[d] nonempty
* Example for d = D1 (duplicate for D2, D3 or generate with macro)

A_IN1   STJ     AIN1_RET

        * Try dequeue from GHD1
        LDA     =GHD1=
        ENTA    =GHD1=
        ENTX    0               * X acts as result
        JMP     AIN1_DEQ
AIN1_DEQ
        * call DEQ with A=addr(GHD1)
        * (Assume A already = addr(GHD1))
        JSR     DEQ
        JXZ     AIN1_BLOCK      * X=0 means empty -> block

        * Got buffer b in X: mark YELLOW and credit token
        * STATE[b] := YELLOW
        LDA     =BDST1-1=,X
        ENTA    STATE_YEL
        STA     0,X
        * TOK1 := min(TOK1+1, CAP1)
        LDA     TOK1
        ADD     ONE
        CMPA    CAP1
        JGE     AIN1_TOKCAP
        STA     TOK1
        JMP     AIN1_DONE
AIN1_TOKCAP
        STA     TOK1            * (A   CAP → store A anyway; CAP1 already bounds)
AIN1_DONE
        JMP     AIN1_RET
```

```
AIN1_BLOCK
        * Signal need and invoke scheduler
        STA     NEED1           * store nonzero to NEED1
        JSR     B_POOL

        * Busy-wait until GHD1 nonempty (you may also yield via CHECK_IO)
AIN1_WAIT
        LDA     GHD1
        JAZ     AIN1_SPIN
        JMP     A_IN1           * try again
AIN1_SPIN
        JSR     CHECK_IO        * poll devices and reschedule
        JMP     AIN1_WAIT

AIN1_RET NOP
ONE     CON     1
```

5.2 A_pool.OUT(o)

```
* A_OUT(o): take FREE or block; example for o = O1
A_OUT1  STJ     AOUT1_RET
AOUT1_TRY
        LDA     =FREEHD=
        JSR     DEQ
        JXZ     AOUT1_BLOCK
        * X=b; STATE[b] := YELLOW; DEV[b] := O1
        LDA     =BDST1-1=,X
        ENTA    STATE_YEL
        STA     0,X
        LDA     =BDDEV1-1=,X
        ENTA    O1
        STA     0,X
        JMP     AOUT1_RET
AOUT1_BLOCK
        JSR     B_POOL
AOUT1_WAIT
        LDA     FREEHD
        JAZ     AOUT1_SPIN
        JMP     A_OUT1
AOUT1_SPIN
        JSR     CHECK_IO
```

```
        JMP     AOUT1_WAIT
AOUT1_RET NOP
```

5.3 R_pool.RELEASE

```
* RELEASE for an input buffer (consumed): return to FREE
REL_IN  STJ     RELI_RET
        * IN: X = b
        * STATE[b] := FREE; ENQ b -> FREEHD
        LDA     =BDST1-1=,X
        ENTA    STATE_FREE
        STA     0,X
        ENTA    =FREEHD=
        JSR     ENQ
        JMP     RELI_RET
RELI_RET NOP

* RELEASE for an output buffer (produced): enqueue RED to R[o]
* Example for o=O1, X=b
REL_OUT1 STJ    RELO1_RET
        LDA     =BDST1-1=,X
        ENTA    STATE_RED
        STA     0,X
        LDA     =BDDEV1-1=,X
        ENTA    O1
        STA     0,X
        ENTA    =RHO1=
        JSR     ENQ
        * hint scheduler
        JSR     B_POOL
        JMP     RELO1_RET
RELO1_RET NOP
```

6. Scheduler (B_pool)

Drain outputs → essential inputs (to unblock compute) → anticipatory inputs (tokens + reserve).

```
B_POOL  STJ     BRET

        * ----- B1: drain outputs first -----
        * if O1 idle and RHO1 nonempty and no inflight out: start OUT(O1)
```

```
B1_O1   LDA     RHO1
        JAZ     B1_O2           * nothing ready
        LDA     INFLO1
        JAZ     B1_O1_TRY       * none inflight
        JMP     B1_O2           * already busy for O1
B1_O1_TRY
        ENTA    =RHO1=
        JSR     DEQ             * X=b
        JXZ     B1_O2
        * mark inflight and start OUT
        STA     INFLO1          * INFLO1 := b
        LDA     =BDST1-1=,X
        ENTA    STATE_INFLO
        STA     0,X
        * OUT mem, O1
        LDA     =BDADR1-1=,X
        OUT     0(O1)           * (syntax varies by assembler; concept: OUT at BDADR[b] to O
B1_O2
        * repeat for O2 similarly...
        * (macro in real code)

        * ----- B2: essential inputs (unblock compute) -----
        * For each d with NEED[d]!=0 and INFLI[d]==0 and FREE available (even if count <= RES
B2_D1   LDA     NEED1
        JAZ     B2_D2
        LDA     INFLI1
        JNZ     B2_D2
        * Try FREE
        LDA     FREEHD
        JAZ     B2_D2           * truly no free buffers; cannot help now
        ENTA    =FREEHD=
        JSR     DEQ             * X=b
        JXZ     B2_D2
        * Start IN on D1 without consuming token (exception)
        STA     INFLI1
        LDA     =BDDEV1-1=,X
        ENTA    D1
        STA     0,X
        LDA     =BDST1-1=,X
        ENTA    STATE_INFLI
        STA     0,X
        * IN mem, D1
```

```
        LDA     =BDADR1-1=,X
        IN      0(D1)
        * clear NEED1
        ENTA    0
        STA     NEED1
B2_D2
        * Repeat similarly for D2, D3 ...

        * ----- B3: conservative anticipation -----
        * Only if FREE count > RESV and TOK[d] > 0 and no inflight
        * (Here we approximate "FREE count" with head != NIL and a spillover test,
        *  or keep a FREE_COUNT counter updated by ENQ/DEQ in real code.)

        * We'll implement a minimal guard: require FREEHD != NIL and also
        * require a second FREE to exist (head.next != NIL) when RESV=1, etc.
        * For clarity, use FREE_COUNT (recommended).

        * Example for D1:
B3_D1   LDA     INFLI1
        JNZ     B3_D2
        LDA     TOK1
        JAZ     B3_D2
        LDA     FREE_COUNT
        CMPA    RESV
        JLE     B3_D2
        * OK: schedule anticipatory IN for D1; consume one token
        ENTA    =FREEHD=
        JSR     DEQ             * X=b (must exist by guard)
        JXZ     B3_D2
        STA     INFLI1
        LDA     =BDDEV1-1=,X
        ENTA    D1
        STA     0,X
        LDA     =BDST1-1=,X
        ENTA    STATE_INFLI
        STA     0,X
        * IN mem, D1
        LDA     =BDADR1-1=,X
        IN      0(D1)
        * TOK1 := TOK1 - 1
        LDA     TOK1
        SUB     ONE
```

```
        STA     TOK1
B3_D2
        * Repeat for D2, D3 ...

        JMP     BRET
BRET    NOP
```

7. I/O completion polling (CHECK_IO)

`IN/OUT` start asynchronously; we detect completion with `JBUS` ("jump if busy"). When `JBUS` falls through, the device finished. We then move the buffer to the right queue and clear `INFLI?/INFLO?`.

```
CHECK_IO STJ    CIRET

        * ----- outputs: when OUT completes, return buffer to FREE -----
C_O1    JBUS    C_O1_BUSY,O1    * jump if O1 still busy
        * Completed
        LDA     INFLO1
        JAZ     C_O2            * no inflight -> nothing to do
        ENTX    A               * X=b
        ENTA    0
        STA     INFLO1
        * STATE[b] := FREE; ENQ FREE
        LDA     =BDST1-1=,X
        ENTA    STATE_FREE
        STA     0,X
        ENTA    =FREEHD=
        JSR     ENQ
        * FREE_COUNT++
        LDA     FREE_COUNT
        ADD     ONE
        STA     FREE_COUNT
        JMP     C_O2
C_O1_BUSY
        NOP
C_O2    * repeat for O2 ...

        * ----- inputs: when IN completes, put buffer GREEN on G[d] -----
C_I1    JBUS    C_I1_BUSY,D1
        LDA     INFLI1
        JAZ     C_I2
        ENTX    A               * X=b
```

```
        ENTA    0
        STA     INFLI1
        * STATE[b] := GREEN; DEV[b]:=D1; ENQ to GHD1
        LDA     =BDST1-1=,X
        ENTA    STATE_GREEN
        STA     0,X
        LDA     =BDDEV1-1=,X
        ENTA    D1
        STA     0,X
        ENTA    =GHD1=
        JSR     ENQ
        JMP     C_I2
C_I1_BUSY
        NOP
C_I2    * repeat for D2, D3 ...

        * Optional: immediately run B_POOL to kick off drain/anticipation
        JSR     B_POOL
        JMP     CIRET
CIRET   NOP
```

8. FREE_COUNT maintenance (recommended)

Counting FREE precisely makes the **reserve** rule simple and efficient.

```
FREE_COUNT  CON  B           * initialized to B at startup

* On taking from FREE (DEQ success): FREE_COUNT--
* On returning to FREE (ENQ):        FREE_COUNT++

* You can integrate these bumps directly around the DEQ/ENQ calls that touch FREE.
```

9. Putting it together (call pattern)

- The **compute** routine calls `A_IN?` to obtain a YELLOW buffer (filled input) or `A_OUT?` to obtain a YELLOW buffer for output space. It then does work and calls `REL_IN` or `REL_OUT?` accordingly.
- `A_*` calls may block, which triggers `B_POOL` and `CHECK_IO` until the requested list becomes nonempty.
- `B_POOL` is also called:
  - right after `REL_OUT?` (to drain RED → device),

  - from `CHECK_IO` when completions happen,
  - and opportunistically anywhere you'd like to make the system responsive.

10. Policy knobs (the "anti-flood" part)

- **Token buckets:** `TOK[d]` minted **only** when compute actually consumes a GREEN buffer (in `A_IN?`, after successful dequeue). One token is **spent** when scheduling an anticipatory `IN` in B3. Essential reads in B2 don't spend tokens (that's the "exception when blocked"). Keep `CAP[d]` small (1–2).
- **Reserve:** `RESV = #outputs` is a good default; increase if outputs can bunch up or are slow. The `FREE_COUNT > RESV` guard must hold before B3 starts more anticipation.
- **Output priority:** Encoded by B1 running before B2/B3. This prevents the pool from filling with GREEN while outputs starve.

11. Notes for a real MIX assembler/emulator

- The snippets use "address arithmetic" placeholders like `=BDST1-1=,X`. In a real MIXAL, define macros `BDNXT(i)`, `BDST(i)`, `BDDEV(i)`, `BDADR(i)` that compute addresses from `i` (e.g., via `EQU` tables or by laying out BD fields in contiguous arrays).
- The `IN 0(D)` / `OUT 0(D)` lines indicate "start I/O at address in A to device D". Adapt to your assembler's exact operand format; many MIX tools accept `IN A,D`.
- If your emulator blocks on `IN/OUT` (non-asynchronous), simulate async by splitting start/complete as shown (or just keep the same control logic; `JBUS` tests become no-ops).

18 [30]

Below is a clean, interrupt-driven rewrite of Knuth's multiple-buffer scheme (Algorithms/Programs A, R, B in §1.4.4) for MIX, using the proposed control-state "interrupt" extension described right after Exercise 18. I first restate the interrupt model very briefly, then give the modified algorithms and a compact MIX sketch, and finally explain why it works and how it maps to the original buffering logic.

Interrupt model (per the exercise)

- MIX gains control memory locations −3999..−0001 and two internal states: **normal** and **control**. On an interrupt, registers rA, rI1..rI6, rX, and (rJ, OV, CI, "next-inst") are saved into −0009..−0001, and MIX **enters control state** at an address determined by the interrupt type. There is also a "clock" at −0010 that ticks each 1000u and, on zero, interrupts to −0011. The instruction **INT**: in normal state, forces an interrupt to −0012; in control state, it **restores** registers from −0009..−0001 and returns to normal state. I/O completion interrupts vector to −(0020+unit). (All as specified in the exercise statement.)

Variables and notation

We keep Knuth's §1.4.4 notation:

- `N` = total buffers in the circle.
- `n` = number of *green* buffers (assignable).
- `NEXTG`, `NEXTR`, `CURRENT` = ring pointers as in the text.
- **New:** `t` = 0 if the device is idle, 1 if a transfer is in progress.
- Device unit number is `U`.

(Initial conditions are as in §1.4.4: for input, start with all buffers green; for output, with all buffers red; and `NEXTR = NEXTG` at start. See the section text around Algorithms A/R/B.)

The modified algorithms

These are exactly the changes needed to eliminate any polling/JRED and run purely interrupt-driven. The outline matches Knuth's official answer ("Partial solution") but with full context and linkage.

A — ASSIGN (normal state, called by the compute routine)

Unchanged from Algorithm A in the text:

- **A1.** Wait for `n < N`.
- **A2.** `CURRENT ← NEXTG`.
- **A3.** Advance `NEXTG` (clockwise).

(Identical to §1.4.4A.)

R — RELEASE (normal state, inlined where the compute routine is done with the current buffer)

- **R1 .** Increase `n` by 1.
- **R2 .** If `t = 0` (device idle), **force an interrupt** with `INT` so that control jumps immediately to the control routine's step **B3** (see below).

This is exactly the little "nudge" we need so that when the CPU frees a buffer and the device happens to be idle, the control routine can immediately kick off the next I/O without delay.

B — BUFFER CONTROL (runs in control state; it *processes interrupts*)

Think of B as the *interrupt handler framework* for buffering; it is entered:

- on a software interrupt from **R2** (vector −0012),
- or on an I/O completion interrupt (vector −(0020+U)).

It uses `t` to remember whether the device is currently busy.

- **B1 . Restart the main program** (execute `INT`) if there's nothing useful to do right now.
- **B2 .** If `n = 0`, set `t ← 0` and **go to B1** (no buffers to service, stay idle and let the CPU compute).
- **B3 .** Set `t ← 1` and **initiate I/O** on unit `U` using the buffer pointed to by `NEXTR` (e.g., `IN` for input flow, `OUT` for output flow).
- **B4 . Restart the main program** (`INT`). An I/O completion will later interrupt and re-enter control state.
- **B5 .** On the *I/O completion interrupt*, advance `NEXTR` to the next buffer in the ring.
- **B6 .** Decrease `n` by 1 (we've consumed a green buffer for input, or emptied a red buffer for output), and **go to B2** .

This is the full interrupt-driven analogue of the original B; no busy-waiting or `JRED` appears anywhere.

Minimal MIX skeleton (how to wire it)

Below is a compact sketch to show where the control-state entry points jump. It's deliberately schematic (addresses/labels are illustrative), but it shows the exact linkage you can adopt in your own code.

```
* --------------------------
* Data (normal state)
N       CON   ...          * total buffers
n       CON   0            * current green count
t       CON   0            * 0 = idle, 1 = busy
NEXTG   CON   ...          * ring pointers per §1.4.4
NEXTR   CON   ...
CURRENT CON   ...

* --------------------------
* R  (normal state, inline macro)
Rprime: INC   n,1          * R1
        LDA   t
        JAZ   ForceINT     * if t = 0 -> nudge the control routine
        JMP   *+1
ForceINT:
        INT                * software interrupt -> control entry at -0012

* A  (normal state): identical to Algorithm A in text (A1-A3)
* ... your A code from §1.4.4 ...

* --------------------------
* CONTROL STATE VECTORS
```

```
* (hardware jumps here on different interrupts)

ORG    -12                 * -0012: software INT from R2
INT_SW: JMP   B3prime

ORG    -(20+U)             * I/O completion for device U
INT_IO: JMP   B5prime

* (Clock -0011 not needed here, but could JMP B1  if you like.)

* --------------------------
* B  control routine (control state)
B1prime:
        INT                * Return to normal state immediately when idle

B2prime:
        LDA   n
        JAZ   Idle         * if n = 0: idle the device
        JMP   B3prime

Idle:   STA   t            * store 0 (accumulator assumed 0 from JAZ path)
        JMP   B1prime

B3prime:
        ENTA  1
        STA   t            * t ← 1 (device busy)
        * Initiate I/O on buffer at NEXTR:
        *   For input:  IN  BUF(NEXTR)(U)
        *   For output: OUT BUF(NEXTR)(U)
        * (Addressing of BUF(NEXTR) per your ring layout.)
        INT                * B4 : restart main program; completion will re-enter at INT_IO

B5prime:
        * advance NEXTR one buffer clockwise
        * NEXTR ← next(NEXTR)
        DEC   n,1          * B6 : n ← n - 1
        JMP   B2prime
```

(You'll splice in the actual `IN`/`OUT` origin fields to address the buffer at `NEXTR`, exactly as you did in your non-interrupt version.)

Why this is correct

- **No polling:** The original B used a "wait until device ready" flavor (conceptually via `JBUS/JRED`). Here we *never* poll. The control logic either restarts the main program (B1 /B4) or is re-entered **only** when something interesting happens (a release when idle, or I/O completion). This directly matches the exercise's intent.
- **Immediate kick-off when possible:** If the CPU frees a buffer while the device is idle (`t=0`), **R2** forces a software interrupt, which vectors straight to **B3** and starts I/O at once. This eliminates latent delays that might otherwise occur before the next timer/clock tick.
- **Invariant preservation:** The same invariants from §1.4.4 hold:
  - `n` counts available (green) buffers; is incremented only by RELEASE (R1) and decremented only when a transfer is actually begun (B6).
  - `t` tracks device occupancy; set in B3 and implicitly cleared by the **path** B2 takes when `n=0` (and by the fact that once completion fires, the next cycle will either start another I/O or idle again).
  - `NEXTR` advances **only** upon I/O completion (B5), exactly mirroring the original control logic.
- **State transitions match hardware:** `INT` in control state restores registers and resumes normal execution (B1 , B4), and I/O completion vectors to −(0020+U) (B5 path). That's precisely how the exercise defines INT and the I/O interrupts.

Mapping to Knuth's official answer

Knuth's answer gives the same three changes concisely:

- "In the algorithms below, **t** is 0 when idle, 1 when active."
- **A** unchanged.
- **R :** increment `n`; if `t=0` force an interrupt to B3 .
- **B :** (i) restart; (ii) if `n=0`, set `t ← 0` and restart; (iii) set `t ← 1` and initiate I/O; (iv) restart; (v) on I/O complete, advance `NEXTR`; (vi) `n ← n - 1`; loop.

Notes and practical tips

- If you use the "green–yellow–red–purple" combined input/output circle (Fig. 26) elsewhere in §1.4.4, the same interrupt framework applies; you just choose `IN` vs. `OUT` in **B3** based on which half (input or output) is being serviced at that moment.
- You do **not** need the clock interrupt (−0011) for correctness; it can be ignored or used for other housekeeping. The key events are **R2** 's software INT and device completion interrupts.

If you'd like, I can drop this into your specific MIX buffer layout (addresses for `BUF1..BUFN`, ring macros for `NEXTR/NEXTG`, etc.) and produce a runnable MIXAL file.

19 [M28]

We process $n \geq 2$ consecutive blocks from a rotating device. Block k can begin input only when its start passes the head, namely at times

$$t_k \equiv (k-1)L \pmod{P}, \qquad t_1 = 0,$$

and it becomes available T time units later, i.e. it may be **assigned** at

$$u_k \;\geq\; t_k + T,$$

then takes C time units to compute, so it is **released** at

$$v_k = u_k + C.$$

Processing is sequential so $u_k \geq v_{k-1}$ for $k > 1$. With only N buffers we also need $t_k \geq v_{k-N}$ for $k > N$.

Question: what is the least N that lets us achieve the theoretical optimum finishing time

$$v_n^{\min} = T + C + (n-1)\max(L, C)?$$

Give a general rule and then work it out for $L = 1,\ P = 100,\ T = 0.5,\ n = 100$ with several C's.

Lower bound and target schedule

The lower bound arises because each additional block must wait at least L time units (until its start again appears) and at least C time units (to compute). Thus the best possible cadence is by $\max(L, C)$, with the pipeline start-up cost $T + C$. We therefore try to **hit the bound** by choosing

$$v_k = \begin{cases} T + C, & k = 1, \\ v_{k-1} + C, & C \geq L \quad \text{(compute-bound)}, \\ v_{k-1} + L, & C \leq L \quad \text{(I/O-bound)}. \end{cases}$$

Case A — I/O-bound ($C \leq L$)

Here we can target $u_k = t_k + T = (k-1)L + T$ (use the *earliest* revolution), giving

$$v_k = T + C + (k-1)L.$$

To be feasible with N buffers we must not start input before a buffer frees:

$$t_k \;=\; (k-1)L \;\geq\; v_{k-N} = T + C + (k-N-1)L \;\iff\; (N-1)L \;\geq\; T + C.$$

Hence the **minimum number of buffers** is

$$\boxed{N_{\min} = \left\lceil 1 + \frac{T+C}{L} \right\rceil, \qquad C \leq L.}$$

Case B — Compute-bound ($C \geq L$)

Now we target $u_k = v_{k-1}$ so $v_k = T + kC$. Block k must have finished input by $v_{k-1} - T$. Among the allowed input start times for block k,

$$\mathcal{T}_k = \{(k-1)L + mP \mid m \in \mathbb{Z}_{\geq 0}\},$$

let t_k be the **latest** $\leq v_{k-1} - T$. The lag we need to "absorb" with buffering is the backward distance

$$\delta_k \stackrel{\text{def}}{=} ((v_{k-1} - T) - t_k) = ((k-1)(C-L)) \bmod P \in [0, P).$$

(That identity uses $v_{k-1} - T = (k-1)C$.) With N buffers the available "window" before compute touches the block is

$$(v_{k-1} - T) - v_{k-N} = (N-1)C - T,$$

so feasibility for **all** k requires

$$\delta_k \leq (N-1)C - T \quad \text{for } k = 2, \dots, n.$$

Let

$$\Delta_n \;=\; \max_{1 \leq j \leq n-1} \big(j(C-L) \bmod P\big),$$

then a necessary and sufficient condition to attain the optimum is

$$(N-1)C - T \;\geq\; \Delta_n.$$

Therefore the **minimum number of buffers** is

$$\boxed{N_{\min} = \left\lceil 1 + \frac{T + \Delta_n}{C} \right\rceil, \qquad C \geq L,}$$

with $\Delta_n = \max_{1 \leq j \leq n-1} \big(j(C-L) - P\lfloor j(C-L)/P \rfloor\big)$.

Remarks.

- If $(C-L)/P$ is rational $= p/q$ in lowest terms and $n-1 \geq q$, then $\Delta_n = P - P/q$. If n is smaller, compute the max over $j = 1, \dots, n-1$ explicitly.
- If $C = L$, then $\Delta_n = 0$ and the formula gives $N_{\min} = \lceil 1 + T/C \rceil$.

Worked examples ($L = 1,\ P = 100,\ T = 0.5,\ n = 100$)

For each C we evaluate $\Delta_{100} = \max_{1 \leq j \leq 99}(j(C-1) \bmod 100)$ and then $N_{\min}$.

Case	C	Regime	Δ_{100}	$N_{\min}$
(a)	0.5	$C \leq L$	—	$\lceil 1 + \frac{0.5+0.5}{1} \rceil = 2$
(b)	1.0	$C = L$	0	$\lceil 1 + \frac{0.5+0}{1} \rceil = 2$
(c)	1.01	$C \geq L$	$\max_{j \leq 99} 0.01j = 0.99$	$\lceil 1 + \frac{0.5+0.99}{1.01} \rceil = 3$
(d)	1.5	$C \geq L$	$\max_{j \leq 99} 0.5j = 49.5$	$\lceil 1 + \frac{0.5+49.5}{1.5} \rceil = 35$
(e)	2.0	$C \geq L$	$\max_{j \leq 99} 1 \cdot j = 99$	$\lceil 1 + \frac{0.5+99}{2} \rceil = 51$
(f)	2.5	$C \geq L$	$\max_{j \leq 99}(1.5j \bmod 100) = 99$	$\lceil 1 + \frac{0.5+99}{2.5} \rceil = 41$
(g)	10.0	$C \geq L$	$\max_{j \leq 99}(9j \bmod 100) = 99$	$\lceil 1 + \frac{0.5+99}{10} \rceil = 11$
(h)	50.0	$C \geq L$	$\max_{j \leq 99}(49j \bmod 100) = 99$	$\lceil 1 + \frac{0.5+99}{50} \rceil = 3$
(i)	200.0	$C \geq L$	$\max_{j \leq 99}(199j \bmod 100) = 99$	$\lceil 1 + \frac{0.5+99}{200} \rceil = 2$

These values are the least buffer counts that allow the optimal makespan $v_n^{\min} = T + C + (n-1)\max(L, C)$ to be realized under the exercise's constraints.